



**UNIVERSIDAD PABLO DE
OLAVIDE**

Escuela Politécnica Superior

Grado en Ingeniería Informática en Sistemas de Información

**Desarrollo de una aplicación basada
en visión por computadora para la
detección automática de anomalías
en radiografías
25-26-C23**

Trabajo fin de grado

Luis Carmona Berdugo

DIRIGIDO POR

Aurelio López Fernández

Sevilla, Septiembre de 2026



**UNIVERSIDAD PABLO DE
OLAVIDE**

**Desarrollo de una aplicación basada
en visión por computadora para la
detección automática de anomalías
en radiografías**

MEMORIA QUE PRESENTA

Luis Carmona Berdugo

DIRIGIDO POR

Aurelio López Fernández

Escuela Politécnica Superior

**Grado en Ingeniería Informática en Sistemas de
Información**

Sevilla, Septiembre de 2026

Agradecimientos

Resumen

Abstract

Índice general

Parte I.....	24
Capítulo 1. Estado del Arte y Contextualización.....	25
1.1. La neumonía, un problema de salud global.....	25
1.2. La radiografía de tórax como imagen digital.....	26
1.2.1. ¿Qué es una radiografía de tórax y por qué es la prueba de referencia?.....	26
1.2.2. ¿Cómo ve el ordenador una radiografía?.....	26
1.2.3. Conjuntos de datos utilizados.....	27
1.3. Inteligencia Artificial, aprendizaje profundo y visión artificial.....	28
1.3.1. De la IA al aprendizaje profundo.....	28
1.3.2. La visión artificial.....	29
1.3.3. CheXNet y el estudio de Kermany.....	29
1.4. Las redes neuronales convolucionales (CNN).....	30
1.4.1. Qué es una red neuronal convolucional.....	30
1.4.2. Las arquitecturas convolucionales del banco de pruebas.....	30
1.5. Los Transformers en visión artificial.....	34
1.5.1. El mecanismo de atención.....	34
1.5.2. De las palabras a los píxeles.....	35
1.5.3. Las arquitecturas Transformer del proyecto.....	36
1.6. La explicabilidad: inteligencia artificial explicable.....	37
1.6.1. El problema de la caja negra.....	37
1.6.2. Técnicas de explicabilidad.....	37
1.6.3. La evaluación cuantitativa.....	38
1.6.4. La calibración.....	39
1.7. Las barreras para la adopción clínica de la IA.....	40
1.8. Los asistentes conversacionales y los grandes modelos de lenguaje.....	41
1.8.1. ¿Qué son los grandes modelos de lenguaje?.....	41
1.8.2. El panorama de los modelos de lenguaje y la elección inicial de Llama 3.....	42
Capítulo 2. Metas y Propósitos del Proyecto.....	43
Capítulo 3. Estructura organizativa y partes interesadas del proyecto.....	46
3.1. Estructura organizativa del proyecto.....	46
3.2. Partes interesadas del proyecto.....	48
3.3. Responsabilidades, funciones y participación de las partes interesadas.....	49
Capítulo 4. Marco Metodológico de Gestión.....	52
4.1. Justificación del enfoque metodológico.....	52
4.2. Roles y recursos del proyecto.....	53
4.2.1. Roles dentro del marco de trabajo.....	53
4.2.2. Dedicación y recursos materiales.....	53
4.3. Ciclo de trabajo iterativo.....	54
4.3.1. Los sprints y la cadencia del trabajo.....	54

4.3.2. Ceremonias y reuniones de seguimiento.....	54
4.3.3. Artefactos del proceso de gestión.....	55
4.4. Seguimiento visual del trabajo con Kanban.....	55
4.5. Alcance de la metodología.....	56
Capítulo 5. Cronograma y Plan de Trabajo.....	57
5.1. Marco general y objetivos de la planificación.....	57
5.2. Desglose de tareas por Sprint.....	58
5.2.1. Sprint 0 – Planificación del proyecto.....	60
5.2.2. Sprint 1 – Infraestructura y seguridad.....	62
5.2.3. Sprint 2 – Motor de diagnóstico.....	65
5.2.4. Sprint 3 – Explicabilidad.....	67
5.2.5. Sprint 4 – Entrenamiento CNN.....	70
5.2.6. Sprint 5 – Transformers y análisis XAI.....	73
5.2.7. Sprint 6 – Validación externa.....	76
5.2.8. Sprint 7 – Laboratorio MLOps.....	78
5.2.9. Sprint 8 – Documentación y cierre.....	81
5.3. Recursos y costes del proyecto.....	84
5.4. Reparto de responsabilidades.....	86
Capítulo 6. Identificación y Mitigación de Riesgos.....	87
6.1. Escala de valoración de riesgos.....	88
6.2. Catálogo de riesgos identificados.....	90
6.3. Plan de mitigación (medidas preventivas).....	92
6.4. Plan de contingencia (medidas correctivas).....	93
Capítulo 7. Planes de apoyo a la gestión del proyecto.....	94
7.1. Plan de Comunicación.....	95
7.2. Plan de Calidad.....	95
7.3. Plan de Gestión de la Configuración.....	96
Capítulo 8. Cuestiones abiertas y decisiones adoptadas.....	97
8.1. Cuestiones abiertas.....	100
Capítulo 9. Consideraciones complementarias del proyecto.....	101
9.1. Marco regulatorio.....	101
Parte II.....	103
Capítulo 10. Planteamiento del análisis del sistema.....	104
10.1. Propósito del análisis.....	104
10.2. Dimensiones del análisis.....	104
Capítulo 11. Definición del ámbito del sistema.....	106
11.1. Objetivos del sistema.....	107
11.2. Contexto tecnológico y restricciones de entorno.....	120
11.3. Normativa y estándares de referencia.....	122
11.4. Perfiles de usuario del sistema.....	124
Capítulo 12. Especificación de requisitos del sistema.....	125

12.1. Obtención y organización de los requisitos.....	126
12.1.1. Requisitos funcionales del sistema.....	127
12.1.1.1. Módulo de autenticación y cuenta.....	127
12.1.1.2. Módulo de diagnóstico clínico.....	134
12.1.1.3. Módulo de historial de consultas.....	140
12.1.1.4. Módulo de laboratorio MLOps.....	144
12.1.1.5. Módulo de administración.....	160
12.1.1.6. Módulo transversal.....	163
12.1.1.7. Matriz de trazabilidad entre requisitos funcionales y objetivos del sistema....	169
12.1.2. Requisitos no funcionales del sistema.....	173
12.1.2.1. Seguridad de la plataforma y del acceso.....	173
12.1.2.2. Confidencialidad y protección de los datos.....	185
12.1.2.3. Rendimiento y capacidad de respuesta.....	192
12.1.2.4. Sencillez de uso y accesibilidad.....	196
12.1.2.5. Robustez y disponibilidad del servicio.....	200
12.1.2.6. Persistencia y salvaguarda de los datos.....	204
12.1.2.7. Reproducibilidad y sostenibilidad del desarrollo.....	206
12.1.2.8. Compromisos del proceso de desarrollo y entregables del proyecto.....	208
12.1.2.9. Matriz de trazabilidad entre requisitos no funcionales y objetivos del sistema.....	212
12.2. Especificación de los casos de uso.....	215
12.2.1. Diagramas de casos de uso por módulo.....	216
12.2.1.1. Gestión del acceso y de la cuenta.....	217
12.2.1.2. Interfaz de diagnóstico asistido.....	218
12.2.1.3. Laboratorio de experimentación MLOps.....	220
12.2.1.4. Supervisión y administración de la plataforma.....	222
12.2.1.5. Capacidades transversales de la plataforma.....	224
12.2.2. Actores del sistema y su ámbito de interacción.....	225
12.2.3. Actores del sistema y su ámbito de interacción.....	227
12.2.3.1. Módulo de gestión del acceso y de la cuenta.....	229
12.2.3.2. Módulo de interfaz de diagnóstico asistido.....	234
12.2.3.3. Módulo de laboratorio de experimentación MLOps.....	245
12.2.3.4. Módulo de supervisión y administración de la plataforma.....	262
12.2.3.5. Módulo de capacidades transversales de la plataforma.....	266
12.2.3.6. Matriz de trazabilidad requisitos – casos de uso.....	273
Capítulo 13. Identificación de subsistemas de análisis.....	280
Capítulo 14. Modelo de dominio y entidades del sistema.....	282
14.1. Estructura estática de las entidades del sistema.....	282
14.2. Comportamiento dinámico del sistema y DSS.....	285
14.2.1. SS-001 – Subsistema de Acceso y Gestión de Cuentas.....	286
14.2.2. SS-002 – Subsistema de Diagnóstico Asistido.....	290

14.2.3. SS-003 – Subsistema de Gestión del Historial.....	297
14.2.4. SS-004 – Subsistema de Laboratorio MLOps.....	301
14.2.5. SS-005 – Subsistema de Supervisión y Administración.....	318
14.2.6. SS-006 – Subsistema de Capacidades Transversales.....	322
Capítulo 15. Verificación de la consistencia del análisis.....	325
15.1. Criterio de verificación.....	325
15.2. Objetivos y requisitos.....	325
15.3. Requisitos, casos de uso y subsistemas.....	326
15.4. Requisitos no funcionales y responsabilidades.....	327
15.5. Modelo de dominio y limitaciones.....	328
15.6. Resultado de la verificación.....	328
Capítulo 16. Plan de pruebas.....	329
16.1. Pruebas de componentes individuales.....	329
16.2. Pruebas de los flujos entre subsistemas.....	330
16.3. Pruebas de protección y control de acceso.....	331
16.4. Pruebas de rendimiento y capacidad de respuesta.....	331
16.5. Pruebas de sistema y validación funcional.....	332
16.6. Matriz de trazabilidad entre requisitos y pruebas.....	332
Parte III.....	335
Capítulo 17. Diseño y arquitectura de vitalXAI.....	336
17.1. Organización arquitectónica de la solución.....	337
17.1.1. Capa de presentación y acceso desde el navegador.....	337
17.1.2. Capa HTTP y coordinación de los subsistemas funcionales.....	338
17.1.3. Servicios de aplicación y motores de inteligencia artificial.....	338
17.1.4. Persistencia relacional y ejecución asíncrona.....	339
17.1.5. Vista global de la arquitectura.....	340
17.2. Condiciones de calidad, normativa y restricciones del diseño.....	342
17.2.1. Requisitos no funcionales y respuesta arquitectónica.....	342
17.2.1.1. Seguridad, identidad y control del acceso.....	344
17.2.1.2. Confidencialidad y tratamiento de la información.....	345
17.2.1.3. Rendimiento, concurrencia y ejecución sin bloqueo.....	345
17.2.1.4. Usabilidad, internacionalización y presentación.....	346
17.2.1.5. Robustez, persistencia y recuperación.....	346
17.2.2. Estándares y normas aplicables.....	347
17.2.3. Restricciones técnicas y límites del entorno.....	348
17.3. Subsistemas de Diseño.....	349
17.3.1. SD-001: Acceso, identidad y gestión de sesiones.....	350
17.3.2. SD-002: Diagnóstico asistido y generación de resultados.....	351
17.3.3. SD-003: Historial y gestión de consultas.....	353
17.3.4. SD-004: Laboratorio de experimentación MLOps.....	354
17.3.5. SD-005: Supervisión y administración.....	356
17.3.6. SD-006: Cola de trabajos y capacidades transversales.....	357

17.3.7. Relaciones de integración entre los subsistemas de diseño.....	359
17.4. Operación segura y controles de protección.....	361
17.4.1. Control de identidad, sesión y autorización.....	362
17.4.2. Protección de los datos y de los artefactos.....	363
17.4.3. Comunicaciones, configuración y secretos.....	364
17.4.4. Seguridad de la ejecución asíncrona.....	365
17.4.5. Persistencia, arranque y recuperación operativa.....	366
17.4.6. Límites actuales y responsabilidades del operador.....	367
Capítulo 18. Arquitectura de soporte y mecanismos transversales.....	367
18.1. Subsistemas de Soporte.....	369
18.1.1. SSOP-001: Persistencia relacional y conexiones de base de datos.....	370
18.1.2. SSOP-002: Almacenamiento de ficheros y recursos de aplicación.....	371
18.1.3. SSOP-003: Entorno de ejecución e integraciones externas.....	372
18.1.4. SSOP-003: Entorno de ejecución e integraciones externas.....	373
18.2. Mecanismos de diseño transversales.....	374
18.2.1. Configuración externa y separación de secretos.....	374
18.2.2. Validación en las fronteras de entrada.....	374
18.2.3. Autenticación, autorización y propiedad de recursos.....	375
18.2.4. Persistencia de estado y transacciones simples.....	375
18.2.5. Ejecución asíncrona y aislamiento de trabajos.....	376
18.2.6. Errores, estados y comunicación con el usuario.....	376
18.2.7. Modularidad, pruebas y evolución controlada.....	377
18.2.8. Ciclo de vida y sustitución controlada de componentes.....	377
Capítulo 19. Persistencia relacional y esquema de datos de vitalXAI.....	378
19.1. Diseño del esquema de persistencia.....	380
19.2. Acceso a la capa de persistencia.....	394
Capítulo 20. Diseño de los casos de uso.....	396
20.1. SD-001: Acceso, identidad y gestión de sesiones.....	397
20.2. SD-002: Diagnóstico asistido y resultados.....	399
20.3. SD-003: Historial y gestión de consultas.....	401
20.4. SD-004: Laboratorio de experimentación MLOps.....	402
20.5. SD-005: Supervisión y administración.....	404
20.6. SD-006: Cola de trabajos y capacidades transversales.....	405
20.7. Cierre del diseño de los casos de uso.....	406
Capítulo 21. Estructura de clases del diseño.....	406
21.1. Subsistema SD-001: Acceso, identidad y gestión de sesiones.....	407
21.1.1. Diagrama de clases del subsistema.....	408
21.1.2. Especificación de las clases.....	409
21.2. Subsistema SD-002: Diagnóstico asistido y generación de resultados.....	413
21.2.1. Diagrama de clases del subsistema.....	413
21.2.2. Especificación de las clases.....	415
21.3. Subsistema SD-003: Historial y gestión de consultas.....	420

21.3.1. Diagrama de clases del subsistema.....	420
21.3.2. Especificación de las clases.....	422
21.4. Subsistema SD-004: Laboratorio de experimentación MLOps.....	423
21.4.1. Diagrama de clases del subsistema.....	423
21.4.2. Especificación de las clases.....	425
21.5. Subsistema SD-005: Supervisión y administración.....	432
21.5.1. Diagrama de clases del subsistema.....	432
21.5.2. Especificación de las clases.....	434
21.6. Subsistema SD-006: Cola de trabajos y capacidades transversales.....	435
21.6.1. Diagrama de clases del subsistema.....	435
21.6.2. Especificación de las clases.....	437
Capítulo 22. Diseño de interfaces de los subsistemas.....	440
22.1. Subsistema SD-001: Acceso, identidad y gestión de sesiones.....	443
22.1.1. Flujo de navegación.....	443
22.1.2. Especificación de las interfaces.....	444
22.1.3. Informes del subsistema.....	446
22.2. Subsistema SD-002: Diagnóstico asistido y generación de resultados.....	447
22.2.1. Flujo de navegación.....	447
22.2.2. Especificación de las interfaces.....	448
22.2.3. Informes del subsistema.....	450
22.3. Subsistema SD-003: Historial y gestión de consultas.....	451
22.3.1. Flujo de navegación.....	451
22.3.2. Especificación de las interfaces.....	452
22.3.3. Informes del subsistema.....	453
22.4. Subsistema SD-004: Laboratorio de experimentación MLOps.....	454
22.4.1. Flujo de navegación.....	454
22.4.2. Especificación de las interfaces.....	456
22.4.3. Informes del subsistema.....	458
22.5. Subsistema SD-005: Supervisión y administración.....	460
22.5.1. Flujo de navegación.....	460
22.5.2. Especificación de las interfaces.....	462
22.5.3. Informes del subsistema.....	463
22.6. Subsistema SD-006: Cola de trabajos y capacidades transversales.....	464
22.6.1. Flujo de navegación.....	464
22.6.2. Especificación de las interfaces.....	465
22.6.3. Informes del subsistema.....	466
22.7. Criterios transversales de accesibilidad.....	466
Capítulo 23. Especificación de la construcción del sistema.....	468
23.1. Entorno tecnológico de construcción.....	469
23.2. Paquetes y componentes de construcción.....	473
23.3. Proceso de construcción del software.....	476

23.4. Elaboración de Especificaciones del Modelo Físico de Datos.....	479
Capítulo 24. Preparación inicial de los datos del sistema.....	480
24.1. Entorno de Carga Inicial o Migración.....	481
24.2. Procedimientos de Carga Inicial o Migración.....	484
Capítulo 25. Estrategia de verificación del sistema.....	487
25.1. Entornos de verificación.....	488
25.2. Niveles de verificación y criterios de aceptación.....	491
25.2.1. Nivel unitario.....	492
25.2.2. Nivel de integración.....	493
25.2.3. Nivel de sistema.....	494
Capítulo 26. Implantación del sistema.....	495
26.1. Documentación necesaria para la operación.....	497
26.2. Requisitos de Implantación.....	498
Capítulo 27. Introducción a la codificación.....	502
27.1. Visión general de la implementación.....	503
27.2. Estructura del repositorio.....	504
27.3. Convenciones de código y decisiones transversales.....	506
27.4. Herramientas y calidad del código.....	506
Capítulo 28. Codificación del backend.....	507
28.1. Arranque y configuración de la aplicación.....	508
28.2. Capa HTTP: los routers.....	510
28.3. Servicios de aplicación.....	512
28.4. Persistencia y acceso a los datos.....	514
28.5. Seguridad transversal.....	516
Capítulo 29. Codificación de la ejecución asíncrona.....	518
29.1. La cola de trabajos y su máquina de estados.....	518
29.2. El bucle del worker.....	520
29.3. Reclamación, despacho y procesamiento de los trabajos.....	522
29.4. Tolerancia a fallos y recuperación.....	524
29.5. La cola desde la API.....	526
Capítulo 30. Codificación del motor de diagnóstico y XAI.....	528
30.1. Carga y caché de los modelos.....	529
30.2. Preprocesado y predicción.....	531
30.3. Generación de los mapas de explicabilidad.....	533
30.4. Generación del informe PDF.....	535
Capítulo 31. Codificación del laboratorio MLOps.....	537
31.1. Metodología experimental del pipeline.....	537
31.2. Orquestación del pipeline de entrenamiento.....	538
31.3. Validación externa y comparación estadística.....	540
31.4. Gestión de las sesiones de experimentación.....	542
31.5. Lectura de los resultados.....	544

31.6. Generación del informe de la sesión.....	546
Capítulo 32. Codificación del frontend.....	548
32.1. Las plantillas de presentación.....	548
32.2. Los recursos JavaScript por ámbito.....	550
32.3. Comunicación asíncrona con la API.....	550
32.4. Internacionalización y tema visual.....	552
Capítulo 33. Introducción a la verificación del sistema.....	555
33.1. Alcance de la verificación.....	556
33.2. Entornos y herramientas de verificación.....	557
33.3. Organización de la parte.....	558
33.4. Trazabilidad de los objetivos específicos.....	558
Capítulo 34. Pruebas automatizadas.....	560
34.1. Las pruebas unitarias.....	560
34.2. Las pruebas de integración.....	562
34.3. Ejecución y resultados de la batería.....	563
34.4. Cobertura de los módulos.....	563
Capítulo 35. Pruebas de seguridad.....	565
35.1. Protección CSRF.....	565
35.2. Cabeceras de seguridad.....	566
35.3. Limitación de peticiones.....	566
35.4. Gestión de los tokens de sesión.....	567
35.5. Resultados de la verificación de seguridad.....	568
Capítulo 36. Pruebas de sistema y validación funcional.....	569
36.1. PE-001: Flujo clínico completo.....	569
36.2. PE-002: Gestión del historial.....	572
36.3. PE-003: Flujo del laboratorio.....	575
36.4. PE-004: Ejecución asíncrona de la cola.....	579
36.5. PE-005: Supervisión administrativa.....	582
36.6. PE-006: Capacidades transversales.....	585
36.7. Resultados de la validación funcional.....	588
Capítulo 37. Referencias bibliográficas.....	591

[OBJ]

Índice de Tablas

Índice de Ilustraciones

Índice de Ilustraciones

Control	
Proyecto	Desarrollo de una aplicación basada en visión por computadora para la detección automática de anomalías en radiografías
Fecha	Septiembre 2026
Edición	v4
Autores	Luis Carmona Berdugo

Registro de Cambios		
Versión	Descripción del Cambio	Fecha del Cambio
v1	Creación del documento	Junio 2026
v2	Modificación de objetivos y requisitos del proyecto	Julio 2026
v3	Depuración de diagramas de los Documentos de Análisis y Diseño	Agosto 2026
v4	Repaso final y definición de conclusiones	Septiembre 2026

Parte I.

Plan de Proyecto

[OBJ.]

[OBJ.]

[OBJ.]

Capítulo 1. Estado del Arte y Contextualización

Este capítulo presenta el estado del arte y la contextualización del Trabajo Fin de Grado titulado "vitalXAI: Plataforma MLOps con inteligencia artificial explicable para el diagnóstico asistido de neumonía mediante radiografías de tórax", realizado por el alumno Luis Carmona Berdugo y tutorizado por el profesor Aurelio López Fernández. En él se introduce el problema clínico, se describen las tecnologías y métodos relevantes y se sitúa la propuesta del proyecto dentro del contexto de la inteligencia artificial aplicada al diagnóstico por imagen.

Este capítulo cumple dos funciones. Primero, presenta el problema que motiva el proyecto: la neumonía, su diagnóstico mediante radiografías de tórax y las limitaciones de los métodos actuales. Después, introduce el estado del arte de las tecnologías relacionadas con la propuesta, como la inteligencia artificial, las redes neuronales convolucionales, los Transformers, la inteligencia artificial explicable y los asistentes conversacionales basados en grandes modelos de lenguaje. Las decisiones propias de la plataforma y de su evaluación se desarrollan en los capítulos correspondientes de diseño, construcción y codificación.

1.1. La neumonía, un problema de salud global

La neumonía es una infección que afecta a los pulmones y, en particular, a los alvéolos, las estructuras donde se produce el intercambio de gases. La acumulación de líquido o pus en estas estructuras dificulta la llegada de oxígeno a la sangre. Entre sus manifestaciones habituales se encuentran la tos, la fiebre, la dificultad respiratoria y el dolor torácico (Torres & al., 2021).

La neumonía puede estar causada por diferentes microorganismos, siendo los más frecuentes las bacterias y los virus, aunque también existen neumonías producidas por hongos. La causa concreta condiciona el tratamiento: las neumonías bacterianas se tratan habitualmente con antibióticos, mientras que las virales no responden a estos medicamentos y requieren, en general, cuidados de soporte. Distinguir el tipo de neumonía no siempre es sencillo, y las radiografías de tórax, junto con los síntomas y las pruebas de laboratorio, son herramientas clave para orientar el diagnóstico.

Tiene una relevancia clínica y social considerable. Según la Organización Mundial de la Salud, fue responsable de aproximadamente 2,5 millones de fallecimientos en 2019 y constituye una de las principales causas de muerte por infección en el mundo (World Health Organization, 2024). Su impacto es especialmente elevado entre los grupos más vulnerables, como los menores de cinco años y las personas mayores de sesenta y cinco. En los países de ingresos bajos y medios, las dificultades de acceso a la atención sanitaria agravan especialmente la mortalidad asociada a la neumonía infantil.

La detección temprana y el diagnóstico correcto son importantes porque un retraso puede demorar el tratamiento y aumentar el riesgo de complicaciones. Sin embargo, la interpretación de una radiografía de tórax puede presentar dificultades incluso para especialistas con experiencia. Los estudios muestran variabilidad entre observadores, de modo que dos radiólogos pueden discrepar sobre la presencia de signos compatibles con neumonía (Hopstaken & al., 2004). Esta variabilidad motiva la búsqueda de herramientas informáticas que ayuden a los profesionales sanitarios a fundamentar sus decisiones, sin sustituir su valoración clínica.

1.2. La radiografía de tórax como imagen digital

1.2.1. ¿Qué es una radiografía de tórax y por qué es la prueba de referencia?

La radiografía de tórax es una prueba de imagen que utiliza rayos X para obtener una representación del interior del tórax. Los rayos X son una forma de radiación electromagnética de alta energía que atraviesa los tejidos del cuerpo en mayor o menor medida según su densidad. Los huesos, que son muy densos, absorben la mayor parte de la radiación y aparecen en la imagen de color blanco; los tejidos blandos, como el músculo o el corazón, la absorben en menor grado y se muestran en tonos grises; y las zonas llenas de aire, como los pulmones sanos, apenas la absorben y aparecen prácticamente en negro (Gonzalez & Woods, 2008).

Esta propiedad es la que permite al radiólogo detectar una neumonía: cuando los alvéolos se llenan de líquido o de pus, la zona afectada del pulmón deja de ser transparente a los rayos X y se muestra como una mancha más blanca y densa de lo normal, lo que se conoce como opacidad o infiltrado (Franquet, 2001). La radiografía de tórax es la técnica de imagen de primera línea para el diagnóstico de la neumonía porque combina tres ventajas fundamentales: es muy accesible (existe en prácticamente todos los centros sanitarios), tiene un coste bajo en comparación con otras técnicas como el TAC, y es muy rápida de adquirir. Por estos motivos, sigue siendo la herramienta estándar que se utiliza en primer lugar ante la sospecha de neumonía (Mandell & al., 2007).

1.2.2. ¿Cómo ve el ordenador una radiografía?

Para los algoritmos es necesario representar la radiografía como datos numéricos. Una imagen digital puede describirse como una cuadrícula de píxeles. En una radiografía en escala de grises, cada píxel contiene un valor asociado al nivel de intensidad. En una representación de 8 bits, esos valores suelen situarse entre 0 y 255. Por ejemplo, una imagen de 512 por 512 píxeles puede tratarse como una matriz de 262.144 valores (Szeliski, 2010).

Los algoritmos de visión artificial no interpretan las imágenes como lo hace una persona, sino que procesan valores organizados en estructuras matemáticas, como matrices y tensores. Las redes neuronales profundas ajustan sus parámetros para identificar patrones en esas representaciones, como se explica en las secciones siguientes.

1.2.3. Conjuntos de datos utilizados

Para entrenar y evaluar los modelos de inteligencia artificial es imprescindible disponer de un conjunto de imágenes etiquetadas, es decir, imágenes de las que se sabe con certeza si corresponden o no a un paciente con neumonía. En este proyecto se emplean dos conjuntos de datos públicos y anonimizados, siguiendo la metodología establecida en el estudio de referencia (Kermany & al., 2018).

El conjunto de datos de entrenamiento es el publicado por Kermany y colaboradores, procedente del Guangzhou Women and Children's Medical Center y disponible en la plataforma Kaggle. Este conjunto contiene 5.856 radiografías de tórax pediátricas en formato JPEG, clasificadas en dos categorías: NORMAL, con 1.583 imágenes, y PNEUMONIA, con 4.273 imágenes. La categoría PNEUMONIA se subdivide en neumonía bacteriana y neumonía viral. La diferencia entre el número de imágenes de ambas clases constituye un desbalanceo que se aborda más adelante en la metodología. El conjunto coincide con el utilizado en el estudio de referencia, lo que facilita la comparación contextual, aunque el protocolo experimental y las arquitecturas pueden diferir.

El segundo conjunto de datos se emplea para la validación externa, un concepto que se desarrollará en profundidad más adelante y que, de forma resumida, consiste en comprobar que los modelos funcionan también con imágenes que nunca han visto y que proceden de un entorno distinto al del entrenamiento. Este segundo conjunto es el COVID-19 Radiography Database, desarrollado por investigadores de la Universidad de Qatar y la Universidad de Dhaka (Chowdhury & al., 2020; Rahman & al., 2021). Se trata de radiografías de tórax de pacientes adultos, adquiridas con equipos y protocolos diferentes a los del dataset pediátrico de entrenamiento. De este conjunto se extraen quinientas imágenes de pacientes sanos y quinientas de pacientes con neumonía viral, garantizando así un balance perfecto entre las dos clases. La población adulta, el equipo de adquisición y los protocolos de imagen diferentes convierten esta prueba en un escenario potencialmente exigente para evaluar la capacidad de generalización de los modelos. No obstante, el conjunto combina imágenes normales y patológicas procedentes de fuentes y repositorios distintos, por lo que existe el riesgo de que un modelo aprenda señales relacionadas con el origen de la imagen en lugar de la patología (DeGrave, Janizek, & Lee, 2021).

Durante el desarrollo y la depuración se han utilizado subconjuntos reducidos de ambos conjuntos de datos para acortar los ciclos de prueba. Estos subconjuntos permiten comprobar el funcionamiento del código, pero no representan la evaluación experimental completa. La configuración del proyecto está preparada para trabajar con las 5.856 imágenes de Kermany destinadas al entrenamiento y con las 1.000 imágenes seleccionadas del COVID-19 Radiography Database para la validación externa.

1.3. Inteligencia Artificial, aprendizaje profundo y visión artificial

1.3.1. De la IA al aprendizaje profundo

La inteligencia artificial reúne técnicas que permiten a las máquinas realizar tareas asociadas habitualmente con capacidades humanas, como el aprendizaje o el reconocimiento de patrones. El aprendizaje automático es una de sus ramas y se basa en ajustar modelos a partir de ejemplos, en lugar de programar de forma explícita todas las reglas de la tarea (Bishop, 2006). En este proyecto, los ejemplos son radiografías etiquetadas y el modelo aprende una relación entre sus características visuales y las clases PNEUMONIA y NORMAL.

El aprendizaje profundo es una rama del aprendizaje automático basada en redes neuronales con múltiples capas. Cada unidad combina sus entradas mediante pesos y una función de activación. Las capas iniciales reciben la representación de la imagen y las siguientes transforman esa información hasta producir una salida relacionada con la tarea de clasificación (LeCun, Bengio, & Hinton, 2015).

El proceso mediante el que la red ajusta sus parámetros se denomina entrenamiento. De forma resumida, comprende los pasos siguientes (Goodfellow, Bengio, & Courville, 2016):

1. **Propagación hacia delante:** se introduce una imagen en la red y se calcula su salida con los pesos actuales.
2. **Cálculo del error:** se compara la salida de la red con la respuesta correcta mediante una función matemática denominada función de coste o pérdida (loss), que cuantifica cuánto se ha equivocado la red.
3. **Propagación hacia atrás:** se calcula cómo afectaría una pequeña variación de cada peso al error final, mediante un algoritmo llamado retropropagación del error.
4. **Actualización de los pesos:** se modifican ligeramente los pesos en la dirección que reduce el error, mediante un algoritmo de optimización como el descenso del gradiente.

Este ciclo se repite con todas las imágenes del conjunto de entrenamiento, y el resultado es que, progresivamente, la red va ajustando sus pesos hasta producir salidas cada vez más cercanas a las correctas. En el caso concreto de este proyecto, las redes se entrenan para resolver un problema de clasificación binaria: dada una radiografía, decidir si pertenece a la clase PNEUMONIA o a la clase NORMAL. La red no devuelve una etiqueta rígida, sino una probabilidad entre 0 y 1 (la salida de una neurona con activación sigmoide), que puede interpretarse como el nivel de confianza del modelo.

1.3.2. La visión artificial

La visión artificial (computer vision) es la rama de la inteligencia artificial que se ocupa de que las máquinas puedan interpretar el mundo visual: imágenes, vídeos, fotografías y, en el caso que nos ocupa, radiografías (Szeliski, 2010). La tarea concreta que aborda este proyecto es la clasificación de imágenes: asignar una categoría (en este caso, "neumonía" o "sano") a cada imagen de entrada.

La visión artificial puede apoyar el análisis de radiografías mediante modelos capaces de extraer patrones visuales y asignar una clase a cada imagen. Esta automatización no elimina la dificultad de la tarea: las radiografías presentan variaciones de adquisición y hallazgos sutiles, y el resultado del modelo debe interpretarse dentro del contexto clínico.

1.3.3. CheXNet y el estudio de Kermay

Para contextualizar el estado del arte se revisan dos trabajos especialmente relacionados con los datos y los modelos empleados en este proyecto.

El primero es CheXNet, presentado por Rajpurkar y colaboradores en 2017 (Rajpurkar & al., 2017). Se trata de una red convolucional de 121 capas, basada en DenseNet, entrenada con ChestX-ray14, una base de datos de más de 112.000 radiografías de unos 30.000 pacientes y con anotaciones para catorce enfermedades torácicas. El estudio comparó el rendimiento del modelo con el de un grupo de radiólogos en la detección de neumonía. Sus resultados ilustran el potencial del aprendizaje profundo en esta tarea, aunque deben interpretarse dentro de las condiciones concretas del conjunto de datos y del protocolo utilizado.

El segundo hito es el estudio de Kermay y colaboradores, publicado en la revista Cell en 2018 (Kermay & al., 2018), que constituye una referencia metodológica y de datos para este proyecto. El trabajo presenta un marco de aprendizaje profundo basado en transferencia de conocimiento y demuestra su aplicación a la identificación de neumonía pediátrica mediante radiografías de tórax, utilizando el conjunto de 5.856 imágenes descrito en la sección anterior. Su aportación no consiste en comparar sistemáticamente varias arquitecturas convolucionales, sino en mostrar cómo un sistema de aprendizaje profundo puede utilizarse para apoyar el diagnóstico por imagen y acompañar sus predicciones de elementos interpretables. La comparación con el rendimiento de expertos se refiere principalmente a las tareas de diagnóstico oftalmológico analizadas en el estudio, por lo que no debe trasladarse directamente a una afirmación de equivalencia entre el modelo de neumonía y los radiólogos. El artículo resulta relevante para este proyecto por el uso del dataset pediátrico, la transferencia de conocimiento y la preocupación por la interpretabilidad, pero no constituye un benchmark de cinco arquitecturas de neumonía.

Este trabajo toma ambos estudios como referencia y desarrolla una propuesta propia en tres direcciones: prepara un pipeline para comparar diecinueve arquitecturas, incorpora métricas cuantitativas de explicabilidad y calibración, y reúne el procedimiento en una plataforma web MLOps orientada a usuarios sin formación técnica. La ejecución experimental documentada es parcial y contiene resultados de ocho arquitecturas CNN. Por tanto, la aportación se presenta como la construcción de un entorno comparativo y operativo, no como una ampliación de los resultados publicados por Kermay.

1.4. Las redes neuronales convolucionales (CNN)

1.4.1. Qué es una red neuronal convolucional

Las redes neuronales convolucionales (del inglés convolutional neural networks, CNN) son el tipo de arquitectura que ha dominado históricamente la visión artificial. Su nombre procede de la operación matemática central que realizan: la convolución.

La convolución aplica filtros numéricos sobre regiones locales de la imagen. Cada filtro responde a determinados patrones, como bordes o texturas, y produce un mapa de características. El mismo filtro se aplica en distintas posiciones, lo que reduce el número de parámetros y hace que el modelo pueda reconocer un patrón con independencia de su posición exacta. Las capas de agrupación reducen la resolución espacial y resumen la información de regiones próximas (Krizhevsky, Sutskever, & Hinton, 2012).

Las CNN se estructuran en capas que extraen progresivamente características cada vez más abstractas. Las primeras capas detectan elementos sencillos como bordes y cambios de brillo; las capas intermedias combinan estos elementos en texturas y formas; y las capas finales integran el todo en conceptos de alto nivel relevantes para la tarea, en nuestro caso, la presencia o ausencia de las opacidades características de la neumonía. Entre las capas convolucionales se intercalan capas de agrupación (pooling) que reducen el tamaño de los mapas de características, condensando la información y ayudando a la red a ser más robusta y eficiente. Al final de la red, las características se aplanan y se conectan a una o varias capas densas (como las de cualquier red neuronal clásica) que producen la clasificación final.

En el contexto de este proyecto, una CNN ajusta sus parámetros para distinguir patrones visuales asociados a las clases del conjunto de entrenamiento. Esos patrones pueden incluir variaciones de densidad en los campos pulmonares, pero el modelo no garantiza que toda señal utilizada corresponda a una causa clínica relevante.

1.4.2. Las arquitecturas convolucionales del banco de pruebas

El proyecto contempla dieciséis arquitecturas convolucionales. Se agrupan por familias para resumir sus características principales sin reproducir una descripción exhaustiva de cada modelo. Todas parten de pesos preentrenados en ImageNet (Deng & al., 2009; Russakovsky & al., 2015). La configuración concreta del entrenamiento se documenta en la implementación del laboratorio MLOps.

Familia ResNet (He, Zhang, Ren, & Sun, 2016): el problema que ResNet vino a resolver es el de la degradación de las redes muy profundas. Cuando una red tiene decenas o cientos de capas, entrenarla de forma directa se vuelve extremadamente difícil, y el rendimiento, lejos de mejorar, empeora. La solución propuesta son las conexiones residuales o saltos: cada bloque de la red aprende no una transformación completa, sino la diferencia (el residuo) entre su entrada y su salida, sumando la entrada original. Esta idea, aparentemente sencilla, permitió entrenar redes de cientos de capas y es una de las arquitecturas más influyentes de la década. En este proyecto se evalúan:

- ResNet50: la versión de 50 capas, la más popular y equilibrada de la familia.

- ResNet101: versión más profunda (101 capas), que puede captar patrones más complejos a costa de mayor coste computacional.
- ResNet152: la más profunda de la serie (152 capas), con mayor capacidad pero también mayor demanda de recursos.
- ResNet50V2: una revisión de ResNet50 que reorganiza el orden de las operaciones dentro de cada bloque (normalizando antes de la convolución), lo que mejora el flujo de gradientes y, en general, la convergencia del entrenamiento (He, Zhang, Ren, & Sun, 2016).

Familia DenseNet (Huang, Liu, Van Der Maaten, & Weinberger, 2017): DenseNet lleva la idea de las conexiones a un extremo: en lugar de saltos puntuales, cada capa recibe como entrada las características de todas las capas anteriores, concatenándolas. Esto se denomina conectividad densa y favorece la reutilización de las características, el flujo de información y la regularización natural. DenseNet es especialmente relevante para este proyecto porque CheXNet, el sistema que igualó a los radiólogos en la detección de neumonía, se basaba precisamente en esta arquitectura (Rajpurkar & al., 2017). Se evalúan:

- DenseNet121: la versión de 121 capas, la más ligera y utilizada de la familia.
- DenseNet201: la versión más profunda (201 capas), con mayor capacidad de representación.

Familia EfficientNet (Tan & Le, 2019): EfficientNet plantea la pregunta de cómo escalar una red de forma óptima. En lugar de decidir a mano cuánto agrandar la profundidad, la anchura o la resolución de la imagen, propone un método de escalado compuesto que equilibra las tres dimensiones mediante un coeficiente único. El resultado es una familia de modelos que ofrece una excelente relación entre precisión y coste computacional. Se evalúan:

- EfficientNetB0: el modelo base de la familia, el más ligero.
- EfficientNetB3 y EfficientNetB7: versiones escaladas con mayor capacidad.
- EfficientNetV2S: la segunda generación de EfficientNet, que mejora la eficiencia del entrenamiento introduciendo capas de entrenamiento progresivo y ajustes en las operaciones (Tan & Le, 2021).

Familia MobileNet (Howard & al., 2017): MobileNet está diseñada para funcionar en dispositivos con recursos limitados, como teléfonos móviles. Su idea clave es la convolución separable por profundidad, que descompone una convolución estándar en dos operaciones más baratas, reduciendo drásticamente el número de parámetros y de operaciones sin sacrificar demasiada precisión. Se evalúan:

- MobileNetV2: segunda generación, que introduce los bloques residuales invertidos (Sandler & al., 2018).
- MobileNetV3Large: tercera generación, optimizada además con técnicas de búsqueda de arquitecturas (Howard & al., 2019).

VGG16 (Simonyan & Zisserman, 2015): VGG mostró que una arquitectura sencilla, basada en una pila uniforme de capas convolucionales de 3×3 seguidas de capas de agrupación, podía alcanzar un rendimiento competitivo al aumentar su profundidad. Su simplicidad la convierte en un modelo de referencia para comparar arquitecturas más complejas.

InceptionV3 (Szegedy & al., 2016): la idea de Inception es aplicar varios filtros de distinto tamaño (1×1, 3×3 y 5×5) en paralelo sobre la misma región, de modo que la red pueda captar patrones a diferentes escalas simultáneamente. InceptionV3 refina esta idea con convoluciones factorizadas que reducen el coste computacional.

Xception (Chollet, 2017): Xception lleva la idea de Inception al extremo y se basa exclusivamente en convoluciones separables por profundidad, demostrando que esta operación, combinada de la forma adecuada, puede superar a las arquitecturas clásicas de Inception. Es una arquitectura especialmente eficiente en términos de parámetros.

ConvNeXtTiny (Liu & al., 2022): ConvNeXt es una relectura moderna de las CNN: tomando como base ResNet, incorpora de forma sistemática las ideas que hicieron exitosos a los Transformers (normalizaciones, funciones de activación modernas, menor número de bloques de normalización), demostrando que las CNN "modernizadas" pueden competir con los Transformers en igualdad de condiciones. ConvNeXtTiny es la versión pequeña de esta familia.

Las dieciséis arquitecturas cubren distintas capacidades y costes computacionales, desde modelos ligeros como MobileNetV2 hasta modelos de mayor tamaño como ConvNeXt-Tiny. Esta variedad, junto con los estudios comparativos de CNN y Transformers en radiografías, proporciona un marco amplio para analizar la detección de neumonía (Murphy & al., 2022).

RED NEURONAL CONVOLUCIONAL (CNN) PARA RADIOGRAFÍAS DE TÓRAX

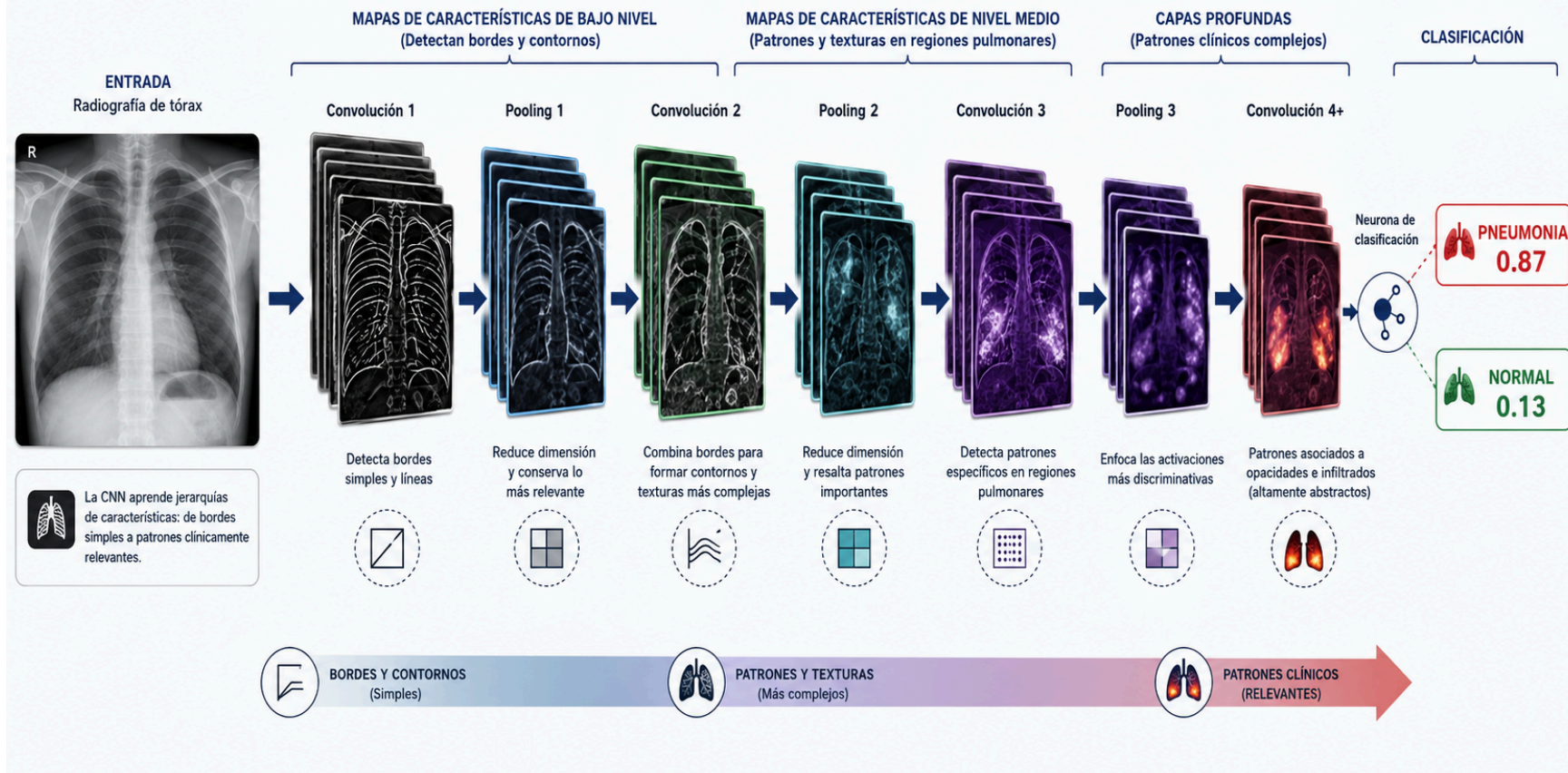


Ilustración x - Representación CNN para Radiografías de Tórax

1.5. *Los Transformers en visión artificial*

1.5.1. El mecanismo de atención

Para entender los modelos más recientes de este proyecto es necesario introducir otra familia de arquitecturas: los Transformers. Su historia comienza en el procesamiento del lenguaje natural. Durante años, las tareas de lenguaje se resolvían con redes recurrentes que procesaban las palabras de una frase una a una, en orden, manteniendo un estado interno. Este enfoque era lento y tenía dificultades para recordar relaciones entre palabras muy separadas dentro del texto.

En 2017, el artículo "Attention is all you need" (Vaswani & al., 2017) propuso una arquitectura radicalmente distinta: el Transformer, basada en un mecanismo denominado atención. La idea esencial de la atención es que, al procesar un elemento de una secuencia (por ejemplo, una palabra), el modelo puede "prestar atención" a todos los demás elementos de la secuencia simultáneamente, ponderando cuánto influye cada uno de ellos. Así, al interpretar la palabra "neumonía" en la frase "la radiografía muestra una neumonía en el lóbulo inferior", el modelo aprende a relacionarla con "radiografía" y "lóbulo inferior" aunque estén lejos en la frase.

Dos conceptos acompañan a la atención: la **autoatención**, que establece relaciones entre todos los elementos de una misma secuencia (cada palabra se relaciona con todas las demás), y las **cabezas de atención**, que son múltiples mecanismos de atención ejecutados en paralelo, cada uno especializado en un tipo distinto de relación. Además, como el Transformer no procesa los elementos en orden, necesita incorporar una **codificación posicional**, una información que le indica en qué posición de la secuencia se encuentra cada elemento (Vaswani & al., 2017).

1.5.2. De las palabras a los píxeles

La aplicación de Transformers a imágenes requiere convertir la información espacial en una representación que el modelo pueda procesar como una secuencia. El enfoque adoptado por los Vision Transformers consiste en dividir la imagen en fragmentos y representar cada uno como un elemento de esa secuencia.

El Vision Transformer (ViT) propuesto por Dosovitskiy y colaboradores en 2021 (Dosovitskiy & al., 2021) divide la imagen en una cuadrícula de pequeños fragmentos cuadrados llamados patches (parches). Cada parche se convierte en un vector numérico mediante una transformación lineal, de forma análoga a como cada palabra se convierte en un vector que la representa. Estos vectores, llamados tokens, se introducen en un Transformer junto con su codificación posicional, de modo que el modelo aplica autoatención entre todos los parches de la imagen. Para producir la clasificación final, el ViT utiliza un token especial denominado token de clasificación, cuyo estado final después de atravesar el modelo se conecta a una capa de clasificación.

La diferencia conceptual con las CNN es profunda y muy relevante para este proyecto. Una CNN procesa la imagen de forma local y progresiva: cada neurona solo "ve" una pequeña región, y la visión global se construye capa a capa. Un Transformer, en cambio, relaciona desde la primera capa todas las regiones de la imagen entre sí, lo que le permite captar dependencias de largo alcance que a una CNN podrían pasarle desapercibidas. En el contexto de la neumonía, esto significa que un Transformer puede relacionar directamente, por ejemplo, la densidad anómala de un lóbulo pulmonar con la expansión del pulmón opuesto, sin necesidad de que esa relación emerja capa tras capa. La contrapartida es que los Transformers son muy "hambrientos de datos": necesitan grandes cantidades de imágenes para aprender correctamente y tienen un coste computacional elevado, especialmente cuando la imagen se divide en muchos parches (Dosovitskiy & al., 2021). Precisamente por eso su aplicación al ámbito médico, donde los conjuntos de datos suelen ser reducidos, exige una evaluación cuidadosa, que es uno de los objetivos de este trabajo.

1.5.3. Las arquitecturas Transformer del proyecto

Este proyecto evalúa tres arquitecturas basadas en atención:

- **DeiT** (Data-efficient Image Transformers) (Touvron & al., 2021): DeiT aborda directamente el problema del hambre de datos de los ViT. Su idea central es el uso de un token de destilación: durante el entrenamiento, un profesor (típicamente una CNN muy potente) "enseña" al Transformer, y el token de destilación es un elemento adicional que el modelo aprende a usar para imitar al profesor. Esto permite que un Transformer se entrene correctamente con muchos menos datos. En este proyecto se emplea la variante deit-base-distilled, preentrenada a 224×224 píxeles.
- **Swin Transformer** (Liu & al., 2021): Swin ataca el problema del coste computacional de los Transformers. En lugar de calcular la autoatención sobre todos los parches de la imagen (lo que crece de forma cuadrática con el número de parches), Swin la calcula dentro de ventanas locales, que además se desplazan entre capas consecutivas (ventanas desplazadas). Este diseño jerárquico, inspirado en las CNN, reduce el coste y permite captar información a múltiples escalas. La variante evaluada es swin-base, con parches de 4×4 y ventanas de 7×7 , a resolución 224×224 .
- **ViT-384**: es el Vision Transformer estándar, pero configurado para trabajar con una resolución de entrada mayor (384×384 píxeles en lugar de los 224 habituales). Trabajar con más píxeles equivale a dividir la imagen en más parches, lo que proporciona más detalle espacial al modelo (al precio de un mayor coste computacional). La variante utilizada es vit-base-patch16-384.

La inclusión de estos tres modelos permite comparar las CNN y los Transformers dentro de un mismo marco experimental, aunque no mediante un protocolo de optimización idéntico. Se mantienen comunes las particiones estratificadas, el número de pliegues, el balanceo, la aumentación, el conjunto de validación y las métricas. Las CNN utilizan una base convolucional congelada y entrenan un nuevo cabezal, mientras que los Transformers ajustan sus pesos preentrenados con una tasa de aprendizaje menor y AdamW (Loshchilov & Hutter, 2019). Los resultados deben interpretarse como una comparación exploratoria entre familias con protocolos adaptados, no como una atribución causal de las diferencias únicamente a la arquitectura (Murphy & al., 2022).

1.6. La explicabilidad: inteligencia artificial explicable

1.6.1. El problema de la caja negra

Los modelos de aprendizaje profundo, tanto las CNN como los Transformers, contienen numerosos parámetros cuyo ajuste no ofrece una explicación directa de cada predicción. Incluso cuando una clasificación es correcta, puede ser difícil determinar qué regiones o características de la radiografía han influido en ella. En un contexto clínico resulta necesario analizar si el modelo utiliza señales relacionadas con la patología o si su decisión puede estar condicionada por artefactos de la imagen o del conjunto de datos.

Esta cuestión es especialmente relevante en el ámbito clínico, donde las decisiones deben poder analizarse y auditarse. La inteligencia artificial explicable estudia métodos para hacer más comprensibles las decisiones de los modelos de aprendizaje automático (Holzinger & al., 2022; Linardatos, Papastefanopoulos, & Kotsiantis, 2021; Tjoa & Guan, 2020). En este proyecto, los mapas muestran las regiones que contribuyen a la salida del modelo, pero no constituyen por sí mismos una prueba de causalidad ni garantizan la validez clínica de la predicción (Rajaraman & al., 2019).

1.6.2. Técnicas de explicabilidad

Este proyecto utiliza tres familias de técnicas de explicabilidad:

Mapas de prominencia o Saliency Maps (Simonyan, Vedaldi, & Zisserman, 2014): la idea es preguntarse directamente al modelo qué píxeles de la imagen han influido más en su decisión. Para ello se calcula la derivada (el gradiente) de la salida de la red respecto a cada píxel de la imagen de entrada. Intuitivamente, el gradiente indica cómo cambiaría la predicción si se modificase un píxel concreto: si un pequeño cambio en un píxel altera mucho la decisión, ese píxel es importante. El resultado es un mapa que resalta las regiones más influyentes.

SmoothGrad (Smilkov & al., 2017): los mapas de saliencia crudos suelen ser muy ruidosos, con pequeñas variaciones que no tienen significado clínico. SmoothGrad mejora esta situación añadiendo ruido aleatorio a la imagen de entrada y promediando los mapas de saliencia obtenidos sobre varias versiones con ruido. El resultado es un mapa más suave y estable, en el que los patrones realmente consistentes del modelo destacan sobre el ruido.

Grad-CAM (Selvaraju & al., 2017): Grad-CAM produce mapas de activación de clase, que resaltan las regiones de la imagen que más contribuyen a la clase predicha. Para ello combina los mapas de características de la última capa convolucional del modelo con los gradientes de la clase predicha, ponderando cada mapa de características por su importancia. El resultado es un mapa de calor grueso y fácil de interpretar, que se superpone a la radiografía original. Grad-CAM es especialmente valioso en medicina porque sus mapas son suaves y corresponden de forma natural a las regiones anatómicas que el clínico puede inspeccionar.

Para las arquitecturas Transformer se emplea una variante complementaria: los mapas de atención, que extraen directamente los pesos de atención de la última capa del modelo. Como se explicó en la sección anterior, los Transformers deciden cuánta importancia prestar a cada parche de la imagen; esos pesos de atención indican, por tanto, qué regiones de la radiografía está "mirando" el modelo (Chefer, Gur, & Wolf, 2021). En este proyecto, los mapas de atención se obtienen promediando las cabezas de atención de la última capa.

1.6.3. La evaluación cuantitativa

Observar mapas de calor a simple vista es necesario, pero no es suficiente para una evaluación científica rigurosa: dos observadores pueden opinar de forma distinta sobre la calidad de un mismo mapa. Por ello, este proyecto incorpora métricas cuantitativas que puntúan de forma objetiva distintas propiedades de las explicaciones generadas. La evaluación de fidelidad mediante borrado e inserción se fundamenta en Petsiuk, Das y Saenko (2018), mientras que la consideración de medidas de concentración y complejidad se encuadra en la literatura general sobre evaluación de métodos XAI (Linardatos, Papastefanopoulos, & Kotsiantis, 2021):

- **Deletion AUC** (área bajo la curva de borrado): mide qué ocurre cuando se borran progresivamente de la imagen las regiones que el mapa considera más importantes. Si al borrarlas la predicción del modelo se degrada rápidamente, el mapa señala realmente las regiones determinantes (cuanto más baja la curva, mejor la explicación).
- **Insertion AUC** (área bajo la curva de inserción): el proceso inverso; se parte de una imagen borrosa y se van añadiendo progresivamente las regiones importantes. Si el modelo acierta pronto al incorporarlas, la explicación es buena (cuanto más alta, mejor).
- **Sparsity** (dispersión): mide si el mapa es compacto y se concentra en pocas regiones, o si por el contrario es una mancha difusa que lo abarca todo. Un buen mapa debe ser directo (Linardatos, Papastefanopoulos, & Kotsiantis, 2021).
- **Entropy** (entropía): evalúa la concentración de la información del mapa desde el punto de vista de la teoría de la información; complementa a la dispersión (Linardatos, Papastefanopoulos, & Kotsiantis, 2021).
- **Stability SSIM**: mide la consistencia de las explicaciones ante pequeñas perturbaciones de la imagen de entrada, utilizando el índice de similitud estructural SSIM (Wang & al., 2004). Un modelo estable produce mapas similares para imágenes casi idénticas.

1.6.4. La calibración

Existe una dimensión adicional de fiabilidad que a menudo se pasa por alto y que este proyecto incorpora como mejora respecto al estudio de referencia: la calibración. Cuando una red dice "neumonía con un 95% de confianza", ese 95% debería significar que, de cada cien casos clasificados con ese nivel de confianza, aproximadamente noventa y cinco son efectivamente neumonía. En la práctica, muchas redes modernas están mal calibradas: son demasiado optimistas, y sus probabilidades declaradas no se corresponden con su precisión real (Guo, Pleiss, Sun, & Weinberger, 2017).

La métrica estándar para medir esta desviación es el Expected Calibration Error (ECE), que cuantifica la diferencia media entre la confianza declarada por el modelo y su precisión real, agrupando las predicciones en intervalos de confianza. Complementariamente se utiliza el Brier Score, una métrica que evalúa la calidad global de las probabilidades predichas. La calibración es un aspecto crítico en entornos clínicos: una alta confianza en una predicción errónea puede tener consecuencias graves, y un sistema que sobrestima su seguridad no es clínicamente fiable. Incluir estas métricas permite evaluar no solo si los modelos aciertan, sino también si saben cuándo aciertan.

En este punto conviene aclarar, por honestidad con el lector, cómo se distribuyen las cuatro técnicas XAI entre las dos vías de uso de la plataforma. En la vía de diagnóstico de la aplicación web, el módulo de explicabilidad implementa las cuatro técnicas visuales descritas (Saliency Maps, SmoothGrad y Grad-CAM para redes convolucionales, y mapas de atención para Transformers), que se generan de forma automática en cada consulta del profesional sanitario. En la vía de laboratorio MLOps, los scripts del pipeline de entrenamiento generan los mapas de explicabilidad cualitativos (Grad-CAM y mapas de saliencia para las CNN, y mapas de saliencia para los Transformers) y calculan las métricas cuantitativas y de calibración descritas en esta sección.

1.7. Las barreras para la adopción clínica de la IA

A pesar del potencial de la inteligencia artificial en el diagnóstico por imagen, su adopción clínica plantea dificultades relacionadas con la transparencia, la generalización y la facilidad de uso. La primera se refiere a la dificultad de interpretar las predicciones de los modelos. vitalXAI aborda esta cuestión mediante los mapas de explicabilidad y las métricas de evaluación descritos en la sección anterior. Estos elementos aportan información adicional sobre el comportamiento del modelo, pero no sustituyen la validación clínica.

La segunda barrera es la robustez y la generalización. Un modelo puede obtener buenos resultados en las condiciones controladas del laboratorio y comportarse peor ante poblaciones, equipos o protocolos de adquisición diferentes (Nagendran & al., 2020). Por este motivo, la validación externa sobre imágenes no utilizadas durante el entrenamiento es una comprobación relevante antes de considerar un posible uso clínico (Zech & al., 2018). En este proyecto se utiliza el COVID-19 Radiography Database como fuente de imágenes adultas para explorar ese cambio de distribución.

El COVID-19 Radiography Database fue desarrollado por investigadores de la Universidad de Qatar y la Universidad de Dhaka para apoyar la investigación con radiografías de tórax (Chowdhury & al., 2020; Rahman & al., 2021). En este proyecto se utiliza como fuente de imágenes adultas para la validación externa de modelos entrenados con imágenes pediátricas. Los modelos se entrenan para distinguir neumonía de normalidad, no para detectar específicamente COVID-19. Además, la combinación de fuentes y condiciones de adquisición puede introducir señales que permitan identificar el origen de las imágenes en lugar de la patología (DeGrave, Janizek, & Lee, 2021). Por ello, los resultados de esta cohorte deben considerarse evidencia complementaria y no una demostración de generalización clínica.

La tercera barrera es la usabilidad. Muchas herramientas de investigación requieren instalar entornos, ejecutar scripts o interpretar ficheros de resultados, lo que puede dificultar su uso por profesionales sanitarios. Las prácticas MLOps buscan automatizar y organizar el ciclo de vida de los modelos, desde el entrenamiento y la validación hasta el despliegue y la monitorización (Kreuzberger, Kühl, & Hirschl, 2023). En este contexto, una interfaz que reúna estas operaciones puede reducir la carga técnica para el usuario.

vitalXAI aborda esta barrera mediante un laboratorio con asistente conversacional. El usuario puede describir en lenguaje natural la configuración del experimento y revisar los parámetros antes de lanzarlo, sin tener que escribir el código del pipeline. La interfaz no elimina la complejidad del entrenamiento, sino que la organiza y la presenta mediante operaciones orientadas al flujo de trabajo del usuario.

1.8. Los asistentes conversacionales y los grandes modelos de lenguaje

1.8.1. ¿Qué son los grandes modelos de lenguaje?

Dado que el asistente conversacional es uno de los componentes de vitalXAI, es necesario situar esta tecnología en su contexto. Los grandes modelos de lenguaje son redes neuronales basadas habitualmente en la arquitectura Transformer y entrenadas con grandes colecciones de texto. Durante el entrenamiento aprenden a predecir elementos posteriores de una secuencia, lo que les permite modelar regularidades del lenguaje y generar texto a partir de un contexto dado (Brown & al., 2020; Vaswani & al., 2017).

El entrenamiento para seguir instrucciones y responder en formato de diálogo permite adaptar estos modelos a aplicaciones conversacionales. Entre las técnicas utilizadas se encuentra el aprendizaje por refuerzo con retroalimentación humana (RLHF), aunque su aplicación concreta depende del modelo y del proveedor (Ouyang & al., 2022). En vitalXAI, el modelo de lenguaje se utiliza para extraer parámetros de configuración a partir de los mensajes del usuario y no para realizar el diagnóstico.

La aplicación de los grandes modelos de lenguaje al ámbito de la medicina es un campo de investigación muy activo. Se han publicado numerosos trabajos que exploran su uso como apoyo en tareas clínicas: responder preguntas médicas, redactar informes o asistir en el diagnóstico, con resultados prometedores pero también con advertencias importantes sobre la necesidad de validación rigurosa antes de su uso clínico (Thirunavukarasu & al., 2023; Singhal & al., 2023). En este proyecto, sin embargo, el modelo de lenguaje no desempeña funciones de diagnóstico: su papel es el de un asistente de configuración que interpreta las peticiones del usuario sobre qué experimento de entrenamiento quiere lanzar y las traduce a los parámetros técnicos que necesita el sistema.

1.8.2. El panorama de los modelos de lenguaje y la elección inicial de Llama 3

En el momento de desarrollo de este proyecto existen numerosos grandes modelos de lenguaje, con características muy diferentes. Por un lado están los modelos propietarios y cerrados, como GPT-4 de OpenAI, Claude de Anthropic o Gemini de Google, accesibles únicamente a través de sus APIs comerciales. Estos modelos son muy potentes, pero su código no está disponible, el usuario no puede adaptarlos ni ejecutarlos en su propia infraestructura, y su uso está sujeto a las políticas y precios de cada proveedor. Por otro lado están los modelos de acceso abierto, entre los que destacan la familia Llama de Meta y los modelos de Mistral AI, cuyos pesos son publicados y pueden descargarse y utilizarse libremente (Touvron & al., 2023; Jiang & al., 2023).

La elección inicial de Llama 3 (Meta AI, 2024) para el asistente de vitalXAI se justificaba por varias razones. En primer lugar, era un modelo de pesos abiertos, lo que se alineaba con el espíritu del proyecto: transparencia, reproducibilidad y ausencia de dependencia de un proveedor concreto. En segundo lugar, ofrecía un rendimiento competitivo con los mejores modelos propietarios en razonamiento y seguimiento de instrucciones, según las evaluaciones publicadas por el propio equipo de Meta, manteniendo un tamaño razonable (el modelo inicialmente empleado, Llama 3.3 de 70.000 millones de parámetros, ofrecía una relación calidad-coste excelente). En tercer lugar, su licencia permitía tanto el uso en investigación como el uso comercial, lo que facilitaba la eventual transferencia de la tecnología. Y en cuarto lugar, al tratarse de un modelo abierto, era posible adaptarlo o sustituirlo en el futuro sin necesidad de reescribir el sistema.

La ejecución del modelo se delega en la plataforma Groq (Groq, 2024). Groq ofrece inferencia de modelos de lenguaje a través de una API en la nube, pero con una particularidad hardware: utiliza unos chips propios denominados unidades de procesamiento de lenguaje (LPU, del inglés Language Processing Units), diseñados específicamente para ejecutar modelos de Transformer a velocidades extremadamente altas y con una latencia muy reducida, muy por debajo de la que ofrecen las GPUs convencionales. Esta velocidad es clave para la experiencia de usuario: un asistente conversacional que responde con fluidez se siente natural y cercano a lo que el usuario espera de un sistema tipo ChatGPT, mientras que un asistente lento arruinaría la sensación de conversación. Además, Groq ofrece un plan gratuito con cuotas suficientes para un proyecto académico, lo que encaja con las restricciones de coste de un Trabajo Fin de Grado.

El asistente conversacional de vitalXAI se configura con un prompt de sistema cuidadosamente diseñado (una técnica denominada prompt engineering) que define su comportamiento como experto en MLOps médico. El asistente guía al usuario, en lenguaje natural, por los cinco parámetros que definen un experimento de entrenamiento: la ruta del dataset, las arquitecturas a entrenar, el número de épocas, el tamaño de lote y la tasa de aprendizaje. Cuando el usuario ha proporcionado todos los datos, el asistente devuelve un objeto JSON estructurado con la configuración completa, que el sistema interpreta automáticamente para lanzar el pipeline de entrenamiento. El asistente está además internacionalizado en cuatro idiomas (español, inglés, chino e hindi), ampliando su accesibilidad a usuarios de distintas procedencias.

Capítulo 2. Metas y Propósitos del Proyecto

Este Trabajo Fin de Grado parte de la necesidad de reunir en un mismo entorno el diagnóstico asistido por inteligencia artificial, la explicabilidad de las predicciones y la evaluación de los modelos. En la práctica, estas capacidades suelen aparecer separadas entre repositorios de código, cuadernos de experimentación y plataformas cerradas, lo que puede dificultar la reproducción de los procedimientos y la inspección de los métodos utilizados (Liu & al., 2019; Varoquaux & Cheplygina, 2019). Aunque diversos estudios han mostrado resultados prometedores en imagen médica, esos resultados dependen de los datos, del protocolo y del entorno de evaluación empleados (Esteve & al., 2017; De Fauw & al., 2018; McKinney & al., 2020). La literatura también advierte de que la aplicación clínica requiere validación rigurosa, análisis estadístico y una exposición clara de las limitaciones (Nagendran & al., 2020; Kelly & al., 2019; Aggarwal & al., 2021). En este contexto se plantea vitalXAI como una plataforma experimental y de apoyo, no como un producto sanitario certificado.

Este capítulo define los fines del proyecto. Primero se presenta el objetivo general y, después, los objetivos específicos que lo descomponen en metas de carácter científico, metodológico y técnico. Los objetivos describen el resultado y el alcance previstos, mientras que las decisiones sobre arquitecturas, técnicas e implementación se justifican en los capítulos correspondientes de análisis, diseño y codificación.

El objetivo general de este Trabajo Fin de Grado es desarrollar vitalXAI, una plataforma web MLOps que integre inteligencia artificial explicable para el diagnóstico asistido de neumonía mediante radiografías de tórax. La plataforma debe reunir en un mismo entorno la ejecución de diagnósticos, la comparación de arquitecturas y la evaluación de las explicaciones generadas por los modelos, con criterios de reproducibilidad y trazabilidad. Su orientación es experimental y de apoyo a la investigación. La interfaz se diseña para que profesionales sanitarios e investigadores puedan utilizar estas capacidades sin necesidad de programar, sin que ello convierta la plataforma en un producto sanitario certificado ni permita sustituir la valoración clínica.

Del objetivo general se derivan once objetivos específicos. Se organizan en dos bloques. El primero, científico y metodológico, reúne la fundamentación teórica, el diseño experimental y la evaluación de los modelos. El segundo, relacionado con la ingeniería web, reúne la construcción de la plataforma, su interfaz, su seguridad, su usabilidad y los servicios que ofrece al usuario.

OE1 - Revisar el estado del arte. Analizar la literatura científica sobre la aplicación de la inteligencia artificial al diagnóstico por imagen médica, con especial atención a la detección de neumonía mediante radiografías de tórax, los métodos de inteligencia artificial explicable y las prácticas de MLOps para gestionar el ciclo de vida de los modelos. Esta revisión, realizada en la tarea 0.2 del Sprint 0, sirve de base para las decisiones de diseño e implementación. También permite situar el proyecto frente al estudio de Kermany y colaboradores (Kermany & al., 2018) y considerar aspectos como la calibración, la validación estadística y la reproducibilidad (Park & Han, 2018; Varoquaux & Cheplygina, 2019).

OE2 - Diseñar e implementar la arquitectura de persistencia. Crear una infraestructura que soporte el acceso concurrente de varios usuarios y la gestión del historial de diagnósticos y experimentos. Esta capa debe registrar los parámetros de cada inferencia, como la imagen, el modelo, la predicción, la confianza y los artefactos generados, y conservar la configuración, las métricas y los resultados de las sesiones de entrenamiento para poder consultarlos posteriormente.

OE3 - Diseñar e implementar el control de acceso. Desarrollar un mecanismo de autenticación y autorización que mantenga separados los datos de cada usuario y proteja las operaciones de la plataforma. El objetivo incluye el registro, el inicio y el cierre de sesión, además de medidas frente a amenazas habituales de las aplicaciones web (OWASP, 2021), como la inyección, la falsificación de peticiones entre sitios y los intentos de acceso por fuerza bruta.

OE4 - Implementar el pipeline de entrenamiento. Automatizar el entrenamiento y la evaluación de un conjunto amplio de arquitecturas de deep learning, tanto redes convolucionales como modelos basados en atención, mediante validación cruzada y particiones balanceadas. El pipeline debe incluir el preprocesamiento de las imágenes, la transferencia de conocimiento, el cálculo de métricas y el almacenamiento de los artefactos. Las diferencias de optimización entre familias deben quedar documentadas para que la comparación pueda interpretarse correctamente.

OE5 - Desarrollar el módulo de explicabilidad (XAI). Implementar dos formas complementarias de analizar las decisiones del modelo. La primera generará mapas visuales para inspeccionar las zonas de la radiografía que influyen en cada predicción. La segunda calculará métricas sobre la fidelidad de esas explicaciones. Además, se evaluará la calibración para comprobar si la confianza del modelo se corresponde con su precisión.

OE6 - Ejecutar la validación externa y el análisis estadístico. Evaluar los modelos sobre una cohorte independiente, sin reaprendizaje, para observar cómo responden ante una población, unos equipos y unos protocolos de imagen distintos. El análisis estadístico debe distinguir entre resultados descriptivos y contrastes inferenciales, tener en cuenta la dependencia entre pliegues y aplicar correcciones por comparaciones múltiples cuando corresponda (Park & Han, 2018; Nagendran & al., 2020; Nadeau & Bengio, 2003).

OE7 - Favorecer la reproducibilidad y la trazabilidad de los experimentos. Fijar las semillas aleatorias, guardar la configuración de cada experimento y registrar los resultados y artefactos generados. Estas medidas permiten comprobar y repetir los resultados bajo las mismas condiciones, dentro de las limitaciones propias del hardware y del entorno de ejecución (Varoquaux & Cheplygina, 2019).

OE8 - Desarrollar la interfaz clínica de diagnóstico asistido. Diseñar una interfaz web que permita cargar radiografías, obtener el diagnóstico y su confianza, consultar los mapas de explicabilidad y descargar un informe PDF desde un mismo panel. La interfaz debe estar dirigida a usuarios sin formación específica en inteligencia artificial y seguir las pautas de interacción humano-inteligencia artificial (Amershi & al., 2019).

OE9 - Desarrollar el laboratorio MLOps y el asistente conversacional. Integrar un laboratorio de entrenamiento en la plataforma web con un asistente conversacional que ayude a configurar y lanzar experimentos sin escribir código. El sistema debe orquestar el pipeline descrito en los objetivos OE4, OE5 y OE6 y presentar sus resultados, comparativas e informes. La elección de este tipo de interfaz se apoya en trabajos que estudian el uso de agentes conversacionales en aplicaciones sanitarias (Laranjo & al., 2018; Tudor Car & al., 2020).

OE10 - Diseñar e implementar el procesamiento asíncrono de tareas. Ejecutar los entrenamientos, los análisis de explicabilidad y las validaciones externas sin bloquear la interfaz. El sistema debe mostrar el estado y el progreso de cada tarea, y permitir cancelar los trabajos pendientes cuando sea posible. Esta separación entre las solicitudes web y los procesos prolongados reduce parte de la complejidad operativa de los sistemas de aprendizaje automático (Sculley & al., 2015).

OE11 - Incorporar internacionalización. Añadir soporte para que la interfaz, los informes y el asistente conversacional estén disponibles en español, inglés, chino e hindi. Con ello se amplía el uso de la plataforma a otros entornos lingüísticos.

En conjunto, estos objetivos cubren la revisión del problema, el diseño experimental, la construcción de la plataforma y sus mecanismos de evaluación. Cada uno se relaciona a lo largo de la memoria con los requisitos, casos de uso y componentes de diseño que le corresponden. La trazabilidad, la evidencia disponible y las limitaciones de su comprobación se resumen en el capítulo 33, sin confundir la verificación funcional de la plataforma con la interpretación de los resultados de los modelos. La administración avanzada de la plataforma queda fuera de este conjunto de objetivos y del alcance planificado del proyecto.

Capítulo 3. Estructura organizativa y partes interesadas del proyecto

Este capítulo describe la estructura organizativa del proyecto y las personas e instituciones relacionadas con su desarrollo. Al tratarse de un trabajo unipersonal y multidisciplinar, es necesario distinguir quién participa, qué función desempeña y qué tipo de relación mantiene con el alumno desarrollador. El capítulo se divide en tres apartados. El apartado 3.1 presenta el organigrama y las relaciones entre las partes interesadas. El apartado 3.2 identifica esos perfiles y resume su papel general. El apartado 3.3 detalla sus responsabilidades y presenta la matriz RACI para las principales fases y entregables.

3.1. Estructura organizativa del proyecto

El organigrama representa las relaciones entre las partes interesadas del proyecto. Incluye nueve nodos: Marc Ríos Cadenas, Iván Segura Carmona, Aurelio López Fernández, Luis Carmona Berdugo, el consultor de persistencia y bases de datos Vicente de Vides Rodríguez, Rubén Pérez Garrido, el Tribunal de Evaluación del TFG, el Laboratorio Synergia y los facultativos e investigadores clínicos. Estos dos últimos representan a los usuarios objetivo de la plataforma y no mantienen una relación jerárquica con los integrantes del equipo.

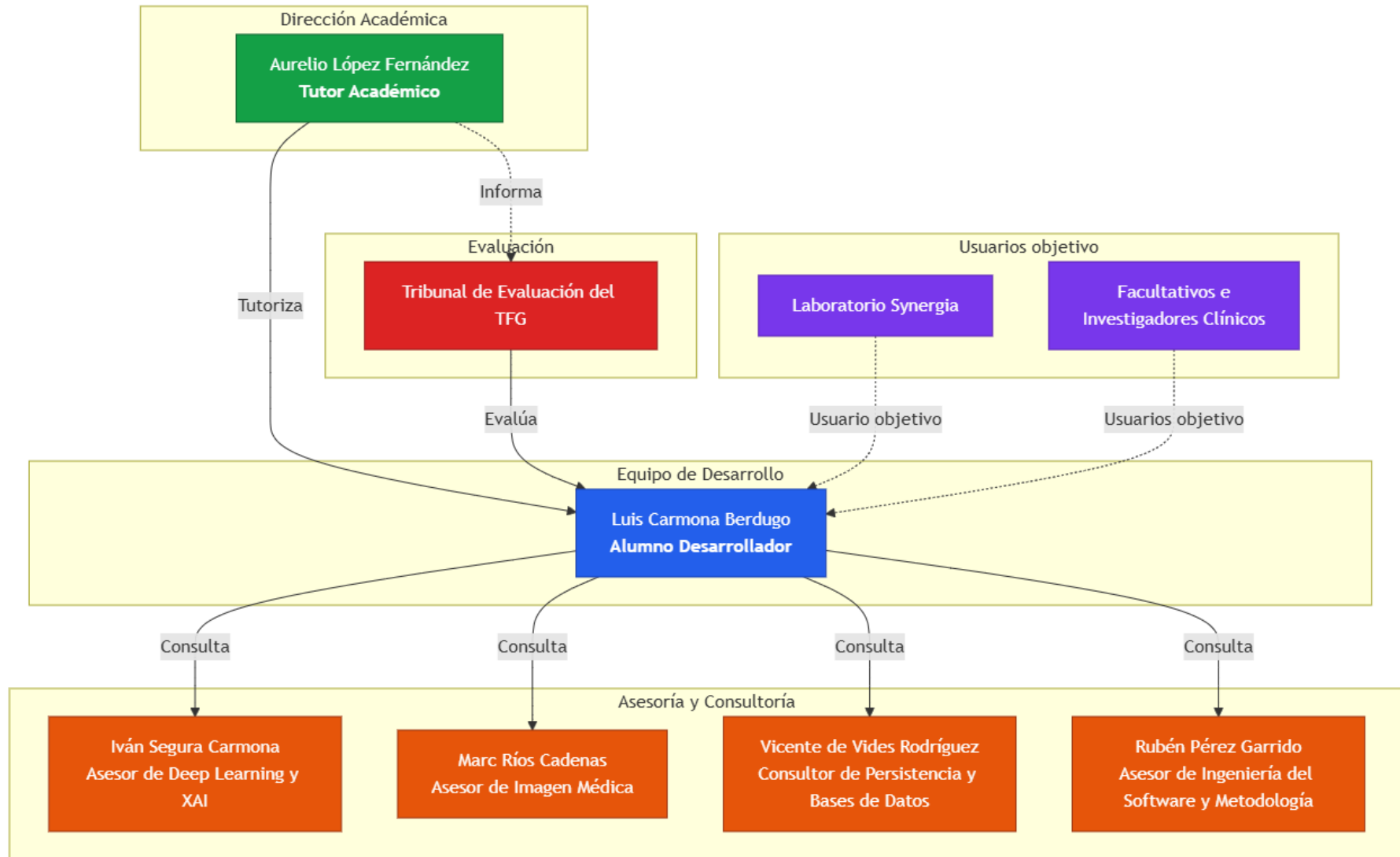


Ilustración x - Organigrama del Proyecto

3.2. Partes interesadas del proyecto

La identificación de las partes interesadas permite distinguir quién se ve afectado por el proyecto, quién aporta conocimiento y quién participa en la revisión del resultado. En este caso conviven el alumno desarrollador, el tutor, varios asesores, el tribunal y los colectivos que representan a los usuarios objetivo. A continuación se resume el papel general de cada uno:

- **Luis Carmona Berdugo (Alumno Desarrollador):** Asume la responsabilidad principal de la ejecución técnica y documental. Diseña, implementa, prueba y documenta el sistema, y coordina la comunicación con el tutor y los asesores.
- **Aurelio López Fernández (Tutor Académico):** Supervisa el desarrollo académico del trabajo, revisa las entregas y orienta sobre su alcance, metodología y presentación.
- **Tribunal de Evaluación del TFG (Cliente / Evaluador):** Examina la memoria, el sistema y la defensa oral conforme a los criterios académicos de la universidad, y determina la calificación final.
- **Vicente de Vides Rodríguez (Consultor de Persistencia y Bases de Datos, Rol Consultivo):** Aporta asesoramiento puntual sobre el diseño, el modelado y la persistencia de los datos. La decisión final y la implementación corresponden al alumno.
- **Iván Segura Carmona (Asesor de Deep Learning y XAI):** Aporta asesoramiento sobre las arquitecturas de aprendizaje profundo, las técnicas de explicabilidad y la interpretación de los resultados experimentales.
- **Marc Ríos Cadenas (Asesor de Imagen Médica):** Aporta contexto sobre la interpretación de radiografías y sobre el uso previsto de la plataforma en un entorno clínico, sin participar en la implementación.
- **Rubén Pérez Garrido (Asesor de Ingeniería del Software y Metodología):** Es consultado puntualmente sobre la metodología de desarrollo, las prácticas de ingeniería y la calidad del proceso.
- **Laboratorio Synergia (Parte interesada / Usuario objetivo):** Representa un posible entorno de adopción cuyas líneas de investigación son afines a los objetivos de vitalXAI. En el proyecto no se realizaron sesiones de uso ni evaluaciones con este laboratorio.
- **Facultativos e investigadores clínicos (Usuarios objetivo):** Constituyen el colectivo al que se dirige el sistema. Sus necesidades se utilizan como referencia para definir el alcance y las funcionalidades, pero no se realizó una evaluación empírica con usuarios reales durante el proyecto.

3.3. Responsabilidades, funciones y participación de las partes interesadas

La definición de responsabilidades ayuda a ordenar el trabajo y a distinguir quién ejecuta, quién aprueba y quién aporta asesoramiento. Esta distinción es especialmente útil en un proyecto multidisciplinar, porque las partes interesadas intervienen en fases diferentes y con niveles de participación distintos.

Alumno Desarrollador (Luis Carmona Berdugo, LCB): Ejecuta el proyecto y asume la responsabilidad de sus decisiones técnicas y documentales. Revisa el estado del arte, diseña la arquitectura, implementa los módulos y pipelines, prepara las pruebas, analiza los resultados disponibles y redacta la memoria y el manual de usuario. También mantiene la planificación, gestiona los riesgos y coordina la comunicación con el tutor y los asesores. La validez y el alcance de los resultados deben interpretarse según las condiciones experimentales y las limitaciones documentadas en la memoria.

Tutor Académico (Aurelio López Fernández, ALF): Revisa las entregas y proporciona retroalimentación sobre el rigor académico, la coherencia del trabajo y su adecuación a los objetivos del TFG. Orienta al alumno en las decisiones de alcance y presentación, pero la ejecución y las decisiones finales del proyecto corresponden al alumno desarrollador.

Tribunal de Evaluación del TFG (T. TFG): Evalúa la memoria, el sistema y la defensa oral según los criterios académicos establecidos por la universidad. No participa directamente en el desarrollo y su intervención se produce en la evaluación final. La claridad de la exposición, la justificación de las decisiones y la delimitación de las conclusiones forman parte de los aspectos que debe valorar.

Consultor de Persistencia y Bases de Datos (Vicente de Vides Rodríguez, VVR): Su intervención se concentra en la tarea 1.1 y tiene carácter consultivo. Aporta criterios sobre el diseño, el modelado y la optimización del esquema de datos, mientras que la decisión definitiva y la implementación corresponden al alumno desarrollador.

Asesor de Deep Learning y XAI (Iván Segura Carmona, ISC): Participa con carácter consultivo en la revisión de las arquitecturas, las técnicas de explicabilidad, las métricas y los resultados del benchmarking. El alumno conserva la responsabilidad de diseñar, implementar, ejecutar y documentar estas partes del proyecto.

Asesor de Imagen Médica (Marc Ríos Cadenas, MRC): Participa de forma consultiva en la revisión del contexto médico y radiológico, especialmente en la interpretación de radiografías y de los mapas de explicabilidad. Sus observaciones sirven como referencia de dominio; la traducción de esas observaciones a requisitos, diseño e implementación corresponde al alumno.

Asesor de Ingeniería del Software y Metodología (Rubén Pérez Garrido, RPG): Su participación es puntual y consultiva. Aporta criterios sobre la aplicación de la metodología, las prácticas de desarrollo guiado por pruebas y la calidad del proceso de ingeniería en las tareas de implementación. La ejecución concreta de dichas tareas y la aplicación de la metodología son responsabilidad del alumno.

Laboratorio Synergia y Facultativos e investigadores clínicos (LS/FIC): Mantienen una posición externa al núcleo de programación y representan a los usuarios objetivo del sistema. No participaron en el desarrollo ni realizaron sesiones de uso o evaluaciones de usabilidad. Por tanto, sus necesidades se consideran una referencia de diseño, no una medida empírica de la utilidad de la plataforma. La usabilidad se aborda mediante los requisitos y las decisiones de interfaz, pero queda como una limitación del proyecto la ausencia de una evaluación con usuarios reales.

La matriz RACI se utiliza en este proyecto para resumir la participación de cada interesado en las fases y entregables principales. Sus letras representan Responsable, quien ejecuta el trabajo; Accountable, quien asume la aprobación final; Consulted, quien aporta conocimiento; e Informed, quien recibe información sobre el avance o el resultado.

En esta aplicación, R identifica al alumno como responsable de ejecutar la tarea y A al tutor como responsable de su supervisión académica, salvo en la defensa final, cuya evaluación corresponde al tribunal. Los asesores y el consultor aparecen como C cuando su conocimiento puede orientar una decisión, mientras que I indica que el interesado solo recibe información. Un guion representa que no se ha asignado participación en esa fase o entregable.

A continuación, se presenta la matriz RACI del proyecto, en la que las filas recogen las principales fases y entregables del ciclo de vida del proyecto, y las columnas representan los interesados identificados en el apartado anterior.

Fase / Entregable	LCB	ALF	T. TFG	VVR	ISC	MRC	RPG	LS/FIC
Revisión del estado del arte	R	A	I	-	C	C	-	-
Diseño de la arquitectura del sistema	R	A	I	C	-	-	-	-
Diseño e implementación de la base de datos	R	A	I	C	-	-	-	-
Implementación del pipeline de entrenamiento CNN y Transformer	R	A	I	-	C	-	C	-
Implementación del módulo XAI (cualitativo y cuantitativo)	R	A	I	-	C	C	C	-
Desarrollo de la interfaz web y chatbot conversacional	R	A	I	-	-	-	C	-
Seguridad y control de acceso	R	A	I	-	-	-	C	-

Fase / Entregable	LCB	ALF	T. TFG	VVR	ISC	MRC	RPG	LS/FIC
Pruebas y verificación del sistema	R	A	I	-	C	C	C	-
Laboratorio MLOps	R	A	I	-	C	-	C	-
Procesamiento asíncrono de tareas	R	A	I	-	-	-	C	-
Internacionalización de la plataforma	R	A	I	-	-	-	C	-
Panel de administración	R	A	I	-	-	-	C	-
Gestión de riesgos	R	A	I	-	-	-	C	-
Benchmarking y análisis de resultados	R	A	I	-	C	C	-	-
Validación del contexto clínico	R	A	I	-	-	C	-	-
Validación externa y tests estadísticos	R	A	I	-	C	-	-	-
Documentación del TFG	R	A	I	-	-	-	I	-
Defensa y evaluación final	R	C	A	-	-	-	-	-

Tabla x - Matriz RACI

Capítulo 4. Marco Metodológico de Gestión

Todo proyecto necesita un marco de gestión que ordene su ejecución y que sea coherente con la naturaleza del trabajo. Este capítulo describe cómo se gestiona el presente proyecto: los criterios que llevaron a elegir un enfoque ágil, la adaptación de Scrum a un desarrollo unipersonal, las herramientas de seguimiento utilizadas y el alcance de la metodología. La gestión se plantea como un apoyo práctico para ordenar el trabajo, hacer visible su estado y adaptar las tareas cuando los resultados o las restricciones del proyecto lo requieran.

4.1. *Justificación del enfoque metodológico*

La naturaleza de este proyecto combina desarrollo de software con investigación computacional aplicada al diagnóstico por imagen médica. Esta combinación introduce incertidumbre técnica: el comportamiento de las arquitecturas de deep learning frente a los conjuntos de datos no siempre es predecible, y los resultados del benchmarking pueden obligar a ajustar el diseño, los hiperparámetros o el alcance inicialmente previsto. Además, el proyecto cuenta con un único desarrollador y una duración acotada, por lo que una metodología con una carga procedimental elevada resultaría desproporcionada para esta escala de trabajo. La metodología debe seleccionarse atendiendo al tamaño del proyecto, la dinámica de sus requisitos y el entorno en el que se desarrolla (Boehm & Turner, 2004).

A la vista de estas características, se ha adoptado un marco ágil adaptado de Scrum (Schwaber & Sutherland, 2020), complementado con un tablero Kanban para gestionar visualmente el flujo de trabajo. La iteración permite revisar las decisiones a partir de los resultados obtenidos y detectar errores antes de que se acumulen. Esta flexibilidad resulta adecuada tanto para la comparación de arquitecturas y el ajuste de hiperparámetros como para un proyecto unipersonal con recursos y plazo limitados. La literatura sobre métodos ágiles y su aplicación también respalda la adaptación de estas prácticas a contextos de desarrollo individual (Dybå & Dingsøyr, 2008; Sutherland, 2014).

4.2. Roles y recursos del proyecto

4.2.1. Roles dentro del marco de trabajo

Scrum define las responsabilidades de Product Owner, Scrum Master y Developers (Schwaber & Sutherland, 2020). En este proyecto, al tratarse de un desarrollo unipersonal, se adopta la siguiente distribución práctica:

- **Product Owner, en sentido adaptado (Aurelio López Fernández):** El tutor académico orienta sobre el alcance, revisa las entregas y aporta retroalimentación sobre su adecuación a los objetivos del TFG. No sustituye al alumno en la toma de decisiones técnicas ni en la ejecución del proyecto.
- **Scrum Master y Developer (Luis Carmona Berdugo):** El alumno concentra ambos roles. Organiza el trabajo, mantiene los artefactos de seguimiento y ejecuta la totalidad de la implementación, las pruebas y la documentación.
- **Asesores científicos (consultor de persistencia y bases de datos, asesores de deep learning y XAI, de imagen médica y de ingeniería del software):** La terminología de Scrum omite esta figura. En este proyecto intervienen como especialistas externos, aportando conocimiento puntual cuando una decisión requiere experiencia específica.

4.2.2. Dedicación y recursos materiales

La dedicación de los recursos humanos no es uniforme a lo largo del proyecto, sino que varía en función de la naturaleza de cada tarea y de la participación concreta de cada persona. El alumno asume la mayor parte de la ejecución, pero su porcentaje de dedicación también se ajusta a cada actividad: en las reuniones inicial y final comparte la dedicación al 50 % con el tutor, mientras que en las tareas técnicas suele asumir el 100 %. El tutor, en su papel de Product Owner, participa al 50 % en las tareas 0.1 y 8.3. Los asesores intervienen con porcentajes variables, que en el cronograma oscilan entre el 3 %, el 5 %, el 8 %, el 10 %, el 15 % y el 25 %, según la tarea y el tipo de revisión o consulta requerida. Estas proporciones son las que sustentan el cálculo de horas y costes del apartado 5.3; no existe, por tanto, una dedicación uniforme de los asesores.

En cuanto a los recursos materiales, el proyecto requiere un equipo con capacidad de cómputo GPU para el entrenamiento de los modelos de deep learning, particularmente durante las fases de validación cruzada de las arquitecturas CNN y Transformer. Se emplea una estación de trabajo local equipada con una GPU NVIDIA compatible con CUDA, cuyo coste de uso está incluido en la infraestructura hardware disponible para el desarrollo del TFG, sin necesidad de recurrir a instancias de computación en la nube. Todo el software utilizado es de código abierto y gratuito, por lo que no se incurre en costes de licenciamiento.

4.3. Ciclo de trabajo iterativo

4.3.1. Los sprints y la cadencia del trabajo

El Sprint es el evento central de Scrum y, en este proyecto, se entiende como un ciclo de trabajo con un objetivo y un resultado verificable (Schwaber & Sutherland, 2020). Los nueve sprints tienen duración desigual, porque la planificación real reserva más tiempo a la fase inicial, a los bloques técnicos de mayor complejidad y a la documentación final. No se aplica Scrum de forma ortodoxa ni se mantiene una cadencia temporal fija: se conservan la planificación, la revisión, la retrospectiva, el backlog y el incremento, adaptados a las restricciones académicas y al volumen de cada bloque. Cuando la naturaleza del trabajo lo permite, cada sprint integra diseño, implementación y pruebas. Las pruebas se realizan de forma continua, incluidas las de seguridad de los componentes afectados.

La ejecución de cada tarea de implementación se apoya, además, en un ciclo de desarrollo basado en especificaciones y guiado por pruebas (SDD/TDD). Primero se define el comportamiento esperado, después se escriben las pruebas correspondientes y, finalmente, se implementa el código necesario para superarlas y se refactoriza el resultado (Beck, 2000). Esta práctica favorece que las funcionalidades cubiertas por el ciclo dispongan de pruebas automatizadas y contribuye a la calidad y reproducibilidad del desarrollo.

4.3.2. Ceremonias y reuniones de seguimiento

Cada Sprint se organiza, cuando la fase y la disponibilidad de los participantes lo permiten, mediante actividades adaptadas de planificación, revisión y retrospectiva. El alumno desglosa los elementos del Product Backlog en tareas concretas y utiliza el tablero para realizar el seguimiento. Las revisiones con el tutor se celebran en los hitos previstos y mediante reuniones puntuales. No se presupuestan dieciocho ceremonias formales, y la retrospectiva se realiza como revisión del proceso al cierre de cada bloque cuando resulta aplicable.

En este proyecto no se realiza ninguna Daily Scrum, ya que no resulta operativo en un contexto de desarrollo individual. Para cubrir la función de estas reuniones diarias se realizan reuniones periódicas de seguimiento con el tutor y, cuando es necesario, con los asesores. En estas reuniones el alumno presenta los avances realizados, expone las dudas técnicas surgidas, recibe retroalimentación sobre los resultados obtenidos y acuerda las tareas a acometer hasta el siguiente encuentro. El registro de estas reuniones queda recogido en el Anexo de Seguimiento del Proyecto, que actúa como bitácora formal del proceso a lo largo del ciclo de vida del desarrollo.

La ejecución del proyecto no ha seguido un calendario artificialmente uniforme, sino la cadencia real que impone un trabajo académico con un único desarrollador. El proyecto se inició a finales de noviembre de 2025. Durante el periodo comprendido entre ese momento y finales de febrero de 2026, la actividad se limitó a un par de reuniones de seguimiento con el tutor y a una pequeña inicialización del marco tecnológico: la selección de las librerías, la preparación del entorno y el arranque del repositorio. A partir de principios de marzo de 2026 comenzó el desarrollo completo del sistema, que se prolongó hasta principios de junio de 2026, con algunas reuniones puntuales, pocas pero decisivas, con el tutor y, cuando fue necesario, con los asesores, para revisar los avances y acordar los pasos siguientes. Desde principios de junio de 2026 hasta septiembre de 2026 se desarrolló la documentación completa del proyecto, junto con las ejecuciones parciales del benchmarking, las correcciones y la entrega, de nuevo con algunas reuniones de seguimiento puntuales. Esta distribución temporal, lejos de ser un detalle administrativo, refleja la naturaleza real del trabajo: la fase de implementación concentró la aplicación efectiva de los sprints, mientras que las fases inicial y final fueron más intensivas en documentación, revisión y cierre.

4.3.3. Artefactos del proceso de gestión

El proceso de gestión se apoya en tres artefactos de Scrum. El Product Backlog reúne y ordena el trabajo pendiente: requisitos del sistema, tareas de implementación, experimentos de benchmarking y documentos del TFG. Lo mantiene el alumno y lo revisa con el tutor en los puntos de seguimiento. El Sprint Backlog contiene las tareas seleccionadas para el bloque en curso, desglosadas de forma que puedan guiar el trabajo. El Increment reúne los elementos completados durante el Sprint, una vez comprobado que cumplen el criterio de finalización acordado, la Definition of Done (Schwaber & Sutherland, 2020).

4.4. Seguimiento visual del trabajo con Kanban

Además del marco Scrum, se utiliza un tablero Kanban, gestionado a través de GitHub Projects, como instrumento visual para administrar el flujo de trabajo. La elección de GitHub Projects responde a que el repositorio del proyecto está alojado en GitHub, lo que permite asociar de manera directa las tareas del tablero a los commits, pull requests e issues del código, unificando toda la trazabilidad del desarrollo en una única plataforma.

El tablero se organiza en cuatro columnas que representan los estados por los que transita cada tarea: Por Hacer (To Do), para las tareas planificadas y aún no iniciadas; En Progreso (In Progress), para las tareas en desarrollo activo; En Revisión (In Review), para las tareas completadas por el alumno y pendientes de validación por el tutor; y Hecho (Done), para las tareas validadas que cumplen la Definition of Done.

La combinación del marco ágil adaptado y Kanban permite unir la planificación iterativa con la visibilidad continua del trabajo (Kniberg & Skarin, 2010; Reddy, 2016). El tablero ayuda a limitar el trabajo en curso y a detectar bloqueos (Anderson, 2010), algo útil en un proyecto unipersonal en el que el alumno debe organizar y priorizar su propia carga de trabajo.

4.5. Alcance de la metodología

El marco de gestión descrito en este capítulo se aplica exclusivamente al trabajo comprendido en este Plan de Proyecto, es decir, a la concepción, la implementación y la documentación de vitalXAI hasta su entrega. Quedan fuera las actividades que solo tendrían sentido durante una explotación continuada, como el mantenimiento del software, el soporte a usuarios, la actualización de los modelos o su despliegue en un centro sanitario. La planificación tiene como horizonte la finalización y defensa de la memoria. Las posibles líneas de evolución futura se describen como trabajo futuro, sin ampliar el alcance ni la estimación de recursos de este documento. De acuerdo con el calendario del apartado 4.3.2, la metodología se aplicó principalmente durante el desarrollo activo, mientras que las fases inicial y final se apoyaron más en las reuniones de seguimiento y en el tablero Kanban que en una cadencia formal de sprints.

Capítulo 5. Cronograma y Plan de Trabajo

5.1. Marco general y objetivos de la planificación

Este capítulo recoge el programa de trabajo del proyecto: el marco temporal, el desglose de las tareas en sprints y la asignación de recursos y responsabilidades. El punto de partida es vitalXAI, la plataforma web MLOps que integra inteligencia artificial explicable para el diagnóstico asistido de neumonía mediante radiografías de tórax, definida en el capítulo de metas y propósitos. La planificación cubre los once objetivos específicos del proyecto, organizados en los bloques científico-metodológico y de ingeniería web, desde la revisión del estado del arte hasta la construcción de la interfaz clínica, el laboratorio MLOps, la seguridad y la internacionalización.

El programa de trabajo cubre el ciclo completo de concepción, implementación y documentación del sistema, y tiene un horizonte claro: la entrega y defensa del Trabajo Fin de Grado el 2 de septiembre de 2026. La ejecución se ha organizado siguiendo el marco ágil adaptado de Scrum descrito en el capítulo de gestión: las iteraciones de trabajo se aplicaron de forma plena durante la fase de implementación, mientras que las fases de planificación y de documentación se apoyaron en reuniones periódicas de seguimiento y en el tablero Kanban.

De acuerdo con el calendario real del proyecto, la planificación se articula en tres fases. La fase inicial, de finales de noviembre de 2025 a finales de febrero de 2026, estableció las bases organizativas, técnicas y documentales. La fase de desarrollo, de principios de marzo a principios de junio de 2026, concentró la implementación del sistema en siete sprints. Por último, la fase de documentación, desde principios de junio hasta septiembre de 2026, se dedicó a la elaboración de la memoria, del manual de usuario y al cierre del proyecto.

El esfuerzo total planificado asciende a 474 horas, distribuidas en nueve sprints de duración desigual que reflejan la carga real de cada bloque de trabajo: la infraestructura y el laboratorio MLOps son los bloques más pesados, mientras que la validación externa es el más contenido. Las horas de las tareas representan esfuerzo estimado, mientras que las duraciones del diagrama de Gantt representan ventanas de calendario; ambas magnitudes no se convierten mediante una jornada diaria uniforme. La dedicación efectiva varía según la fase, la naturaleza de las tareas y la participación de los asesores, tal como se detalla en la asignación de recursos del apartado 5.3. Por tanto, el cronograma debe interpretarse como una planificación temporal y de esfuerzo, no como una deducción de horas diarias constantes.

5.2. Desglose de tareas por Sprint

El trabajo se estructura en nueve sprints, cada uno con un propósito definido y un resultado verificable. Sus duraciones son desiguales y responden a la carga y naturaleza de cada bloque. El Sprint 0 corresponde a la fase de planificación, los siete siguientes a la fase de desarrollo y el Sprint 8 a la documentación y el cierre, de acuerdo con el calendario descrito en el capítulo de gestión.

El camino crítico del proyecto pasa por las tareas de mayor carga técnica y menor posibilidad de ejecución simultánea: la implementación del pipeline de entrenamiento de las redes convolucionales (Sprint 4) y su continuación en el pipeline Transformer (Sprint 5), necesarias antes de ejecutar el benchmarking (Sprint 8). Estas tareas concentran el mayor riesgo de desviación cronológica del proyecto. En la siguiente figura se representa la estructura de desglose del trabajo (EDT), elaborada manualmente.

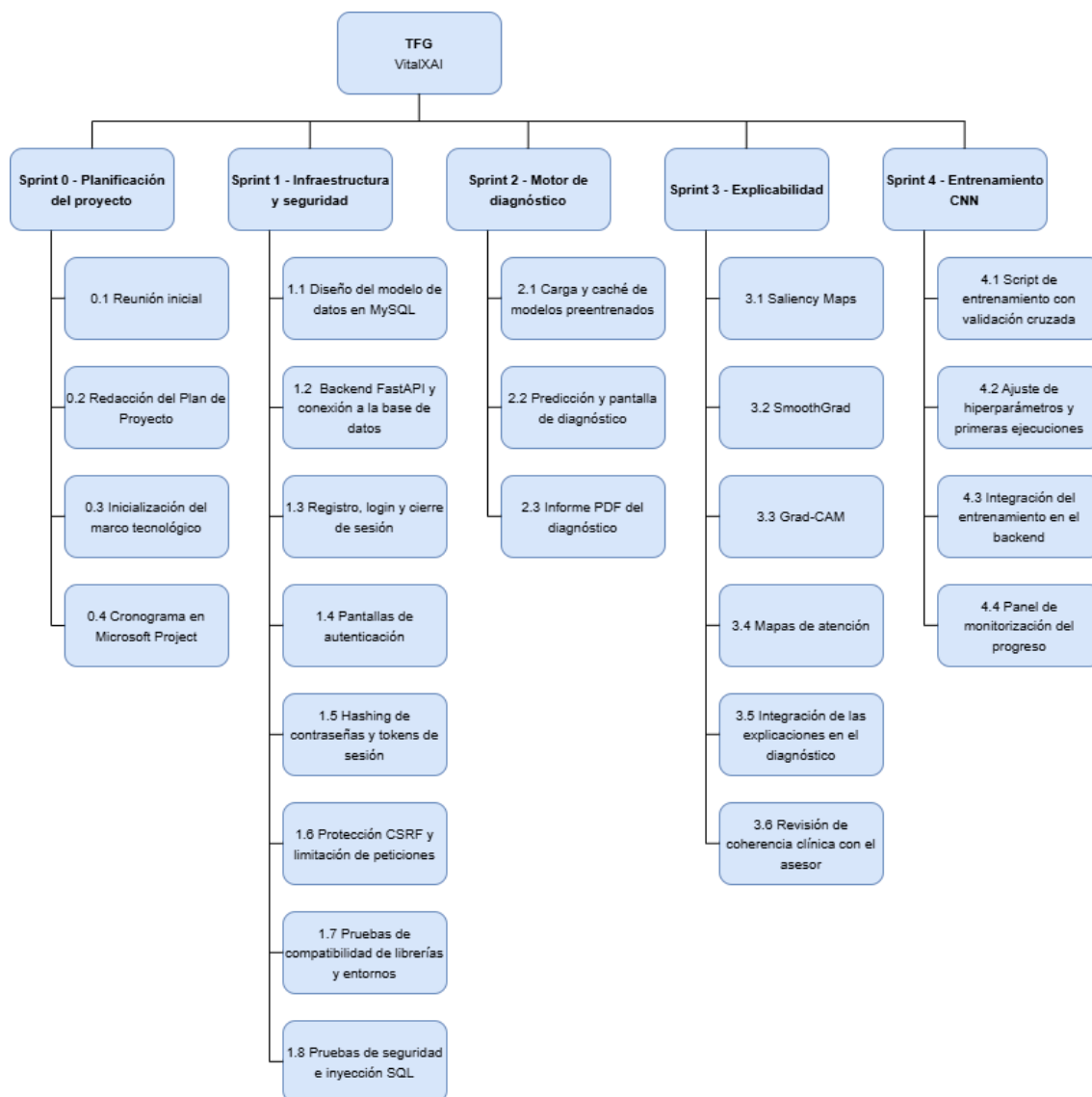


Ilustración x - Diagrama EDT I

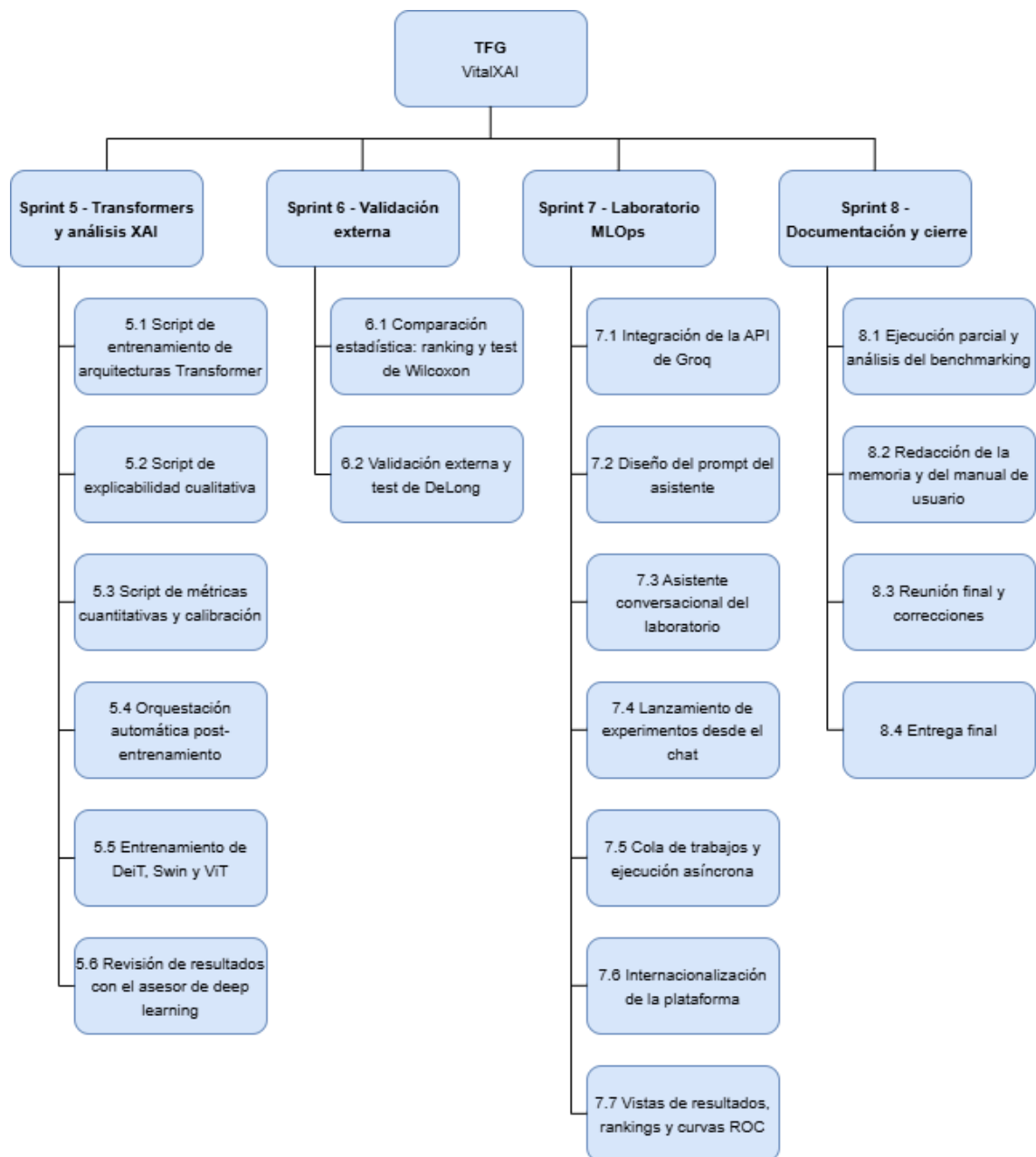


Ilustración x - Diagrama EDT II

5.2.1. Sprint 0 – Planificación del proyecto

Este sprint inicial cubre la preparación organizativa, técnica y documental del proyecto. Se desarrolla entre finales de noviembre de 2025 y finales de febrero de 2026, con 42 horas de esfuerzo repartidas en cuatro tareas. También incluye la revisión bibliográfica que fundamenta el estado del arte y las decisiones metodológicas posteriores.

Código	Nombre	Responsable(s)	Estimación (h)
0.1	Reunión inicial	Luis Carmona Berdugo, Aurelio López Fernández	3
0.2	Redacción del Plan de Proyecto y revisión bibliográfica	Luis Carmona Berdugo	25
0.3	Inicialización del marco tecnológico	Luis Carmona Berdugo	8
0.4	Cronograma en Microsoft Project	Luis Carmona Berdugo	6

Tabla x - Recursos y estimaciones Sprint 0

Tarea 0.1 – Reunión inicial: encuentro con el tutor para fijar objetivos, alcance, tecnologías (FastAPI, TensorFlow, MySQL y Groq) y metodología.

Tarea 0.2 – Redacción del Plan de Proyecto y revisión bibliográfica: el alumno redacta el documento marco que integra los objetivos, la organización, la metodología de gestión, el plan de tareas y la evaluación de riesgos. Esta tarea establece la base documental y metodológica de los capítulos iniciales y orienta la ejecución de los sprints posteriores.

Tarea 0.3 – Inicialización del marco tecnológico: se fijan las versiones de las librerías, se preparan los entornos virtuales de Anaconda y se crea el repositorio. Esta tarea establece el entorno de trabajo inicial y reduce el riesgo de incompatibilidades entre dependencias (riesgo R02).

Tarea 0.4 – Cronograma en Microsoft Project: se construye el diagrama de Gantt con las tareas, sus dependencias, los recursos y el camino crítico, que servirá de referencia para el seguimiento.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
■ 0. Planificación del Proyecto	70 días	lun 24/11/25	vie 27/02/26		42 horas	
0.1 Reunión Inicial	1 día	lun 24/11/25	lun 24/11/25	Luis Carmona Berdugo[50%]; Aurelio López Fernández[50%]	3 horas	
0.2 Redacción del Plan de Proyecto	49 días	mar 25/11/25	vie 30/01/26	Luis Carmona Berdugo	25 horas	3
0.3 Inicialización del marco tecnológico	10 días	lun 02/02/26	vie 13/02/26	Luis Carmona Berdugo	8 horas	4
0.4 Cronograma en Microsoft Project	10 días	lun 16/02/26	vie 27/02/26	Luis Carmona Berdugo	6 horas	5

Ilustración x - Sprint 0 en Project

5.2.2. Sprint 1 – Infraestructura y seguridad

El primer sprint de desarrollo reúne la infraestructura y la seguridad inicial del sistema, con 70 horas de esfuerzo. En él se construyen el backend sobre FastAPI y la base de datos MySQL, se implementa la autenticación y se incorporan controles de seguridad, pruebas de compatibilidad y pruebas frente a inyección SQL. Con ocho tareas, es el sprint con más entregables del cronograma. Se desarrolla durante las dos primeras semanas de marzo de 2026 y prepara la plataforma para integrar el motor de inferencia.

Código	Nombre	Responsable(s)	Estimación (h)
1.1	Diseño del modelo de datos en MySQL	Luis Carmona Berdugo, Vicente de Vides Rodríguez	10
1.2	Backend FastAPI y conexión a la base de datos	Luis Carmona Berdugo	14
1.3	Registro, login y cierre de sesión	Luis Carmona Berdugo	12
1.4	Pantallas de autenticación	Luis Carmona Berdugo	8
1.5	Hashing de contraseñas y tokens de sesión	Luis Carmona Berdugo	6
1.6	Protección CSRF y limitación de peticiones	Luis Carmona Berdugo	6
1.7	Pruebas de compatibilidad de librerías y entornos	Luis Carmona Berdugo	6
1.8	Pruebas de seguridad e inyección SQL	Luis Carmona Berdugo	8

Tabla x - Recursos y estimaciones Sprint 1

Tarea 1.1 – Diseño del modelo de datos en MySQL: se diseña el esquema relacional del sistema (usuarios, consultas de diagnóstico, sesiones de entrenamiento, cola de trabajos y tokens de refresco). El consultor de persistencia y bases de datos aporta su criterio sobre el modelado y la optimización del esquema.

Tarea 1.2 – Backend FastAPI y conexión a la base de datos: se monta el backend con FastAPI y se configura la conexión a MySQL mediante un pool de conexiones. El renderizado de las páginas con Jinja2 y el servidor Uvicorn quedan operativos al cierre de esta tarea, sobre cuya base se construyen los endpoints posteriores del sistema.

Tarea 1.3 – Registro, login y cierre de sesión: se implementan los flujos de creación de cuenta, inicio de sesión y cierre de sesión. Las contraseñas se cifran con bcrypt y la sesión se gestiona mediante tokens JWT almacenados en cookies seguras, con tokens de refresco y su rotación en cada uso.

Tarea 1.4 – Pantallas de autenticación: se construyen las interfaces de login y registro con el sistema de estilos de Tailwind. El flujo completo de acceso se verifica de forma integral, desde el formulario hasta la entrada al panel.

Tarea 1.5 – Hashing de contraseñas y tokens de sesión: se integra la capa de cifrado de contraseñas y la emisión y validación de los tokens de sesión y de refresco.

Tarea 1.6 – Protección CSRF y limitación de peticiones: se incorpora el middleware de doble cookie frente a la falsificación de peticiones y la limitación de intentos para frenar los ataques por fuerza bruta.

Tarea 1.7 – Pruebas de compatibilidad de librerías y entornos: se ejecutan pruebas sobre los entornos virtuales de Anaconda para garantizar que las versiones de TensorFlow y Keras conviven sin conflictos antes de construir el motor de inferencia (riesgo R02).

Tarea 1.8 – Pruebas de seguridad e inyección SQL: se validan los mecanismos de autenticación y autorización y se prueban los endpoints frente a inyección SQL y exposición de datos sensibles.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
▲ 1. Infraestructura y Seguridad	12 días	lun 02/03/26	mar 17/03/26		70 horas	
1.1 Diseño del modelo de datos en MySQL	2 días	lun 02/03/26	mar 03/03/26	Luis Carmona Berdugo[85%]; Vicente de Vides Rodríguez[15%]	10 horas	6
1.2 Backend FastAPI y conexión a la BD	2 días	mié 04/03/26	jue 05/03/26	Luis Carmona Berdugo	14 horas	8
1.3 Registro, login y cierre de sesión	2 días	vie 06/03/26	lun 09/03/26	Luis Carmona Berdugo	12 horas	9
1.4 Pantallas de autenticación	2 días	lun 09/03/26	mar 10/03/26	Luis Carmona Berdugo	8 horas	
1.5 Hashing de contraseñas y tokens de sesión	3 días	mar 10/03/26	jue 12/03/26	Luis Carmona Berdugo	6 horas	
1.6 Protección CSRF y limitación de peticiones	2 días	mié 11/03/26	vie 13/03/26	Luis Carmona Berdugo	6 horas	
1.7 Pruebas de compatibilidad de librerías y entorno	2 días	jue 12/03/26	lun 16/03/26	Luis Carmona Berdugo	6 horas	11
1.8 Pruebas de seguridad e inyección SQL	2 días	lun 16/03/26	mar 17/03/26	Luis Carmona Berdugo	8 horas	

Ilustración x - Sprint 1 en Project

5.2.3. Sprint 2 – Motor de diagnóstico

Este sprint construye el núcleo del diagnóstico clínico: la carga de los modelos preentrenados, el endpoint de predicción y la interfaz que permite al facultativo subir una radiografía y ver el resultado. Con 41 horas en tres tareas, cierra la primera versión funcional del flujo de diagnóstico en la segunda mitad de marzo.

Código	Nombre	Responsable(s)	Estimación (h)
2.1	Carga y caché de modelos preentrenados	Luis Carmona Berdugo	15
2.2	Predicción y pantalla de diagnóstico	Luis Carmona Berdugo	18
2.3	Informe PDF del diagnóstico	Luis Carmona Berdugo	8

Tabla x - Recursos y estimaciones Sprint 2

Tarea 2.1 – Carga y caché de modelos preentrenados: se desarrolla el motor de carga de los modelos CNN y Transformer desde sus pesos. Un sistema de caché en memoria evita recargar el modelo en cada petición después de su primera carga, reduciendo el tiempo necesario para las consultas posteriores.

Tarea 2.2 – Predicción y pantalla de diagnóstico: se implementa el endpoint de predicción y la pantalla de diagnóstico en la que el facultativo sube la radiografía, selecciona la arquitectura y obtiene el diagnóstico con su nivel de confianza. El procesamiento se encola de forma asíncrona para no bloquear la interfaz durante la inferencia. Es la tarea central de este sprint: sobre ella se apoyan los mapas de explicabilidad del sprint siguiente. La pantalla presenta el resultado de forma clara y sin tecnicismos, pensando en un usuario sin formación técnica.

Tarea 2.3 – Informe PDF del diagnóstico: se implementa la generación del informe PDF que recoge la radiografía, el diagnóstico, el nivel de confianza y el modelo empleado, listo para su descarga y archivo.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
2. Motor de diagnóstico	10 días	mié 18/03/21	mar 31/03/26		41 horas	
2.1 Carga y caché de modelos preentrenados	3 días	mié 18/03/21	vie 20/03/26	Luis Carmona Berdugo	15 horas	15
2.2 Predicción y pantalla de diagnóstico	3 días	lun 23/03/26	mié 25/03/26	Luis Carmona Berdugo	18 horas	17
2.3 Informe PDF del diagnóstico	4 días	jue 26/03/26	mar 31/03/26	Luis Carmona Berdugo	8 horas	18

Ilustración x - Sprint 2 en Project

5.2.4. Sprint 3 – Explicabilidad

Este sprint incorpora técnicas de inteligencia artificial explicable al sistema. Se implementan Saliency Maps, SmoothGrad y Grad-CAM para las CNN, además de los mapas de atención para los Transformers, y se integran en el flujo de diagnóstico. Con 48 horas repartidas en seis tareas, el sprint se desarrolla durante las dos primeras semanas de abril. La revisión del asesor de imagen médica aporta una valoración de contexto sobre las explicaciones generadas.

Código	Nombre	Responsable(s)	Estimación (h)
3.1	Saliency Maps	Luis Carmona Berdugo, Iván Segura-Carmona	10
3.2	SmoothGrad	Luis Carmona Berdugo	8
3.3	Grad-CAM	Luis Carmona Berdugo, Iván Segura-Carmona	12
3.4	Mapas de atención	Luis Carmona Berdugo	8
3.5	Integración de las explicaciones en el diagnóstico	Luis Carmona Berdugo	6
3.6	Revisión de coherencia clínica con el asesor	Luis Carmona Berdugo, Marc Ríos Cadenas	4

Tabla x - Recursos y estimaciones Sprint 3

Tarea 3.1 – Saliency Maps: se implementa el mapa de prominencia que calcula el gradiente de la clase predicha respecto a cada píxel, resaltando las regiones más influyentes en la decisión del modelo. La revisión del asesor de deep learning y XAI valida la corrección del cálculo.

Tarea 3.2 – SmoothGrad: se implementa la variante que promedia los mapas de saliencia sobre versiones con ruido gaussiano. El resultado es un mapa más suave y estable, en el que los patrones consistentes del modelo destacan sobre el ruido.

Tarea 3.3 – Grad-CAM: se implementan los mapas de activación de clase a partir de la última capa convolucional, que superponen a la radiografía un mapa de calor de resolución reducida. La representación puede facilitar la inspección visual por parte del profesional clínico. Se valida su correcta implementación con el asesor de deep learning y XAI.

Tarea 3.4 – Mapas de atención: se implementa la extracción de los pesos de atención de la última capa de los Transformers, promediando las cabezas de atención para obtener el mapa de relevancia de cada parche.

Tarea 3.5 – Integración de las explicaciones en el diagnóstico: se integra la generación de los mapas en el flujo de predicción, de modo que cada consulta genera automáticamente sus explicaciones.

Tarea 3.6 – Revisión de coherencia clínica con el asesor: el asesor de imagen médica comprueba que los mapas apuntan a las regiones pulmonares relevantes y no a artefactos.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
4 3. Explicabilidad	8 días	mié 01/04/21	vie 10/04/26		48 horas	
3.1 Saliency Maps	2 días	mié 01/04/21	jue 02/04/26	Luis Carmona Berdugo[90%];Iván Segura Carmona[10%]	10 horas	19
3.2 SmoothGrad	1 día	vie 03/04/26	vie 03/04/26	Luis Carmona Berdugo	8 horas	21
3.3 Grad-CAM	2 días	lun 06/04/26	mar 07/04/26	Iván Segura Carmona[8%];Luis Carmona Berdugo[92%]	12 horas	22
3.4 Mapas de atención	1 día	mié 08/04/21	mié 08/04/26	Luis Carmona Berdugo	8 horas	23
3.5 Integración de las explicaciones en el diagnóstico	1 día	jue 09/04/26	jue 09/04/26	Luis Carmona Berdugo	6 horas	24
3.6 Revisión de coherencia clínica con asesor	1 día	vie 10/04/26	vie 10/04/26	Luis Carmona Berdugo[75%];Marc Ríos Cadena[25%]	4 horas	25

Ilustración x - Sprint 3 en Project

5.2.5. Sprint 4 – Entrenamiento CNN

Este sprint implementa el primer pipeline de entrenamiento automatizado. Se desarrolla el script que entrena las arquitecturas convolucionales con validación cruzada de cinco pliegues y balanceo de clases, se integra con el backend para poder lanzar experimentos desde la plataforma y se construye el panel de monitorización del progreso. Es, junto con el sprint siguiente, el corazón del camino crítico del proyecto: sin el pipeline de entrenamiento no existe benchmarking posible. Con 52 horas en cuatro tareas, ocupa la segunda quincena de abril. El asesor de deep learning y XAI revisa la corrección del pipeline y de sus métricas.

Código	Nombre	Responsable(s)	Estimación (h)
4.1	Script de entrenamiento con validación cruzada	Luis Carmona Berdugo, Iván Segura-Carmona	20
4.2	Ajuste de hiperparámetros y primeras ejecuciones	Luis Carmona Berdugo	14
4.3	Integración del entrenamiento en el backend	Luis Carmona Berdugo	8
4.4	Panel de monitorización del progreso	Luis Carmona Berdugo	10

Tabla x - Recursos y estimaciones Sprint 4

Tarea 4.1 – Script de entrenamiento con validación cruzada: se implementa el script que entrena dinámicamente cualquier arquitectura de `tf.keras.applications` con cinco pliegues estratificados y submuestreo aleatorio de la clase mayoritaria. La base preentrenada en ImageNet se mantiene congelada y se sustituye la cabeza de clasificación por una propia, con callbacks de guardado del mejor modelo, parada temprana y reducción de la tasa de aprendizaje. La estimación incluye la resolución de las incompatibilidades entre Keras y `tf_keras`, que consumió más tiempo del previsto. Es la tarea más costosa del sprint y la base de todo el benchmarking posterior.

Tarea 4.2 – Ajuste de hiperparámetros y primeras ejecuciones: se lanzan las primeras ejecuciones sobre arquitecturas ligeras (MobileNetV2, EfficientNetB0) para calibrar el hardware y ajustar el número de épocas, el tamaño de lote y la tasa de aprendizaje. Esta fase acumula experiencia sobre los tiempos reales de entrenamiento y permite estimar con realismo la duración de los sprints siguientes. Es un paso previo imprescindible antes de liberar a las arquitecturas más pesadas.

Tarea 4.3 – Integración del entrenamiento en el backend: se conecta el script de entrenamiento con la API, de modo que un experimento pueda lanzarse desde la plataforma y ejecutarse como un proceso independiente sin bloquear la interfaz.

Tarea 4.4 – Panel de monitorización del progreso: se construye la vista que muestra el estado del entrenamiento en tiempo real, con el progreso por modelo y la consulta del estado de la cola de trabajos. El facultativo investigador puede así seguir la evolución de sus experimentos sin salir de la plataforma.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
4. Entrenamiento CNN	10 días	lun 13/04/26	vie 24/04/26		52 horas	
4.1 Script de entrenamiento con validación cruzada	4 días	lun 13/04/26	jue 16/04/26	Iván Segura Carmona[3%]; Luis Carmona Berdugo[97%]	20 horas	26
4.2 Ajuste de hiperparámetros y primeras ejecuciones	2 días	vie 17/04/26	lun 20/04/26	Luis Carmona Berdugo	14 horas	28
4.3 Integración del entrenamiento en el backend	1 día	mar 21/04/26	mar 21/04/26	Luis Carmona Berdugo	8 horas	29
4.4 Panel de monitorización del progreso	3 días	mié 22/04/26	vie 24/04/26	Luis Carmona Berdugo	10 horas	30

Ilustración x - Sprint 4 en Project

5.2.6. Sprint 5 – Transformers y análisis XAI

Este sprint amplía el marco de entrenamiento a las arquitecturas Transformer y desarrolla los módulos de análisis XAI que se ejecutan automáticamente tras cada entrenamiento. Es el segundo bloque del camino crítico del proyecto: el entrenamiento de los Vision Transformers y la evaluación de su explicabilidad condicionan directamente el benchmarking previsto. Con 62 horas en seis tareas, el sprint se extiende desde finales de abril hasta el 19 de mayo, con el entrenamiento de DeiT, Swin-Base y ViT-384 prolongándose hasta mediados de mayo y la revisión final del asesor realizándose el 18 de mayo. Incluye la implementación de los scripts de análisis cualitativo y cuantitativo, la orquestación automática que garantiza que ningún modelo quede sin su evaluación de explicabilidad y la revisión final del asesor de deep learning y XAI sobre los resultados.

Código	Nombre	Responsable(s)	Estimación (h)
5.1	Script de entrenamiento de arquitecturas Transformer	Luis Carmona Berdugo, Iván Segura-Carmona	16
5.2	Script de explicabilidad cualitativa	Luis Carmona Berdugo	10
5.3	Script de métricas cuantitativas y calibración	Luis Carmona Berdugo, Iván Segura-Carmona	12
5.4	Orquestación automática post-entrenamiento	Luis Carmona Berdugo	6
5.5	Entrenamiento de DeiT, Swin y ViT	Luis Carmona Berdugo	14
5.6	Revisión de resultados con el asesor de deep learning	Luis Carmona Berdugo, Iván Segura-Carmona	4

Tabla x - Recursos y estimaciones Sprint 5

Tarea 5.1 – Script de entrenamiento de arquitecturas Transformer: se implementa el script que entrena DeiT, Swin-Base y ViT-384 con HuggingFace Transformers y validación cruzada de cinco pliegues, reutilizando el esquema de balanceo y de guardado de pesos del pipeline convolucional. La integración con la librería de Transformers obliga a adaptar el manejo de pesos y la tasa de aprendizaje, con ajustes específicos para lograr la convergencia. Es la tarea más exigente del sprint desde el punto de vista técnico. Su correcta ejecución era condición necesaria para el benchmarking previsto, aunque estas arquitecturas no forman parte de los resultados parciales conservados.

Tarea 5.2 – Script de explicabilidad cualitativa: se implementa el script que genera los mapas de explicabilidad visuales sobre imágenes de ejemplo, con Grad-CAM y saliencia para las CNN y saliencia para los Transformers. El resultado es la galería de mapas de cada modelo, que permite la inspección visual de la coherencia de las decisiones.

Tarea 5.3 – Script de métricas cuantitativas y calibración: se implementa el cálculo de las métricas de fidelidad de las explicaciones (Deletion AUC, Insertion AUC, Sparsity, Entropy y Stability SSIM) y de la calibración de las predicciones mediante el Expected Calibration Error y el Brier Score. La revisión del asesor de deep learning y XAI valida el cálculo de estas métricas.

Tarea 5.4 – Orquestación automática post-entrenamiento: se automatiza la ejecución de los análisis XAI al finalizar cada entrenamiento, garantizando que ningún modelo quede sin su evaluación de explicabilidad.

Tarea 5.5 – Entrenamiento de DeiT, Swin y ViT: se planifica el entrenamiento de las tres arquitecturas Transformer sobre el dataset. La duración de esta tarea refleja el coste computacional previsto, superior al de las CNN. Se aplican los ajustes de tasa de aprendizaje necesarios durante las ejecuciones disponibles.

Tarea 5.6 – Revisión de resultados con el asesor de deep learning: el asesor valida la interpretación de los resultados del entrenamiento y de la explicabilidad de los Transformers.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
➤ 5. Transformers y análisis XAI	17 días	lun 27/04/26	mar 19/05/26		62 horas	
5.1 Script de entrenamiento de arquitecturas Transformer	3 días	lun 27/04/26	mié 29/04/26	Iván Segura Carmona[3%];Luis Carmona Berdugo[97%]	16 horas	31
5.2 Script de explicabilidad cualitativa	3 días	jue 30/04/26	lun 04/05/26	Luis Carmona Berdugo	10 horas	33
5.3 Script de métricas cuantitativas y calibración	3 días	lun 04/05/26	mié 06/05/26	Iván Segura Carmona[8%];Luis Carmona Berdugo[92%]	12 horas	
5.4 Orquestación automática post-entreno	1 día	jue 07/05/26	vie 08/05/26	Luis Carmona Berdugo	6 horas	35
5.5 Entrenamiento de Deit, Swin y ViT	5 días	lun 11/05/26	vie 15/05/26	Luis Carmona Berdugo	14 horas	36
5.6 Revisión de resultados con el asesor de deep learning	1 día	lun 18/05/26	lun 18/05/26	Iván Segura Carmona[25%];Luis Carmona Berdugo[75%]	4 horas	37

Ilustración x - Sprint 5 en Project

5.2.7. Sprint 6 – Validación externa

Este es el sprint más breve del proyecto, con 26 horas en dos tareas y una duración de cuatro días. Implementa los mecanismos de validación externa y de análisis estadístico previstos para comparar los modelos. Se desarrolla en la segunda mitad de mayo.

Código	Nombre	Responsable(s)	Estimación (h)
6.1	Comparación estadística: ranking y test de Wilcoxon	Luis Carmona Berdugo, Iván Segura-Carmona	12
6.2	Validación externa y test de DeLong	Luis Carmona Berdugo	14

Tabla x - Recursos y estimaciones Sprint 6

Tarea 6.1 – Comparación estadística: ranking y test de Wilcoxon: ranking y análisis exploratorio: se implementa el script que genera el ranking global de los modelos por su AUC medio y calcula una matriz exploratoria de p-valores por pares. Como solo se dispone de cinco folds dependientes, el test de Wilcoxon (Wilcoxon, 1945) no se utilizará para afirmar significación confirmatoria. Cualquier contraste por pares deberá incorporar una corrección por comparaciones múltiples y sus limitaciones deberán quedar documentadas.

Tarea 6.2 – Validación externa y test de DeLong: se implementa la evaluación de los modelos congelados sobre la cohorte independiente de pacientes adultos y el test de DeLong (DeLong, DeLong, & Clarke-Pearson, 1988) para comparar las curvas ROC. La tarea aporta evidencia sobre el comportamiento de los modelos en la cohorte externa y genera las matrices de significación correspondientes.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
6. Validación externa	4 días	mar 19/05/21	vie 22/05/26		26 horas	
6.1 Comparación estadística: ranking y test de Wilcoxon	2 días	mar 19/05/21	mié 20/05/26	Iván Segura Carmona[8%];Luis Carmona Berdugo[92%]	12 horas	38
6.2 Validación externa y test de DeLong	2 días	jue 21/05/26	vie 22/05/26	Luis Carmona Berdugo	14 horas	40

Ilustración x - Sprint 6 en Project

5.2.8. Sprint 7 – Laboratorio MLOps

El laboratorio MLOps es, junto con la infraestructura, el bloque de mayor carga del proyecto, con 66 horas en siete tareas. Integra el asistente conversacional basado en Groq y en el modelo openai/gpt-oss-120b, que permite configurar y lanzar experimentos en lenguaje natural. El sprint incorpora también la cola de trabajos para diagnósticos, entrenamientos y validaciones externas, la internacionalización de la plataforma en cuatro idiomas y las vistas de resultados con el ranking, las matrices de significación y las curvas ROC. Se desarrolla en la última semana de mayo y la primera de junio.

Código	Nombre	Responsable(s)	Estimación (h)
7.1	Integración de la API de Groq	Luis Carmona Berdugo	8
7.2	Diseño del prompt del asistente	Luis Carmona Berdugo	12
7.3	Asistente conversacional del laboratorio	Luis Carmona Berdugo	14
7.4	Lanzamiento de experimentos desde el chat	Luis Carmona Berdugo	8
7.5	Cola de trabajos y ejecución asíncrona	Luis Carmona Berdugo	8
7.6	Internacionalización de la plataforma	Luis Carmona Berdugo	8
7.7	Vistas de resultados, rankings y curvas ROC	Luis Carmona Berdugo	8

Tabla x - Recursos y estimaciones Sprint 7

Tarea 7.1 – Integración de la API de Groq: se configura el cliente de Groq y la conexión con el modelo openai/gpt-oss-120b que alimenta al asistente conversacional del laboratorio.

Tarea 7.2 – Diseño del prompt del asistente: se diseña el prompt de sistema que define al asistente como experto en MLOps médico. Incluye las reglas de extracción de los cinco parámetros del experimento (ruta del dataset, arquitecturas, épocas, lote y tasa de aprendizaje) y el formato JSON de salida. Es una tarea de diseño más que de código, y por eso su estimación es moderada.

Tarea 7.3 – Asistente conversacional del laboratorio: se implementa el endpoint de chat y la sala de conversación, con el historial de mensajes por sesión. El asistente interpreta la configuración devuelta por el modelo y la traduce al panel del experimento, para que el usuario pueda revisarla y confirmarla. Es la tarea más costosa del sprint.

Tarea 7.4 – Lanzamiento de experimentos desde el chat: se conecta la configuración detectada por el asistente con el lanzamiento del pipeline de entrenamiento, de modo que el experimento se encola y arranca sin escribir código.

Tarea 7.5 – Cola de trabajos y ejecución asíncrona: se implementa la cola de trabajos que gestiona los diagnósticos, los entrenamientos y las validaciones externas, con monitorización de estado y cancelación de las tareas pendientes.

Tarea 7.6 – Internacionalización de la plataforma: se incorpora el soporte multilingüe de la interfaz, los informes y el asistente en los cuatro idiomas de la plataforma, mediante atributos de traducción y diccionarios en JavaScript.

Tarea 7.7 – Vistas de resultados, rankings y curvas ROC: se construyen las vistas del laboratorio que muestran el ranking de modelos, las matrices de significación y las curvas ROC de la validación externa.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
🔪 7. Laboratorio MLOps	10 días	lun 25/05/26	vie 05/06/26		66 horas	
7.1 Integración de la API de Groq	1 día	lun 25/05/26	lun 25/05/26	Luis Carmona Berdugo	8 horas	41
7.2 Diseño del prompt del asistente	2 días	mar 26/05/26	mié 27/05/26	Luis Carmona Berdugo	12 horas	43
7.3 Asistente conversacional del laboratorio	2 días	jue 28/05/26	vie 29/05/26	Luis Carmona Berdugo	14 horas	44
7.4 Lanzamiento de experimentos desde el chat	1 día	lun 01/06/26	lun 01/06/26	Luis Carmona Berdugo	8 horas	45
7.5 Cola de trabajos y ejecución asíncrona	1 día	mar 02/06/26	mar 02/06/26	Luis Carmona Berdugo	8 horas	46
7.6 Internacionalización de la plataforma	1 día	mié 03/06/26	mié 03/06/26	Luis Carmona Berdugo	8 horas	47
7.7 Vistas de resultados, rankings y curvas ROC	1 día	jue 04/06/26	jue 04/06/26	Luis Carmona Berdugo	8 horas	48

Ilustración x - Sprint 7 en Project

5.2.9. Sprint 8 – Documentación y cierre

El sprint final combina las ejecuciones parciales del benchmarking con la redacción de la memoria y del manual de usuario, las correcciones finales y la entrega. Con 67 horas en cuatro tareas, se desarrolla desde el 5 de junio hasta el 2 de septiembre de 2026, con una dedicación repartida entre la preparación y el análisis de las ejecuciones, la documentación y el cierre. La redacción de la memoria es la tarea más extensa del sprint, mientras que las correcciones finales y la entrega se han separado en tareas específicas. El sprint culmina con la entrega del 2 de septiembre de 2026.

Código	Nombre	Responsable(s)	Estimación (h)
8.1	Ejecución parcial y análisis del benchmarking	Luis Carmona Berdugo, Iván Segura-Carmona	18
8.2	Redacción de la memoria y del manual de usuario	Luis Carmona Berdugo	30
8.3	Reunión final y correcciones	Luis Carmona Berdugo, Aurelio López Fernández	16
8.4	Entrega final	Luis Carmona Berdugo	3

Tabla x - Recursos y estimaciones Sprint 8

Tarea 8.1 – Ejecución parcial y análisis del benchmarking: se ejecutan y analizan las ocho arquitecturas CNN de las que se conservan resultados en el directorio del proyecto —ResNet50, DenseNet121, EfficientNetB0, EfficientNetV2B0, MobileNetV2, InceptionV3, Xception y ConvNeXtTiny—. Cada una cuenta con cinco ejecuciones de validación cruzada, lo que supone cuarenta entrenamientos con sus métricas y predicciones asociadas. No se completó el banco de pruebas previsto de diecinueve arquitecturas: no se incorporan resultados de los tres Transformers ni de las ocho CNN restantes. Las dieciocho horas corresponden al trabajo del alumno y de los asesores para preparar los lanzamientos, revisar su configuración y analizar los resultados; el tiempo de cómputo que transcurrió sin supervisión directa no se contabiliza como dedicación humana. El asesor de deep learning y XAI y el asesor de imagen médica revisan la coherencia de los resultados disponibles. La reducción del alcance se documenta como consecuencia de la carga computacional identificada en el riesgo R04.

Tarea 8.2 – Redacción de la memoria y del manual de usuario: se redacta la memoria del Trabajo Fin de Grado, integrando el plan de proyecto, el análisis, el diseño, la implementación, las pruebas y las conclusiones, junto con el manual de usuario orientado a facultativos sin formación técnica. La redacción se realiza de forma incremental durante la fase de documentación y se incorporan las revisiones del tutor. Cada capítulo se elabora y revisa por separado antes de integrarlo en el documento final. El manual de usuario emplea un lenguaje no técnico y recoge capturas de los flujos principales.

Tarea 8.3 – Reunión final y cierre: reunión con el tutor y periodo de corrección programados del 28 de agosto al 1 de septiembre. Esta tarea permite revisar el conjunto del trabajo, aplicar las correcciones detectadas, preparar la defensa y dejar la versión final lista antes de la entrega.

Tarea 8.4 – Entrega final: depósito de la memoria y de los entregables del proyecto el 2 de septiembre de 2026. Esta tarea constituye el cierre administrativo posterior a la revisión y corrección final.

8. Documentación y cierre	63 días?	lun 08/06/26	mié 02/09/26		52 horas
8.1 Ejecución parcial y análisis del benchmarking	10 días	lun 08/06/26	vie 19/06/26	Iván Segura Carmona[5%];Luis Carmona Berdugo[90%];Marc Ríos Cadena[5%]	18 horas
8.2 Redacción de la memoria y del manual de usuario	50 días	lun 22/06/26	vie 28/08/26	Luis Carmona Berdugo	30 horas
8.3 Reunión final y cierre	3 días	lun 31/08/26	mié 02/09/26	Aurelio López Fernández[50%];Luis Carmona Berdugo[50%]	4 horas

Ilustración x - Sprint 8 en Project

5.3. Recursos y costes del proyecto

Este apartado detalla la asignación de recursos del proyecto. Se distinguen los recursos de trabajo, correspondientes a las personas que participan en las tareas, y los recursos materiales, que incluyen la infraestructura y el equipamiento tecnológico.

Microsoft Project centraliza la administración del proyecto. Los recursos ocasionales, como el tutor y los asesores, reciben un horario fijo; esto impide que el software reasigne automáticamente su esfuerzo ante variaciones en la duración de las tareas. Todas las asignaciones se han auditado manualmente para que la carga de los recursos laborables sea coherente con la dedicación real de cada fase del proyecto.

Recursos de trabajo:

- **Luis Carmona Berdugo:** asume la carga técnica integral del proyecto. Acumula 455,14 horas distribuidas a lo largo de todos los sprints. Con una tarifa estándar de 20 €/h, su esfuerzo se cuantifica en 9.102,80 €.
- **Aurelio López Fernández (tutor):** interviene estratégicamente en los hitos de apertura (tarea 0.1) y de clausura y correcciones (tarea 8.3). Dedicar 9,5 horas de supervisión a 50 €/h, sumando un total de 475,00 €.
- **Vicente de Vides Rodríguez (consultor de persistencia y bases de datos):** aporta su experiencia estructural en el diseño y la optimización de la persistencia de datos. Participa en la tarea 1.1 con 1,5 horas a 50 €/h, con un total de 75,00 €.
- **Iván Segura Carmona (asesor Deep Learning y XAI):** participa en la revisión de las implementaciones de explicabilidad, los pipelines CNN y Transformer, la comparación estadística y el benchmarking. Acumula 5,96 horas a 50 €/h, con un total de 298,00 €.
- **Marc Ríos Cadenas (asesor Imagen Médica):** participa en la revisión de la coherencia de los mapas de explicabilidad y de los resultados disponibles del benchmarking. Dedicar 1,9 horas a 50 €/h, sumando 95,00 €.

El coste total de los recursos de trabajo asciende, por tanto, a aproximadamente 10.045,80 €.

Nombre del recurso ▼	Trabajo ▼	Costo ▼	Comienzo ▼	Fin ▼
▷ Luis Carmona Berdugo	455,14 horas	9.102,80 €	lun 24/11/25	mié 02/09/26
▷ Aurelio López Fernández	9,5 horas	475,00 €	lun 24/11/25	mar 01/09/26
▷ Vicente de Vides Rodríguez	1,5 horas	75,00 €	lun 02/03/26	mar 03/03/26
▷ Iván Segura Carmona	5,96 horas	298,00 €	mié 01/04/26	jue 18/06/26
▷ Marc Ríos Cadena	1,9 horas	95,00 €	vie 10/04/26	jue 18/06/26

Ilustración x - Vista “Uso de Recursos” en Project

Recursos Materiales:

- **Equipo de desarrollo:** se utiliza un equipo local con GPU NVIDIA compatible con CUDA, disponible para el desarrollo. El plan no especifica el modelo de GPU ni su memoria VRAM, por lo que la viabilidad de las arquitecturas con mayor resolución o capacidad no puede justificarse únicamente desde este apartado. Al tratarse de infraestructura ya disponible, no se imputa una amortización adicional al TFG.
- **Licencias de software:** coste nulo. La plataforma descansa enteramente sobre tecnologías de código abierto (Python, TensorFlow, FastAPI, MySQL), eliminando cualquier gasto de licenciamiento.
- **Servicios cloud:** no se presupuesta ningún servicio cloud en el plan base. La nube aparece únicamente como contingencia ante un fallo del equipo local, pero no existe una reserva económica asignada; si fuera necesario activarla, habría que aprobar un coste adicional o reducir el alcance del benchmarking y documentar la decisión.
- **Electricidad y API de Groq:** no se dispone de una medición o factura imputable exclusivamente al proyecto, por lo que ambos conceptos quedan fuera del presupuesto cuantificado. El consumo energético se reconoce como impacto operativo y el uso de Groq queda sujeto al plan y a las cuotas vigentes del servicio.

El coste económico documentado del proyecto asciende a 10.045,80 €, correspondiente a los recursos humanos presupuestados. Esta cifra no debe interpretarse como un coste completo de explotación: excluye la amortización del equipo ya disponible, la electricidad, una eventual infraestructura cloud y cualquier consumo facturado de la API de Groq.

5.4. Reparto de responsabilidades

Este apartado recoge la distribución de responsabilidades sobre las tareas definidas en el apartado 5.2. Para cada tarea se identifica la persona responsable, los interesados que intervienen de forma puntual y la proporción asignada. El reparto porcentual recogido en las tablas del apartado 5.2 sirve de base para calcular las horas y los costes del apartado 5.3, y se relaciona con los roles definidos en la matriz RACI del capítulo de organización. El alumno desarrollador concentra la mayor parte de la carga de trabajo, mientras que los demás interesados participan de forma puntual y consultiva.

Luis Carmona Berdugo es responsable de la ejecución técnica y documental del proyecto. A lo largo de los nueve sprints acumula 455,14 horas de trabajo, que representan la mayor parte del esfuerzo planificado. Sus tareas incluyen el análisis de requisitos, el diseño de la arquitectura, la implementación del backend y la interfaz, la construcción de los pipelines de entrenamiento y los módulos XAI, el benchmarking y la documentación. En las tareas con participación de asesores, la dedicación del alumno sigue siendo mayoritaria, con porcentajes que oscilan entre el 50 % de las reuniones y el 97 % de las tareas de entrenamiento.

El carácter multidisciplinar del sistema requiere el apoyo puntual y consultivo de varios interesados. Su participación se concentra en los hitos en los que se necesita conocimiento especializado. A continuación se detalla la participación de cada uno.

Aurelio López Fernández participa como tutor académico del proyecto. Su dedicación presupuestada se concentra en la tarea 0.1, correspondiente a la reunión inicial, y en la tarea 8.3, correspondiente a la reunión final y al periodo de correcciones. En ambas comparte la dedicación al 50 % con el alumno, lo que supone 9,5 horas y un coste de 475 €. El resto del seguimiento se realiza mediante comunicaciones y revisiones puntuales que no se contabilizan como horas adicionales.

Iván Segura Carmona participa como asesor de aprendizaje profundo e inteligencia artificial explicable. Su intervención consultiva se distribuye entre los sprints técnicos: revisa las técnicas XAI en el Sprint 3, el pipeline convolucional en el Sprint 4, el entrenamiento Transformer y las métricas cuantitativas en el Sprint 5, la comparación estadística en el Sprint 6 y los resultados disponibles del benchmarking en el Sprint 8. Su dedicación total asciende a 5,96 horas, con un coste de 298 €.

Marc Ríos Cadenas interviene como asesor especialista en imagen médica. Su participación mantiene un perfil consultivo y se concentra en dos hitos que exigen criterio clínico: la revisión de la coherencia de los mapas de explicabilidad en la tarea 3.6, donde comprueba que los mapas apuntan a las regiones pulmonares relevantes y no a artefactos, y la revisión de los resultados disponibles del benchmarking en la tarea 8.1. Su dedicación, del 25% en la tarea 3.6 y del 5% en la tarea 8.1, asciende a 1,9 horas y a un coste de 95 €.

Vicente de Vides Rodríguez actúa como consultor de persistencia y bases de datos. Su intervención es puntual y estrictamente consultiva, concentrándose en la tarea 1.1, correspondiente al diseño del modelo de datos en MySQL. En ella aporta criterios sobre el modelo de persistencia y sus necesidades de almacenamiento, sin asumir la implementación, que corresponde al alumno. Su dedicación del 15 % sobre dicha tarea supone 1,5 horas y un coste de 75 €.

El resto de las partes interesadas identificadas en el capítulo de organización, como el **asesor de ingeniería del software y metodología**, participan únicamente a través de la matriz RACI, sin asumir tareas concretas ni horas asignadas en el cronograma del proyecto. En conjunto, el reparto refleja un trabajo académico unipersonal: la carga técnica recae principalmente en el alumno y los asesores intervienen de forma puntual. Sus costes se consolidan en el apartado 5.3.

Capítulo 6. Identificación y Mitigación de Riesgos

La gestión de riesgos forma parte de la planificación del proyecto. Su importancia aumenta cuando el sistema combina aprendizaje profundo, inteligencia artificial explicable y varias arquitecturas algorítmicas. En este capítulo se considera riesgo cualquier evento que pueda afectar negativamente al alcance, al calendario, a la calidad técnica o al coste del proyecto. El objetivo es identificar estos eventos, valorar su probabilidad e impacto y definir una respuesta ordenada para reducir sus consecuencias.

Para cada riesgo se definen dos respuestas. El plan de mitigación reúne las medidas preventivas que se aplican antes de que ocurra el evento, mientras que el plan de contingencia recoge las acciones que se adoptan cuando el riesgo ya se ha materializado. Ambos planes buscan limitar el impacto sobre el desarrollo del proyecto.

6.1. Escala de valoración de riesgos

Antes de presentar el catálogo de riesgos se establece la escala utilizada para valorar su probabilidad, severidad y prioridad.

La probabilidad estima la posibilidad de que el evento ocurra durante el ciclo de vida del proyecto. Se divide en tres niveles:

- **Baja:** El evento es poco probable o está limitado por medidas de control conocidas.
- **Media:** El evento puede producirse bajo determinadas condiciones del proyecto.
- **Alta:** El evento es frecuente o resulta difícil de controlar en proyectos de esta naturaleza.

La severidad cuantifica el impacto sobre el proyecto si el riesgo se materializa. Se establecen cuatro niveles:

- **Baja:** El impacto puede corregirse sin cambios relevantes en el alcance o el calendario.
- **Media:** El impacto afecta al tiempo, al coste o a la calidad y requiere medidas correctoras.
- **Alta:** El impacto compromete una parte importante del proyecto y puede exigir cambios de alcance o calendario.
- **Muy Alta:** El impacto puede impedir la finalización del proyecto o invalidar una parte esencial de sus resultados.

La prioridad se obtiene combinando la probabilidad y la severidad. La matriz siguiente muestra el resultado de esa combinación:

Probabilidad \ Severidad	Baja	Media	Alta	Muy Alta
Baja	Baja	Baja	Media	Alta
Media	Baja	Media	Alta	Alta
Alta	Media	Alta	Alta	Muy Alta

Tabla x - Matriz de Prioridades

6.2. Catálogo de riesgos identificados

A continuación, se presentan los riesgos identificados para este proyecto, clasificados por su origen, evaluados según su probabilidad y la severidad de su impacto, y priorizados mediante la matriz del apartado anterior. Se han identificado nueve riesgos relacionados con la gestión, la infraestructura y la implementación tecnológica, entre ellos la incompatibilidad de librerías, la disponibilidad de los conjuntos de datos médicos y el rendimiento de los modelos de aprendizaje profundo.

Id	Origen	Descripción	Probabilidad	Severidad	Prioridad
R01	Gestión	Planificación incorrecta de la duración, terminando el proyecto después de la fecha prevista.	Media	Media	Media
R02	Tecnológico	Incompatibilidades entre versiones de librerías de deep learning (TensorFlow, Keras, Transformers, CUDA Toolkit).	Alta	Alta	Alta
R03	Tecnológico	Dificultad en la integración de los módulos de entrenamiento Python con el backend FastAPI.	Media	Alta	Alta
R04	Gestión	Subestimación del tiempo de entrenamiento de los modelos de deep learning, especialmente en los Sprints de validación cruzada.	Alta	Alta	Alta
R05	Tecnológico	Dataset médico no disponible, inadecuado o con licencia incompatible para las pruebas de validación.	Media	Media	Media
R06	Tecnológico	Bloqueo o degradación de la interfaz web durante la ejecución prolongada de un entrenamiento.	Baja	Media	Baja
R07	Externo	Pérdida de código fuente por fallo del sistema de archivos o del equipo de desarrollo.	Baja	Muy Alta	Alta
R08	Infraestructura	Sobrecalentamiento o agotamiento de recursos de la GPU durante entrenamientos prolongados, provocando interrupciones.	Media	Alta	Alta
R09	Tecnológico	Falta de convergencia o rendimiento insuficiente de los modelos entrenados, invalidando los resultados del benchmarking.	Media	Alta	Alta

Tabla x - Riesgos Identificados

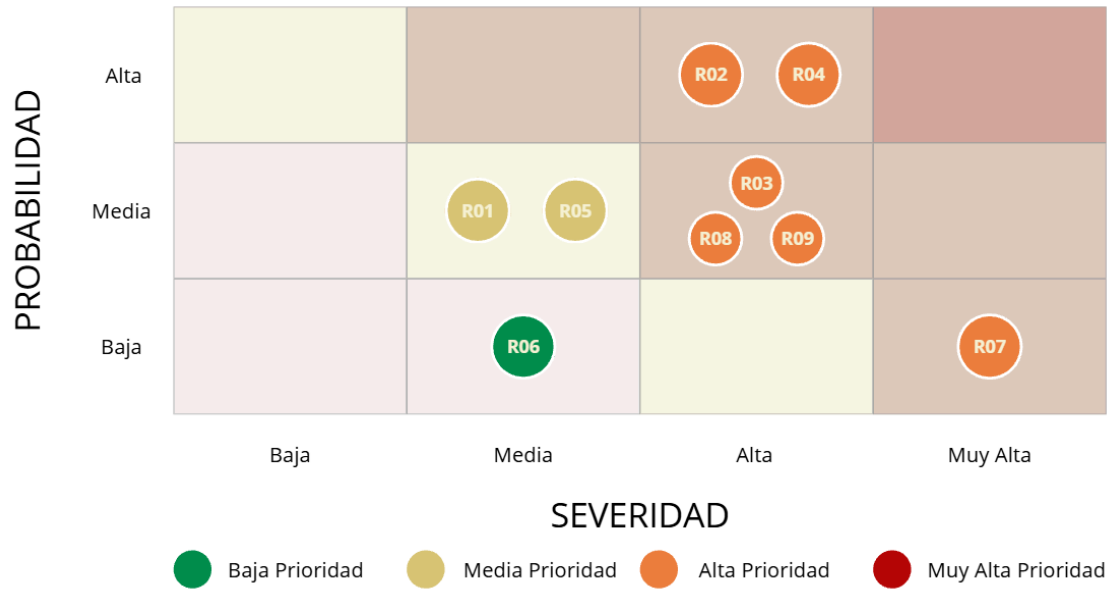


Ilustración x - Mapa de Prioridad

6.3. Plan de mitigación (medidas preventivas)

El plan de mitigación reúne las medidas preventivas previstas para reducir la probabilidad de cada riesgo o limitar sus efectos.

Frente al riesgo de una planificación incorrecta (R01), el alumno revisa el avance al cierre de cada Sprint junto con el tutor y compara el trabajo realizado con el previsto. Si se observa una desviación relevante, el alumno ajusta manualmente el cronograma y la carga pendiente antes de continuar con la siguiente iteración.

Para reducir las incompatibilidades entre librerías de aprendizaje profundo (R02), se fijan las versiones de las dependencias y se utilizan entornos virtuales de Anaconda. Además, durante el Sprint 1 se comprueba la compatibilidad de las librerías antes de construir el motor de inferencia.

Ante la dificultad de integrar los módulos de Python con el backend FastAPI (R03), los scripts de entrenamiento se ejecutan como procesos independientes de la capa web. Durante el Sprint 4 se comprueba la comunicación entre la API y los scripts antes de ampliar el resto de los endpoints.

Para reducir el efecto de la subestimación del tiempo de entrenamiento (R04), el trabajo comienza con arquitecturas más ligeras, como MobileNetV2 y EfficientNetB0. Sus tiempos de ejecución permiten calibrar el hardware y revisar el alcance del benchmarking antes de abordar modelos de mayor coste.

Respecto a la posible indisponibilidad del conjunto de datos médicos (R05), las radiografías se descargan localmente durante el Sprint 1. También se comprueban su formato, integridad y condiciones de uso, para reducir la dependencia de los repositorios externos durante el desarrollo.

Para evitar el bloqueo de la interfaz durante los procesos de cálculo prolongados (R06), las tareas pesadas se ejecutan de forma asíncrona. FastAPI delega estas operaciones al sistema de trabajos y la interfaz consulta su estado y progreso.

Frente a la pérdida de código fuente (R07), el proyecto utiliza un repositorio privado en GitHub y realiza copias de seguridad periódicas en un dispositivo externo. Los cambios relevantes se registran mediante commits descriptivos.

Para reducir el sobrecalentamiento o el agotamiento de recursos de la GPU (R08), las ejecuciones se separan mediante pausas y se controla la temperatura del equipo. Las arquitecturas más exigentes se ejecutan en periodos concretos y bajo supervisión.

Finalmente, para reducir el riesgo de falta de convergencia (R09), durante los sprints de entrenamiento se monitorizan las curvas de pérdida. El código utiliza parada temprana y reducción de la tasa de aprendizaje cuando detecta estancamiento. Las configuraciones iniciales de hiperparámetros se documentan y se ajustan a partir de los resultados de las primeras ejecuciones.

6.4. Plan de contingencia (medidas correctivas)

El plan de contingencia recoge las medidas correctoras que se aplican cuando un riesgo se materializa, con el fin de limitar sus efectos y mantener el proyecto dentro de un alcance viable.

R01 – Desviación temporal consolidada: Si el cronograma se desvía, el alumno revisa manualmente la ruta crítica y reduce, si es necesario, los elementos no esenciales, como variantes adicionales de Transformer o mejoras gráficas. La reducción del alcance se documenta y justifica en la memoria.

R02 – Conflicto de librerías: Si aparece una incompatibilidad entre versiones, se aísla la dependencia afectada y se consulta la documentación oficial para identificar una combinación compatible. Si es necesario, se crea un entorno de Anaconda con las versiones fijadas y se repiten las pruebas de compatibilidad antes de continuar. La configuración utilizada y la solución adoptada se registran para poder reproducir el entorno del proyecto.

R03 – Fallo en la integración del backend: Si la integración presenta problemas, los scripts de entrenamiento se mantienen como procesos independientes, invocados desde FastAPI mediante el módulo de subprocesos (`subprocess`). Esta separación permite comprobar la comunicación entre la API y los scripts por partes.

R04 – Tiempo de cómputo insuficiente: El riesgo se materializó durante el benchmarking. El alcance se redujo al subconjunto de ocho arquitecturas CNN con resultados conservados (ResNet50, DenseNet121, EfficientNetB0, EfficientNetV2B0, MobileNetV2, InceptionV3, Xception y ConvNeXtTiny), mientras que las arquitecturas restantes, incluidos los Transformers, quedaron fuera de la ejecución documentada. Esta reducción se recoge en la memoria para presentar los resultados como una evaluación parcial.

R05 – Cambio o no disponibilidad del dataset: Si los datos dejan de estar disponibles o no cumplen las condiciones previstas, no se sustituyen automáticamente. Primero se identifica un conjunto alternativo que sea accesible, anonimizado, compatible con la licencia y adecuado al problema clínico. Si se adopta, se repiten el entrenamiento, la validación cruzada, la validación externa y los análisis de explicabilidad, y se documenta el cambio y su efecto sobre la interpretación de los resultados.

R06 – Bloqueo de la interfaz gráfica: Si una tarea pesada bloquea la interfaz, se revisa su ejecución mediante la cola asíncrona y se comprueba el estado del worker. Si el bloqueo persiste, se informa al usuario del estado de la tarea y se registra la incidencia para ajustar la configuración del proceso.

R07 – Pérdida física del código fuente: Ante un fallo del equipo, se recupera el último estado disponible desde GitHub y se restaura el entorno de trabajo. A continuación, se revisa el trabajo pendiente y se reajusta el Sprint afectado.

R08 – Sobrecalentamiento o falta de recursos de la GPU: Se interrumpe el entrenamiento si la temperatura o el consumo de recursos alcanza un nivel problemático. Las ejecuciones se dividen en bloques más pequeños y, si es necesario, se reducen el tamaño de lote (batch size) o el número de épocas.

R09 – Falta de convergencia de un modelo: El desarrollador revisa las tasas de aprendizaje, el preprocesamiento y la configuración de validación. Si el modelo no converge después de los ajustes previstos, se excluye del resultado consolidado o se presenta como resultado no concluyente. La sustitución de la arquitectura y la repetición completa del protocolo quedan como trabajo posterior.

Capítulo 7. Planes de apoyo a la gestión del proyecto

Este capítulo reúne los planes de apoyo que complementan el marco de gestión descrito en el capítulo 4. Además de la metodología y de la organización del trabajo, el proyecto necesita procedimientos para gestionar la comunicación, revisar la calidad del producto y del proceso, y conservar de forma ordenada el código fuente y sus entregables. El capítulo se estructura en tres apartados dedicados a estas dimensiones.

7.1. Plan de Comunicación

La comunicación del proyecto se adapta a un desarrollo unipersonal. La planificación y la revisión de las iteraciones se realizan mediante el tablero y reuniones de seguimiento programadas según la fase y la disponibilidad del tutor, no mediante dieciocho ceremonias formales con dedicación presupuestada. En la práctica, hubo un par de reuniones durante la planificación inicial, reuniones más frecuentes durante el desarrollo activo y reuniones puntuales durante la fase de documentación. Las horas imputadas al tutor se concentran en las tareas 0.1 y 8.3; el resto de las comunicaciones y revisiones puntuales no se contabiliza como dedicación adicional en el apartado de costes.

Al margen de estos hitos formales, la sincronización operativa se realiza mediante el tablero Kanban, que muestra el estado del desarrollo, y el correo electrónico para la gestión logística. Las consultas técnicas con los asesores se organizan cuando la complejidad del trabajo lo requiere, especialmente durante la integración de los módulos XAI y la construcción de los pipelines algorítmicos.

7.2. Plan de Calidad

El aseguramiento de la calidad del sistema se organiza en cuatro líneas de actuación.

A nivel de pruebas, el código se somete a pruebas unitarias sobre los módulos de procesamiento y predicción, pruebas de integración sobre los flujos del sistema, pruebas de seguridad sobre la autenticación y el control de acceso, y pruebas de rendimiento sobre la inferencia, la ejecución asíncrona y el acceso concurrente. Estas comprobaciones se describen en el capítulo 16. El desarrollo se apoya además en el ciclo basado en especificaciones y guiado por pruebas (SDD/TDD) descrito en el capítulo 4. Las funcionalidades cubiertas por este ciclo disponen de pruebas automatizadas antes de integrarse.

Respecto a los estándares estructurales, el código Python sigue las convenciones de PEP 8, que se revisan mediante herramientas de análisis estático (van Rossum, Warsaw, & Coghlan, 2001; Python Software Foundation, 2024). Los scripts de entrenamiento y el backend mantienen separadas sus responsabilidades, y las funciones principales incluyen docstrings con información sobre su propósito, sus entradas y su retorno.

En cuanto a la automatización, el repositorio utiliza un flujo de integración continua (CI) mediante GitHub Actions. En los eventos configurados para main y refactorizacion se ejecutan las pruebas con pytest, el análisis de estilo con Ruff y la comprobación de que la cobertura no desciende por debajo del 70 % (pytest, 2024; Ruff, 2024). Estas comprobaciones sirven como control previo a la integración de cambios.

Finalmente, para mejorar la reproducibilidad de los experimentos frente a la variabilidad debida a la inicialización aleatoria, el código fija semillas en las librerías críticas (TensorFlow, NumPy y Python) y almacena las configuraciones de cada experimento en archivos JSON vinculados a la sesión de entrenamiento. Estas medidas facilitan la repetición y la trazabilidad de los resultados, pero no garantizan por sí solas un determinismo bit a bit: las operaciones concurrentes de la GPU, las versiones del software y otros factores del entorno pueden introducir pequeñas diferencias entre ejecuciones. Por ello, los resultados deben reproducirse bajo las mismas condiciones documentadas y compararse atendiendo a la variabilidad observada.

7.3. Plan de Gestión de la Configuración

El código fuente del proyecto se almacena en un repositorio de GitHub. La rama principal (main) contiene el estado integrado y el trabajo puede organizarse en ramas con nombres descriptivos, como feature/xai-quantitative-metrics o feature/training-pipeline-cnn. La incorporación de cambios a main puede realizarse mediante Pull Requests para ejecutar las validaciones de GitHub Actions antes de la integración. Los commits descriptivos y las copias externas del repositorio se utilizan como medidas de trazabilidad y recuperación cuando resultan necesarios.

El repositorio integra un archivo .gitignore que excluye del control de versiones los conjuntos de datos radiológicos, los pesos de los modelos entrenados (archivos .keras y .h5), los artefactos generados (radiografías subidas, PDF y mapas de calor) y los residuos del entorno de desarrollo, como las cachés de Python, los entornos virtuales y las configuraciones locales. Esta configuración evita almacenar datos sensibles y mantiene en el repositorio los elementos necesarios para reproducir el entorno.

Los estados estables del código pueden identificarse mediante commits o etiquetas de Git cuando sea necesario relacionarlos con un entregable o un punto concreto de la planificación. El repositorio no mantiene una política formal de versiones etiquetadas para cada Sprint.

Capítulo 8. Cuestiones abiertas y decisiones adoptadas

A fecha de elaboración de este documento, algunas decisiones técnicas ya están consolidadas y otras cuestiones permanecen abiertas porque dependen de trabajo futuro o de resultados todavía incompletos. El capítulo presenta primero las decisiones adoptadas y después las cuestiones pendientes.

Dataset de entrenamiento: Se adopta el conjunto de datos público de radiografías de tórax pediátricas publicado por Kermany y colaboradores, procedente del Guangzhou Women and Children's Medical Center y disponible en Kaggle. Este dataset contiene 5.856 radiografías en formato JPEG, clasificadas en dos categorías: NORMAL (1.583 imágenes) y PNEUMONIA (4.273 imágenes), estas últimas subdivididas en neumonía bacteriana y viral. Se ha seleccionado por ser el mismo utilizado en el estudio de referencia (Kermany & al., 2018), sin asumir que ambos trabajos utilicen el mismo protocolo experimental o comparen las mismas arquitecturas. El desbalanceo entre clases se aborda mediante submuestreo aleatorio en cada pliegue de la validación cruzada. Durante el desarrollo y la depuración se han empleado subconjuntos reducidos para agilizar las pruebas, aunque la metodología está preparada para trabajar con el conjunto completo.

Dataset de validación externa: Se adopta el conjunto COVID-19 Radiography Database, desarrollado por investigadores de la Universidad de Qatar y la Universidad de Dhaka, que contiene radiografías de tórax de pacientes adultos (Chowdhury & al., 2020; Rahman & al., 2021). Para la validación externa se seleccionan 500 imágenes de pacientes sanos y 500 de pacientes con neumonía viral. El dataset representa una población distinta de la de entrenamiento, con equipos y protocolos de imagen diferentes, por lo que ofrece un escenario de evaluación de la generalización. Durante las pruebas de desarrollo también se han utilizado subconjuntos reducidos del conjunto externo.

Arquitecturas de deep learning: La selección inicial del banco de pruebas comprende dieciséis arquitecturas convolucionales y tres arquitecturas Transformer. Esta selección cubre modelos de distinta capacidad y permite comparar redes convolucionales con modelos basados en atención. La ejecución documentada es parcial: se completaron cinco folds para ocho CNN (ResNet50, DenseNet121, EfficientNetB0, EfficientNetV2B0, MobileNetV2, InceptionV3, Xception y ConvNeXtTiny). La finalización del banco de pruebas queda como cuestión abierta y se detalla al final del capítulo.

Técnicas XAI: Se han adoptado cuatro técnicas de inteligencia artificial explicable: Saliency Maps, SmoothGrad y Grad-CAM para arquitecturas convolucionales, y mapas de atención para arquitecturas Transformer (Simonyan, Vedaldi, & Zisserman, 2014; Smilkov & al., 2017; Selvaraju & al., 2017; Chefer, Gur, & Wolf, 2021). La evaluación cuantitativa utiliza Deletion AUC, Insertion AUC, Sparsity, Entropy y Stability SSIM. También se contempla el Expected Calibration Error (ECE) para valorar la calibración de las predicciones. Las técnicas visuales están disponibles en el módulo de explicabilidad del diagnóstico. En el laboratorio MLOps, los scripts generan los mapas previstos y calculan las métricas cuantitativas y de calibración.

Tecnologías del sistema: La pila tecnológica queda definida en este apartado por capas, sobre la base de las tecnologías empleadas en el desarrollo.

En cuanto al **backend**, se utiliza Python 3.11 como lenguaje principal. Sobre él se asienta FastAPI, que gestiona las rutas HTTP y la validación de los datos de entrada mediante Pydantic (versión 2) (FastAPI, 2024). El servidor se ejecuta con Uvicorn (Uvicorn, 2024) y las páginas se renderizan en el servidor mediante plantillas Jinja2. La persistencia se apoya en MySQL, con MariaDB en el entorno de desarrollo mediante XAMPP, y se accede a ella mediante mysql-connector-python y un pool de conexiones. El esquema relacional almacena los usuarios, las consultas de diagnóstico, las sesiones de entrenamiento, la cola de trabajos y los tokens de refresco de sesión.

En cuanto a la **seguridad**, el sistema incorpora una capa de protección transversal. Las contraseñas se almacenan aplicando el algoritmo de hash bcrypt, que incluye un factor de coste configurable. La autenticación se gestiona mediante tokens JWT (generados con la librería python-jose, algoritmo HS256) almacenados en cookies httpOnly, complementados con tokens de refresco opacos persistidos en la base de datos y sometidos a rotación en cada uso, con detección de su reutilización para revocar la sesión. Las peticiones que modifican estado están protegidas por un middleware de CSRF de doble cookie. Se aplica además limitación de peticiones con la librería slowapi (un límite general y un límite específico y más estricto sobre el inicio de sesión) y un middleware que inyecta cabeceras de seguridad (Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, entre otras).

En cuanto al **frontend**, la interfaz se construye con HTML5, CSS y JavaScript vanilla, descartándose de forma deliberada los frameworks de componentes como React, Vue o Angular. La decisión no ignora las ventajas de estos frameworks, sino que responde a la adecuación al proyecto: la aplicación no requiere un estado de interfaz altamente reactivo ni componentes complejos, y el renderizado en el servidor con Jinja2 permite entregar una interfaz accesible y mantenible sin introducir una cadena de herramientas de compilación (npm, webpack o similares) que añadiría complejidad operativa. Las páginas de la aplicación, que incluyen login, registro, panel de diagnóstico, laboratorio de entrenamiento y panel de administración, se componen en el servidor a partir de plantillas Jinja2 que inyectan dinámicamente los datos del sistema y los textos de la interfaz.

El **estilo visual** se apoya en Tailwind CSS, cargado mediante CDN y configurado con modo oscuro mediante la estrategia de clase (darkMode: 'class'), y en la librería de iconos Font Awesome (versión 6.4), también cargada por CDN. Las utilidades de Tailwind y un conjunto reducido de estilos propios se utilizan en las distintas pantallas, incluidas las tablas de resultados, los gráficos y los visores de imágenes.

La **interactividad** se implementa en JavaScript nativo (ES6), organizado en módulos por ámbito funcional: dashboard.js para el panel de diagnóstico, training.js para el laboratorio de entrenamiento, admin.js para la administración e i18n.js, compartido, para la internacionalización. La comunicación con la API se realiza mediante fetch, enviando los formularios de forma asíncrona con FormData para no recargar la página; un interceptor global de peticiones añade automáticamente la cabecera de protección CSRF (X-CSRF-Token) y gestiona la renovación automática de la sesión cuando el token de acceso expira, reintentando la petición original. El estado de los procesos en segundo plano se actualiza mediante polling periódico del panel de la cola de trabajos, y las imágenes de resultados pueden inspeccionarse a tamaño completo mediante un visor modal. La internacionalización se apoya en atributos data-i18n distribuidos en las plantillas y en un diccionario de traducciones en JavaScript que cubre cuatro idiomas (español, inglés, chino e hindú), de modo que al cambiar el idioma la interfaz se re-renderiza al instante, incluyendo los informes y el asistente conversacional.

En cuanto al **motor de inteligencia artificial**, el proyecto utiliza TensorFlow (versión 2.18) y Keras (versión 3) para la construcción, el entrenamiento y la inferencia de las arquitecturas convolucionales. Las arquitecturas se cargan dinámicamente desde `tf.keras.applications` con pesos preentrenados en ImageNet, aplicando transferencia de conocimiento: la base del modelo se mantiene congelada y se sustituye el cabezal de clasificación por uno propio adaptado al problema binario de la neumonía (Deng & al., 2009). La librería `tf.keras` se incorpora para garantizar la compatibilidad con los pesos guardados en formatos heredados de Keras. Los modelos Transformer, DeiT, Swin-Base y ViT-384, se integran mediante Hugging Face Transformers, empleando la clase `TFAutoModelForImageClassification` para su entrenamiento y carga en el backend (Touvron & al., 2021; Liu & al., 2021; Dosovitskiy & al., 2021). El preprocesamiento de las imágenes se realiza con OpenCV y con las utilidades de datos de TensorFlow.

En cuanto al **análisis de datos** y la **generación de informes**, el sistema utiliza pandas y NumPy para la gestión y el procesamiento de los datos, scikit-learn para las métricas de rendimiento, la validación cruzada estratificada y las curvas ROC, y SciPy para el test estadístico de Wilcoxon (Wilcoxon, 1945). El test de DeLong para la comparación de curvas ROC se implementa de forma propia en el pipeline de validación externa (DeLong, DeLong, & Clarke-Pearson, 1988). Las representaciones gráficas, como curvas ROC, matrices de calor de significación, rankings y mapas de explicabilidad, se generan con matplotlib y seaborn. Los informes descargables en formato PDF se generan con la librería fpdf2, tanto para los diagnósticos individuales como para los informes completos de las sesiones de entrenamiento.

En cuanto a la **inteligencia conversacional**, la plataforma se conecta a la API de Groq y utiliza el modelo openai/gpt-oss-120b servido mediante ese proveedor. El asistente se configura con un prompt de sistema que solicita una salida JSON estructurada con la configuración de los experimentos y está disponible en los cuatro idiomas de la plataforma. La elección del proveedor responde a su disponibilidad dentro del plan utilizado y a su integración sencilla con el proyecto (Groq, 2024).

Nota de actualización de la configuración final: durante la elaboración de la memoria, el modelo llama-3.3-70b-versatile dejó de estar disponible para la cuenta utilizada en Groq. La implementación final se actualizó a openai/gpt-oss-120b, manteniendo Groq como proveedor, la variable GROQ_API_KEY, el prompt de sistema y el contrato JSON del asistente. Por tanto, las referencias a Llama 3 describen la configuración inicialmente planificada; el modelo utilizado en la versión entregada es openai/gpt-oss-120b.

Por último, en cuanto al **despliegue**, el sistema se publica exponiendo el servidor local a través de un túnel de Cloudflare. El entorno de ejecución permanece en la máquina anfitriona y las versiones de sus librerías se mantienen documentadas antes de exponer el servicio mediante el túnel.

8.1. *Cuestiones abiertas*

Finalización del benchmarking. La selección inicial comprendía diecinueve arquitecturas, pero la ejecución experimental conservada solo cubre ocho CNN con cinco folds cada una. Quedan pendientes las ocho CNN restantes y las tres arquitecturas Transformer. Por tanto, la memoria debe presentar los resultados disponibles como una evaluación parcial y no como un benchmark completo. La ampliación del experimento queda como trabajo futuro, condicionada al tiempo de cómputo disponible.

Corrección por comparaciones múltiples. La comparación entre arquitecturas mediante pruebas estadísticas requiere aplicar una corrección por comparaciones múltiples antes de interpretar los contrastes entre pares. Esta corrección queda pendiente de aplicar en una ejecución estadística definitiva. Hasta entonces, la matriz de resultados debe tratarse como apoyo exploratorio y no como evidencia confirmatoria de diferencias entre modelos.

Progreso del entrenamiento en el catálogo de requisitos. El laboratorio calcula y muestra el progreso del entrenamiento por épocas, pero esta capacidad todavía no dispone de un requisito funcional propio en el catálogo. Su incorporación al catálogo y la actualización de la matriz de trazabilidad quedan pendientes de una revisión posterior de la especificación.

Capítulo 9. Consideraciones complementarias del proyecto

Las consideraciones éticas y de privacidad forman parte del alcance del proyecto, dado que se utilizan radiografías de tórax de origen humano para entrenar y evaluar los modelos de inteligencia artificial. En el alcance actual, vitalXAI es un prototipo académico de investigación y no un producto sanitario autorizado ni una herramienta destinada a tomar decisiones clínicas. Este capítulo resume el marco jurídico y las condiciones que tendría una futura puesta en servicio.

Todos los datasets utilizados para el entrenamiento y la validación externa proceden de repositorios públicos y se emplean como conjuntos anonimizados. El dataset de Kermany y colaboradores procede del Guangzhou Women and Children's Medical Center y fue publicado posteriormente en Kaggle (Kermany & al., 2018). El dataset COVID-19 Radiography Database también se utiliza como conjunto anonimizado y fue desarrollado por investigadores de la Universidad de Qatar y de la Universidad de Dhaka (Chowdhury & al., 2020; Rahman & al., 2021). Este enfoque reduce los riesgos asociados al tratamiento de datos personales, pero no sustituye la revisión ética y jurídica necesaria para utilizar datos clínicos reales.

El sistema se ha probado con datasets públicos y anonimizados. Aunque la aplicación dispone de una función para cargar radiografías en el flujo de demostración, no se han realizado pruebas con imágenes de pacientes identificables. Cualquier uso futuro con datos clínicos exigiría aplicar las condiciones de protección de datos y las autorizaciones que correspondan a la institución responsable.

En el alcance actual, vitalXAI no está autorizado para prestar asistencia clínica ni para utilizarse como sustituto del criterio profesional. Las predicciones generadas por los modelos solo se evalúan como resultados experimentales. La eventual utilización clínica requeriría la supervisión de profesionales sanitarios y el cumplimiento de las condiciones regulatorias descritas en este capítulo.

9.1. Marco regulatorio

Las radiografías de tórax pueden constituir datos relativos a la salud y, cuando permiten relacionarse con una persona identificada o identificable, forman parte de las categorías especiales de datos personales previstas en el artículo 9 del Reglamento General de Protección de Datos (RGPD) (Parlamento Europeo y Consejo de la Unión Europea, 2016). En este TFG se utilizan datasets públicos y anonimizados, y no se realizan pruebas ni demostraciones con radiografías de pacientes identificables. La anonimización previa es una condición del uso de los datos, no una funcionalidad que el sistema compruebe automáticamente.

Si una versión futura tratase datos clínicos, la entidad que determinase los fines y medios del tratamiento asumiría la posición de responsable. Según la organización concreta, podría ser la universidad, un hospital o la entidad de investigación que decidiese esos fines. Un proveedor que tratase los datos siguiendo instrucciones documentadas podría actuar como encargado, en cuyo caso sería necesario formalizar la relación conforme al artículo 28 del RGPD. Estas posiciones no se asignan en el TFG porque no existe un tratamiento de datos clínicos reales ni un servicio de producción (Parlamento Europeo y Consejo de la Unión Europea, 2016).

La consideración de vitalXAI como software sanitario dependería de su finalidad prevista y de su puesta en servicio. Si una versión futura proporcionase información utilizada para decisiones diagnósticas, podría quedar dentro del Reglamento (UE) 2017/745 sobre productos sanitarios y tendría que someterse a la clasificación y evaluación de conformidad correspondientes. Del mismo modo, si se integrase como componente de seguridad de un producto sanitario sujeto a evaluación por terceros, podría quedar dentro de los supuestos de alto riesgo del Reglamento (UE) 2024/1689 sobre inteligencia artificial. En ese escenario serían necesarias, entre otras medidas, la gestión de riesgos, la documentación técnica, la supervisión humana, la trazabilidad y los controles de calidad (Parlamento Europeo y Consejo de la Unión Europea, 2017, 2024). El prototipo actual no dispone de marcado CE, no se presenta como producto sanitario y no realiza esa evaluación de conformidad.

Por último, el consumo energético de los entrenamientos constituye una consideración medioambiental del propio desarrollo. En este proyecto, los entrenamientos se han realizado preferentemente en un equipo local con GPU y la ejecución documentada se ha limitado a los experimentos disponibles. La preferencia por la computación local responde a criterios de coste y de disponibilidad de la infraestructura. No se ha imputado al presupuesto una medición específica de electricidad ni existe una reserva económica para cloud; si el hardware local dejara de estar disponible, el uso de infraestructura en la nube requeriría una decisión adicional de coste o de alcance y quedaría documentado explícitamente. La reducción del número de ejecuciones documentadas limita el consumo asociado al desarrollo, aunque no se dispone de una medición que permita cuantificar su huella de carbono.

Parte II.

Análisis del Sistema

Capítulo 10. Planteamiento del análisis del sistema

Para diseñar e implementar un sistema es necesario concretar la idea inicial en afirmaciones verificables sobre sus capacidades, sus restricciones y sus usuarios. Esta parte de la memoria desarrolla esa concreción y este capítulo introduce el propósito y las dimensiones del análisis (Larman, 2004; Wiegers & Beatty, 2013).

10.1. *Propósito del análisis*

El análisis fija las bases sobre las que se construirá el sistema. Por un lado, evita asumir durante el diseño y la implementación capacidades que no forman parte del alcance o dejar fuera requisitos acordados. Por otro, documenta las restricciones del entorno y las decisiones de alcance, de modo que el tutor, el tribunal o un futuro mantenedor puedan entender qué debe ofrecer el sistema y cuáles son sus límites.

En este proyecto, la formalización del análisis es especialmente relevante porque el sistema combina una aplicación web orientada al diagnóstico asistido y un laboratorio de investigación de aprendizaje automático. La primera requiere claridad, seguridad y facilidad de uso; el segundo requiere flexibilidad, reproducibilidad y capacidad para ejecutar experimentos. El análisis permite considerar ambas perspectivas desde el inicio y dejar documentadas las decisiones que afectan a cada una.

10.2. *Dimensiones del análisis*

El análisis del sistema comienza con la definición de sus objetivos y de su ámbito. El capítulo 11 delimita las capacidades de la plataforma, el entorno tecnológico, la normativa aplicable y los perfiles de usuario. Estos elementos establecen el contexto del producto, pero no agotan el análisis. A partir de ellos, los capítulos siguientes transforman las necesidades del proyecto en requisitos verificables, casos de uso, subsistemas, entidades del dominio y pruebas de consistencia y comportamiento.

Sobre esa base, el análisis especifica qué debe hacer el sistema y cómo se organiza para ofrecer esas capacidades. Esta parte se estructura en los capítulos 12 a 16:

- **Especificación de requisitos y casos de uso (capítulo 12).** Se definen los requisitos funcionales y no funcionales del sistema y los casos de uso que describen la interacción de cada perfil de usuario con la plataforma.
- **Identificación de subsistemas (capítulo 13).** Las capacidades del sistema se agrupan en subsistemas de análisis, lo que permite organizar el resto del análisis y, posteriormente, el diseño.
- **Modelo de dominio (capítulo 14).** Se identifican las entidades que el sistema debe gestionar y se describe su comportamiento dinámico mediante secuencias de interacción.
- **Verificación de la consistencia (capítulo 15).** Se comprueba la coherencia entre los objetivos, los requisitos, los casos de uso y los subsistemas.
- **Plan de pruebas (capítulo 16).** Se define la estrategia de pruebas que verificará el cumplimiento de lo especificado.

El conjunto de estas dimensiones constituye el marco de referencia del diseño del sistema, desarrollado en la parte siguiente de la memoria.

Capítulo 11. Definición del ámbito del sistema

El contexto y el estado del arte en el que se inscribe vitalXAI se presentaron en el capítulo 1, donde se describieron el problema de la neumonía, la radiografía de tórax como imagen digital, la evolución del aprendizaje profundo aplicado al diagnóstico por imagen, los conjuntos de datos y los modelos de referencia, y las plataformas MLOps existentes. Este capítulo no repite ese análisis: asume lo allí expuesto y se centra en delimitar el ámbito del sistema, es decir, qué capacidades incluye vitalXAI como producto, bajo qué entorno tecnológico, con qué normativa y para qué usuarios.

El capítulo delimita el alcance funcional de vitalXAI y las condiciones generales en las que debe operar. No incorpora una revisión bibliográfica ni compara soluciones existentes, porque esa fundamentación ya se encuentra en el capítulo 1.

La plataforma se concibe como una aplicación web de servidor. El usuario final accede mediante un navegador y no necesita instalar componentes locales. La infraestructura de ejecución sí requiere un entorno compatible con el motor de aprendizaje automático, capacidad de cómputo acelerado y almacenamiento para los datos y artefactos del sistema. Estas condiciones se analizan en el apartado de contexto tecnológico, mientras que las tecnologías concretas se justifican en los capítulos 17 y 23.

Desde el punto de vista funcional, el alcance se organiza en dos núcleos: el diagnóstico asistido y el laboratorio de experimentación MLOps. Ambos se complementan con capacidades transversales de autenticación, aislamiento de datos, ejecución asíncrona, internacionalización y administración. La definición formal de estas capacidades comienza con los objetivos del sistema y continúa con los requisitos y casos de uso del análisis.

11.1. Objetivos del sistema

Los objetivos del sistema describen lo que vitalXAI debe ofrecer como producto. Se obtienen a partir de las reuniones mantenidas con el tutor y de los objetivos generales del proyecto, y sirven de base para los requisitos de los capítulos siguientes. Cada objetivo se presenta mediante una ficha con su identificador, nombre, descripción, importancia y estado. Estos objetivos se diferencian de los objetivos del plan de proyecto, que describen lo que el alumno debe conseguir durante el desarrollo.

OBJ-001 – Diagnóstico asistido con inteligencia artificial explicable

El diagnóstico asistido es el núcleo de la plataforma y el punto de partida de todo el sistema. Un profesional sanitario, sin conocimientos de inteligencia artificial, debe poder cargar una radiografía de tórax y obtener, en un tiempo razonable, un diagnóstico con su nivel de confianza y una explicación visual de los motivos que sustentan la decisión del modelo. La confianza es importante porque permite al facultativo calibrar la fiabilidad de la predicción, y la explicación visual es importante porque le permite comprobar si el modelo se está fijando en las regiones pulmonares correctas. Esta ficha fija ese compromiso.

Campo	Contenido
ID	OBJ-001
Nombre	Diagnóstico asistido con inteligencia artificial explicable
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe permitir al profesional sanitario cargar una radiografía de tórax, seleccionar una arquitectura de deep learning y obtener un diagnóstico con su nivel de confianza, acompañado de los mapas de explicabilidad que justifican la predicción.
Importancia	Alta
Estado	Aprobado
Comentarios	Este objetivo recoge el núcleo clínico del sistema. El mecanismo concreto de generación de las explicaciones corresponde al ámbito del diseño.

Tabla x - OBJ-001: Diagnóstico asistido con inteligencia artificial explicable

OBJ-002 – Gestión del historial de consultas

Un facultativo realiza numerosas consultas a lo largo del tiempo, y cada una de ellas debe quedar registrada para poder ser consultada, revisada o corregida posteriormente. Sin un historial organizado, el profesional perdería la capacidad de comparar la evolución de un mismo caso o de recuperar un diagnóstico anterior. Este objetivo garantiza que el sistema conserve el historial completo de diagnósticos de cada usuario de forma aislada y gestionable.

Campo	Contenido
ID	OBJ-002
Nombre	Gestión del historial de consultas
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe conservar el historial de consultas de diagnóstico de cada usuario, permitiendo consultarlas, renombrarlas y eliminarlas, de modo que cada profesional gestione únicamente sus propios registros.
Importancia	Media
Estado	Aprobado
Comentarios	Este objetivo complementa al OBJ-001 y refuerza el aislamiento de datos entre usuarios.

Tabla x - OBJ-002: Gestión del historial de consultas

OBJ-003 – Generación de informes descargables

Los resultados de un diagnóstico no deben quedarse en la pantalla: el profesional necesita un documento que pueda archivar, imprimir o incorporar a su flujo de trabajo habitual. Este objetivo cubre la generación de informes en formato PDF. El informe consolidado de una sesión de entrenamiento está formalizado en el requisito RF-030, que fija su contenido (configuración del experimento, ranking de modelos, comparativas estadísticas, validación externa y métricas de explicabilidad y calibración). El informe individual de un diagnóstico se formaliza en el requisito RF-039 y se verifica en la prueba PU-017.

Campo	Contenido
ID	OBJ-003
Nombre	Generación de informes descargables
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe generar informes en formato PDF. El informe consolidado de cada sesión de entrenamiento se especifica en el requisito RF-030. El informe individual de cada diagnóstico se genera junto con el resultado de la consulta y se verifica en la prueba PU-017.
Importancia	Media
Estado	Aprobado
Comentarios	El informe de sesión se formaliza en RF-030 y el informe individual del diagnóstico en RF-039. La implementación del informe individual se verifica en PU-017.

Tabla x - OBJ-003: Generación de informes descargables

OBJ-004 – Laboratorio de entrenamiento MLOps

El segundo núcleo del sistema es el laboratorio de entrenamiento. Su propósito es que un investigador pueda configurar y lanzar experimentos de entrenamiento de modelos sin escribir código, apoyándose en un asistente conversacional que interpreta sus indicaciones y las traduce a la configuración técnica del pipeline. El usuario debe poder consultar los resultados de forma organizada y conocer el estado de sus trabajos. Conviene distinguir aquí dos nociones relacionadas pero distintas: el estado de la cola de trabajos, cubierto por el requisito RF-036 (pendiente, en ejecución, completado o fallido), y el progreso del entrenamiento por épocas, que el laboratorio calcula y muestra mientras la ejecución avanza. El cálculo de ese progreso está implementado y verificado en la prueba PU-023, aunque la especificación de requisitos no le dedica un requisito funcional propio, laguna que deberá resolverse en una revisión posterior del catálogo de requisitos.

Campo	Contenido
ID	OBJ-004
Nombre	Laboratorio de entrenamiento MLOps
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe permitir al usuario configurar y lanzar experimentos de entrenamiento de modelos mediante un asistente conversacional, consultar los resultados generados por el pipeline de forma organizada y conocer el estado de sus trabajos.
Importancia	Alta
Estado	Aprobado
Comentarios	Este objetivo engloba el ciclo completo del experimento desde la perspectiva del usuario: configurar, lanzar, consultar el estado y recuperar los resultados. El estado de la cola está cubierto por el requisito RF-036; el progreso por épocas está implementado y verificado en la prueba PU-023, pero carece de un requisito funcional propio en la especificación actual.

Tabla x - OBJ-004: Laboratorio de entrenamiento MLOps

OBJ-005 – Evaluación rigurosa de los modelos

El sistema debe permitir comparar el rendimiento de los modelos y observar su comportamiento sobre una cohorte independiente. Este objetivo incluye la evaluación estadística, la validación externa y la presentación de los resultados de forma comprensible para el investigador. Los contrastes deben interpretarse teniendo en cuenta las limitaciones del protocolo experimental.

Campo	Contenido
ID	OBJ-005
Nombre	Evaluación rigurosa de los modelos
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe automatizar la evaluación estadística de los modelos entrenados: comparación de rendimiento, validación externa sobre una cohorte independiente y contrastes de significación, presentando los resultados de forma comprensible para el usuario.
Importancia	Alta
Estado	Aprobado
Comentarios	La concreción de las pruebas estadísticas se especifica en los requisitos derivados de este objetivo.

Tabla x - OBJ-005: Evaluación rigurosa de los modelos

OBJ-006 – Reproducibilidad de los experimentos

La reproducibilidad es especialmente relevante en el aprendizaje automático, donde una inicialización aleatoria o un orden de datos diferente pueden alterar los resultados. Este objetivo busca asociar cada experimento con su configuración y sus semillas aleatorias, de modo que los resultados puedan verificarse y repetirse bajo las mismas condiciones (Varoquaux & Cheplygina, 2019).

Campo	Contenido
ID	OBJ-006
Nombre	Reproducibilidad de los experimentos
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe favorecer la reproducibilidad de los experimentos de entrenamiento, fijando las semillas aleatorias y almacenando la configuración completa de cada sesión junto con sus resultados, de modo que los resultados puedan verificarse y replicarse bajo las mismas condiciones.
Importancia	Media
Estado	Aprobado
Comentarios	Este objetivo responde a las limitaciones de reproducibilidad identificadas en la literatura sobre aprendizaje automático (Varoquaux & Cheplygina, 2019).

Tabla x - OBJ-006: Reproducibilidad de los experimentos

OBJ-007 – Acceso seguro y personalizado al sistema

El sistema trata información de carácter clínico, aunque anonimizada, y cada usuario debe disponer de un espacio de trabajo aislado. Este objetivo cubre el registro, la autenticación, el cierre de sesión y el acceso de cada profesional a sus propios datos. El modelo de autorización distingue estas operaciones de las funciones de gobierno reservadas al administrador, que se recogen en el OBJ-011.

Campo	Contenido
ID	OBJ-007
Nombre	Acceso seguro y personalizado al sistema
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe disponer de un módulo de gestión de usuarios que permita el registro, la autenticación y el cierre de sesión de forma segura, garantizando que cada usuario opere únicamente sobre sus propios datos y consultas. El rol de administrador y las operaciones de gobierno se recogen en el OBJ-011.
Importancia	Alta
Estado	Aprobado
Comentarios	La autenticación es necesaria por el carácter clínico de los datos gestionados. El mecanismo concreto de autenticación corresponde al diseño.

Tabla x - OBJ-007: Acceso seguro y personalizado al sistema

OBJ-008 – Persistencia y trazabilidad de la información

El sistema debe almacenar de forma persistente las consultas de diagnóstico y las sesiones de entrenamiento, junto con su configuración y sus resultados, de modo que el usuario pueda consultar su historial en cualquier momento. La persistencia no es una funcionalidad visible para el usuario final, sino la base sobre la que se sustentan el historial clínico, la comparación de experimentos y la trazabilidad de los resultados.

Campo	Contenido
ID	OBJ-008
Nombre	Persistencia y trazabilidad de la información
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe almacenar de forma persistente las consultas de diagnóstico y las sesiones de entrenamiento, junto con su configuración y sus resultados, de modo que el usuario pueda consultar su historial en cualquier momento.
Importancia	Media
Estado	Aprobado
Comentarios	La persistencia es un requisito transversal: sustenta al resto de los objetivos aunque no sea una funcionalidad visible para el usuario final.

Tabla x - OBJ-008: Persistencia y trazabilidad de la información

OBJ-009 – Usabilidad e internacionalización de la interfaz

El sistema está pensado para usuarios sin formación técnica, por lo que la interfaz debe ser intuitiva y no exigir en ningún momento interactuar con código ni con entornos de programación. Además, el sistema debe estar disponible en varios idiomas, de modo que el usuario pueda cambiar de idioma de forma dinámica, ampliando la accesibilidad de la plataforma a entornos multilingües.

Campo	Contenido
ID	OBJ-009
Nombre	Usabilidad e internacionalización de la interfaz
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe ofrecer una interfaz web intuitiva para usuarios con perfil no técnico, que permita realizar todas las operaciones sin interactuar con código ni con entornos de programación, y debe estar disponible en varios idiomas, permitiendo cambiar de idioma de forma dinámica.
Importancia	Media
Estado	Aprobado
Comentarios	La usabilidad orienta el diseño de la interfaz, pero no determina la corrección funcional de los modelos.

Tabla x - OBJ-009: Usabilidad e internacionalización de la interfaz

OBJ-010 – Ejecución asíncrona de tareas de larga duración

El entrenamiento de un modelo de deep learning puede prolongarse durante horas, y durante ese tiempo la plataforma debe seguir respondiendo al usuario. Este objetivo garantiza que las tareas largas (entrenamientos, análisis de explicabilidad y validación externa) se ejecuten de forma asíncrona a través de una cola de trabajos, de modo que la interfaz permanezca operativa y el usuario pueda consultar el estado de cada tarea e incluso cancelar las tareas pendientes.

Campo	Contenido
ID	OBJ-010
Nombre	Ejecución asíncrona de tareas de larga duración
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe ejecutar las tareas de larga duración (entrenamientos, análisis de explicabilidad y validación externa) de forma asíncrona, de modo que la interfaz permanezca operativa durante su ejecución y el usuario pueda consultar el estado de cada tarea.
Importancia	Media
Estado	Aprobado
Comentarios	Este objetivo favorece la disponibilidad de la plataforma durante los procesos computacionales más costosos.

Tabla x - OBJ-010: Ejecución asíncrona de tareas de larga duración

OBJ-011 – Administración de la plataforma

El gobierno de una plataforma con múltiples usuarios requiere de un perfil con privilegios de administración que pueda gestionar las cuentas y supervisar la actividad registrada. Este objetivo cubre la existencia de un panel de administración y de un rol diferenciado dentro del mecanismo de autorización. La frontera con el OBJ-007 queda así definida: el acceso seguro garantiza quién puede entrar y sobre qué datos, mientras que la administración cubre las operaciones de gobierno sobre el conjunto de la plataforma, reservadas al administrador. Esta frontera se refleja en la trazabilidad de los requisitos: el requisito transversal de roles y control de acceso (RF-006) se asocia a ambos objetivos, mientras que la auditoría de la actividad administrativa (RNF-006) se asocia únicamente al OBJ-011.

Campo	Contenido
ID	OBJ-011
Nombre	Administración de la plataforma
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe disponer de un panel de administración que permita gestionar las cuentas de usuario y supervisar las consultas y sesiones de entrenamiento registradas en la plataforma, con acceso restringido al rol de administrador.
Importancia	Media
Estado	Aprobado
Comentarios	Este objetivo define la administración como las operaciones de gobierno reservadas al administrador, complementando al OBJ-007, que cubre el acceso seguro y el aislamiento de datos. La auditoría de la actividad administrativa se especifica en el requisito no funcional RNF-006.

Tabla x - OBJ-011: Administración de la plataforma

La siguiente tabla establece la correspondencia entre los once objetivos del sistema y los objetivos específicos del plan de proyecto presentados en el capítulo 2 del Plan de Proyecto (OE1 a OE11). No se trata de una relación uno a uno: varios objetivos del sistema se apoyan en un mismo objetivo del plan, y algunos objetivos del plan son de carácter científico-metodológico y sustentan a más de un objetivo del producto. El objetivo específico OE1 (revisar el estado del arte) no figura en la correspondencia porque pertenece a la fase de fundamentación del proyecto y no se materializa como una capacidad del producto.

Objetivo del sistema	Objetivo(s) específico(s) del plan	Relación
OBJ-001 Diagnóstico asistido con IA explicable	OE5, OE8	La explicabilidad (OE5) y la interfaz clínica (OE8) sustentan el diagnóstico explicable.
OBJ-002 Gestión del historial de consultas	OE2	La arquitectura de persistencia (OE2) soporta el historial.
OBJ-003 Generación de informes descargables	OE8, OE9	El informe individual se entrega con el diagnóstico (OE8); el informe de sesión se consolida en el laboratorio (OE9).
OBJ-004 Laboratorio de entrenamiento MLOps	OE9	El laboratorio y el asistente conversacional (OE9) materializan el laboratorio.
OBJ-005 Evaluación rigurosa de los modelos	OE4, OE6	El pipeline (OE4) produce las métricas; la validación externa y el análisis estadístico (OE6) las evalúan.
OBJ-006 Reproducibilidad de los experimentos	OE7	La reproducibilidad y la trazabilidad (OE7) se materializan en este objetivo.
OBJ-007 Acceso seguro y personalizado al sistema	OE3	El control de acceso (OE3) sustenta este objetivo.
OBJ-008 Persistencia y trazabilidad de la información	OE2	La arquitectura de persistencia (OE2) sustenta este objetivo.
OBJ-009 Usabilidad e internacionalización de la interfaz	OE8, OE11	La interfaz aborda la usabilidad (OE8) y el soporte multilingüe corresponde a OE11.

Objetivo del sistema	Objetivo(s) específico(s) del plan	Relación
OBJ-010 Ejecución asíncrona de tareas de larga duración	OE10	El procesamiento asíncrono (OE10) se materializa en este objetivo.
OBJ-011 Administración de la plataforma	No aplica	Sin objetivo específico directo: la administración amplía el alcance planificado y se apoya, para el rol, en el control de acceso (OE3).

11.2. Contexto tecnológico y restricciones de entorno

El sistema se desarrolla sobre un entorno tecnológico que establece determinadas condiciones de trabajo. Este apartado distingue entre las restricciones que condicionan el diseño y las decisiones tecnológicas propias del proyecto, que se justifican en la parte de diseño de la memoria.

La principal restricción del entorno es la compatibilidad con Python, lenguaje utilizado por las librerías de aprendizaje automático, los formatos de los modelos y las herramientas de evaluación del proyecto. Por ello, el entorno de ejecución debe mantener la compatibilidad con ese ecosistema.

El entrenamiento requiere capacidad de cómputo acelerado mediante GPU, especialmente cuando se ejecuta validación cruzada sobre arquitecturas de mayor tamaño. El proyecto utiliza un equipo portátil con una GPU NVIDIA RTX 4060 Laptop y 8 GB de VRAM. Esta infraestructura permite realizar las ejecuciones previstas con tamaños de lote reducidos y precisión mixta, aunque el tiempo disponible limita el número de experimentos que pueden completarse.

El equipo forma parte de la infraestructura ya disponible y su amortización no se imputa al coste del TFG. Se descartó contratar instancias en la nube por el coste asociado y porque el trabajo local permite disponer de los pesos y los datos sin depender de un servicio externo. Como consecuencia, los tiempos de entrenamiento deben estimarse con prudencia y los experimentos más costosos deben programarse con antelación.

Sobre estas restricciones, el sistema opera con un servicio web, un motor de aprendizaje automático, una base de datos relacional y un asistente conversacional conectado a un servicio externo. La elección de los frameworks, las librerías y sus versiones corresponde al diseño. En el análisis solo se fija el contexto tecnológico resumido en la tabla.

Aspecto del entorno	Contexto relevante para el análisis
Lenguaje y entorno de ejecución	Python y un entorno compatible con las librerías de aprendizaje automático.
Aplicación web	Aplicación servidor accesible desde un navegador.
Persistencia	Base de datos relacional para cuentas, consultas, sesiones y trabajos.
Aprendizaje automático	Frameworks de deep learning para arquitecturas convolucionales y basadas en atención.
Procesamiento y evaluación	Herramientas Python para preparar datos, calcular métricas y generar representaciones.
Asistente conversacional	Servicio externo de inferencia lingüística integrado mediante una API.

Tabla x - Contexto tecnológico del sistema

La tabla anterior recoge el contexto tecnológico de trabajo, no la especificación de implementación. Las tecnologías concretas, las versiones de las dependencias, las librerías de seguridad y las herramientas de calidad y pruebas se seleccionan y justifican en la parte de diseño, especialmente en los capítulos 17 y 23.

En cuanto al despliegue, el sistema debe poder publicarse desde la máquina de ejecución y ser accesible mediante un navegador. La forma concreta de exponer el servicio, el proveedor utilizado y las alternativas descartadas son decisiones de diseño y se documentan en los capítulos 17 y 23. En este capítulo solo se establece que el acceso remoto debe protegerse mediante un canal cifrado y que la infraestructura local no debe exponer directamente sus servicios internos.

En cuanto a la reproducibilidad, la fijación de versiones y el aislamiento del entorno reducen una fuente de variabilidad, pero no garantizan resultados idénticos en todas las ejecuciones. Como se señala en el objetivo OBJ-006, se busca que los resultados puedan verificarse y repetirse bajo las mismas condiciones, no un determinismo bit a bit. La gestión de las dependencias se orienta a mantener un entorno coherente y reconstruible.

En definitiva, la elección de Python y la disponibilidad de una GPU son restricciones del entorno actual del proyecto. El framework web, la base de datos, las librerías de seguridad, las versiones concretas y el mecanismo de despliegue son decisiones de diseño que se justifican en los capítulos correspondientes.

11.3. Normativa y estándares de referencia

El desarrollo del sistema está condicionado por normas y estándares relacionados con la protección de datos, el ámbito sanitario, la seguridad de las comunicaciones, la calidad del código y la documentación del TFG. A continuación se resume su relación con el proyecto.

En primer lugar, el sistema gestiona cuentas de usuarios e imágenes de origen médico. Cuando estos datos permiten identificar a una persona, pueden quedar sujetos a las categorías especiales previstas en el artículo 9 del Reglamento General de Protección de Datos (RGPD). El proyecto utiliza datasets públicos y anonimizados, aplica la minimización de la información y limita el acceso mediante cuentas y roles (Parlamento Europeo y Consejo de la Unión Europea, 2016; España, 2018). El acceso administrativo a datos de otros usuarios se limita a las funciones de supervisión descritas en el requisito RF-005 y en los requisitos RF-033 a RF-035.

Conviene ser preciso sobre la anonimización. El proyecto asume que las imágenes que llegan al sistema están anonimizadas y que los conjuntos de datos utilizados son públicos y anónimos, pero esa asunción no equivale a una garantía implementada: si un facultativo sube una radiografía con identificadores visibles en la imagen o con metadatos DICOM que revelen la identidad del paciente, el sistema los almacena junto con la imagen. En el estado actual del proyecto no existe un mecanismo que despoje automáticamente esos metadatos ni una validación que compruebe la ausencia de identificadores antes de aceptar una carga. Esta circunstancia debe documentarse como una limitación: la anonimización previa recae hoy en el responsable de la captura, y una evolución del sistema debería incorporar la limpieza de metadatos y la detección de identificadores como requisitos explícitos.

El asistente conversacional del laboratorio envía a la API de Groq los mensajes utilizados para configurar un experimento. Aunque esos mensajes se orientan a parámetros técnicos y no a datos clínicos, existe una comunicación con un proveedor externo. Por ello, el despliegue debe documentar el tratamiento y revisar las condiciones aplicables al proveedor.

En segundo lugar, el sistema se inscribe en el ámbito de las tecnologías sanitarias y de la inteligencia artificial. El marco regulatorio aplicable al tratamiento de radiografías, a la posible consideración del software como producto sanitario y a la clasificación futura del sistema de IA se desarrolla en el capítulo 9. En el alcance actual, vitalXAI se limita a un prototipo académico de investigación, sin intención de comercialización ni puesta en servicio clínica.

En tercer lugar, el sistema mantiene una comunicación constante entre el servidor y el navegador del cliente, a través de la cual viajan credenciales de acceso y datos de las consultas. Estas comunicaciones deben protegerse mediante el protocolo HTTPS, que se fundamenta en el estándar TLS (IETF, 2018) y cifra el contenido de la comunicación, de modo que ni las credenciales ni los datos transmitidos puedan ser leídos por un tercero durante el tránsito por la red. Conviene precisar dónde termina ese cifrado cuando el servicio se publica mediante un túnel, como se describe en el apartado 11.1. En el modelo habitual, el certificado TLS termina en el servidor que aloja la aplicación; con un túnel, el cifrado TLS termina en el borde del proveedor del túnel, y el tramo entre ese borde y el servidor local viaja por el canal cifrado que el propio túnel establece. Esto significa que el proveedor del túnel actúa como extremo de confianza de la conexión HTTPS y, por tanto, debe tratarse como un tercero que puede inspeccionar el tráfico que no está cifrado de extremo a extremo. Esta circunstancia no invalida la protección de las comunicaciones, pero obliga a tener presente que la confianza no termina en el servidor de la aplicación, sino en el operador del túnel.

En cuarto lugar, dado que se trata de una aplicación web accesible desde Internet, la seguridad de la aplicación se fundamenta en el catálogo de riesgos de aplicaciones web de OWASP. OWASP es una organización internacional sin ánimo de lucro dedicada a la seguridad del software, y su publicación más conocida, el Top 10, reúne los diez riesgos de seguridad más críticos que afectan a las aplicaciones web, como la inyección de código, la exposición de datos sensibles y la falsificación de peticiones entre sitios (OWASP, 2021). Este catálogo guía las medidas de protección implementadas en el sistema, que incluyen el cifrado de contraseñas, la gestión segura de sesiones, la protección frente a la falsificación de peticiones entre sitios, la limitación de peticiones y el establecimiento de cabeceras de seguridad.

En quinto lugar, el código del sistema se desarrolla íntegramente en Python y debe seguir la guía de estilo PEP 8. Una guía de estilo reúne convenciones sobre nombres de variables, sangrado, longitud de las líneas y organización de las importaciones para mantener un código coherente y legible. PEP 8 es un Python Enhancement Proposal, un documento de la comunidad Python redactado originalmente por Guido van Rossum, Barry Warsaw y Nick Coghlan y publicado por la Python Software Foundation (van Rossum, Warsaw, & Coghlan, 2001; Python Software Foundation, 2024). En este proyecto, el cumplimiento se revisa mediante herramientas de análisis estático.

Por último, la documentación del proyecto se rige por la normativa de Trabajo Fin de Grado de la Escuela Politécnica Superior de la Universidad Pablo de Olavide, que establece la estructura, el formato y los requisitos formales que deben cumplir los documentos entregables asociados a este TFG (Universidad Pablo de Olavide, 2014). La Tabla resume los estándares y normativas aplicables al sistema.

Estándar / Norma	Ámbito de aplicación
RGPD (Reglamento UE 2016/679)	Protección de datos y tratamiento de información biomédica.
LOPDGDD (Ley Orgánica 3/2018)	Adaptación del RGPD al ordenamiento jurídico español.
MDR (Reglamento UE 2017/745)	Requisitos de los productos sanitarios destinados a un fin médico.
Reglamento de IA (Reglamento UE 2024/1689)	Clasificación y obligaciones de los sistemas de inteligencia artificial.
HTTPS / TLS	Protocolo seguro de comunicación entre cliente y servidor.
OWASP Top 10	Seguridad de las aplicaciones web.
PEP 8	Guía de estilo y nomenclatura del código Python.
Normativa TFG EPS-UPO	Estructura y formato de la documentación del proyecto.

Tabla x - Estándares y normativas aplicables al sistema

11.4. Perfiles de usuario del sistema

Identificar correctamente a los usuarios que van a utilizar el sistema es una tarea fundamental dentro del análisis, porque determina desde qué perspectivas se diseña el sistema y qué funcionalidades debe ofrecer a cada tipo de actor. En este apartado conviene distinguir dos planos complementarios que con frecuencia se confunden. El primero es el análisis de los perfiles de usuario, que caracteriza a los colectivos que usarán la plataforma por sus necesidades y por su actividad. El segundo es el modelo de autorización, que define los roles de acceso mediante los que se controla qué puede hacer cada cuenta. Ambos planos se describen a continuación: los perfiles de usuario, en primer lugar, y los roles de acceso, a continuación.

El plan de proyecto identifica dos colectivos de usuarios objetivo a los que se dirige vitalXAI. El primero lo forman los facultativos e investigadores clínicos, profesionales que podrían incorporar el diagnóstico asistido a su rutina médica e investigadora. Sus necesidades sirven como referencia para definir el alcance del sistema: obtener un diagnóstico claro sobre la radiografía, comprender los motivos que lo sustentan mediante los mapas de explicabilidad, descargar un informe y disponer de un laboratorio para configurar, comparar y evaluar modelos sin necesidad de escribir código. El segundo colectivo es el Laboratorio Synergia, cuyas líneas de investigación son afines a los objetivos de vitalXAI y representan un posible entorno de adopción. Ninguno de estos colectivos participó en sesiones de uso o evaluaciones durante el proyecto, por lo que la facilidad de uso y la claridad de la interfaz se mantienen como requisitos de diseño, no como propiedades validadas empíricamente.

Una decisión de diseño relevante es que la plataforma no restringe el acceso en función del perfil profesional: cualquier usuario con una cuenta puede utilizar tanto la interfaz de diagnóstico como el laboratorio de entrenamiento. Esta decisión simplifica el modelo de acceso, pero implica que cualquier facultativo o investigador autenticado puede lanzar entrenamientos que consumen la GPU del servidor durante horas y compiten por la cola de trabajos. Un uso intensivo del laboratorio puede afectar a la experiencia del resto de los usuarios. La ejecución asíncrona y la cola persistente de trabajos permiten que las peticiones no queden bloqueadas (OBJ-010), aunque no eliminan la competencia por los recursos computacionales. Una saturación prolongada de la GPU puede degradar el tiempo de respuesta de los diagnósticos procesados simultáneamente. La limitación de entrenamientos concurrentes o la priorización de trabajos queda como posible evolución.

Sobre los perfiles anteriores se define el modelo de autorización, que materializa el control de acceso mediante tres roles. El usuario visitante accede a la aplicación sin autenticarse y solo puede utilizar las funciones públicas de registro, inicio de sesión y cambio de idioma. El usuario autenticado agrupa a los profesionales con cuenta registrada y puede utilizar la funcionalidad no administrativa de diagnóstico, historial, laboratorio MLOps y resultados. El usuario administrador es un usuario autenticado con capacidades adicionales de gobierno sobre las cuentas y la actividad de la plataforma. Ninguno de estos roles exige conocimientos técnicos específicos, de acuerdo con el requisito RNF-013.

La siguiente tabla resume los roles de acceso identificados, con su descripción, el perfil técnico exigido y su nivel de acceso.

Tipo de usuario	Descripción	Perfil técnico	Nivel de acceso
Usuario visitante	Persona que accede a la plataforma sin autenticarse.	No se exigen conocimientos técnicos.	Registro, inicio de sesión y cambio de idioma.
Usuario autenticado	Profesional con cuenta (clínica o investigadora) que utiliza el sistema.	No se exigen conocimientos técnicos.	Acceso completo a la funcionalidad no administrativa: diagnóstico, historial, laboratorio MLOps y resultados.
Usuario administrador	Usuario autenticado con capacidades de gobierno sobre la plataforma.	No se exigen conocimientos técnicos.	Gestión de usuarios y supervisión de la actividad.

Tabla x - Roles de acceso del sistema

Con la definición de los objetivos del sistema, el contexto tecnológico, la normativa aplicable y los perfiles de usuario, queda delimitado el ámbito del sistema. Sobre este marco, los capítulos siguientes del análisis descomponen el sistema en subsistemas, traducen las necesidades de los usuarios en requisitos y casos de uso, y definen el plan de pruebas que verificará el cumplimiento de lo especificado.

Capítulo 12. Especificación de requisitos del sistema

Los requisitos traducen las necesidades del proyecto a condiciones que pueden comprobarse en la solución construida (Wieggers & Beatty, 2013). Para que cumplan esa función, deben expresar una intención sin ambigüedades, permitir una comprobación objetiva y conservar el vínculo con el objetivo o necesidad que los origina. Estas propiedades permiten utilizar el catálogo como referencia común entre el análisis, el diseño y la validación, en lugar de limitarlo a una declaración general sobre el sistema (Jacobson, Booch, & Rumbaugh, 1999; Larman, 2004). Este capítulo recoge las especificaciones de vitalXAI a partir de los objetivos definidos en el capítulo de metas y propósitos y de los casos de uso identificados durante el análisis, manteniendo la trazabilidad entre necesidad, caso de uso y requisito.

12.1. *Obtención y organización de los requisitos*

Los requisitos del sistema se han obtenido de tres fuentes complementarias. La primera son los objetivos específicos del capítulo 2, que descomponen el objetivo general en metas concretas y verificables y actúan como punto de anclaje de cada requisito. La segunda son los casos de uso recogidos en el análisis, que describen las interacciones de los actores con el sistema y permiten identificar las capacidades concretas que deben implementarse. La tercera son las reuniones mantenidas con el tutor y con los asesores, que aportan el criterio académico y de dominio necesario para matizar el alcance de cada capacidad y para validar la coherencia de los requisitos con la realidad clínica y técnica del proyecto.

A partir de estas fuentes, los requisitos se han clasificado en dos grandes grupos. Los requisitos funcionales delimitan las capacidades que el usuario debe poder utilizar, sin anticipar la solución técnica. Los no funcionales fijan condiciones de calidad, seguridad, rendimiento o cumplimiento que deben respetarse al ofrecer esas capacidades (Wieggers & Beatty, 2013). En este catálogo, cada requisito se identifica mediante un código, RF-XXX para los funcionales y RNF-XXX para los no funcionales, y se relaciona con los objetivos y casos de uso correspondientes para facilitar su seguimiento durante el desarrollo.

12.1.1. Requisitos funcionales del sistema

Los requisitos funcionales describen las capacidades y funcionalidades que el sistema debe ofrecer al usuario para cumplir con los objetivos definidos en el capítulo de metas y propósitos. Cada requisito funcional especifica una acción concreta, definida desde el punto de vista del usuario y con total independencia de cómo se implemente posteriormente; en consecuencia, los requisitos se formulan sin referirse a tecnologías o mecanismos de construcción concretos, que quedan reservados para la parte de diseño de la memoria.

Los requisitos funcionales se organizan en seis módulos que corresponden a las áreas funcionales del sistema: autenticación y cuenta, diagnóstico clínico, historial de consultas, laboratorio MLOps, administración y capacidades transversales. Cada módulo agrupa requisitos y casos de uso de un mismo ámbito y sirve como referencia estable para ordenar el análisis posterior. El capítulo 13 conserva esta organización mediante los subsistemas SS-001 a SS-006, sin introducir una división adicional ni decisiones de diseño. En esta sección se detalla el primero de ellos, el módulo de autenticación y cuenta, que constituye la puerta de acceso al resto de la funcionalidad del sistema.

12.1.1.1. Módulo de autenticación y cuenta

Este módulo agrupa los requisitos relacionados con el control de acceso al sistema. Su propósito es garantizar que únicamente los usuarios registrados y autenticados puedan acceder a las funcionalidades privadas de la plataforma, y que cada usuario pueda operar únicamente con sus propios datos. La autenticación es un requisito transversal: ninguna de las funcionalidades de diagnóstico, historial, laboratorio o administración puede utilizarse sin completar previamente este proceso. Los requisitos de este módulo se corresponden con los casos de uso CU-001 a CU-004 y dan respuesta al objetivo específico de diseñar e implementar el control de acceso del sistema.

RF-001 - Registro de usuario

La plataforma está concebida para ser utilizada por múltiples profesionales sanitarios e investigadores, y cada uno de ellos debe disponer de un espacio de trabajo propio y aislado. Para que ese modelo de uso sea posible, el sistema debe ofrecer un proceso de registro que permita a cualquier persona crear una cuenta. El registro es el punto de entrada de todos los usuarios del sistema: sin una cuenta, no existe forma de acceder a las funcionalidades privadas de la plataforma. Es especialmente relevante que las contraseñas se almacenen de forma segura, de modo que, ante un acceso no autorizado a la base de datos, las credenciales de los usuarios no queden expuestas. El requisito RF-001 recoge este comportamiento.

Campo	Contenido
ID	RF-001
Nombre	Registro de usuario
Objetivos relacionados	OBJ-007
Descripción	El sistema debe permitir a cualquier usuario no autenticado crear una cuenta proporcionando nombre de usuario, nombre, apellidos, correo electrónico y contraseña. El sistema debe validar que los datos proporcionados son correctos y que el usuario no existe previamente, y debe almacenar la contraseña de forma segura, de modo que nunca quede en texto plano.
Importancia	Alta
Estado	Aprobado
Comentarios	El mecanismo concreto de cifrado de la contraseña es una decisión de diseño; el requisito garantiza que la contraseña nunca se almacena en claro en la base de datos.

Tabla x - RF-001 - Registro de usuario

RF-002 - Inicio de sesión

Una vez que el usuario dispone de una cuenta, el siguiente paso es poder acceder al sistema. El inicio de sesión es la acción mediante la cual el sistema verifica la identidad del usuario a partir de sus credenciales y, si estas son correctas, le otorga acceso a las áreas privadas de la plataforma. Se trata del punto de entrada a toda la funcionalidad privada: ninguna operación de diagnóstico, historial o laboratorio puede realizarse sin haber superado este proceso. El sistema debe validar las credenciales contra las almacenadas en la base de datos y, únicamente si la validación es correcta, iniciar una sesión segura para el usuario. Esta es la primera línea de defensa del sistema frente a accesos no autorizados, por lo que su correcto funcionamiento es condición indispensable para el resto de las funcionalidades. El mecanismo concreto de gestión de la sesión, emisión y validación de tokens o cookies, se deja al diseño. El requisito RF-002 especifica este comportamiento.

Campo	Contenido
ID	RF-002
Nombre	Inicio de sesión
Objetivos relacionados	OBJ-007
Descripción	El sistema debe permitir al usuario registrado iniciar sesión mediante sus credenciales. Tras validar que las credenciales son correctas, el sistema debe otorgar al usuario acceso a las áreas privadas de la plataforma.
Importancia	Alta
Estado	Aprobado
Comentarios	El mecanismo concreto de gestión de la sesión y la protección frente a intentos de acceso reiterados se definen en el diseño.

Tabla x - RF-002 - Inicio de sesión

RF-003 - Cierre de sesión

El proceso de acceso no estaría completo sin la posibilidad de cerrar la sesión. El cierre de sesión permite al usuario finalizar su sesión de forma segura, de modo que, cuando comparte un equipo o trabaja desde un dispositivo público, ningún intento posterior de acceder a las áreas privadas tenga éxito sin volver a autenticarse. Este requisito es necesario para dotar al sistema de seguridad en entornos de uso compartido y constituye una de las operaciones básicas de cualquier aplicación con control de acceso. El requisito RF-003 recoge este comportamiento.

Campo	Contenido
ID	RF-003
Nombre	Cierre de sesión
Objetivos relacionados	OBJ-007
Descripción	El sistema debe permitir al usuario autenticado cerrar su sesión de forma segura, de modo que los intentos posteriores de acceder a las áreas privadas de la plataforma sean rechazados hasta un nuevo inicio de sesión.
Importancia	Alta
Estado	Aprobado
Comentarios	Tras el cierre de sesión, cualquier intento de acceso a las funcionalidades privadas debe ser rechazado por el sistema.

Tabla x - RF-003 - Cierre de sesión

RF-004 - Cambio de idioma de la interfaz

La plataforma está dirigida a un público sanitario e investigador que puede proceder de entornos lingüísticos muy diversos, y uno de sus objetivos es ampliar la accesibilidad de la herramienta. Para ello, la interfaz debe poder presentarse en varios idiomas y el usuario debe poder cambiar de idioma en cualquier momento, sin necesidad de reconfigurar nada. Este cambio debe aplicarse de forma dinámica en toda la interfaz y, cuando proceda, también en los informes generados y en el asistente conversacional, de modo que la experiencia de uso resulte coherente en el idioma elegido. El requisito RF-004 recoge este comportamiento y está disponible tanto para visitantes como para usuarios autenticados.

Campo	Contenido
ID	RF-004
Nombre	Cambio de idioma de la interfaz
Objetivos relacionados	OBJ-009
Descripción	El sistema debe permitir a cualquier usuario, esté o no autenticado, cambiar el idioma de la interfaz entre los disponibles (español, inglés, chino e hindú). El cambio debe aplicarse de forma dinámica en toda la interfaz y, cuando proceda, en los informes generados y en el asistente conversacional.
Importancia	Media
Estado	Aprobado
Comentarios	Este requisito responde al objetivo de incorporar internacionalización a la plataforma y aplica tanto a visitantes como a usuarios autenticados.

Tabla x - RF-004 - Cambio de idioma de la interfaz

RF-005 - Aislamiento de datos entre usuarios

El sistema gestiona información de carácter clínico, aunque anonimizada, y cada usuario desarrolla su actividad sobre sus propios datos: sus consultas de diagnóstico, sus sesiones de entrenamiento y sus resultados. Garantizar que ningún usuario autenticado pueda acceder a los datos de otro en el uso ordinario de la plataforma es un requisito de seguridad y de privacidad de primer orden, y de especial relevancia en un ámbito donde los datos tratados pueden ser sensibles. Este aislamiento no debe ser una propiedad coyuntural de una u otra funcionalidad, sino un principio transversal que se garantice en todas las operaciones de acceso a datos: tanto en la consulta del historial como en la gestión de las sesiones de entrenamiento o en la descarga de informes, el sistema debe verificar siempre que el recurso solicitado pertenece al usuario que lo solicita. De este modo, el aislamiento se convierte en una salvaguarda aplicada de forma sistemática en toda la plataforma. El acceso del administrador a los datos de otros usuarios, limitado a sus funciones de supervisión y definido en el módulo de administración (RF-033 a RF-035), constituye la única excepción a este principio. El requisito RF-005 recoge el aislamiento entre usuarios.

Campo	Contenido
ID	RF-005
Nombre	Aislamiento de datos entre usuarios
Objetivos relacionados	OBJ-007
Descripción	El sistema debe garantizar que, en el uso ordinario de la plataforma, cada usuario autenticado solo puede acceder a sus propias consultas de diagnóstico, sesiones de entrenamiento y resultados. El acceso del administrador a los datos de otros usuarios, limitado a sus funciones de supervisión y regulado por el módulo de administración (RF-033 a RF-035), constituye la única excepción a este aislamiento.
Importancia	Alta
Estado	Aprobado
Comentarios	Este requisito es transversal a todos los módulos del sistema y debe estar garantizado en todas las operaciones que impliquen el acceso a datos por parte de los usuarios autenticados. La excepción administrativa queda acotada al rol de administrador y se justifica por la función de supervisión de la plataforma.

Tabla x - RF-005 - Aislamiento de datos entre usuarios

RF-006 - Roles y control de acceso

La plataforma distingue dos formas de participación: la del usuario que utiliza las funcionalidades de diagnóstico y de laboratorio, y la del administrador que gobierna el sistema. La experiencia de acceso es la misma para todos los usuarios autenticados (cualquier usuario con cuenta puede utilizar tanto la interfaz de diagnóstico como el laboratorio, sin restricción por perfil profesional), pero las funcionalidades de administración deben quedar reservadas para el rol de administrador, que gestiona las cuentas y supervisa la actividad registrada. Esta distinción entre el rol de usuario y el rol de administrador debe materializarse en el mecanismo de autorización del sistema. El requisito RF-006 recoge este comportamiento.

Campo	Contenido
ID	RF-006
Nombre	Roles y control de acceso
Objetivos relacionados	OBJ-007, OBJ-011
Descripción	El sistema debe contemplar dos roles de usuario: el usuario autenticado, con acceso a toda la funcionalidad no administrativa de la plataforma, y el usuario administrador, con acceso adicional a las funcionalidades de administración, que permanecen restringidas para el resto de los usuarios.
Importancia	Media
Estado	Aprobado
Comentarios	La existencia del rol de administrador responde al objetivo de administración de la plataforma. La distinción entre ambos roles se materializa en el mecanismo de autorización del sistema.

Tabla x - RF-006 - Roles y control de acceso

12.1.1.2. Módulo de diagnóstico clínico

Este módulo agrupa los requisitos de la interfaz clínica, el primer núcleo funcional del sistema. A través de ella, el usuario autenticado puede realizar un diagnóstico asistido de neumonía a partir de una radiografía de tórax: cargar la imagen, seleccionar la arquitectura con la que desea realizar la consulta, obtener un diagnóstico con su nivel de confianza y visualizar los mapas de explicabilidad que lo justifican. El diagnóstico se procesa de forma asíncrona, de modo que la interfaz permanece operativa mientras el sistema analiza la imagen. Los requisitos de este módulo se corresponden con los casos de uso CU-005 a CU-010 y se asocian al objetivo de diagnóstico asistido con inteligencia artificial explicable (OBJ-001).

RF-007 - Acceso al panel de diagnóstico

Toda la funcionalidad de diagnóstico se organiza en torno a un panel que el usuario autenticado utiliza como punto de partida de sus consultas. Este panel reúne, en una única vista, la carga de la radiografía, la selección del modelo y el acceso al historial, de modo que el profesional dispone de un entorno de trabajo claro y ordenado, sin necesidad de navegar entre secciones. El requisito RF-007 recoge la necesidad de que este panel exista y sea accesible para todo usuario autenticado.

Campo	Contenido
ID	RF-007
Nombre	Acceso al panel de diagnóstico
Objetivos relacionados	OBJ-001
Descripción	La plataforma debe proporcionar un panel operativo para el usuario autenticado que centralice la carga de imágenes radiológicas, la elección del modelo y la supervisión del historial de sus consultas clínicas.
Importancia	Alta
Estado	Aprobado
Comentarios	Dicha interfaz se sitúa como el eje fundamental para el despliegue de las capacidades clínicas dentro de la solución tecnológica.

Tabla x - RF-007 - Acceso al panel de diagnóstico

RF-008 - Subida de una radiografía de tórax

El primer paso de cualquier diagnóstico es incorporar la imagen al sistema. El facultativo debe poder seleccionar una radiografía de tórax desde su equipo, y el sistema debe validarla antes de aceptarla: el formato debe ser un tipo de imagen admitido (JPEG o PNG) y el tamaño no debe superar los 10 MB. Esta validación evita que un archivo incorrecto o excesivamente grande llegue al motor de inferencia, donde no podría procesarse y degradaría el flujo de trabajo del profesional. La imagen se conserva en el servidor asociada a la consulta, de modo que pueda recuperarse desde el historial del usuario. El requisito RF-008 recoge este comportamiento.

Campo	Contenido
ID	RF-008
Nombre	Subida de una radiografía de tórax
Objetivos relacionados	OBJ-001
Descripción	El sistema debe permitir al usuario autenticado subir una radiografía de tórax desde su equipo, validando que el formato sea JPEG o PNG y que su tamaño no supere los 10 MB, y conservarla asociada a la consulta para que pueda recuperarse desde el historial.
Importancia	Alta
Estado	Aprobado
Comentarios	La validación del formato y del tamaño se realiza antes de encolar la consulta. El límite de 10 MB es un valor único del catálogo y debe mantenerse coherente con el caso de uso CU-006.

Tabla x - RF-008 - Subida de una radiografía de tórax

RF-009 - Selección de la arquitectura para el diagnóstico

El sistema dispone de varias arquitecturas de deep learning entrenadas, y el resultado del diagnóstico depende del modelo empleado. El usuario debe poder elegir, entre las arquitecturas disponibles, el modelo con el que desea realizar la consulta, de modo que pueda comparar el comportamiento de distintos modelos sobre una misma imagen. El requisito RF-009 recoge esta capacidad de selección.

Campo	Contenido
ID	RF-009
Nombre	Selección de la arquitectura para el diagnóstico
Objetivos relacionados	OBJ-001
Descripción	El sistema debe permitir al usuario autenticado seleccionar, entre las arquitecturas de deep learning disponibles, el modelo con el que desea realizar el diagnóstico.
Importancia	Media
Estado	Aprobado
Comentarios	La lista de arquitecturas disponibles se muestra en el panel de diagnóstico.

Tabla x - RF-009 - Selección de la arquitectura para el diagnóstico

RF-010 - Solicitud de un diagnóstico

Con la imagen cargada y el modelo seleccionado, el usuario solicita el diagnóstico. Esta petición es el punto de entrada del flujo clínico: el sistema debe aceptar la solicitud, encolarla y procesarla en segundo plano, de modo que la interfaz permanezca operativa mientras el trabajo se ejecuta. Una vez finalizado el procesamiento, la consulta queda registrada en el historial del usuario con su resultado. La presentación de la predicción y de su confianza se recoge en el requisito RF-011, la generación de los mapas de explicabilidad en el RF-012 y la ejecución asíncrona de tareas de larga duración es una capacidad transversal del sistema (OBJ-010). El requisito RF-010 recoge exclusivamente la solicitud y el encolado del diagnóstico.

Campo	Contenido
ID	RF-010
Nombre	Solicitud de un diagnóstico
Objetivos relacionados	OBJ-001, OBJ-010
Descripción	El sistema debe permitir al usuario autenticado solicitar un diagnóstico sobre la imagen cargada y con la arquitectura seleccionada, encolando la consulta para su procesamiento en segundo plano y registrándola en el historial cuando finalice.
Importancia	Alta
Estado	Aprobado
Comentarios	La generación de los mapas de explicabilidad se recoge en el requisito RF-012 y la ejecución asíncrona es una capacidad transversal (OBJ-010), de modo que cada capacidad es verificable de forma independiente.

Tabla x - RF-010 - Solicitud de un diagnóstico

RF-011 - Visualización del resultado del diagnóstico

Cuando la consulta finaliza, el sistema debe presentar al usuario el resultado del diagnóstico: la predicción (PNEUMONIA o NORMAL), el nivel de confianza asociado y el modelo empleado. Esta información se muestra de forma clara y sin tecnicismos, de modo que el profesional pueda interpretarla de inmediato y decidir si confía en ella. La presentación del nivel de confianza es especialmente relevante, porque permite al facultativo calibrar la fiabilidad de la predicción antes de incorporarla a su valoración clínica. La consulta queda registrada en el historial con su resultado. El requisito RF-011 recoge esta presentación del resultado.

Campo	Contenido
ID	RF-011
Nombre	Visualización del resultado del diagnóstico
Objetivos relacionados	OBJ-001
Descripción	El sistema debe mostrar al usuario el resultado de cada consulta de diagnóstico: la predicción, el nivel de confianza y el modelo empleado, de forma clara y comprensible.
Importancia	Alta
Estado	Aprobado
Comentarios	La consulta queda registrada en el historial con su resultado.

Tabla x - RF-011 - Visualización del resultado del diagnóstico

RF-012 - Visualización de los mapas de explicabilidad

Un diagnóstico asistido debe acompañarse de información que permita inspeccionar la decisión del modelo. Por ello, cada consulta incluye mapas de explicabilidad: Saliency Maps, que resaltan la influencia estimada de los píxeles; SmoothGrad, que reduce el ruido de los mapas de saliencia; y Grad-CAM, que localiza activaciones relevantes para la clase predicha (Simonyan, Vedaldi, & Zisserman, 2014; Smilkov & al., 2017; Selvaraju & al., 2017). Para las arquitecturas Transformer se emplean mapas de atención, cuya interpretación se apoya en el análisis de estos modelos (Chefer, Gur, & Wolf, 2021). Estas visualizaciones sirven como apoyo para inspeccionar si el modelo atiende a regiones pulmonares plausibles, pero no demuestran por sí solas la validez clínica del diagnóstico. El requisito RF-012 recoge esta visualización.

Campo	Contenido
ID	RF-012
Nombre	Visualización de los mapas de explicabilidad
Objetivos relacionados	OBJ-001
Descripción	El sistema debe mostrar, junto al resultado de cada consulta, los mapas de explicabilidad que justifican la predicción, superpuestos sobre la radiografía original: Saliency Maps, SmoothGrad y Grad-CAM para las arquitecturas convolucionales, y mapas de atención para las arquitecturas Transformer.
Importancia	Alta
Estado	Aprobado
Comentarios	La generación de las explicaciones se apoya en las técnicas XAI descritas en el capítulo 1, y su evaluación cuantitativa se recoge en el laboratorio (RF-022).

Tabla x - RF-012 - Visualización de los mapas de explicabilidad

12.1.1.3. Módulo de historial de consultas

Este módulo agrupa los requisitos relacionados con la gestión del historial de consultas de diagnóstico. Un facultativo realiza numerosas consultas a lo largo del tiempo, y cada una de ellas debe quedar registrada para poder ser consultada, revisada o corregida posteriormente. El sistema conserva el historial de cada usuario de forma aislada, de modo que cada profesional gestione únicamente sus propios registros. Los requisitos de este módulo se corresponden con los casos de uso CU-011 a CU-014 y se asocian al objetivo de gestión del historial de consultas (OBJ-002).

RF-013 - Consultar el historial de consultas

El profesional necesita recuperar en cualquier momento sus consultas anteriores para revisar un diagnóstico, comparar la evolución de un caso o reutilizar una imagen. El sistema debe mostrar el listado de sus consultas, con los datos esenciales de cada una, fecha, modelo empleado, resultado y confianza, de modo que pueda localizarlas con rapidez. El requisito RF-013 recoge este comportamiento.

Campo	Contenido
ID	RF-013
Nombre	Consultar el historial de consultas
Objetivos relacionados	OBJ-002, OBJ-008
Descripción	El sistema debe permitir al usuario autenticado consultar el listado de sus consultas de diagnóstico, mostrando para cada una la fecha, el modelo empleado, el resultado y el nivel de confianza.
Importancia	Media
Estado	Aprobado
Comentarios	Solo se muestran las consultas del propio usuario, en cumplimiento del aislamiento de datos entre usuarios.

Tabla x - RF-013 - Consultar el historial de consultas

RF-014 - Visualización del detalle de una consulta del historial

Además del listado, el profesional debe poder abrir el detalle completo de una consulta concreta para recuperar la imagen original, el resultado, la confianza y los mapas de explicabilidad asociados. Este detalle permite revisar un diagnóstico en profundidad o utilizarlo como referencia para una nueva consulta, por lo que constituye una operación habitual en el uso clínico de la plataforma. El requisito RF-014 recoge esta visualización del detalle.

Campo	Contenido
ID	RF-014
Nombre	Visualización del detalle de una consulta del historial
Objetivos relacionados	OBJ-002, OBJ-008
Descripción	El sistema debe permitir al usuario autenticado visualizar el detalle de una consulta de su historial, incluyendo la imagen original, el resultado, el nivel de confianza y los mapas de explicabilidad.
Importancia	Media
Estado	Aprobado
Comentarios	El acceso al detalle está restringido a las consultas del propio usuario.

Tabla x - RF-014 - Visualización del detalle de una consulta del historial

RF-015 - Renombrar una consulta del historial

El profesional puede querer dar a sus consultas un nombre más descriptivo para identificarlas mejor, por ejemplo cuando un mismo paciente tiene varias placas. El sistema debe permitir modificar el nombre de una consulta propia. El requisito RF-015 recoge esta operación.

Campo	Contenido
ID	RF-015
Nombre	Renombrar una consulta del historial
Objetivos relacionados	OBJ-002
Descripción	El sistema debe permitir al usuario autenticado modificar el nombre de una de sus consultas del historial.
Importancia	Baja
Estado	Aprobado
Comentarios	El nuevo nombre no puede estar vacío y solo aplica a consultas del propio usuario.

Tabla x - RF-015 - Renombrar una consulta del historial

RF-016 - Eliminar una consulta del historial

El profesional debe poder depurar su historial, eliminando las consultas que ya no necesita del almacenamiento operativo. El registro deja de aparecer en el listado y no queda accesible para el administrador en el ejercicio ordinario de su supervisión. El alcance de la eliminación sobre copias de seguridad y registros externos debe definirse mediante una política de retención. El requisito RF-016 recoge esta operación.

Campo	Contenido
ID	RF-016
Nombre	Eliminar una consulta del historial
Objetivos relacionados	OBJ-002
Descripción	El sistema debe permitir al usuario autenticado eliminar una de sus consultas del historial de forma definitiva, de modo que el registro y sus artefactos asociados dejen de estar disponibles tanto para el propio usuario como para el administrador.
Importancia	Baja
Estado	Aprobado
Comentarios	El sistema debe permitir al usuario autenticado eliminar una de sus consultas del historial de forma definitiva, de modo que el registro y sus artefactos asociados dejen de estar disponibles tanto para el propio usuario como para el administrador.

Tabla x - RF-016 - Eliminar una consulta del historial

12.1.1.4. Módulo de laboratorio MLOps

Este módulo agrupa los requisitos del laboratorio de entrenamiento MLOps, el segundo núcleo funcional del sistema. A través de él, el usuario investigador puede configurar y lanzar experimentos de entrenamiento mediante un asistente conversacional, monitorizar su progreso y consultar los resultados comparativos y estadísticos. El laboratorio orquesta automáticamente la secuencia de entrenamiento, análisis de explicabilidad, comparación estadística y validación externa, de modo que el usuario no necesita escribir código en ningún momento. Los requisitos de este módulo se corresponden con los casos de uso CU-015 a CU-030 y se asocian a los objetivos de laboratorio de entrenamiento MLOps (OBJ-004), evaluación rigurosa de los modelos (OBJ-005) y ejecución asíncrona de tareas (OBJ-010).

RF-017 - Acceso al laboratorio de entrenamiento

El laboratorio de entrenamiento es el espacio desde el que el usuario lanza y consulta sus experimentos. Al igual que el panel de diagnóstico para la actividad clínica, el laboratorio debe ser accesible para todo usuario autenticado y presentar, en una única vista, el asistente conversacional, las sesiones de entrenamiento y el acceso a los resultados. Esta accesibilidad es condición necesaria para que el resto de los requisitos de este módulo tengan sentido. El requisito RF-017 recoge la necesidad de que el laboratorio exista y sea accesible.

Campo	Contenido
ID	RF-017
Nombre	Acceso al laboratorio de entrenamiento
Objetivos relacionados	OBJ-004
Descripción	El sistema debe permitir al usuario autenticado acceder al laboratorio de entrenamiento, desde el que pueda conversar con el asistente, lanzar experimentos y consultar sus sesiones.
Importancia	Alta
Estado	Aprobado
Comentarios	El laboratorio es accesible para todo usuario autenticado, con independencia de su perfil profesional.

Tabla x - RF-017 - Acceso al laboratorio de entrenamiento

RF-018 - Conversación con el asistente para configurar un experimento

El laboratorio se maneja mediante un asistente conversacional que permite al usuario definir un experimento en lenguaje natural, sin escribir código. El usuario debe poder comunicarse con el asistente indicando qué arquitecturas quiere entrenar y con qué hiperparámetros, y el asistente debe interpretar esas indicaciones y traducirlas a la configuración técnica del experimento. Para ello, el asistente debe conocer los parámetros que definen el experimento, las arquitecturas a entrenar, el número de épocas, el tamaño de lote y la tasa de aprendizaje, y, cuando disponga de todos ellos, devolver la configuración estructurada para que el usuario pueda revisarla y confirmarla. Si falta algún parámetro, el asistente debe solicitarlo antes de completar la configuración. La ruta del dataset no forma parte de la conversación: se selecciona de forma validada mediante el requisito RF-019 y se incorpora a la configuración del experimento. La conversación debe desarrollarse en el idioma seleccionado por el usuario. El requisito RF-018 recoge este comportamiento.

Campo	Contenido
ID	RF-018
Nombre	Conversación con el asistente para configurar un experimento
Objetivos relacionados	OBJ-004
Descripción	El sistema debe permitir al usuario configurar un experimento de entrenamiento mediante un asistente conversacional en lenguaje natural, que interprete los parámetros del experimento (arquitecturas, épocas, tamaño de lote y tasa de aprendizaje) y devuelva la configuración estructurada para su revisión, incorporando la ruta del dataset seleccionada conforme al RF-019.
Importancia	Alta
Estado	Aprobado
Comentarios	El asistente debe completar los parámetros faltantes preguntando al usuario antes de devolver la configuración. La ruta del dataset se obtiene mediante RF-019.

Tabla x - RF-018 - Conversación con el asistente para configurar un experimento

RF-019 - Selección de la carpeta del dataset

Antes de lanzar un experimento, el sistema debe disponer de la ruta del dataset sobre el que se entrenarán los modelos. La selección no debe exponer el sistema de ficheros completo del servidor ni permitir el acceso a rutas arbitrarias. El requisito RF-019 prevé confinar la selección a un directorio de datasets permitido y aplicar una validación de la ruta. La misma operación cubre la selección del dataset externo empleado en la validación externa. Esta restricción de ruta queda pendiente de implementación en el prototipo actual.

Campo	Contenido
ID	RF-019
Nombre	Selección de la carpeta del dataset
Objetivos relacionados	OBJ-004
Descripción	El sistema debe permitir al usuario seleccionar el dataset de entrenamiento y el dataset externo de validación dentro de un directorio raíz de datasets permitido, validando que la ruta resultante permanece dentro de ese directorio y sin exponer el resto del sistema de ficheros del servidor.
Importancia	Media
Estado	Aprobado
Comentarios	La selección se confina al directorio de datasets permitido para impedir el acceso a rutas arbitrarias del servidor y respetar el aislamiento entre usuarios (RF-005).

Tabla x - RF-019 - Selección de la carpeta del dataset

RF-020 - Lanzamiento de un experimento de entrenamiento

Con la configuración definida, el usuario lanza el experimento. El sistema debe crear una sesión de entrenamiento y encolar la ejecución del entrenamiento de las arquitecturas solicitadas en segundo plano, de modo que la interfaz permanezca operativa y el usuario pueda monitorizar el progreso y continuar utilizando la plataforma. La metodología experimental concreta del entrenamiento (particiones de validación cruzada, métricas y evaluación estadística) corresponde al diseño del pipeline y sus resultados se consultan mediante los requisitos del laboratorio (RF-022 a RF-025); no forma parte de este requisito. El requisito RF-020 recoge únicamente el lanzamiento del experimento y su encolado asíncrono.

Campo	Contenido
ID	RF-020
Nombre	Lanzamiento de un experimento de entrenamiento
Objetivos relacionados	OBJ-004, OBJ-010, OBJ-006
Descripción	El sistema debe permitir al usuario lanzar un experimento de entrenamiento con la configuración definida, creando la sesión y encolando la ejecución de forma asíncrona, de modo que la interfaz permanezca operativa durante el entrenamiento.
Importancia	Alta
Estado	Aprobado
Comentarios	El análisis de explicabilidad y la comparación estadística se consultan mediante los requisitos RF-023 a RF-025; su ejecución automática tras el entrenamiento es una decisión del diseño del pipeline. La creación de la sesión con su configuración completa sustenta el objetivo de reproducibilidad (OBJ-006).

Tabla x - RF-020 - Lanzamiento de un experimento de entrenamiento

RF-021 - Consulta de las sesiones de entrenamiento

El usuario puede lanzar varios experimentos a lo largo del tiempo, y cada uno queda registrado como una sesión de entrenamiento. El sistema debe mostrar el listado de sus sesiones, con su estado y sus modelos, de modo que el usuario pueda localizarlas y consultar sus resultados. El requisito RF-021 recoge este listado.

Campo	Contenido
ID	RF-021
Nombre	Consulta de las sesiones de entrenamiento
Objetivos relacionados	OBJ-004, OBJ-008
Descripción	El sistema debe permitir al usuario autenticado consultar el listado de sus sesiones de entrenamiento con su estado y sus modelos.
Importancia	Media
Estado	Aprobado
Comentarios	Solo se muestran las sesiones del propio usuario.

Tabla x - RF-021 - Consulta de las sesiones de entrenamiento

RF-022 - Consulta de los resultados de un modelo de la sesión

Cada sesión produce, para cada modelo entrenado, un conjunto de resultados cuantitativos que permiten evaluar su rendimiento y su fiabilidad. El sistema debe mostrar, para un modelo concreto, las métricas de la validación cruzada (exactitud, precisión, sensibilidad, F1 y AUC), las métricas cuantitativas de explicabilidad (Deletion AUC, Insertion AUC, Sparsity, Entropy y Stability SSIM) y las métricas de calibración (Brier Score y Expected Calibration Error). La interpretación de las métricas XAI se apoya en el marco de métodos de explicabilidad descrito por Linardatos, Papastefanopoulos y Kotsiantis (2021), y la de la similitud estructural en Wang et al. (2004); las métricas de calibración se fundamentan en Guo et al. (2017). El sistema conserva además la media y la desviación de las métricas sobre los pliegues. La desagregación por pliegue es un formato de presentación que corresponde al diseño y debe ser coherente con el modelo de resultados persistido de la sesión. El requisito RF-022 recoge esta consulta.

Campo	Contenido
ID	RF-022
Nombre	Consulta de los resultados de un modelo de la sesión
Objetivos relacionados	OBJ-005
Descripción	El sistema debe mostrar al usuario, para un modelo de la sesión, las métricas de la validación cruzada (exactitud, precisión, sensibilidad, F1 y AUC), las métricas cuantitativas de explicabilidad y las métricas de calibración, con su media y su desviación sobre los pliegues.
Importancia	Alta
Estado	Aprobado
Comentarios	La presentación de los resultados debe ser coherente con el modelo de resultados persistido de la sesión; la desagregación por pliegue es una decisión del diseño de presentación.

Tabla x - RF-022 - Consulta de los resultados de un modelo de la sesión

RF-023 - Visualización de los mapas de explicabilidad de un modelo

Además de las métricas numéricas, el laboratorio debe permitir la inspección visual de las explicaciones de cada modelo. El sistema debe mostrar la galería de mapas de calor generados por el análisis XAI cualitativo sobre imágenes de ejemplo, lo que permite comprobar si el modelo se fija en las regiones pulmonares relevantes. El requisito RF-023 recoge esta visualización.

Campo	Contenido
ID	RF-023
Nombre	Visualización de los mapas de explicabilidad de un modelo
Objetivos relacionados	OBJ-005
Descripción	El sistema debe permitir al usuario visualizar la galería de mapas de explicabilidad generados por el análisis XAI cualitativo de un modelo de la sesión.
Importancia	Media
Estado	Aprobado
Comentarios	Los mapas se generan sobre imágenes de ejemplo del dataset de entrenamiento.

Tabla x - RF-023 - Visualización de los mapas de explicabilidad de un modelo

RF-024 - Consulta del ranking de modelos de la sesión

Una vez completada la comparación estadística, el sistema debe presentar el ranking global de los modelos de la sesión, ordenados por su AUC medio de la validación cruzada. Este ranking permite identificar de un vistazo las arquitecturas con mejor rendimiento dentro de la sesión. El requisito RF-024 recoge esta consulta.

Campo	Contenido
ID	RF-024
Nombre	Consulta del ranking de modelos de la sesión
Objetivos relacionados	OBJ-005
Descripción	El sistema debe mostrar el ranking global de los modelos de la sesión, ordenados por su AUC medio de la validación cruzada, junto con su desviación típica.
Importancia	Media
Estado	Aprobado
Comentarios	El ranking se regenera al ejecutar la comparación estadística.

Tabla x - RF-024 - Consulta del ranking de modelos de la sesión

RF-025 - Consulta de la comparativa estadística de la sesión

Determinar si las diferencias entre modelos son compatibles con la variabilidad de los datos es una exigencia metodológica del proyecto. El sistema debe mostrar la matriz de significación estadística que compara los modelos de la sesión, con los p-valores del test de Wilcoxon (Wilcoxon, 1945) sobre el AUC de los pliegues y, cuando la validación externa se ha ejecutado, la matriz del test de DeLong (DeLong, DeLong, & Clarke-Pearson, 1988) sobre las curvas ROC. El AUC resume el comportamiento de la curva ROC sin fijar un único umbral de decisión (Fawcett, 2006). Estas matrices sirven para identificar diferencias compatibles o no con la variabilidad observada, pero no prueban por sí solas superioridad clínica ni causalidad. El requisito RF-025 recoge esta consulta.

Campo	Contenido
ID	RF-025
Nombre	Consulta de la comparativa estadística de la sesión
Objetivos relacionados	OBJ-005
Descripción	El sistema debe mostrar la matriz de significación estadística de la sesión, con los p-valores del test de Wilcoxon y, cuando proceda, los del test de DeLong sobre las curvas ROC de la validación externa.
Importancia	Alta
Estado	Aprobado
Comentarios	La interpretación de la significación debe tener en cuenta la potencia limitada de la prueba con el número de pliegues disponible, cuestión que se analiza en la parte experimental de la memoria; este requisito no fija un umbral de significación.

Tabla x - RF-025 - Consulta de la comparativa estadística de la sesión

RF-026 - Solicitud del recálculo de la comparativa estadística

La comparativa estadística de la sesión puede quedar incompleta en dos circunstancias: cuando un entrenamiento se interrumpe o falla a mitad de su ejecución, o cuando se incorporan nuevos resultados a una sesión ya finalizada. En esas circunstancias, el usuario debe poder solicitar la regeneración de la comparativa estadística y del ranking. El sistema debe recalcular el ranking y el test de Wilcoxon en segundo plano y notificar al usuario cuando el proceso finalice. El requisito RF-026 recoge esta operación de recuperación.

Campo	Contenido
ID	RF-026
Nombre	Solicitud del recálculo de la comparativa estadística
Objetivos relacionados	OBJ-005
Descripción	El sistema debe permitir al usuario solicitar el recálculo de la comparativa estadística de la sesión, regenerando el ranking y el test de Wilcoxon de forma asíncrona.
Importancia	Media
Estado	Aprobado
Comentarios	El usuario es notificado cuando el recálculo finaliza.

Tabla x - RF-026 - Solicitud del recálculo de la comparativa estadística

RF-027 - Ejecución del análisis de explicabilidad de un modelo

El pipeline automático genera el análisis de explicabilidad al completar el entrenamiento de cada modelo, pero ese análisis puede quedar sin ejecutarse si el entrenamiento se interrumpe o si se incorpora a la sesión un modelo que no llegó a analizarse. En esas circunstancias, el usuario debe poder ejecutar manualmente el análisis XAI cualitativo y cuantitativo del modelo, regenerando sus mapas y sus métricas. El requisito RF-027 recoge esta operación de recuperación.

Campo	Contenido
ID	RF-027
Nombre	Ejecución del análisis de explicabilidad de un modelo
Objetivos relacionados	OBJ-005
Descripción	El sistema debe permitir al usuario ejecutar manualmente el análisis de explicabilidad de un modelo de la sesión, regenerando sus mapas y sus métricas cuantitativas.
Importancia	Media
Estado	Aprobado
Comentarios	La ejecución manual complementa el análisis automático del pipeline.

Tabla x - RF-027 - Ejecución del análisis de explicabilidad de un modelo

RF-028 - Solicitud de la validación externa de la sesión

La validación externa constituye la prueba de generalización de los modelos. El usuario debe poder solicitar la evaluación de los modelos entrenados sobre el dataset externo de pacientes adultos, con los modelos congelados y sin ningún tipo de reaprendizaje. El sistema debe encolar la tarea, evaluar cada modelo sobre la cohorte externa calculando las cinco métricas y las curvas ROC, y aplicar el test de DeLong (DeLong, DeLong, & Clarke-Pearson, 1988) para comparar las curvas entre modelos. Dado su coste computacional, la validación externa se ejecuta de forma asíncrona. El requisito RF-028 recoge este comportamiento.

Campo	Contenido
ID	RF-028
Nombre	Solicitud de la validación externa de la sesión
Objetivos relacionados	OBJ-005, OBJ-010
Descripción	El sistema debe permitir al usuario solicitar la validación externa de la sesión, evaluando los modelos congelados sobre el dataset externo y aplicando el test de DeLong, de forma asíncrona.
Importancia	Alta
Estado	Aprobado
Comentarios	La validación externa se encola y se procesa en segundo plano.

Tabla x - RF-028 - Solicitud de la validación externa de la sesión

RF-029 - Consulta de los resultados de la validación externa

Cuando la validación externa finaliza, el sistema debe presentar sus resultados: las métricas de cada modelo sobre la cohorte externa, las curvas ROC y la matriz de significación del test de DeLong. Estos datos determinan qué arquitecturas generalizan mejor a poblaciones y condiciones de adquisición distintas. El requisito RF-029 recoge esta consulta.

Campo	Contenido
ID	RF-029
Nombre	Consulta de los resultados de la validación externa
Objetivos relacionados	OBJ-005
Descripción	El sistema debe mostrar los resultados de la validación externa de la sesión: métricas sobre la cohorte externa, curvas ROC y matriz de significación del test de DeLong.
Importancia	Media
Estado	Aprobado
Comentarios	Los resultados solo están disponibles tras completar la validación externa.

Tabla x - RF-029 - Consulta de los resultados de la validación externa

RF-030 - Generación del informe PDF de la sesión

El laboratorio debe consolidar los resultados de una sesión en un informe descargable en PDF, que recoge la configuración del experimento, el ranking de modelos, la matriz de Wilcoxon, los resultados de la validación externa con su matriz de DeLong y las métricas de explicabilidad y calibración por modelo. Este informe permite archivar y compartir los resultados de la sesión. El requisito RF-030 recoge la generación del informe.

Campo	Contenido
ID	RF-030
Nombre	Generación del informe PDF de la sesión
Objetivos relacionados	OBJ-004, OBJ-003
Descripción	El sistema debe generar un informe descargable en PDF que consolide la configuración, las métricas, las comparativas estadísticas y las gráficas de la sesión de entrenamiento.
Importancia	Media
Estado	Aprobado
Comentarios	El informe integra ranking, matrices de significación, curvas ROC y métricas XAI.

Tabla x - RF-030 - Generación del informe PDF de la sesión

RF-031 - Renombrar una sesión de entrenamiento

El usuario debe poder dar a sus sesiones de entrenamiento un nombre más descriptivo para identificarlas mejor. El sistema debe permitir modificar el nombre de una sesión propia. El requisito RF-031 recoge esta operación.

Campo	Contenido
ID	RF-031
Nombre	Renombrar una sesión de entrenamiento
Objetivos relacionados	OBJ-004
Descripción	El sistema debe permitir al usuario autenticado modificar el nombre de una de sus sesiones de entrenamiento.
Importancia	Baja
Estado	Aprobado
Comentarios	El nuevo nombre no puede estar vacío y solo aplica a sesiones del propio usuario.

Tabla x - RF-031 - Renombrar una sesión de entrenamiento

RF-032 - Eliminar una sesión de entrenamiento

El usuario debe poder eliminar las sesiones de entrenamiento que ya no necesita. La eliminación retira la sesión y sus resultados del laboratorio del usuario. El requisito RF-032 recoge esta operación.

Campo	Contenido
ID	RF-032
Nombre	Eliminar una sesión de entrenamiento
Objetivos relacionados	OBJ-004
Descripción	El sistema debe permitir al usuario autenticado eliminar una de sus sesiones de entrenamiento y sus resultados asociados.
Importancia	Baja
Estado	Aprobado
Comentarios	Solo se pueden eliminar sesiones del propio usuario.

Tabla x - RF-032 - Eliminar una sesión de entrenamiento

12.1.1.5. Módulo de administración

Este módulo agrupa los requisitos del panel de administración, destinados al administrador de la plataforma. Su propósito es permitir el gobierno del sistema: la gestión y supervisión de las cuentas de usuario y de la actividad registrada. Los requisitos de este módulo se corresponden con los casos de uso CU-031 a CU-033 y se asocian al objetivo de administración de la plataforma (OBJ-011). La gestión de cuentas de usuario (RF-040) completa este módulo. Todas las funcionalidades de este módulo están restringidas al rol de administrador, que constituye el único perfil con acceso a estas operaciones.

RF-033 - Consulta del listado de usuarios

El administrador necesita una visión global de quién utiliza la plataforma. El sistema debe mostrar el listado de los usuarios registrados, de modo que el administrador pueda identificar cuentas y comprobar el estado de la plataforma. Esta consulta constituye la base de la supervisión del sistema y del resto de las operaciones de administración. El requisito RF-033 recoge este listado.

Campo	Contenido
ID	RF-033
Nombre	Consulta del listado de usuarios
Objetivos relacionados	OBJ-011
Descripción	El sistema debe permitir al administrador consultar el listado de usuarios registrados en la plataforma.
Importancia	Media
Estado	Aprobado
Comentarios	Esta funcionalidad está restringida al rol de administrador. El acceso a los datos de otros usuarios queda acotado a la función de supervisión definida como excepción en RF-005 y se registra conforme al requisito de auditoría RNF-006.

Tabla x - RF-033 - Consulta del listado de usuarios

RF-034 - Consulta de las consultas de un usuario

La supervisión de la actividad registrada exige poder examinar la actividad de un usuario concreto. El sistema debe permitir al administrador seleccionar un usuario y consultar su historial de consultas de diagnóstico. Esta capacidad permite al administrador comprobar el uso que se hace de la plataforma y detectar posibles incidencias. El requisito RF-034 recoge esta consulta.

Campo	Contenido
ID	RF-034
Nombre	Consulta de las consultas de un usuario
Objetivos relacionados	OBJ-011
Descripción	El sistema debe permitir al administrador consultar el historial de consultas de diagnóstico de un usuario concreto de la plataforma.
Importancia	Media
Estado	Aprobado
Comentarios	Esta funcionalidad está restringida al rol de administrador. El acceso al historial de otro usuario queda acotado a la función de supervisión definida como excepción en RF-005, se limita a las consultas no eliminadas (RF-016) y se registra conforme al requisito de auditoría RNF-006.

Tabla x - RF-034 - Consulta de las consultas de un usuario

RF-035 - Visualización del detalle de una consulta de un usuario

Para completar la supervisión, el administrador debe poder abrir el detalle completo de una consulta de un usuario, incluyendo la imagen, el resultado, la confianza y los metadatos asociados. Esta operación permite auditar un caso concreto ante cualquier incidencia. El requisito RF-035 recoge esta visualización.

Campo	Contenido
ID	RF-035
Nombre	Visualización del detalle de una consulta de un usuario
Objetivos relacionados	OBJ-011
Descripción	El sistema debe permitir al administrador visualizar el detalle de una consulta de diagnóstico de un usuario de la plataforma.
Importancia	Media
Estado	Aprobado
Comentarios	Esta funcionalidad está restringida al rol de administrador. La visualización de la imagen de una consulta de otro usuario queda acotada a la función de supervisión definida como excepción en RF-005, se limita a las consultas no eliminadas (RF-016) y se registra conforme al requisito de auditoría RNF-006.

Tabla x - RF-035 - Visualización del detalle de una consulta de un usuario

12.1.1.6. Módulo transversal

Este módulo agrupa los requisitos que no pertenecen a un ámbito funcional concreto, sino que afectan a toda la plataforma: la consulta y cancelación de los trabajos de la cola, que cubren los diagnósticos, los entrenamientos y las validaciones externas, y la personalización del tema visual de la interfaz. Los requisitos de este módulo se corresponden con los casos de uso CU-034 a CU-036 y se asocian al objetivo de ejecución asíncrona de tareas (OBJ-010) y, en el caso del tema visual, al objetivo de usabilidad e internacionalización de la interfaz (OBJ-009).

RF-036 - Consulta del estado de la cola de trabajos

El sistema ejecuta de forma asíncrona los diagnósticos, los entrenamientos y las validaciones externas, y el usuario necesita conocer en cada momento el estado de sus trabajos. El sistema debe mostrar un panel de la cola de trabajos que refleje el estado de cada tarea (pendiente, en ejecución, completada o fallida) y que se actualice automáticamente al menos cada cinco segundos, sin exigir que el usuario recargue la página. Esta visibilidad es la que permite al usuario saber cuándo estará disponible un resultado o si un trabajo ha fallado. El requisito RF-036 recoge este panel.

Campo	Contenido
ID	RF-036
Nombre	Consulta del estado de la cola de trabajos
Objetivos relacionados	OBJ-010
Descripción	El sistema debe mostrar al usuario el estado de los trabajos de la cola (diagnósticos, entrenamientos y validaciones externas), actualizado automáticamente al menos cada cinco segundos.
Importancia	Media
Estado	Aprobado
Comentarios	El panel cubre todos los tipos de trabajo y muestra su estado.

Tabla x - RF-036 - Consulta del estado de la cola de trabajos

RF-037 - Cancelación de un trabajo de la cola

En determinadas circunstancias, el usuario puede necesitar cancelar un trabajo que ha encolado por error o que ya no le interesa. El sistema debe permitir cancelar un trabajo pendiente, de modo que no llegue a ejecutarse. La interrupción de un trabajo en ejecución no forma parte de la implementación actual y queda como comportamiento pendiente. El requisito RF-037 recoge esta operación.

Campo	Contenido
ID	RF-037
Nombre	Cancelación de un trabajo de la cola
Objetivos relacionados	OBJ-010
Descripción	El sistema debe permitir al usuario cancelar un trabajo pendiente de la cola. La interrupción de un trabajo en ejecución queda como comportamiento previsto, pero no implementado.
Importancia	Media
Estado	Aprobado
Comentarios	La implementación actual permite cancelar trabajos pendientes. La interrupción de trabajos en ejecución y la gestión de resultados parciales quedan pendientes.

Tabla x - RF-037 - Cancelación de un trabajo pendiente de la cola

RF-038 - Cambio del tema visual de la interfaz

El usuario debe poder personalizar la apariencia de la interfaz eligiendo entre el tema claro y el tema oscuro, según su preferencia visual. El cambio debe aplicarse de forma inmediata en toda la interfaz. El requisito RF-038 recoge esta personalización.

Campo	Contenido
ID	RF-038
Nombre	Cambio del tema visual de la interfaz
Objetivos relacionados	OBJ-009
Descripción	El sistema debe permitir al usuario autenticado alternar entre el tema claro y el tema oscuro de la interfaz.
Importancia	Baja
Estado	Aprobado
Comentarios	La preferencia se aplica de forma inmediata en toda la interfaz.

Tabla x - RF-038 - Cambio del tema visual de la interfaz

RF-039 - Generación del informe PDF del diagnóstico

Cada consulta de diagnóstico debe poder descargarse como un informe en PDF que el profesional pueda archivar, imprimir o incorporar a su flujo de trabajo habitual. El informe debe recoger la identificación de la consulta, la imagen original, la predicción, el nivel de confianza, el modelo empleado y los mapas de explicabilidad. Este requisito cubre el informe individual de la consulta; el informe consolidado de la sesión de entrenamiento se recoge en el requisito RF-030. El requisito RF-039 pertenece al módulo de diagnóstico clínico.

Campo	Contenido
ID	RF-039
Nombre	Generación del informe PDF del diagnóstico
Objetivos relacionados	OBJ-001, OBJ-003
Descripción	El sistema debe generar un informe descargable en PDF para cada consulta de diagnóstico, que recoja la imagen original, la predicción, el nivel de confianza, el modelo empleado y los mapas de explicabilidad.
Importancia	Media
Estado	Aprobado
Comentarios	Este requisito completa la cobertura del objetivo de generación de informes (OBJ-003), junto con el informe de sesión (RF-030). Su verificación se apoya en la prueba PU-017.

Tabla x - RF-039 - Generación del informe PDF del diagnóstico

RF-040 - Gestión de cuentas de usuario por el administrador

La administración de la plataforma no se limita a la consulta: el objetivo OBJ-011 contempla también la gestión de las cuentas de usuario. El administrador debería poder desactivar una cuenta, cambiar el rol de un usuario y eliminar una cuenta, con registro de las operaciones conforme a la auditoría administrativa (RNF-006) y respeto del aislamiento de datos (RF-005). Estas operaciones no están implementadas en el prototipo actual. El requisito RF-040 pertenece al módulo de administración.

Campo	Contenido
ID	RF-040
Nombre	Gestión de cuentas de usuario por el administrador
Objetivos relacionados	OBJ-011
Descripción	El sistema debe permitir al administrador gestionar las cuentas de usuario de la plataforma: desactivar una cuenta, cambiar el rol de un usuario y eliminar una cuenta, con registro de la operación conforme al requisito RNF-006. Estas operaciones quedan pendientes de implementación.
Importancia	Media
Estado	Aprobado, pendiente de implementación
Comentarios	Este requisito completa la cobertura prevista del objetivo de administración. Las operaciones de gestión de cuentas no están implementadas en el prototipo actual.

Tabla x - RF-040 - Gestión de cuentas de usuario por el administrador

RF-041 - Limitación de entrenamientos simultáneos y de la cola de trabajos

El entrenamiento de modelos compite por los recursos computacionales de la estación de trabajo, que cuenta con una única GPU. Para que un uso intensivo del laboratorio por parte de un usuario no degrade el servicio del resto, y para que un lanzamiento accidental no sature la plataforma, el sistema debe limitar el número de entrenamientos que se ejecutan simultáneamente y el número de trabajos de entrenamiento que pueden permanecer encolados. Los entrenamientos que excedan el límite de ejecución concurrente deben esperar en cola y ejecutarse cuando haya recursos disponibles. El requisito RF-041 pertenece al módulo de laboratorio MLOps.

Campo	Contenido
ID	RF-041
Nombre	Limitación de entrenamientos simultáneos y de la cola
Objetivos relacionados	OBJ-004, OBJ-010
Descripción	El sistema debe limitar el número de entrenamientos que se ejecutan simultáneamente en la estación de trabajo y el número de trabajos de entrenamiento encolados, de modo que la competición por la GPU no degrade el servicio para el resto de los usuarios y un lanzamiento masivo no sature la plataforma.
Importancia	Media
Estado	Aprobado
Comentarios	Esta limitación es global a la plataforma y complementa el requisito de disponibilidad del servicio (RNF-017). Los valores concretos de los límites se configuran en el entorno.

Tabla x - RF-041 - Limitación de entrenamientos simultáneos y de la cola de trabajos

12.1.1.7. Matriz de trazabilidad entre requisitos funcionales y objetivos del sistema

La matriz de trazabilidad que se presenta a continuación relaciona cada requisito funcional con el objetivo u objetivos del sistema a los que da respuesta. Su finalidad es permitir una comprobación en ambas direcciones: qué objetivos tienen requisitos asociados y qué objetivo justifica cada requisito funcional. Esta relación sirve para detectar objetivos sin capacidades asociadas y requisitos sin fundamento declarado, pero no demuestra por sí sola que el catálogo sea completo ni que todas las capacidades del sistema estén especificadas. Las columnas representan los objetivos del sistema definidos en el capítulo de ámbito del sistema (OBJ-001 a OBJ-011) y las filas los requisitos funcionales; una X en la intersección indica que el requisito contribuye a la consecución del objetivo correspondiente. Conviene señalar el criterio aplicado a dos requisitos transversales: el aislamiento de datos (RF-005) se asocia al objetivo de acceso seguro (OBJ-007), porque protege a los usuarios entre sí en su uso ordinario; el requisito de roles y control de acceso (RF-006) se asocia además al objetivo de administración (OBJ-011), porque define el rol de administrador sobre el que se sustenta la gestión de la plataforma.

RF	Denominación	OBJ-001	OBJ-002	OBJ-003	OBJ-004	OBJ-005	OBJ-006	OBJ-007	OBJ-008	OBJ-009	OBJ-010	OBJ-011
RF-001	Registro de usuario							X				
RF-002	Inicio de sesión							X				
RF-003	Cierre de sesión							X				
RF-004	Cambio de idioma de la interfaz									X		
RF-005	Aislamiento de datos entre usuarios							X				
RF-006	Roles y control de acceso							X				X
RF-007	Acceso al panel de diagnóstico	X										
RF-008	Subida de una radiografía de tórax	X										
RF-009	Selección de la arquitectura para el diagnóstico	X										
RF-010	Solicitud de un diagnóstico	X									X	

RF	Denominación	OBJ-001	OBJ-002	OBJ-003	OBJ-004	OBJ-005	OBJ-006	OBJ-007	OBJ-008	OBJ-009	OBJ-010	OBJ-011
RF-011	Visualización del resultado del diagnóstico	X										
RF-012	Visualización de los mapas de explicabilidad	X										
RF-013	Consultar el historial de consultas		X						X			
RF-014	Visualización del detalle de una consulta del historial		X						X			
RF-015	Renombrar una consulta del historial		X									
RF-016	Eliminar una consulta del historial		X									
RF-017	Acceso al laboratorio de entrenamiento				X							
RF-018	Conversación con el asistente para configurar un experimento				X							
RF-019	Selección de la carpeta del dataset				X							
RF-020	Lanzamiento de un experimento de entrenamiento				X		X				X	
RF-021	Consulta de las sesiones de entrenamiento				X				X			
RF-022	Consulta de los resultados de un modelo de la sesión					X						
RF-023	Visualización de los mapas de explicabilidad de un modelo					X						

RF	Denominación	OBJ-001	OBJ-002	OBJ-003	OBJ-004	OBJ-005	OBJ-006	OBJ-007	OBJ-008	OBJ-009	OBJ-010	OBJ-011
RF-024	Consulta del ranking de modelos de la sesión					X						
RF-025	Consulta de la comparativa estadística de la sesión					X						
RF-026	Solicitud del recálculo de la comparativa estadística					X						
RF-027	Ejecución del análisis de explicabilidad de un modelo					X						
RF-028	Solicitud de la validación externa de la sesión					X					X	
RF-029	Consulta de los resultados de la validación externa					X						
RF-030	Generación del informe PDF de la sesión			X	X							
RF-031	Renombrar una sesión de entrenamiento				X							
RF-032	Eliminar una sesión de entrenamiento				X							
RF-033	Consulta del listado de usuarios											X
RF-034	Consulta de las consultas de un usuario											X
RF-035	Visualización del detalle de una consulta de un usuario											X

RF	Denominación	OBJ-001	OBJ-002	OBJ-003	OBJ-004	OBJ-005	OBJ-006	OBJ-007	OBJ-008	OBJ-009	OBJ-010	OBJ-011
RF-036	Consulta del estado de la cola de trabajos										X	
RF-037	Cancelación de un trabajo de la cola										X	
RF-038	Cambio del tema visual de la interfaz									X		
RF-039	Generación del informe PDF del diagnóstico	X		X								
RF-040	Gestión de cuentas de usuario por el administrador											X
RF-041	Limitación de entrenamientos simultáneos y de la cola				X						X	

Tabla x - Matriz de trazabilidad entre requisitos funcionales y objetivos del sistema

La matriz muestra que todos los objetivos del sistema tienen al menos un requisito funcional asociado y que cada requisito está justificado por uno o varios objetivos: los objetivos de diagnóstico asistido con inteligencia artificial explicable (OBJ-001), gestión del historial (OBJ-002), laboratorio de entrenamiento (OBJ-004) y evaluación rigurosa de los modelos (OBJ-005) concentran la mayor parte de los requisitos. El objetivo de generación de informes (OBJ-003) se cubre con el informe de sesión (RF-030) y con el informe individual del diagnóstico (RF-039); el objetivo de administración de la plataforma (OBJ-011) se cubre con las operaciones de consulta (RF-033 a RF-035) y con la gestión de cuentas de usuario (RF-040); y el objetivo de ejecución asíncrona de tareas (OBJ-010) se cubre con los requisitos de la cola de trabajos (RF-036, RF-037 y RF-041). Esta cobertura es formal: el progreso del entrenamiento por épocas, implementado y probado mediante PU-023, continúa sin un requisito funcional propio. La verificación de la cobertura entre requisitos y casos de uso se aborda, de forma análoga, en el apartado correspondiente del modelado de casos de uso.

12.1.2. Requisitos no funcionales del sistema

Los requisitos no funcionales describen las condiciones de calidad en las que el sistema debe satisfacer su comportamiento, con independencia de las funcionalidades concretas que ofrece. A diferencia de los requisitos funcionales, que responden a la pregunta de qué debe hacer el sistema, los requisitos no funcionales responden a la pregunta de cómo debe comportarse: qué nivel de seguridad debe garantizar, cómo debe responder en términos de rendimiento, qué experiencia de uso debe ofrecer o cómo debe salvaguardar los datos. Cada requisito no funcional se formula con un umbral o una condición comprobable, de modo que pueda verificarse de forma objetiva; cuando el umbral depende de una configuración, se indica el valor aplicado en este proyecto. Estos requisitos condicionan de forma transversal todo el diseño e influyen de manera decisiva en la aceptación del sistema por parte de sus usuarios finales, especialmente en un ámbito clínico donde la seguridad, la fiabilidad y la privacidad no son negociables.

Los requisitos no funcionales se han obtenido de los objetivos del sistema, de las restricciones del entorno tecnológico descritas en el análisis y de las buenas prácticas recogidas en la literatura y en los planes de apoyo del proyecto. Se organizan en siete grupos: seguridad de la plataforma y del acceso, confidencialidad y protección de los datos, rendimiento y capacidad de respuesta, sencillez de uso y accesibilidad, robustez y disponibilidad del servicio, persistencia y salvaguarda de los datos, y reproducibilidad del sistema. Al final de la sección se incluye, por un lado, una subsección dedicada a los compromisos del proceso de desarrollo y a los entregables del proyecto, que no constituyen requisitos del sistema y se presentan por separado para no mezclar ambas naturalezas, y por otro, la matriz de trazabilidad que permite revisar, de forma análoga a la de los requisitos funcionales, la relación entre los objetivos con implicaciones de calidad y los requisitos no funcionales.

12.1.2.1. Seguridad de la plataforma y del acceso

Este grupo agrupa los requisitos no funcionales relacionados con la protección de la plataforma y de las credenciales de sus usuarios. La seguridad es un requisito transversal de primer orden: un fallo en el tratamiento de las contraseñas, en la gestión de las sesiones o en la protección frente a los ataques web más comunes podría comprometer la confidencialidad de los datos clínicos gestionados por el sistema. Los requisitos de este grupo se asocian a los objetivos de acceso seguro y personalizado al sistema (OBJ-007) y de administración de la plataforma (OBJ-011). El marco de referencia de este grupo es el catálogo de riesgos de aplicaciones web de OWASP (OWASP, 2021), descrito en el análisis, por lo que los requisitos cubren, al menos, los riesgos más críticos: inyección, exposición de datos sensibles, pérdida de control de acceso, configuraciones inseguras, gestión vulnerable de componentes, autenticación fallida y falsificación de peticiones.

RNF-001 - Cifrado de contraseñas

La contraseña es la credencial principal con la que el usuario accede al sistema, y su almacenamiento debe protegerla frente a un acceso no autorizado a la base de datos. El sistema debe almacenarla de forma que no pueda recuperarse en claro. Para ello debe utilizar una función de hash con un coste configurable que eleve el esfuerzo necesario para probar contraseñas de forma masiva, conforme a la práctica de seguridad recogida por OWASP (OWASP, 2021). El requisito RNF-001 recoge esta exigencia.

Campo	Contenido
ID	RNF-001
Nombre	Cifrado de contraseñas
Objetivos relacionados	OBJ-007
Descripción	El sistema debe almacenar las contraseñas de los usuarios mediante una función de hash no reversible con un factor de coste configurable, de modo que ninguna contraseña quede en claro y que la verificación se realice siempre sobre el valor derivado.
Importancia	Alta
Estado	Aprobado
Comentarios	Esta exigencia complementa al requisito funcional de registro de usuario. La elección concreta del algoritmo es una decisión de diseño.

Tabla x - RNF-001 - Cifrado de contraseñas

RNF-002 - Gestión segura de la sesión

Una vez autenticado el usuario, la sesión debe gestionarse de forma que sus credenciales no queden expuestas en cada petición. El sistema debe basar la sesión en credenciales que el navegador no pueda leer desde scripts y que se asocien a un periodo de validez limitado, y debe permitir renovar el acceso sin que el usuario tenga que volver a autenticarse mediante una credencial de refresco de vida limitada y revocable. Esta gestión evita que las credenciales viajen o se almacenen de forma vulnerable. El requisito RNF-002 recoge esta exigencia.

Campo	Contenido
ID	RNF-002
Nombre	Gestión segura de la sesión
Objetivos relacionados	OBJ-007
Descripción	El sistema debe gestionar las sesiones mediante credenciales que el navegador no pueda leer desde scripts, con validez limitada, y debe permitir renovar el acceso sin una nueva autenticación mediante una credencial de refresco de vida limitada y revocable.
Importancia	Alta
Estado	Aprobado
Comentarios	El mecanismo concreto de cookies y de rotación de la credencial de refresco son decisiones de diseño.

Tabla x - RNF-002 - Gestión segura de la sesión

RNF-003 - Protección frente a la falsificación de peticiones entre sitios

Cuando una aplicación utiliza cookies para mantener la sesión, un sitio externo puede intentar inducir al navegador a enviar una petición con esas credenciales. Este riesgo, identificado entre las amenazas habituales de las aplicaciones web (OWASP, 2021), se aborda en el sistema exigiendo un token de protección en las peticiones que modifican el estado. El requisito RNF-003 recoge esta exigencia.

Campo	Contenido
ID	RNF-003
Nombre	Protección frente a la falsificación de peticiones entre sitios
Objetivos relacionados	OBJ-007
Descripción	El sistema debe proteger las peticiones que modifican el estado frente a ataques de falsificación de peticiones entre sitios, exigiendo un token de protección válido en cada petición.
Importancia	Alta
Estado	Aprobado
Comentarios	El mecanismo concreto de generación y verificación del token es una decisión de diseño.

Tabla x - RNF-003 - Protección frente a la falsificación de peticiones entre sitios

RNF-004 - Limitación de las peticiones de acceso

El proceso de inicio de sesión es el principal objetivo de los ataques de fuerza bruta, en los que un atacante prueba contraseñas de forma masiva hasta acertar. El sistema debe limitar el número de intentos de acceso fallidos desde una misma dirección y, al superar el umbral, bloquear nuevas peticiones durante un intervalo de tiempo definido. En este proyecto el umbral es de cinco peticiones fallidas por minuto desde una misma dirección, con un bloqueo de diez minutos. El requisito RNF-004 recoge esta exigencia.

Campo	Contenido
ID	RNF-004
Nombre	Limitación de las peticiones de acceso
Objetivos relacionados	OBJ-007
Descripción	El sistema debe limitar a cinco peticiones fallidas por minuto los intentos de acceso desde una misma dirección y bloquear nuevas peticiones durante diez minutos cuando se supere el umbral.
Importancia	Media
Estado	Aprobado
Comentarios	Se aplica un límite estricto sobre el inicio de sesión y un límite general sobre el resto de las peticiones. Los valores pueden ajustarse mediante configuración, pero los indicados son los aplicados en este proyecto.

Tabla x - RNF-004 - Limitación de las peticiones de acceso

RNF-005 - Cabeceras de seguridad en las respuestas

Además de los mecanismos de autenticación, el sistema debe reforzar la seguridad del navegador mediante cabeceras HTTP de protección. Estas cabeceras restringen las políticas de contenido que el navegador puede cargar, fuerzan el uso de conexiones seguras y bloquean la inclusión de la aplicación en marcos de terceros. Aunque el usuario no las percibe directamente, reducen la superficie de ataque de la plataforma. El requisito RNF-005 recoge esta exigencia.

Campo	Contenido
ID	RNF-005
Nombre	Cabeceras de seguridad en las respuestas
Objetivos relacionados	OBJ-007
Descripción	El sistema debe incluir en sus respuestas cabeceras HTTP de seguridad que restrinjan las políticas de contenido del navegador, fuercen el uso de conexiones seguras y bloqueen la inclusión en marcos de terceros.
Importancia	Media
Estado	Aprobado
Comentarios	El conjunto concreto de cabeceras y sus políticas se define en la configuración del middleware de seguridad, como decisión de diseño.

Tabla x - RNF-005 - Cabeceras de seguridad en las respuestas

RNF-006 - Auditoría de la actividad administrativa

Las operaciones de administración afectan al conjunto de la plataforma, incluida la gestión de usuarios, la supervisión de consultas y la configuración general. El sistema debe registrar la actividad administrativa para dejar constancia de la operación realizada, de quién la ejecutó y del momento en que se produjo. Este registro tendría acceso restringido al administrador. El requisito RNF-006 recoge esta exigencia y permanece pendiente de implementación.

Campo	Contenido
ID	RNF-006
Nombre	Auditoría de la actividad administrativa
Objetivos relacionados	OBJ-007, OBJ-011
Descripción	El sistema debe registrar la actividad de administración, de modo que quede constancia de las operaciones realizadas, del usuario que las ejecutó y del momento en que se produjeron, con acceso restringido al administrador. Este requisito permanece pendiente de implementación.
Importancia	Media
Estado	Aprobado, pendiente de implementación
Comentarios	Requisito aprobado, pendiente de implementación en el prototipo actual. Su incorporación deberá definir el formato, la retención y el acceso a los registros.

Tabla x - RNF-006 - Auditoría de la actividad administrativa

RNF-007 - Validación y saneamiento de las entradas

La inyección de código, en particular la inyección SQL, es el riesgo de seguridad más crítico de las aplicaciones web según el catálogo de OWASP. El sistema debe tratar toda entrada procedente del usuario como no confiable y procesarla mediante consultas parametrizadas que separen la instrucción de sus valores, sin concatenar nunca el contenido recibido en una sentencia SQL. Esta garantía debe aplicarse en todos los puntos en los que el sistema accede a la base de datos. El requisito RNF-007 recoge esta exigencia.

Campo	Contenido
ID	RNF-007
Nombre	Validación y saneamiento de las entradas
Objetivos relacionados	OBJ-007
Descripción	El sistema debe tratar las entradas del usuario como no confiables y procesarlas mediante consultas parametrizadas que eviten la inyección de código, en particular la inyección SQL, en todos los accesos a la base de datos.
Importancia	Alta
Estado	Aprobado
Comentarios	La verificación de este requisito se apoya en las pruebas de seguridad del plan de pruebas y en el análisis estático del código.

Tabla x - RNF-007 - Validación y saneamiento de las entradas

RNF-008 - Subida de ficheros segura

El sistema acepta la subida de imágenes de diagnóstico, por lo que debe validar el contenido real del fichero y no solo su extensión o su tipo declarado. El sistema debe comprobar que el contenido corresponde a una imagen admitida (JPEG o PNG) y rechazar cualquier fichero cuyo contenido no sea el esperado, con independencia de su nombre o de su tipo MIME declarado. El requisito RNF-008 recoge esta exigencia.

Campo	Contenido
ID	RNF-008
Nombre	Subida de ficheros segura
Objetivos relacionados	OBJ-007
Descripción	El sistema debe validar el contenido real de los ficheros de imagen subidos, y no solo su extensión o su tipo MIME declarado, rechazando cualquier fichero cuyo contenido no corresponda a una imagen admitida.
Importancia	Alta
Estado	Aprobado
Comentarios	Complementa al requisito funcional RF-008, que fija el formato y el tamaño máximo de las imágenes.

Tabla x - RNF-008 - Subida de ficheros segura

RNF-009 - Registro de eventos de seguridad

La detección y el análisis de incidentes requieren un registro de los eventos relevantes de seguridad. El sistema debería registrar los intentos de acceso fallidos, los bloqueos por límite de peticiones, las operaciones de administración y los errores de autenticación, junto con la fecha, el origen y, cuando proceda, el usuario implicado. Este registro permanece pendiente de implementación. El requisito RNF-009 recoge esta exigencia.

Campo	Contenido
ID	RNF-009
Nombre	Registro de eventos de seguridad
Objetivos relacionados	OBJ-007, OBJ-011
Descripción	El sistema debe registrar los eventos de seguridad relevantes (accesos fallidos, bloqueos por límite de peticiones y operaciones de administración) con su fecha, su origen y, cuando proceda, el usuario implicado.
Importancia	Media
Estado	Aprobado, pendiente de implementación
Comentarios	El registro de eventos de seguridad complementa al requisito de auditoría de la actividad administrativa (RNF-006).

Tabla x - RNF-009 - Registro de eventos de seguridad

RNF-010 - Política de contraseñas

La fortaleza de las credenciales depende en parte de las contraseñas elegidas por los usuarios. El sistema debe exigir una política mínima de contraseñas en el registro y en el cambio de contraseña: una longitud mínima de ocho caracteres y la comprobación de que no sea idéntica a datos personales básicos del usuario, como el nombre de usuario o el correo electrónico. El requisito RNF-010 recoge esta exigencia.

Campo	Contenido
ID	RNF-010
Nombre	Política de contraseñas
Objetivos relacionados	OBJ-007
Descripción	El sistema debe exigir una contraseña de al menos ocho caracteres en el registro y en el cambio de contraseña, y rechazar las contraseñas idénticas a datos personales básicos del usuario.
Importancia	Media
Estado	Aprobado
Comentarios	La política complementa al requisito de cifrado de contraseñas (RNF-001).

Tabla x - RNF-010 - Política de contraseñas

RNF-011 - Gestión de secretos

El sistema depende de credenciales externas que no deben quedar incrustadas en el código ni en los ficheros versionados, como la clave de la API del asistente conversacional, la clave de firma de las sesiones o las credenciales de acceso a la base de datos. El sistema debe obtener estos secretos de la configuración del entorno y no exponerlos en las respuestas, en los registros ni en la interfaz. El requisito RNF-011 recoge esta exigencia.

Campo	Contenido
ID	RNF-011
Nombre	Gestión de secretos
Objetivos relacionados	OBJ-007
Descripción	El sistema debe obtener las credenciales externas (clave de la API del asistente, clave de firma y credenciales de la base de datos) de la configuración del entorno, sin incrustarlas en el código ni exponerlas en respuestas, registros o interfaz.
Importancia	Alta
Estado	Aprobado
Comentarios	Esta exigencia cubre la clave de la API de Groq y el resto de secretos del entorno.

Tabla x - RNF-011 - Gestión de secretos

12.1.2.2. Confidencialidad y protección de los datos

Este grupo agrupa los requisitos no funcionales relacionados con la privacidad y la protección de los datos que el sistema trata. El sistema maneja información de origen médico, aunque anonimizada, y su tratamiento debe ajustarse a la normativa vigente en materia de protección de datos personales. Los requisitos de este grupo se asocian al objetivo de acceso seguro y personalizado al sistema (OBJ-007) y se apoyan en el análisis de la normativa presentado en el capítulo de ámbito del sistema.

RNF-012 - Cumplimiento del Reglamento General de Protección de Datos

El sistema gestiona cuentas de usuarios registrados y procesa imágenes de origen médico, por lo que el tratamiento de esos datos debe ajustarse al Reglamento General de Protección de Datos y a la Ley Orgánica 3/2018 (Parlamento Europeo y Consejo de la Unión Europea, 2016; España, 2018). En particular, los principios de minimización y limitación de la finalidad condicionan el registro, el almacenamiento y el tratamiento de las imágenes. El sistema debe observar estas obligaciones de forma transversal. El requisito RNF-012 recoge esta exigencia.

Campo	Contenido
ID	RNF-012
Nombre	Cumplimiento del Reglamento General de Protección de Datos
Objetivos relacionados	OBJ-007
Descripción	El sistema debe cumplir la normativa vigente en materia de protección de datos personales, en particular el Reglamento General de Protección de Datos (RGPD) y la Ley Orgánica 3/2018, en el tratamiento de los datos de los usuarios y de las imágenes gestionadas.
Importancia	Alta
Estado	Aprobado
Comentarios	El cumplimiento condiciona el diseño del registro, el almacenamiento y el tratamiento de los datos.

Tabla x - RNF-012 - Cumplimiento del Reglamento General de Protección de Datos

RNF-013 - Anonimización de los conjuntos de datos

Los conjuntos de datos utilizados para el entrenamiento y la validación de los modelos proceden de repositorios públicos y están anonimizados, es decir, desprovistos de cualquier información que permita identificar a los pacientes. El sistema debe operar sobre los conjuntos de datos de entrenamiento y de validación externa empleados en el laboratorio, que son públicos y anónimos. Conviene precisar el alcance de esta garantía: el sistema no puede verificar, por sí mismo, que una imagen subida por un facultativo en el flujo de diagnóstico esté anonimizada, como reconoce el requisito RNF-014; esa verificación recae en el responsable de la captura. Por ello, este requisito se limita a los conjuntos de datos gestionados por el propio sistema. El requisito RNF-013 recoge esta exigencia.

Campo	Contenido
ID	RNF-013
Nombre	Anonimización de los conjuntos de datos
Objetivos relacionados	OBJ-007
Descripción	El sistema debe operar exclusivamente sobre conjuntos de datos de entrenamiento y de validación externa que sean públicos y estén anonimizados. Esta garantía se limita a los conjuntos gestionados por el sistema y no puede extenderse a las imágenes subidas por los usuarios en el diagnóstico.
Importancia	Alta
Estado	Aprobado
Comentarios	La verificación de la anonimización de las imágenes subidas por los facultativos no es realizable por el sistema y recae en el responsable de la captura, como señala el requisito RNF-014.

Tabla x - RNF-013 - Anonimización de los conjuntos de datos

RNF-014 - Exclusión de datos personales identificables

El sistema no debe almacenar ni procesar datos de pacientes individuales identificables. Las radiografías que los facultativos suben para su diagnóstico se conservan asociadas a la consulta del profesional que realizó el diagnóstico, de modo que pueden recuperarse desde el historial (RF-008), sin exponer información del paciente en la interfaz. Es responsabilidad del personal clínico garantizar la anonimización previa de las imágenes que introduce en el sistema, dado que el sistema no dispone de un mecanismo que la verifique. El requisito RNF-014 recoge esta exigencia.

Campo	Contenido
ID	RNF-014
Nombre	Exclusión de datos personales identificables
Objetivos relacionados	OBJ-007
Descripción	El sistema no debe almacenar ni procesar datos de pacientes individuales identificables, y debe asociar los artefactos generados a la cuenta del profesional que realizó la consulta, conservando las imágenes mientras exista la consulta asociada.
Importancia	Alta
Estado	Aprobado
Comentarios	La anonimización previa de las imágenes es responsabilidad del personal clínico, dado que el sistema no puede verificarla. La conservación de las imágenes está vinculada a la consulta y a su recuperación desde el historial, conforme al requisito RF-008.

Tabla x - RNF-014 - Exclusión de datos personales identificables

RNF-015 - Confidencialidad de las comunicaciones

Las comunicaciones entre el navegador y el servidor transportan credenciales de acceso y datos de las consultas, por lo que deben protegerse frente a su interceptación. El sistema debe servir sus páginas mediante un canal de comunicación cifrado basado en TLS (IETF, 2018), de modo que ni las credenciales ni los datos transmitidos puedan ser leídos por un tercero durante el tránsito por la red. Esta protección es una condición de seguridad transversal que afecta a todas las comunicaciones de la plataforma. El requisito RNF-015 recoge esta exigencia.

Campo	Contenido
ID	RNF-015
Nombre	Confidencialidad de las comunicaciones
Objetivos relacionados	OBJ-007
Descripción	El sistema debe proteger las comunicaciones entre el cliente y el servidor mediante un canal de comunicación cifrado, de modo que las credenciales y los datos no viajen en claro.
Importancia	Alta
Estado	Aprobado
Comentarios	El mecanismo concreto de cifrado de las comunicaciones es una decisión de diseño.

Tabla x - RNF-015 - Confidencialidad de las comunicaciones

RNF-016 - Retención y borrado de los datos

Los datos almacenados, en particular las imágenes de origen clínico y sus artefactos, no deben conservarse indefinidamente. El sistema debe disponer de una política de retención que fije el plazo de conservación de las consultas y de sus artefactos, y debe soportar el borrado definitivo de una consulta cuando el usuario lo solicita, conforme al derecho de supresión del RGPD (RF-016). El requisito RNF-016 recoge esta exigencia.

Campo	Contenido
ID	RNF-016
Nombre	Retención y borrado de los datos
Objetivos relacionados	OBJ-007
Descripción	El sistema debe aplicar una política de retención de los datos y de sus artefactos, y debe permitir el borrado definitivo de una consulta y de sus ficheros asociados cuando el usuario lo solicita, en coherencia con el requisito RF-016 y con el derecho de supresión (Parlamento Europeo y Consejo de la Unión Europea, 2016).
Importancia	Alta
Estado	Aprobado, pendiente de implementación
Comentarios	El borrado debe abarcar los registros y los ficheros asociados en el almacenamiento operativo. El alcance sobre copias de seguridad queda pendiente de definir.

Tabla x - RNF-016 - Retención y borrado de los datos

RNF-017 - Cifrado de los datos en reposo

El sistema almacena imágenes de origen clínico y credenciales de usuarios, por lo que los datos persistidos deben estar protegidos ante un acceso no autorizado al almacenamiento. El sistema debe conservar la información sensible cifrada en reposo, de modo que una exposición de los ficheros o de la base de datos no revele directamente su contenido. El requisito RNF-017 recoge esta exigencia.

Campo	Contenido
ID	RNF-017
Nombre	Cifrado de los datos en reposo
Objetivos relacionados	OBJ-007
Descripción	El sistema debe conservar cifrados en reposo los datos sensibles que persiste, incluidas las imágenes de diagnóstico y las credenciales, de modo que una exposición del almacenamiento no revele su contenido.
Importancia	Media
Estado	Aprobado, pendiente de implementación
Comentarios	Requisito aprobado, pendiente de implementación en el prototipo actual. Las contraseñas se cubren mediante RNF-001; este requisito se refiere al resto de datos sensibles persistidos.

Tabla x - RNF-017 - Cifrado de los datos en reposo

RNF-018 - Tratamiento de datos por terceros

El sistema envía al proveedor externo del asistente conversacional los mensajes que el usuario introduce en el laboratorio para configurar un experimento. Aunque el contenido de esas conversaciones se refiere a parámetros técnicos del experimento y no a datos clínicos de pacientes, constituye una transferencia de información del usuario a un tercero. El sistema debe limitar esa transferencia al contenido necesario para la configuración del experimento, no ofrecer campos para introducir datos personales y dejar constancia del tratamiento, de modo que el responsable pueda formalizar, en cada despliegue, las condiciones de protección de datos con el proveedor. El requisito RNF-018 recoge esta exigencia.

Campo	Contenido
ID	RNF-018
Nombre	Tratamiento de datos por terceros
Objetivos relacionados	OBJ-007
Descripción	El sistema debe limitar la transferencia al proveedor externo del asistente conversacional al contenido necesario para la configuración del experimento, sin ofrecer campos para datos personales, y dejar constancia de ese tratamiento para que pueda formalizarse con el proveedor.
Importancia	Media
Estado	Aprobado
Comentarios	El tratamiento por el tercero debe valorarse en cada despliegue, conforme al análisis de la normativa del capítulo 11.

Tabla x - RNF-018 - Tratamiento de datos por terceros

12.1.2.3. Rendimiento y capacidad de respuesta

Este grupo agrupa los requisitos no funcionales relacionados con el rendimiento del sistema y con su capacidad para responder de forma ágil a los usuarios. El rendimiento condiciona directamente la experiencia de uso: un diagnóstico que tarda demasiado, una interfaz que se bloquea durante un entrenamiento o una plataforma que se degrada ante varios usuarios simultáneos comprometerían la utilidad de la herramienta en un entorno clínico real. Los requisitos de este grupo se asocian a los objetivos de diagnóstico asistido con inteligencia artificial explicable (OBJ-001) y de ejecución asíncrona de tareas de larga duración (OBJ-010).

RNF-019 - Tiempo de respuesta de la inferencia

El diagnóstico asistido se realiza en el momento de la consulta, por lo que la inferencia debe completarse en un tiempo acotado. En este proyecto se fija que la inferencia de un diagnóstico con una arquitectura ya cargada debe completarse en menos de quince segundos, y que el envío de la petición debe devolver el identificador del trabajo encolado en menos de dos segundos. La primera consulta con una arquitectura puede requerir un tiempo adicional de carga de los pesos del modelo, que no se computa en el umbral anterior. El requisito RNF-019 recoge esta exigencia.

Campo	Contenido
ID	RNF-019
Nombre	Tiempo de respuesta de la inferencia
Objetivos relacionados	OBJ-001
Descripción	El sistema debe completar la inferencia de un diagnóstico con una arquitectura ya cargada en menos de quince segundos y devolver el identificador del trabajo encolado en menos de dos segundos.
Importancia	Alta
Estado	Aprobado
Comentarios	La primera carga de una arquitectura puede exceder el umbral y no se computa en él. La reutilización de los modelos cargados en memoria es una solución de diseño, no el requisito en sí.

Tabla x - RNF-019 - Tiempo de respuesta de la inferencia

RNF-020 - Ejecución sin bloqueo de la interfaz

Las tareas de larga duración (entrenamientos, análisis de explicabilidad y validación externa) pueden prolongarse durante horas, y durante ese tiempo la plataforma debe seguir respondiendo al usuario. El sistema debe ejecutar estas tareas de forma asíncrona, de modo que la interfaz permanezca operativa y el usuario pueda continuar utilizando la plataforma, consultar el estado de sus trabajos o lanzar nuevas operaciones. Sin esta condición, un único entrenamiento bloquearía el acceso a toda la plataforma durante horas. El requisito RNF-020 recoge esta exigencia.

Campo	Contenido
ID	RNF-020
Nombre	Ejecución sin bloqueo de la interfaz
Objetivos relacionados	OBJ-010
Descripción	El sistema debe ejecutar las tareas de larga duración de forma asíncrona, de modo que la interfaz permanezca operativa durante su ejecución.
Importancia	Alta
Estado	Aprobado
Comentarios	La cola de trabajos permite separar la interfaz de los procesos computacionales, aunque este comportamiento debe verificarse.

Tabla x - RNF-020 - Ejecución sin bloqueo de la interfaz

RNF-021 - Capacidad de acceso concurrente

La plataforma está concebida para ser utilizada por varios profesionales de forma simultánea. En este proyecto se fija que el sistema debe soportar al menos diez usuarios autenticados concurrentes realizando operaciones de consulta y de gestión del historial sin que el tiempo de respuesta de esas operaciones se degrade por encima del doble del tiempo con carga baja, y debe preservar el aislamiento de datos entre usuarios incluso bajo esa carga. El requisito RNF-021 recoge esta exigencia.

Campo	Contenido
ID	RNF-021
Nombre	Capacidad de acceso concurrente
Objetivos relacionados	OBJ-010
Descripción	El sistema debe soportar al menos diez usuarios autenticados concurrentes en operaciones de consulta y gestión del historial sin degradar el tiempo de respuesta por encima del doble del tiempo con carga baja, preservando el aislamiento de datos entre usuarios.
Importancia	Media
Estado	Aprobado
Comentarios	El umbral de diez usuarios debe validarse mediante pruebas de rendimiento. El pool de base de datos tiene un tamaño predeterminado de cinco conexiones, configurable mediante el entorno.

Tabla x - RNF-021 - Capacidad de acceso concurrente

RNF-022 - Comportamiento del sistema con la GPU ocupada

El entrenamiento de modelos compite por la GPU de la estación de trabajo, que es un recurso único. El sistema debe garantizar que, cuando la GPU está ocupada por uno o varios entrenamientos, las peticiones de diagnóstico continúan encolándose y procesándose cuando el recurso queda libre, sin pérdida de trabajos y sin que un entrenamiento monopolice el servicio indefinidamente. La duración máxima de un entrenamiento no se fija como umbral del sistema, sino que se determina por la configuración del experimento y por el límite de trabajos del requisito RF-041. El requisito RNF-022 recoge esta exigencia.

Campo	Contenido
ID	RNF-022
Nombre	Comportamiento del sistema con la GPU ocupada
Objetivos relacionados	OBJ-010
Descripción	El sistema debe encolar y procesar las peticiones de diagnóstico y de entrenamiento cuando la GPU está ocupada, sin pérdida de trabajos y de modo que un entrenamiento no monopolice el recurso indefinidamente.
Importancia	Media
Estado	Aprobado, pendiente de verificación
Comentarios	Complementa al requisito funcional RF-041. El comportamiento debe verificarse sobre el entorno de ejecución del proyecto.

Tabla x - RNF-022 - Comportamiento del sistema con la GPU ocupada

12.1.2.4. Sencillez de uso y accesibilidad

Este grupo agrupa los requisitos no funcionales relacionados con la facilidad de uso de la plataforma y con su accesibilidad para los distintos perfiles de usuarios. vitalXAI está concebido para ser utilizado por profesionales sanitarios sin formación técnica en inteligencia artificial o informática, por lo que la interfaz debe emplear un lenguaje claro y no exigir conocimientos de programación. Los requisitos de este grupo se asocian al objetivo de usabilidad e internacionalización de la interfaz (OBJ-009). En este proyecto, la usabilidad se especifica mediante criterios de diseño y se revisa sobre la interfaz, pero no se valida empíricamente con usuarios reales.

RNF-023 - Interfaz intuitiva para usuarios sin formación técnica

El usuario principal de la plataforma es el profesional sanitario, que no dispone de formación en inteligencia artificial ni en informática. La interfaz debe emplear un lenguaje no técnico, presentar la información de forma clara y guiar al usuario en cada flujo sin exigirle conocer el funcionamiento interno del sistema. La comprobación prevista para este requisito consiste en revisar la interfaz y ejecutar los flujos principales —registro, diagnóstico, consulta del historial y lanzamiento de un experimento— según los criterios definidos. Esta comprobación no permite medir la experiencia de usuarios reales, ya que no se realizaron sesiones con representantes del perfil sanitario. El requisito RNF-023 recoge esta exigencia.

Campo	Contenido
ID	RNF-023
Nombre	Interfaz intuitiva para usuarios sin formación técnica
Objetivos relacionados	OBJ-009
Descripción	El sistema debe permitir que una persona que no participe en el desarrollo complete correctamente, sin asistencia del evaluador, las cuatro tareas principales (registro, diagnóstico, consulta del historial y lanzamiento de un experimento) en el flujo principal. La tasa de éxito exigida será del 100 %.
Importancia	Alta
Estado	Aprobado
Comentarios	La comprobación se realizará mediante una revisión estructurada de la interfaz y la ejecución de los cuatro flujos principales. El resultado no equivale a una evaluación con usuarios reales, que no se realizó durante el proyecto.

Tabla x - RNF-023 - Interfaz intuitiva para usuarios sin formación técnica

RNF-024 - Soporte plurilingüe de la plataforma

La plataforma está dirigida a un público sanitario e investigador que puede proceder de entornos lingüísticos diversos, por lo que debe poder presentarse en varios idiomas. El sistema debe ofrecer la interfaz, los informes generados y el asistente conversacional en los cuatro idiomas de la plataforma (español, inglés, chino e hindi), de modo que el usuario pueda cambiar de idioma de forma dinámica sin reconfigurar nada. El requisito RNF-024 recoge esta exigencia.

Campo	Contenido
ID	RNF-024
Nombre	Soporte plurilingüe de la plataforma
Objetivos relacionados	OBJ-009
Descripción	El sistema debe ofrecer la interfaz, los informes y el asistente conversacional en los cuatro idiomas de la plataforma, permitiendo cambiar de idioma de forma dinámica.
Importancia	Media
Estado	Aprobado
Comentarios	El cambio de idioma se aplica en toda la plataforma y en el asistente conversacional.

Tabla x - RNF-024 - Soporte plurilingüe de la plataforma

RNF-025 - Legibilidad y presentación adaptada a la interfaz web

La interfaz debe presentar la información de forma legible y adaptarse a distintos tamaños de pantalla, de modo que el profesional pueda utilizarla desde su equipo habitual sin pérdida de información. El diseño debe garantizar una jerarquía visual clara en las tablas de resultados y una disposición ordenada de los elementos en los paneles de diagnóstico y de laboratorio. La comprobación se realiza verificando que las vistas principales se adaptan a las resoluciones de pantalla de uso habitual sin pérdida de contenido y sin barras de desplazamiento horizontales no deseadas. El requisito RNF-025 recoge esta exigencia.

Campo	Contenido
ID	RNF-025
Nombre	Legibilidad y presentación adaptada a la interfaz web
Objetivos relacionados	OBJ-009
Descripción	El sistema debe presentar la información de forma legible y ordenada, con jerarquía visual clara, y adaptarse a las resoluciones de referencia de escritorio (1366 × 768 píxeles) y móvil (390 × 844 píxeles) sin pérdida de contenido ni barras de desplazamiento horizontales no deseadas.
Importancia	Media
Estado	Aprobado
Comentarios	La verificación se realiza sobre las vistas principales del sistema en 1366 × 768 píxeles y 390 × 844 píxeles.

Tabla x - RNF-025 - Legibilidad y presentación adaptada a la interfaz web

RNF-026 - Accesibilidad de la interfaz

El sistema debe ser utilizable por personas con discapacidad, conforme a las pautas de accesibilidad para contenido web (WCAG) en su nivel de conformidad AA. La interfaz debe ser navegable por teclado, los elementos interactivos deben tener etiquetas descriptivas que puedan interpretar los lectores de pantalla y los elementos textuales deben mantener un contraste suficiente con su fondo. La comprobación se realiza mediante una revisión de las pautas WCAG sobre las vistas principales del sistema (W3C, 2023). El requisito RNF-026 recoge esta exigencia.

Campo	Contenido
ID	RNF-026
Nombre	Accesibilidad de la interfaz
Objetivos relacionados	OBJ-009
Descripción	El sistema debe cumplir las pautas de accesibilidad WCAG en su nivel AA: navegación por teclado, etiquetas descriptivas interpretables por lectores de pantalla y contraste suficiente entre texto y fondo.
Importancia	Media
Estado	Aprobado
Comentarios	La verificación se realiza mediante revisión de las pautas sobre las vistas principales.

Tabla x - RNF-026 - Accesibilidad de la interfaz

12.1.2.5. Robustez y disponibilidad del servicio

Este grupo agrupa los requisitos no funcionales relacionados con la robustez del sistema y con su disponibilidad continua durante el uso. En un entorno clínico, la plataforma debe permanecer operativa mientras se ejecutan las tareas de larga duración, y los errores aislados de un trabajo no deben comprometer el servicio en su conjunto. Los requisitos de este grupo se asocian al objetivo de ejecución asíncrona de tareas de larga duración (OBJ-010).

RNF-027 - Disponibilidad del servicio durante las tareas de larga duración

El sistema debe permanecer disponible y operativo mientras se ejecutan los entrenamientos y las validaciones externas, que pueden prolongarse durante horas. Un entrenamiento no debe dejar la plataforma inaccesible para el resto de los usuarios: durante su ejecución, el profesional debe poder continuar utilizando la interfaz, realizar diagnósticos o consultar el estado de sus trabajos. El requisito RNF-027 recoge esta exigencia.

Campo	Contenido
ID	RNF-027
Nombre	Disponibilidad del servicio durante las tareas de larga duración
Objetivos relacionados	OBJ-010
Descripción	El sistema debe permanecer disponible y operativo mientras se ejecutan las tareas de larga duración, de modo que un entrenamiento no deje la plataforma inaccesible para el resto de los usuarios.
Importancia	Alta
Estado	Aprobado
Comentarios	La ejecución asíncrona garantiza la disponibilidad continua del servicio.

Tabla x - RNF-027 - Disponibilidad del servicio durante las tareas de larga duración

RNF-028 - Gestión de los errores de los trabajos de la cola

Las tareas que se ejecutan en segundo plano pueden fallar por motivos diversos (una incompatibilidad, un dataset inaccesible o un error en los scripts). El sistema debe gestionar estos fallos de forma ordenada: registrar el error asociado al trabajo, marcarlo como fallido y notificar al usuario, de modo que la plataforma no se bloquee y el usuario pueda conocer la causa del fallo y decidir cómo proceder. Un error aislado en un trabajo no debe interrumpir el resto de los trabajos de la cola. El requisito RNF-028 recoge esta exigencia.

Campo	Contenido
ID	RNF-028
Nombre	Gestión de los errores de los trabajos de la cola
Objetivos relacionados	OBJ-010
Descripción	El sistema debe gestionar los errores de los trabajos de la cola registrando el fallo y marcando el trabajo como fallido, sin interrumpir el resto de los trabajos.
Importancia	Alta
Estado	Aprobado
Comentarios	Un error aislado no debe comprometer la disponibilidad del servicio.

Tabla x - RNF-028 - Gestión de los errores de los trabajos de la cola

RNF-029 - Recuperación de los trabajos tras un reinicio del servidor

Un reinicio del servidor no debe provocar la pérdida de los trabajos que estaban en ejecución. El sistema debe recuperar la cola de trabajos al arrancar, de modo que las tareas que quedaron en ejecución vuelvan a quedar pendientes y puedan procesarse de nuevo. Dado que un trabajo interrumpido a mitad de su ejecución pudo escribir resultados parciales, la recuperación debe ser idempotente: el sistema debe comprobar, antes de reprocesar un trabajo, si ya existen artefactos del trabajo anterior y, en tal caso, sustituirlos de forma consistente para evitar duplicados o artefactos corruptos. El requisito RNF-029 recoge esta exigencia.

Campo	Contenido
ID	RNF-029
Nombre	Recuperación de los trabajos tras un reinicio del servidor
Objetivos relacionados	OBJ-010
Descripción	El sistema debe recuperar la cola de trabajos al arrancar, devolviendo a pendiente los trabajos que quedaron en ejecución, y debe reprocesarlos de forma idempotente, sustituyendo de forma consistente los artefactos parciales del intento anterior para evitar duplicados o resultados corruptos.
Importancia	Media
Estado	Aprobado
Comentarios	La recuperación evita la pérdida de trabajos ante un reinicio; la idempotencia evita que el reprocesamiento genere duplicados o artefactos incompletos.

Tabla x - RNF-029 - Recuperación de los trabajos tras un reinicio del servidor

RNF-030 - Aislamiento de los fallos entre trabajos

Cada trabajo de la cola debe ejecutarse de forma aislada, de modo que un fallo en uno de ellos no afecte al resto. Esta separación es especialmente relevante en el laboratorio de entrenamiento, donde los experimentos se procesan de forma independiente: un error en el entrenamiento de un modelo no debe impedir la ejecución de los demás trabajos encolados. El requisito RNF-030 recoge esta exigencia.

Campo	Contenido
ID	RNF-030
Nombre	Aislamiento de los fallos entre trabajos
Objetivos relacionados	OBJ-010
Descripción	El sistema debe ejecutar los trabajos de la cola de forma aislada, de modo que un fallo en uno de ellos no afecte al resto de los trabajos.
Importancia	Media
Estado	Aprobado
Comentarios	El aislamiento garantiza la robustez del procesamiento asíncrono.

Tabla x - RNF-030 - Aislamiento de los fallos entre trabajos

12.1.2.6. Persistencia y salvaguarda de los datos

Este grupo agrupa los requisitos no funcionales relacionados con la integridad y la salvaguarda de la información persistida. La capa de persistencia almacena las cuentas de usuario, las consultas de diagnóstico con sus artefactos y las configuraciones de los experimentos, y esa información debe conservarse de forma íntegra y recuperable. Los requisitos de este grupo se asocian al objetivo de persistencia y trazabilidad de la información (OBJ-008).

RNF-031 - Integridad y durabilidad de la persistencia

La capa de persistencia es la encargada de almacenar la información que la plataforma genera: las cuentas de usuario, las consultas de diagnóstico con sus artefactos y las configuraciones de los experimentos. Esta información debe conservarse de forma íntegra y duradera: los registros deben mantenerse coherentes entre sí y no corromperse con el paso del tiempo ni ante la ejecución concurrente de operaciones. El sistema debe garantizar la coherencia de las operaciones de escritura y la conservación de los datos a lo largo del ciclo de vida de la plataforma. El requisito RNF-031 recoge esta exigencia.

Campo	Contenido
ID	RNF-031
Nombre	Integridad y durabilidad de la persistencia
Objetivos relacionados	OBJ-008
Descripción	El sistema debe conservar la información almacenada de forma íntegra y duradera, garantizando la coherencia de los registros ante el paso del tiempo y la ejecución concurrente de operaciones.
Importancia	Media
Estado	Aprobado
Comentarios	Aplica a las cuentas de usuario, las consultas de diagnóstico y las configuraciones de los experimentos.

Tabla x - RNF-031 - Integridad y durabilidad de la persistencia

RNF-032 - Copia de seguridad y recuperación de los datos

Los datos almacenados por la plataforma deben poder recuperarse ante un incidente que los ponga en riesgo, como un fallo del hardware, un error de configuración o una corrupción accidental de la base de datos. El sistema debe disponer de un mecanismo de copia de seguridad de la información almacenada y de un procedimiento de recuperación que permita restaurarla. En este proyecto, la copia de seguridad se realiza con una periodicidad diaria e incluye la base de datos y los ficheros de artefactos necesarios para reconstruir una consulta completa (imágenes, mapas de explicación e informes). El objetivo de recuperación se fija en un máximo de veinticuatro horas tras el incidente. El requisito RNF-032 recoge esta exigencia.

Campo	Contenido
ID	RNF-032
Nombre	Copia de seguridad y recuperación de los datos
Objetivos relacionados	OBJ-008
Descripción	El sistema debe disponer de una copia de seguridad diaria que incluya la base de datos y los ficheros de artefactos necesarios para reconstruir una consulta completa, y de un procedimiento de recuperación que permita restaurar los datos en un máximo de veinticuatro horas.
Importancia	Media
Estado	Aprobado, pendiente de implementación
Comentarios	El alcance de la copia incluye los ficheros de imagen, explicación e informe, no solo la base de datos, porque su pérdida impide reconstruir una consulta completa.

Tabla x - RNF-032 - Copia de seguridad y recuperación de los datos

12.1.2.7. Reproducibilidad y sostenibilidad del desarrollo

Este grupo agrupa los requisitos no funcionales relacionados con la reproducibilidad de los experimentos del laboratorio. En el alcance de este proyecto, un resultado se considera reproducible cuando puede volver a obtenerse bajo las condiciones documentadas del experimento y del entorno de ejecución. Los requisitos de este grupo se asocian al objetivo de reproducibilidad de los experimentos (OBJ-006).

RNF-033 - Reproducibilidad de los experimentos

Los experimentos de entrenamiento pueden verse afectados por la inicialización aleatoria de los pesos, el orden de procesamiento de los datos o la concurrencia del hardware, de modo que dos ejecuciones del mismo experimento pueden producir resultados ligeramente distintos. Para que los resultados sean verificables y replicables, el sistema debe fijar las semillas aleatorias de las librerías de aprendizaje automático y de la biblioteca estándar de Python, y almacenar la configuración completa de cada experimento en un archivo asociado a la sesión de entrenamiento. Con esta información, cualquier resultado empírico puede reproducirse bajo las mismas condiciones y puede trazarse su origen. El requisito RNF-033 recoge esta exigencia.

Campo	Contenido
ID	RNF-033
Nombre	Reproducibilidad de los experimentos
Objetivos relacionados	OBJ-006
Descripción	El sistema debe fijar las semillas aleatorias de las librerías de aprendizaje automático y de la biblioteca estándar de Python, y almacenar la configuración completa de cada sesión de entrenamiento, de modo que cualquier resultado pueda reproducirse bajo las mismas condiciones.
Importancia	Alta
Estado	Aprobado
Comentarios	La configuración almacenada permite replicar y trazar cualquier resultado empírico. No se exige determinismo bit a bit entre ejecuciones.

Tabla x - RNF-033 - Reproducibilidad de los experimentos

RNF-034 - Reproducibilidad del entorno de ejecución

Además de la semilla aleatoria, la reproducibilidad depende del entorno de ejecución: una versión distinta de una librería puede alterar los resultados o romper la compatibilidad entre componentes. El sistema debe fijar las versiones exactas de las dependencias y aislar el entorno de ejecución, de modo que el conjunto de librerías pueda reconstruirse de forma reproducible. Esta fijación de versiones se refiere a un entorno con las características de la estación de trabajo descrita en el análisis, incluida la GPU y el toolkit CUDA correspondiente; no se pretende la reproducibilidad en máquinas de características distintas. El requisito RNF-034 recoge esta exigencia.

Campo	Contenido
ID	RNF-034
Nombre	Reproducibilidad del entorno de ejecución
Objetivos relacionados	OBJ-006
Descripción	El sistema debe fijar las versiones exactas de las dependencias y aislar el entorno de ejecución, de modo que el conjunto de librerías pueda reconstruirse de forma reproducible en un entorno con las características de la estación de trabajo, incluida su GPU y su toolkit CUDA.
Importancia	Alta
Estado	Aprobado
Comentarios	La reproducibilidad se garantiza para el entorno concreto de la estación de trabajo, no para máquinas de características distintas.

Tabla x - RNF-034 - Reproducibilidad del entorno de ejecución

12.1.2.8. Compromisos del proceso de desarrollo y entregables del proyecto

RNF-035 - Cumplimiento de estándares de codificación

El código del sistema debe seguir las convenciones de estilo de Python recogidas en PEP 8 (Python Software Foundation, 2024), verificadas mediante herramientas de análisis estático, y sus funciones deben documentarse con docstrings que describan su propósito, sus parámetros y su valor de retorno. Este compromiso facilita la legibilidad y la revisión del código.

Campo	Contenido
ID	RNF-035
Nombre	Cumplimiento de estándares de codificación
Objetivos relacionados	N/A
Descripción	El código del proyecto debe cumplir las convenciones de estilo PEP 8, verificadas con herramientas de análisis estático, y documentar las funciones mediante docstrings.
Importancia	Media
Estado	Aprobado
Comentarios	Compromiso del proceso de desarrollo, no requisito del sistema.

Tabla x - RNF-035 - Cumplimiento de estándares de codificación

RNF-036 - Separación de responsabilidades y modularidad

El código del proyecto debe mantener una separación clara de responsabilidades entre sus componentes: los scripts de entrenamiento y evaluación deben permanecer desacoplados de la capa web, de modo que puedan ejecutarse como procesos independientes y evolucionar por separado. Este compromiso reduce el acoplamiento entre el motor científico y la plataforma.

Campo	Contenido
ID	RNF-036
Nombre	Separación de responsabilidades y modularidad
Objetivos relacionados	N/A
Descripción	El código del proyecto debe mantener separados los módulos de computación científica de la capa web, de modo que puedan ejecutarse de forma independiente y mantenerse de forma aislada.
Importancia	Media
Estado	Aprobado
Comentarios	Compromiso del proceso de desarrollo, no requisito del sistema.

Tabla x - RNF-036 - Separación de responsabilidades y modularidad

RNF-037 - Documentación de la memoria del Trabajo Fin de Grado (entregable)

El proyecto debe documentarse mediante la memoria del Trabajo Fin de Grado, que recoge de forma estructurada el plan de proyecto, el análisis, el diseño, la implementación, las pruebas y las conclusiones del trabajo. La memoria debe cumplir la estructura y el formato exigidos por la normativa universitaria. Este documento es el entregable central del proyecto.

Campo	Contenido
ID	RNF-037
Nombre	Documentación de la memoria del Trabajo Fin de Grado
Objetivos relacionados	N/A
Descripción	El proyecto debe documentarse mediante la memoria del Trabajo Fin de Grado, que cumpla la estructura y el formato exigidos por la normativa universitaria.
Importancia	Alta
Estado	Aprobado
Comentarios	Entregable del proyecto, no requisito del sistema.

Tabla x - RNF-037 - Documentación de la memoria del Trabajo Fin de Grado (entregable)

RNF-038 - Manual de usuario (entregable)

El proyecto debe acompañarse de un manual de usuario orientado a los profesionales sanitarios que no disponen de formación técnica. El manual debe explicar, con un lenguaje no técnico y con capturas de los flujos principales, cómo registrarse, realizar un diagnóstico, interpretar los mapas de explicabilidad y utilizar el laboratorio de entrenamiento.

Campo	Contenido
ID	RNF-038
Nombre	Manual de usuario
Objetivos relacionados	N/A
Descripción	El proyecto debe incluir un manual de usuario en lenguaje no técnico, dirigido a los profesionales sanitarios, que describa paso a paso los flujos principales de la plataforma.
Importancia	Media
Estado	Aprobado
Comentarios	Entregable del proyecto, no requisito del sistema.

Tabla x - RNF-038 - Manual de usuario (entregable)

Conviene señalar que el manual de instalación y configuración, que en una primera versión del catálogo se recogía como requisito, no aparece en la planificación del proyecto: el cronograma contempla la memoria y el manual de usuario como entregables finales, pero no asigna horas a un manual de instalación. Por ello, no se incluye como entregable en esta sección; si se desea incorporar en el futuro, debe planificarse previamente.

12.1.2.9. Matriz de trazabilidad entre requisitos no funcionales y objetivos del sistema

La matriz de trazabilidad que se presenta a continuación relaciona cada requisito no funcional con el objetivo u objetivos del sistema a los que da respuesta, siguiendo el mismo criterio que la matriz de los requisitos funcionales. Esta matriz permite verificar que los objetivos con implicaciones transversales de calidad quedan cubiertos por los requisitos no funcionales correspondientes. Los objetivos de carácter puramente funcional (como la gestión del historial de consultas (OBJ-002) o la generación de informes descargables (OBJ-003)) se materializan mediante requisitos funcionales y no presentan asignaciones en esta matriz. No ocurre lo mismo con los objetivos que sí tienen implicaciones de calidad: el laboratorio de entrenamiento (OBJ-004) y la evaluación rigurosa de los modelos (OBJ-005) se asocian, respectivamente, al requisito de comportamiento del sistema con la GPU ocupada y a los requisitos de reproducibilidad de los experimentos, que condicionan la calidad de sus resultados. Los compromisos del proceso de desarrollo y los entregables (RNF-035 a RNF-038) no se incluyen en la matriz por no ser requisitos del sistema. Las columnas representan los objetivos del sistema y las filas los requisitos no funcionales; una X en la intersección indica que el requisito contribuye a la consecución del objetivo correspondiente.

RNF	Nombre	OBJ-001	OBJ-004	OBJ-005	OBJ-006	OBJ-007	OBJ-008	OBJ-009	OBJ-010	OBJ-011
RNF-001	Cifrado de contraseñas					X				
RNF-002	Gestión segura de la sesión					X				
RNF-003	Protección frente a la falsificación de peticiones entre sitios					X				
RNF-004	Limitación de las peticiones de acceso					X				
RNF-005	Cabeceras de seguridad en las respuestas					X				
RNF-006	Auditoría de la actividad administrativa					X				X
RNF-007	Validación y saneamiento de las entradas					X				
RNF-008	Subida de ficheros segura					X				
RNF-009	Registro de eventos de seguridad					X				X
RNF-010	Política de contraseñas					X				

RNF	Nombre	OBJ -001	OBJ -004	OBJ -005	OBJ -006	OBJ -007	OBJ -008	OBJ -009	OBJ -010	OBJ -011
RNF-011	Gestión de secretos					X				
RNF-012	Cumplimiento del Reglamento General de Protección de Datos					X				
RNF-013	Anonimización de los conjuntos de datos					X				
RNF-014	Exclusión de datos personales identificables					X				
RNF-015	Confidencialidad de las comunicaciones					X				
RNF-016	Retención y borrado de los datos					X				
RNF-017	Cifrado de los datos en reposo					X				
RNF-018	Tratamiento de datos por terceros					X				
RNF-019	Tiempo de respuesta de la inferencia	X								
RNF-020	Ejecución sin bloqueo de la interfaz								X	
RNF-021	Capacidad de acceso concurrente								X	
RNF-022	Comportamiento del sistema con la GPU ocupada				X				X	
RNF-023	Interfaz intuitiva para usuarios sin formación técnica							X		
RNF-024	Soporte plurilingüe de la plataforma							X		
RNF-025	Legibilidad y presentación adaptada a la interfaz web							X		

RNF	Nombre	OBJ-001	OBJ-004	OBJ-005	OBJ-006	OBJ-007	OBJ-008	OBJ-009	OBJ-010	OBJ-011
RNF-026	Accesibilidad de la interfaz							X		
RNF-027	Disponibilidad del servicio durante las tareas de larga duración								X	
RNF-028	Gestión de los errores de los trabajos de la cola								X	
RNF-029	Recuperación de los trabajos tras un reinicio del servidor								X	
RNF-030	Aislamiento de los fallos entre trabajos								X	
RNF-031	Integridad y durabilidad de la persistencia						X			
RNF-032	Copia de seguridad y recuperación de los datos						X			
RNF-033	Reproducibilidad de los experimentos			X	X					
RNF-034	Reproducibilidad del entorno de ejecución			X	X					

Tabla x - Matriz de trazabilidad entre requisitos no funcionales y objetivos del sistema

La matriz muestra la relación entre los objetivos con implicaciones de calidad y los requisitos no funcionales: la seguridad y la confidencialidad se concentran en el objetivo de acceso seguro y personalizado (OBJ-007); la usabilidad, la accesibilidad y el multilingüismo en el de usabilidad e internacionalización (OBJ-009); la disponibilidad y la robustez en el de ejecución asíncrona de tareas (OBJ-010); la integridad y la salvaguarda de los datos en el de persistencia y trazabilidad (OBJ-008); y la reproducibilidad en el de reproducibilidad de los experimentos (OBJ-006), que además se asocia a la evaluación rigurosa de los modelos (OBJ-005), porque la calidad de los resultados estadísticos depende de esas condiciones. El objetivo de diagnóstico asistido (OBJ-001) se complementa con el requisito de rendimiento de la inferencia, y el laboratorio de entrenamiento (OBJ-004) con el comportamiento del sistema cuando la GPU está ocupada. Esta relación no certifica por sí sola que todos los aspectos de calidad estén especificados, implementados o verificados.

12.2. Especificación de los casos de uso

Un caso de uso describe una interacción concreta entre un actor y el sistema, orientada a obtener un resultado que aporta valor a dicho actor (Jacobson, Booch, & Rumbaugh, 1999; Larman, 2004). Para reconocer qué es y qué no es un caso de uso basta con aplicar una sencilla regla práctica: si la acción la inicia el actor y produce un efecto observable para él, es un caso de uso; si se trata de un mecanismo interno que el actor no percibe ni desencadena, no lo es. Así, «el facultativo sube una radiografía y recibe un diagnóstico explicado» es un caso de uso, mientras que «la base de datos persiste la consulta» no lo es, porque constituye un detalle de implementación ajeno a la experiencia del usuario (Cockburn, 2001).

La especificación de los casos de uso ocupa un lugar central en esta memoria porque tiende el puente entre lo que el análisis afirma que el sistema debe hacer y la forma concreta en que cada usuario lo realiza: mientras que los requisitos funcionales definen las capacidades del sistema de forma técnica y verificable, los casos de uso las describen desde la perspectiva de quien las emplea, relatando el flujo de acciones y las respuestas del sistema paso a paso (Wieggers & Beatty, 2013). Sobre esta especificación se apoyan, además, tres actividades posteriores del proyecto: el diseño de los subsistemas, la definición de las pruebas de sistema y la validación final de que la plataforma entrega lo prometido. Este apartado modela los treinta y nueve casos de uso de vitalXAI, agrupados en cinco módulos funcionales que se corresponden con los ámbitos de la plataforma identificados en el análisis: la gestión del acceso y de la cuenta, la interfaz de diagnóstico asistido, el laboratorio de experimentación MLOps, la supervisión y administración de la plataforma y las capacidades transversales. Para cada módulo se presenta, en primer lugar, el diagrama de casos de uso y, a continuación, la especificación detallada de cada caso con sus flujos normal y alternativo y las condiciones previas y posteriores que lo delimitan.

12.2.1. Diagramas de casos de uso por módulo

El diagrama de casos de uso proporciona una visión de conjunto de las interacciones que los actores pueden mantener con la plataforma, representando de forma gráfica a los actores, a los casos de uso y a las relaciones de asociación que los vinculan. En la notación empleada, cada actor se representa mediante una figura acompañada de su nombre, cada caso de uso mediante una elipse con su denominación y su código identificador, y cada asociación mediante una línea continua que une al actor con el caso de uso que inicia. Para facilitar la lectura y la comprensión del conjunto, el diagrama general se ha dividido en cinco diagramas independientes, uno por cada módulo funcional, de modo que cada figura recoge exclusivamente los casos de uso de su ámbito y los actores que participan en ellos. Esta división responde a un criterio de legibilidad: el sistema completo reúne treinta y nueve casos de uso, y su representación conjunta en un único diagrama dificultaría la interpretación sin aportar información adicional.

Respecto a la notación de las relaciones entre casos de uso, conviene recordar el significado de los dos estereotipos empleados en los diagramas. Una relación de extensión, representada con el estereotipo «extend», indica un comportamiento opcional que amplía el flujo de un caso de uso base: el caso de uso que extiende solo se activa cuando el actor decide utilizarlo, y no forma parte del flujo principal. Una relación de inclusión, representada con el estereotipo «include», indica un paso obligatorio que el caso de uso base delega en otro para poder completarse: el caso de uso incluido se ejecuta siempre como parte del flujo del que depende (Jacobson, Booch, & Rumbaugh, 1999; Larman, 2004). En los diagramas de este capítulo las flechas de extensión se dibujan en la dirección de la navegación, es decir, desde el caso de uso base hacia la extensión, de modo que reflejan el orden en que el actor recorre la interfaz; la flecha de inclusión, por el contrario, se dibuja desde el caso de uso que incluye hacia el caso incluido, conforme a la notación estándar.

En vitalXAI la mayoría de los casos de uso representan interacciones directas e independientes del actor con el sistema, pero existen relaciones de extensión allí donde la navegación de la interfaz conduce de una consulta general a un detalle o a una acción opcional sobre ese detalle, por ejemplo, abrir el detalle de una consulta del historial o solicitar el recálculo de una comparativa estadística, y una relación de inclusión en el laboratorio, donde el lanzamiento de un experimento no puede completarse sin la configuración previa definida con el asistente. Estas relaciones se representan en los diagramas mediante flechas discontinuas etiquetadas con el estereotipo correspondiente. Finalmente, cabe aclarar el modelo de actores empleado en las figuras: el Administrador no constituye un perfil aislado, sino un usuario autenticado que conserva todas las capacidades del resto de los perfiles y añade las operaciones de gobierno del sistema (CU-031 a CU-033). Por ello, en los diagramas de los módulos no administrativos el actor se denomina «Usuario autenticado» y representa a todos los perfiles con cuenta, incluido el administrador. A continuación se presentan los diagramas de los cinco módulos, precedidos cada uno de una breve descripción de su propósito y de los casos de uso que agrupa.

12.2.1.1. Gestión del acceso y de la cuenta

Este módulo agrupa los casos de uso relacionados con el control de acceso a la plataforma, el punto de partida de cualquier uso del sistema. Su propósito es doble. Por un lado, garantiza que únicamente las personas registradas y autenticadas puedan acceder a las funcionalidades privadas de la plataforma, de modo que la información clínica y los resultados de los experimentos queden reservados a quienes disponen de una cuenta. Por otro lado, asegura que cada usuario opere exclusivamente con sus propios datos, en cumplimiento del aislamiento de información entre cuentas que exige la plataforma y que constituye uno de los principios de diseño del sistema. La autenticación se configura así como una condición transversal: ninguna de las capacidades de diagnóstico, laboratorio o administración puede utilizarse sin haberla completado, y cualquier intento de acceder a ellas sin una sesión activa debe ser rechazado por el sistema.

El módulo comprende cuatro casos de uso, todos ellos de naturaleza directa. La creación de la cuenta (CU-001) permite al visitante registrarse aportando sus datos personales y una contraseña, que el sistema almacena siempre cifrada; el acceso con credenciales (CU-002) verifica la identidad del usuario y abre la sesión, habilitando el resto de los módulos; el cierre seguro de la sesión (CU-003) revoca el acceso a las áreas privadas cuando el usuario termina su trabajo; y el cambio del idioma de la interfaz (CU-004) permite adaptar el idioma de la plataforma (entre los disponibles, que incluyen el español, el inglés, el chino y el hindi) tanto desde el área pública como desde el área privada, sin perder el estado de navegación.

Conviene aclarar la naturaleza del actor Administrador en este módulo: se trata de un usuario autenticado con las mismas capacidades que el resto de los perfiles, cuyas operaciones específicas de gobierno se recogen en el módulo de supervisión y administración. Por ello, en este diagrama el actor «Usuario autenticado» representa a todos los perfiles con cuenta, incluido el administrador. Al no existir comportamientos opcionales ni pasos obligatorios compartidos, los cuatro casos de uso se representan como interacciones directas e independientes del actor con el sistema, sin relaciones de inclusión o extensión entre sí.

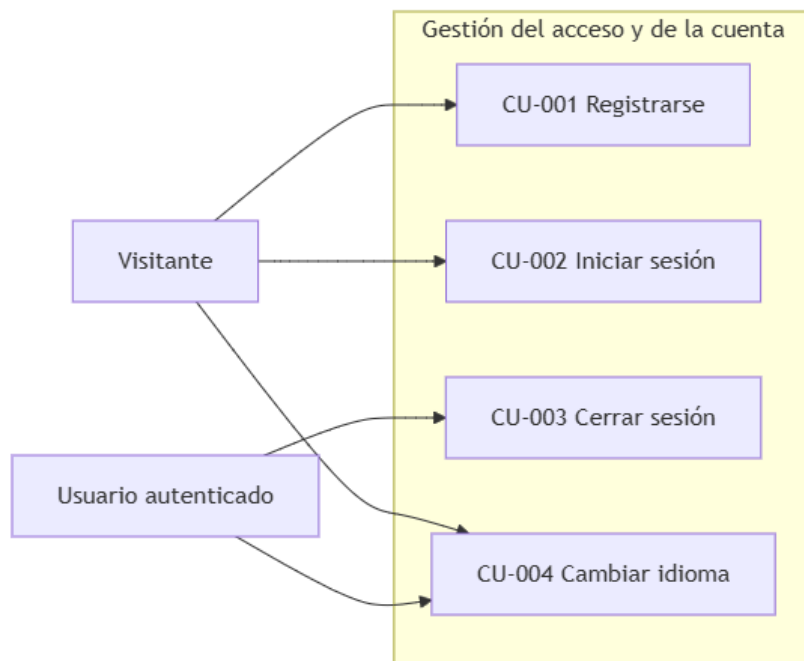


Ilustración x - Casos de uso del módulo de gestión del acceso y de la cuenta

12.2.1.2. Interfaz de diagnóstico asistido

Este módulo agrupa los casos de uso de la interfaz clínica, el primer núcleo funcional del sistema. A través de ella, el usuario autenticado realiza un diagnóstico asistido de neumonía a partir de una radiografía de tórax: carga la imagen, elige la arquitectura con la que desea realizar la consulta, solicita el diagnóstico y obtiene el resultado junto con su nivel de confianza y los mapas de explicabilidad que lo justifican. El diagnóstico se procesa de forma asíncrona mediante la cola de trabajos: en cuanto el usuario envía la petición, el sistema encola el análisis y la interfaz permanece operativa, de modo que el facultativo puede seguir trabajando mientras el modelo procesa la imagen, genera la predicción y produce la explicación visual.

El módulo cubre también la gestión del historial de consultas, que permite al facultativo recuperar en cualquier momento sus diagnósticos anteriores, revisar un resultado, comparar la evolución de un caso o reutilizar una imagen ya analizada. Cada consulta queda registrada de forma aislada en el historial del usuario que la realizó, y solo ese usuario puede consultarla, renombrarla o eliminarla. En total, el módulo reúne once casos de uso: el acceso al panel (CU-005), que presenta en una única vista la carga de la imagen, la selección del modelo y el acceso al historial; la subida de la radiografía (CU-006); la selección de la arquitectura para el diagnóstico (CU-007); la solicitud del diagnóstico (CU-008); la visualización del resultado (CU-009) y de los mapas de explicabilidad (CU-010); la consulta del listado del historial (CU-011); el detalle de una consulta (CU-012); su renombrado (CU-013), su eliminación (CU-014) y la generación del informe PDF de la consulta (CU-037).

En la parte del historial, las relaciones de extensión reflejan el orden en que el actor recorre la interfaz. Desde el listado de consultas (CU-011) se accede, de forma opcional, al detalle de una de ellas (CU-012), donde el usuario recupera la imagen original, el resultado, la confianza y los mapas de explicabilidad asociados; y desde ese detalle pueden ejecutarse igualmente, de forma opcional, el renombrado (CU-013) o la eliminación (CU-014) de la consulta. El resto de los casos de uso del módulo, centrados en la solicitud y visualización del diagnóstico, se representan como interacciones directas e independientes del actor con el sistema.

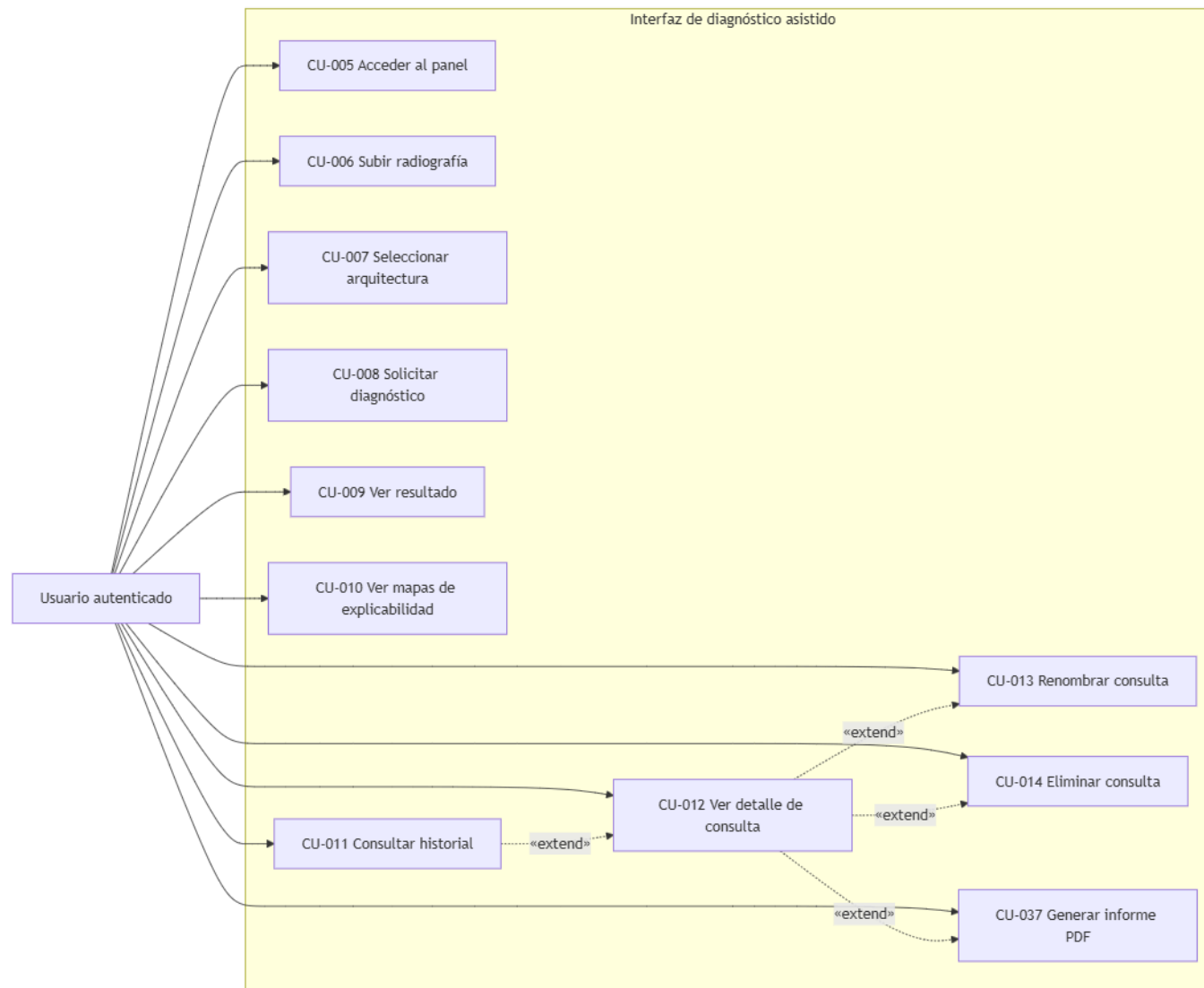


Ilustración x - Casos de uso del módulo de interfaz de diagnóstico asistido

12.2.1.3. Laboratorio de experimentación MLOps

Este módulo agrupa los casos de uso del laboratorio de entrenamiento, el segundo núcleo funcional del sistema. A través de él, el usuario configura y lanza experimentos de entrenamiento mediante un asistente conversacional, monitoriza la ejecución del pipeline y consulta los resultados comparativos y estadísticos de sus sesiones. El laboratorio está concebido para que el investigador no necesite escribir código en ningún momento: el asistente, conectado a un modelo de lenguaje externo mediante una API, interpreta las indicaciones del usuario en lenguaje natural, extrae los parámetros del experimento y rellena la configuración correspondiente.

El laboratorio orquesta automáticamente la secuencia de entrenamiento, análisis de explicabilidad, comparación estadística y validación externa. Una vez lanzado el experimento, el pipeline entrena cada arquitectura seleccionada mediante validación cruzada, genera los mapas de calor de explicabilidad y las métricas cuantitativas asociadas, compara estadísticamente los modelos mediante el ranking y el test de Wilcoxon y, cuando el usuario lo solicita, evalúa los modelos congelados sobre un conjunto de datos externo aplicando el test de DeLong. Este módulo es el más extenso de la plataforma y reúne diecisiete casos de uso: el acceso al laboratorio (CU-015), la conversación con el asistente (CU-016), la selección de la carpeta del dataset (CU-017), el lanzamiento del experimento (CU-018), la consulta de las sesiones (CU-019), los resultados de un modelo (CU-020), los mapas de calor de explicabilidad (CU-021), el ranking de modelos (CU-022), la comparativa estadística (CU-023), el recálculo de la comparativa (CU-024), la ejecución del análisis de explicabilidad (CU-025), la solicitud de la validación externa (CU-026) y la consulta de sus resultados (CU-027), la generación del informe PDF de la sesión (CU-028), el renombrado (CU-029) y eliminación (CU-030) de sesiones, y la comprobación de la limitación de entrenamientos simultáneos y encolados (CU-039).

El lanzamiento del experimento (CU-018) depende de que la configuración del experimento, incluidas las arquitecturas, el número de épocas, el tamaño de lote y la tasa de aprendizaje, esté completa antes de iniciar el entrenamiento. La selección del dataset se realiza mediante CU-017 y ambas operaciones deben completarse antes del lanzamiento.

Las relaciones de extensión, por su parte, siguen la navegación del usuario por el laboratorio. Desde la consulta de las sesiones (CU-019) se accede a los resultados de un modelo concreto (CU-020), al ranking de la sesión (CU-022), a la comparativa estadística (CU-023) o a la generación del informe consolidado (CU-028); desde los resultados de un modelo se alcanzan sus mapas de calor de explicabilidad (CU-021); y desde la comparativa estadística puede solicitarse su recálculo (CU-024) cuando el usuario desea actualizar los resultados. El resto de las operaciones del módulo se representan como interacciones directas e independientes del actor con el sistema.

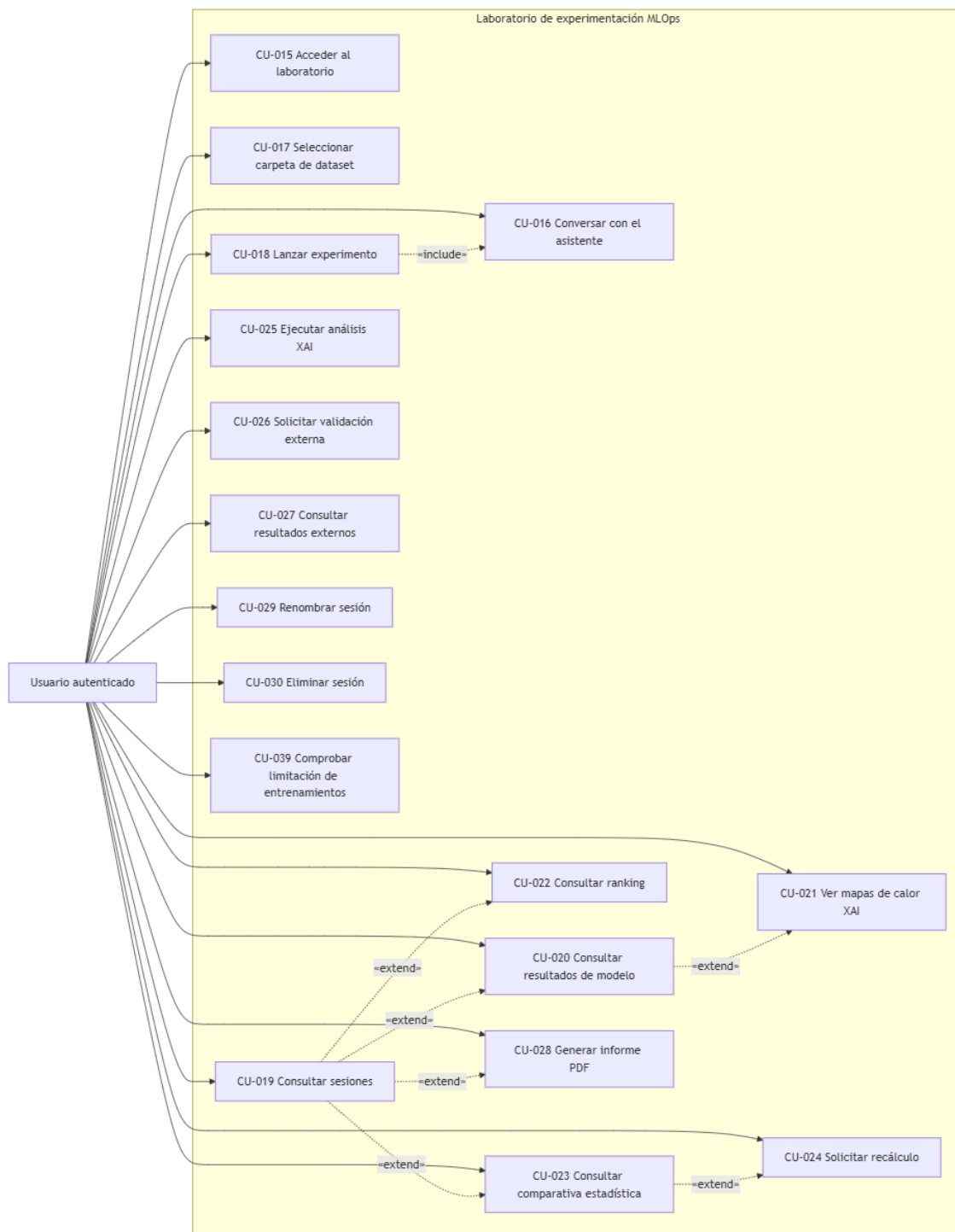


Ilustración x - Casos de uso del módulo de laboratorio de experimentación MLOps

12.2.1.4. Supervisión y administración de la plataforma

La gestión de cuentas de usuario (CU-038) se representa en este módulo junto con los casos de uso CU-031 a CU-033.

Este módulo agrupa los casos de uso del panel de administración, destinados al gobierno de la plataforma: la gestión y supervisión de las cuentas de usuario y de la actividad registrada en el sistema. Su existencia responde a la necesidad de que alguien asuma la responsabilidad sobre el conjunto de la plataforma: conocer quién la utiliza, supervisar la actividad que en ella se genera y disponer de los medios para auditar cualquier incidencia. Todas las operaciones de este módulo están restringidas al rol de administrador, que constituye el único perfil con acceso a ellas.

Es importante insistir en la naturaleza de este actor. El Administrador es, ante todo, un usuario autenticado que conserva todas las capacidades descritas en los demás módulos (diagnóstico, historial, laboratorio y funcionalidades transversales) y que añade, además, las operaciones de gobierno que aquí se recogen. En la figura se representa como Administrador para distinguir las operaciones exclusivas de este rol; el resto de sus capacidades coinciden con las del actor «Usuario autenticado» de los diagramas anteriores. El módulo comprende cuatro casos de uso: la consulta del listado de usuarios (CU-031), que ofrece una visión global de las cuentas registradas; la consulta de las consultas de diagnóstico de un usuario concreto (CU-032), que permite revisar la actividad clínica de una cuenta; la visualización del detalle de una de esas consultas (CU-033), con la imagen, el resultado, la confianza y los metadatos asociados; y la gestión de las cuentas de usuario (CU-038), que permite desactivar una cuenta, cambiar un rol o eliminar una cuenta.

Estos casos de uso permiten al administrador identificar las cuentas activas de la plataforma, supervisar la actividad que en ella se registra, auditar un caso concreto ante cualquier incidencia y ejercer el gobierno de las cuentas. Las relaciones de extensión reflejan la navegación desde el listado general hasta el caso concreto: desde el listado de usuarios (CU-031) se accede a las consultas de un usuario (CU-032), y desde ellas al detalle completo de una consulta (CU-033), de modo que el administrador pueda profundizar progresivamente en la información que necesita. La gestión de cuentas (CU-038) se representa como una operación independiente del administrador sobre el conjunto de cuentas.

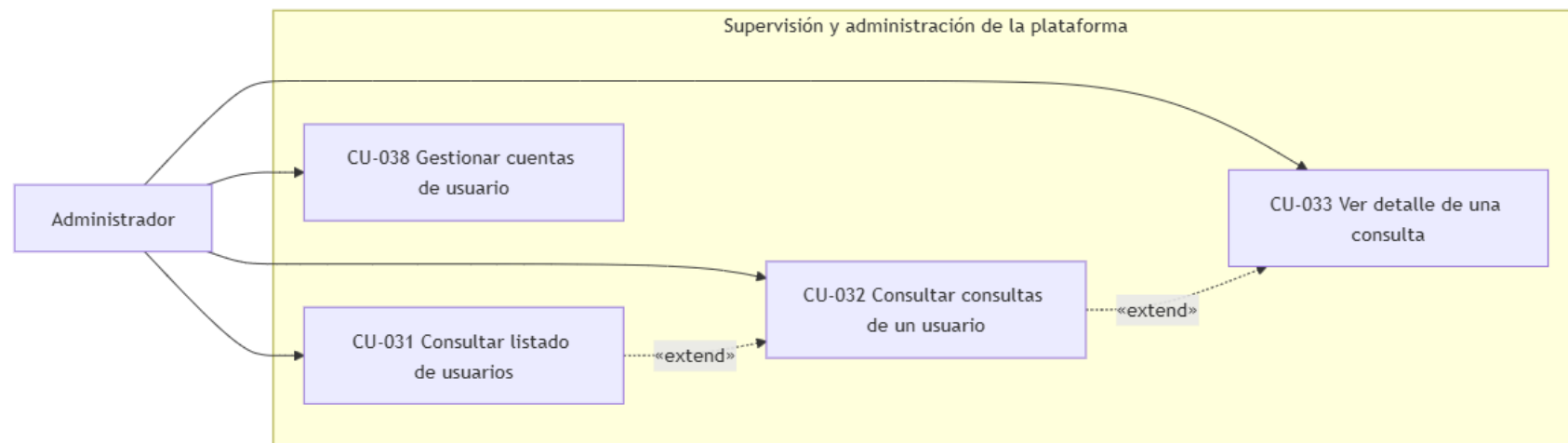


Ilustración x - Casos de uso del módulo de supervisión y administración de la plataforma

12.2.1.5. Capacidades transversales de la plataforma

Este módulo agrupa los casos de uso que no pertenecen a un ámbito funcional concreto, sino que afectan a toda la plataforma. Su carácter transversal se aprecia en que están disponibles para todo usuario autenticado, con independencia del núcleo funcional en el que trabaje: un profesional puede consultar el estado de sus diagnósticos mientras un investigador supervisa sus entrenamientos, y ambos pueden cancelar un trabajo que haya quedado pendiente en la cola. El cambio del tema visual, por su parte, afecta a la totalidad de la interfaz y responde a una preferencia personal del usuario.

El módulo comprende tres casos de uso. La consulta del estado de la cola de trabajos (CU-034) muestra periódicamente el estado de cada tarea pendiente, en ejecución, completada o fallida, cubriendo los tres tipos de trabajo que la plataforma ejecuta en segundo plano: los diagnósticos, los entrenamientos y las validaciones externas; el panel se actualiza de forma periódica y permite al usuario conocer en qué momento estará disponible un resultado o si un trabajo ha fallado. La cancelación de un trabajo de la cola (CU-035) impide que un trabajo pendiente llegue a ejecutarse. La interrupción de un trabajo en ejecución queda fuera de la implementación actual. La alternancia entre el tema claro y el tema oscuro de la interfaz (CU-036) permite al usuario elegir la presentación visual que mejor se adapte a sus condiciones de trabajo.

Ninguno de los tres casos de uso presenta relaciones de inclusión o extensión con otros casos de uso de la plataforma: la consulta de la cola, la cancelación de trabajos y el cambio de tema se ejecutan de forma autónoma, sin delegar pasos obligatorios en otros casos ni ampliar ningún flujo base.

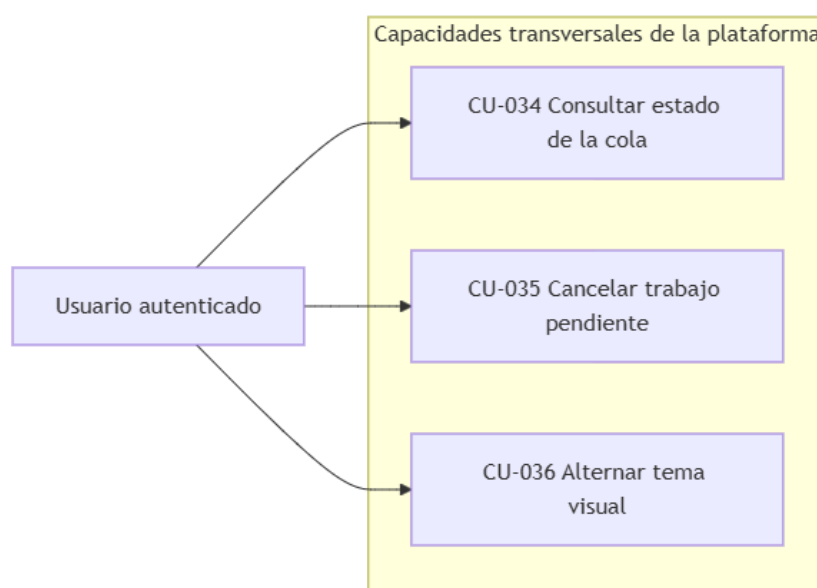


Ilustración x - Casos de uso del módulo de capacidades transversales de la plataforma

12.2.2. Actores del sistema y su ámbito de interacción

Un actor representa todo aquello que interactúa con el sistema desde el exterior, ya sea una persona o un sistema externo que produce o consume información de la plataforma (Jacobson, Booch, & Rumbaugh, 1999; Cockburn, 2001). Conviene distinguir con claridad entre los actores del sistema y los interesados del proyecto: un interesado es cualquier persona u organización que participa en el desarrollo o se ve afectada por su resultado, el tutor, los asesores, la universidad o los profesionales que aportaron su criterio durante el análisis, mientras que un actor es únicamente quien mantiene una interacción directa con el sistema en tiempo de ejecución. Esta distinción evita confundir la lista de participantes en el proyecto con los roles que el sistema debe reconocer a la hora de autorizar y atender sus operaciones.

En vitalXAI se han identificado tres actores, todos ellos perfiles humanos que interactúan con la plataforma a través de la interfaz web. Los servicios internos que ejecutan los procesos, la cola de trabajos, el motor de inteligencia artificial y el asistente conversacional, forman parte de la arquitectura interna del sistema y no se modelan como actores, porque nunca inician una interacción con el usuario: responden a las solicitudes que este realiza a través de la interfaz, y su papel se describe en la parte de diseño de la memoria. Los perfiles humanos se derivan de la definición de usuarios del análisis: el visitante, que accede sin autenticarse; el usuario autenticado, que reúne a todos los profesionales con cuenta registrada, con independencia de que su actividad sea clínica o investigadora; y el administrador, responsable del gobierno de la plataforma. La Tabla resume las características de los tres actores.

Actor	Tipo	Descripción	Alcance de acceso
Visitante	Primario	Persona que interactúa con la aplicación sin haber realizado el proceso de autenticación.	Funcionalidades de carácter público: registro, validación de credenciales y ajuste de idioma.
Usuario autenticado	Primario	Profesional registrado con acceso unificado a las dimensiones clínicas y de investigación del sistema.	Diagnóstico clínico, gestión de historial, laboratorio de entrenamiento MLOps y servicios transversales.
Administrador	Primario	Perfil facultado para el gobierno global y la supervisión de la actividad en la plataforma.	Capacidades operativas completas, gestión de cuentas y auditoría de consultas (CU-031 a CU-033).

Tabla x - Actores del sistema

El visitante es toda persona que accede a la plataforma sin haber iniciado sesión. Su interacción se limita a las funcionalidades públicas del sistema: la creación de una cuenta, el inicio de sesión y el cambio del idioma de la interfaz. Este perfil existe para que la plataforma permanezca abierta a nuevos registros sin exponer las funcionalidades de diagnóstico o de entrenamiento a quien no está autenticado. Tras completar el registro o el inicio de sesión, el visitante deja de serlo y pasa a disponer de una cuenta autenticada, con lo que se incorpora al conjunto de usuarios que pueden operar sobre las áreas privadas.

El usuario autenticado es el actor central del sistema y agrupa a todos los profesionales con cuenta registrada. Como se señaló en el análisis, la plataforma no restringe el acceso en función del perfil profesional: cualquier usuario con una cuenta puede utilizar tanto la interfaz de diagnóstico como el laboratorio de entrenamiento, de modo que la distinción entre el uso clínico y el uso investigador no se traduce en perfiles de acceso distintos, sino en la manera en que cada profesional emplea las mismas funcionalidades. El facultativo sin formación técnica encuentra en el diagnóstico asistido una herramienta clara y sin tecnicismos, mientras que el investigador dispone en el laboratorio de las capacidades de experimentación y de análisis estadístico que necesita. Este actor protagoniza la práctica totalidad de los casos de uso de la plataforma.

El administrador es un usuario autenticado que, además de conservar todas las capacidades del resto de los perfiles, dispone de las operaciones de gobierno de la plataforma recogidas en el módulo de supervisión y administración: la consulta del listado de usuarios, la consulta de las consultas de diagnóstico de un usuario concreto y la visualización del detalle de una de esas consultas. Su función es conocer quién utiliza la plataforma, supervisar la actividad que en ella se genera y auditar cualquier incidencia. La existencia de este rol se materializa mediante el mecanismo de roles y control de acceso del sistema, que restringe estas operaciones al perfil correspondiente.

Los tres actores identificados representan las interacciones externas documentadas: el visitante accede a las funciones públicas, el usuario autenticado utiliza las funciones privadas y el administrador dispone de las operaciones de supervisión. Los diagramas muestran esta asignación y el alcance de cada rol.

12.2.3. Actores del sistema y su ámbito de interacción

Los diagramas del apartado 12.2.1 muestran los casos de uso y los actores que los inician, pero no detallan el desarrollo de cada interacción. Por ello, cada caso se describe mediante las acciones del actor, las respuestas del sistema, las condiciones previas, los flujos normal y alternativo y el estado final (Cockburn, 2001). Estas fichas sirven como referencia para el diseño y las pruebas, sin sustituir la comprobación de la implementación.

A diferencia de los requisitos funcionales, que declaran qué debe hacer el sistema de forma técnica y verificable, los casos de uso describen esa misma capacidad desde la perspectiva del usuario, sin entrar en los detalles de implementación (Jacobson, Booch, & Rumbaugh, 1999; Larman, 2004). Un requisito funcional afirma, por ejemplo, que el sistema debe permitir iniciar sesión y otorgar acceso a las áreas privadas; el caso de uso correspondiente describe el recorrido completo que realiza la persona para lograrlo: cómo llega al formulario, qué introduce, qué verifica el sistema y qué ocurre tanto cuando todo es correcto como cuando no lo es. Esta perspectiva centrada en el usuario es precisamente lo que permite validar que el sistema responde a la manera en que los profesionales trabajan en la práctica y no solo a cómo se ha especificado sobre el papel: al leer un caso de uso, cualquier persona implicada en el proyecto, el tutor, un facultativo o el propio tribunal, puede seguir la secuencia y comprobar si refleja el comportamiento esperado. La redacción se mantiene, no obstante, al nivel de detalle suficiente para que la secuencia sea reproducible por quien vaya a diseñar o probar la funcionalidad, sin descender a mecanismos internos que el actor no percibe.

Cada caso de uso se identifica de forma inequívoca mediante el código CU-XXX y se especifica mediante una ficha que recoge los siguientes campos. El identificador y el nombre del caso de uso permiten referenciarlo de forma estable en todo el resto de la memoria. La fuente expresa el requisito o requisitos funcionales de los que deriva, de modo que quede constancia de qué capacidad del sistema materializa la interacción. Los actores indican qué perfiles participan en ella, y la descripción resume en una frase el propósito de la interacción. Las precondiciones enumeran las condiciones que deben cumplirse antes de iniciar el flujo; el flujo normal enumera, paso a paso y en orden, la secuencia de acciones y respuestas del camino de éxito; los flujos alternativos recogen los caminos que se desvían de ese recorrido, como los errores de validación, los datos ya existentes o las situaciones excepcionales, etiquetados con el número del paso del flujo normal en el que se producen; las postcondiciones describen el estado en que queda el sistema al finalizar, tanto tras el éxito como tras los caminos alternativos que sí dejan efectos; y, finalmente, la importancia y el estado del caso de uso completan la ficha. La trazabilidad con los requisitos funcionales queda así garantizada, porque cada caso de uso mantiene la misma estructura de módulos y deriva directamente de los requisitos del módulo correspondiente (Wieggers & Beatty, 2013).

Conviene precisar dos convenciones empleadas en la redacción de los flujos. En primer lugar, cada paso se numera de forma correlativa y se describe desde la perspectiva del actor: los pasos que inicia el actor se expresan en voz activa («el usuario sube la radiografía»), mientras que las respuestas del sistema se expresan como acciones de este («el sistema encola el análisis»), de modo que el lector distingue en todo momento quién actúa y quién responde. En segundo lugar, los flujos alternativos se referencian con el número del paso del flujo normal en el que se desvían, seguido de una letra que distingue cada alternativa: por ejemplo, la alternativa 4a se produce durante el paso 4 del flujo normal, y la 4b es una segunda desviación posible en ese mismo paso. Esta notación, tomada de la práctica habitual en la especificación de casos de uso, permite localizar con precisión dónde y en qué condiciones se produce cada desviación sin romper la lectura del flujo principal (Cockburn, 2001).

La relación entre los casos de uso y los requisitos funcionales no es, sin embargo, biunívoca: mientras que la mayoría de los requisitos funcionales se materializan en un caso de uso concreto, algunos requisitos de carácter transversal, como el aislamiento de datos entre usuarios o los roles y el control de acceso, no dan lugar a una interacción específica del actor, sino que condicionan la forma en que se ejecutan el resto de las interacciones. Estos requisitos se reflejan en las precondiciones y en los flujos alternativos de los casos de uso a los que afectan, por ejemplo, la comprobación de que un recurso pertenece al usuario que lo solicita, y su cumplimiento global se verifica mediante los requisitos no funcionales correspondientes. De esta manera, la especificación de casos de uso no duplica la de requisitos, sino que la complementa: los requisitos declaran qué debe ofrecerse y bajo qué condiciones de calidad, y los casos de uso concretan cómo se materializa cada capacidad en la interacción diaria del usuario con la plataforma.

A continuación se especifican los treinta y nueve casos de uso de vitalXAI, organizados en los cinco módulos funcionales presentados en el apartado de diagramas. El módulo de gestión del acceso y de la cuenta (CU-001 a CU-004) reúne las interacciones necesarias para crear una cuenta, entrar, salir y adaptar el idioma; la interfaz de diagnóstico asistido (CU-005 a CU-014 y CU-037) cubre el ciclo completo de una consulta clínica, desde la carga de la radiografía hasta la gestión del historial y la generación del informe; el laboratorio de experimentación MLOps (CU-015 a CU-030 y CU-039) agrupa la configuración, el lanzamiento y el análisis de los resultados de los experimentos de entrenamiento, además de la comprobación de sus límites; la supervisión y administración de la plataforma (CU-031 a CU-033 y CU-038) reúne las operaciones reservadas al administrador; y las capacidades transversales (CU-034 a CU-036) recogen la consulta de la cola de trabajos, la cancelación de trabajos pendientes y el cambio del tema visual. Los flujos se describen desde el punto de vista del actor, indicando en cada paso la acción que este realiza y la respuesta del sistema, de modo que la especificación pueda servir de base para el diseño de los subsistemas, para la elaboración de las pruebas de sistema y para la verificación final de que la plataforma entrega las capacidades comprometidas.

12.2.3.1. Módulo de gestión del acceso y de la cuenta

Este módulo agrupa los casos de uso que permiten a los usuarios acceder a la plataforma y gestionar su identidad en ella, y constituye el punto de entrada de todo el sistema: ninguna funcionalidad privada puede utilizarse sin haber completado previamente el proceso de autenticación que aquí se define. El módulo comprende cuatro casos de uso: la creación de la cuenta (CU-001), el inicio de sesión (CU-002), el cierre de sesión (CU-003) y el cambio del idioma de la interfaz (CU-004). Los tres primeros se corresponden con los requisitos funcionales de registro, acceso y cierre de sesión (RF-001 a RF-003), mientras que el cambio de idioma deriva del requisito de internacionalización (RF-004); los restantes requisitos del módulo (el aislamiento de datos entre usuarios (RF-005) y los roles y el control de acceso (RF-006)) no se materializan en casos de uso propios, porque constituyen condiciones transversales que se aplican a todas las interacciones de la plataforma y no interacciones concretas del actor.

Como se señaló al presentar los actores, el actor Administrador es un usuario autenticado con las mismas capacidades que el resto de los perfiles, por lo que en este módulo el actor «Usuario autenticado» lo incluye. Al tratarse de interacciones directas e independientes, ninguno de los casos de uso del módulo presenta relaciones de inclusión o extensión con los demás: el registro, el acceso, el cierre de sesión y el cambio de idioma se ejecutan de forma autónoma. A continuación se especifica cada uno de ellos.

CU-001 - Registrarse

Campo	Contenido
ID	CU-001
Nombre	Registrarse
Fuente	RF-001
Actores	Visitante
Descripción	El visitante crea una cuenta en la plataforma proporcionando nombre de usuario, nombre, apellidos, correo electrónico y contraseña. El sistema valida los datos, comprueba que la identidad no esté ya registrada y crea el registro almacenando la contraseña cifrada, de modo que nunca quede en texto plano.
Precondiciones	El visitante no está autenticado. El nombre de usuario y el correo electrónico no están registrados previamente.
Flujo normal	1. El visitante accede al formulario de registro desde la página de inicio de sesión. 2. El sistema muestra el formulario con los campos nombre de usuario, nombre, apellidos, correo electrónico y contraseña. 3. El visitante introduce sus datos y confirma el registro. 4. El sistema valida el formato de los datos y la fortaleza de la contraseña. 5. El sistema comprueba que el nombre de usuario y el correo electrónico no existen previamente. 6. El sistema transforma la contraseña mediante un mecanismo de almacenamiento seguro y crea el registro del usuario. 7. El sistema muestra una confirmación y redirige al visitante a la página de inicio de sesión.
Flujo alternativo	4a. Si algún dato es inválido o la contraseña no cumple los requisitos de fortaleza, el sistema muestra un mensaje de error indicando el campo afectado y no crea el registro. 5a. Si el nombre de usuario o el correo electrónico ya están registrados, el sistema muestra un mensaje de error y no crea el registro.
Postcondiciones	El usuario queda registrado en la plataforma y puede iniciar sesión con sus credenciales.
Importancia	Alta
Estado	Aprobado
Comentarios	La contraseña se almacena siempre cifrada; los detalles del mecanismo concreto de cifrado son una decisión de diseño. El tratamiento de los datos personales del registro se ajusta al RGPD.

Tabla x - CU-001 - Registrarse

CU-002 - Iniciar sesión

Campo	Contenido
ID	CU-002
Nombre	Iniciar sesión
Fuente	RF-002
Actores	Visitante
Descripción	El visitante introduce sus credenciales y el sistema, tras verificarlas contra las almacenadas, inicia una sesión segura y le otorga acceso a las áreas privadas de la plataforma.
Precondiciones	El visitante dispone de una cuenta registrada. El visitante no tiene una sesión activa.
Flujo normal	1. El visitante accede al formulario de inicio de sesión. 2. El sistema muestra el formulario con los campos nombre de usuario y contraseña. 3. El visitante introduce sus credenciales y confirma. 4. El sistema verifica que la contraseña coincide con el hash almacenado para ese nombre de usuario. 5. El sistema genera el token de acceso y el token de refresco, y los establece en cookies seguras. 6. El sistema redirige al usuario a su panel.
Flujo alternativo	4a. Si las credenciales son incorrectas, el sistema muestra un mensaje de error genérico que no indica si el fallo está en el nombre de usuario o en la contraseña. 4b. Si se supera el límite de intentos permitidos, el sistema bloquea temporalmente las peticiones desde esa dirección y lo informa al usuario.
Postcondiciones	La sesión queda iniciada y el usuario accede a las áreas privadas de la plataforma.
Importancia	Alta
Estado	Aprobado
Comentarios	Los tokens se gestionan mediante cookies seguras. El límite de intentos y los mensajes genéricos previenen los ataques de fuerza bruta y de enumeración de usuarios, en línea con los requisitos no funcionales de seguridad.

Tabla x - CU-002 - Iniciar sesión

CU-003 - Cerrar sesión

Campo	Contenido
ID	CU-003
Nombre	Cerrar sesión
Fuente	RF-003
Actores	Usuario autenticado
Descripción	El usuario selecciona una imagen de su equipo para ser analizada. El sistema valida el tipo de archivo (JPEG o PNG) y su tamaño (máximo 10 MB) y la conserva en el servidor asociada a la consulta, de modo que pueda recuperarse desde el historial.
Precondiciones	El usuario tiene una sesión iniciada.
Flujo normal	1. El usuario selecciona el archivo de imagen desde su equipo. 2. El sistema valida el formato y el tamaño del archivo. 3. El sistema conserva la imagen asociada a la consulta, de modo que pueda recuperarse desde el historial.
Flujo alternativo	N/A
Postcondiciones	La sesión queda revocada y el acceso a las áreas privadas queda bloqueado hasta un nuevo inicio de sesión.
Importancia	Alta
Estado	Aprobado
Comentarios	El actor Administrador, por ser también un usuario autenticado, participa en este caso de uso con el mismo comportamiento.

Tabla x - CU-003 - Cerrar sesión

CU-004 - Cambiar el idioma de la interfaz

Campo	Contenido
ID	CU-004
Nombre	Cambiar el idioma de la interfaz
Fuente	RF-004
Actores	Visitante, Usuario autenticado
Descripción	El actor selecciona el idioma de la interfaz entre los disponibles (español, inglés, chino e hindi). El sistema guarda la preferencia y aplica las traducciones en la interfaz, en los informes generados y en el asistente conversacional.
Precondiciones	El actor accede a la plataforma, esté o no autenticado.
Flujo normal	1. El actor selecciona el idioma deseado en el selector de idioma de la interfaz. 2. El sistema guarda la preferencia. 3. El sistema aplica las traducciones correspondientes en toda la interfaz sin recargar la página.
Flujo alternativo	N/A
Postcondiciones	La interfaz se muestra en el idioma seleccionado y la preferencia queda guardada.
Importancia	Media
Estado	Aprobado
Comentarios	El actor Administrador, por ser también un usuario autenticado, queda incluido en el actor «Usuario autenticado» de este caso de uso.

Tabla x - CU-004 - Cambiar el idioma de la interfaz

12.2.3.2. Módulo de interfaz de diagnóstico asistido

Este módulo agrupa los casos de uso de la interfaz clínica, el primer núcleo funcional del sistema y el eje de la actividad de diagnóstico del profesional. A través de ella, el usuario autenticado realiza un diagnóstico asistido de neumonía a partir de una radiografía de tórax: carga la imagen, elige la arquitectura con la que desea realizar la consulta, solicita el diagnóstico y obtiene el resultado junto con su nivel de confianza y los mapas de explicabilidad que lo justifican. El diagnóstico se procesa de forma asíncrona mediante la cola de trabajos: en cuanto el usuario envía la petición, el sistema encola el análisis y la interfaz permanece operativa, de modo que el facultativo puede seguir trabajando mientras el modelo procesa la imagen, genera la predicción y produce la explicación visual. El módulo comprende los casos de uso CU-005 a CU-014, que se corresponden con los requisitos funcionales de la interfaz clínica (RF-007 a RF-012) y de la gestión del historial de consultas (RF-013 a RF-016). El informe PDF individual del diagnóstico se recoge en el caso de uso CU-037, asociado al requisito RF-039.

El módulo se organiza en dos bloques. El primero, centrado en la consulta de diagnóstico, reúne el acceso al panel (CU-005), la subida de la radiografía (CU-006), la selección de la arquitectura (CU-007), la solicitud del diagnóstico (CU-008), la visualización del resultado (CU-009) y la visualización de los mapas de explicabilidad (CU-010). El segundo, centrado en la gestión del historial, reúne la consulta del listado (CU-011), el detalle de una consulta (CU-012), su renombrado (CU-013) y su eliminación (CU-014). En este segundo bloque, las relaciones de extensión reflejan el orden en que el actor recorre la interfaz: desde el listado (CU-011) se accede al detalle (CU-012), y desde el detalle pueden ejecutarse, de forma opcional, el renombrado (CU-013) o la eliminación (CU-014). El aislamiento de datos entre usuarios (RF-005) se materializa en este módulo como una condición aplicada a todas las operaciones sobre consultas: cada consulta solo puede ser consultada, renombrada o eliminada por el usuario que la realizó.

CU-005 - Acceder al panel de diagnóstico

Campo	Contenido
ID	CU-005
Nombre	Acceder al panel de diagnóstico
Fuente	RF-007
Actores	Usuario autenticado
Descripción	El usuario autenticado accede al panel de diagnóstico, desde el que puede cargar una radiografía, seleccionar un modelo y consultar su historial de consultas.
Precondiciones	El usuario tiene una sesión iniciada.
Flujo normal	1. El usuario accede al panel de diagnóstico desde la navegación principal. 2. El sistema valida la sesión activa del usuario. 3. El sistema carga el panel y lo muestra.
Flujo alternativo	2a. Si el usuario no tiene una sesión activa, el sistema deniega el acceso y redirige al usuario a la página de inicio de sesión.
Postcondiciones	El panel de diagnóstico queda visible y operativo para el usuario.
Importancia	Alta
Estado	Aprobado
Comentarios	El panel constituye el punto de entrada a la funcionalidad clínica de la plataforma.

Tabla x - CU-005 - Acceder al panel de diagnóstico

CU-006 - Subir una radiografía de tórax

Campo	Contenido
ID	CU-006
Nombre	Subir una radiografía de tórax
Fuente	RF-008
Actores	Usuario autenticado
Descripción	El usuario selecciona una imagen de su equipo para ser analizada. El sistema valida el tipo de archivo (JPEG o PNG) y su tamaño (máximo 10 MB) y la almacena de forma temporal en el servidor, asociándola a la consulta en curso.
Precondiciones	El usuario se encuentra en el panel de diagnóstico.
Flujo normal	1. El usuario selecciona el archivo de imagen desde su equipo. 2. El sistema valida el formato y el tamaño del archivo. 3. El sistema almacena la imagen de forma temporal y la asocia a la consulta en curso.
Flujo alternativo	2a. Si el formato o el tamaño no son válidos, el sistema muestra un mensaje de error y rechaza la imagen, que no queda asociada a ninguna consulta.
Postcondiciones	La imagen queda disponible para el diagnóstico y asociada a la consulta en curso.
Importancia	Alta
Estado	Aprobado
Comentarios	La validación se realiza antes de encolar la consulta, en línea con el requisito RF-008.

Tabla x - CU-006 - Subir una radiografía de tórax

CU-007 - Seleccionar la arquitectura para el diagnóstico

Campo	Contenido
ID	CU-007
Nombre	Selección de arquitectura de inferencia
Fuente	RF-009
Actores	Usuario autenticado
Descripción	El usuario elige, entre las arquitecturas de deep learning disponibles en la interfaz, el modelo con el que desea realizar el diagnóstico.
Precondiciones	El usuario se encuentra en el panel de diagnóstico.
Flujo normal	1. El usuario despliega el selector de modelos del panel. 2. El sistema muestra la lista de arquitecturas disponibles. 3. El usuario selecciona la arquitectura deseada.
Flujo alternativo	N/A
Postcondiciones	La arquitectura queda seleccionada para la consulta en curso.
Importancia	Media
Estado	Aprobado
Comentarios	La lista de arquitecturas disponibles se muestra en el panel de diagnóstico.

Tabla x - CU-007 - Seleccionar la arquitectura para el diagnóstico

CU-008 - Solicitar un diagnóstico

Campo	Contenido
ID	CU-008
Nombre	Solicitar un diagnóstico
Fuente	RF-010
Actores	Usuario autenticado
Descripción	El usuario envía la petición de diagnóstico con la imagen y el modelo seleccionados. El sistema valida la petición, encola el trabajo de diagnóstico y lo procesa en segundo plano, sin bloquear la interfaz.
Precondiciones	Hay una imagen subida y un modelo seleccionado en la consulta en curso.
Flujo normal	1. El usuario envía la petición de diagnóstico. 2. El sistema valida la imagen y el modelo seleccionado. 3. El sistema encola el trabajo de diagnóstico. 4. El worker procesa el trabajo y genera la predicción, los mapas de explicabilidad y el informe de la consulta. 5. El sistema notifica al usuario la finalización del diagnóstico.
Flujo alternativo	2a. Si la petición no es válida, el sistema muestra un mensaje de error y no encola el trabajo. 4a. Si el procesamiento falla, el trabajo pasa a estado fallido, se registra el error y se informa al usuario.
Postcondiciones	La consulta queda registrada en el historial con su resultado, su confianza, sus mapas de explicabilidad y su informe.
Importancia	Alta
Estado	Aprobado
Comentarios	La presentación del resultado (CU-009), de los mapas (CU-010) y del informe (CU-037) son capacidades verificables por separado; este caso de uso cubre exclusivamente la solicitud y el encolado del diagnóstico.

Tabla x - CU-008 - Solicitar un diagnóstico

CU-009 - Visualizar el resultado del diagnóstico

Campo	Contenido
ID	CU-009
Nombre	Visualizar el resultado del diagnóstico
Fuente	RF-011
Actores	Usuario autenticado
Descripción	Cuando el trabajo de diagnóstico finaliza, el sistema muestra el resultado de la consulta: la predicción (PNEUMONIA o NORMAL), el nivel de confianza asociado y el modelo utilizado.
Precondiciones	La consulta ha sido procesada por el worker y ha pasado a estado completado.
Flujo normal	1. El usuario espera a que la consulta pase a estado completado. 2. El sistema muestra el resultado con su nivel de confianza y el modelo empleado.
Flujo alternativo	1a. Si la consulta ha pasado a estado fallido, el sistema muestra el estado fallido e informa del error, sin presentar ningún resultado.
Postcondiciones	El resultado queda visible para el usuario y registrado en su historial.
Importancia	Alta
Estado	Aprobado
Comentarios	La consulta queda registrada en el historial con su resultado, en línea con el requisito RF-011.

Tabla x - CU-009 - Visualizar el resultado del diagnóstico

CU-010 - Visualizar los mapas de explicabilidad

Campo	Contenido
ID	CU-010
Nombre	Visualización de los mapas de explicabilidad
Fuente	RF-012
Actores	Usuario autenticado
Descripción	El usuario visualiza los mapas de calor generados por el sistema (Saliency Maps, SmoothGrad y Grad-CAM para arquitecturas convolucionales, o mapas de atención para arquitecturas Transformer), superpuestos sobre la radiografía original.
Precondiciones	La consulta ha sido procesada y los mapas de explicabilidad han sido generados.
Flujo normal	1. El usuario selecciona la consulta completada. 2. El sistema comprueba que la consulta pertenece al usuario. 3. El sistema muestra el mosaico con la radiografía original y los mapas de explicabilidad.
Flujo alternativo	2a. Si la consulta no pertenece al usuario, el sistema deniega el acceso y no muestra la información.
Postcondiciones	Los mapas de explicabilidad quedan visibles para su inspección.
Importancia	Alta
Estado	Aprobado
Comentarios	La generación de las explicaciones se apoya en las técnicas XAI descritas en el capítulo 1, y su evaluación cuantitativa se recoge en el laboratorio (CU-020).

Tabla x - CU-010 - Visualizar los mapas de explicabilidad

CU-011 - Consultar el historial de consultas

Campo	Contenido
ID	CU-011
Nombre	Consulta del registro de diagnósticos
Fuente	RF-013
Actores	Usuario autenticado
Descripción	El usuario consulta el listado de sus consultas de diagnóstico, en el que se muestran la fecha, el modelo empleado, el resultado y la confianza de cada una.
Precondiciones	El usuario tiene una sesión iniciada.
Flujo normal	1. El usuario accede a su historial desde el panel de diagnóstico. 2. El sistema recupera y muestra únicamente las consultas del propio usuario.
Flujo alternativo	2a. Si el usuario no tiene consultas registradas, el sistema muestra un mensaje informativo y un listado vacío.
Postcondiciones	El listado del historial queda visible para el usuario.
Importancia	Media
Estado	Aprobado
Comentarios	Solo se muestran las consultas del propio usuario, en cumplimiento del aislamiento de datos entre usuarios.

Tabla x - CU-011 - Consultar el historial de consultas

CU-012 - Ver el detalle de una consulta del historial

Campo	Contenido
ID	CU-012
Nombre	Ver el detalle de una consulta del historial
Fuente	RF-014
Actores	Usuario autenticado
Descripción	El usuario abre el detalle de una consulta de su historial, que incluye la radiografía original, el resultado, la confianza, los mapas de explicabilidad y los metadatos asociados
Precondiciones	La consulta pertenece al usuario.
Flujo normal	1. El usuario selecciona una consulta del listado del historial. 2. El sistema comprueba que la consulta pertenece al usuario. 3. El sistema muestra el detalle completo de la consulta.
Flujo alternativo	2a. Si la consulta no pertenece al usuario, el sistema deniega el acceso y no muestra la información.
Postcondiciones	El detalle de la consulta queda visible para el usuario.
Importancia	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-011: el detalle se alcanza desde el listado del historial.

Tabla x - CU-012 - Ver el detalle de una consulta del historial

CU-013 - Renombrar una consulta del historial

Campo	Contenido
ID	CU-013
Nombre	Renombrar una consulta del historial
Fuente	RF-015
Actores	Usuario autenticado
Descripción	El usuario modifica el nombre de una de sus consultas del historial para identificarla mejor.
Precondiciones	La consulta pertenece al usuario.
Flujo normal	1. El usuario indica el nuevo nombre de la consulta desde el detalle de la misma. 2. El sistema comprueba que la consulta pertenece al usuario. 3. El sistema valida que el nuevo nombre no está vacío. 4. El sistema actualiza el nombre de la consulta.
Flujo alternativo	2a. Si la consulta no pertenece al usuario, el sistema deniega el acceso y no modifica nada. 3a. Si el nuevo nombre está vacío, el sistema muestra un mensaje de error y mantiene el nombre anterior.
Postcondiciones	La consulta queda renombrada en el historial del usuario.
Importancia	Baja
Estado	Aprobado
Comentarios	El renombrado no afecta a la imagen, al resultado ni a los mapas de la consulta. Este caso de uso extiende CU-012.

Tabla x - CU-013 - Renombrar una consulta del historial

CU-014 - Eliminar una consulta del historial

Campo	Contenido
ID	CU-014
Nombre	Eliminar una consulta del historial
Fuente	RF-016
Actores	Usuario autenticado
Descripción	El usuario elimina una consulta de su historial de forma definitiva, de modo que el registro y sus artefactos asociados dejan de estar disponibles tanto para el propio usuario como para el administrador.
Precondiciones	La consulta pertenece al usuario.
Flujo normal	1. El usuario solicita la eliminación de la consulta desde el detalle de la misma. 2. El sistema comprueba que la consulta pertenece al usuario. 3. El sistema solicita confirmación de la eliminación. 4. El usuario confirma la eliminación. 5. El sistema elimina el registro de la consulta y sus artefactos asociados.
Flujo alternativo	2a. Si la consulta no pertenece al usuario, el sistema deniega el acceso y no elimina nada. 4a. Si el usuario cancela la operación, la consulta se conserva sin cambios.
Postcondiciones	La consulta y sus artefactos quedan eliminados de forma definitiva del sistema.
Importancia	Baja
Estado	Aprobado
Comentarios	La eliminación se realiza únicamente sobre consultas del propio usuario y es definitiva, en coherencia con el derecho de supresión del RGPD. Este caso de uso extiende CU-012.

Tabla x - CU-014 - Eliminar una consulta del historial

12.2.3.3. Módulo de laboratorio de experimentación MLOps

Este módulo agrupa los casos de uso del laboratorio de entrenamiento, el segundo núcleo funcional del sistema y el espacio en el que se desarrolla la actividad investigadora. A través de él, el usuario configura y lanza experimentos de entrenamiento mediante un asistente conversacional, monitoriza la ejecución del pipeline y consulta los resultados comparativos y estadísticos de sus sesiones. El laboratorio orquesta automáticamente la secuencia de entrenamiento, análisis de explicabilidad, comparación estadística y validación externa, de modo que el investigador no necesita escribir código en ningún momento: el asistente interpreta sus indicaciones, el pipeline ejecuta los procesos en segundo plano y la interfaz presenta los resultados de forma organizada. El módulo comprende los casos de uso CU-015 a CU-030, que se corresponden con los requisitos funcionales del laboratorio (RF-017 a RF-032), y el escenario previsto CU-039, asociado al requisito RF-041. Estos casos se asocian a los objetivos de laboratorio de entrenamiento MLOps (OBJ-004), evaluación rigurosa de los modelos (OBJ-005) y ejecución asíncrona de tareas (OBJ-010).

El lanzamiento del experimento (CU-018) depende de que la configuración del experimento debe quedar completa antes de iniciar el entrenamiento. Las relaciones de extensión siguen la navegación del usuario por el laboratorio: desde la consulta de las sesiones (CU-019) se accede a los resultados de un modelo (CU-020), al ranking (CU-022), a la comparativa estadística (CU-023) o al informe PDF (CU-028); desde los resultados de un modelo se alcanzan sus mapas de explicabilidad (CU-021); y desde la comparativa estadística puede solicitarse su recálculo (CU-024). El aislamiento de datos entre usuarios (RF-005) se aplica también en este módulo: cada sesión solo puede ser consultada, renombrada o eliminada por el usuario que la lanzó.

CU-015 - Acceso al laboratorio de entrenamiento

Campo	Contenido
ID	CU-015
Nombre	Acceso al laboratorio de entrenamiento
Fuente	RF-017
Actores	Usuario autenticado
Descripción	El usuario autenticado accede al laboratorio de entrenamiento, desde el que puede conversar con el asistente, lanzar experimentos y consultar sus sesiones.
Precondiciones	El usuario tiene una sesión iniciada.
Flujo normal	1. El usuario accede al laboratorio desde la navegación principal. 2. El sistema valida la sesión activa del usuario. 3. El sistema carga el entorno del laboratorio y lo muestra.
Flujo alternativo	2a. Si el usuario no tiene una sesión activa, el sistema deniega el acceso y redirige al usuario a la página de inicio de sesión.
Postcondiciones	El laboratorio queda visible y operativo para el usuario.
Importancia	Alta
Estado	Aprobado
Comentarios	El laboratorio es accesible para todo usuario autenticado, con independencia de su perfil profesional.

Tabla x - CU-015 - Acceso al laboratorio de entrenamiento

CU-016 - Conversar con el asistente para configurar un experimento

Campo	Contenido
ID	CU-016
Nombre	Conversar con el asistente para configurar un experimento
Fuente	RF-018
Actores	Usuario autenticado
Descripción	El usuario conversa en lenguaje natural con el asistente conversacional para definir los parámetros del experimento: arquitecturas a entrenar, número de épocas, tamaño de lote y tasa de aprendizaje. El asistente interpreta las indicaciones y, cuando dispone de todos los parámetros, devuelve la configuración estructurada. La ruta del dataset se incorpora mediante el caso de uso CU-017.
Precondiciones	El usuario se encuentra en el laboratorio.
Flujo normal	1. El usuario envía un mensaje al asistente indicando los parámetros del experimento. 2. El sistema envía la petición al modelo de lenguaje con el prompt de sistema definido. 3. El modelo extrae los parámetros mencionados en el mensaje. 4. Si todos los parámetros están definidos, el asistente devuelve la configuración estructurada. 5. El sistema rellena el panel de configuración con los valores obtenidos.
Flujo alternativo	4a. Si faltan parámetros, el asistente pregunta por ellos y la conversación continúa hasta completarlos. 2a. Si el servicio del modelo de lenguaje no está disponible, el sistema informa del error y la configuración no se completa.
Postcondiciones	La configuración del experimento queda disponible para su revisión y lanzamiento, con la ruta del dataset incorporada.
Importancia	Alta
Estado	Aprobado
Comentarios	La conversación se desarrolla en el idioma seleccionado por el usuario. La ruta del dataset se obtiene mediante CU-017, no mediante la conversación.

Tabla x - CU-016 - Conversar con el asistente para configurar un experimento

CU-017 - Seleccionar la carpeta del dataset

Campo	Contenido
ID	CU-017
Nombre	Seleccionar la carpeta del dataset
Fuente	RF-019
Actores	Usuario autenticado
Descripción	El usuario selecciona el dataset de entrenamiento y el dataset externo de validación dentro del directorio raíz de datasets permitido. El sistema valida que la ruta permanece dentro de ese directorio y no expone el resto del sistema de ficheros del servidor.
Precondiciones	El usuario se encuentra en el laboratorio y está configurando un experimento.
Flujo normal	1. El usuario solicita seleccionar el dataset. 2. El sistema muestra las opciones disponibles dentro del directorio raíz de datasets permitido. 3. El usuario confirma la ruta. 4. El sistema valida que la ruta permanece dentro del directorio permitido.
Flujo alternativo	2a. Si la ruta indicada no es accesible, el sistema muestra un mensaje de error y permite seleccionar otra.
Postcondiciones	La ruta del dataset queda disponible para el experimento.
Importancia	Media
Estado	Aprobado
Comentarios	La selección se confina al directorio de datasets permitido para impedir el acceso a rutas arbitrarias del servidor y respetar el aislamiento entre usuarios (RF-005).

Tabla x - CU-017 - Seleccionar la carpeta del dataset

CU-018 - Lanzar un experimento de entrenamiento

Campo	Contenido
ID	CU-018
Nombre	Lanzar un experimento de entrenamiento
Fuente	RF-020
Actores	Usuario autenticado
Descripción	El usuario lanza el experimento con la configuración definida. El sistema crea una sesión de entrenamiento y encola la ejecución de forma asíncrona, de modo que la interfaz permanece operativa durante el entrenamiento.
Precondiciones	La configuración del experimento está completa, incluida la ruta del dataset (CU-016 y CU-017).
Flujo normal	1. El usuario envía la configuración del experimento. 2. El sistema crea la sesión de entrenamiento y encola el trabajo. 3. El worker ejecuta el entrenamiento de las arquitecturas solicitadas. 4. El sistema actualiza el estado de la sesión al finalizar.
Flujo alternativo	3a. Si algún script del pipeline falla, la sesión queda registrada con el error correspondiente y el usuario es informado.
Postcondiciones	La sesión de entrenamiento queda creada con su estado.
Importancia	Alta
Estado	Aprobado
Comentarios	Este caso de uso depende de CU-016 y CU-017: la configuración y el dataset deben estar definidos antes de lanzar el experimento. El análisis de explicabilidad y la comparación estadística se consultan mediante CU-020 a CU-023.

Tabla x - CU-018 - Lanzar un experimento de entrenamiento

CU-019 - Consultar las sesiones de entrenamiento

Campo	Contenido
ID	CU-019
Nombre	Consultar las sesiones de entrenamiento
Fuente	RF-021
Actores	Usuario autenticado
Descripción	El usuario consulta el listado de sus sesiones de entrenamiento con su estado y sus modelos.
Precondiciones	El usuario tiene una sesión iniciada.
Flujo normal	1. El usuario accede al listado de sesiones del laboratorio. 2. El sistema recupera y muestra únicamente las sesiones del propio usuario, con su estado y sus modelos.
Flujo alternativo	2a. Si el usuario no tiene sesiones de entrenamiento, el sistema muestra un mensaje informativo y un listado vacío.
Postcondiciones	El listado de sesiones queda visible para el usuario.
Importancia	Media
Estado	Aprobado
Comentarios	Solo se muestran las sesiones del propio usuario, en cumplimiento del aislamiento de datos entre usuarios.

Tabla x - CU-019 - Consultar las sesiones de entrenamiento

CU-020 - Consultar los resultados de un modelo de la sesión

Campo	Contenido
ID	CU-020
Nombre	Consultar los resultados de un modelo de la sesión
Fuente	RF-022
Actores	Usuario autenticado
Descripción	El usuario consulta los resultados cuantitativos de un modelo concreto de la sesión: las métricas de la validación cruzada (exactitud, precisión, sensibilidad, F1 y AUC), las métricas XAI cuantitativas y las métricas de calibración, con su media y su desviación sobre los pliegues.
Precondiciones	La sesión dispone de resultados para el modelo seleccionado.
Flujo normal	1. El usuario selecciona un modelo de la sesión. 2. El sistema recupera y muestra las métricas del modelo con su media y desviación sobre los pliegues.
Flujo alternativo	2a. Si el modelo no dispone de resultados, el sistema muestra un mensaje informativo.
Postcondiciones	Los resultados del modelo quedan visibles para el usuario.
Importancia	Alta
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-019. La desagregación por pliegue es una decisión del diseño de presentación, coherente con el modelo de resultados persistido de la sesión.

Tabla x - CU-020 - Consultar los resultados de un modelo de la sesión

CU-021 - Visualizar los mapas de calor de explicabilidad de un modelo

Campo	Contenido
ID	CU-021
Nombre	Visualizar los mapas de calor de explicabilidad de un modelo
Fuente	RF-023
Actores	Usuario autenticado
Descripción	El usuario visualiza la galería de mapas de calor de explicabilidad del modelo, generados por el análisis XAI cualitativo sobre imágenes de ejemplo.
Precondiciones	El modelo dispone de mapas de explicabilidad generados.
Flujo normal	1. El usuario selecciona el modelo de la sesión. 2. El sistema muestra la galería de imágenes XAI del modelo.
Flujo alternativo	2a. Si el modelo no dispone de mapas de explicabilidad, el sistema muestra un mensaje informativo.
Postcondiciones	Los mapas de calor quedan visibles para su inspección.
Media	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-020: la galería se alcanza desde los resultados del modelo.

Tabla x - CU-021 - Visualizar los mapas de calor de explicabilidad de un modelo

CU-022 - Consultar el ranking de modelos de la sesión

Campo	Contenido
ID	CU-022
Nombre	Consultar el ranking de modelos de la sesión
Fuente	RF-024
Actores	Usuario autenticado
Descripción	El usuario consulta el ranking global de los modelos de la sesión, ordenado por su AUC medio de la validación cruzada, junto con su desviación típica.
Precondiciones	La comparación estadística de la sesión se ha generado.
Flujo normal	1. El usuario solicita el ranking de la sesión. 2. El sistema recupera y muestra el ranking de los modelos ordenados por su AUC medio.
Flujo alternativo	2a. Si la comparación estadística no se ha generado, el sistema muestra un mensaje informativo.
Postcondiciones	El ranking de modelos queda visible para el usuario.
Importancia	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-019. El ranking se regenera al ejecutar la comparación estadística.

Tabla x - CU-022 - Consultar el ranking de modelos de la sesión

CU-023 - Consultar la comparativa estadística de la sesión

Campo	Contenido
ID	CU-023
Nombre	Consultar la comparativa estadística de la sesión
Fuente	RF-025
Actores	Usuario autenticado
Descripción	El usuario consulta la matriz de significación estadística que compara los modelos de la sesión: los p-valores del test de Wilcoxon y, cuando la validación externa se ha ejecutado, los del test de DeLong sobre las curvas ROC.
Precondiciones	La comparación estadística de la sesión se ha generado.
Flujo normal	1. El usuario accede a la vista de comparativa de la sesión. 2. El sistema muestra la matriz de significación correspondiente.
Flujo alternativo	2a. Si la comparación estadística no se ha generado, el sistema muestra un mensaje informativo.
Postcondiciones	La comparativa estadística queda visible para el usuario.
Importancia	Alta
Estado	Aprobado
Comentarios	La interpretación de la significación debe considerar la potencia limitada de la prueba con el número de pliegues disponible; este caso de uso no fija un umbral. Extiende CU-019.

Tabla x - CU-023 - Consultar la comparativa estadística de la sesión

CU-024 - Solicitar el recálculo de la comparativa estadística

Campo	Contenido
ID	CU-024
Nombre	Solicitar el recálculo de la comparativa estadística
Fuente	RF-026
Actores	Usuario autenticado
Descripción	El usuario solicita el recálculo de la comparativa estadística de la sesión, regenerando el ranking y el test de Wilcoxon de forma asíncrona.
Precondiciones	La sesión dispone de resultados de sus modelos.
Flujo normal	1. El usuario solicita el recálculo de la comparativa. 2. El sistema lanza el proceso de recálculo en segundo plano. 3. El sistema actualiza la comparativa y notifica al usuario cuando el recálculo finaliza.
Flujo alternativo	2a. Si el proceso de recálculo falla, el sistema registra el error e informa al usuario.
Postcondiciones	La comparativa estadística de la sesión queda actualizada.
Importancia	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-023: el recálculo se solicita desde la comparativa estadística.

Tabla x - CU-024 - Solicitar el recálculo de la comparativa estadística

CU-025 - Ejecutar el análisis de explicabilidad de un modelo

Campo	Contenido
ID	CU-025
Nombre	Ejecutar el análisis de explicabilidad de un modelo
Fuente	RF-027
Actores	Usuario autenticado
Descripción	El usuario solicita la ejecución manual del análisis de explicabilidad (cualitativo y cuantitativo) de un modelo de la sesión, regenerando sus mapas y sus métricas.
Precondiciones	El modelo pertenece a una sesión del usuario.
Flujo normal	1. El usuario solicita generar el análisis XAI del modelo. 2. El sistema ejecuta los scripts de explicabilidad en segundo plano. 3. El sistema notifica al usuario cuando el análisis finaliza.
Flujo alternativo	2a. Si el análisis falla, el sistema registra el error e informa al usuario.
Postcondiciones	Los mapas y las métricas XAI del modelo quedan regenerados.
Importancia	Media
Estado	Aprobado
Comentarios	La ejecución manual complementa el análisis automático del pipeline.

Tabla x - CU-025 - Ejecutar el análisis de explicabilidad de un modelo

CU-026 - Solicitar la validación externa de la sesión

Campo	Contenido
ID	CU-026
Nombre	Solicitar la validación externa de la sesión
Fuente	RF-028
Actores	Usuario autenticado
Descripción	El usuario solicita la validación externa de la sesión sobre el dataset externo de pacientes adultos. El sistema encola el trabajo, que evalúa los modelos congelados y aplica el test de DeLong para comparar sus curvas ROC, de forma asíncrona.
Precondiciones	La sesión dispone de modelos entrenados y de un dataset externo disponible.
Flujo normal	1. El usuario solicita la validación externa de la sesión. 2. El sistema encola el trabajo de validación externa. 3. El worker evalúa los modelos congelados y aplica el test de DeLong. 4. El sistema notifica al usuario cuando la validación finaliza.
Flujo alternativo	3a. Si la validación externa falla, el sistema registra el error e informa al usuario.
Postcondiciones	Los resultados de la validación externa quedan disponibles para su consulta.
Importancia	Alta
Estado	Aprobado
Comentarios	La validación externa se encola y se procesa en segundo plano, en línea con el objetivo de ejecución asíncrona (OBJ-010).

Tabla x - CU-026 - Solicitar la validación externa de la sesión

CU-027 - Consultar los resultados de la validación externa

Campo	Contenido
ID	CU-027
Nombre	Consultar los resultados de la validación externa
Fuente	RF-029
Actores	Usuario autenticado
Descripción	El usuario consulta los resultados de la validación externa de la sesión: las métricas sobre la cohorte externa, las curvas ROC de los modelos y la matriz de significación del test de DeLong.
Precondiciones	La validación externa de la sesión se ha ejecutado.
Flujo normal	1. El usuario accede a los resultados externos de la sesión. 2. El sistema muestra las métricas, las curvas ROC y la matriz de DeLong.
Flujo alternativo	2a. Si la validación externa no se ha ejecutado, el sistema muestra un mensaje informativo.
Postcondiciones	Los resultados externos quedan visibles para el usuario.
Importancia	Media
Estado	Aprobado
Comentarios	Los resultados solo están disponibles tras completar la validación externa.

Tabla x - CU-027 - Consultar los resultados de la validación externa

CU-028 - Generar el informe PDF de la sesión

Campo	Contenido
ID	CU-028
Nombre	Generar el informe PDF de la sesión
Fuente	RF-030
Actores	Usuario autenticado
Descripción	El usuario genera y descarga el informe PDF consolidado de la sesión, que recoge la configuración, el ranking, la matriz de Wilcoxon, los resultados de la validación externa con su matriz de DeLong y las métricas de explicabilidad y calibración por modelo.
Precondiciones	La sesión pertenece al usuario y dispone de resultados.
Flujo normal	1. El usuario solicita el informe de la sesión. 2. El sistema genera el documento PDF con los resultados consolidados. 3. El sistema lo descarga al equipo del usuario.
Flujo alternativo	2a. Si la generación del informe falla, el sistema informa del error y no descarga el documento.
Postcondiciones	El usuario dispone del informe PDF de la sesión.
Importancia	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-019: el informe se genera desde la sesión.

Tabla x - CU-028 - Generar el informe PDF de la sesión

CU-029 - Renombrar una sesión de entrenamiento

Campo	Contenido
ID	CU-029
Nombre	Renombrar una sesión de entrenamiento
Fuente	RF-031
Actores	Usuario autenticado
Descripción	El usuario modifica el nombre de una de sus sesiones de entrenamiento para identificarla mejor.
Precondiciones	La sesión pertenece al usuario.
Flujo normal	1. El usuario indica el nuevo nombre de la sesión. 2. El sistema comprueba que la sesión pertenece al usuario. 3. El sistema valida que el nuevo nombre no está vacío. 4. El sistema actualiza el nombre de la sesión.
Flujo alternativo	2a. Si la sesión no pertenece al usuario, el sistema deniega el acceso y no modifica nada. 3a. Si el nuevo nombre está vacío, el sistema muestra un mensaje de error y mantiene el nombre anterior.
Postcondiciones	La sesión queda renombrada en el laboratorio del usuario.
Baja	Baja
Estado	Aprobado
Comentarios	El renombrado no afecta a los modelos ni a los resultados de la sesión.

Tabla x - CU-029 - Renombrar una sesión de entrenamiento

CU-030 - Eliminar una sesión de entrenamiento

Campo	Contenido
ID	CU-030
Nombre	Eliminar una sesión de entrenamiento
Fuente	RF-032
Actores	Usuario autenticado
Descripción	El usuario elimina una de sus sesiones de entrenamiento y sus resultados asociados.
Precondiciones	La sesión pertenece al usuario.
Flujo normal	1. El usuario solicita la eliminación de la sesión. 2. El sistema comprueba que la sesión pertenece al usuario. 3. El sistema solicita confirmación de la eliminación. 4. El usuario confirma la eliminación. 5. El sistema elimina la sesión y sus artefactos asociados.
Flujo alternativo	2a. Si la sesión no pertenece al usuario, el sistema deniega el acceso y no elimina nada. 4a. Si el usuario cancela la operación, la sesión se conserva sin cambios.
Postcondiciones	La sesión desaparece del laboratorio del usuario.
Baja	Baja
Estado	Aprobado
Comentarios	La eliminación se realiza únicamente sobre sesiones del propio usuario.

Tabla x - CU-030 - Eliminar una sesión de entrenamiento

12.2.3.4. Módulo de supervisión y administración de la plataforma

Este módulo agrupa los casos de uso del panel de administración, destinados al gobierno de la plataforma: la gestión y supervisión de las cuentas de usuario y de la actividad registrada en el sistema. Su existencia responde a la necesidad de que alguien asuma la responsabilidad sobre el conjunto de la plataforma: conocer quién la utiliza, supervisar la actividad que en ella se genera y disponer de los medios para auditar cualquier incidencia. Todas las operaciones de este módulo están restringidas al rol de administrador, que constituye el único perfil con acceso a ellas, de modo que el mecanismo de roles y control de acceso (RF-006) garantiza que ningún otro usuario puede invocar estos casos de uso. El módulo comprende los casos de uso CU-031 a CU-033 y CU-038, que se corresponden con los requisitos funcionales de administración (RF-033 a RF-035 y RF-040) y se asocian al objetivo de administración de la plataforma (OBJ-011).

El Administrador es, ante todo, un usuario autenticado que conserva todas las capacidades descritas en los demás módulos y que añade las operaciones de gobierno que aquí se recogen. Los tres casos de uso se encadenan mediante relaciones de extensión que reflejan la navegación del administrador desde el listado general hasta el caso concreto: desde el listado de usuarios (CU-031) se accede a las consultas de un usuario (CU-032), y desde ellas al detalle completo de una consulta (CU-033). Esta progresión permite al administrador profundizar gradualmente en la información, partiendo de una visión global de las cuentas y llegando, si lo necesita, al detalle de una consulta individual con su imagen, su resultado y sus metadatos.

CU-031 - Consultar el listado de usuarios

Campo	Contenido
ID	CU-031
Nombre	Consultar el listado de usuarios
Fuente	RF-033
Actores	Administrador
Descripción	El administrador consulta el listado de usuarios registrados en la plataforma, lo que le permite identificar cuentas y comprobar el estado del sistema.
Precondiciones	El administrador tiene una sesión iniciada con rol de administración.
Flujo normal	1. El administrador accede al panel de administración. 2. El sistema verifica el rol de administración del actor. 3. El sistema recupera y muestra el listado de usuarios registrados.
Flujo alternativo	2a. Si el actor no tiene rol de administración, el sistema deniega el acceso y no muestra el listado.
Postcondiciones	El listado de usuarios queda visible para el administrador.
Importancia	Media
Estado	Aprobado
Comentarios	Esta funcionalidad está restringida al rol de administrador.

Tabla x - CU-031 - Consultar el listado de usuarios

CU-032 - Consultar las consultas de un usuario

Campo	Contenido
ID	CU-032
Nombre	Consultar las consultas de un usuario
Fuente	RF-034
Actores	Administrador
Descripción	El administrador consulta el historial de consultas de diagnóstico no eliminadas de un usuario concreto de la plataforma, dentro de su función de supervisión.
Precondiciones	El administrador tiene una sesión iniciada con rol de administración.
Flujo normal	1. El administrador selecciona un usuario del listado. 2. El sistema verifica el rol de administración del actor. 3. El sistema recupera y muestra las consultas de diagnóstico no eliminadas de ese usuario. 4. El sistema registra la operación de auditoría.
Flujo alternativo	2a. Si el actor no tiene rol de administración, el sistema deniega el acceso y no muestra las consultas.
Postcondiciones	El listado de consultas del usuario queda visible para el administrador y la operación queda registrada.
Importancia	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-031. El acceso del administrador a los datos de otro usuario es la excepción al aislamiento definida en RF-005, queda acotado a la función de supervisión, se limita a las consultas no eliminadas (RF-016) y se registra conforme a RNF-006.

Tabla x - CU-032 - Consultar las consultas de un usuario

CU-033 - Ver el detalle de una consulta de un usuario

Campo	Contenido
ID	CU-033
Nombre	Ver el detalle de una consulta de un usuario
Fuente	RF-035
Actores	Administrador
Descripción	El administrador consulta el detalle completo de una consulta de diagnóstico no eliminada de un usuario, incluidos la imagen, el resultado, la confianza y los metadatos asociados.
Precondiciones	El administrador tiene una sesión iniciada con rol de administración.
Flujo normal	1. El administrador selecciona una consulta de las consultas de un usuario. 2. El sistema verifica el rol de administración del actor. 3. El sistema recupera y muestra el detalle completo de la consulta. 4. El sistema registra la operación de auditoría.
Flujo alternativo	2a. Si el actor no tiene rol de administración, el sistema deniega el acceso y no muestra el detalle.
Postcondiciones	El detalle de la consulta queda visible para el administrador y la operación queda registrada.
Importancia	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-032. El acceso del administrador a la imagen de una consulta de otro usuario es la excepción al aislamiento definida en RF-005, queda acotado a la función de supervisión, se limita a las consultas no eliminadas (RF-016) y se registra conforme a RNF-006.

Tabla x - CU-033 - Ver el detalle de una consulta de un usuario

12.2.3.5. Módulo de capacidades transversales de la plataforma

Este módulo agrupa los casos de uso que no pertenecen a un ámbito funcional concreto, sino que afectan a toda la plataforma. Incluye la consulta y la cancelación de los trabajos de la cola (que cubren los diagnósticos, los entrenamientos y las validaciones externas que el sistema ejecuta de forma asíncrona) y la personalización del tema visual de la interfaz. Su carácter transversal se aprecia en que estos casos de uso están disponibles para todo usuario autenticado, con independencia del núcleo funcional en el que trabaje: un profesional puede consultar el estado de sus diagnósticos mientras un investigador supervisa sus entrenamientos, y ambos pueden cancelar un trabajo que haya quedado pendiente en la cola. El módulo comprende los casos de uso CU-034 a CU-036, que se corresponden con los requisitos funcionales transversales (RF-036 a RF-038) y se asocian al objetivo de ejecución asíncrona de tareas (OBJ-010) y, en el caso del tema visual, al objetivo de usabilidad e internacionalización de la interfaz (OBJ-009).

Ninguno de los tres casos de uso presenta relaciones de inclusión o extensión con otros casos de uso de la plataforma: la consulta de la cola, la cancelación de trabajos y el cambio de tema se ejecutan de forma autónoma, sin delegar pasos obligatorios en otros casos ni ampliar ningún flujo base. La cancelación de un trabajo (CU-035), no obstante, se describe de forma coherente con la consulta de la cola (CU-034), porque es desde el panel de la cola donde el usuario identifica el trabajo que desea cancelar, aunque ambas operaciones se representan como casos de uso independientes.

CU-034 - Consultar el estado de la cola de trabajos

Campo	Contenido
ID	CU-034
Nombre	Consultar el estado de la cola de trabajos
Fuente	RF-036
Actores	Usuario autenticado
Descripción	El usuario consulta el panel de la cola de trabajos, que muestra el estado de los trabajos pendientes, en ejecución, completados o fallidos, y que se actualiza automáticamente al menos cada cinco segundos. El panel cubre los diagnósticos, los entrenamientos y las validaciones externas.
Precondiciones	El usuario tiene una sesión iniciada.
Flujo normal	1. El usuario accede al panel de la cola de trabajos. 2. El sistema muestra el estado de los trabajos del usuario. 3. El sistema actualiza el estado de los trabajos automáticamente al menos cada cinco segundos.
Flujo alternativo	2a. Si el usuario no tiene trabajos en la cola, el sistema muestra un panel con el estado vacío.
Postcondiciones	El usuario conoce el estado de sus trabajos en la cola.
Importancia	Media
Estado	Aprobado
Comentarios	El panel cubre todos los tipos de trabajo y muestra su estado.

Tabla x - CU-034 - Consultar el estado de la cola de trabajos

CU-035 - Cancelar un trabajo de la cola

Campo	Contenido
ID	CU-035
Nombre	Cancelar un trabajo de la cola
Fuente	RF-037
Actores	Usuario autenticado
Descripción	El usuario cancela un trabajo pendiente de la cola, que no llega a ejecutarse. La interrupción de un trabajo en ejecución queda fuera de la implementación actual.
Precondiciones	El usuario tiene un trabajo en la cola.
Flujo normal	1. El usuario solicita la cancelación de un trabajo pendiente de la cola. 2. El sistema comprueba que el trabajo sigue pendiente. 3. El sistema lo marca como cancelado y evita su ejecución..
Flujo alternativo	2a. Si el trabajo ya está en ejecución o ha finalizado, el sistema informa de que no puede cancelarse y no lo modifica.
Postcondiciones	El trabajo pendiente queda cancelado o no se modifica si ya ha comenzado o ha finalizado.
Importancia	Media
Estado	Aprobado
Comentarios	La implementación actual solo permite cancelar trabajos pendientes.

Tabla x - CU-035 - Cancelar un trabajo pendiente de la cola

CU-036 - Alternar el tema visual de la interfaz

Campo	Contenido
ID	CU-036
Nombre	Alternar el tema visual de la interfaz
Fuente	RF-038
Actores	Usuario autenticado
Descripción	El usuario alterna entre el tema claro y el tema oscuro de la interfaz, según su preferencia visual.
Precondiciones	El usuario tiene una sesión iniciada.
Flujo normal	1. El usuario activa el cambio de tema en la interfaz. 2. El sistema aplica el tema seleccionado en toda la interfaz de forma inmediata.
Flujo alternativo	N/A
Postcondiciones	La interfaz se muestra con el tema seleccionado.
Importancia	Baja
Estado	Aprobado
Comentarios	La preferencia se aplica de forma inmediata en toda la interfaz.

Tabla x - CU-036 - Alternar el tema visual de la interfaz

CU-037 - Generar el informe PDF del diagnóstico

Campo	Contenido
ID	CU-037
Nombre	Generar el informe PDF del diagnóstico
Fuente	RF-039
Actores	Usuario autenticado
Descripción	El usuario genera y descarga el informe PDF de una consulta de diagnóstico, que recoge la imagen original, la predicción, el nivel de confianza, el modelo empleado y los mapas de explicabilidad.
Precondiciones	La consulta pertenece al usuario y dispone de resultado.
Flujo normal	1. El usuario solicita el informe de la consulta desde su detalle. 2. El sistema comprueba que la consulta pertenece al usuario. 3. El sistema genera el documento PDF. 4. El sistema lo descarga al equipo del usuario.
Flujo alternativo	2a. Si la consulta no pertenece al usuario, el sistema deniega el acceso y no genera el informe. 3a. Si la generación del informe falla, el sistema informa del error y no descarga el documento.
Postcondiciones	La interfaz se muestra con el tema seleccionado.
Importancia	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-012: el informe se genera desde el detalle de la consulta. Cubre el informe individual del diagnóstico; el informe consolidado de la sesión se recoge en el CU-028.

Tabla x - CU-037 - Generar el informe PDF del diagnóstico

CU-038 - Gestionar las cuentas de usuario

Campo	Contenido
ID	CU-038
Nombre	Gestionar las cuentas de usuario
Fuente	RF-040
Actores	Administrador
Descripción	El administrador gestiona las cuentas de usuario de la plataforma: desactiva una cuenta, cambia el rol de un usuario o elimina una cuenta, con registro de la operación.
Precondiciones	El administrador tiene una sesión iniciada con rol de administración.
Flujo normal	1. El administrador selecciona la cuenta sobre la que desea actuar. 2. El sistema verifica el rol de administración del actor. 3. El administrador ejecuta la operación (desactivar, cambiar rol o eliminar). 4. El sistema aplica la operación y registra la auditoría.
Flujo alternativo	2a. Si el actor no tiene rol de administración, el sistema deniega el acceso y no modifica nada.
Postcondiciones	La cuenta queda gestionada y la operación registrada.
Importancia	Media
Estado	Aprobado, pendiente de implementación
Comentarios	Este caso de uso completa la cobertura prevista del objetivo de administración. Las operaciones de gestión de cuentas no están implementadas en el prototipo actual.

Tabla x - CU-038 - Gestionar las cuentas de usuario

CU-039 - Comprobar la limitación de entrenamientos

Campo	Contenido
ID	CU-039
Nombre	Comprobar la limitación de entrenamientos
Fuente	RF-041
Actores	Usuario autenticado
Descripción	Escenario previsto en el que, al superar el límite de entrenamientos simultáneos o encolados, el sistema mantiene el trabajo en espera o lo rechaza según la política definida.
Precondiciones	El usuario ha lanzado o va a lanzar un experimento de entrenamiento.
Flujo normal	1. El usuario lanza un experimento. 2. El sistema comprueba el límite de entrenamientos simultáneos y encolados. 3. Si se supera el límite, el sistema encola el trabajo en espera. 4. El usuario consulta el estado del trabajo en el panel de la cola.
Flujo alternativo	3a. Si no se supera el límite, el entrenamiento se ejecuta de forma inmediata.
Postcondiciones	El trabajo queda encolado o en ejecución conforme al límite de recursos.
Importancia	Media
Estado	Aprobado, pendiente de implementación
Comentarios	La limitación está prevista para proteger la disponibilidad del servicio. La implementación actual no aplica estos límites. Este escenario se relaciona con la consulta de la cola (CU-034).

Tabla x - CU-039 - Comprobar la limitación de entrenamientos

12.2.3.6. Matriz de trazabilidad requisitos – casos de uso

La matriz de trazabilidad que se presenta a continuación relaciona cada requisito funcional con los casos de uso en los que se materializa, de modo que pueda verificarse en ambas direcciones que todos los requisitos quedan cubiertos por al menos un caso de uso y que todos los casos de uso están respaldados por al menos un requisito. Esta relación permite revisar la coherencia entre los dos niveles de especificación del capítulo: los requisitos funcionales declaran qué debe hacer el sistema y los casos de uso concretan cómo se realiza cada capacidad, por lo que no debe existir ningún requisito sin caso de uso que lo materialice ni ningún caso de uso sin requisito que lo justifique.

Como se señaló al especificar los casos de uso, la mayoría de los requisitos funcionales se materializan en un caso de uso concreto del mismo módulo, lo que se refleja en las matrices de forma directa: cada caso de uso marca con una X la celda correspondiente a su requisito fuente. No obstante, dos requisitos de carácter transversal no dan lugar a un caso de uso propio: el aislamiento de datos entre usuarios (RF-005) y los roles y el control de acceso (RF-006). Estos requisitos condicionan la ejecución del resto de las interacciones y se reflejan en las matrices marcando las celdas de los casos de uso a los que afectan, las operaciones sobre consultas de diagnóstico, sesiones de entrenamiento y datos de administración, sin constituir por ello interacciones independientes. Para facilitar la lectura, la matriz se divide en cinco tablas, una por módulo funcional, de modo que cada tabla relaciona los requisitos del módulo con los casos de uso que los materializan.

Matriz de trazabilidad del módulo de gestión del acceso y de la cuenta

En este módulo, cada requisito se materializa en un único caso de uso: el registro (RF-001) se concreta en la creación de la cuenta (CU-001), el acceso (RF-002) en el inicio de sesión (CU-002), el cierre de sesión (RF-003) en la finalización de la sesión (CU-003) y el cambio de idioma (RF-004) en la adaptación de la interfaz (CU-004). Los requisitos transversales de aislamiento y de roles afectan a la totalidad de las interacciones de la plataforma, por lo que también se reflejan en este módulo.

RF	Denominación	CU-001	CU-002	CU-003	CU-004
RF-001	Registro de usuario	X			
RF-002	Inicio de sesión		X		
RF-003	Cierre de sesión			X	
RF-004	Cambio de idioma de la interfaz				X
RF-005	Aislamiento de datos entre usuarios	X		X	
RF-006	Roles y control de acceso		X	X	

Tabla x - Matriz de trazabilidad RF-CU del módulo de gestión del acceso y de la cuenta

Matriz de trazabilidad del módulo de interfaz de diagnóstico asistido

En este módulo, cada caso de uso del bloque de consulta de diagnóstico se corresponde con su requisito fuente: el acceso al panel (CU-005) con RF-007, la subida de la radiografía (CU-006) con RF-008, la selección de la arquitectura (CU-007) con RF-009, la solicitud del diagnóstico (CU-008) con RF-010, la visualización del resultado (CU-009) con RF-011 y la visualización de los mapas de explicabilidad (CU-010) con RF-012. Los casos de uso del bloque de historial se corresponden con los requisitos de gestión del historial: la consulta del listado (CU-011) con RF-013, el detalle (CU-012) con RF-014, el renombrado (CU-013) con RF-015 y la eliminación (CU-014) con RF-016. El aislamiento de datos (RF-005) se aplica a todas las operaciones sobre las consultas del usuario.

RF	Denominación	CU-005	CU-006	CU-007	CU-008	CU-009	CU-010	CU-011	CU-012	CU-013	CU-014	CU-037
RF-007	Acceso al panel de diagnóstico	X										
RF-008	Subida de una radiografía de tórax		X									
RF-009	Selección de la arquitectura para el diagnóstico			X								
RF-010	Solicitud de un diagnóstico				X							
RF-011	Visualización del resultado del diagnóstico					X						
RF-012	Visualización de los mapas de explicabilidad						X					
RF-013	Consultar el historial de consultas							X				
RF-014	Visualización del detalle de una consulta del historial								X			
RF-015	Renombrar una consulta del historial									X		
RF-016	Eliminar una consulta del historial										X	
RF-039	Generación del informe PDF del diagnóstico											X
RF-005	Aislamiento de datos entre usuarios					X	X	X	X	X	X	X

Tabla x - Matriz de trazabilidad RF-CU del módulo de interfaz de diagnóstico asistido

Matriz de trazabilidad del módulo de laboratorio de experimentación MLOps

En este módulo, los dieciséis casos de uso se corresponden de forma biunívoca con sus requisitos fuente, desde el acceso al laboratorio (CU-015 con RF-017) hasta la eliminación de la sesión (CU-030 con RF-032). El aislamiento de datos (RF-005) se aplica a las operaciones sobre las sesiones de entrenamiento del usuario.

RF	Denominación	CU-015	CU-016	CU-017	CU-018	CU-019	CU-020	CU-021	CU-022
RF-017	Acceso al laboratorio de entrenamiento	X							
RF-018	Conversación con el asistente para configurar un experimento		X						
RF-019	Selección de la carpeta del dataset			X					
RF-020	Lanzamiento de un experimento de entrenamiento				X				
RF-021	Consulta de las sesiones de entrenamiento					X			
RF-022	Consulta de los resultados de un modelo de la sesión						X		
RF-023	Visualización de los mapas de explicabilidad de un modelo							X	
RF-024	Consulta del ranking de modelos de la sesión								X

Tabla x - Matriz de trazabilidad RF-CU del módulo de laboratorio de experimentación MLOps (I)

RF	Denominación	CU-023	CU-024	CU-025	CU-026	CU-027	CU-028	CU-029	CU-030	CU-039
RF-025	Consulta de la comparativa estadística de la sesión	X								
RF-026	Solicitud del recálculo de la comparativa estadística		X							
RF-027	Ejecución del análisis de explicabilidad de un modelo			X						
RF-028	Solicitud de la validación externa de la sesión				X					
RF-029	Consulta de los resultados de la validación externa					X				
RF-030	Generación del informe PDF de la sesión						X			
RF-031	Renombrar una sesión de entrenamiento							X		
RF-032	Eliminar una sesión de entrenamiento								X	
RF-041	Limitación de entrenamientos simultáneos y de la cola									X
RF-005	Aislamiento de datos entre usuarios	X	X	X	X	X	X	X	X	

Tabla x - Matriz de trazabilidad RF-CU del módulo de laboratorio de experimentación MLOps (II)

Matriz de trazabilidad del módulo de supervisión y administración de la plataforma

En este módulo, los cuatro casos de uso se corresponden con sus requisitos fuente y se refleja además la incidencia del requisito de roles y control de acceso (RF-006), que restringe estas operaciones al administrador. El acceso del administrador a los datos de los usuarios supervisados es la excepción al aislamiento de datos (RF-005) definida en este módulo: no contradice el requisito porque queda limitada al rol y a la función de supervisión, y porque el administrador solo puede examinar los datos a los que su rol le da acceso.

RF	Denominación	CU-031	CU-032	CU-033	CU-038
RF-033	Consulta del listado de usuarios	X			
RF-034	Consulta de las consultas de un usuario		X		
RF-035	Visualización del detalle de una consulta de un usuario			X	
RF-040	Gestión de cuentas de usuario por el administrador				X
RF-006	Roles y control de acceso	X	X	X	X

Tabla x - Matriz de trazabilidad RF-CU del módulo de supervisión y administración de la plataforma

Matriz de trazabilidad del módulo de capacidades transversales de la plataforma

En este módulo, los tres casos de uso se corresponden con sus requisitos fuente: la consulta de la cola (CU-034) con RF-036, la cancelación de trabajos (CU-035) con RF-037 y el cambio del tema visual (CU-036) con RF-038.

RF	Denominación	CU-034	CU-035	CU-036
RF-036	Consulta del estado de la cola de trabajos	X		
RF-037	Cancelación de un trabajo de la cola		X	
RF-038	Cambio del tema visual de la interfaz			X

Tabla x - Matriz de trazabilidad RF-CU del módulo de capacidades transversales de la plataforma

El conjunto de las cinco tablas permite revisar la correspondencia entre los requisitos funcionales y los casos de uso. La mayoría de los requisitos se relaciona con un caso de uso concreto, mientras que RF-005 y RF-006 actúan como condiciones transversales. Los únicos requisitos que no poseen un caso de uso propio son los de carácter transversal, el aislamiento de datos (RF-005) y los roles y el control de acceso (RF-006), cuya cobertura se refleja en las matrices mediante las celdas marcadas sobre los casos de uso a los que condicionan. Los requisitos añadidos para completar la cobertura de objetivos (RF-039 informe del diagnóstico, RF-040 gestión de cuentas y RF-041 limitación de entrenamientos) se materializan en los casos de uso CU-037, CU-038 y CU-039, incorporados a las matrices de sus módulos. De este modo, la trazabilidad entre ambos niveles de especificación queda garantizada en todo el capítulo, de forma análoga a la matriz de trazabilidad entre requisitos y objetivos del sistema presentada en la sección 12.1.

Capítulo 13. Identificación de subsistemas de análisis

El capítulo 12 organiza los requisitos funcionales y los casos de uso en seis módulos: autenticación y cuenta, diagnóstico clínico, historial de consultas, laboratorio MLOps, administración y capacidades transversales. Este capítulo conserva esa organización mediante la nomenclatura de subsistemas utilizada en los capítulos posteriores. No introduce una descomposición distinta ni toma decisiones de diseño; establece la correspondencia entre los módulos funcionales y los identificadores que se utilizarán en el análisis dinámico y en las pruebas.

Módulo funcional del capítulo 12	Subsistema de análisis	Contenido principal
Módulo de autenticación y cuenta	SS-001, Subsistema de acceso y gestión de cuentas	Registro, inicio y cierre de sesión, cambio de idioma, aislamiento de datos y control de roles.
Módulo de diagnóstico clínico	SS-002, Subsistema de diagnóstico asistido	Acceso al panel, carga de radiografías, diagnóstico, resultados, explicabilidad e informes de las consultas.
Módulo de historial de consultas	SS-003, Subsistema de gestión del historial de consultas	Consulta, detalle, renombrado y eliminación de consultas propias.
Módulo de laboratorio MLOps	SS-004, Subsistema de laboratorio de experimentación MLOps	Configuración, entrenamiento, resultados, explicabilidad, comparación, validación externa, informes de experimentación y limitación prevista de entrenamientos.
Módulo de administración	SS-005, Subsistema de supervisión y administración	Consulta de usuarios, supervisión de consultas y gestión prevista de cuentas por parte del administrador.
Módulo transversal	SS-006, Subsistema de capacidades transversales	Consulta y cancelación de trabajos de la cola y personalización del tema visual.

Tabla x - Identificación de subsistemas de análisis

La cola de trabajos es un mecanismo común del sistema del que dependen el diagnóstico, el laboratorio y la validación externa. Por ese motivo, su relación con SS-002 y SS-004 se tratará como una dependencia entre subsistemas, no como una capacidad exclusiva de un único ámbito funcional. El cambio de tema visual se mantiene en SS-006 porque pertenece al módulo transversal definido en el capítulo 12 y no forma parte de un flujo clínico, de laboratorio o de administración.

Esta organización permite conservar la trazabilidad sin duplicar la especificación de requisitos ni los casos de uso. El capítulo 14 utiliza estos identificadores para ordenar las secuencias de interacción y el capítulo 15 comprueba la correspondencia entre requisitos, casos de uso y subsistemas. Las decisiones sobre componentes, tecnologías y distribución de responsabilidades internas se reservan para los capítulos de diseño.

Por tanto, los subsistemas de este capítulo deben entenderse como identificadores estables para organizar el análisis. La descripción detallada de cada capacidad, sus condiciones y sus flujos se encuentra en el capítulo 12, que es la referencia principal para los requisitos y los casos de uso.

Capítulo 14. Modelo de dominio y entidades del sistema

En este capítulo se identifican las entidades que intervienen en vitalXAI, la información que representan y las relaciones que mantienen. El resultado es un modelo conceptual, no una descripción de las tablas o ficheros que se utilizan en la implementación. El modelo de clases complementa los casos de uso del capítulo 12 y los subsistemas del capítulo 13: aquellos describen las interacciones, estos agrupan responsabilidades y el modelo de dominio organiza las entidades sobre las que se apoyan (Larman, 2004).

El modelado del dominio se organiza en dos secciones complementarias. La primera, dedicada a la estructura estática, presenta las entidades del negocio, sus atributos y sus relaciones. La segunda recoge el comportamiento dinámico mediante secuencias de interacción vinculadas a los casos de uso del capítulo 12. Estas secuencias no amplían el modelo de entidades: muestran cómo el actor utiliza el sistema y cómo este responde. La separación permite distinguir el modelo conceptual de los flujos de comportamiento sin perder la relación entre ambos.

14.1. Estructura estática de las entidades del sistema

El diagrama de clases del sistema recoge las once clases de negocio identificadas durante el análisis del dominio de vitalXAI: Usuario, ConsultaDiagnostico, MapaXAI, SesionExperimentacion, ModeloEntrenado, ResultadoModelo, ResultadoPliegue, ValidacionExterna, ResultadoValidacionExterna, ComparacionModelos y TrabajoCola. Estas clases representan las entidades centrales que la plataforma debe gestionar a lo largo de su ciclo de vida. Las nuevas clases de resultados hacen explícita la diferencia entre los valores agregados de un modelo, sus métricas por pliegue, los resultados individuales de una validación externa y las comparaciones estadísticas entre dos modelos.

A nivel de modelado se han adoptado dos simplificaciones. En primer lugar, las arquitecturas y los hiperparámetros del experimento, el número de épocas, el tamaño de lote y la tasa de aprendizaje, se asocian en el modelo conceptual a ModeloEntrenado, aunque la configuración se guarda realmente en los ficheros de la sesión. Esta decisión evita introducir una clase separada para una configuración que, en el flujo del proyecto, termina vinculada a cada modelo entrenado. La ruta del dataset se asocia a SesionExperimentacion, porque una sesión agrupa los modelos entrenados sobre un mismo conjunto de datos y la ruta se obtiene mediante CU-017. En segundo lugar, Usuario actúa como raíz conceptual: las entidades se relacionan con un propietario de forma directa o a través de la sesión o consulta correspondiente. En la implementación actual, las sesiones y los resultados del laboratorio se conservan como carpetas sin user_id, por lo que esa propiedad conceptual no tiene el mismo respaldo físico en todas las entidades.

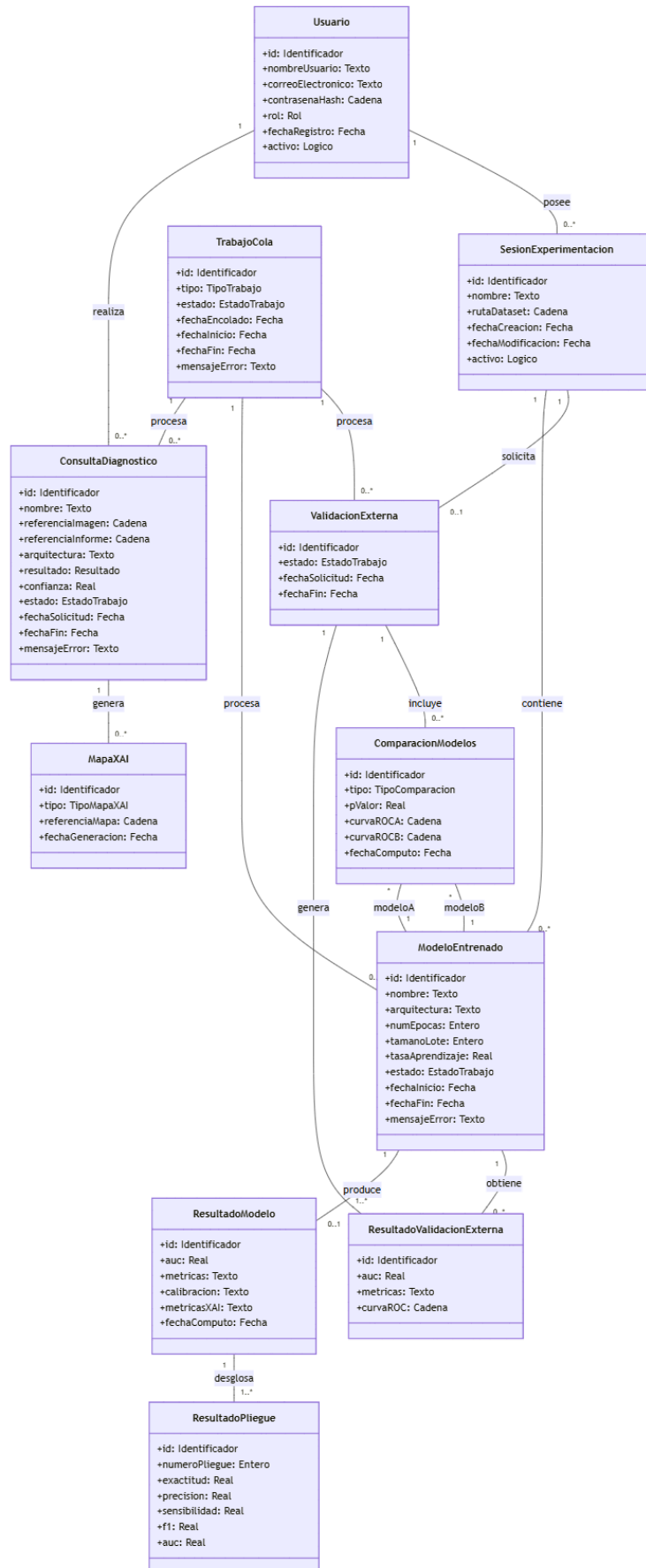


Ilustración x - Diagrama de clases del sistema

La clase Usuario representa la identidad de cada cuenta, incluido su nombre de usuario, correo electrónico, hash de contraseña, rol y, en el modelo conceptual, su estado y fechas relevantes. En la persistencia actual no todos esos atributos tienen una columna propia, como se detalla en el capítulo 19. Un usuario puede realizar múltiples consultas de diagnóstico, representadas por ConsultaDiagnostico, que reúne la referencia a la imagen, el informe PDF, la arquitectura seleccionada, el resultado, la confianza y el estado del trabajo. Cada consulta puede generar mapas de explicabilidad, representados por MapaXAI, cuyo atributo tipo distingue el método empleado.

Por otra parte, un usuario puede poseer múltiples sesiones de experimentación, representadas por SesionExperimentacion, que agrupa los modelos entrenados sobre un mismo dataset. ModeloEntrenado representa cada modelo de la sesión, con su arquitectura, sus hiperparámetros, su estado y sus fechas de ejecución. Cuando finaliza el entrenamiento, el modelo produce un ResultadoModelo con los valores agregados de rendimiento, calibración y explicabilidad. Los valores de validación cruzada de cada pliegue se representan mediante ResultadoPliegue, asociado al resultado del modelo y con las cinco métricas exigidas por RF-022. La validación cruzada y sus posibles sesgos de estimación se tratan como decisiones metodológicas del proyecto, no como una garantía de generalización (Varma & Simon, 2006).

ValidacionExterna representa una ejecución de evaluación sobre una cohorte independiente. Sus resultados se desglosan por modelo mediante ResultadoValidacionExterna, que reúne las métricas y la curva ROC de cada modelo. Las comparaciones entre dos modelos se representan mediante ComparacionModelos. Esta entidad recoge el tipo de contraste, las curvas comparadas y el p-valor correspondiente, incluido el del test de DeLong cuando se comparan curvas ROC (DeLong, DeLong, & Clarke-Pearson, 1988). De este modo, el p-valor se asocia a la comparación entre dos modelos dentro de una misma ejecución y no a la validación considerada de forma aislada.

Finalmente, TrabajoCola representa los trabajos asíncronos de diagnóstico, entrenamiento y validación externa, que se procesan sin bloquear la interfaz conforme al requisito RF-036.

El modelo de relaciones expresa el aislamiento de datos previsto en RF-005: todas las entidades de negocio parten conceptualmente de Usuario, de modo que cada consulta, sesión, modelo y resultado quedan asociados a un propietario en el modelo. Las relaciones entre SesionExperimentacion, ModeloEntrenado y ResultadoModelo reflejan el ciclo de vida de los experimentos. ResultadoPliegue depende de ResultadoModelo, mientras que ResultadoValidacionExterna depende de una ejecución de ValidacionExterna y se asocia al modelo evaluado. ComparacionModelos se vincula con los dos modelos que intervienen en el contraste y con la ejecución de validación externa cuando el contraste corresponde al test de DeLong. En la persistencia real, sin embargo, las sesiones y sus resultados se almacenan en carpetas sin una relación física con users; esta diferencia entre propiedad conceptual y garantía implementada queda documentada en el capítulo 19.

El diagrama de clases presentado en la figura 8 constituye el modelo estático de dominio de vitalXAI. Las secuencias de la sección 14.2 lo complementan desde la perspectiva del comportamiento, mostrando cómo el actor utiliza el sistema en los casos de uso del capítulo 12.

14.2. Comportamiento dinámico del sistema y DSS

El modelo estático no basta para describir el desarrollo de las interacciones. Por ello, la sección 14.2 utiliza secuencias asociadas a los casos de uso del capítulo 12. Estas secuencias complementan el modelo de entidades y muestran el intercambio observable entre el actor y el sistema, sin convertir los mecanismos internos en participantes externos (Larman, 2004).

En la mayoría de las secuencias intervienen dos participantes: el actor que inicia la acción y el sistema en su conjunto. Los mensajes del actor representan acciones sobre la interfaz, mientras que el sistema procesa la información y devuelve una respuesta. Cuando el flujo requiere una tarea asíncrona, el encolado, el procesamiento del trabajo y la actualización del resultado se representan como operaciones internas del sistema. El worker no aparece como participante porque forma parte de ese sistema y no es un actor externo. La secuencia de configuración del experimento incorpora además el modelo de lenguaje como participante externo de la integración conversacional.

Las secuencias se presentan organizadas por subsistemas, siguiendo la estructura definida en el capítulo 13, de forma que resulte sencillo localizar el comportamiento de la plataforma para cada funcionalidad. Para mantener la claridad de la presentación, cada diagrama recoge únicamente el flujo habitual de la acción, es decir, el camino que sigue la interacción cuando todo funciona de forma correcta. Los comportamientos alternativos, como los mensajes de error o la cancelación de una operación, se describen en la especificación detallada de cada caso de uso del capítulo 12 (sección 12.2.3), donde se documentan los flujos normal y alternativo de cada caso.

14.2.1. SS-001 – Subsistema de Acceso y Gestión de Cuentas

Este subapartado recoge las secuencias de interacción correspondientes a los casos de uso del subsistema de acceso y gestión de cuentas, descrito en el capítulo 13. Todos ellos tienen como actor al visitante o al usuario autenticado según corresponda, e interactúan con el sistema para gestionar la identidad y el acceso a la plataforma: el registro de una nueva cuenta (CU-001), el inicio de sesión (CU-002), el cierre de sesión (CU-003) y el cambio del idioma de la interfaz (CU-004). Se presentan a continuación, precedidos cada uno de una breve descripción de la interacción que modelan.

Secuencia CU-001: Registrarse

El registro es la puerta de entrada de la plataforma y la primera interacción que un visitante mantiene con ella. El visitante accede al formulario de registro desde la página de inicio de sesión, introduce su nombre de usuario, nombre, apellidos, correo electrónico y contraseña, y los envía al sistema. Este valida el formato de los datos recibidos y la fortaleza de la contraseña, comprueba que no exista ninguna cuenta previa con el mismo nombre de usuario o correo electrónico y, si todo es correcto, aplica un hash a la contraseña y crea el registro del nuevo usuario. Finalmente, el sistema confirma el alta y redirige al visitante a la página de inicio de sesión.

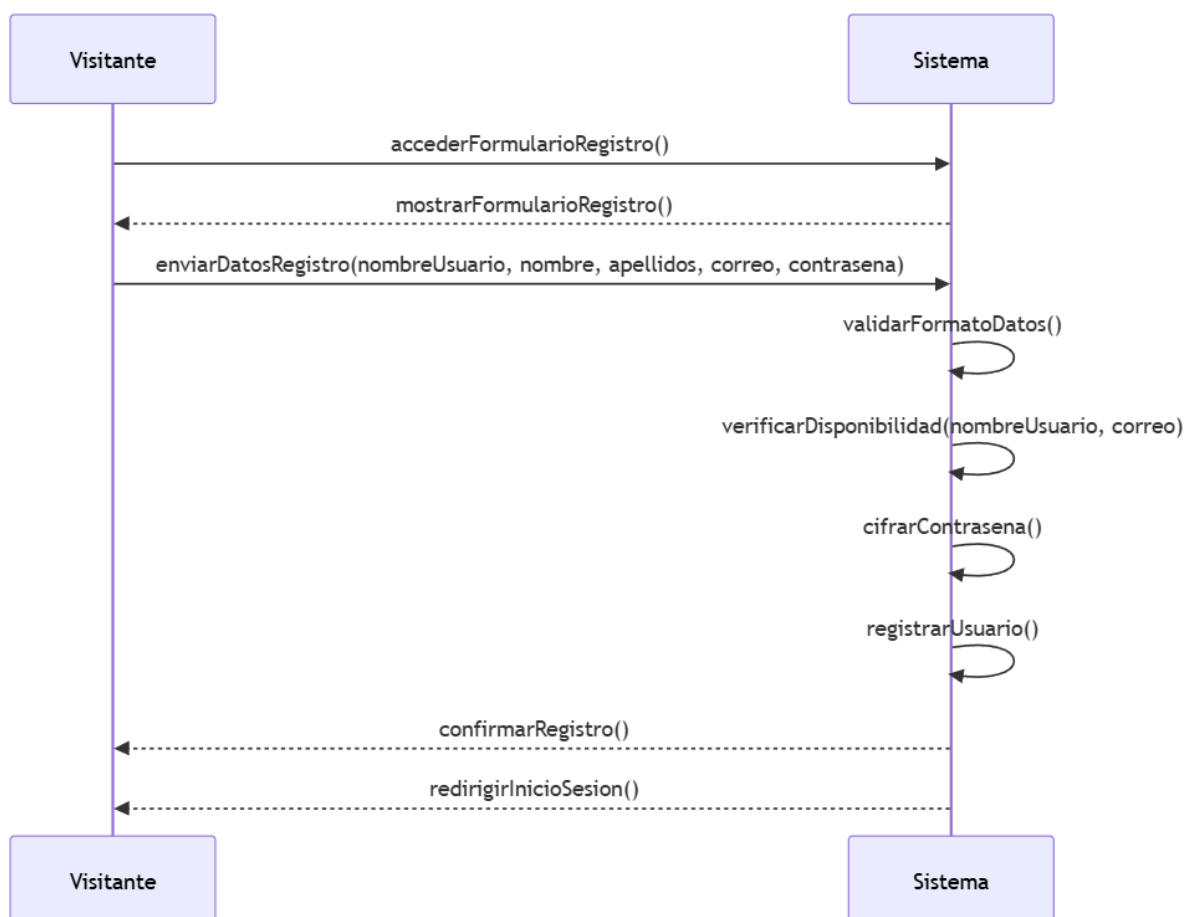


Ilustración x - Secuencia de interacción del registro de una cuenta (CU-001)

Secuencia CU-002: Iniciar sesión

El inicio de sesión es el punto de acceso a toda la funcionalidad privada de la plataforma. El visitante accede al formulario de inicio de sesión, introduce su nombre de usuario y contraseña y las envía al sistema. Este verifica que la contraseña introducida coincida con el hash almacenado para ese nombre de usuario y, si la verificación es correcta, genera el token de acceso y el token de refresco, los establece en cookies seguras y redirige al usuario a su panel, concediéndole acceso a las áreas privadas.

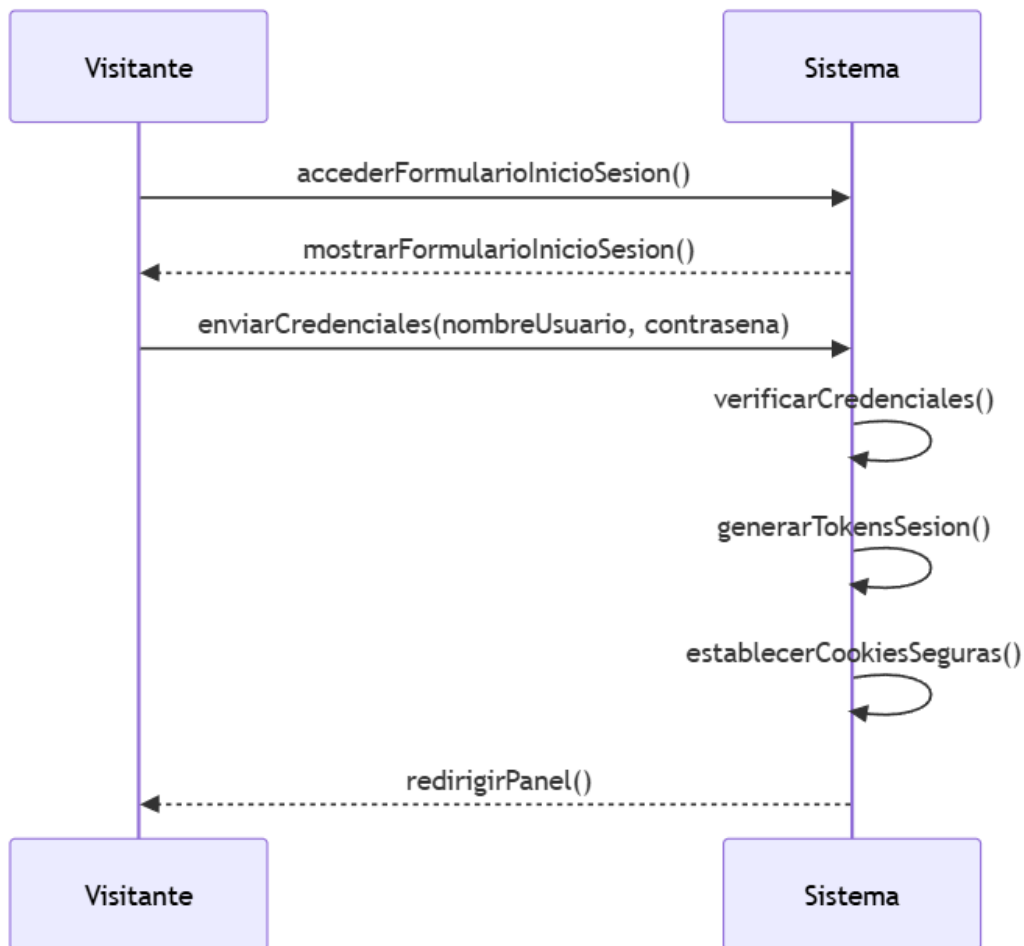


Ilustración x - Secuencia de interacción del inicio de sesión (CU-002)

Secuencia CU-003: Cerrar sesión

El cierre de sesión completa el ciclo de acceso y es especialmente relevante en entornos de uso compartido. El usuario autenticado selecciona la opción de cerrar sesión y el sistema revoca el token de refresco asociado a su sesión, elimina las cookies de sesión del navegador y redirige al usuario a la página de inicio de sesión. A partir de ese momento, cualquier intento de acceder a las áreas privadas desde el mismo equipo es rechazado hasta que se vuelva a autenticar.

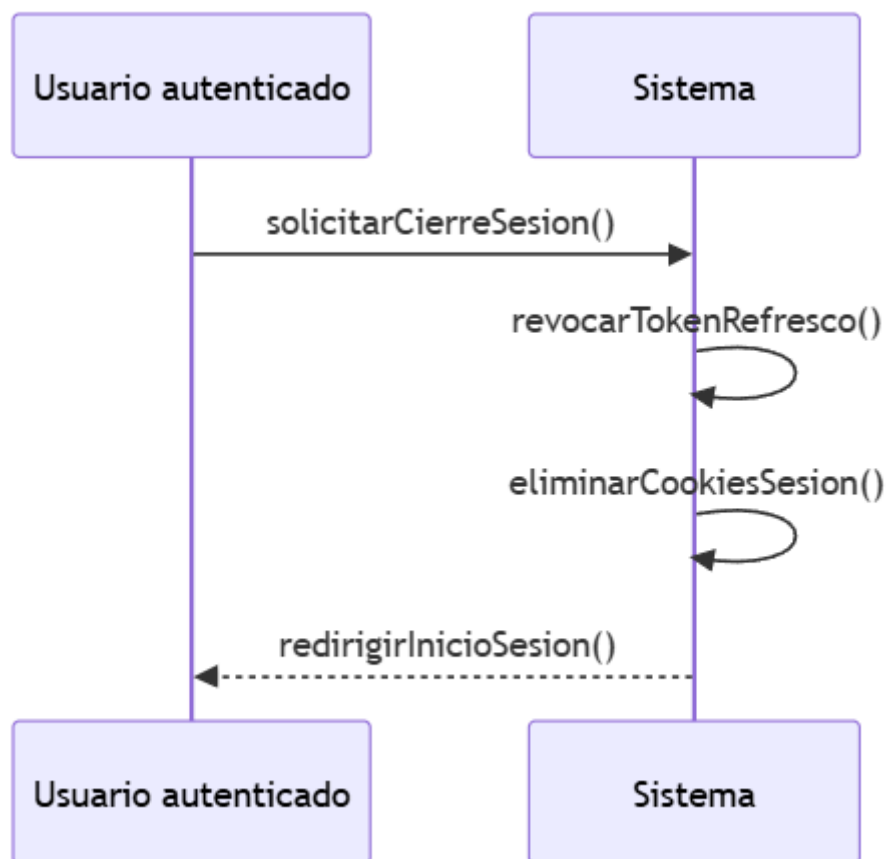


Ilustración x - Secuencia de interacción del cierre de sesión (CU-003)

Secuencia CU-004: Cambiar el idioma de la interfaz

El cambio de idioma es una interacción transversal disponible tanto para el visitante en el área pública como para el usuario autenticado en el área privada. El actor selecciona el idioma deseado en el selector de idioma de la interfaz y el sistema guarda la preferencia seleccionada, de modo que persista durante la sesión, y aplica las traducciones correspondientes en toda la interfaz sin necesidad de recargar la página, sin interrumpir el estado de navegación y, cuando procede, también en los informes generados y en el asistente conversacional.

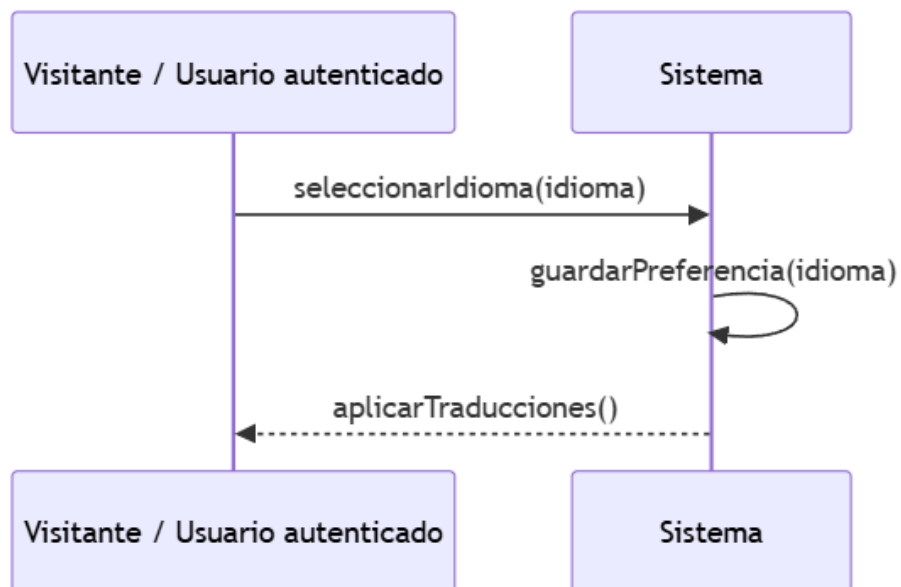


Ilustración x - Secuencia de interacción del cambio de idioma (CU-004)

14.2.2. SS-002 – Subsistema de Diagnóstico Asistido

Este subapartado recoge las secuencias de interacción correspondientes a los casos de uso del subsistema de diagnóstico asistido, descrito en el capítulo 13. Este subsistema agrupa el flujo clínico de la plataforma: el acceso al panel de diagnóstico (CU-005), la subida de una radiografía de tórax (CU-006), la selección de la arquitectura del modelo (CU-007), la solicitud del diagnóstico (CU-008), la visualización del resultado (CU-009) y la visualización de los mapas de explicabilidad (CU-010). Todos ellos tienen como actor al usuario autenticado, que opera sobre su propia consulta en curso. Las tres primeras secuencias preparan la consulta sobre la que se ejecutará el diagnóstico, mientras que las tres últimas completan el flujo clínico desde la solicitud hasta la presentación de los resultados. Se presentan a continuación, precedidos cada uno de una breve descripción de la interacción que modelan.

Secuencia CU-005: Acceder al panel de diagnóstico

El panel de diagnóstico es el punto de partida de toda la actividad clínica del usuario y el acceso a él constituye el primer contacto del usuario autenticado con la funcionalidad clínica de la plataforma. El usuario accede al panel desde la navegación principal y el sistema valida que su sesión se encuentra activa. Si la sesión es válida, el sistema carga el panel de diagnóstico y lo muestra, quedando disponibles la carga de la radiografía, el selector de modelos y el acceso al historial.

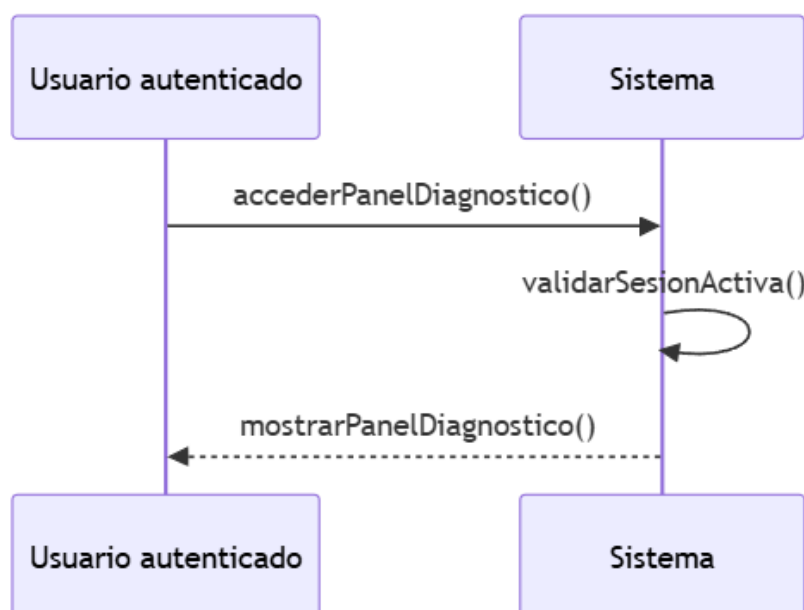


Ilustración x - Secuencia de interacción del acceso al panel de diagnóstico (CU-005)

Secuencia CU-006: Subir una radiografía de tórax

El primer paso de cualquier diagnóstico es incorporar la imagen al sistema. El usuario selecciona el archivo de la radiografía desde su equipo y el sistema valida que el formato sea un tipo de imagen admitido (JPEG o PNG) y que el tamaño no supere el límite máximo fijado. Una vez validada, la imagen se almacena de forma temporal en el servidor y queda asociada a la consulta en curso, de modo que el usuario puede continuar con la selección del modelo.

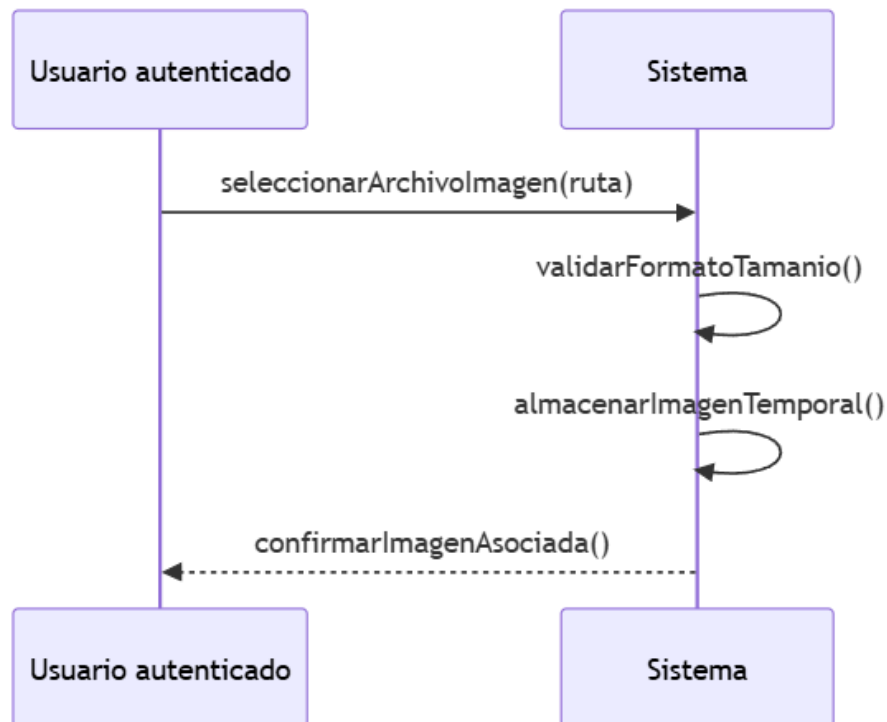


Ilustración x - Secuencia de interacción de la subida de una radiografía (CU-006)

Secuencia CU-007: Seleccionar la arquitectura para el diagnóstico

El resultado del diagnóstico depende del modelo empleado, por lo que el usuario debe poder elegir la arquitectura con la que desea realizar la consulta. El usuario despliega el selector de modelos del panel y el sistema muestra la lista de arquitecturas de deep learning disponibles. El usuario selecciona la arquitectura deseada y esta queda asociada a la consulta en curso.

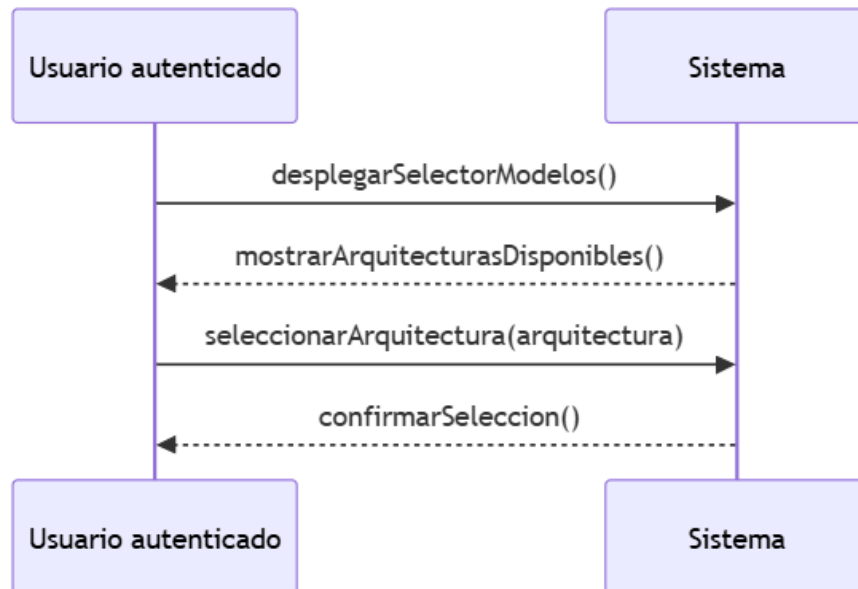


Ilustración x - Secuencia de interacción de la selección de la arquitectura (CU-007)

Secuencia CU-008: Solicitar un diagnóstico

La solicitud del diagnóstico es el punto de entrada del flujo clínico. El usuario envía la petición con la imagen cargada y el modelo seleccionado, y el sistema valida que la petición sea correcta. Para evitar bloquear la interfaz durante el procesamiento, el sistema encola el trabajo de diagnóstico y lo procesa en segundo plano: el worker realiza la predicción, genera los mapas de explicabilidad, genera el informe de la consulta y guarda la consulta. Cuando el trabajo finaliza, el sistema notifica al usuario la disponibilidad del resultado. La presentación del resultado, de los mapas y del informe se describe en las secuencias CU-009, CU-010 y CU-037.

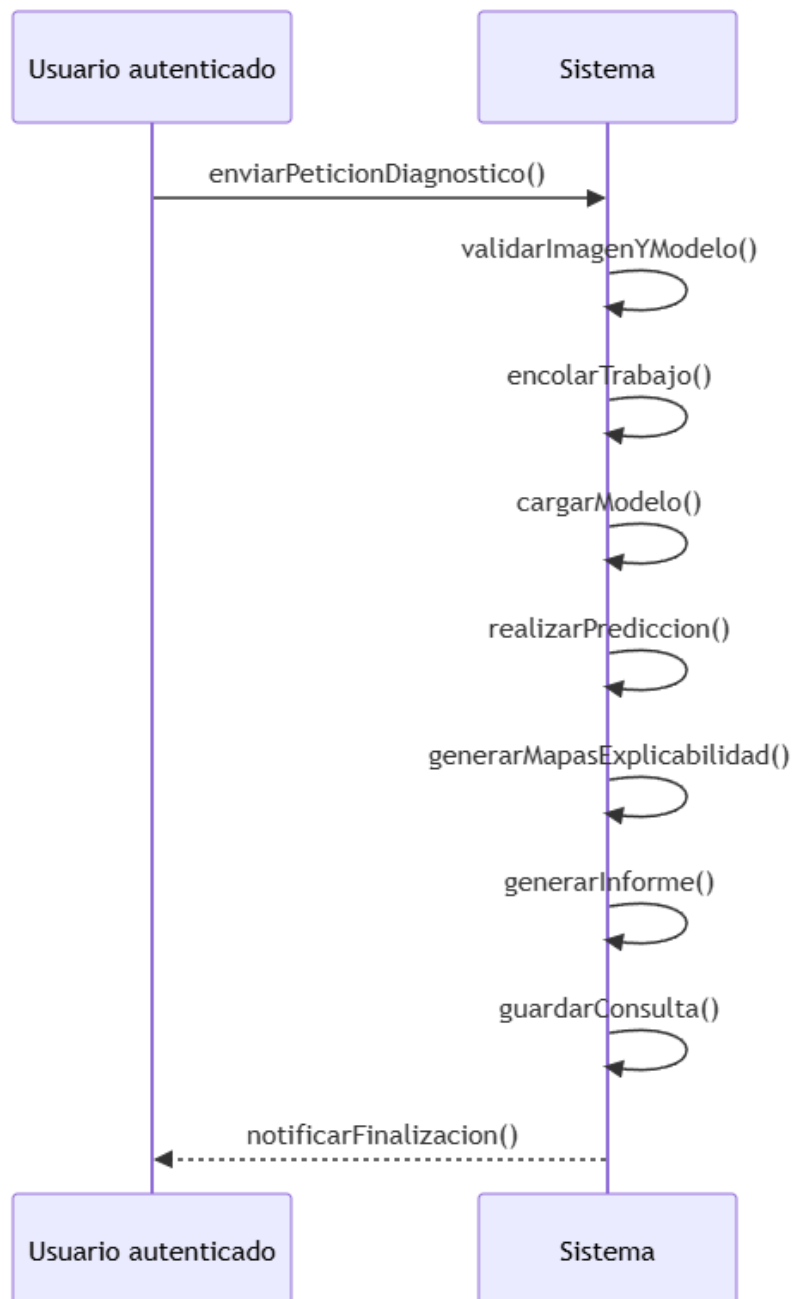


Ilustración x - Secuencia de interacción de la solicitud de un diagnóstico (CU-008)

Secuencia CU-009: Visualizar el resultado del diagnóstico

Cuando el trabajo de diagnóstico finaliza, el usuario consulta el estado de su consulta y el sistema comprueba que ha pasado a estado completado. Si es así, el sistema muestra el resultado de la consulta: la predicción (PNEUMONIA o NORMAL), el nivel de confianza asociado y el modelo empleado, presentados de forma clara y sin tecnicismos. El resultado queda registrado en el historial, de modo que pueda recuperarse en cualquier momento.

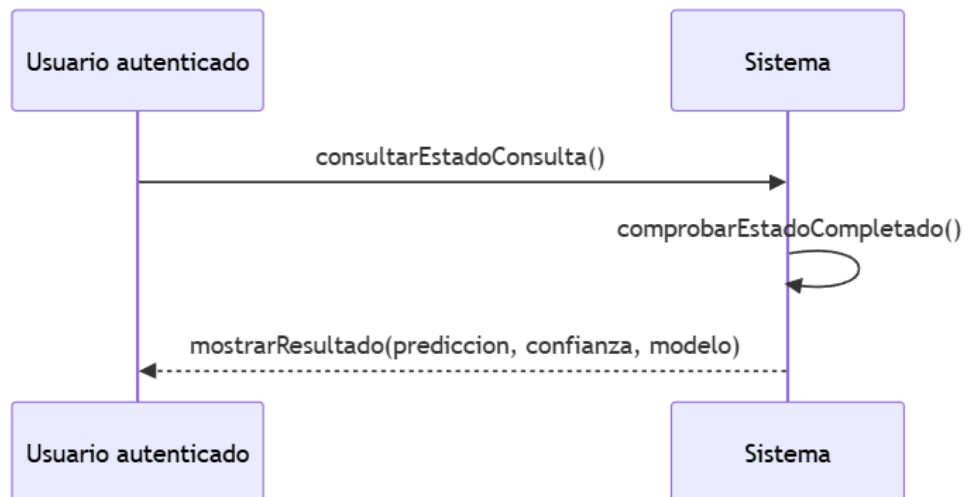


Ilustración x - Secuencia de interacción de la visualización del resultado (CU-009)

Secuencia CU-010: Visualizar los mapas de explicabilidad

Cuando la consulta está completada, el usuario la selecciona para inspeccionar sus explicaciones y el sistema comprueba que la consulta pertenece al usuario. El sistema muestra entonces el mosaico con la radiografía original y los mapas de explicabilidad generados: Saliency Maps, SmoothGrad y Grad-CAM para las arquitecturas convolucionales, o mapas de atención para las arquitecturas Transformer (Simonyan, Vedaldi, & Zisserman, 2014; Smilkov & al., 2017; Selvaraju & al., 2017; Chefer, Gur, & Wolf, 2021). Estas visualizaciones sirven para inspeccionar las regiones resaltadas, pero no prueban por sí solas la validez clínica de la predicción.

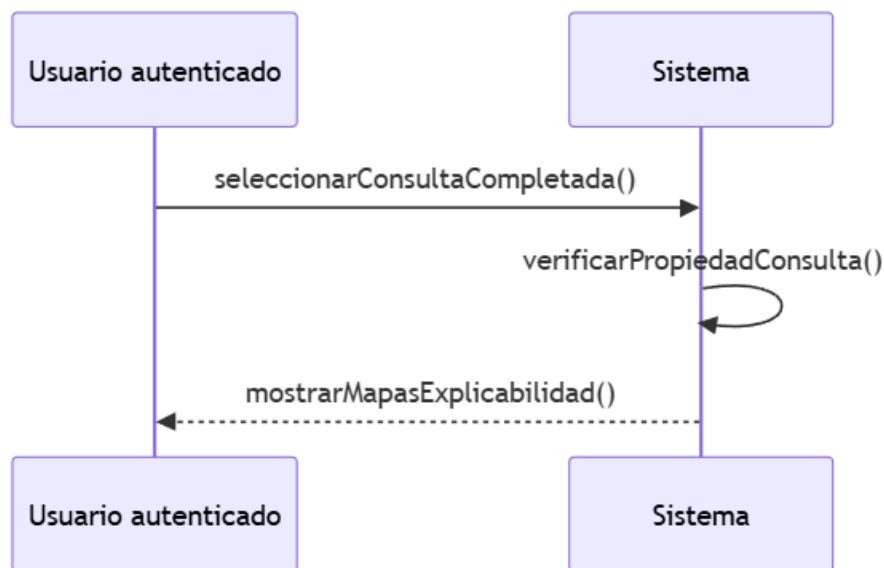


Ilustración x - Secuencia de interacción de la visualización de los mapas de explicabilidad (CU-010)

Secuencia CU-037: Generar el informe PDF del diagnóstico

Un diagnóstico asistido no es útil si el profesional no puede comprobar los motivos de la decisión del modelo. Cuando la consulta está completada, el usuario la selecciona para inspeccionar sus explicaciones y el sistema comprueba que la consulta pertenece al usuario, garantizando el aislamiento de datos. El sistema muestra entonces el mosaico con la radiografía original y los mapas de explicabilidad generados: Saliency Maps, SmoothGrad y Grad-CAM para las arquitecturas convolucionales, o mapas de atención para las arquitecturas Transformer.

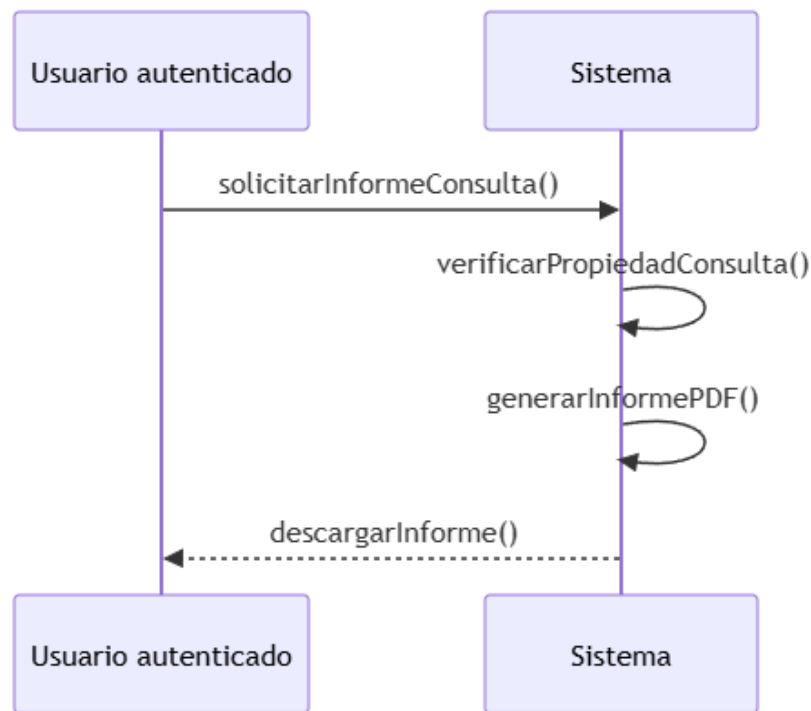


Ilustración x - Secuencia de interacción de la generación del informe PDF del diagnóstico (CU-037)

14.2.3. SS-003 – Subsistema de Gestión del Historial

Este subapartado recoge las secuencias de interacción correspondientes a los casos de uso del subsistema de gestión del historial, descrito en el capítulo 13. Este subsistema agrupa la recuperación y la gestión de las consultas de diagnóstico ya realizadas: la consulta del listado del historial (CU-011), la visualización del detalle de una consulta (CU-012), el renombrado de una consulta (CU-013) y su eliminación (CU-014). Todos ellos tienen como actor al usuario autenticado y operan exclusivamente sobre las consultas del propio usuario, en cumplimiento del aislamiento de datos entre usuarios: antes de mostrar o modificar cualquier información, el sistema verifica que la consulta pertenece al usuario que la solicita. La secuencia de CU-011 constituye el punto de entrada del subsistema, desde el que se alcanzan opcionalmente el detalle (CU-012) y, desde este, el renombrado (CU-013) y la eliminación (CU-014). Se presentan a continuación, precedidos cada uno de una breve descripción de la interacción que modelan.

Secuencia CU-011: Consultar el historial de consultas

El profesional recupera sus consultas anteriores para revisar un diagnóstico, comparar la evolución de un caso o reutilizar una imagen. El usuario accede a su historial desde el panel de diagnóstico y el sistema recupera únicamente las consultas del propio usuario, mostrando el listado con los datos esenciales de cada una: fecha, modelo empleado, resultado y confianza.

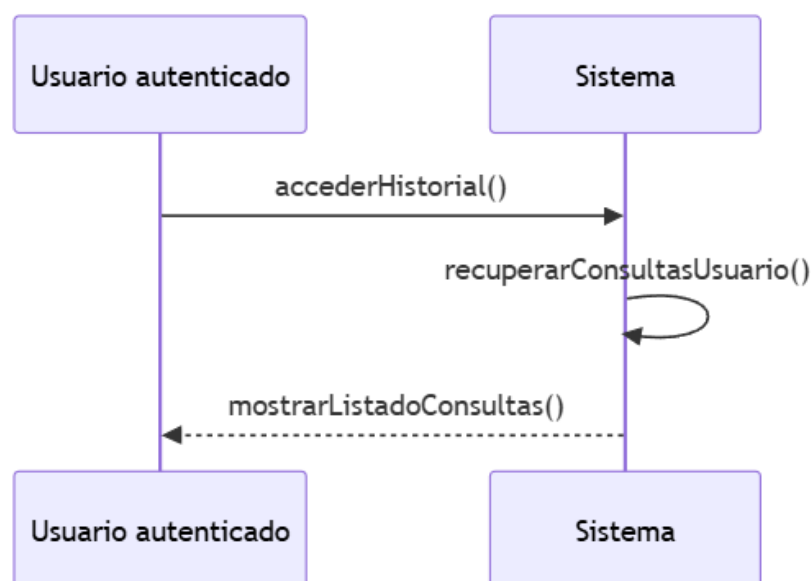


Ilustración x - Secuencia de interacción de la consulta del historial (CU-011)

Secuencia CU-012: Ver el detalle de una consulta del historial

El usuario abre el detalle completo de una consulta concreta para recuperar la imagen original, el resultado, la confianza y los mapas de explicabilidad asociados. El usuario selecciona una consulta del listado del historial y el sistema comprueba que la consulta pertenece al usuario. Si la verificación es correcta, el sistema muestra el detalle completo de la consulta, incluyendo los metadatos asociados. La figura muestra esta secuencia.

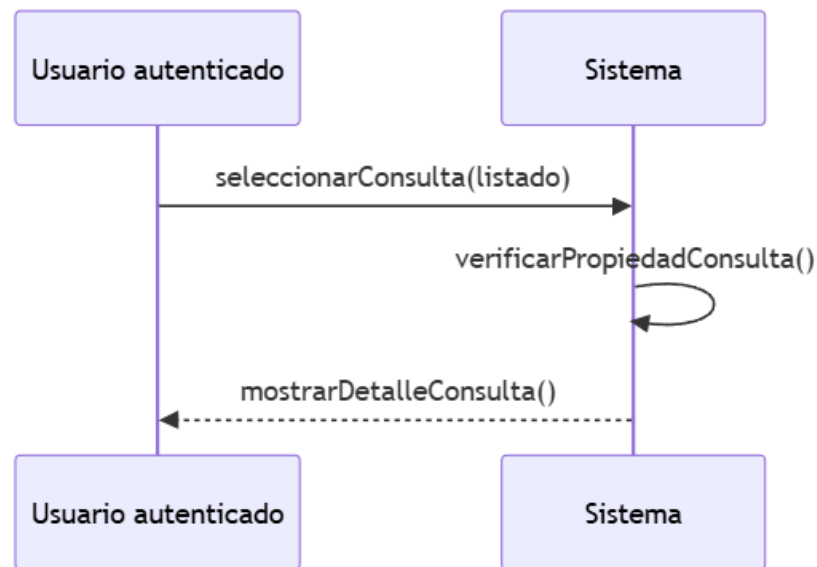


Ilustración x - Secuencia de interacción de la visualización del detalle (CU-012)

Secuencia CU-013: Renombrar una consulta del historial

El usuario modifica el nombre de una de sus consultas para identificarla mejor, por ejemplo cuando un mismo paciente tiene varias placas. El usuario indica el nuevo nombre desde el detalle de la consulta, el sistema comprueba que la consulta pertenece al usuario y valida que el nuevo nombre no esté vacío. Si la validación es correcta, el sistema actualiza el nombre de la consulta, sin alterar la imagen, el resultado ni los mapas de explicabilidad.

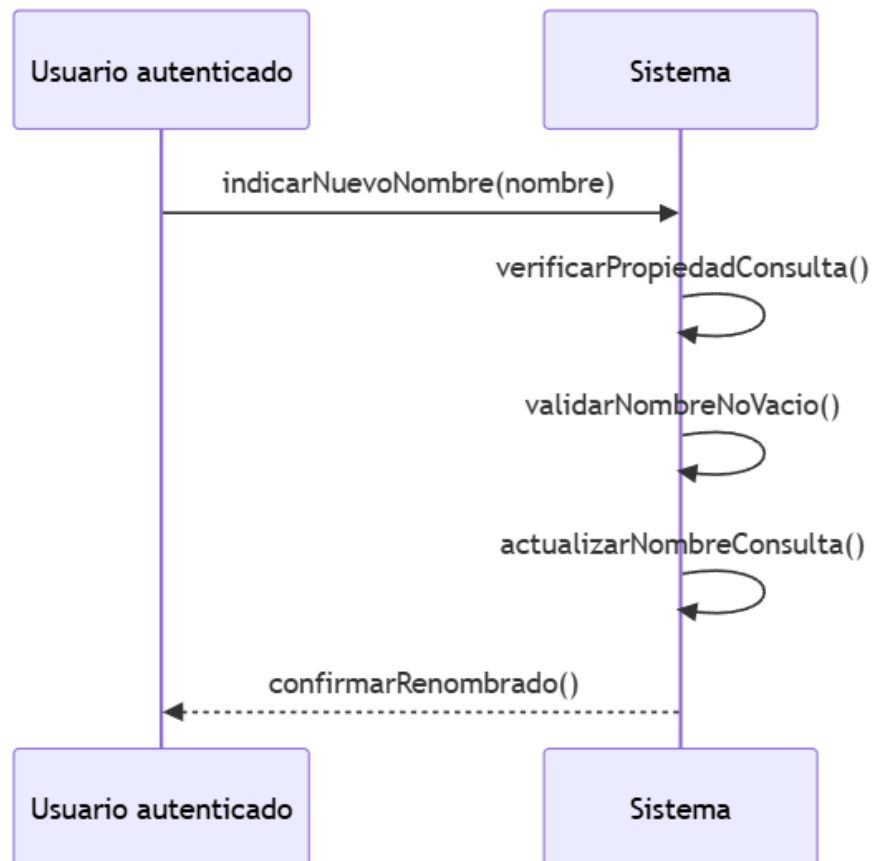


Ilustración x - Secuencia de interacción del renombrado de una consulta (CU-013)

Secuencia CU-014: Eliminar una consulta del historial

El usuario depura su historial eliminando las consultas que ya no necesita. Al tratarse de una operación que retira información clínica, el sistema comprueba primero que la consulta pertenece al usuario y solicita confirmación antes de ejecutarla. El usuario confirma la eliminación y el sistema retira el registro de la consulta, que deja de aparecer en su listado activo.

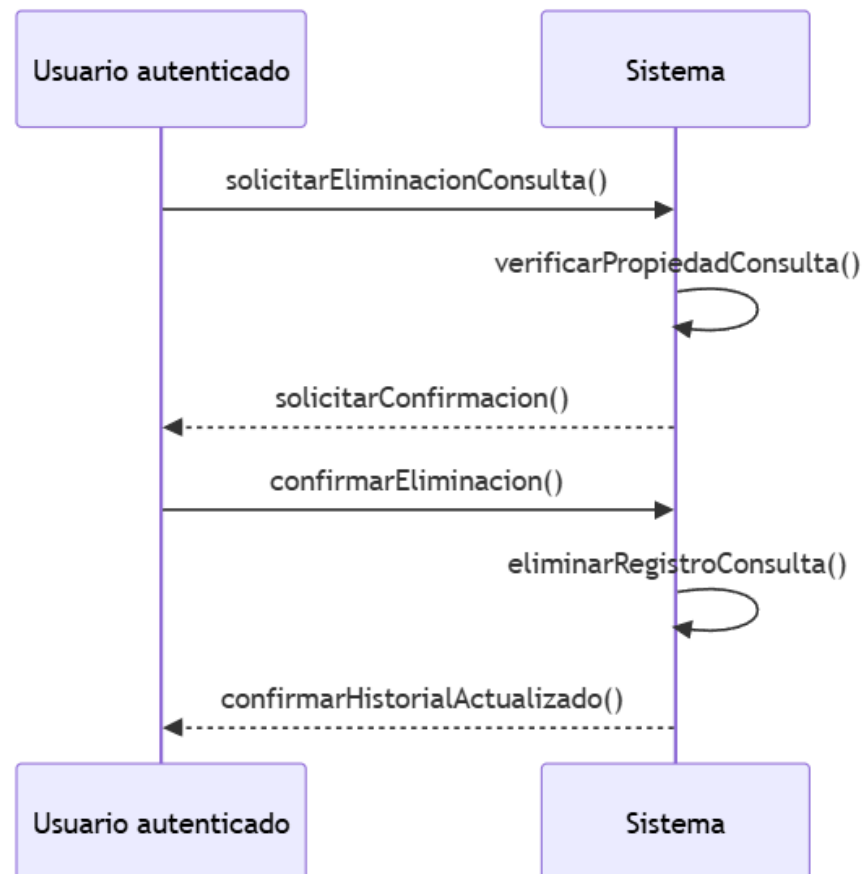


Ilustración x - Secuencia de interacción de la eliminación de una consulta (CU-014)

14.2.4. SS-004 – Subsistema de Laboratorio MLOps

Este subapartado recoge las secuencias de interacción correspondientes a los casos de uso del subsistema de laboratorio MLOps, descrito en el capítulo 13. Este subsistema agrupa la actividad investigadora de la plataforma: el acceso al laboratorio (CU-015), la conversación con el asistente para configurar un experimento (CU-016), la selección de la carpeta del dataset (CU-017), el lanzamiento del experimento (CU-018), la consulta de las sesiones (CU-019), la consulta de los resultados de un modelo (CU-020), la visualización de sus mapas de explicabilidad (CU-021), la consulta del ranking (CU-022), la comparativa estadística (CU-023) y su recálculo (CU-024), el análisis de explicabilidad (CU-025), la validación externa (CU-026) y su consulta (CU-027), la generación del informe PDF (CU-028), el renombrado y la eliminación de sesiones (CU-029 y CU-030) y la comprobación de la limitación de entrenamientos simultáneos y encolados (CU-039). Todos ellos tienen como actor al usuario autenticado, que opera exclusivamente sobre sus propias sesiones en cumplimiento del aislamiento de datos entre usuarios. Las secuencias se presentan siguiendo el orden natural de la actividad del laboratorio, desde la configuración del experimento hasta la consulta y gestión de los resultados. Se presentan a continuación, precedidos cada uno de una breve descripción de la interacción que modelan.

Secuencia CU-015: Acceder al laboratorio de entrenamiento

El laboratorio de entrenamiento es el espacio en el que se desarrolla la actividad investigadora de la plataforma. El usuario accede al laboratorio desde la navegación principal y el sistema valida que su sesión se encuentra activa. Si la sesión es válida, el sistema carga el entorno del laboratorio y lo muestra, quedando disponibles el asistente conversacional, el listado de sesiones y el resto de las funcionalidades del módulo.

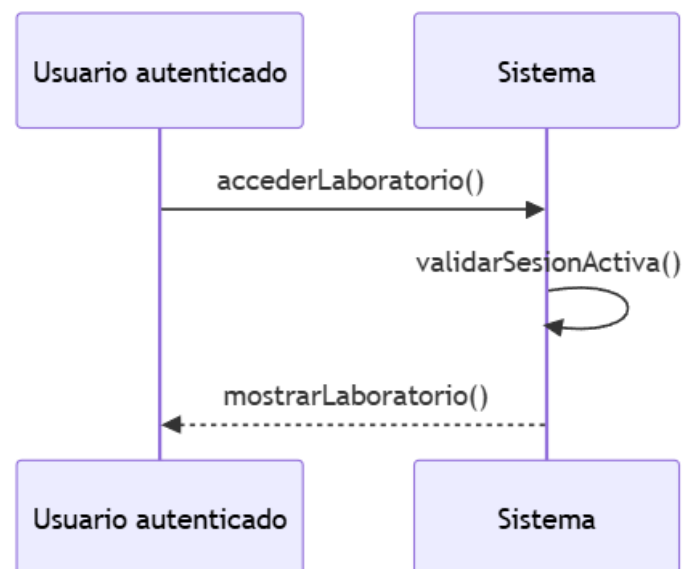


Ilustración x - Secuencia de interacción del acceso al laboratorio (CU-015)

Secuencia CU-016: Conversar con el asistente para configurar un experimento

El asistente conversacional es la interfaz principal de configuración del experimento y su interacción es un diálogo bidireccional: el usuario no entrega la configuración completa de una sola vez, sino que mantiene un intercambio con el asistente hasta que todos los parámetros quedan definidos. El usuario envía un mensaje en lenguaje natural indicando los parámetros que desea y el sistema prepara el prompt de sistema definido y envía la petición al modelo de lenguaje. El modelo extrae los parámetros mencionados en el mensaje y, si falta alguno de los parámetros que definen el experimento, como las arquitecturas, el número de épocas, el tamaño de lote o la tasa de aprendizaje, el asistente pregunta por el parámetro faltante y la conversación continúa hasta completarlo. La ruta del dataset no forma parte de la conversación: se selecciona de forma validada mediante el caso de uso CU-017 y se incorpora a la configuración. Cuando el asistente dispone de todos los parámetros, devuelve la configuración estructurada, el sistema rellena el panel de configuración con los valores obtenidos y el usuario la revisa y confirma.

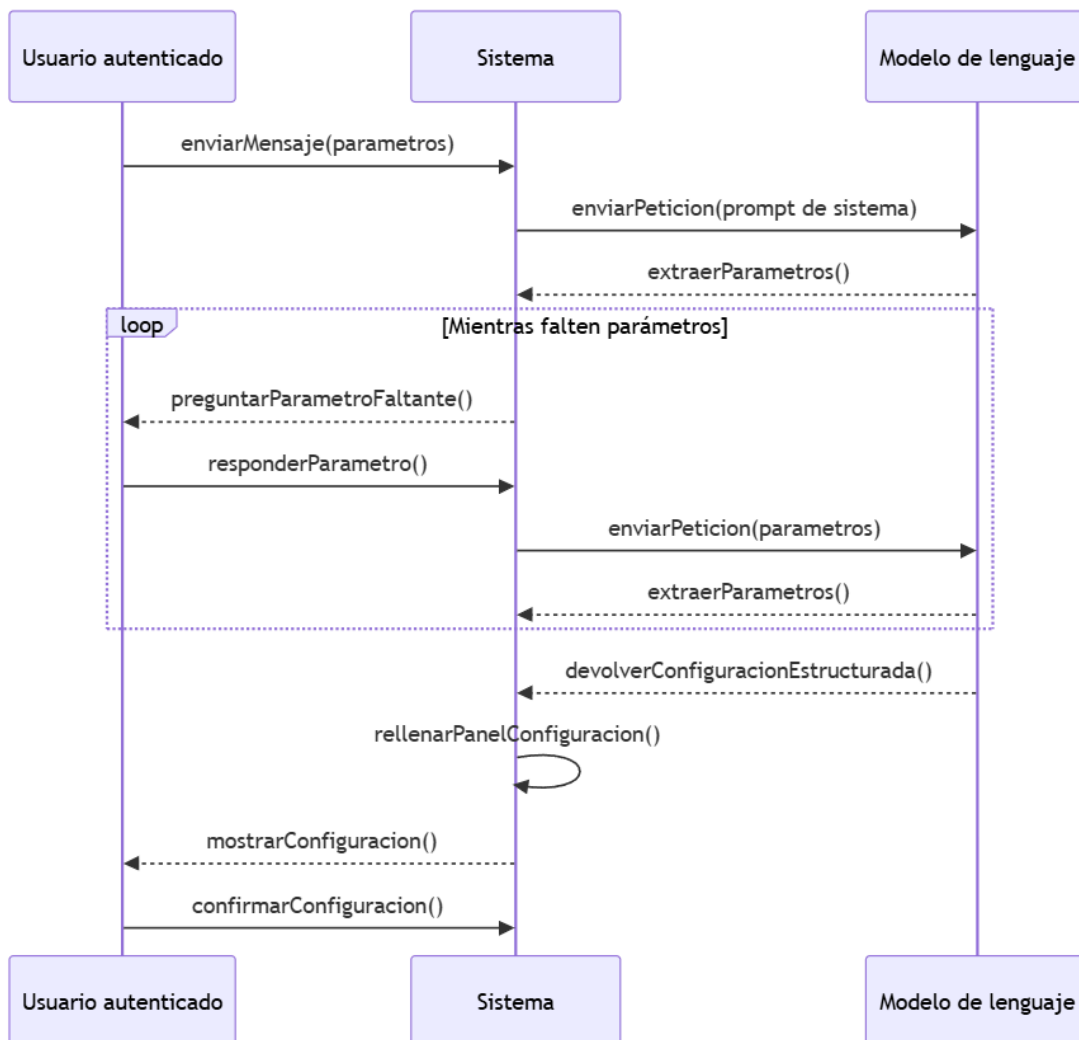


Ilustración x - Secuencia de interacción de la conversación con el asistente (CU-016)

Secuencia CU-017: Seleccionar la carpeta del dataset

Antes de lanzar el experimento, el usuario debe indicar la carpeta del conjunto de datos sobre el que se entrenará el modelo. El usuario solicita explorar la carpeta y el sistema devuelve la ruta, preconfigurada en el entorno o seleccionada por el propio usuario. El usuario confirma la ruta y esta queda asociada a la configuración del experimento. La restricción que debe impedir el acceso a rutas fuera del directorio permitido permanece pendiente de implementación, como se indica en el capítulo 12.

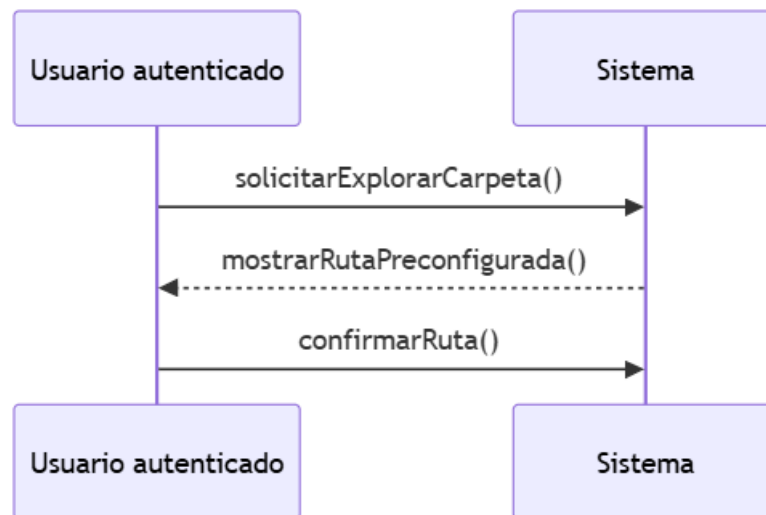


Ilustración x - Secuencia de interacción de la selección de la carpeta del dataset (CU-017)

Secuencia CU-018: Lanzar un experimento de entrenamiento

El lanzamiento del experimento inicia el proceso de entrenamiento. El usuario envía la configuración del experimento, ya completada mediante la conversación con el asistente, y el sistema crea la sesión de entrenamiento y encola el trabajo de ejecución. El worker procesa el trabajo en segundo plano: ejecuta el entrenamiento con validación cruzada, el análisis de explicabilidad y la comparación estadística. Al finalizar, el sistema actualiza el estado de la sesión y notifica al usuario.

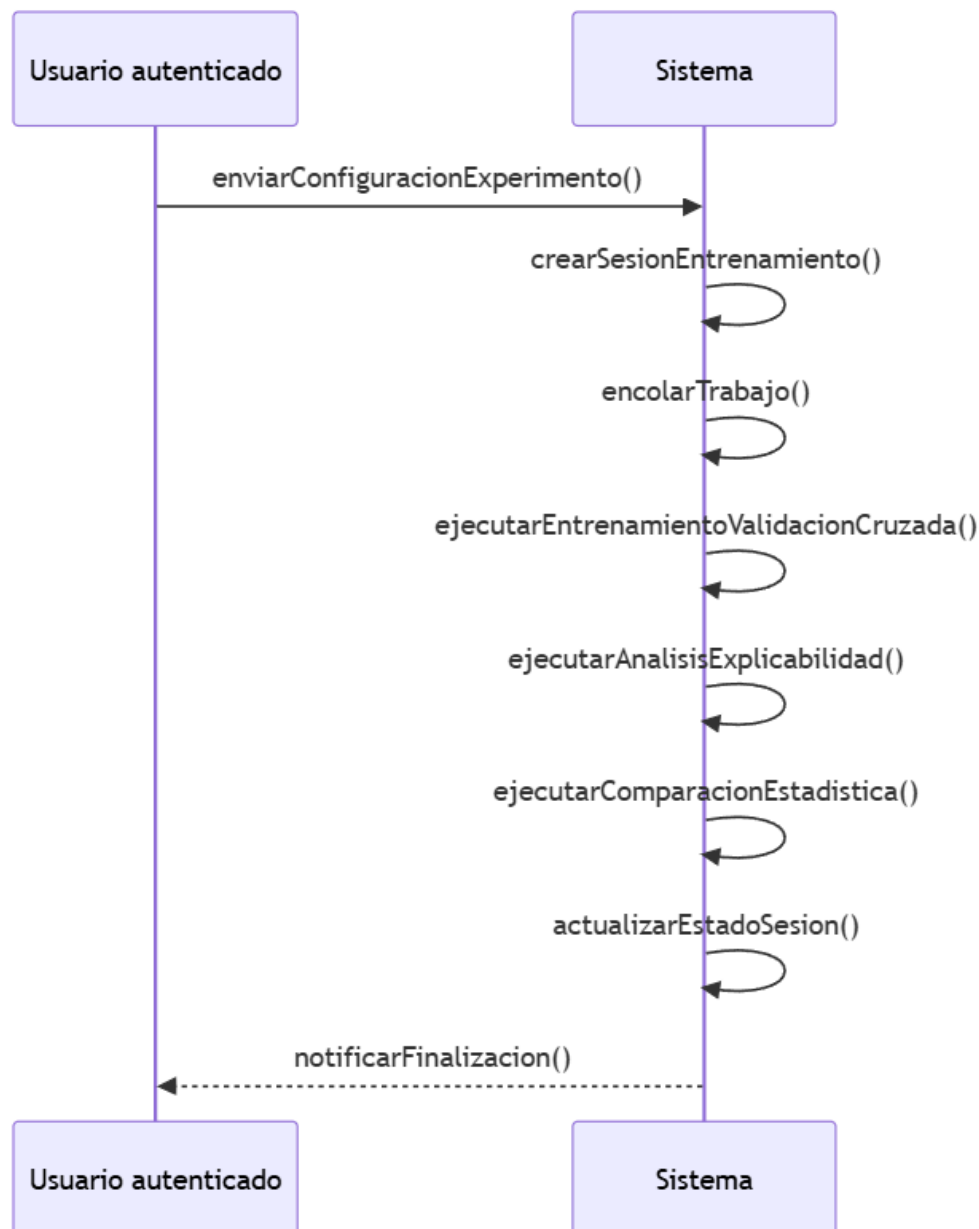


Ilustración x - Secuencia de interacción del lanzamiento de un experimento (CU-018)

Secuencia CU-019: Consultar las sesiones de entrenamiento

El investigador necesita recuperar sus sesiones de entrenamiento para revisar su estado y sus resultados. El usuario accede al listado de sesiones del laboratorio y el sistema recupera únicamente las sesiones del propio usuario, mostrando para cada una su estado y los modelos generados.

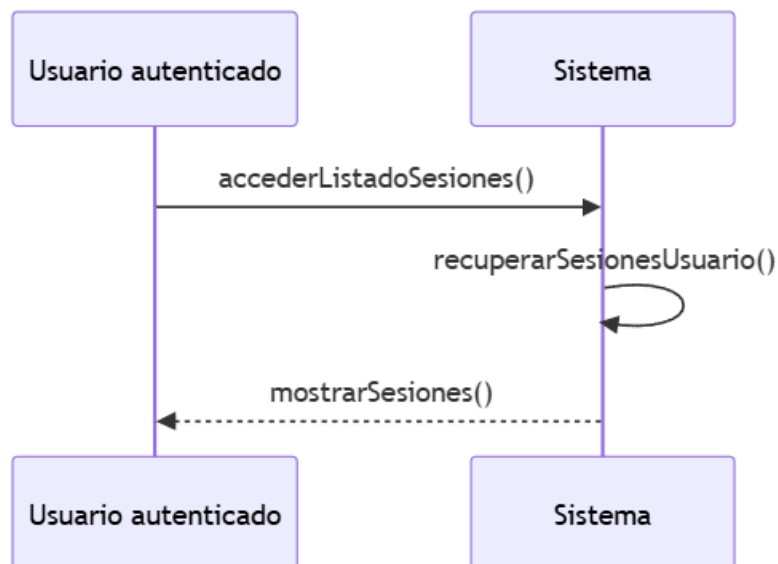


Ilustración x - Secuencia de interacción de la consulta de las sesiones (CU-019)

Secuencia CU-020: Consultar los resultados de un modelo de la sesión

El usuario selecciona un modelo de la sesión para inspeccionar sus resultados. El sistema recupera y muestra las métricas del modelo, con su media y desviación sobre los pliegues de la validación cruzada, junto con los artefactos asociados. Esta información permite al investigador evaluar el rendimiento del modelo antes de decidir sobre su uso.

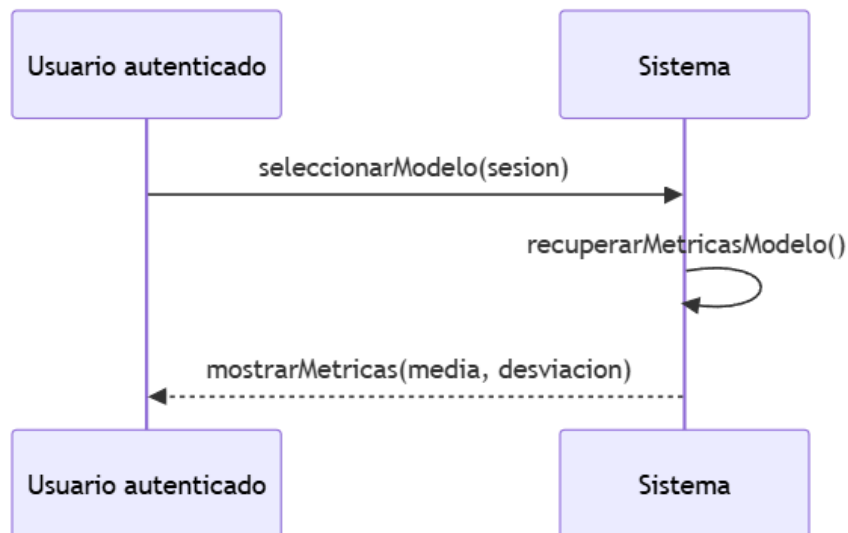


Ilustración x - Secuencia de interacción de la consulta de los resultados de un modelo (CU-020)

Secuencia CU-021: Visualizar los mapas de calor de explicabilidad de un modelo

La explicabilidad de los modelos es uno de los pilares de la plataforma. El usuario selecciona un modelo de la sesión y el sistema recupera las imágenes XAI generadas durante el análisis de explicabilidad, mostrándolas en una galería. De este modo, el investigador puede inspeccionar visualmente qué regiones de las imágenes pondera cada modelo antes de tomar decisiones.

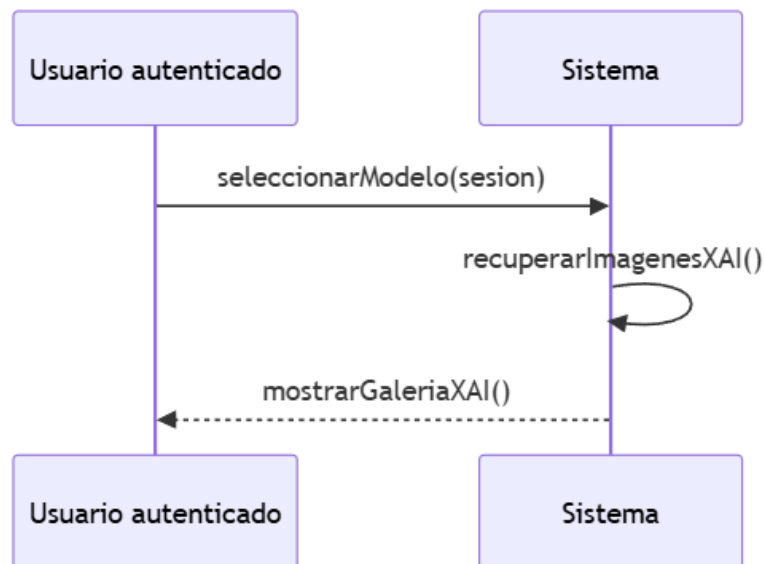


Ilustración x - Secuencia de interacción de la visualización de los mapas de explicabilidad (CU-021)

Secuencia CU-022: Consultar el ranking de modelos de la sesión

Para comparar rápidamente los modelos generados, el usuario puede consultar el ranking de la sesión. El usuario solicita el ranking y el sistema recupera y muestra los modelos ordenados por su AUC medio, una medida habitual del rendimiento discriminativo basada en la curva ROC (Fawcett, 2006).

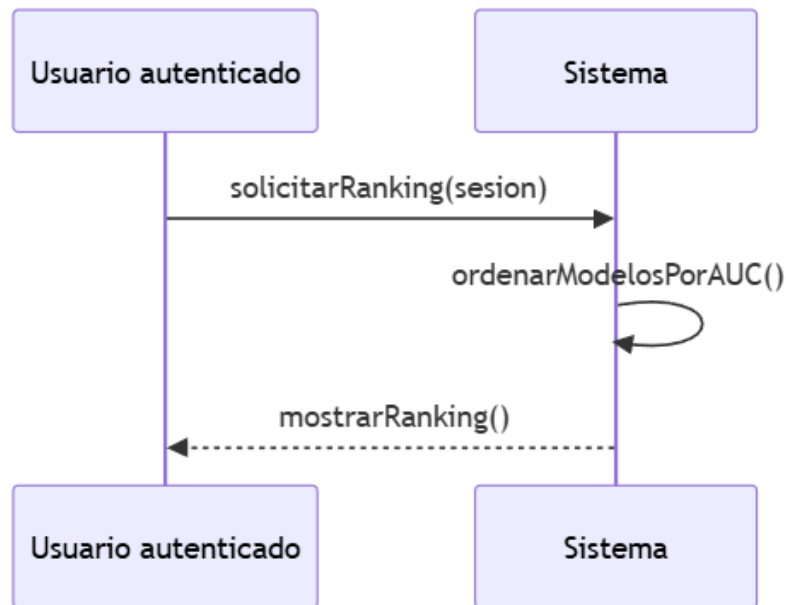


Ilustración x - Secuencia de interacción de la consulta del ranking (CU-022)

Secuencia CU-023: Consultar la comparativa estadística de la sesión

La comparativa estadística complementa el ranking con el resultado de los contrastes entre modelos. El usuario accede a la vista de comparativa de la sesión y el sistema muestra la matriz de significación correspondiente, calculada mediante el test de Wilcoxon sobre los valores disponibles (Wilcoxon, 1945). La matriz informa sobre la compatibilidad de las diferencias observadas con la variabilidad de los datos, pero no establece por sí sola superioridad clínica.

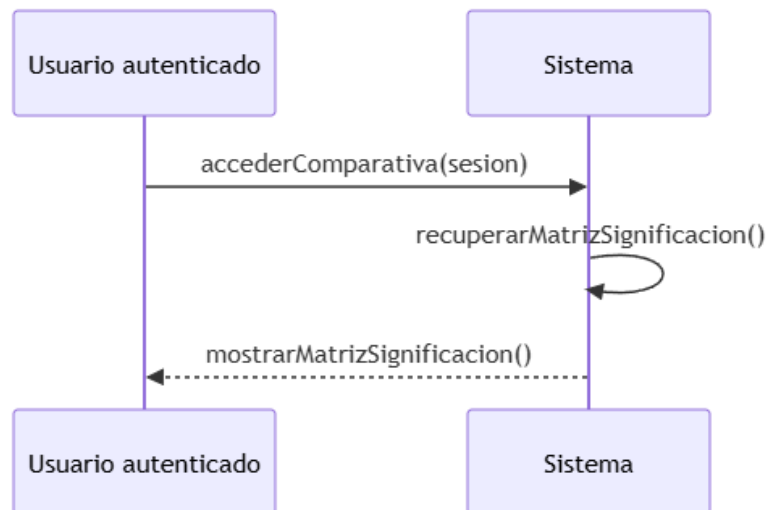


Ilustración x - Secuencia de interacción de la consulta de la comparativa estadística (CU-023)

Secuencia CU-024: Solicitar el recálculo de la comparativa estadística

Si el investigador modifica las condiciones del análisis, puede solicitar que la comparativa estadística se recalcule. El usuario solicita el recálculo y el sistema lanza el proceso en segundo plano, de modo que la interfaz permanece operativa. Cuando el recálculo finaliza, el sistema actualiza la comparativa y notifica al usuario la disponibilidad de los nuevos resultados.

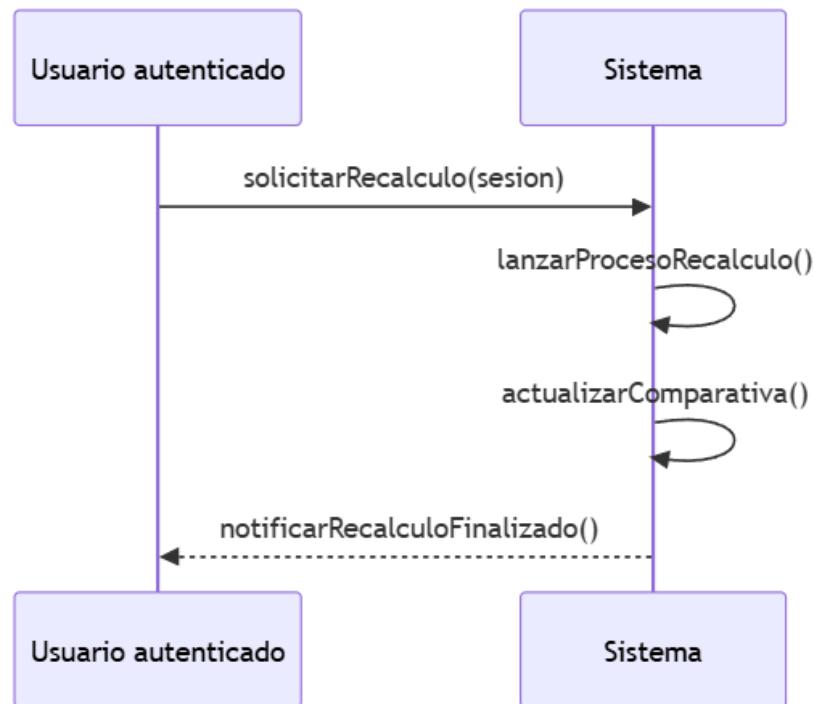


Ilustración x - Secuencia de interacción del recálculo de la comparativa (CU-024)

Secuencia CU-025: Ejecutar el análisis de explicabilidad de un modelo

El análisis de explicabilidad puede ejecutarse de forma independiente sobre un modelo concreto de la sesión. El usuario solicita generar el análisis XAI del modelo y el sistema ejecuta los scripts de explicabilidad en segundo plano. Cuando el análisis finaliza, el sistema notifica al usuario, que puede consultar las imágenes XAI generadas. Las técnicas utilizadas se describen en la bibliografía específica de Saliency Maps, SmoothGrad, Grad-CAM y mapas de atención (Simonyan, Vedaldi, & Zisserman, 2014; Smilkov & al., 2017; Selvaraju & al., 2017; Chefer, Gur, & Wolf, 2021).

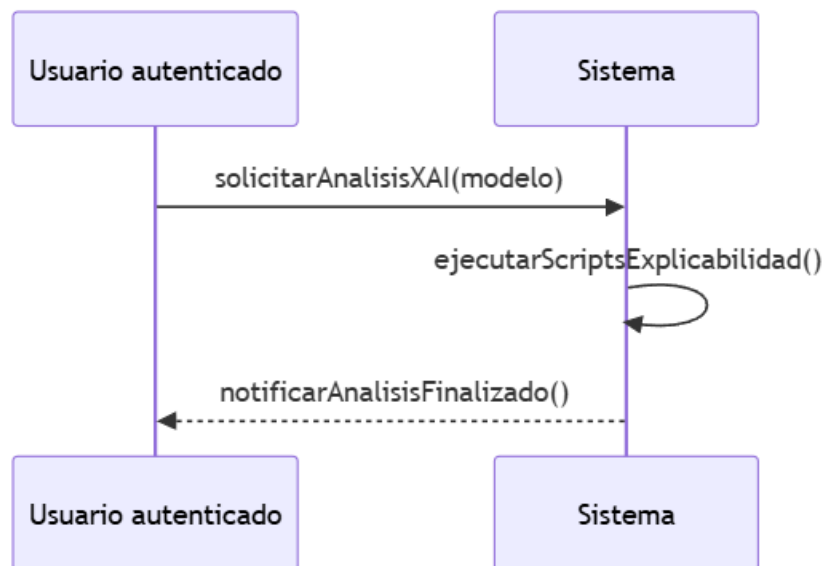


Ilustración x - Secuencia de interacción de la ejecución del análisis de explicabilidad (CU-025)

Secuencia CU-026: Solicitar la validación externa de la sesión

La validación externa aporta información adicional sobre el rendimiento de los modelos en datos independientes. El usuario solicita la validación externa de la sesión y el sistema encola el trabajo de validación. El worker evalúa los modelos congelados sobre el conjunto externo y aplica el test de DeLong para comparar sus curvas ROC (DeLong, DeLong, & Clarke-Pearson, 1988). Cuando la validación finaliza, el sistema notifica al usuario.

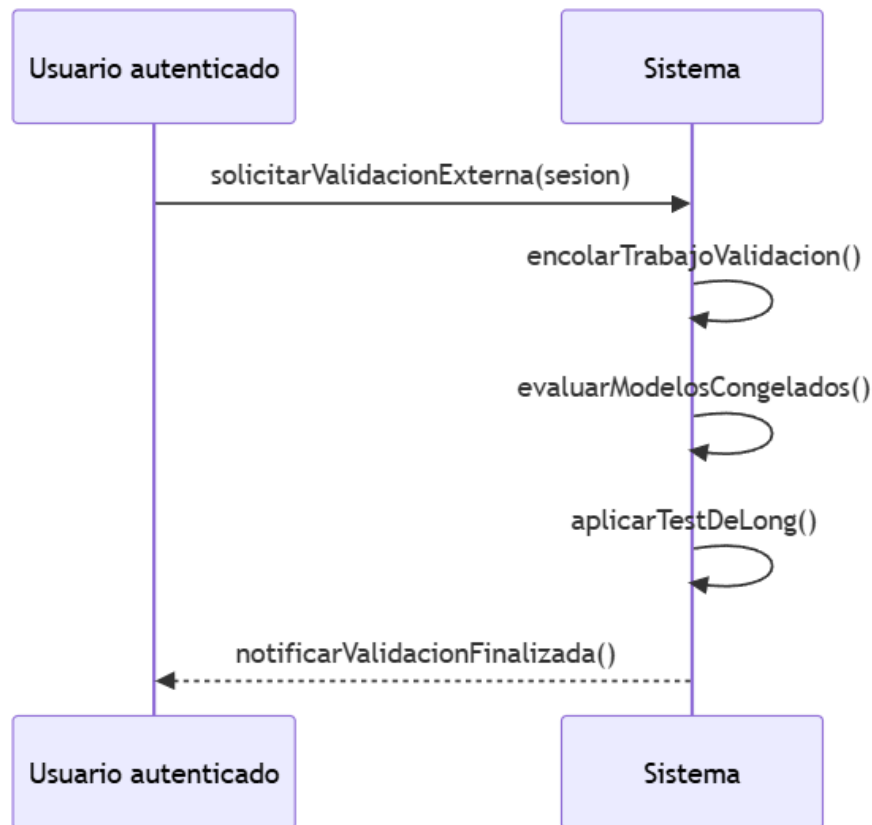


Ilustración x - Secuencia de interacción de la solicitud de la validación externa (CU-026)

Secuencia CU-027: Consultar los resultados de la validación externa

Una vez finalizada la validación externa, el usuario consulta sus resultados. El usuario accede a los resultados externos de la sesión y el sistema muestra las métricas de los modelos sobre el conjunto externo, las curvas ROC y la matriz de DeLong resultante del test estadístico.

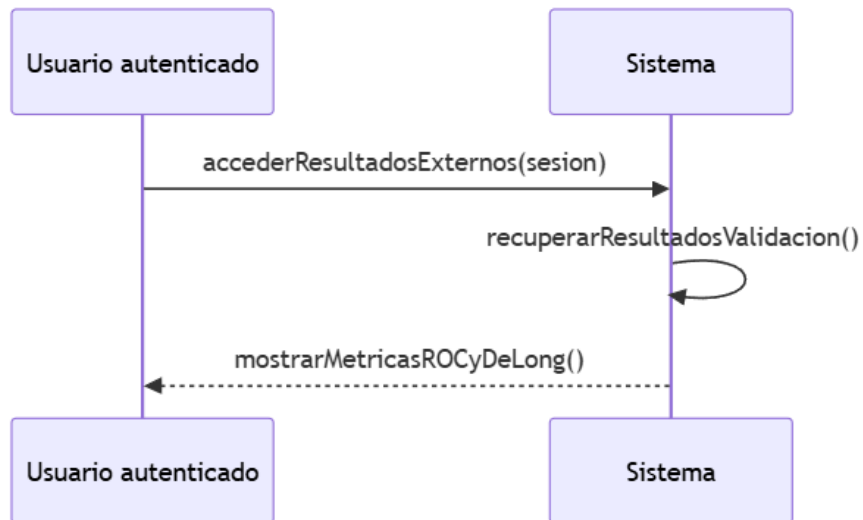


Ilustración x - Secuencia de interacción de la consulta de la validación externa (CU-027)

Secuencia CU-028: Generar el informe PDF de la sesión

El informe PDF consolida los resultados de la sesión para su difusión o conservación. El usuario solicita el informe de la sesión y el sistema genera el documento PDF con los resultados consolidados, incluyendo las métricas, los mapas de explicabilidad y la comparativa. Finalmente, el sistema inicia la descarga del documento al equipo del usuario.

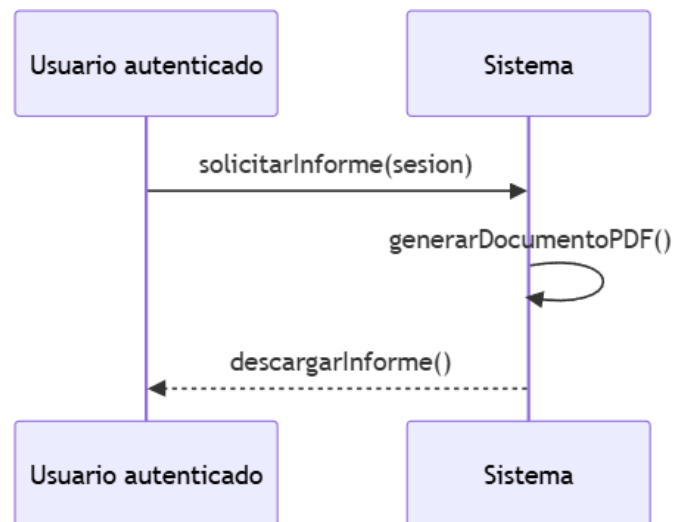


Ilustración x - Secuencia de interacción de la generación del informe PDF (CU-028)

Secuencia CU-029: Renombrar una sesión de entrenamiento

El investigador puede dar a sus sesiones un nombre más descriptivo para identificarlas mejor. El usuario indica el nuevo nombre de la sesión y el sistema comprueba que la sesión pertenece al usuario y valida que el nuevo nombre no esté vacío. Si la validación es correcta, el sistema actualiza el nombre de la sesión, sin alterar los modelos ni los artefactos asociados.

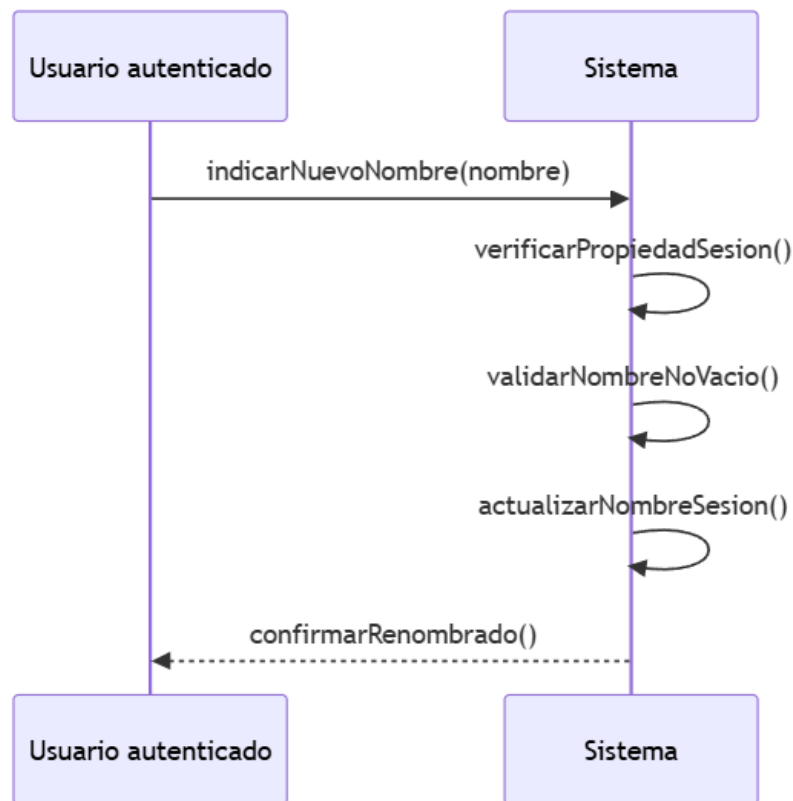


Ilustración x - Secuencia de interacción del renombrado de una sesión (CU-029)

Secuencia CU-030: Eliminar una sesión de entrenamiento

El investigador puede eliminar las sesiones que ya no necesita. Al tratarse de una operación que retira artefactos de entrenamiento, el sistema comprueba primero que la sesión pertenece al usuario y solicita confirmación antes de ejecutarla. El usuario confirma la eliminación y el sistema retira la sesión junto con sus artefactos asociados.

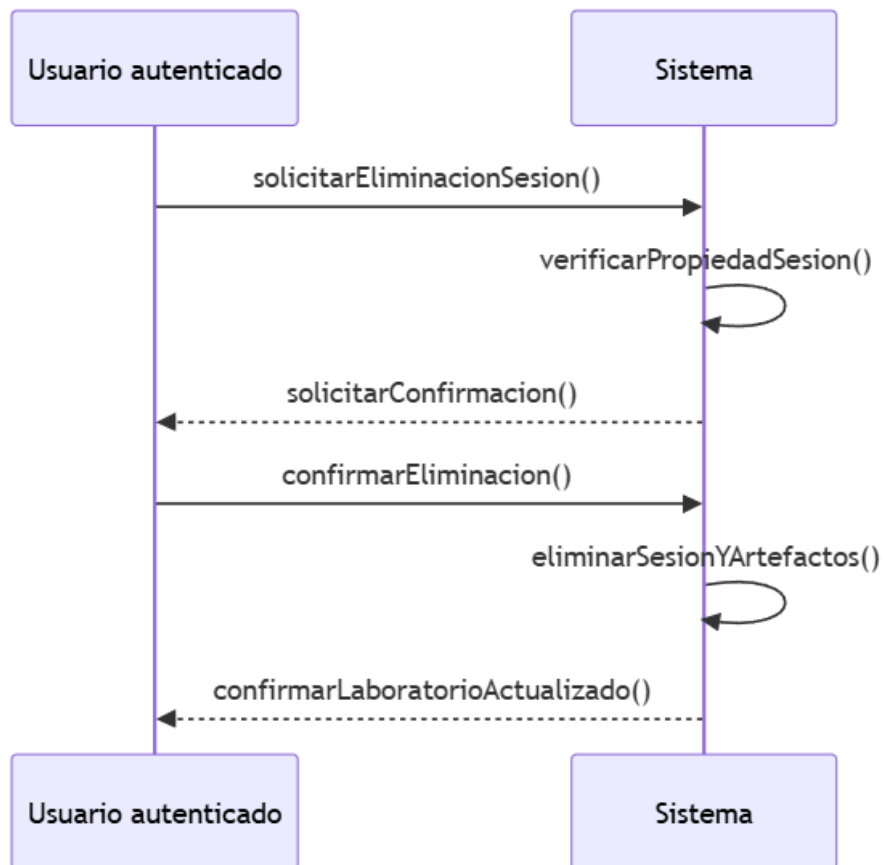


Ilustración x - Secuencia de interacción de la eliminación de una sesión (CU-030)

Secuencia CU-039: Comprobar la limitación de entrenamientos

La limitación de entrenamientos es una capacidad prevista para evitar que la competición por la GPU degrade el servicio para el resto de los usuarios. Cuando el usuario lance un experimento y se supere el límite de entrenamientos simultáneos o encolados, el trabajo debería permanecer en espera o ser rechazado según la política definida. Esta capacidad permanece pendiente de implementación en el prototipo actual.

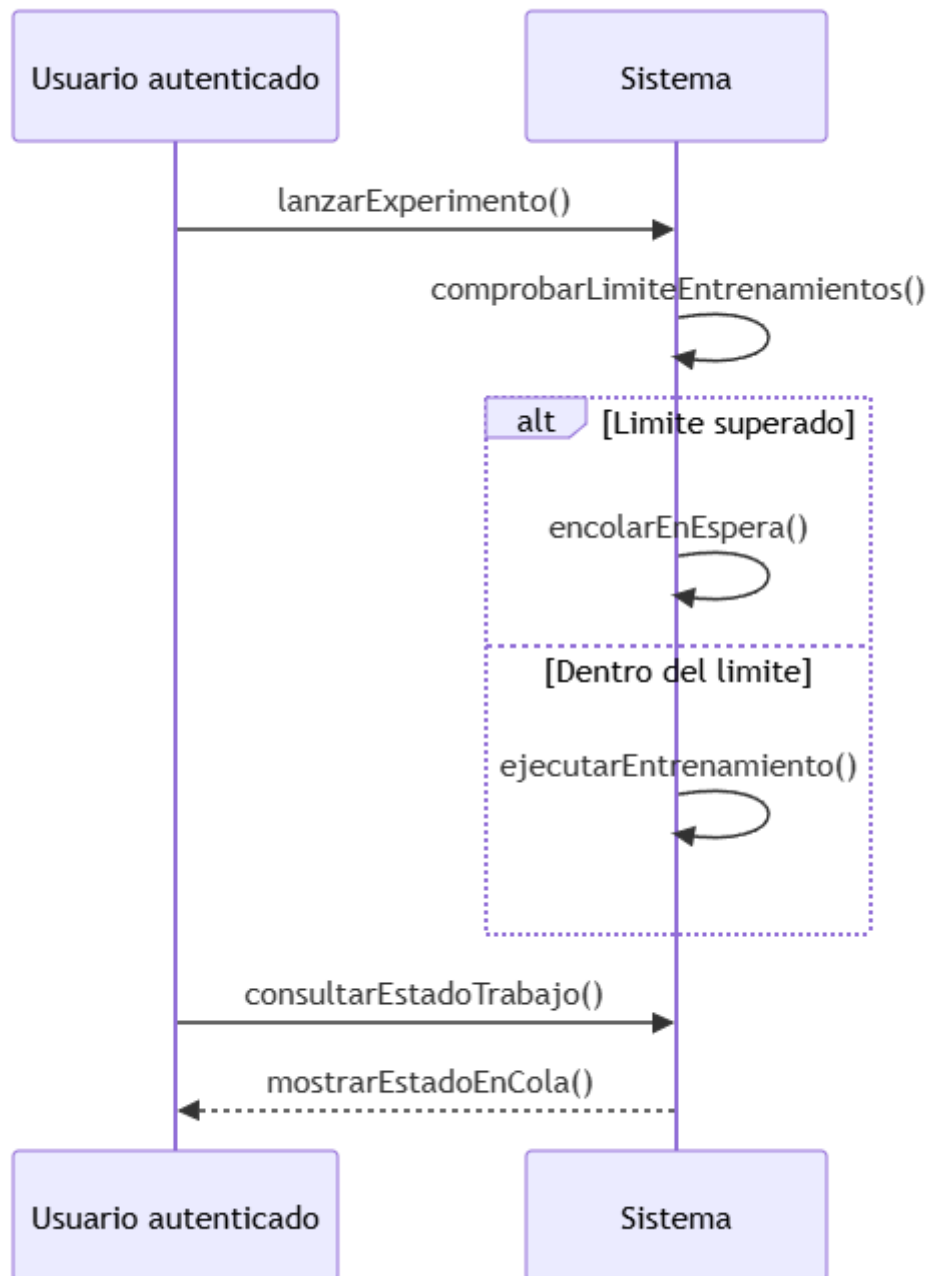


Ilustración x - Secuencia de interacción de la limitación de entrenamientos (CU-039)

14.2.5. SS-005 – Subsistema de Supervisión y Administración

Este subapartado recoge las secuencias de interacción correspondientes a los casos de uso del subsistema de supervisión y administración, descrito en el capítulo 13. Este subsistema agrupa las operaciones de supervisión de la plataforma: la consulta del listado de usuarios (CU-031), la consulta de las consultas de diagnóstico de un usuario concreto (CU-032), la visualización del detalle de una de esas consultas (CU-033) y la gestión prevista de cuentas (CU-038). Todos ellos tienen como actor al administrador y están restringidos a su rol: en cada operación el sistema verifica el rol del actor antes de mostrar información o aplicar una operación. Las secuencias de supervisión siguen la navegación desde la visión global del listado hasta el detalle de una consulta concreta; la gestión de cuentas se describe como una operación independiente. Se presentan a continuación, precedidos cada uno de una breve descripción de la interacción que modelan.

Secuencia CU-031: Consultar el listado de usuarios

El listado de usuarios constituye la base de la supervisión de la plataforma. El administrador accede al panel de administración y el sistema verifica que el actor dispone del rol de administración. Si la verificación es correcta, el sistema recupera y muestra el listado de usuarios registrados, permitiendo al administrador identificar cuentas y comprobar el estado del sistema.

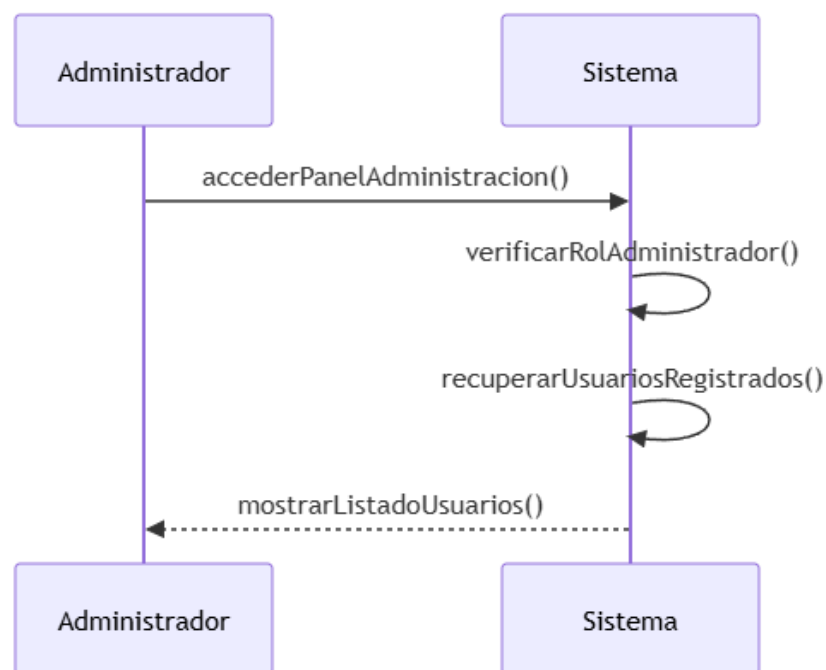


Ilustración x - Secuencia de interacción de la consulta del listado de usuarios (CU-031)

Secuencia CU-032: Consultar las consultas de un usuario

La supervisión de la actividad registrada exige poder examinar la actividad de un usuario concreto. El administrador selecciona un usuario del listado y el sistema verifica de nuevo el rol de administración del actor. Si la verificación es correcta, el sistema recupera y muestra las consultas de diagnóstico de ese usuario, permitiendo comprobar el uso que se hace de la plataforma y detectar posibles incidencias.

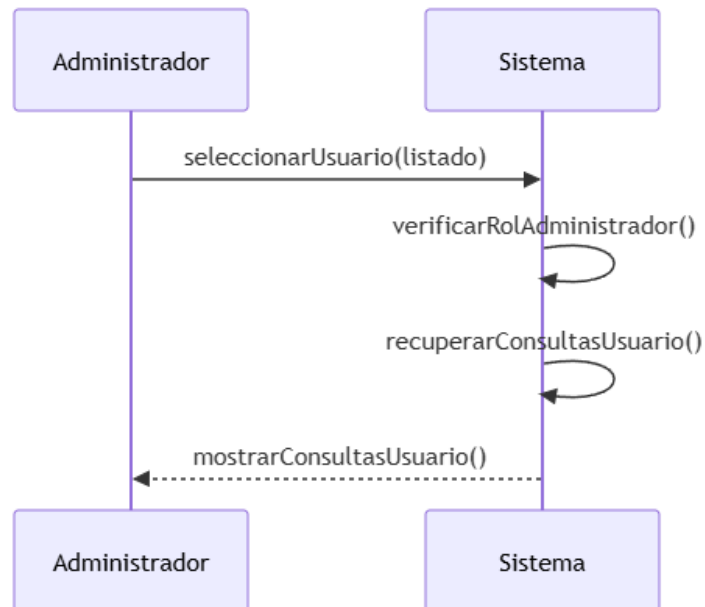


Ilustración x - Secuencia de interacción de la consulta de las consultas de un usuario (CU-032)

Secuencia CU-033: Ver el detalle de una consulta de un usuario

Para completar la supervisión, el administrador puede abrir el detalle completo de una consulta de un usuario para auditarla ante una incidencia. El administrador selecciona una consulta del listado y el sistema verifica su rol. Si la verificación es correcta, el sistema recupera y muestra el detalle de la consulta, incluida la imagen, el resultado, la confianza y los metadatos asociados.

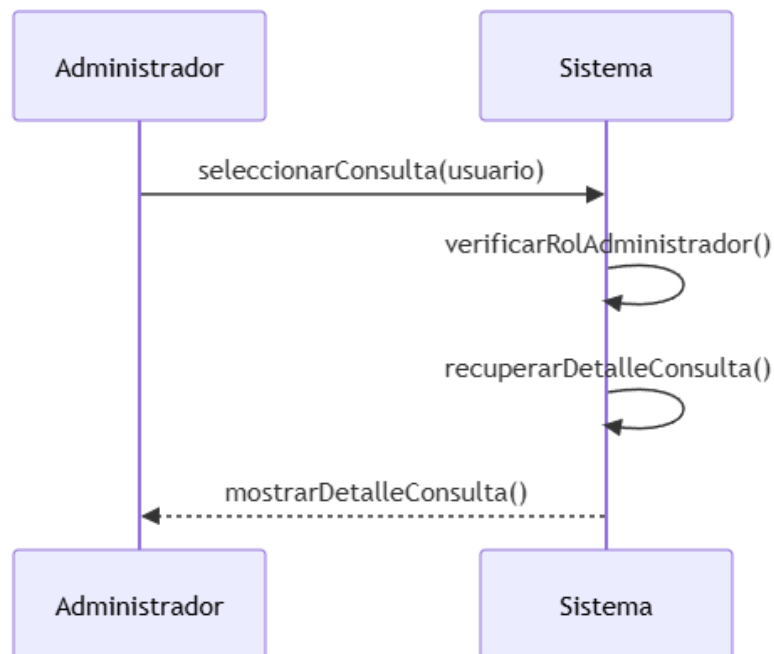


Ilustración x - Secuencia de interacción de la visualización del detalle de una consulta (CU-033)

Secuencia CU-038: Gestionar las cuentas de usuario

La administración de la plataforma no se limita a la consulta: el objetivo OBJ-011 contempla también la gestión de las cuentas de usuario. En el escenario previsto, el administrador selecciona una cuenta, el sistema verifica su rol y el administrador solicita desactivarla, cambiar su rol o eliminarla. El sistema aplicaría la operación y registraría la auditoría. Esta capacidad permanece pendiente de implementación en el prototipo actual.

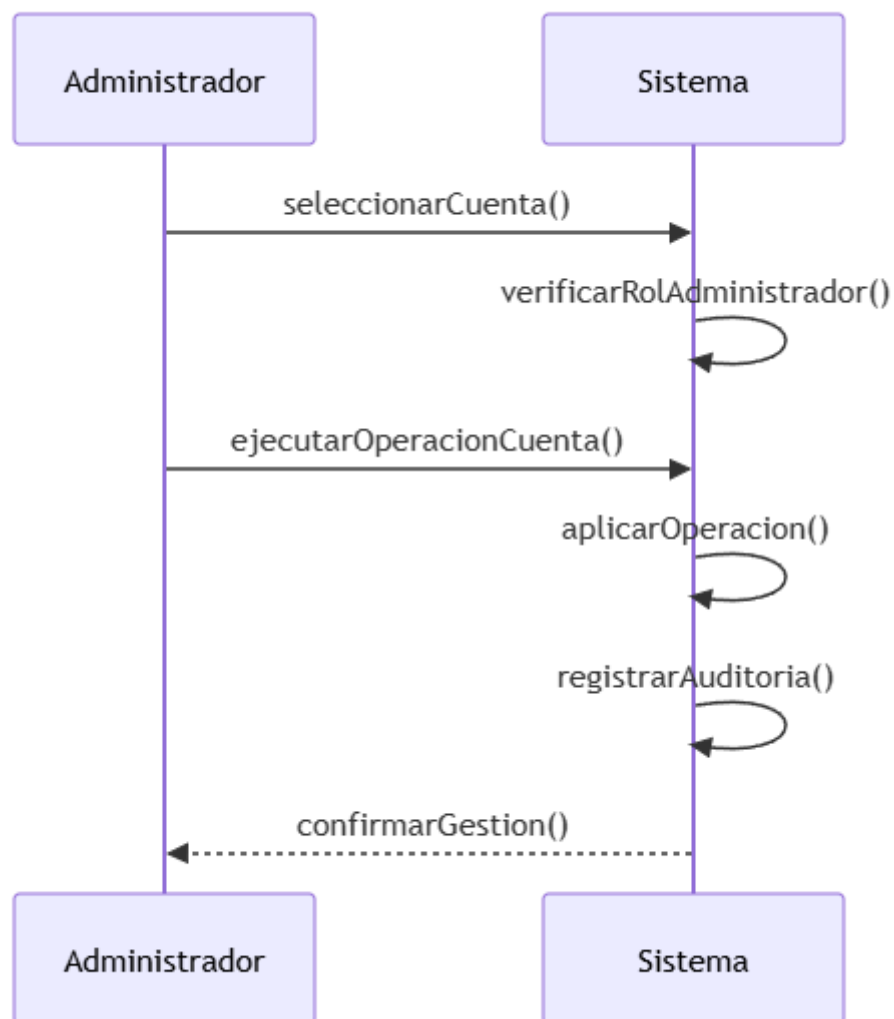


Ilustración x - Secuencia de interacción de la gestión de las cuentas de usuario (CU-038)

14.2.6. SS-006 – Subsistema de Capacidades Transversales

Este subapartado recoge las secuencias de interacción correspondientes a los casos de uso del subsistema de capacidades transversales, descrito en el capítulo 13. Este subsistema agrupa las funcionalidades que no pertenecen a un ámbito funcional concreto, sino que afectan a toda la plataforma y están disponibles para todo usuario autenticado: la consulta del estado de la cola de trabajos (CU-034), la cancelación de un trabajo pendiente (CU-035) y el cambio del tema visual de la interfaz (CU-036). Las dos primeras están vinculadas a la ejecución asíncrona de los diagnósticos, los entrenamientos y las validaciones externas, mientras que la tercera responde a la personalización de la interfaz. A diferencia del resto de subsistemas, ninguna de estas secuencias verifica la propiedad de un recurso concreto del usuario ni un rol específico, puesto que sus operaciones son de carácter general sobre la propia sesión. Se presentan a continuación, precedidos cada uno de una breve descripción de la interacción que modelan.

Secuencia CU-034: Consultar el estado de la cola de trabajos

El sistema ejecuta de forma asíncrona los diagnósticos, los entrenamientos y las validaciones externas, y el usuario necesita conocer en cada momento el estado de sus trabajos. El usuario accede al panel de la cola de trabajos y el sistema muestra el estado de los trabajos del usuario: pendientes, en ejecución, completados o fallidos. La interfaz consulta periódicamente ese estado, de modo que el usuario sabe cuándo estará disponible un resultado o cuándo debe repetir un trabajo que ha fallado.

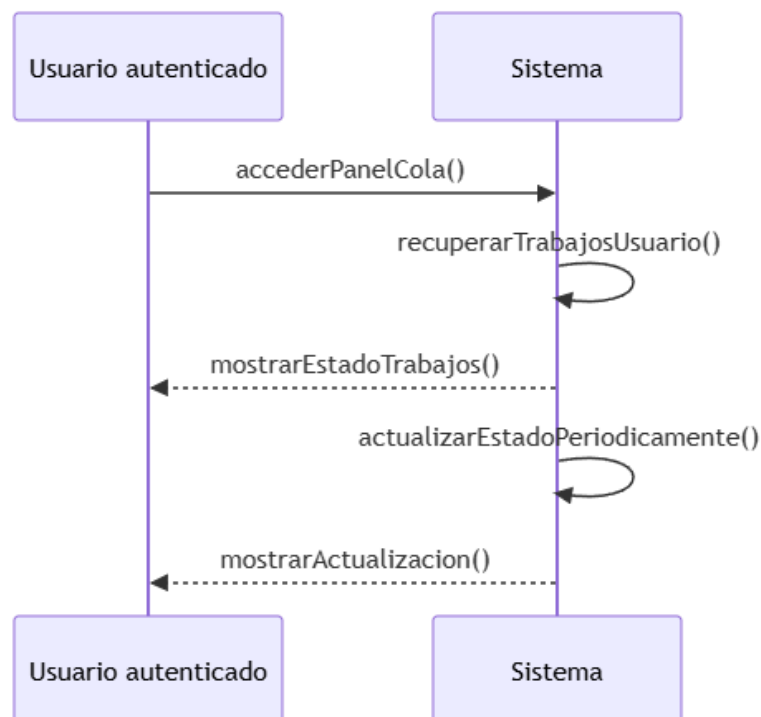


Ilustración x - Secuencia de interacción de la consulta del estado de la cola (CU-034)

Secuencia CU-035: Cancelar un trabajo de la cola

El usuario puede necesitar cancelar un trabajo pendiente que ha encolado por error o que ya no le interesa. El usuario solicita la cancelación y el sistema comprueba que el trabajo todavía no ha comenzado. Si sigue pendiente, el sistema lo marca como cancelado y evita su ejecución. La interrupción de un trabajo en ejecución no forma parte de la implementación actual.

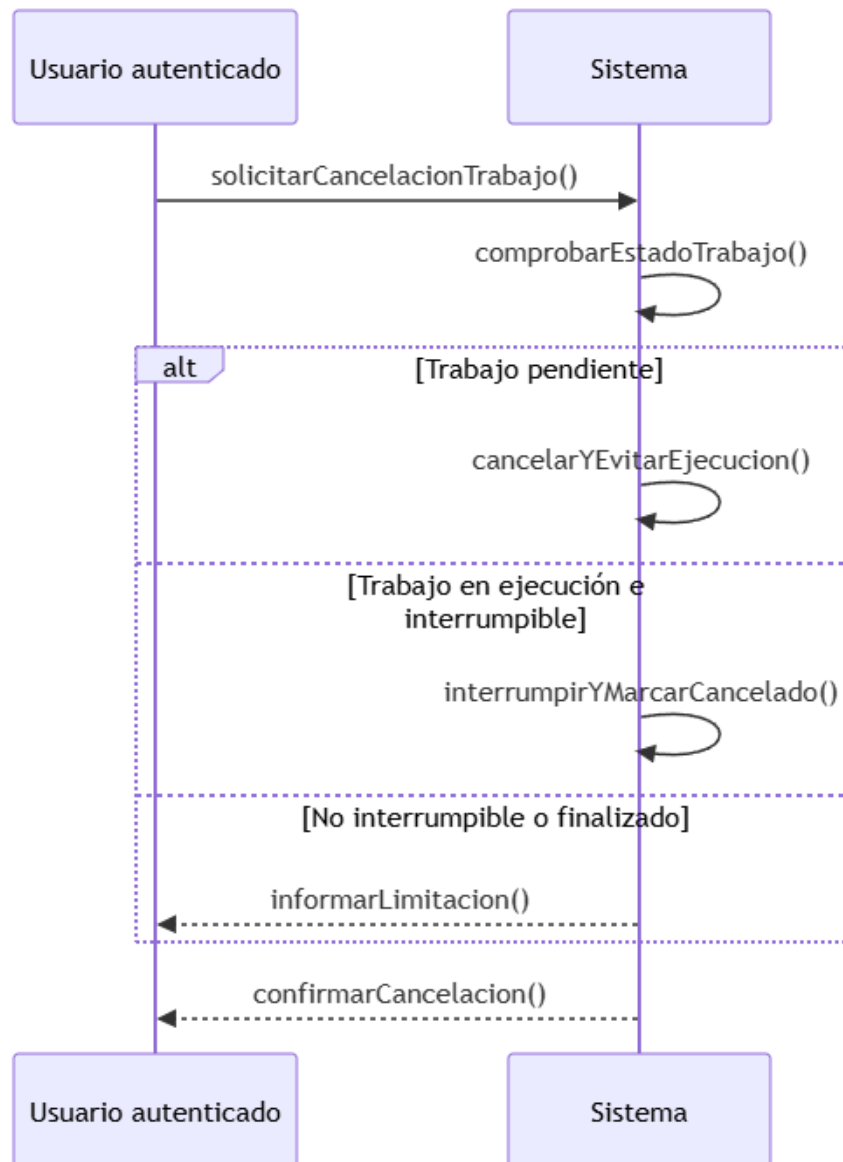


Ilustración x - Secuencia de interacción de la cancelación de un trabajo de la cola (CU-035)

Secuencia CU-036: Alternar el tema visual de la interfaz

El usuario personaliza la apariencia de la interfaz eligiendo entre el tema claro y el tema oscuro, según su preferencia visual y sus condiciones de trabajo. El usuario activa el cambio de tema en la interfaz y el sistema aplica el tema seleccionado en toda la interfaz de forma inmediata, sin interrumpir la navegación ni el estado del trabajo que el usuario esté realizando.

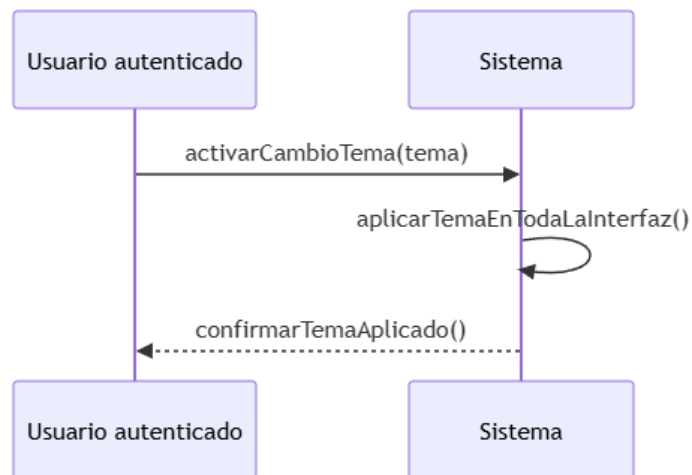


Ilustración x - Secuencia de interacción del cambio del tema visual (CU-036)

Capítulo 15. Verificación de la consistencia del análisis

Este capítulo contrasta los elementos documentales del análisis de vitalXAI y registra las correspondencias confirmadas y las lagunas que siguen abiertas. La revisión se limita a comprobar la trazabilidad entre objetivos, requisitos, casos de uso, subsistemas y entidades del dominio. No demuestra que todas las capacidades estén implementadas ni que todos los requisitos hayan sido verificados mediante pruebas.

15.1. Criterio de verificación

La revisión utiliza como fuentes los objetivos del sistema del capítulo 11, la especificación de requisitos y los casos de uso del capítulo 12, la organización de subsistemas del capítulo 13 y el modelo de dominio del capítulo 14. La comprobación se plantea como una revisión de trazabilidad entre artefactos, no como una prueba de aceptación del producto (Jacobson, Booch, & Rumbaugh, 1999; Larman, 2004). Para cada relación se distingue entre tres situaciones: correspondencia documentada, correspondencia que requiere una interpretación transversal y laguna abierta. Esta distinción evita convertir una referencia existente en una certificación de cumplimiento.

15.2. Objetivos y requisitos

El objetivo del laboratorio de entrenamiento mantiene una laguna conocida. El capítulo 11 declara que el progreso del entrenamiento por épocas está implementado y lo asocia a la prueba PU-023, pero esta capacidad no dispone de un requisito funcional propio. Por tanto, la matriz puede mostrar que OBJ-004 está relacionado con RF-017 a RF-021, RF-030 a RF-032 y RF-041, pero no demuestra que el progreso por épocas esté cubierto por la especificación. Esta cuestión queda abierta para una revisión posterior del catálogo.

La situación del informe individual del diagnóstico se ha actualizado en el capítulo 11: el capítulo 12 ya contiene RF-039 y lo asocia a PU-017. Por ello, esa laguna queda cerrada a nivel de trazabilidad documental; la ejecución y el resultado de la prueba deben comprobarse en los capítulos de verificación correspondientes.

Resultado de la revisión	Estado
Cada objetivo tiene al menos un requisito asociado en la matriz del capítulo 12.	Confirmado como relación documental.
El progreso del entrenamiento por épocas tiene un requisito funcional propio.	No confirmado. La laguna permanece abierta.
El informe individual del diagnóstico tiene un requisito funcional propio.	Confirmado mediante RF-039, su prueba está asociada a PU-017.
La existencia de una relación objetivo requisito demuestra que el objetivo está completamente especificado.	No confirmado.

Tabla x - Resultado de las revisiones

15.3. Requisitos, casos de uso y subsistemas

La matriz de la sección 12.2.4 establece relaciones entre los requisitos funcionales y los casos de uso. La revisión confirma que los requisitos funcionales enumerados en esa matriz tienen una referencia a uno o más casos de uso, con la excepción de RF-005 y RF-006, que son requisitos transversales y condicionan otras interacciones en lugar de constituir casos de uso independientes.

Esta correspondencia tampoco prueba que las funcionalidades estén implementadas. Solo demuestra que la especificación funcional tiene una relación explícita con su comportamiento esperado. En particular, la asociación del progreso del entrenamiento con PU-023 no elimina la ausencia de un requisito funcional específico para esa capacidad.

El capítulo 13 conserva los identificadores SS-001 a SS-006 y documenta su correspondencia con los seis módulos del capítulo 12. Esta relación es útil para ordenar las secuencias del capítulo 14 y las pruebas posteriores, pero no constituye una descomposición independiente ni permite concluir que se hayan demostrado formalmente los niveles de cohesión y acoplamiento.

Comprobación	Resultado
Los requisitos funcionales aparecen relacionados con casos de uso en la matriz del capítulo 12.	Confirmado a nivel documental.
RF-005 y RF-006 tienen casos de uso independientes.	No aplica; son requisitos transversales.
Cada caso de uso se organiza mediante un identificador de subsistema.	Confirmado mediante la correspondencia del capítulo 13.
La organización de subsistemas demuestra por sí sola una arquitectura de bajo acoplamiento.	No confirmado. Esa decisión corresponde al diseño posterior.

Tabla x - Comprobación de requisitos

15.4. Requisitos no funcionales y responsabilidades

La asignación de requisitos no funcionales a subsistemas debe interpretarse como una indicación de ámbito principal, porque muchos de estos requisitos afectan a más de un subsistema. La tabla siguiente identifica el ámbito principal de cada grupo y evita presentar esa asignación como una responsabilidad exclusiva o como una certificación de cumplimiento.

Grupo de requisitos	Requisitos	Ámbito principal de aplicación
Seguridad del acceso	RNF-001 a RNF-005, RNF-007, RNF-008, RNF-010 y RNF-011	SS-001, con aplicación transversal al resto de la plataforma.
Auditoría y eventos de seguridad	RNF-006 y RNF-009	SS-005, con apoyo de los componentes que ejecutan las operaciones supervisadas.
Confidencialidad y protección de datos	RNF-012 a RNF-018	Transversal a SS-001, SS-002, SS-003, SS-004 y SS-005.
Rendimiento y capacidad de respuesta	RNF-019 a RNF-022	SS-002, SS-004 y SS-006 según la operación afectada.
Usabilidad, internacionalización y accesibilidad	RNF-023 a RNF-026	SS-006 y las interfaces de los demás subsistemas.
Robustez y disponibilidad	RNF-027 a RNF-030	SS-006 como ámbito de la cola, con efecto sobre SS-002 y SS-004.

Tabla x - Requisitos no funcionales y responsabilidades

La tabla asigna a cada grupo un ámbito principal coherente con la organización de los capítulos 12 y 13, pero no confirma que todos estén implementados. En el capítulo 12 constan como aprobados y pendientes los requisitos RNF-006, RNF-009, RNF-016, RNF-017, RNF-021, RNF-022, RNF-029 y RNF-032. Por tanto, no puede afirmarse que todos los requisitos no funcionales estén satisfechos o verificados en el estado actual.

15.5. Modelo de dominio y limitaciones

El capítulo 14 contiene once entidades conceptuales. La incorporación de ResultadoPliegue permite representar las métricas de cada pliegue de validación cruzada. ResultadoValidacionExterna representa los resultados de cada modelo sobre la cohorte independiente y ComparacionModelos representa los contrastes entre pares de modelos, incluido el p-valor del test de DeLong (DeLong, DeLong, & Clarke-Pearson, 1988). Estas relaciones hacen que el modelo de dominio sea compatible con RF-022 y RF-029 a nivel conceptual.

La verificación del modelo de dominio tiene un alcance limitado. El capítulo 19 materializa parte de estas entidades mediante ficheros y no mediante tablas. En particular, las carpetas de sesiones y resultados del laboratorio no incluyen una asociación física con user_id, por lo que RF-005 y OBJ-008 no quedan respaldados de forma equivalente a las entidades almacenadas en tablas. La existencia de una clase conceptual no demuestra que todos sus datos estén disponibles ni aislados correctamente en la implementación actual. Esta limitación debe resolverse en una evolución posterior y comprobarse mediante pruebas específicas del laboratorio.

15.6. Resultado de la verificación

La revisión confirma las siguientes correspondencias: los objetivos tienen referencias a requisitos, los requisitos funcionales tienen referencias a casos de uso, los casos de uso conservan identificadores de subsistema y el modelo de dominio contempla las métricas por pliegue y las comparaciones entre modelos.

También confirma las siguientes limitaciones:

- El progreso del entrenamiento por épocas se declara implementado y asociado a PU-023, pero no tiene un requisito funcional propio ni un resultado de verificación documentado en este capítulo.
- Los requisitos no funcionales pendientes no pueden considerarse satisfechos ni verificados.
- La asignación de requisitos no funcionales a subsistemas indica ámbitos principales, pero no demuestra una responsabilidad exclusiva ni un cumplimiento efectivo.
- La existencia de matrices de trazabilidad no demuestra por sí sola la implementación ni la validación del sistema.

En consecuencia, el análisis es trazable en sus relaciones principales, pero no puede certificarse como completamente consistente ni cerrado sin resolver la laguna del progreso del entrenamiento y sin aportar la verificación de los requisitos pendientes. Estas conclusiones se mantienen explícitas para que los capítulos de diseño, implementación y pruebas no presenten como demostrado aquello que el análisis todavía identifica como abierto.

Capítulo 16. Plan de pruebas

Este capítulo define las comprobaciones previstas para evaluar si vitalXAI se comporta conforme a los requisitos del capítulo 12. El plan establece qué se comprueba, qué resultado permitiría considerar superada cada prueba y qué requisitos se relacionan con ella. No documenta ejecuciones ni resultados. Esa información se conserva en los capítulos 33 a 36, mientras que el capítulo 25 define los entornos y niveles de verificación.

La planificación distingue entre pruebas de componentes, pruebas de integración, pruebas de protección, pruebas de rendimiento y pruebas de sistema. Cada prueba recibe un identificador estable para relacionarla con los requisitos, los casos de uso y los resultados que se documenten posteriormente. Los criterios se formulan sobre el comportamiento observable y no sobre los nombres de los ficheros o la organización interna del repositorio (Myers, Sandler, & Badgett, 2011). Los identificadores del plan agrupan el alcance previsto de las comprobaciones; no implican que exista un test con el mismo nombre en el repositorio.

16.1. Pruebas de componentes individuales

Estas pruebas comprueban unidades funcionales de forma aislada. Las dependencias externas se sustituyen por dobles de prueba y se incluyen tanto los caminos normales como las principales condiciones de error, siguiendo el alcance previsto para las pruebas unitarias (Myers, Sandler, & Badgett, 2011).

Identificadores	Ámbito	Criterio general
PU-001 a PU-009	Registro, autenticación, sesiones, validación de entradas y usuarios duplicados.	Las entradas válidas producen el resultado esperado y las inválidas o no autorizadas son rechazadas sin efectos indebidos.
PU-010 a PU-017	Inferencia, confianza, mapas de explicabilidad e informes del diagnóstico.	El sistema produce una predicción válida, los artefactos esperados y un informe accesible cuando los datos son correctos.
PU-018 a PU-027	Asistente, configuración, entrenamiento, sesiones y resultados del laboratorio.	Las operaciones del laboratorio conservan la configuración, actualizan los estados correctos y respetan la propiedad del usuario.
PU-028 a PU-030	Consulta, detalle, renombrado y propiedad del historial.	El usuario solo consulta o modifica sus propias consultas y las operaciones válidas actualizan el historial correctamente.
PU-031 a PU-032	Operaciones de supervisión administrativa.	Las operaciones reservadas al administrador rechazan a los usuarios sin el rol requerido.

Identificadores	Ámbito	Criterio general
PU-033 a PU-035	Consulta, cancelación y procesamiento de trabajos.	Los trabajos cambian de estado conforme al flujo previsto y solo pueden cancelarse mientras permanecen pendientes.
PU-036 a PU-037	Internacionalización de la interfaz.	El idioma seleccionado se aplica correctamente y se utiliza el idioma de respaldo cuando falta una traducción.

Tabla x - Pruebas de componentes individuales

16.2. Pruebas de los flujos entre subsistemas

Estas pruebas comprueban la colaboración entre varias partes del sistema. Su objetivo es comprobar que una operación iniciada por el usuario atraviesa correctamente la autenticación, la lógica de aplicación, la persistencia y, cuando corresponde, la cola de trabajos (Myers, Sandler, & Badgett, 2011).

Identificador	Flujo	Criterio de éxito
PI-001	Registro, inicio de sesión y acceso al panel.	El usuario se registra, inicia sesión y accede al área privada.
PI-002	Acceso privado sin sesión.	El sistema rechaza o redirige al usuario no autenticado.
PI-003	Historial de un usuario recién registrado.	El historial inicial aparece vacío para el usuario recién registrado.
PI-004	Diagnóstico completo desde la carga hasta el resultado.	La imagen se valida, el trabajo se encola y el resultado queda disponible.
PI-005	Aislamiento de datos entre usuarios.	Un usuario no puede consultar ni modificar consultas o sesiones ajenas.
PI-006	Supervisión administrativa.	El administrador puede consultar la información permitida y un usuario ordinario no puede hacerlo.
PI-007	Laboratorio desde la configuración hasta la cola.	La configuración válida crea la sesión y el trabajo se encola para su procesamiento.

Tabla x - Pruebas de los flujos entre subsistemas

16.3. Pruebas de protección y control de acceso

Estas pruebas comprueban los requisitos de seguridad del capítulo 12 mediante escenarios controlados y comprobaciones de las respuestas del sistema. El catálogo de amenazas de OWASP constituye el marco de referencia de los controles de seguridad del proyecto (OWASP, 2024).

Identificador	Protección	Criterio de éxito
PS-001 a PS-004	Protección frente a falsificación de peticiones.	Las peticiones que modifican el estado requieren un token válido y las peticiones fraudulentas son rechazadas.
PS-005 a PS-007	Cabeceras de seguridad.	Las respuestas incluyen las protecciones exigidas por el requisito correspondiente.
PS-008	Limitación de peticiones de acceso.	Tras cinco peticiones fallidas desde una misma dirección en un minuto, la siguiente petición de acceso debe rechazarse y el bloqueo debe mantenerse durante el intervalo definido.
PS-009	Tokens de sesión revocados.	Una credencial de refresco revocada no permite renovar la sesión.

Tabla x - Pruebas de protección y control de acceso

Los requisitos de auditoría y registro de eventos de seguridad, así como los controles de protección de datos pendientes, se mantienen identificados en la matriz de trazabilidad como comprobaciones que requieren una prueba específica antes de poder declararse verificados.

16.4. Pruebas de rendimiento y capacidad de respuesta

Estas pruebas se ejecutan sobre el entorno de sistema definido en el capítulo 25. Sus criterios proceden directamente de los requisitos del capítulo 12.

Identificador	Requisito	Criterio de éxito
PR-001	RNF-019	El identificador del trabajo se devuelve en menos de 2 segundos.
PR-002	RNF-019	La inferencia con el modelo ya cargado finaliza en menos de 15 segundos.
PR-003	RNF-020	La interfaz continúa respondiendo durante una tarea de larga duración.
PR-004	RNF-021	Diez usuarios autenticados realizan operaciones concurrentes sin superar el doble del tiempo de carga baja y preservando el aislamiento.
PR-005	RNF-021	Las operaciones concurrentes se atienden sin errores de conexión.

Tabla x - Pruebas de rendimiento y capacidad de respuesta

16.5. Pruebas de sistema y validación funcional

Estas pruebas comprueban los flujos completos desde la interfaz y sobre el entorno de sistema. Incluyen los escenarios principales de uso y las operaciones que atraviesan varios subsistemas.

Identificador	Flujo	Criterio de éxito
PE-001	Diagnóstico completo, incluidos resultado, explicabilidad e informe.	El profesional completa el flujo y obtiene todos los artefactos previstos.
PE-002	Gestión del historial.	El usuario consulta, visualiza, renombra y elimina sus consultas correctamente.
PE-003	Laboratorio MLOps.	El usuario configura y lanza un experimento y puede consultar sus resultados e informe.
PE-004	Cola de trabajos.	El usuario consulta el estado y cancela trabajos conforme a las condiciones permitidas.
PE-005	Supervisión administrativa.	El administrador consulta usuarios y consultas conforme a sus permisos.
PE-006	Idioma y tema visual.	Las preferencias se aplican sin perder el estado de navegación.

Tabla x - Pruebas de sistema y validación funcional

16.6. Matriz de trazabilidad entre requisitos y pruebas

La matriz relaciona los requisitos con las pruebas previstas para aportar evidencia sobre su comportamiento. Una relación en esta tabla no demuestra que la prueba se haya ejecutado ni que el requisito esté verificado. Una celda vacía o una observación de pendiente indica que el plan no define una comprobación suficiente para declararlo verificado.

Requisitos	Pruebas relacionadas	Alcance de la comprobación
RF-001 a RF-006	PU-001 a PU-009, PI-001 a PI-003, PS-008 y PS-009	Cuenta, acceso, sesión, validación, propiedad y control de acceso.
RF-007 a RF-012	PU-010 a PU-016, PI-004 y PE-001	Flujo de diagnóstico, resultado y explicabilidad.
RF-013 a RF-016	PU-028 a PU-030, PI-003, PI-005 y PE-002	Consulta y gestión del historial propio.
RF-017 a RF-032	PU-018 a PU-027, PI-007 y PE-003	Configuración, entrenamiento, resultados, comparación y validación del laboratorio.

Requisitos	Pruebas relacionadas	Alcance de la comprobación
RF-033 a RF-035	PU-031 y PU-032, PI-006 y PE-005	Supervisión administrativa y restricciones de rol.
RF-036 a RF-038	PU-033 a PU-037, PE-004 y PE-006	Cola de trabajos, cancelación e internacionalización. La cancelación prevista se limita a trabajos pendientes.
RF-039	PU-017 y PE-001	Informe PDF individual del diagnóstico.
RF-040	PI-006 y PE-005	Gestión de cuentas por el administrador.
RF-041	PI-007 y PE-003	Limitación de entrenamientos simultáneos y encolados.
RNF-001 a RNF-005, RNF-007, RNF-010 y RNF-011	PU-001, PU-007 a PU-009, PS-001 a PS-009	Protección de credenciales, entradas, peticiones, respuestas y sesiones. La cobertura de cada requisito debe confirmarse en los resultados específicos.
RNF-006 y RNF-009	Pendiente de prueba específica	Auditoría administrativa y registro de eventos de seguridad.
RNF-012 a RNF-018	PI-004 a PI-007 y PE-001 a PE-005	Comprobaciones previstas sobre protección y tratamiento de datos en los flujos que los utilizan. La cobertura normativa completa y la evidencia de ejecución requieren revisión específica.
RNF-019 a RNF-022	PR-001 a PR-005, PI-004, PI-007 y PE-003	Rendimiento, ejecución asíncrona, concurrencia y comportamiento con recursos ocupados.
RNF-023 a RNF-026	PU-036 a PU-037 y PE-006	Comprobaciones previstas sobre idioma, uso, presentación y accesibilidad. La usabilidad no se valida con usuarios reales y la evidencia de todos los criterios requiere revisión específica.
RNF-027 a RNF-030	PU-033 a PU-035, PR-003, PE-003 y PE-004	Disponibilidad, errores, recuperación y aislamiento de fallos en trabajos.
RNF-031 y RNF-032	PU-018 a PU-027 y PE-002 a PE-003	Integridad, persistencia y recuperación de la información.
RNF-033 y RNF-034	PU-018 a PU-027 y PE-003	Reproducibilidad de experimentos y del entorno de ejecución.
RNF-035 a RNF-038	No aplica como prueba de sistema	Compromisos del proceso y entregables del proyecto. Se prevé verificarlos mediante revisión documental y de calidad.

Tabla x - Matriz de trazabilidad entre requisitos y pruebas

La matriz permite localizar qué comprobaciones previstas deben contrastarse en los capítulos 33 a 36. También deja visibles las áreas sin prueba específica y evita afirmar que una relación documental equivale a una verificación superada. Los resultados de cada prueba, los recuentos y la cobertura real se documentan únicamente en la parte de verificación.

Parte III.

Diseño del Sistema

Capítulo 17. Diseño y arquitectura de vitalXAI

El presente capítulo inicia la parte de diseño de vitalXAI y transforma las decisiones funcionales establecidas durante el análisis en una organización técnica concreta. Mientras que los capítulos anteriores describen qué debe hacer la plataforma, qué actores intervienen y qué requisitos debe satisfacer, el diseño determina cómo se estructuran los componentes que permiten ofrecer esas capacidades. Esta transición se realiza manteniendo la trazabilidad con los objetivos, requisitos, casos de uso y subsistemas definidos en los capítulos 11 a 14.

El diseño se apoya en una aplicación web de servidor desarrollada con Python y FastAPI (FastAPI, 2024). La solución proporciona al usuario una interfaz accesible desde el navegador, expone los servicios funcionales mediante rutas HTTP, persiste la información operativa en MySQL (Oracle, 2024) y delega los trabajos de mayor duración en un worker interno de ejecución asíncrona. Sobre esta estructura se integran el motor de inferencia de imágenes, los módulos de explicabilidad, el laboratorio de experimentación MLOps, el asistente conversacional conectado con Groq (Groq, 2024) y los generadores de informes PDF.

La organización técnica no reproduce de forma literal la distribución de los subsistemas de análisis. Estos subsistemas continúan siendo la referencia funcional para garantizar la trazabilidad, pero en el diseño se agrupan los elementos según sus responsabilidades de ejecución: presentación, entrada HTTP, lógica de aplicación y aprendizaje automático, persistencia y procesamiento asíncrono. Esta separación permite que la interfaz no tenga que conocer los detalles del motor de inferencia o de la base de datos, y que las tareas computacionalmente costosas puedan continuar sin mantener bloqueada la petición que las originó (Larman, 2004).

La arquitectura descrita en este capítulo corresponde a la implementación actual del sistema. Por este motivo, se documentan FastAPI, las plantillas Jinja2, los scripts JavaScript, MySQL, el pool de conexiones, la cola persistida en la base de datos y el worker basado en ``asyncio``, sin sustituirlos por tecnologías previstas en versiones anteriores de la documentación o por componentes que no forman parte del código ejecutable. La correspondencia entre esta solución y los subsistemas de análisis se desarrolla en los apartados posteriores del documento de diseño.

17.1. Organización arquitectónica de la solución

La solución adopta una arquitectura cliente-servidor organizada en capas lógicas y servicios internos especializados. El cliente es el navegador del usuario, que recibe las páginas HTML renderizadas mediante Jinja2 y ejecuta los recursos JavaScript y CSS de la interfaz. El servidor concentra la lógica de negocio, la validación de las peticiones, la gestión de la identidad, el acceso a los datos y la coordinación de los trabajos de inteligencia artificial. La base de datos MySQL proporciona la persistencia estructurada, mientras que la cola de trabajos y el worker interno desacoplan las operaciones de larga duración del ciclo de petición HTTP.

Esta decisión responde a dos características esenciales de vitalXAI. La primera es que la complejidad computacional debe permanecer en el servidor: el navegador envía una imagen, una configuración o una solicitud de consulta, pero no carga los modelos de aprendizaje profundo ni ejecuta los entrenamientos. La segunda es que algunas operaciones no pueden resolverse de manera fiable dentro del tiempo de una petición ordinaria. La inferencia, la generación de mapas de explicabilidad, el entrenamiento y la validación externa pueden requerir un tiempo considerable; por ello, se registran como trabajos en `job_queue` y se procesan posteriormente mediante el worker de la aplicación.

17.1.1. Capa de presentación y acceso desde el navegador

La capa de presentación está formada por las plantillas HTML del directorio `templates`, los recursos estáticos del directorio `static` y el navegador del usuario. La aplicación no utiliza una SPA independiente: las páginas se generan en el servidor mediante Jinja2 y se sirven como respuestas HTML desde las rutas correspondientes. Entre las vistas principales se encuentran las páginas de inicio de sesión y registro, el panel de diagnóstico, el laboratorio de entrenamiento y la interfaz de administración.

Los scripts JavaScript asociados a las páginas complementan este renderizado con interacciones asíncronas. Por medio de `fetch`, el navegador puede enviar formularios, cargar imágenes, consultar estados de trabajos, recuperar resultados y actualizar partes de la interfaz sin tener que reconstruir toda la página en cada operación. Los recursos `dashboard.js`, `training.js`, `admin.js` e `i18n.js` concentran la lógica de interacción propia de las vistas, mientras que los estilos y las imágenes se sirven como recursos estáticos.

La capa de presentación no constituye una frontera de seguridad suficiente por sí misma. Su función es mostrar la información y recoger las acciones del usuario, pero la autenticación, la autorización, la validación definitiva de los datos y el aislamiento entre usuarios se vuelven a comprobar en el servidor. Esta decisión evita que una modificación de las peticiones desde el navegador permita acceder a consultas, sesiones de entrenamiento o funciones administrativas que no correspondan al usuario autenticado.

17.1.2. Capa HTTP y coordinación de los subsistemas funcionales

FastAPI constituye la entrada principal al servidor. La instancia se crea en `main.py`, donde se registra el ciclo de vida de la aplicación, se montan los recursos estáticos y se incorporan los routers funcionales. La organización por routers mantiene separadas las responsabilidades de acceso y evita concentrar todos los endpoints en un único módulo. La aplicación incorpora los routers de autenticación, diagnóstico, historial, cola de trabajos, laboratorio MLOps y administración.

Cada router recibe la petición HTTP, valida sus parámetros, obtiene la identidad del usuario cuando es necesario y delega la operación en el componente especializado. Así, `auth.py` coordina el registro, el inicio y el cierre de sesión; `inference.py` inicia las consultas de diagnóstico; `history.py` gestiona el historial; `trainer.py` coordina el laboratorio MLOps; `queue.py` expone el estado y la cancelación de trabajos; y `admin.py` restringe las operaciones de supervisión al rol de administrador. Esta distribución mantiene la relación entre las rutas de la API y los casos de uso sin convertir cada router en el responsable de la persistencia o del cálculo científico.

La instancia de FastAPI también concentra los elementos transversales que deben aplicarse a todas o a gran parte de las peticiones. `CSRFMiddleware` comprueba el token de protección en las operaciones que modifican el estado, `SecurityHeadersMiddleware` añade las cabeceras de seguridad configuradas y el limitador basado en `slowapi` controla la frecuencia de las operaciones sensibles, especialmente el acceso. Estos mecanismos se registran en el punto de creación de la aplicación para que no dependan de que cada router los aplique manualmente.

17.1.3. Servicios de aplicación y motores de inteligencia artificial

La lógica que no pertenece directamente al protocolo HTTP se organiza en el directorio `services`. Esta capa contiene servicios de aplicación y motores especializados que son invocados por los routers o por el worker de la cola. La separación es especialmente importante en `vitalXAI` porque las operaciones de una consulta clínica no se limitan a producir una etiqueta: también pueden generar un mapa de explicabilidad, crear un informe PDF y persistir el resultado asociado al usuario.

El servicio de autenticación encapsula la creación y comprobación de credenciales, el hash de contraseñas mediante `bcrypt`, la generación de tokens y la gestión de su renovación y revocación. `ml_engine.py` prepara la imagen, carga las arquitecturas disponibles y ejecuta la inferencia mediante TensorFlow/Keras o mediante modelos de Transformers (TensorFlow, 2024; Hugging Face, 2024). Las arquitecturas basadas en atención se obtienen a través de la configuración de Hugging Face utilizada por el proyecto. `xai_generator.py` produce los mapas de explicabilidad correspondientes a la arquitectura seleccionada, y `pdf_generator.py` transforma el resultado de una consulta en un informe descargable.

El laboratorio de experimentación se divide en servicios que reflejan sus distintas responsabilidades. `chatbot_service.py` encapsula la comunicación con la API de Groq y permite que el asistente interprete los parámetros del experimento; `trainer_engine.py` prepara los datos y ejecuta el entrenamiento; `mlops_engine.py` gestiona las sesiones, los modelos y los resultados; y `pdf_generator_mlops.py` genera los informes de las sesiones de experimentación. Esta división permite que el router del laboratorio coordine el flujo sin contener la implementación completa del entrenamiento o de la comunicación con el servicio externo.

Los servicios externos se tratan como fronteras de integración. La API de Groq se utiliza únicamente desde el servicio conversacional y requiere la variable de entorno GROQ_API_KEY. Los modelos de Transformers se resuelven desde Hugging Face cuando corresponde y se cargan en el motor de aprendizaje automático. El worker transforma los fallos de estos servicios en trabajos fallidos, pero algunos routers todavía pueden devolver detalles de excepciones en sus respuestas de error; esta exposición debe endurecerse antes de un despliegue de producción.

17.1.4. Persistencia relacional y ejecución asíncrona

La persistencia se implementa mediante MySQL y el conector oficial mysql-connector-python. El módulo database.py proporciona un MySQLConnectionPool reutilizable, obtiene los parámetros de conexión desde variables de entorno y expone la función que entrega conexiones a los routers y servicios que necesitan consultar o modificar datos. La inicialización de la aplicación verifica la existencia de las tablas principales: users, consultations, training_jobs, job_queue y refresh_tokens.

Las relaciones de propiedad de las consultas y los trabajos se mantienen en la base de datos mediante claves ajenas que los vinculan con los usuarios. En las sesiones del laboratorio, el user_id se conserva en config.json y la aplicación lo comprueba al acceder, aunque no existe una relación relacional equivalente. Por ello, la propiedad de las sesiones tiene un respaldo distinto al del historial y los trabajos. El diseño también permite persistir el resultado de una operación larga después de que la petición HTTP inicial haya finalizado: el cliente conserva el identificador del trabajo y consulta posteriormente su estado y su resultado.

La ejecución asíncrona no depende de un broker externo. queue.py ofrece las operaciones de consulta y cancelación, mientras que queue_worker.py implementa el consumidor interno. Durante el ciclo de vida de FastAPI, start_worker restablece los trabajos que quedaron en ejecución tras una interrupción y crea una tarea asyncio que inspecciona periódicamente la tabla job_queue. El worker selecciona el siguiente trabajo pendiente, intenta reclamarlo de forma condicional y ejecuta el diagnóstico, el entrenamiento o la validación externa según el tipo registrado.

El procesamiento pesado se ejecuta mediante run_in_executor, evitando que el cálculo bloquee el bucle de eventos de la aplicación. Una vez terminada la operación, el worker almacena el resultado y marca el trabajo como completado; si se produce una excepción, registra el error y lo marca como fallido. Esta máquina de estados, queued, running, completed, failed y cancelled, permite que la interfaz informe del progreso sin mantener abierta la petición que creó el trabajo. La cancelación se limita a los trabajos que aún están pendientes, lo que evita interrumpir de forma inconsistente una operación que ya ha comenzado.

17.1.5. Vista global de la arquitectura

La siguiente ilustración resume las relaciones entre los elementos principales de la solución. El navegador consume las páginas y recursos de presentación y se comunica con FastAPI mediante peticiones HTTP. FastAPI dirige cada operación hacia el router y el servicio correspondiente, utiliza MySQL para la información persistente y registra en `job_queue` las operaciones que deben ejecutarse fuera del ciclo inmediato de la petición. El worker recupera esos trabajos y utiliza los motores de inferencia, explicabilidad, entrenamiento y generación de informes antes de persistir sus resultados.

La arquitectura resultante conserva una separación clara entre la interacción del usuario, la coordinación HTTP, la lógica especializada, la persistencia y la ejecución de trabajos. La aplicación sigue siendo desplegable como una única solución, pero sus responsabilidades internas están suficientemente delimitadas para que una modificación en la interfaz no obligue a reescribir el motor de inferencia, que una nueva arquitectura de modelo se incorpore dentro de los servicios de aprendizaje automático y que los cambios en la presentación no alteren el mecanismo de cola. Esta organización materializa las dependencias identificadas en el análisis: los subsistemas de diagnóstico y laboratorio dependen de la identidad y la persistencia, y ambos utilizan el mecanismo transversal de trabajos asíncronos para ejecutar las operaciones de larga duración.

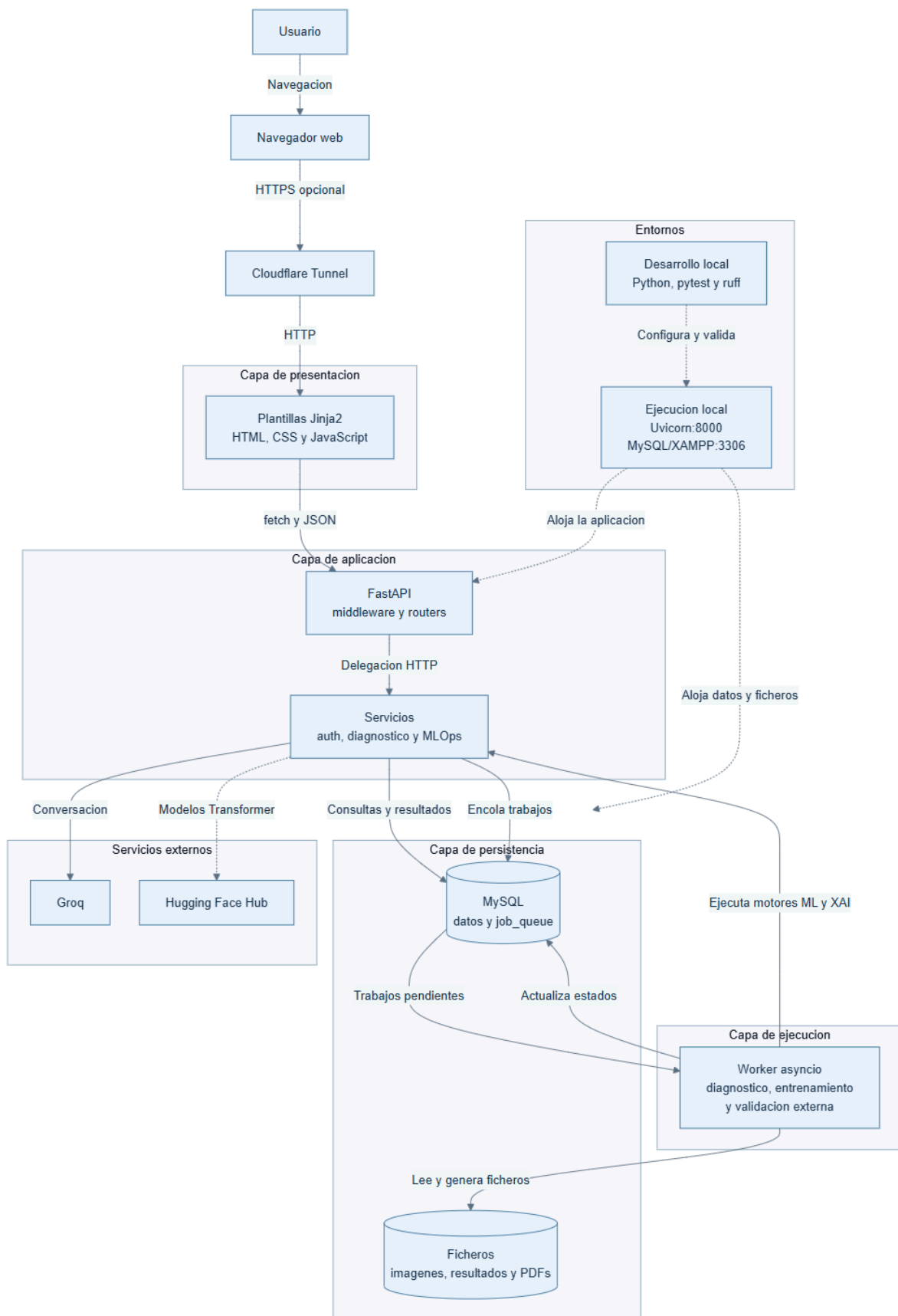


Ilustración x - Organización arquitectónica de vitalXAI

17.2. Condiciones de calidad, normativa y restricciones del diseño

La arquitectura definida en el apartado anterior no constituye una elección aislada de tecnologías. Cada una de sus decisiones responde a los requisitos no funcionales, a las normas aplicables al dominio sanitario y a las limitaciones del entorno en el que se construye y ejecuta vitalXAI. En consecuencia, este apartado relaciona las condiciones de calidad del capítulo 12 con las decisiones técnicas que las materializan, distingue los estándares que deben observarse durante el desarrollo y delimita las restricciones que condicionan la solución.

17.2.1. Requisitos no funcionales y respuesta arquitectónica

Los requisitos no funcionales se han agrupado en el análisis en siete dimensiones del sistema: seguridad y acceso, confidencialidad de los datos, rendimiento, sencillez de uso, robustez, persistencia y reproducibilidad. A estas dimensiones se añaden los compromisos del proceso y los entregables, que el capítulo 12 mantiene separados de los requisitos del sistema. En la fase de diseño no todos ellos se traducen en una clase o un módulo independiente. Algunos se materializan mediante middleware, configuración o separación de responsabilidades; otros corresponden a procedimientos del entorno de ejecución y a los documentos que acompañan al software. La tabla resume la relación entre cada grupo y la respuesta adoptada en la implementación actual.

Grupo de requisitos	Requisitos relacionados	Decisión de diseño y mecanismo aplicado
Seguridad del acceso	RNF-001 a RNF-011	Hash de contraseñas con bcrypt, tokens de acceso y refresco, cookies HttpOnly con SameSite=Lax, middleware CSRF, limitación de acceso mediante slowapi y cabeceras de seguridad. Las operaciones administrativas se protegen mediante el rol del usuario, la auditoría y el registro formal de eventos permanecen pendientes.
Confidencialidad de los datos	RNF-012 a RNF-018	Comprobación de propiedad en las rutas privadas, uso de datasets anonimizados y exposición del servicio mediante HTTPS cuando se accede desde el exterior. Las sesiones del laboratorio se relacionan con el usuario mediante user_id en su configuración, pero no mediante una relación relacional.
Rendimiento y respuesta	RNF-019 a RNF-022	Reutilización de modelos cargados, pool de conexiones MySQL, ejecución de trabajos mediante job_queue y worker asyncio, y delegación del cálculo pesado en run_in_executor.
Usabilidad y accesibilidad	RNF-023 a RNF-026	Interfaz HTML/Jinja2 con JavaScript vanilla, asistencia conversacional para configurar experimentos, servicio de traducciones, cambio de tema visual y diseño adaptable de las vistas.

Grupo de requisitos	Requisitos relacionados	Decisión de diseño y mecanismo aplicado
Robustez y disponibilidad	RNF-027 a RNF-030	Estados persistentes de los trabajos, registro de errores, recuperación de trabajos que quedaron en ejecución al reiniciar el servidor y aislamiento de fallos.
Persistencia y salvaguarda de los datos	RNF-031 y RNF-032	Integridad de los registros, copias de seguridad y recuperación de la base de datos y de los ficheros de artefactos.
Reproducibilidad del sistema	RNF-033 y RNF-034	Versiones fijadas en requirements.txt, entorno virtual de Python, separación entre routers, servicios y persistencia, y ejecución de comprobaciones automatizadas mediante pytest y ruff.
Compromisos del proceso	RNF-035 y RNF-036	Estándares de codificación y separación de responsabilidades aplicados durante el desarrollo.
Documentación y entregables	RNF-037 y RNF-038	Memoria estructurada por fases y manual de usuario. La información de configuración y despliegue se mantiene como documentación interna de apoyo.

Tabla x - Relación entre los requisitos no funcionales y las decisiones de diseño

17.2.1.1. Seguridad, identidad y control del acceso

La identidad del usuario constituye la primera condición de seguridad de la plataforma. El servicio de autenticación no almacena contraseñas originales, sino valores generados mediante bcrypt, y la tabla users conserva únicamente el campo password_hash. La sesión se articula mediante un token de acceso y un token de refresco. Ambos se entregan mediante cookies protegidas frente al acceso de JavaScript mediante el atributo HttpOnly, mientras que el token de refresco limita además su ruta de envío al endpoint de renovación. La rotación y revocación de los tokens se apoya en la tabla refresh_tokens, que conserva su hash, su fecha de expiración y su estado de revocación.

El diseño incorpora una protección adicional frente a peticiones CSRF. El middleware genera una cookie csrf_token accesible desde el código de la interfaz y exige que las peticiones que modifican el estado incluyan el mismo valor en la cabecera X-CSRF-Token. Las operaciones de lectura permanecen fuera de esta comprobación y la ruta de inicio de sesión se trata como excepción específica para permitir el acceso inicial. La comparación se realiza en el servidor antes de entregar la petición al router correspondiente.

La limitación de frecuencia se aplica especialmente al inicio de sesión, que se encuentra limitado a cinco peticiones por minuto mediante slowapi. Esta medida está prevista para reducir la viabilidad de ataques automatizados de fuerza bruta, en línea con las recomendaciones de seguridad web de OWASP (OWASP, 2024), sin incorporar la lógica de limitación dentro del router de autenticación. A este mecanismo se suman las cabeceras emitidas por SecurityHeadersMiddleware, entre ellas Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security y Referrer-Policy.

El control de roles se aplica en el router de administración. El usuario administrador puede consultar el listado de usuarios y supervisar las consultas de una cuenta, mientras que un usuario ordinario no puede invocar esas operaciones. No obstante, el endpoint actual de registro acepta un campo role enviado por el cliente y lo persiste, por lo que la asignación inicial del rol no debe considerarse un control seguro; debería fijarse en el servidor a un rol ordinario. El aislamiento de datos se comprueba de forma independiente en cada operación que recupera o modifica una consulta, una sesión de entrenamiento o un trabajo. Esta comprobación no se delega en la interfaz, ya que el navegador puede ser manipulado por el usuario que realiza la petición.

Debe distinguirse entre el control de acceso administrativo y la auditoría formal de la actividad administrativa. La implementación actual restringe las rutas de administración mediante el rol y materializa RF-006, pero no introduce una tabla independiente de trazas con el detalle histórico de cada operación. Por ello, RNF-006, que exige la auditoría de esas operaciones, permanece pendiente de implementación. El mecanismo de autorización sí forma parte de la arquitectura implementada, pero no sustituye al registro histórico exigido por RNF-006.

17.2.1.2. Confidencialidad y tratamiento de la información

La aplicación trabaja con cuentas de usuario, radiografías y resultados de diagnóstico. El diseño relacional vincula las consultas, los trabajos y los entrenamientos previstos con el identificador del usuario propietario, y las rutas utilizan esa relación para limitar los resultados relacionales. Las sesiones y los resultados efectivos del laboratorio se conservan en `training_results` sin esa asociación física, por lo que el aislamiento de ese contenido constituye una limitación pendiente.

El cumplimiento de los requisitos de privacidad no se resuelve únicamente con una configuración de software. Los datasets de entrenamiento y validación deben estar anonimizados antes de incorporarse al proyecto, y el personal que sube una imagen debe garantizar que no contiene información identificable del paciente. La plataforma almacena rutas de ficheros y resultados asociados al usuario profesional, pero no incorpora un módulo de identificación del paciente ni debe utilizarse para introducir datos personales que no sean necesarios para la consulta. Además, `training_results` está montado como recurso estático en `main.py`, por lo que la protección de las URL de los artefactos requiere una revisión antes de cualquier despliegue con datos sensibles.

La confidencialidad durante el tránsito depende del entorno de exposición. En el uso local, Uvicorn atiende la aplicación en el puerto 8000; cuando se publica el sistema mediante el túnel configurado para la demostración, el acceso exterior se realiza mediante HTTPS. Las cookies de sesión incorporan actualmente `HttpOnly` y `SameSite=Lax`; la protección del canal debe mantenerse como condición obligatoria del entorno de despliegue para evitar que las credenciales y los datos viajen sin cifrar.

17.2.1.3. Rendimiento, concurrencia y ejecución sin bloqueo

El requisito de rendimiento más relevante no consiste únicamente en reducir el tiempo de una función concreta, sino en impedir que una operación de aprendizaje automático monopolice el ciclo de petición. Por este motivo, el diagnóstico, el entrenamiento y la validación externa se registran primero como trabajos en la tabla `job_queue`. La petición HTTP puede devolver el identificador y el estado inicial del trabajo, mientras que el worker continúa el procesamiento de forma independiente.

El worker se inicia durante el ciclo de vida de FastAPI y consulta periódicamente los trabajos pendientes. Para evitar que el cálculo de TensorFlow, la generación de mapas XAI o la ejecución del entrenamiento bloqueen el bucle de eventos, el trabajo se envía a `run_in_executor`. El resultado o el mensaje de error se almacena después en MySQL. Así, el usuario puede consultar la cola mientras el procesamiento continúa y la plataforma puede atender otras peticiones.

La concurrencia de acceso a la persistencia se aborda mediante `MySQLConnectionPool`. El tamaño del pool se configura con `DB_POOL_SIZE`, mientras que los restantes datos de conexión se obtienen mediante variables de entorno. Esta decisión evita crear una conexión nueva de forma indiscriminada para cada operación y permite controlar el número de conexiones simultáneas en función del entorno disponible. La concurrencia no elimina la necesidad de validar la propiedad de cada registro: el rendimiento y el aislamiento de datos son condiciones simultáneas.

17.2.1.4. Usabilidad, internacionalización y presentación

La elección de plantillas Jinja2 y JavaScript vanilla reduce la complejidad del cliente y evita introducir una cadena de compilación independiente. Las páginas se entregan directamente desde el servidor y los scripts añaden únicamente las interacciones necesarias para cargar imágenes, consultar estados, mostrar resultados y actualizar el laboratorio. La arquitectura facilita que la interfaz pueda funcionar desde un navegador sin instalar software adicional.

El servicio lang.py centraliza la recuperación de textos traducidos y permite seleccionar el idioma mediante la cookie correspondiente. Esta capacidad se utiliza desde las vistas de autenticación, el panel de diagnóstico y el laboratorio. El cambio de tema visual se realiza en el cliente y se conserva como preferencia de la interfaz. La presentación de los resultados se organiza alrededor de las necesidades del usuario: diagnóstico y confianza, mapas de explicabilidad, histórico de consultas, progreso de trabajos y resultados del laboratorio.

17.2.1.5. Robustez, persistencia y recuperación

La tabla job_queue actúa como soporte persistente de la ejecución asíncrona. A diferencia de una cola exclusivamente residente en memoria, permite conservar el estado de un trabajo y recuperarlo después de reiniciar la aplicación. Al iniciar el worker, _reset_running_jobs() devuelve al estado queued los trabajos que quedaron en running, de modo que no permanezcan bloqueados indefinidamente. Cada ejecución puede terminar en completed o failed, y en el segundo caso se conserva un mensaje de error limitado en longitud.

Las claves ajenas de las tablas consultations, training_jobs, job_queue y refresh_tokens apuntan a users. Esta relación refuerza la integridad del modelo y permite eliminar los datos dependientes de un usuario cuando la operación está configurada con ON DELETE CASCADE. Las escrituras se confirman explícitamente mediante commit y las conexiones se cierran al finalizar cada operación.

La copia de seguridad periódica y la restauración de MySQL pertenecen al entorno operativo, no a un servicio interno de la aplicación. El diseño identifica esta dependencia para que el despliegue no confunda la persistencia local con una estrategia de recuperación completa. La guía de despliegue debe conservar, por tanto, las instrucciones necesarias para iniciar MySQL y verificar la disponibilidad de la base de datos antes de ejecutar la plataforma.

17.2.2. Estándares y normas aplicables

El sistema se desarrolla bajo normas legales y técnicas que afectan a distintos niveles de la solución. En materia de privacidad se aplica el Reglamento General de Protección de Datos, Reglamento (UE) 2016/679, junto con la Ley Orgánica 3/2018 de protección de datos personales y garantía de los derechos digitales. Estas normas condicionan la minimización de la información recogida, la anonimización de las imágenes, la asociación de los datos a una cuenta y la forma en que se documenta el tratamiento de la información sanitaria (Parlamento Europeo y Consejo de la Unión Europea, 2016; España, 2018).

Para la protección de las comunicaciones se adopta HTTPS apoyado en TLS. El túnel utilizado para exponer la aplicación debe proporcionar el canal cifrado entre el navegador y el entorno local, y las cabeceras de seguridad del servidor complementan esa protección. La existencia de HTTPS no sustituye a la autenticación, a la autorización ni a la protección CSRF; son mecanismos que operan en capas distintas y deben mantenerse conjuntamente (IETF, 2018).

El código Python sigue las convenciones de PEP 8. La aplicación utiliza ruff como comprobación automatizada de estilo y mantiene un fichero requirements.txt con versiones fijadas de sus dependencias principales. La memoria y el manual de usuario se elaboran conforme a la normativa del Trabajo Fin de Grado de la Escuela Politécnica Superior de la Universidad Pablo de Olavide, que constituye el marco formal de los entregables del proyecto (Python Software Foundation, 2024; Universidad Pablo de Olavide, 2014).

La siguiente tabla resume el ámbito de aplicación de las normas anteriores.

Estándar / Norma	Ámbito de aplicación
RGPD (Reglamento UE 2016/679)	Protección de datos y tratamiento de información biomédica.
LOPDGDD (Ley Orgánica 3/2018)	Adaptación del RGPD al ordenamiento jurídico español.
HTTPS / TLS	Protocolo seguro de comunicación entre cliente y servidor.
OWASP Top 10	Seguridad de las aplicaciones web.
PEP 8	Estándar de estilo y nomenclatura del código Python.

Tabla x - Normas y estándares aplicables al diseño

17.2.3. Restricciones técnicas y límites del entorno

La primera restricción técnica es el ecosistema de aprendizaje automático. La aplicación se ejecuta sobre Python 3.11 y depende de TensorFlow, Keras, OpenCV, scikit-learn y Transformers (TensorFlow, 2024; Hugging Face, 2024). Esta combinación condiciona la versión del intérprete, el sistema operativo y la disponibilidad de recursos de memoria. Los modelos convolucionales y Transformer no se consideran componentes ligeros del servidor: sus pesos deben cargarse en memoria y los entrenamientos pueden requerir una GPU compatible con el entorno de TensorFlow.

La segunda restricción es la capacidad de cómputo local. El entrenamiento y la validación de las arquitecturas demandan más recursos que una consulta web ordinaria, por lo que el entorno de desarrollo debe disponer de hardware suficiente, especialmente durante la ejecución del laboratorio MLOps. La interfaz y los routers no deben asumir que el resultado está disponible de forma inmediata; esta condición es precisamente la que justifica el diseño basado en trabajos persistentes y worker interno.

La tercera restricción es la disponibilidad de MySQL. La aplicación depende de una instancia accesible mediante los parámetros DB_HOST, DB_USER, DB_PASSWORD, DB_NAME y DB_POOL_SIZE. En el entorno de demostración, MySQL se ejecuta mediante XAMPP. Si el servicio no está iniciado o las credenciales no son válidas, la inicialización de la base de datos no puede completarse, aunque el ciclo de vida de FastAPI informa del problema y continúa hasta iniciar el worker.

La cuarta restricción procede de los servicios externos. El asistente conversacional requiere GROQ_API_KEY y conectividad con la API de Groq. La carga de los modelos Transformer puede requerir acceso a Hugging Face, salvo que los modelos ya estén disponibles localmente. Estas dependencias introducen límites de disponibilidad y latencia que el sistema debe tratar como fallos de integración, no como errores de lógica interna. Por este motivo, las pruebas unitarias sustituyen los servicios externos por mocks y la ejecución de producción debe conservar las variables de entorno fuera del código fuente.

Finalmente, la exposición remota mediante el túnel no convierte el entorno local en una infraestructura distribuida. La aplicación, el worker, los ficheros y la base de datos continúan alojados en la máquina de ejecución. El túnel únicamente proporciona el canal de acceso desde el navegador externo. Esta decisión simplifica la demostración y mantiene bajo control las versiones del entorno, pero limita la escalabilidad, la tolerancia a fallos hardware y la disponibilidad de la plataforma si la máquina anfitriona se apaga o pierde conectividad.

En conjunto, estas restricciones explican por qué el diseño prioriza una aplicación servidor sencilla, una persistencia relacional accesible, un mecanismo de trabajos basado en MySQL y una separación clara entre la petición web y la computación científica. No se introducen colas, bases de datos o servicios de despliegue adicionales que no sean necesarios para la implementación actual. La arquitectura conserva así una relación directa entre los requisitos del análisis, las condiciones de calidad, el entorno disponible y los componentes que realmente ejecuta vitalXAI.

17.3. Subsistemas de Diseño

Los subsistemas de diseño son la traducción técnica de los seis subsistemas de análisis definidos en el capítulo 13. La correspondencia se mantiene de forma directa: cada subsistema SS conserva sus responsabilidades funcionales y recibe un identificador SD que agrupa los componentes de implementación que las realizan. Esta decisión permite recorrer la trazabilidad desde los requisitos y casos de uso hasta los routers, servicios, mecanismos de persistencia y procesos que intervienen en la ejecución.

La correspondencia no implica que cada subsistema de diseño sea una aplicación independiente. Todos forman parte de la misma aplicación FastAPI y comparten la base de datos, la autenticación, la capa de presentación y el worker de trabajos. La separación se realiza por responsabilidades y límites funcionales, no mediante despliegues separados. De esta manera, el código mantiene una organización modular sin introducir una complejidad distribuida que no exige el entorno actual.

Subsistema de diseño	Subsistema de análisis	Responsabilidad principal	Componentes de implementación
SD-001	SS-001	Gestionar la identidad, las sesiones y el acceso a la plataforma.	routers/auth.py, services/auth_service.py, users, refresh_tokens y middleware de seguridad.
SD-002	SS-002	Recibir una radiografía, solicitar el diagnóstico y producir sus artefactos.	routers/inference.py, services/ml_engine.py, services/xai_generator.py, services/pdf_generator.py y job_queue.
SD-003	SS-003	Consultar y administrar el historial de diagnósticos del usuario.	routers/history.py, tabla consultations, ficheros de resultados y dashboard.js.
SD-004	SS-004	Configurar, ejecutar y consultar experimentos MLOps.	routers/trainer.py, chatbot_service.py, trainer_engine.py, mlops_engine.py y pdf_generator_mlops.py.
SD-005	SS-005	Supervisar usuarios, consultas y sesiones desde el ámbito administrativo.	routers/admin.py, consultas MySQL y funciones de consulta de mlops_engine.py.
SD-006	SS-006	Coordinar trabajos asíncronos y capacidades transversales de la interfaz.	routers/queue.py, services/queue_worker.py, services/lang.py, middleware global y recursos estáticos.

Tabla x - Correspondencia entre subsistemas de análisis y subsistemas de diseño

17.3.1. SD-001: Acceso, identidad y gestión de sesiones

El subsistema SD-001 materializa el subsistema de análisis SS-001 y cubre el registro, el inicio y el cierre de sesión, la renovación de credenciales y la identificación del usuario en las áreas privadas. Su responsabilidad no se limita a validar el formulario de acceso: establece la identidad que utilizarán los demás subsistemas para aplicar el aislamiento de datos y las comprobaciones de rol.

El punto de entrada del subsistema es `routers/auth.py`. El router sirve las páginas de inicio de sesión y registro y procesa los formularios recibidos desde la interfaz. Antes de crear una cuenta valida el formato del nombre de usuario, la longitud mínima de la contraseña y la presencia de los datos personales básicos. A continuación, consulta MySQL para comprobar que el usuario no existe, genera el hash de la contraseña y almacena la cuenta en la tabla `users`. La contraseña original no se transmite a ningún componente de persistencia después de completar el proceso de hash.

La lógica criptográfica y la gestión del ciclo de vida de los tokens se concentra en `services/auth_service.py`. El token de acceso se firma con JWT y contiene el identificador del usuario y su fecha de expiración. El token de refresco se genera como un valor aleatorio, pero en la base de datos solo se conserva su hash SHA-256 junto con el usuario, la fecha de expiración y el indicador de revocación. Esta diferencia permite renovar la sesión sin guardar una credencial reutilizable en texto plano.

El cierre de sesión revoca el token de refresco y elimina las cookies de la respuesta. La rotación genera un token nuevo y revoca el anterior; si se detecta el uso de un token revocado fuera del periodo de gracia configurado, el servicio invalida todos los tokens del usuario. El mecanismo está diseñado para tolerar peticiones concurrentes de renovación durante un intervalo corto sin interpretar automáticamente cada repetición como un robo de sesión.

El subsistema se apoya también en los mecanismos transversales registrados en `main.py`. `CSRFMiddleware` protege las peticiones que modifican el estado, `SecurityHeadersMiddleware` refuerza las respuestas dirigidas al navegador y `slowapi` limita el endpoint de inicio de sesión. La sesión se almacena en cookies `HttpOnly` y con alcance `SameSite=Lax`, mientras que el resto de routers utiliza `get_user_id_from_token()` para obtener la identidad antes de realizar operaciones privadas. SD-001 es, por tanto, un subsistema transversal: SD-002, SD-003, SD-004, SD-005 y SD-006 dependen de la identidad que establece.

Desde el punto de vista de la trazabilidad, SD-001 realiza principalmente los casos de uso CU-001 a CU-004. CU-001 se materializa en la combinación de validación de los campos, consulta de unicidad, generación del hash e inserción en `users`; CU-002 añade la verificación de la contraseña y la creación de las dos cookies de sesión; CU-003 revoca el token de refresco y elimina las cookies; y CU-004 utiliza el servicio de idioma para adaptar la presentación. Los casos de uso de otros subsistemas no necesitan conocer cómo se firma un token: solo reciben la identidad resuelta o una respuesta de acceso no autorizado.

La configuración sensible permanece fuera del código. `JWT_SECRET_KEY`, `JWT_ACCESS_EXPIRE_MINUTES`, `JWT_REFRESH_EXPIRE_DAYS` y `REFRESH_ROTATION_GRACE_SECONDS` se obtienen del entorno, al igual que los parámetros de MySQL. Esta decisión evita que la clave criptográfica o la duración de las credenciales queden ligadas al repositorio. El servicio emite además una advertencia cuando se utiliza la clave de desarrollo por defecto, lo que convierte una configuración insegura en una condición visible durante el arranque.

El subsistema establece contratos sencillos con sus consumidores. Un componente que necesita identificar al usuario entrega el token de acceso a `get_user_id_from_token()` y recibe un identificador entero o `None`; no interpreta directamente el payload JWT ni duplica la lógica de expiración. Los routers, a su vez, traducen la ausencia de identidad en una respuesta HTTP 401. La comprobación del rol se realiza únicamente en las operaciones administrativas, por lo que la existencia de una sesión válida no equivale a disponer de privilegios elevados.

La principal decisión de SD-001 es conservar el token de acceso en la cookie y el token de refresco en una cookie con ruta restringida, en lugar de enviar las credenciales en el cuerpo de cada petición. Esto simplifica la interacción de las plantillas y los scripts del navegador y reduce la exposición accidental de los tokens en el almacenamiento de la interfaz. La protección efectiva depende además de que el entorno de publicación utilice HTTPS y de que la clave secreta de producción esté configurada correctamente.

17.3.2. SD-002: Diagnóstico asistido y generación de resultados

El subsistema SD-002 corresponde a SS-002 y concentra el flujo clínico de la plataforma. Su función es recibir una imagen válida, asociarla al usuario autenticado, registrar una solicitud de diagnóstico, ejecutar la inferencia con la arquitectura seleccionada y conservar los artefactos que permiten consultar y justificar el resultado.

`routers/inference.py` actúa como fachada HTTP del subsistema. La ruta `/predict` comprueba la existencia de una sesión válida y valida el tipo MIME de la imagen. Se aceptan imágenes con tipo MIME `image/jpeg`, `image/jpg` o `image/png`, y se aplica un límite de 10 MB antes de escribir el fichero en `static/uploads`. Esta validación se realiza antes de encolar el trabajo, de modo que una petición inválida no ocupa una posición en la cola ni genera un registro de procesamiento incompleto.

Una vez almacenada temporalmente la imagen, el router crea un registro en `job_queue` con el tipo `diagnosis`, el identificador del usuario y un payload JSON que incluye la arquitectura, la ruta de la imagen y el idioma. La respuesta HTTP informa de que el trabajo ha quedado encolado, devuelve su identificador y calcula su posición. El router no carga el modelo ni ejecuta la predicción, ya que esas operaciones pertenecen al worker y a la capa de servicios.

El worker dirige el trabajo de diagnóstico hacia `services/ml_engine.py`, que prepara la imagen, carga o reutiliza el modelo y produce la etiqueta y la confianza de la predicción. La implementación combina arquitecturas convolucionales gestionadas mediante TensorFlow/Keras y arquitecturas basadas en Transformers. Después de la inferencia, `services/xai_generator.py` genera el mapa de explicabilidad adecuado para el modelo utilizado. Finalmente, `services/pdf_generator.py` genera el informe descargable con la imagen original, el resultado, la confianza y el artefacto XAI.

El resultado se registra en la tabla `consultations`, donde quedan asociados el usuario, el modelo utilizado, las rutas de los ficheros, la etiqueta de predicción, la confianza y el informe PDF. Esta persistencia permite que el historial y la interfaz administrativa consulten el resultado sin repetir la inferencia. Si cualquier fase falla, el worker marca el trabajo como `failed` y conserva el mensaje de error en `job_queue`, evitando que la petición original quede abierta indefinidamente.

El flujo interno de SD-002 se divide en cinco pasos. Primero, el router autentica al usuario y valida el fichero recibido. Segundo, conserva la imagen original en el área de cargas y crea el payload serializable del trabajo. Tercero, el worker reclama el registro y ejecuta la inferencia. Cuarto, genera los artefactos derivados, que son el mapa XAI y el informe PDF. Quinto, persiste la consulta y actualiza el estado de la cola. La división evita que una ruta HTTP tenga que mantener referencias abiertas a un fichero o a un modelo durante toda la operación.

El payload de un trabajo de diagnóstico contiene únicamente la información necesaria para repetir el procesamiento: `model_name`, `image_path` y `lang`, además del identificador del usuario almacenado en la propia fila de la cola. No se serializa el objeto del modelo ni la imagen completa dentro de MySQL. Los datos voluminosos se mantienen en el sistema de ficheros y la base de datos conserva sus rutas, de manera que la cola sigue siendo ligera y puede consultarse con rapidez.

La separación entre `ml_engine`, `xai_generator` y `pdf_generator` también delimita los fallos. Una arquitectura no reconocida debe fallar antes de producir un resultado ambiguo; una imagen no válida no debe alcanzar el motor; y un error al construir el informe no debe transformarse en una consulta marcada como completada. El worker agrupa el flujo completo dentro del procesamiento del trabajo y utiliza la transición a `failed` como contrato común para que la interfaz pueda mostrar que la operación terminó con error.

La reutilización de modelos cargados responde al RNF-019. El coste de la primera consulta puede incluir la lectura de los pesos y la construcción del modelo, mientras que las consultas posteriores pueden aprovechar el modelo disponible en memoria. Esta optimización pertenece al motor de aprendizaje automático y no al router, por lo que la capa HTTP permanece independiente de si la arquitectura seleccionada es convolucional o Transformer.

SD-002 también actúa como frontera de validación de ficheros. El tipo MIME y el tamaño se comprueban antes de escribir en disco, pero la validez científica de la imagen y la interpretación clínica del resultado siguen siendo responsabilidades distintas. El sistema ofrece una clasificación asistida, una confianza y explicaciones visuales; no sustituye la valoración del profesional sanitario. Esta separación de responsabilidades es importante para no convertir el diseño técnico del subsistema en una afirmación clínica que la implementación no puede garantizar.

17.3.3. SD-003: Historial y gestión de consultas

El subsistema SD-003 materializa SS-003 y proporciona la recuperación y gestión de las consultas ya realizadas. Su entidad persistente principal es `consultations`, que almacena las rutas de los artefactos, la arquitectura utilizada, la predicción, la confianza, el nombre mostrado al usuario y la fecha de la operación.

`routers/history.py` expone la consulta del listado y las operaciones de actualización y eliminación. La consulta del historial filtra siempre por el `user_id` obtenido de la cookie de acceso y ordena los resultados por fecha descendente. Antes de modificar o eliminar una consulta, `_check_consultation_ownership()` comprueba que el registro existe y que pertenece al usuario solicitante. La misma función contempla la excepción controlada del administrador, que puede supervisar consultas desde SD-005.

El subsistema no vuelve a ejecutar el modelo para mostrar una consulta anterior. Recupera desde MySQL los metadatos y devuelve las rutas de la imagen original, el mapa XAI y el informe PDF para que la interfaz pueda presentarlos o descargarlos. Esta decisión reduce el coste de acceso al historial y conserva la separación entre una consulta nueva y la visualización de un resultado ya calculado.

La interfaz del historial se integra en el panel de diagnóstico y utiliza JavaScript para mostrar los resultados, cambiar el nombre visible de una consulta y solicitar su eliminación. El router devuelve códigos diferenciados para la falta de autenticación, la consulta inexistente y la ausencia de permisos. Esta distinción permite que la capa de presentación informe al usuario de la situación sin exponer detalles internos de la base de datos.

La relación entre SD-003 y SD-002 es de productor-consumidor de resultados. SD-002 crea el registro de `consultations` cuando finaliza un diagnóstico; SD-003 no modifica la predicción ni sus artefactos, sino que proporciona operaciones de consulta y organización. El nombre mostrado al usuario se actualiza sobre `patient_name`, que funciona como etiqueta de organización de la consulta y no como identificación clínica del paciente. Esta decisión mantiene separados los metadatos de presentación de los datos técnicos de la inferencia.

El listado devuelve los campos necesarios para construir las tarjetas o filas del historial: identificador, fecha, modelo, rutas de imágenes, etiqueta, confianza, nombre y PDF. Las fechas se convierten a una representación textual antes de formar la respuesta JSON, de modo que el navegador no necesita conocer el tipo específico de fecha de MySQL. Los artefactos se sirven mediante las rutas estáticas configuradas en la aplicación y no mediante una nueva ruta de cálculo.

La eliminación se implementa actualmente como una eliminación física de la fila de consulta. La operación no elimina de forma explícita la imagen, el mapa XAI ni el informe que permanecen en el sistema de ficheros, por lo que puede dejar artefactos huérfanos. Por esa razón, el diseño no debe describir SD-003 como un subsistema de borrado lógico ni afirmar que conserva automáticamente una auditoría de cada eliminación. La integridad de la operación depende de que se compruebe la propiedad antes de ejecutar el DELETE, y de que la transacción se confirme solo después de que la actualización haya sido aceptada por MySQL. La decisión se documenta aquí porque afecta a la interpretación del historial y a futuras ampliaciones del modelo de persistencia.

El administrador constituye una excepción controlada al aislamiento ordinario. `_check_consultation_ownership()` permite continuar cuando el usuario tiene rol admin, pero el acceso administrativo se comprueba en el servidor y no se infiere de un parámetro enviado por el navegador. En todos los demás casos, una consulta de otro usuario produce una respuesta 403. Así, SD-003 mantiene una política única de propiedad y permite que SD-005 la utilice sin duplicar una segunda consulta de autorización.

17.3.4. SD-004: Laboratorio de experimentación MLOps

El subsistema SD-004 corresponde a SS-004 y constituye el bloque de mayor complejidad funcional. Gestiona la configuración conversacional del experimento, la selección del dataset, el entrenamiento de modelos, la ejecución de los análisis de explicabilidad, la comparación estadística, la validación externa, la consulta de resultados y la generación de informes.

`routers/trainer.py` se ha diseñado como una fachada ligera. Sus rutas comprueban la autenticación, validan que la sesión de experimentación pertenece al usuario y delegan las operaciones especializadas en `mlops_engine.py` o en los servicios correspondientes. Entre sus operaciones se encuentran la conversación con el asistente, la exploración de carpetas, el inicio de entrenamientos, la consulta de logs, la recuperación de sesiones, la consulta de resultados, el ranking, el recálculo de la comparativa, la validación externa y la generación de informes.

La configuración conversacional se encapsula en `services/chatbot_service.py`, que se comunica con Groq y devuelve una configuración estructurada cuando se han proporcionado los parámetros necesarios. El router no incorpora la lógica del modelo de lenguaje: únicamente valida la sesión y entrega el mensaje al servicio. Esta separación permite que la comunicación con Groq se trate como una frontera externa y que los errores del proveedor no se confundan con errores de persistencia o de entrenamiento.

`services/mlops_engine.py` organiza las sesiones del laboratorio y resuelve la lectura y escritura de los resultados en `training_results`. `trainer_engine.py` contiene la lógica de preparación del dataset, construcción de modelos, entrenamiento, cálculo del progreso y actualización del estado. Los resultados generados por el pipeline incluyen las métricas de rendimiento, los artefactos de explicabilidad, los datos de comparación y la información necesaria para la validación externa. Los informes finales se generan mediante `pdf_generator_mlops.py`.

El inicio de un entrenamiento crea la sesión, comprueba que la ruta del dataset existe y registra en `job_queue` un trabajo de tipo training con los modelos e hiperparámetros solicitados. La validación externa sigue el mismo mecanismo con un trabajo de tipo `external_validation`. El worker ejecuta ambos tipos fuera del ciclo de petición y actualiza el estado de la cola al finalizar. Esta decisión permite que el laboratorio conserve una interfaz interactiva aunque el entrenamiento tarde mucho más que una petición HTTP normal.

La propiedad de las sesiones es un requisito funcional y se comprueba mediante el `user_id` que la aplicación guarda en `config.json`. Sin embargo, las carpetas y sus resultados no tienen una relación relacional con users, por lo que esta comprobación no cuenta con el mismo respaldo estructural que las consultas y los trabajos almacenados en MySQL. Esta diferencia afecta a RF-005 y OBJ-008 y deberá valorarse en una evolución posterior.

La configuración de una sesión contiene los parámetros que necesita el pipeline: ruta del dataset, modelos seleccionados, número de épocas, tamaño de lote y tasa de aprendizaje. El asistente conversacional permite completar esa información mediante mensajes, pero la ejecución no comienza hasta que el router dispone de una configuración completa y válida. La conversación y el entrenamiento son responsabilidades distintas: Groq ayuda a interpretar la intención del usuario, mientras que mlops_engine y trainer_engine validan y ejecutan la configuración.

El lanzamiento de un entrenamiento sigue una secuencia concreta. La ruta comprueba la autenticación, verifica que existe la ruta del dataset, crea la sesión de entrenamiento y registra en job_queue un payload con la lista de modelos y los hiperparámetros. El identificador de la sesión se conserva tanto en la fila del trabajo como en el directorio de resultados. Esta doble referencia permite consultar el estado desde MySQL y recuperar los artefactos experimentales desde el sistema de ficheros.

La información de una sesión no se reduce a un único valor de rendimiento. El pipeline genera resultados por arquitectura y por pliegue, métricas agregadas, artefactos de explicabilidad, configuraciones y datos de comparación estadística. get_model_results_data() y get_session_ranking_data() transforman esos ficheros en estructuras que el frontend puede mostrar. Esta decisión evita trasladar al navegador la responsabilidad de recorrer todos los artefactos o de repetir los cálculos del ranking.

La comparación estadística tiene un flujo diferenciado del entrenamiento. La ruta de comparación comprueba la propiedad y programa run_statistical_comparison mediante BackgroundTasks, mientras que el entrenamiento y la validación externa se insertan explícitamente en job_queue para ser procesados por queue_worker.py. Esta diferencia es una característica de la implementación actual: ambos mecanismos permiten devolver pronto la respuesta, pero no deben documentarse como si fueran el mismo canal de ejecución. El estado del recálculo se consulta mediante una ruta específica de la sesión.

La validación externa también mantiene la separación entre entrenamiento y evaluación. La ruta verifica que el dataset externo existe y que el usuario tiene acceso a la sesión; después encola un trabajo con el identificador de la sesión y la ruta del dataset. El worker ejecuta la validación sobre los modelos disponibles sin modificar la configuración original del entrenamiento. Los resultados se almacenan como artefactos de la sesión y quedan disponibles para su consulta posterior.

El informe de MLOps se genera en una fase posterior, cuando la sesión dispone de los datos necesarios. pdf_generator_mlops.py recibe la información preparada por el motor y produce un documento descargable, evitando que el router conozca el formato interno del informe. De este modo, el laboratorio puede cambiar la estructura visual del PDF sin cambiar el contrato de las rutas que consultan los resultados.

Desde la perspectiva de los errores, SD-004 debe diferenciar una configuración inválida, un dataset inexistente, una sesión ajena y un fallo del entrenamiento. Las dos primeras situaciones se rechazan antes de crear o encolar el trabajo; la tercera devuelve 403; y la cuarta se registra durante el procesamiento y cambia el estado del trabajo. Esta clasificación permite que la interfaz muestre una corrección de datos, una falta de permisos o un fallo computacional sin confundirlos.

17.3.5. SD-005: Supervisión y administración

El subsistema SD-005 deriva de SS-005 y reúne las operaciones que solo puede realizar un usuario con rol de administrador. Su objetivo es proporcionar una visión supervisada del uso de la plataforma sin mezclar las operaciones administrativas con las rutas que utiliza un usuario ordinario.

`routers/admin.py` centraliza la comprobación de permisos mediante `_require_admin()`. La función obtiene primero la identidad desde el token de acceso y consulta el rol en la tabla `users`. El router diferencia entre una petición no autenticada, una petición autenticada sin permisos y una petición administrativa válida, devolviendo códigos HTTP diferentes en cada caso.

La consulta del listado administrativo combina la información de `users` con el número de consultas de `consultations` y obtiene también el número de sesiones de laboratorio a partir de la información disponible en `training_results`. Las rutas específicas permiten consultar las consultas de un usuario y visualizar el detalle de una consulta concreta. En la segunda operación se incluyen también las sesiones de entrenamiento recuperadas por `mlops_engine`.

SD-005 no duplica la lógica de propiedad del historial ni la del laboratorio. Su responsabilidad es establecer la autorización administrativa y coordinar las consultas globales; la lectura de los datos sigue utilizando las mismas tablas y servicios que el resto de la aplicación. Esta decisión reduce la duplicación y mantiene una frontera clara entre el control de permisos y la representación de la información.

El flujo administrativo comienza siempre por `_require_admin()`. La función identifica al solicitante, consulta su rol y devuelve un resultado ternario en la práctica: no hay identidad, existe identidad sin privilegios o existe identidad con rol de administrador. Cada endpoint interpreta ese resultado antes de abrir la consulta global correspondiente. Esta secuencia evita que una ruta administrativa ejecute una consulta de usuarios o de consultas antes de haber comprobado el permiso.

La vista global combina dos fuentes de información. Los usuarios y el número de diagnósticos se obtienen mediante una consulta agregada sobre `users` y `consultations`; el número de sesiones del laboratorio se calcula inspeccionando las configuraciones disponibles en `training_results`. Esta decisión refleja la persistencia híbrida actual de la plataforma: la información relacional se consulta en MySQL, mientras que parte de los artefactos de las sesiones se conserva en el sistema de ficheros.

Las rutas de supervisión mantienen la granularidad necesaria para la interfaz administrativa. Una ruta devuelve el listado de usuarios, otra recupera las consultas y sesiones asociadas a un usuario concreto y una tercera muestra el detalle de una consulta. El administrador puede acceder a la información de otros usuarios por su rol, pero el sistema no convierte esas rutas en operaciones de modificación general: su responsabilidad actual es de consulta y supervisión.

La protección del subsistema se completa con las medidas de SD-001 y SD-006. El router administrativo no implementa una segunda autenticación ni una segunda gestión de tokens; reutiliza la identidad de la sesión y consulta el rol en la base de datos. Tampoco escribe una traza de auditoría independiente en cada operación, por lo que el requisito de auditoría administrativa debe considerarse una condición pendiente del modelo operativo y no una tabla o servicio que ya exista.

17.3.6. SD-006: Cola de trabajos y capacidades transversales

El subsistema SD-006 materializa SS-006 y reúne las capacidades que sirven de apoyo a varios flujos funcionales. Su núcleo es la cola persistente de trabajos, utilizada por los diagnósticos, los entrenamientos y las validaciones externas. También incluye la consulta de idioma, la personalización de la interfaz y los mecanismos transversales que se aplican desde la configuración principal de FastAPI.

`routers/queue.py` expone la consulta del estado de los trabajos y la cancelación de un trabajo pendiente. La ruta de estado filtra por el usuario autenticado, devuelve los trabajos recientes y calcula la posición de los que permanecen en `queued`. También interpreta el payload según el tipo de trabajo para mostrar el nombre del modelo o el identificador de la sesión sin enviar al navegador el contenido interno completo del payload.

La cancelación se realiza mediante una actualización condicional: solo puede cambiar a `cancelled` un trabajo que pertenezca al usuario y que continúe en estado `queued`. Si el worker ya lo ha reclamado, la operación no lo interrumpe de forma abrupta. Esta restricción evita inconsistencias entre el estado persistido y el cálculo que ya se está ejecutando.

`services/queue_worker.py` es el consumidor de la cola. Al iniciar la aplicación restablece los trabajos que quedaron en estado `running`, selecciona el siguiente trabajo pendiente y lo reclama mediante una actualización que exige que siga en estado `queued`. Después ejecuta el flujo correspondiente en el executor y marca el resultado como completado o fallido. El worker comparte la persistencia con los routers, pero no comparte con ellos el ciclo de petición: esta es la frontera que permite mantener disponible la interfaz durante las tareas de larga duración.

La parte de internacionalización se concentra en `services/lang.py`, que obtiene el idioma seleccionado y devuelve los textos de la interfaz y de los mensajes de respuesta. Los recursos JavaScript aplican el idioma y el tema visual en el navegador. Estas capacidades no necesitan un servicio externo ni una nueva base de datos, por lo que permanecen como mecanismos transversales de presentación y aplicación.

La tabla de responsabilidades concluye la descomposición del diseño. SD-001 establece la identidad; SD-002 produce el diagnóstico; SD-003 conserva y recupera sus consultas; SD-004 gestiona la experimentación; SD-005 supervisa la plataforma; y SD-006 proporciona la ejecución asíncrona y las capacidades comunes. La dependencia entre ellos es deliberada y coincide con la verificación de consistencia del capítulo 15: los subsistemas funcionales necesitan la identidad de SD-001, los trabajos de SD-002 y SD-004 pasan por SD-006, y SD-003 y SD-005 acceden a la persistencia respetando reglas diferentes de propiedad y administración.

La cola aplica además una política de ordenación que prioriza los trabajos de diagnóstico frente a los de entrenamiento. En `_next_job()`, los trabajos de tipo `training` reciben una prioridad posterior, mientras que los diagnósticos y las validaciones externas pueden ser seleccionados antes cuando comparten el estado `queued`. Esta decisión intenta proteger la respuesta del flujo clínico sin eliminar la posibilidad de que los entrenamientos se ejecuten. La posición que observa el usuario se calcula desde el router de cola y se adapta al tipo de trabajo.

El mecanismo de reclamación evita que dos iteraciones del worker procesen el mismo registro. Primero se selecciona una fila pendiente y después `_claim_job()` ejecuta una actualización condicionada a que el estado siga siendo `queued`. Solo si la actualización afecta a una fila se considera que el worker ha adquirido el trabajo. Aunque la configuración actual inicia un worker interno, esta comprobación introduce una garantía de consistencia útil si el proceso evoluciona hacia una ejecución con más de un consumidor.

La máquina de estados de SD-006 es compartida por los subsistemas que generan trabajos. `queued` indica que la petición ha sido aceptada y espera procesamiento; `running` indica que el worker la ha reclamado; `completed` conserva un resultado; `failed` conserva el error; y `cancelled` identifica una cancelación solicitada por el usuario antes del inicio. No todos los estados se pueden alcanzar desde todas las rutas: la cancelación solo opera sobre `queued`, y un trabajo `running` no se interrumpe mediante una actualización administrativa.

El reinicio de la aplicación se trata como un evento de recuperación. Durante el lifespan de FastAPI se inicializa la base de datos y se llama a `start_worker()`. Este método restablece los trabajos `running` y crea la tarea de `worker_loop()`. La recuperación no pretende reconstruir el estado intermedio de un entrenamiento parcialmente ejecutado; vuelve a poner el trabajo en condición de ser procesado, por lo que los motores deben tolerar que una tarea se reinicie desde la información disponible en su payload y en sus artefactos.

El subsistema también resuelve aspectos que no deben mezclarse con la cola. `lang.py` proporciona textos traducidos para los routers y la interfaz, mientras que los scripts JavaScript gestionan el tema visual y la presentación de los estados. El middleware CSRF y el middleware de cabeceras se registran globalmente en `main.py`, aunque funcionalmente apoyan a todos los subsistemas. Se incluyen en la visión transversal de SD-006 por su alcance común, pero sus responsabilidades de seguridad no sustituyen las comprobaciones de propiedad específicas de SD-003 y SD-004.

El límite principal de SD-006 es que la cola está implementada sobre MySQL y el worker se ejecuta dentro del mismo proceso de la aplicación. Esta solución es suficiente para el entorno actual y evita introducir Redis, Celery o un sistema distribuido que no forma parte del código. A cambio, el diseño depende de la disponibilidad del proceso y de la base de datos local, y no proporciona por sí mismo escalado horizontal de workers ni una garantía independiente de disponibilidad. Esta limitación queda recogida para que las futuras decisiones de soporte no confundan la ejecución asíncrona actual con una arquitectura distribuida.

La descomposición de SD-006 permite finalmente que los demás subsistemas compartan un contrato común para las operaciones largas. SD-002 entrega un trabajo de diagnóstico; SD-004 entrega trabajos de entrenamiento y validación; SD-006 los procesa, actualiza sus estados y devuelve la información a los routers de consulta. El usuario observa siempre una misma idea de funcionamiento (petición aceptada, trabajo pendiente, procesamiento y resultado) aunque los motores internos sean diferentes. Esta uniformidad es la principal aportación arquitectónica del subsistema transversal.

17.3.7. Relaciones de integración entre los subsistemas de diseño

La separación anterior permite identificar qué subsistemas inician una operación, cuáles la ejecutan y cuáles presentan sus resultados. SD-001 constituye el punto de entrada común porque proporciona la identidad y el rol. SD-002 y SD-004 son los productores de los trabajos de mayor coste; SD-006 se encarga de procesarlos; SD-003 y SD-005 consumen la información persistida desde perspectivas diferentes. Esta organización evita que el flujo observable de un caso de uso dependa de una llamada directa desde la interfaz a cada motor interno.

En el flujo de diagnóstico, el navegador envía la imagen y la arquitectura a SD-002. El router obtiene el identificador del usuario desde SD-001, valida la carga y registra un trabajo en SD-006. El worker ejecuta el motor de SD-002, produce sus ficheros y escribe la consulta final en MySQL. Cuando el usuario abre el historial, SD-003 recupera la fila de consultations y entrega las rutas de los artefactos ya generados. Si el usuario dispone del rol adecuado, SD-005 puede recuperar una consulta desde la perspectiva administrativa, pero no altera el modo en que SD-002 calculó el resultado.

El flujo del laboratorio es más amplio. El usuario comienza una conversación que SD-004 dirige hacia Groq; después, la configuración devuelta se valida antes de crear una sesión. El lanzamiento registra los parámetros en los artefactos del laboratorio y en el payload de SD-006. El worker invoca a los motores de entrenamiento y evaluación, y los resultados se almacenan en el directorio de la sesión. Las rutas posteriores de SD-004 consultan esos artefactos, calculan o recuperan el ranking, solicitan comparativas y generan el informe. SD-005 puede contabilizar y consultar las sesiones asociadas a un usuario, pero la gestión detallada de los resultados sigue perteneciendo a SD-004.

Las dependencias de persistencia son deliberadamente asimétricas. SD-001 escribe usuarios y tokens; SD-002 escribe trabajos y consultas; SD-003 modifica únicamente los datos organizativos de las consultas; SD-004 combina MySQL para la cola y el sistema de ficheros para los resultados de entrenamiento; SD-005 realiza consultas globales; y SD-006 actualiza los estados operativos de los trabajos. Ningún subsistema debe acceder a los datos de otro sin aplicar la regla de propiedad o la autorización administrativa que corresponda.

La aplicación mantiene también una separación entre datos estructurados y artefactos. Las cuentas, las consultas, los tokens y los estados de la cola se guardan en MySQL porque necesitan filtros, relaciones y actualizaciones transaccionales. Las imágenes, los mapas XAI, los informes PDF, los logs y las configuraciones de entrenamiento se guardan en el sistema de ficheros porque son archivos o estructuras generadas por los motores. Las tablas conservan referencias a los artefactos, pero no incorporan el contenido binario completo en cada registro.

La gestión de errores sigue el límite del subsistema que los origina. Un error de autenticación se resuelve en SD-001 con una respuesta de acceso no autorizado; un fichero que no cumple las restricciones se rechaza en SD-002 antes de generar un trabajo; una consulta ajena se bloquea en SD-003; una configuración de entrenamiento inválida se rechaza en SD-004; un usuario sin rol recibe 403 en SD-005; y un fallo durante una tarea se registra mediante SD-006 como trabajo fallido. Esta distribución evita que los routers tengan que conocer detalles internos de todos los componentes para construir mensajes de error.

La extensibilidad del diseño se apoya en los puntos donde la variación es real. Añadir una nueva arquitectura de inferencia afecta principalmente a `ml_engine` y a las funciones XAI compatibles; añadir un nuevo tipo de tarea requiere ampliar la selección de `_execute_job()` y los datos que presenta `queue.py`; incorporar una nueva operación administrativa se restringe a SD-005; y ampliar los idiomas se concentra en los recursos gestionados por `lang.py` y los scripts de interfaz. Estas extensiones no exigen modificar el mecanismo general de autenticación ni reconstruir la persistencia completa.

El criterio de diseño adoptado es, por tanto, la cohesión funcional con dependencias explícitas. Los componentes de un subsistema colaboran porque comparten una responsabilidad y no únicamente porque estén escritos en Python. Los routers conocen los contratos HTTP; los servicios conocen la lógica de aplicación; el worker conoce la ejecución prolongada; MySQL conoce el estado estructurado; y el navegador conoce la representación y la interacción. Esta distribución hace posible comparar cada subsistema con los casos de uso del capítulo 12 y comprobar, durante las pruebas, si el fallo pertenece a la entrada, a la coordinación, al cálculo o a la persistencia.

Flujo de integración	Subsistema iniciador	Subsistema coordinador	Resultado persistido o presentado
Registro e inicio de sesión	SD-001	SD-001 y middleware global	Cookies de sesión y registro en users y refresh_tokens.
Diagnóstico asistido	SD-002	SD-006 mediante job_queue	Predicción, confianza, mapas XAI, PDF y fila en consultations.
Consulta del historial	SD-003	SD-001 para identidad	Listado de consultas y acceso a sus artefactos.
Entrenamiento MLOps	SD-004	SD-006 mediante job_queue	Sesión, modelos, métricas, artefactos y logs.
Validación externa	SD-004	SD-006 mediante job_queue	Resultados de validación asociados a la sesión.
Supervisión administrativa	SD-005	SD-001 para autenticación y rol	Listado de usuarios, consultas y sesiones supervisadas.
Consulta y cancelación de trabajos	SD-006	SD-001 para identidad	Estado, posición, error o cancelación del trabajo.

Tabla x - Relaciones de integración entre subsistemas de diseño

17.4. Operación segura y controles de protección

La operación de vitalXAI debe conservar las garantías de confidencialidad, integridad y disponibilidad definidas durante el análisis. Estas garantías no dependen de una única función, sino de la combinación de controles distribuidos entre la sesión, los routers, los servicios, la base de datos, el sistema de ficheros y el entorno de publicación. El propósito de este apartado es describir cómo debe funcionar la plataforma de forma segura cuando se inicia, recibe peticiones, procesa trabajos y se detiene, manteniendo la correspondencia con los requisitos RNF-001 a RNF-034 y con los casos de uso que implican autenticación, datos sanitarios o ejecución asíncrona.

La operación segura se organiza en seis ámbitos: control de identidad y permisos, protección de datos y ficheros, comunicaciones y secretos, seguridad de las tareas asíncronas, persistencia y recuperación y condiciones del despliegue. La siguiente tabla resume los controles principales y su punto de aplicación.

Ámbito operativo	Controles principales	Componentes implicados
Identidad y sesión	Hash de contraseñas, JWT con expiración, refresh tokens, rotación, revocación y cookies protegidas.	auth.py, auth_service.py y tabla refresh_tokens en MySQL.
Autorización	Comprobación de autenticación, propiedad de los recursos y rol de administrador.	Routers de history, trainer, queue y administración.
Protección web	CSRF, cabeceras de seguridad y limitación de peticiones.	CSRFMiddleware, SecurityHeadersMiddleware y slowapi.
Datos y ficheros	Datasets anonimizados, validación de imágenes, asociación por usuario y control de rutas.	MySQL, inference.py, consultations y directorios de resultados.
Trabajos asíncronos	Estados persistentes, reclamación condicional, aislamiento de errores y recuperación al reiniciar.	Tabla job_queue y ejecución en queue_worker.py.
Operación y recuperación	Variables de entorno, MySQL/XAMPP, HTTPS mediante el túnel y copias operativas de la base de datos.	main.py, .env, Uvicorn, XAMPP y Cloudflare Tunnel.

Tabla x - Controles de operación segura de vitalXAI

17.4.1. Control de identidad, sesión y autorización

El acceso a las funciones privadas comienza con la identificación del usuario a partir de la cookie `access_token`. El token contiene el identificador de la cuenta y una fecha de expiración, y se firma mediante HS256 con la clave configurada en `JWT_SECRET_KEY`. El servidor no confía en un identificador recibido como un campo ordinario de la petición: cada router obtiene el usuario desde la credencial de sesión y utiliza ese valor para realizar sus consultas.

Durante el registro, `auth.py` valida los campos recibidos y comprueba la unicidad del nombre de usuario antes de insertar la cuenta. La contraseña se transforma mediante `bcrypt` y solo el resultado se almacena en `password_hash`. Durante el inicio de sesión se recupera el hash de la cuenta y se compara con la contraseña recibida; si la comparación falla, no se crean las cookies de sesión. Este comportamiento evita que una petición con credenciales inválidas genere una identidad parcial o un token utilizable.

La sesión utiliza dos credenciales con funciones distintas. El token de acceso se emplea para identificar las peticiones ordinarias y tiene una duración configurable. El token de refresco se utiliza exclusivamente en `/api/token/refresh`, se registra mediante un hash en `refresh_tokens` y permite crear un nuevo acceso sin repetir el formulario de inicio de sesión. La rotación revoca el token anterior y genera otro valor aleatorio. Si se detecta el uso de una credencial revocada fuera del intervalo de tolerancia, el servicio revoca el conjunto de tokens asociado al usuario.

El cierre de sesión debe revocar el refresh token y eliminar las cookies desde la respuesta. La revocación se realiza en la base de datos, por lo que no depende de que el navegador elimine correctamente sus datos locales. El token de acceso puede seguir presente hasta su expiración natural, pero las operaciones de renovación quedan bloqueadas por el estado persistido del token de refresco. Esta diferencia debe considerarse al operar el sistema: cerrar la sesión evita la renovación, pero la protección del canal y la duración razonable del token de acceso siguen siendo necesarias.

La autorización se aplica después de la autenticación. En el historial, las consultas deben pertenecer al usuario salvo cuando la operación se realiza desde el ámbito administrativo. En la cola, tanto la consulta del estado como la cancelación filtran por `user_id`. En administración, `_require_admin()` consulta el rol en `users` y bloquea al usuario autenticado que no tenga el valor `admin`. En el laboratorio, la aplicación comprueba el `user_id` guardado en la configuración de cada sesión; sin embargo, el directorio `training_results` no tiene una relación relacional con `users`, por lo que esta comprobación no ofrece el mismo respaldo estructural que la de las tablas.

Este diseño evita dos errores habituales. El primero es confundir que un usuario esté autenticado con que pueda acceder a cualquier registro; la sesión solo proporciona identidad, mientras que la propiedad decide el alcance. El segundo es dejar la autorización en el cliente; aunque la interfaz oculte el panel administrativo a un usuario ordinario, el servidor vuelve a comprobar el rol cuando recibe la petición.

La protección CSRF se aplica a las operaciones que cambian el estado. El middleware genera una cookie de token y exige que el navegador copie el mismo valor en la cabecera `X-CSRF-Token`. Las peticiones de lectura no necesitan esta comprobación y el inicio de sesión se excluye para permitir que un visitante obtenga su primera sesión. Esta excepción no convierte el resto de las rutas públicas en operaciones sin protección: cada petición que modifica datos debe incluir el token correspondiente.

17.4.2. Protección de los datos y de los artefactos

La plataforma debe operar exclusivamente con imágenes y datasets anonimizados. La arquitectura no incorpora un componente que identifique o elimine automáticamente los datos personales que puedan aparecer dentro de una radiografía; por tanto, la anonimización previa de los ficheros es una condición del procedimiento de uso. La interfaz no debe utilizarse para introducir nombres, identificadores sanitarios u otros metadatos que no sean necesarios para el diagnóstico asistido.

La ruta de diagnóstico aplica controles básicos antes de escribir una imagen: restringe el tipo MIME a JPEG y PNG y limita el tamaño a 10 MB. Después genera un nombre basado en la fecha y conserva el fichero en static/uploads. El trabajo encolado contiene la ruta de la imagen y la arquitectura solicitada, mientras que la fila final de consultations conserva las rutas de los artefactos asociados. Las rutas no deben construirse a partir de concatenaciones posteriores de datos no validados, y el entorno debe restringir los permisos de escritura de los directorios de cargas y resultados al proceso que ejecuta la aplicación.

La asociación por usuario es el control principal de confidencialidad lógica para los datos relacionales. consultations, training_jobs, job_queue y refresh_tokens contienen un user_id que permite aplicar filtros explícitos, aunque training_jobs es una tabla prevista y no participa en el flujo operativo actual. Las consultas de historial y de cola usan ese filtro directamente, y las operaciones administrativas se separan mediante el rol. Las sesiones y los resultados del laboratorio se guardan en carpetas; su configuración incluye user_id, pero no existe una relación relacional equivalente con users. Esta diferencia debe considerarse una limitación de la implementación y no una garantía plenamente respaldada de RF-005.

Los ficheros generados por el diagnóstico incluyen la imagen original, el mapa de explicabilidad y el informe PDF. El informe puede contener el resultado, la confianza, el modelo utilizado y la imagen, por lo que debe considerarse parte de la información protegida aunque se sirva como un documento descargable. El sistema conserva las rutas y la interfaz muestra los artefactos dentro del flujo de la consulta. Como training_results se publica actualmente mediante un montaje estático, las URL de esos artefactos no tienen una comprobación de propiedad equivalente a la del router; esta exposición debe corregirse antes de utilizar datos sensibles fuera del entorno de demostración.

El acceso a MySQL se realiza mediante consultas parametrizadas y conexiones procedentes del pool. Las operaciones de lectura y escritura cierran la conexión al terminar, y las actualizaciones importantes ejecutan commit() de forma explícita. Las claves ajenas vinculan los registros dependientes con users, lo que evita mantener consultas huérfanas cuando se elimina una cuenta con las reglas configuradas en el esquema.

La protección lógica de los datos debe complementarse con una política de retención. La implementación actual permite eliminar una consulta de forma física desde el historial y no incorpora un subsistema de archivado o borrado lógico para las consultas. Por tanto, el operador debe conocer que la eliminación no equivale a una copia de seguridad ni a una auditoría histórica. Las copias de la base de datos y de los directorios de resultados deben gestionarse por separado si se necesita recuperar una operación eliminada.

17.4.3. Comunicaciones, configuración y secretos

En el entorno local, Uvicorn atiende la aplicación en el puerto 8000 y MySQL se ejecuta normalmente mediante XAMPP en el puerto 3306. Para permitir el acceso desde un navegador externo se utiliza el túnel configurado para la demostración. El túnel actúa como punto de publicación y debe proporcionar HTTPS hacia el exterior; el backend no debe exponerse directamente abriendo puertos de la base de datos a Internet.

La comunicación entre el navegador y las rutas de FastAPI transporta credenciales, imágenes y resultados. Por ello, el acceso remoto debe realizarse siempre mediante HTTPS basado en TLS (IETF, 2018). Las cabeceras añadidas por SecurityHeadersMiddleware complementan el cifrado del canal y aplican controles de seguridad web documentados por OWASP (OWASP, 2024): Content-Security-Policy restringe los orígenes de recursos, X-Frame-Options evita la inclusión en marcos, X-Content-Type-Options bloquea la interpretación MIME indebida, Strict-Transport-Security comunica la preferencia por HTTPS y Referrer-Policy limita la información que se comparte entre páginas.

La aplicación utiliza variables de entorno para los valores que no deben formar parte del código fuente. Entre ellas se encuentran la clave JWT, las credenciales de MySQL, el tamaño del pool, las duraciones de los tokens y GROQ_API_KEY. El fichero .env, cuando se utiliza en desarrollo, debe quedar fuera del control de versiones y sus permisos deben limitarse al usuario que ejecuta la aplicación. La clave por defecto de desarrollo que auth_service.py detecta mediante una advertencia no es una configuración válida para producción.

La integración con Groq exige controlar tanto la disponibilidad como el secreto de acceso. chatbot_service.py es el único componente que necesita la clave de la API y traduce la respuesta del servicio externo al flujo conversacional del laboratorio. Los routers no deben incluir la clave en las respuestas JSON ni en los logs. Si Groq no está disponible, la conversación debe informar del error sin revelar la configuración del entorno ni interrumpir la disponibilidad del resto de la plataforma.

La integración con Hugging Face presenta una restricción diferente: puede requerir conectividad durante la descarga inicial de los modelos Transformer y memoria suficiente para mantenerlos cargados. Una vez disponibles, el motor puede reutilizar los modelos según su configuración. La operación debe distinguir entre un fallo de red durante la carga y una predicción inválida, registrando el trabajo como fallido cuando no pueda completarse y evitando devolver al usuario una salida parcial.

La configuración de desarrollo se debe validar antes de publicar el servicio. El operador debe comprobar que Python y las dependencias coinciden con requirements.txt, que MySQL está iniciado, que las variables de entorno están definidas, que existe la estructura de directorios necesaria y que el acceso al túnel funciona mediante HTTPS. Esta verificación se complementa con la ejecución de las pruebas automatizadas y evita que un fallo de configuración se interprete como un error del modelo o de la lógica de negocio.

17.4.4. Seguridad de la ejecución asíncrona

Los trabajos largos se consideran recursos operativos que requieren control de ciclo de vida. La petición que los crea no ejecuta el cálculo directamente: primero valida la identidad, registra el tipo de trabajo, asocia el payload al usuario y devuelve su identificador. El worker recupera la información desde MySQL y no debe aceptar órdenes procedentes directamente del navegador para ejecutar una función arbitraria. El tipo `job_type` determina si se llama al flujo de diagnóstico, entrenamiento o validación externa.

El payload contiene rutas y parámetros que deben haber sido validados por el router que creó el trabajo. El worker vuelve a interpretar el JSON antes de utilizarlo y falla de forma controlada si faltan claves obligatorias. Esta doble frontera es necesaria porque la fila podría conservar datos antiguos o haber sido alterada por un fallo de operación. El worker nunca debe tratar el contenido de un payload como código ejecutable ni utilizarlo para seleccionar un módulo fuera de los tipos explícitamente soportados.

La reclamación condicional del trabajo protege frente a duplicados. `_claim_job()` solo cambia el estado si sigue siendo `queued`; por tanto, un trabajo cancelado o reclamado por otro consumidor no se ejecuta de nuevo desde esa iteración. Después, `_finish_job()` guarda el resultado serializado y `_fail_job()` limita el mensaje de error a 500 caracteres. Este límite reduce el riesgo de llenar la base de datos con trazas descontroladas, aunque los diagnósticos detallados deben consultarse en los logs del entorno cuando sean necesarios.

El aislamiento de errores se basa en el tratamiento individual de cada trabajo. Una excepción durante la inferencia, la generación XAI o el entrenamiento se captura en el bloque de procesamiento y cambia la fila a `failed`. El bucle principal continúa con el siguiente trabajo y, si se produce un error externo al procesamiento, espera antes de volver a consultar la cola. Así, un dataset defectuoso o una arquitectura incompatible no detiene por completo el servicio.

La recuperación tras un reinicio es deliberadamente conservadora. `start_worker()` restablece los trabajos que quedaron en `running` a `queued`, porque no existe un registro externo que garantice en qué punto exacto del cálculo se encontraba el proceso. Esta estrategia puede provocar que una tarea vuelva a comenzar, pero evita dejarla invisible para el usuario. El diseño debe asumir que las operaciones de entrenamiento y validación pueden repetirse después de una interrupción y que los artefactos parciales no deben considerarse resultados válidos hasta que el trabajo se marque como completado.

La cancelación está restringida al estado `queued`. No se intenta finalizar abruptamente un hilo o un cálculo que ya está en `run_in_executor`, porque esa interrupción podría dejar ficheros incompletos, sesiones en un estado ambiguo o conexiones abiertas. El usuario recibe una respuesta de recurso no encontrado cuando el trabajo no existe, no le pertenece o ya ha comenzado. Este comportamiento prioriza la integridad del procesamiento frente a una cancelación inmediata sin garantías.

17.4.5. Persistencia, arranque y recuperación operativa

El arranque de la aplicación se concentra en la función `lifespan` de `main.py`. Primero se carga la configuración del entorno, después se ejecuta `init_db()` para comprobar o crear las tablas principales y, finalmente, se inicia el worker. Si MySQL no está disponible, la aplicación informa de la incidencia con un mensaje destinado al operador. Esta información no debe considerarse una sustitución de la recuperación: la plataforma necesita que el servicio de base de datos esté operativo para autenticar usuarios, consultar trabajos y persistir resultados.

El pool de conexiones se crea de forma diferida cuando se solicita la primera conexión. Esta decisión evita abrir conexiones innecesarias al importar el módulo y permite que los parámetros se obtengan del entorno ya cargado. El tamaño por defecto es cinco, pero puede modificarse con `DB_POOL_SIZE` según la capacidad real del servidor. El operador debe evitar configurar un pool superior al número de conexiones que MySQL puede atender con estabilidad.

Las tablas principales cumplen funciones diferentes. `users` y `refresh_tokens` sostienen la identidad; `consultations` conserva el historial clínico; `training_jobs` está prevista para conservar información de entrenamientos, aunque no se utiliza en el flujo actual; y `job_queue` coordina los trabajos de diagnóstico, entrenamiento y validación externa. La persistencia de los artefactos se completa con `static/uploads`, `static/results` y `training_results`. Un procedimiento de copia completo debe incluir tanto el contenido relacional como los directorios de ficheros, ya que restaurar solo MySQL dejaría rutas sin sus documentos asociados.

La recuperación operativa debe contemplar tres fallos distintos. Ante un fallo de MySQL, no se pueden resolver las operaciones que dependen de la cuenta o de la cola; ante un fallo del sistema de ficheros, los registros pueden conservar rutas cuyos documentos no existen; y ante un fallo del worker, los trabajos running pueden volver a quedar encolados durante el siguiente arranque. La guía de despliegue debe permitir identificar cuál de estos componentes ha fallado antes de reiniciar indiscriminadamente toda la aplicación.

La integridad de los resultados requiere una comprobación adicional durante la operación. Una consulta marcada como `completed` debe disponer de sus artefactos esperados y de la fila de consulta correspondiente. Un trabajo `failed` debe conservar un mensaje suficiente para distinguir un error del modelo, del dataset, de la API externa o de la persistencia. El sistema actual registra estos estados, pero no incorpora un servicio separado de observabilidad; por ello, el operador debe revisar la salida del proceso y los logs disponibles cuando se investigue una incidencia.

17.4.6. Límites actuales y responsabilidades del operador

La solución actual está diseñada para un entorno local o de demostración controlada. La aplicación, MySQL, los ficheros y el worker pueden residir en la misma máquina, y el túnel proporciona el acceso remoto sin convertir el sistema en una plataforma distribuida. Esta decisión reduce la complejidad, pero implica que la disponibilidad depende del equipo anfitrión, de XAMPP, de la conectividad y de la cuota de Groq cuando se utiliza el asistente.

El operador debe iniciar MySQL antes de ejecutar la aplicación, verificar las variables de entorno, evitar el uso de la clave JWT de desarrollo, conservar las dependencias del entorno virtual y comprobar que el directorio de resultados tiene espacio suficiente. También debe controlar los datasets que se cargan, confirmar que están anonimizados y no exponer directamente los puertos 3306 ni otros servicios internos a Internet. La URL pública del túnel debe compartirse solo con los usuarios autorizados durante la demostración.

La implementación actual no proporciona alta disponibilidad, balanceo de workers, almacenamiento de secretos dedicado, cifrado de ficheros en reposo ni un sistema formal de auditoría administrativa. Estas ausencias son límites operativos que deben tenerse en cuenta al valorar la plataforma. Los controles implementados, hashes, tokens, CSRF, cabeceras, rate limiting, filtros de propiedad, estados de cola y recuperación de trabajos, aportan una base de seguridad, pero no equivalen por sí solos a una certificación clínica ni a un entorno de producción de alta disponibilidad.

En consecuencia, el uso de vitalXAI debe mantenerse dentro del alcance declarado en el proyecto. La plataforma ofrece apoyo al diagnóstico y experimentación de modelos, pero la decisión clínica corresponde al profesional. Los datos de pacientes deben anonimizarse antes de la carga, los resultados deben interpretarse junto con los mapas de explicabilidad y los informes no deben considerarse una autorización para automatizar decisiones médicas sin supervisión. La operación segura incluye tanto las medidas técnicas como estas reglas de uso.

La combinación de controles de software y responsabilidades operativas completa el diseño de seguridad. SD-001 protege la identidad; SD-002 limita y procesa las entradas de diagnóstico; SD-003 y SD-004 restringen el acceso a los datos propios; SD-005 separa la supervisión administrativa; SD-006 controla los trabajos y sus estados; y el entorno mantiene MySQL, los ficheros, las claves y el canal HTTPS bajo condiciones conocidas. Esta distribución permite verificar cada requisito de operación en el punto donde realmente se aplica y evita atribuir a una única capa garantías que dependen del sistema completo.

Capítulo 18. Arquitectura de soporte y mecanismos transversales

La arquitectura de soporte reúne los elementos que permiten que los subsistemas funcionales operen de forma coordinada, segura y mantenible. A diferencia de los subsistemas de diseño del capítulo 17, estos componentes no representan una capacidad clínica o investigadora concreta para el usuario. Su responsabilidad es proporcionar persistencia, almacenamiento, configuración, protección, validación y ejecución común a las funcionalidades de autenticación, diagnóstico, historial, laboratorio y administración (Larman, 2004; Jacobson, Booch & Rumbaugh, 1999).

La separación entre funcionalidad y soporte no significa que ambos ámbitos estén desconectados. SD-002 depende del almacenamiento y de la cola; SD-004 necesita el sistema de ficheros, MySQL y la configuración de los servicios externos; SD-001 utiliza el middleware de seguridad; y todos los routers dependen del ciclo de vida de FastAPI. La arquitectura de soporte permite que estas dependencias se apliquen de forma uniforme, evitando que cada módulo implemente su propia conexión, su propio control de errores o su propia interpretación del entorno.

En la implementación actual se distinguen dos dimensiones. La primera está formada por los subsistemas de soporte, que agrupan servicios o recursos con una identidad técnica propia: persistencia relacional, almacenamiento de ficheros, entorno de ejecución e integración externa y calidad operativa. La segunda está formada por los mecanismos de diseño transversales, que son reglas aplicadas a varios componentes: gestión de configuración, validación de entradas, autenticación y autorización, manejo de errores, ejecución asíncrona, modularidad y pruebas automatizadas.

18.1. Subsistemas de Soporte

Los subsistemas de soporte descritos en esta sección no sustituyen a SD-006 ni duplican sus responsabilidades. SD-006 pertenece a la lógica de capacidades transversales que el usuario puede utilizar, como consultar o cancelar un trabajo. En cambio, los subsistemas de soporte proporcionan la infraestructura que hace posible esa capacidad: la conexión a MySQL, el sistema de ficheros, el entorno de ejecución y los servicios que se integran con la plataforma.

Subsistema de soporte	Responsabilidad	Componentes actuales
SSOP-001: Persistencia relacional	Crear conexiones, conservar entidades y asegurar la integridad básica de las relaciones.	database.py, MySQLConnectionPool, MySQL/XAMPP y tablas del esquema.
SSOP-002 Almacenamiento de ficheros y recursos	Conservar imágenes, mapas XAI, informes, resultados MLOps, plantillas y recursos estáticos.	static/, templates/, training_results/ y montajes de main.py.
SSOP-003 Entorno de ejecución e integración	Proporcionar el proceso web, las dependencias Python y el acceso a APIs y repositorios externos.	static/, templates/, training_results/ y montajes de main.py.
SSOP-004 Calidad y verificación	Comprobar el comportamiento de los componentes y detectar regresiones durante el desarrollo.	pytest, fixtures de tests/conftest.py, mocks, pytest-cov y ruff.

Tabla x - Subsistemas de soporte de vitalXAI

18.1.1. SSOP-001: Persistencia relacional y conexiones de base de datos

El subsistema SSOP-001 encapsula el acceso de la aplicación a MySQL. Su componente principal es `database.py`, que mantiene una referencia al pool de conexiones y expone `get_db_connection()` para que los routers y servicios puedan solicitar una conexión sin conocer cómo se crea. La configuración se obtiene de `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME` y `DB_POOL_SIZE`, con valores de desarrollo definidos cuando el entorno no proporciona una variable concreta.

El pool se crea de forma diferida. La primera operación que necesita una conexión invoca `_get_pool()`, que construye un `MySQLConnectionPool` con el tamaño configurado. Las operaciones posteriores reutilizan el mismo objeto y solicitan conexiones disponibles mediante `get_connection()`. Este comportamiento reduce el coste de crear conexiones repetidas y proporciona un punto único desde el que controlar el acceso concurrente a la base de datos.

La inicialización del esquema se realiza mediante `init_db()`, llamada durante el lifespan de FastAPI. El procedimiento comprueba la existencia de las tablas `users`, `consultations`, `training_jobs`, `job_queue` y `refresh_tokens`, y crea las que no existan. Esta estrategia es adecuada para el entorno local y de demostración actual, donde la base de datos se inicializa desde la aplicación. No equivale a un sistema de migraciones versionadas: una evolución posterior del esquema deberá gestionarse con un procedimiento controlado para no depender únicamente de sentencias `CREATE TABLE IF NOT EXISTS`.

La persistencia relacional cumple dos funciones diferentes. Por un lado, conserva información de negocio, como las cuentas y consultas. Por otro, sirve de soporte operacional para la cola de trabajos y los tokens de refresco. En ambos casos se utilizan consultas parametrizadas, se confirma la escritura con `commit()` y se cierra la conexión tras terminar. Las claves ajenas de las tablas de trabajos, consultas y tokens apuntan a `users`, lo que mantiene la relación de propiedad a nivel de esquema.

Los componentes funcionales no comparten una conexión global abierta. Cada router o servicio solicita una conexión para una operación concreta, obtiene un cursor, ejecuta la consulta y cierra la conexión. Esta decisión limita el alcance de un error de conexión y evita que una petición retenga indefinidamente un recurso del pool. El coste de esta estrategia se controla mediante `DB_POOL_SIZE`, que debe ajustarse a la capacidad real de MySQL y al número de peticiones que se espera atender.

El subsistema también define el comportamiento ante indisponibilidad. Si MySQL no puede conectarse durante el arranque, `main.py` muestra un aviso operativo indicando que debe iniciarse XAMPP o revisarse la configuración. El inicio del worker también necesita una conexión a MySQL, por lo que la aplicación no puede operar con normalidad mientras la base de datos permanezca inaccesible. Esta respuesta informa del problema sin ocultar que MySQL es una dependencia obligatoria.

18.1.2. SSOP-002: Almacenamiento de ficheros y recursos de aplicación

El subsistema SSOP-002 gestiona los artefactos que no son adecuados para almacenarse directamente como columnas relacionales. La aplicación utiliza `static/uploads` para las imágenes originales de diagnóstico, `static/results` para los mapas XAI y `training_results` para las configuraciones, logs y resultados del laboratorio. Los informes PDF se generan en las rutas que devuelven `pdf_generator.py` y `pdf_generator_mlops.py`. MySQL conserva las referencias a los ficheros cuando forman parte de una consulta o de un resultado.

La separación entre base de datos y sistema de ficheros evita insertar imágenes y documentos binarios grandes dentro de las tablas principales. La base de datos mantiene los metadatos que necesita el historial, como el modelo, la etiqueta, la confianza, el usuario y las rutas, y el sistema de ficheros conserva el contenido. Cuando SD-003 recupera una consulta, no recalcula ni copia el documento: utiliza las referencias persistidas para que la interfaz solicite el artefacto correspondiente.

Las plantillas Jinja2 y los recursos JavaScript pertenecen también a este subsistema de soporte, aunque se consumen desde la capa de presentación. `main.py` monta `static` como directorio de recursos y crea el directorio de resultados de entrenamiento si no existe. Las plantillas se resuelven mediante Jinja2Templates desde `templates`. Esta configuración permite que la aplicación sirva la interfaz sin un servidor frontend independiente ni un proceso de compilación adicional.

El almacenamiento se organiza según el origen del artefacto. Las cargas de diagnóstico reciben un nombre generado con una marca temporal y el nombre original; los resultados XAI se generan en un subdirectorio específico; y las sesiones de entrenamiento utilizan su identificador para agrupar configuración, logs y resultados. Esta organización facilita que el worker encuentre los artefactos a partir del payload y que el administrador pueda contabilizar sesiones inspeccionando sus configuraciones.

El control de acceso a los ficheros debe entenderse junto al control de acceso de los registros. La aplicación comprueba la propiedad de las consultas y de las sesiones mediante los datos disponibles, incluido el `user_id` guardado en la configuración de cada sesión. Sin embargo, las carpetas no tienen una relación física con las tablas de usuarios, por lo que la protección de esos artefactos no queda respaldada por una clave ajena. Además, el entorno operativo debe limitar los permisos de escritura de los directorios al proceso de la aplicación y evitar la exposición directa de los directorios de datos cuando no sea necesaria. La implementación actual ofrece validación de tipo MIME y tamaño para las imágenes, pero no incorpora un antivirus ni un servicio independiente de análisis de contenido.

La recuperación de este subsistema requiere copiar tanto los directorios de ficheros como las tablas que contienen sus rutas. Una copia de MySQL sin `static/uploads`, `static/results` o `training_results` restauraría referencias incompletas; una copia de los ficheros sin la base de datos perdería la asociación con usuarios, consultas y sesiones. Esta dependencia debe reflejarse en el procedimiento operativo de copia y restauración.

18.1.3. SSOP-003: Entorno de ejecución e integraciones externas

SSOP-003 agrupa los elementos necesarios para iniciar la aplicación y comunicarse con los servicios que no forman parte del proceso local. El servidor se ejecuta con Uvicorn sobre Python 3.11 y utiliza FastAPI como framework web. El fichero requirements.txt fija las versiones de las dependencias principales, entre ellas TensorFlow, Keras, Transformers, mysql-connector-python, Groq, fpdf2, bcrypt, python-jose, slowapi, pytest y ruff.

El entorno virtual aísla estas dependencias de otras instalaciones de Python de la máquina. Antes de iniciar el sistema se debe comprobar que el intérprete activo corresponde al entorno del proyecto y que las versiones instaladas coinciden con el fichero de requisitos. Esta condición es especialmente importante para TensorFlow, Keras y Transformers, cuyas incompatibilidades pueden impedir la importación del motor de inferencia aunque el código de los routers sea correcto.

La aplicación se integra con Groq mediante chatbot_service.py. El servicio externo recibe los mensajes de configuración y devuelve una respuesta que el laboratorio procesa. El subsistema de soporte no expone una ruta genérica para llamar a Groq: la integración se encapsula en un único componente y utiliza GROQ_API_KEY desde el entorno. Esta frontera reduce la propagación de credenciales y permite sustituir o simular el proveedor durante las pruebas.

La carga de modelos Transformer constituye la otra integración externa relevante. ml_engine.py utiliza la configuración de Hugging Face para resolver las arquitecturas y pesos correspondientes. El primer uso puede requerir conectividad y espacio local para descargar o almacenar los modelos; los usos posteriores pueden beneficiarse de la caché disponible. La operación debe registrar como fallo de integración la imposibilidad de cargar un modelo, sin atribuir ese problema a la validación de la imagen o a la base de datos.

El entorno de publicación utiliza el túnel configurado para la demostración para que un navegador externo acceda al servidor local. El túnel no aloja los datos ni ejecuta los modelos; solo proporciona el canal de entrada hasta Uvicorn. MySQL permanece en la máquina de ejecución y sus puertos no deben publicarse. Esta configuración conserva bajo control las versiones y los ficheros del proyecto, aunque no proporciona por sí misma alta disponibilidad ni escalado horizontal.

18.1.4. SSOP-003: Entorno de ejecución e integraciones externas

El subsistema SSOP-004 agrupa las herramientas que permiten comprobar que los cambios no rompen el comportamiento existente. Las pruebas unitarias se encuentran en tests/unit, las de integración en tests/integration y las fixtures compartidas en tests/conftest.py. Los servicios externos se simulan mediante mocks y la base de datos se sustituye en las pruebas de integración por el entorno definido para ese nivel.

La estrategia de pruebas refleja la arquitectura. Los routers se prueban comprobando respuestas, autenticación, propiedad y errores; los servicios de aprendizaje automático se prueban con modelos simulados; el worker se prueba con trabajos controlados; y los middlewares se prueban de forma aislada para confirmar CSRF, cabeceras y limitación. La batería existente incluye pruebas de autenticación, validación de entradas, base de datos, inferencia, XAI, MLOps, cola, seguridad e internacionalización, además del flujo de autenticación integrado.

pytest-cov permite medir la cobertura de los módulos principales de services, routers y database.py. ruff verifica el estilo del código conforme a las convenciones de Python (Python Software Foundation, 2024). El repositorio actual no contiene un workflow de integración continua versionado, por lo que las comprobaciones automatizadas se ejecutan mediante los comandos definidos para el proyecto. La calidad no se concentra en una única prueba de aceptación: se distribuye entre verificaciones rápidas durante el desarrollo y comprobaciones más amplias antes de integrar cambios.

Las fixtures deben mantenerse deterministas y sin datos reales de pacientes. Los modelos, clientes externos y conexiones de producción se sustituyen por dobles de prueba en los niveles que requieren aislamiento. La prueba de integración debe comprobar la colaboración entre capas, pero no convertir el entorno de producción ni la API de Groq en dependencias necesarias para cada ejecución local. Esta política mejora la repetibilidad y evita que una interrupción externa produzca falsos fallos del código.

18.2. Mecanismos de diseño transversales

Los mecanismos de diseño son reglas que se aplican a varios subsistemas independientemente de la funcionalidad concreta que implementen. Se distinguen de los subsistemas de soporte porque no son necesariamente un servicio con una identidad propia: son decisiones repetidas sobre cómo validar, autenticar, persistir, ejecutar y comunicar errores. En vitalXAI se identifican ocho mecanismos principales.

18.2.1. Configuración externa y separación de secretos

La configuración que cambia entre máquinas o que contiene información sensible se obtiene desde variables de entorno. Este mecanismo se aplica a autenticación, MySQL, Groq, duración de tokens y tamaño del pool. El código conserva valores por defecto únicamente para permitir el desarrollo local y emite advertencias cuando una configuración insegura puede afectar a la operación.

La separación de secretos evita que una clave JWT, una contraseña de MySQL o una clave de Groq quede incrustada en un router o en un servicio. Los componentes reciben la configuración en el momento de importación o de ejecución y utilizan nombres de variables comunes. El fichero de ejemplo o la guía de despliegue puede documentar los nombres requeridos, pero no debe contener valores válidos.

Este mecanismo también mejora la portabilidad. El mismo código puede ejecutarse con una base de datos local, una URL pública del túnel diferente o una duración de tokens ajustada al entorno sin modificar los routers. La configuración de desarrollo, pruebas y demostración debe mantenerse separada para evitar que una prueba utilice accidentalmente datos o credenciales del entorno operativo.

18.2.2. Validación en las fronteras de entrada

Toda información procedente del navegador se considera no confiable hasta ser validada. FastAPI valida tipos y parámetros declarados por las rutas; los routers añaden comprobaciones de negocio; y los servicios vuelven a comprobar las condiciones necesarias antes de utilizar los datos en operaciones sensibles.

En el registro se comprueba el correo, la longitud de la contraseña y los campos obligatorios. En el diagnóstico se validan MIME y tamaño de la imagen. En el laboratorio se comprueba la existencia del dataset y la aplicación verifica el `user_id` guardado en la configuración de la sesión. En la cola se valida la identidad y la pertenencia del trabajo. La validación del cliente mediante JavaScript puede mejorar la experiencia, pero no es la referencia definitiva: la petición debe poder rechazarse correctamente aunque se construya con un cliente distinto del frontend oficial.

Las consultas SQL utilizan parámetros separados de las instrucciones, y los nombres de archivo se generan dentro del servidor antes de copiar el contenido recibido. Los datos estructurados del payload de la cola se serializan como JSON y se interpretan según un tipo de trabajo controlado. La combinación de estas decisiones reduce el riesgo de que una entrada del usuario se convierta directamente en una consulta, una ruta o una orden ejecutable.

18.2.3. Autenticación, autorización y propiedad de recursos

El mecanismo de identidad se reutiliza en todos los routers privados mediante `get_user_id_from_token()`. Esto evita que cada subsistema implemente una forma distinta de interpretar la sesión. Después de obtener el usuario, cada operación aplica el alcance correspondiente: propiedad del registro, rol administrativo o acceso general a una capacidad transversal.

La autorización se realiza cerca del recurso que se protege. El historial comprueba la propiedad de la consulta; la cola comprueba el trabajo; administración comprueba el rol; y el laboratorio verifica el `user_id` de la configuración de la sesión. El almacenamiento por carpetas no aporta una relación física equivalente con la cuenta propietaria, por lo que no todos los recursos tienen el mismo nivel de respaldo estructural frente a un identificador ajeno. Las funciones compartidas, como `_check_consultation_ownership()` y `_require_admin()`, concentran reglas repetidas sin convertirlas en un bypass general de seguridad.

CSRF, cabeceras y rate limiting se aplican mediante middleware o decoradores para que no dependan de que cada router recuerde incluirlos. No obstante, estos mecanismos son complementarios: un token CSRF válido no concede permisos sobre otro usuario y una cabecera de seguridad no sustituye a la comprobación de rol.

18.2.4. Persistencia de estado y transacciones simples

La aplicación utiliza MySQL como fuente de verdad para el estado de cuentas, consultas, tokens y trabajos. Las operaciones de escritura siguen una secuencia corta: obtener conexión, preparar una consulta parametrizada, ejecutar, confirmar y cerrar. Esta política reduce el tiempo durante el que se mantiene un recurso del pool y facilita localizar la operación que produjo un error.

Los estados de los trabajos se persisten en lugar de mantenerse solo en una variable del proceso. Esto permite que el router de cola y el worker observen la misma información y que un reinicio pueda volver a encolar trabajos que quedaron en ejecución. Esta recuperación es básica: no reconstruye el punto intermedio del cálculo ni limpia posibles artefactos parciales. La actualización condicional de `_claim_job()` actúa como mecanismo sencillo de exclusión para evitar que dos consumidores reclamen la misma tarea.

La persistencia de MySQL y la de ficheros deben coordinarse de manera explícita. El worker genera primero los artefactos del diagnóstico y, cuando ese proceso termina, inserta la fila de consulta con sus referencias. Si falla la escritura de un informe o de un mapa XAI, el trabajo debe quedar registrado como fallido y no debe crearse una consulta incompleta en el historial. Un reinicio durante otra tarea puede dejar artefactos parciales que requieren limpieza posterior.

18.2.5. Ejecución asíncrona y aislamiento de trabajos

La ejecución asíncrona se implementa mediante una cola persistida y un worker interno. Los routers registran el trabajo y devuelven la respuesta; `queue_worker.py` lo reclama y envía el cálculo pesado a `run_in_executor`. Este patrón libera el ciclo de petición y permite que la interfaz consulte el progreso sin esperar a que termine el entrenamiento o la inferencia.

El worker limita sus operaciones a los tipos explícitos `diagnosis`, `training` y `external_validation`. Cada tipo tiene una función de procesamiento que interpreta su `payload`, invoca al servicio adecuado y devuelve un resultado serializable. El bucle exterior captura los errores de un trabajo, los registra como `failed` y continúa con el siguiente. La recuperación de trabajos `running` al arrancar evita que una interrupción deje tareas invisibles.

La ejecución asíncrona no significa que todas las tareas tengan el mismo nivel de prioridad o el mismo mecanismo. Los entrenamientos y validaciones utilizan `job_queue`; el recálculo estadístico del laboratorio usa `BackgroundTasks`; y las peticiones de consulta permanecen síncronas porque solo recuperan información. Documentar esta diferencia evita que una futura modificación asuma garantías de persistencia para una tarea que actualmente se ejecuta como trabajo en segundo plano del proceso web.

18.2.6. Errores, estados y comunicación con el usuario

Los errores se traducen en respuestas que la interfaz puede interpretar. Las rutas devuelven 401 cuando falta autenticación, 403 cuando el usuario no tiene autorización, 404 cuando el recurso no existe y 400 cuando los datos recibidos no cumplen una condición de negocio. Las excepciones inesperadas se convierten en respuestas 500 y, cuando pertenecen a un trabajo asíncrono, se conservan en el estado `failed` de la cola. Algunos routers todavía incluyen el texto de la excepción en la respuesta 500, por lo que ese comportamiento debe endurecerse antes de un despliegue de producción.

La capa de presentación recibe mensajes localizados mediante `lang.py`, por lo que los routers no deberían mezclar en cada respuesta textos técnicos sin una razón concreta. Al mismo tiempo, el cliente no debe recibir claves secretas, trazas completas ni credenciales. El detalle necesario para diagnosticar debe conservarse en la salida del servidor o en el estado de error del trabajo, mientras que la interfaz debería obtener un mensaje comprensible y seguro.

Los estados `queued`, `running`, `completed`, `failed` y `cancelled` forman un contrato de comunicación entre el worker, los routers y el navegador. El usuario puede saber si la petición fue aceptada, si se está ejecutando, si terminó correctamente o si debe revisar un error. La uniformidad de este contrato reduce la necesidad de que cada vista invente una representación diferente para los trabajos.

18.2.7. Modularidad, pruebas y evolución controlada

La modularidad se aplica separando routers, servicios, worker, persistencia y recursos de presentación. Los routers coordinan HTTP; los servicios encapsulan lógica especializada; database.py proporciona conexiones; queue_worker.py ejecuta trabajos; y las plantillas y scripts representan la interacción. La separación no es absoluta: los routers actuales acceden directamente a `get_db_connection()` en varias operaciones. Aun así, las responsabilidades principales siguen identificables y permiten probar cada grupo de forma aislada.

Las pruebas sirven como contrato evolutivo de esos límites. Los cambios en autenticación deben conservar las pruebas de tokens y cookies; los cambios en diagnóstico deben conservar la validación de imágenes y la integración con la cola; los cambios en MLOps deben conservar la propiedad de sesiones y los estados; y los cambios en middleware deben conservar las pruebas CSRF, cabeceras y rate limiting. Esta relación entre mecanismo de diseño y prueba reduce el riesgo de que una refactorización altere una garantía transversal sin que el cambio sea detectado.

18.2.8. Ciclo de vida y sustitución controlada de componentes

Los componentes de soporte deben poder iniciarse y utilizarse sin dejar estados imposibles de interpretar. Durante el arranque, FastAPI carga la configuración, comprueba la base de datos e inicia el worker. Durante la operación, los routers solicitan conexiones y ficheros únicamente cuando los necesitan. El lifespan actual no implementa un cierre ordenado del worker; si la aplicación se reinicia, `start_worker()` devuelve a la cola los trabajos que quedaron en ejecución. Esta secuencia vincula el ciclo de vida del proceso web con la persistencia de `job_queue`.

La separación de soporte también facilita sustituir un componente si el entorno lo exige. MySQL podría cambiarse por otro motor, pero sería necesario adaptar el contrato de `get_db_connection()`, las consultas, el esquema, las pruebas y el comportamiento transaccional. Groq podría reemplazarse si el servicio conversacional conserva la entrada de mensajes y la salida estructurada; y el directorio local de artefactos podría trasladarse a un almacenamiento especializado si se mantiene la relación entre rutas, usuarios y resultados. Estas posibilidades no forman parte de la implementación actual.

La sustitución no debe realizarse sin revisar los mecanismos transversales. Cambiar MySQL afecta al pool, a las transacciones, a las claves ajenas y a la recuperación de la cola; cambiar el almacenamiento afecta a las rutas de imágenes, informes y resultados; y cambiar el proveedor externo afecta a las credenciales, los tiempos de espera y el tratamiento de errores. Por ello, cualquier cambio de infraestructura debe acompañarse de pruebas de integración y de una comprobación de que los estados y permisos siguen siendo equivalentes.

El entorno de construcción conserva este principio mediante dependencias fijadas y pruebas reproducibles. Una nueva versión de TensorFlow, Keras, Transformers o mysql-connector-python no debe incorporarse únicamente porque resuelva un problema local: debe instalarse en un entorno aislado, ejecutarse la batería de pruebas y comprobarse que los modelos, los ficheros y las consultas mantienen el comportamiento esperado. Esta reproducibilidad está condicionada por la versión de Python, el sistema operativo, los pesos, los datasets, la GPU y las semillas utilizadas. La configuración de producción y la de pruebas deben permanecer separadas para que una migración o una prueba de recuperación no altere los datos de uso.

En conjunto, estos subsistemas completan la estructura técnica de vitalXAI. SSOP-001 sostiene las relaciones y estados persistentes; SSOP-002 conserva los artefactos y recursos; SSOP-003 proporciona el entorno y las integraciones; y SSOP-004 verifica el comportamiento. Sobre ellos se aplican la configuración externa, la validación, la seguridad, la persistencia de estados, la asincronía, la comunicación de errores y la modularidad. Ninguno sustituye a los subsistemas funcionales, pero todos son necesarios para que se ejecuten conforme a los requisitos y restricciones del proyecto.

Las responsabilidades quedan separadas: el desarrollador mantiene el código y las pruebas, el operador prepara el entorno, las credenciales y MySQL, y el sistema conserva los estados y errores necesarios para diagnosticar una ejecución. Así, una incidencia de infraestructura puede distinguirse de un fallo funcional y localizarse en el componente correspondiente.

Esta arquitectura deja definidos los puntos que deben comprobarse cuando se implanta o modifica la plataforma: disponibilidad de MySQL y de los directorios, validez de las variables de entorno, compatibilidad de las dependencias, funcionamiento del worker, conservación de los estados de la cola y ejecución de las pruebas automatizadas. La operación de vitalXAI depende de que estos elementos funcionen conjuntamente; por ello, el soporte no es un añadido posterior, sino la base que permite que la lógica clínica y experimental se ejecute de forma repetible y controlada.

El resultado es una infraestructura ajustada al alcance académico y técnico del proyecto.

Capítulo 19. Persistencia relacional y esquema de datos de vitalXAI

Este capítulo describe cómo vitalXAI conserva la información que produce y utiliza. El análisis del capítulo 14 identificó las entidades y relaciones desde una perspectiva conceptual, mientras que el diseño de los capítulos 17 y 18 fijó la arquitectura y los mecanismos de soporte. Aquí se concreta su representación física: tablas, columnas, tipos de datos, claves y restricciones. El objetivo es que el esquema pueda reproducirse, verificarse y evolucionar sin ambigüedades durante el proyecto.

La definición física también permite comprobar decisiones que, en el modelo conceptual, solo se expresan de forma abstracta. Las claves, restricciones y campos obligatorios muestran si las relaciones de propiedad y la integridad de los datos tienen respaldo en la base de datos. En vitalXAI esta comprobación debe considerar además los artefactos que se conservan fuera de las tablas.

El motor sobre el que se sustenta esta persistencia es MySQL, accedido desde la aplicación mediante `mysql-connector-python` y un pool de conexiones `MySQLConnectionPool` (Oracle, 2024). La definición efectiva del esquema se realiza con sentencias `CREATE TABLE IF NOT EXISTS` que la función `init_db()` ejecuta durante el arranque; no se emplea un ORM ni un sistema de migraciones versionadas. El archivo `agents/db/schema.sql` contiene una representación documental incompleta, ya que no incluye la tabla `job_queue`; por tanto, `database.py` es actualmente la fuente que crea el esquema utilizado por la aplicación. Esta forma de trabajar simplifica la puesta en marcha local, pero cualquier cambio posterior sobre las tablas deberá planificarse y comprobarse de forma independiente.

Conviene destacar, además, que el almacenamiento de vitalXAI no se limita a las tablas de MySQL. El sistema conserva en el sistema de ficheros los artefactos de mayor tamaño o de estructura variable, como las imágenes radiológicas, los mapas de explicabilidad, los informes en PDF y los resultados del laboratorio de entrenamiento, y las tablas guardan únicamente las rutas o referencias que permiten localizarlos. Esta separación entre datos relacionales y artefactos por fichero es una decisión de diseño deliberada, coherente con el subsistema de soporte SSOP-002 del capítulo 18, y condiciona de forma directa el procedimiento de copia y recuperación de la información. Una copia de seguridad que incluya solo la base de datos dejaría referencias a ficheros inexistentes; una que incluya solo los ficheros perdería la asociación con los usuarios y las consultas. El esquema físico debe entenderse, por tanto, como la parte relacional de un modelo de almacenamiento híbrido.

El presente capítulo se limita al esquema de persistencia y a su uso. El apartado 19.1 presenta los criterios de diseño que rigen el esquema, los diagramas que lo representan y la descripción detallada de cada una de las cinco tablas que lo componen, mediante fichas que recogen sus atributos, sus claves y sus restricciones. El apartado 19.2 describe cómo acceden los componentes del sistema a esa capa de persistencia, qué operaciones realiza cada uno y qué consideraciones de rendimiento y de concurrencia deben tenerse en cuenta. El acceso a los datos en su vertiente de infraestructura, incluida la configuración del pool, el ciclo de vida de las conexiones y la inicialización del esquema, se documenta en el capítulo 18 dentro del subsistema de soporte SSOP-001 y, por tanto, no se repite aquí más allá de lo necesario para contextualizar este capítulo.

19.1. *Diseño del esquema de persistencia*

No todas las entidades identificadas en el capítulo 14 se convierten en tablas. El almacenamiento de vitalXAI es híbrido: las cuentas, consultas, cola y tokens se conservan en MySQL, mientras que los resultados variables del laboratorio se guardan principalmente en el sistema de ficheros. El esquema contiene cinco tablas: users, consultations, job_queue, refresh_tokens y training_jobs. La diferencia con las once entidades conceptuales se debe a esta separación entre información estructurada y artefactos.

Antes de describir cada tabla, se fijan los criterios comunes del esquema. Estos criterios permiten interpretar las columnas y restricciones como decisiones relacionadas, no como reglas aisladas.

- **Identificadores enteros autoincrementales.** Cada tabla define su clave primaria como una columna id de tipo INT con la propiedad AUTO_INCREMENT, de modo que MySQL asigna el identificador al insertar una nueva fila (Oracle, 2024). En este proyecto se evita así generar identificadores desde la aplicación. La asignación automática impide valores repetidos dentro de una misma tabla, aunque no implica que la secuencia sea continua.
- **La cuenta de usuario como eje de propiedad relacional.** Las cuatro tablas dependientes de users incorporan una clave ajena user_id que apunta a users.id. Esta clave protege la propiedad de las consultas, los trabajos, los tokens y los entrenamientos representados en esas tablas. La base de datos garantiza la existencia de la cuenta referenciada, y la aplicación complementa esa garantía con las comprobaciones de propiedad y de rol que aplica en sus rutas. Las sesiones del laboratorio se conservan como carpetas y guardan el user_id en su configuración, pero no tienen una relación física con users.
- **Tipos de datos ajustados al contenido.** Las cadenas acotadas se declaran con VARCHAR, los valores numéricos con INT o FLOAT, las marcas temporales con DATETIME y los contenidos variables con JSON o TEXT (Oracle, 2024). En este proyecto, FLOAT se considera suficiente para los valores de confianza y progreso. El uso de DATETIME responde al alcance local del sistema y no pretende resolver las necesidades de un despliegue distribuido con zonas horarias.
- **Borrado de cuentas y datos dependientes.** Las relaciones de job_queue, refresh_tokens y training_jobs con users declaran ON DELETE CASCADE, pero la relación de consultations con users no define una acción de borrado. Por tanto, el borrado de una cuenta que todavía tiene consultas puede ser rechazado por la restricción de clave ajena. Esta decisión no debe justificarse por el flujo de eliminación de una consulta: la clave ajena determina qué ocurre cuando se elimina la cuenta propietaria. Además, los artefactos de consultas y las carpetas del laboratorio no se eliminan automáticamente mediante la base de datos. El comportamiento actual debe considerarse una limitación pendiente: cualquier evolución del borrado de cuentas deberá coordinar las filas relacionales con los ficheros y las carpetas asociadas.
- **Persistencia relacional y artefactos por fichero.** El esquema no pretende guardar en columnas los datos binarios ni la información variable del laboratorio. Las imágenes, los mapas, los informes y los resultados de entrenamiento se escriben en directorios del sistema de ficheros, y las tablas conservan las rutas que los referencian. Esta decisión mantiene las tablas ligeras y evita que el crecimiento de los artefactos degrade el rendimiento de las consultas.

- **Creación y evolución del esquema sin migraciones versionadas.** Las tablas se crean mediante `CREATE TABLE IF NOT EXISTS` durante el arranque. El código Python actúa, por tanto, como fuente efectiva de definición del esquema. Esta estrategia permite levantar una instalación nueva, pero no modifica tablas existentes; la evolución del esquema deberá gestionarse como un cambio de base de datos planificado y probado, sin atribuir a las sentencias de creación una capacidad de migración que no tienen.

En cuanto a los índices, el esquema no declara índices adicionales explícitos. MySQL/InnoDB crea índices asociados a las claves primarias, las restricciones de unicidad y las claves ajenas (Oracle, 2024). En el volumen actual, estos índices cubren las consultas habituales, aunque no se ha documentado una medición mediante `EXPLAIN`. Si el volumen creciera, deberían valorarse índices sobre las columnas de fecha del historial o sobre el estado y tipo de los trabajos; estas optimizaciones quedan fuera de la implementación actual.

La correspondencia entre el modelo conceptual y el esquema físico merece una precisión adicional, porque es aquí donde se materializa la naturaleza híbrida del almacenamiento. De las once entidades del capítulo 14, Usuario se materializa en `users`; ConsultaDiagnostico en `consultations`; y TrabajoCola en `job_queue`. Los mapas de explicabilidad (MapaXAI) no tienen una tabla propia: se representan mediante la columna `xai_image_path` de `consultations`, que apunta al fichero generado, y su tipo de mapa no se almacena en una columna independiente. Las entidades del laboratorio, incluidas `SesionExperimentacion`, `ModeloEntrenado`, `ResultadoModelo`, `ResultadoPliegue`, `ValidacionExterna`, `ResultadoValidacionExterna` y `ComparacionModelos`, se conservan en el directorio `training_results`, organizadas por un identificador de sesión con formato `RUN_AAAAMMDD_HHMMSS`. Cada sesión se materializa como una carpeta que contiene la configuración del experimento, los modelos entrenados, sus métricas por pliegue y agregadas, sus análisis de explicabilidad, los resultados individuales de la validación externa y las comparaciones estadísticas cuando se han solicitado. La configuración de la sesión incluye el `user_id`, aunque la carpeta no tiene una clave ajena ni una relación física con `users`.

Esta decisión se justifica por la estructura variable de los artefactos experimentales. Un entrenamiento produce, para cada modelo y cada pliegue, métricas de validación cruzada, métricas de calibración, métricas de explicabilidad, curvas y mapas, cuya normalización en tablas exigiría un esquema mucho más complejo y rígido, desproporcionado para el alcance del proyecto. La persistencia por ficheros permite, en cambio, que cada nuevo resultado se almacene sin modificar el esquema, y que el laboratorio se gestione con las herramientas habituales del sistema de ficheros. El capítulo 14 describe las once entidades desde el punto de vista conceptual, y este capítulo describe las cinco tablas desde el punto de vista físico; ambos deben leerse de forma complementaria: el primero expresa el modelo de negocio y el segundo su materialización, con la salvedad explícita de que parte de ese modelo vive en los ficheros y no en la base de datos.

El diagrama entidad-relación de la ilustración representa el esquema en notación Chen. En esta notación, las entidades se dibujan como rectángulos, las relaciones como rombos y las cardinalidades se indican sobre las aristas (Chen, 1976; Elmasri & Navathe, 2016). La entidad `USERS` actúa como raíz y participa en cuatro relaciones de tipo uno a muchos con el resto de las entidades. El diagrama se completa a continuación con un boceto en Mermaid que puede adaptarse y refinarse.

En el diagrama, la entidad USERS se sitúa como raíz del modelo y participa en cuatro relaciones de tipo uno a muchos con las tablas relacionales representadas. Un usuario puede realizar cero o muchas consultas de diagnóstico, poseer cero o muchos trabajos de entrenamiento, encolar cero o muchos trabajos asíncronos y disponer de cero o muchos tokens de refresco. Cada relación se materializa mediante la clave ajena user_id que incluyen las tablas correspondientes. Las sesiones y los resultados del laboratorio quedan fuera de este diagrama relacional porque se conservan como carpetas y artefactos del sistema de ficheros.

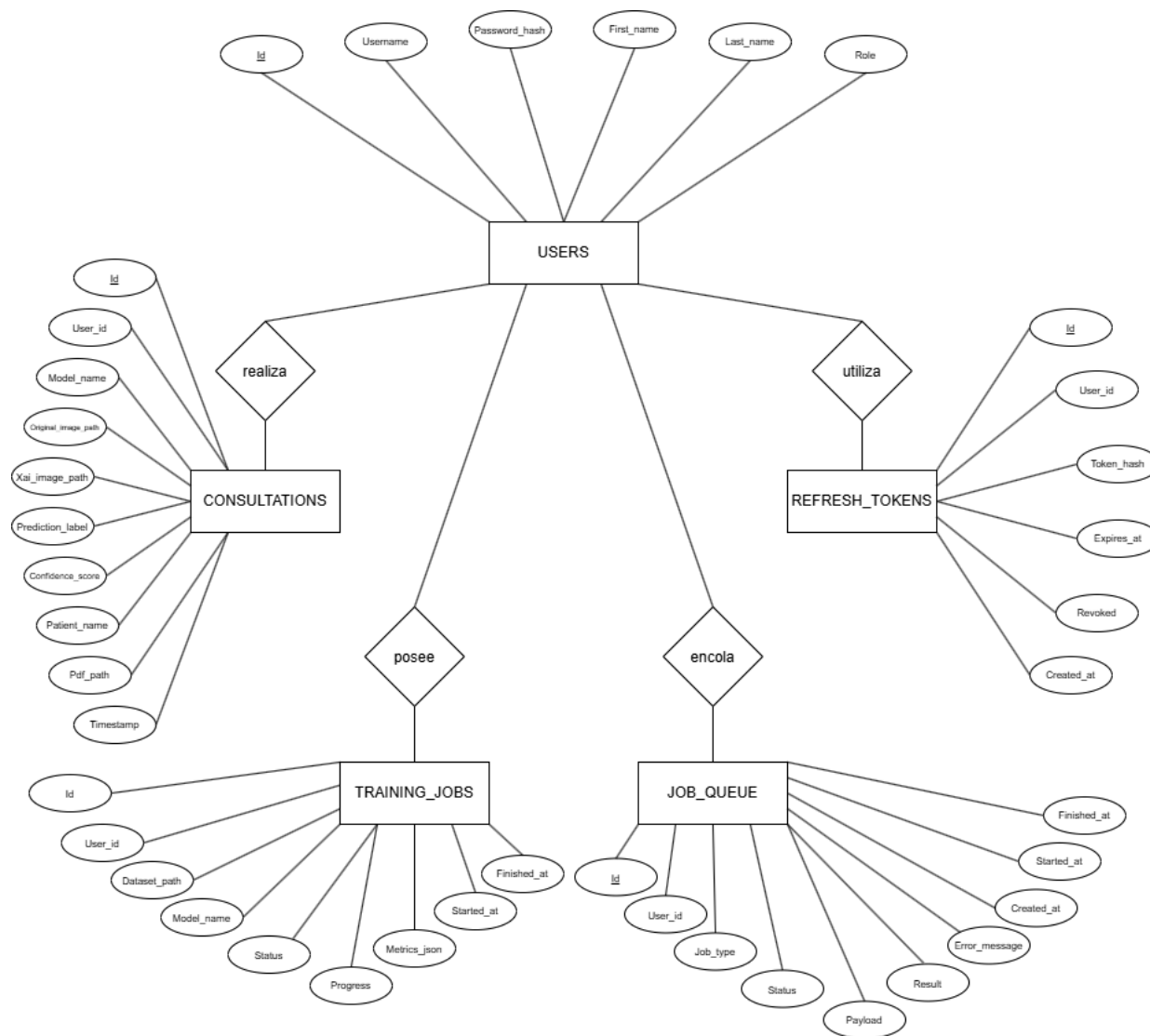


Ilustración x - Diagrama entidad-relación del esquema de persistencia en notación Chen

El diagrama relacional de la siguiente ilustración representa el esquema desde la perspectiva de las tablas y sus claves. A diferencia del anterior, que se centra en las entidades y sus relaciones conceptuales, el diagrama relacional destaca las columnas que actúan como clave primaria y como clave ajena, y la forma en que las tablas se conectan entre sí. Este diagrama es el que más se aproxima a la implementación real: cada caja corresponde a una tabla de MySQL y cada columna de la caja a una columna de esa tabla. Las entidades del laboratorio que se almacenan como ficheros no aparecen como tablas ni como relaciones físicas con users.

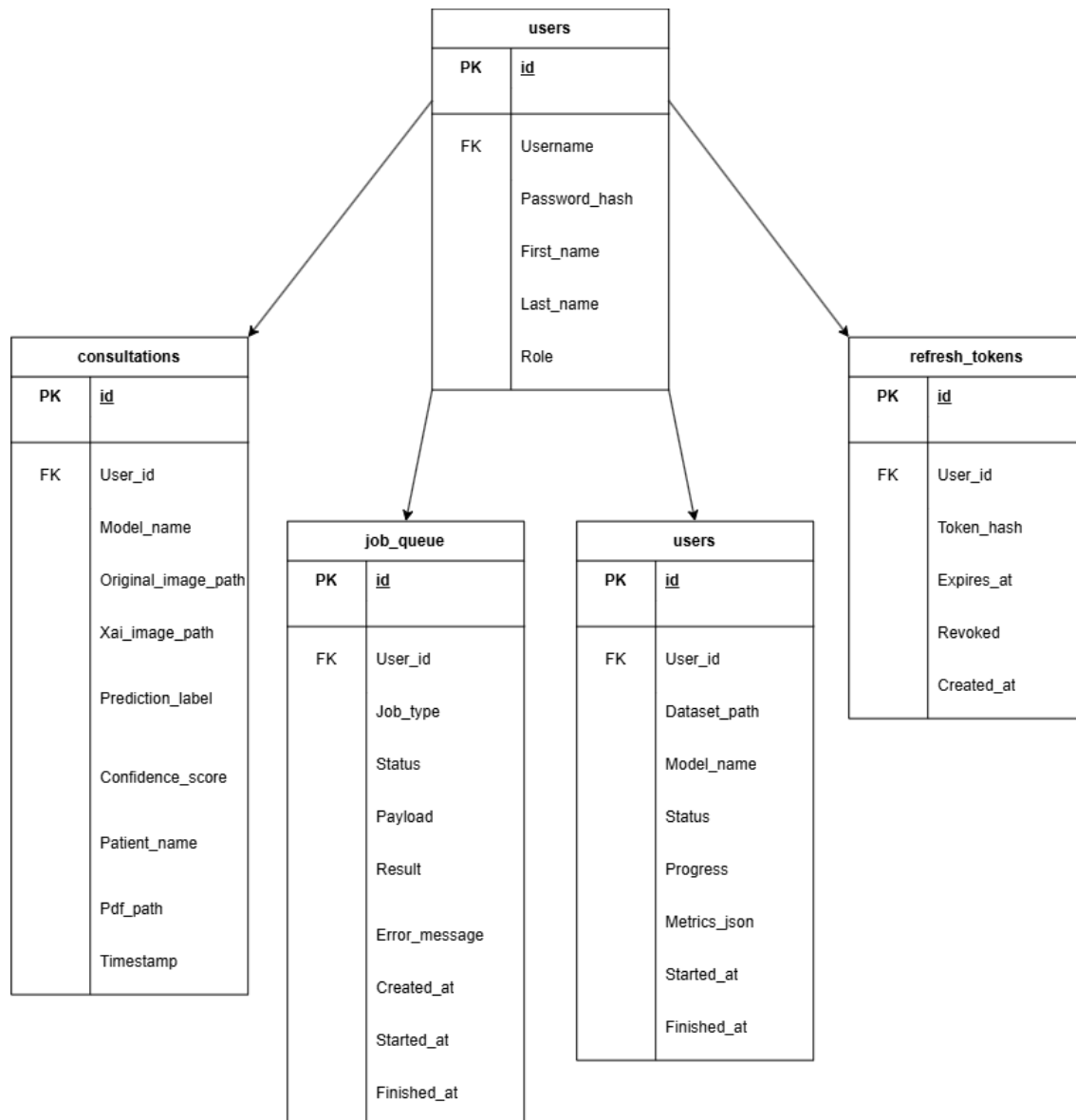


Ilustración x - Diagrama relacional de tablas y claves de vitalXAI

A continuación se describen, mediante una ficha por tabla, la estructura y las restricciones de cada una de las cinco tablas del esquema. Cada ficha indica el nombre, la descripción, los atributos con su tipo y su obligatoriedad, la clave primaria, las claves ajenas, las claves únicas y las restricciones que le aplican. Los identificadores como users_pkey o consultations_user_id_fkey se utilizan aquí como etiquetas documentales: las sentencias actuales no declaran nombres explícitos para todas las restricciones y el nombre efectivo puede depender del motor. Tras cada ficha se incluye una breve discusión de las decisiones que la motivan y de las implicaciones de su uso en el sistema.

Nombre	users		
Descripción	Almacena los datos de identidad de las cuentas registradas. Es la entidad central del modelo y el origen de las relaciones de propiedad del resto de tablas.		
Atributos			
Campo	Tipo	Obligatorio	Descripción
Id	INT	S	Identificador interno de la cuenta. Clave primaria autoincremental.
Username	VARCHAR(255)	S	Identificador de acceso, validado como correo electrónico.
Password_hash	VARCHAR(255)	S	Hash de la contraseña generado con bcrypt. Nunca se almacena la contraseña en claro.
First_name	VARCHAR(255)	S	Nombre mostrado del usuario.
Last_name	VARCHAR(255)	S	Apellidos mostrados del usuario.
Role	VARCHAR(255)	S	Perfil de autorización de la cuenta (ordinario o admin).
Clave primaria			
Nombre	Columnas		Secuencia
users_pkey	Id		Auto incremental de MySQL (AUTO INCREMENT)
Claves ajenas			
Nombre	Destino	Columnas	
N/A	N/A	N/A	
Claves únicas			
Nombre	Columnas		
users_username_key	Username		
Restricciones			
Nombre	Columnas	Restricción	
users_username_unique	Username	Username	
users_role_check	Role	La aplicación restringe el rol a los valores ordinario y admin; el esquema no define un CHECK.	

Tabla x - TB-01: users

La unicidad de username se aplica a nivel de base de datos y constituye la última barrera frente a cuentas duplicadas, incluso cuando dos peticiones de registro llegan de forma concurrente; antes de insertar, el router de autenticación valida el formato y comprueba la disponibilidad del identificador. La columna password_hash conserva el resultado de aplicar la función de hash bcrypt, de modo que la contraseña original nunca queda almacenada, en coherencia con el requisito RNF-001. La columna role es interpretada por la aplicación al proteger las rutas administrativas mediante la comprobación _require_admin() descrita en el capítulo 18; el esquema no introduce una tabla separada de roles, decisión suficiente para el alcance actual.

La columna password_hash conserva el resultado de aplicar la función de hash bcrypt a la contraseña, de modo que la contraseña original nunca queda almacenada, en coherencia con el requisito RNF-001. Esta decisión tiene una consecuencia operativa: la verificación de credenciales en el inicio de sesión se realiza siempre comparando el hash de la contraseña recibida con el hash almacenado, nunca recuperando una contraseña en claro. La columna role es interpretada por la aplicación al proteger las rutas administrativas mediante la comprobación _require_admin() descrita en el capítulo 17. Sin embargo, el endpoint de registro actual acepta el rol enviado por el cliente, por lo que la asignación inicial del perfil no queda restringida de forma segura. El esquema no introduce una tabla separada de roles ni una relación de muchos a muchos entre usuarios y permisos, decisión suficiente para el alcance actual, aunque un sistema con una matriz de permisos más compleja requeriría normalizarlos en tablas propias. La tabla tampoco incluye una columna de borrado lógico: la baja de una cuenta no forma parte del flujo funcional actual, y su posible incorporación futura debería plantearse de forma coordinada con el resto del esquema.

Nombre	consultations		
Descripción	Almacena el historial de consultas de diagnóstico de los usuarios. Cada fila representa una consulta completada y las rutas de sus artefactos asociados.		
Atributos			
Campo	Tipo	Obligatorio	Descripción
Id	INT	S	Identificador interno de la consulta. Clave primaria autoincremental.
User_id	INT	S	Cuenta propietaria de la consulta. Clave ajena a users.
Model_name	VARCHAR(100)	N	Arquitectura utilizada en el diagnóstico.
Original_image_path	VARCHAR(500)	N	Ruta de la imagen radiológica original.
Xai_image_path	VARCHAR(500)	N	Ruta del mapa de explicabilidad generado.
Prediction_label	VARCHAR(50)	N	Clasificación devuelta (PNEUMONIA o NORMAL).
Confidence_score	FLOAT	N	Confianza asociada a la predicción.
Patient_name	VARCHAR(255)	N	Nombre visible de organización de la consulta.
Pdf_path	VARCHAR(500)	N	Ruta del informe PDF generado.
Timestamp	DATETIME	N	Fecha y hora del registro, con valor por defecto CURRENT_TIMESTAMP.
Clave primaria			
Nombre	Columnas	Secuencia	
consultations_pkey	Id	Auto incremental de MySQL (AUTO_INCREMENT)	
Claves ajenas			
Nombre	Destino	Columnas	
consultations_user_id_fkey	users	User_id	
Claves únicas			
Nombre	Columnas		
N/A	N/A		
Restricciones			
Nombre	Columnas	Restricción	
consultations_user_id_not_null	User_id	Valor no nulo: toda consulta debe asociarse a una cuenta válida.	
consultations_prediction_label_check	Prediction_label	La aplicación restringe la etiqueta a PNEUMONIA o NORMAL; el esquema no define un CHECK.	

Tabla x - TB-02: consultations

Esta tabla se alimenta desde el worker de la cola cuando un diagnóstico termina: una vez obtenida la predicción y generados los artefactos de explicabilidad y el informe, el worker registra la consulta con sus metadatos y las rutas de los ficheros. Las columnas de rutas apuntan a los ficheros conservados en el sistema de ficheros, no a su contenido, de modo que reconstruir una consulta completa exige disponer tanto de la fila como de los ficheros referenciados; una copia de seguridad de MySQL sin los directorios correspondientes restauraría referencias incompletas. patient_name actúa como una etiqueta visible que el usuario puede renombrar para organizar su historial, y debe tratarse como un dato de presentación, no como una autorización para guardar identificadores personales del paciente. El estado operativo del diagnóstico no reside en esta tabla: se conserva en job_queue, y la presencia de una fila aquí indica que el diagnóstico alcanzó la fase de persistencia del resultado.

Nombre	job_queue		
Descripción	Almacena los trabajos asíncronos de la plataforma (diagnósticos, entrenamientos y validaciones externas) con su tipo, su estado, sus parámetros y su resultado.		
Atributos			
Campo	Tipo	Obligatorio	Descripción
Id	INT	S	Identificador interno del trabajo. Clave primaria autoincremental.
User_id	INT	S	Cuenta propietaria del trabajo. Clave ajena a users.
Job_type	VARCHAR(20)	S	Tipo de trabajo: diagnosis, training o external_validation.
Status	VARCHAR(20)	N	Estado del ciclo de vida, con valor por defecto queued.
Payload	JSON	N	Parámetros necesarios para ejecutar la tarea.
Result	JSON	N	Resultado serializado del trabajo.
Error_message	TEXT	N	Motivo de un fallo, si lo hubo.
Created_at	DATETIME	N	Fecha de encolado, con valor por defecto CURRENT_TIMESTAMP.
Started_at	DATETIME	N	Fecha de inicio de la ejecución.
Finished_at	DATETIME	N	Fecha de finalización o cancelación.
Clave primaria			
Nombre	Columnas		Secuencia
job_queue_pkey	Id		Auto incremental de MySQL (AUTO_INCREMENT)
Claves ajenas			
Nombre	Destino		Columnas
job_queue_user_id_fkey	users		User_id
Claves únicas			
Nombre	Columnas		
N/A	N/A		
Restricciones			
Nombre	Columnas		Restricción
job_queue_user_id_not_null	User_id		Valor no nulo: todo trabajo debe asociarse a una cuenta válida.
job_queue_job_type_check	Job_type		La aplicación restringe el tipo a diagnosis, training o external_validation; el esquema no define un CHECK.
job_queue_status_check	Status		La aplicación restringe el estado a queued, running, completed, failed o cancelled; el esquema no define un CHECK.

Tabla x - TB-03: job_queue

La tabla `job_queue` es el soporte persistente de la ejecución asíncrona. Permite que el worker reclame y ejecute los trabajos, que el usuario consulte su progreso o los cancele, y que un reinicio del servidor vuelva a encolar las tareas que quedaron en marcha. A diferencia de una cola residente exclusivamente en memoria, esta tabla conserva el estado observable por el worker y por el usuario a través del router de cola; no garantiza por sí sola la recuperación idempotente ni la limpieza de artefactos parciales.

Las columnas de fecha reconstruyen la evolución temporal de cada trabajo: `created_at` indica cuándo se encoló, `started_at` cuándo comenzó la ejecución y `finished_at` cuándo terminó o se canceló. Esta trazabilidad temporal es útil para diagnosticar retrasos o fallos, y constituye la base sobre la que se describe el ciclo de vida de los trabajos en los capítulos 17 y 18. Las columnas `payload` y `result`, de tipo JSON, permiten guardar parámetros heterogéneos sin crear una columna por cada valor posible: el `payload` de un diagnóstico contiene la ruta de la imagen y la arquitectura; el de un entrenamiento, la sesión y los hiperparámetros; el de una validación externa, la sesión y el dataset externo. A cambio, su contenido debe validarse en la aplicación, ya que MySQL no conoce la estructura interna del documento.

La tabla se asocia al usuario mediante `user_id`, lo que permite que la consulta y la cancelación de trabajos se filtren por propietario. No obstante, no mantiene claves ajenas hacia `consultations` ni hacia las sesiones de entrenamiento: la relación entre un trabajo y su resultado es lógica y se expresa mediante el `job_type` y el contenido del `payload`. Esta decisión reduce el número de relaciones rígidas del esquema y evita acoplar la cola a las entidades que procesa, pero traslada al worker la responsabilidad de interpretar correctamente el contenido del `payload` y de coordinarse con la persistencia de los resultados.

Nombre	refresh_tokens		
Descripción	Almacena las credenciales de refresco de las sesiones, con su hash, su expiración y su estado de revocación.		
Atributos			
Campo	Tipo	Obligatorio	Descripción
Id	INT	S	Identificador interno del registro. Clave primaria autoincremental.
User_id	INT	S	Cuenta asociada al token. Clave ajena a users.
Token_hash	VARCHAR(255)	S	Hash del valor aleatorio del token.
Expires_at	DATETIME	S	Fecha y hora de caducidad del token.
Revoked	BOOLEAN	N	Indica si la credencial ha sido invalidada, con valor por defecto false.
Created_at	DATETIME	N	Fecha de creación, con valor por defecto CURRENT_TIMESTAMP.
Clave primaria			
Nombre	Columnas	Secuencia	
refresh_tokens_pkey	Id	Auto incremental de MySQL (AUTO_INCREMENT)	
Claves ajenas			
Nombre	Destino	Columnas	
refresh_tokens_user_id_fkey	users	User_id	
Claves únicas			
Nombre	Columnas		
N/A	N/A		
Restricciones			
Nombre	Columnas	Restricción	
refresh_tokens_user_id_not_null	User_id	Valor no nulo: todo token debe asociarse a una cuenta válida.	
refresh_tokens_token_hash_not_null	Token_hash	Valor no nulo: solo se persiste el hash del token, nunca su valor original.	
refresh_tokens_expires_at_not_n ull	Expires_at	Valor no nulo: todo token debe declarar su fecha de caducidad.	

Tabla x - TB-04: refresh_tokens

La tabla refresh_tokens participa en el mecanismo de autenticación descrito en los capítulos 17 y 18, que articula la sesión mediante un token de acceso con expiración y un token de refresco de mayor duración. El valor original del token de refresco nunca llega a la base de datos; solo se conserva su hash, de modo que una exposición de los datos no comprometa directamente las sesiones. Esta decisión es coherente con el tratamiento que se da a las contraseñas en users: en ambos casos, el sistema persiste un valor derivado y no el secreto original.

Durante una renovación, el servicio de autenticación calcula el hash del valor recibido y busca una fila que coincida, no esté revocada y no haya caducado. La rotación del token invalida la fila anterior y crea una nueva con otro valor aleatorio, de modo que un mismo token de refresco no puede reutilizarse indefinidamente. Si se detecta el uso de un token revocado fuera del periodo de tolerancia, el servicio interpreta que la sesión puede haber sido comprometida y revoca todos los tokens del usuario, limitando el impacto de una posible exposición. La relación con users declara ON DELETE CASCADE, de modo que al eliminar la cuenta desaparecen sus credenciales de refresco, sin sustituir la revocación explícita que realiza el flujo de cierre de sesión.

Nombre	training_jobs		
Descripción	Representación estructurada prevista de los trabajos de entrenamiento de modelos. Definida en el esquema como parte de la inicialización de la base de datos.		
Atributos			
Campo	Tipo	Obligatorio	Descripción
Id	INT	S	Identificador interno del entrenamiento. Clave primaria autoincremental.
User_id	INT	S	Cuenta propietaria del entrenamiento. Clave ajena a users.
Dataset_path	VARCHAR(500)	N	Ruta del dataset utilizado.
Model_name	VARCHAR(100)	N	Arquitectura entrenada.
Status	VARCHAR(50)	N	Estado del entrenamiento, con valor por defecto In Progress.
Progress	FLOAT	N	Porcentaje de progreso, con valor por defecto 0.0.
Metrics_json	TEXT	N	Métricas serializadas del entrenamiento.
Started_at	DATETIME	N	Fecha de inicio, con valor por defecto CURRENT_TIMESTAMP.
Finished_at	DATETIME	N	Fecha de finalización.
Clave primaria			
Nombre	Columnas	Secuencia	
training_jobs_pkey	Id	Auto incremental de MySQL (AUTO_INCREMENT)	
Claves ajenas			
Nombre	Destino	Columnas	
training_jobs_user_id_fkey	users	User_id	
Claves únicas			
Nombre	Columnas		
N/A	N/A		
Restricciones			
Nombre	Columnas	Restricción	
training_jobs_user_id_not_null	User_id	Valor no nulo: todo entrenamiento debe asociarse a una cuenta válida.	

Tabla x - TB-05: training_jobs

La tabla training_jobs está definida en el esquema y se crea durante la inicialización, pero no se utiliza operativamente en la implementación actual: el flujo del laboratorio no inserta filas ni las consulta. La persistencia real del entrenamiento se realiza mediante el sistema de ficheros, en el directorio training_results, y el motor que lanza los entrenamientos (mlops_engine.py) no accede a la base de datos. Existen restos de lógica de actualización sobre la tabla en trainer_engine.py, un módulo que no participa en el flujo productivo, y un volcado histórico contiene filas antiguas, lo que sugiere un uso en versiones previas del proyecto. En consecuencia, esta tabla se documenta como una previsión del esquema cuya materialización se ha trasladado a los ficheros, sin atribuirle un uso que el código actual no realiza.

Existen restos de lógica de actualización sobre la tabla en `trainer_engine.py`, un módulo que no participa en el flujo productivo, y un volcado histórico de la base de datos contiene filas antiguas de entrenamientos, lo que sugiere un uso real en versiones previas del proyecto. En consecuencia, esta ficha la documenta como parte del esquema y señala su situación como recurso previsto o no utilizado en el flujo actual. Esta transparencia es preferible a ocultar la existencia de la tabla o a atribuirle un uso que el código no realiza, porque permite al lector y al evaluador comprender la evolución del diseño y las razones por las que se optó por la persistencia por ficheros para el laboratorio.

Los valores de los campos de dominio se validan en la capa de servicios y no mediante restricciones CHECK del esquema. El estado de la cola admite `queued`, `running`, `completed`, `failed` y `cancelled`; el tipo de trabajo admite `diagnosis`, `training` y `external_validation`; y la etiqueta de predicción admite `PNEUMONIA` y `NORMAL`. Esta decisión centraliza la lógica de validación en la aplicación y evita acoplar el esquema a la evolución de esos valores, de modo que añadir un nuevo tipo de trabajo o un nuevo estado no exige modificar la base de datos. Como contrapartida, el esquema no impide almacenar un valor no permitido si una consulta mal formada lo intentara; esa protección depende por completo de la corrección de la capa de servicios.

La consistencia del esquema se apoya en tres niveles que se complementan entre sí. En el primer nivel, MySQL aplica las claves primarias, las restricciones de obligatoriedad, la unicidad y las claves ajenas, garantizando la integridad estructural de los datos. En el segundo nivel, los routers y los servicios aplican la propiedad, los roles, los estados válidos y las comprobaciones de entrada, protegiendo el acceso a los datos. En el tercer nivel, el worker coordina la relación temporal entre un trabajo encolado, sus artefactos y su resultado, aunque la recuperación tras un reinicio y la limpieza de artefactos parciales requieren comprobaciones adicionales. Ninguno de estos niveles puede sustituir por completo a los demás: una clave ajena no decide si un usuario puede consultar un registro ajeno, y una comprobación de propiedad no corrige una fila sin usuario válido.

El esquema actual prioriza la trazabilidad directa con el código y la facilidad de puesta en marcha. Utiliza identificadores enteros, columnas JSON para parámetros variables y rutas de ficheros para los artefactos. Esta solución es suficiente para el alcance del TFG, pero tiene límites conocidos que conviene enunciar con precisión: no incorpora índices específicos para todas las consultas, no versiona automáticamente el esquema, no normaliza las métricas MLOps y no ofrece una entidad física que vincule cada fila de `job_queue` con una consulta o sesión concreta. Si el volumen de datos creciera, la evolución debería comenzar por los índices de las consultas frecuentes, una estrategia de migraciones versionadas y una política de retención para trabajos y tokens, conservando en todo caso la compatibilidad con los routers, el worker, las pruebas de integración y los documentos de diseño.

Con todo ello, el esquema queda alineado con el estado real del código. MySQL sostiene las entidades y los estados relacionales que necesitan consultas y relaciones; el sistema de ficheros conserva los artefactos grandes o variables; y la aplicación mantiene las reglas que no pueden expresarse solo con claves y tipos SQL. Esta combinación proporciona una persistencia suficiente para la implementación actual y deja identificados los puntos que deberán evolucionar si la plataforma requiere migraciones versionadas, mayor auditoría o una infraestructura distribuida. El apartado siguiente describe cómo se accede a esta capa de persistencia desde los componentes del sistema.

19.2. Acceso a la capa de persistencia

El esquema descrito en el apartado 19.1 define dónde y cómo se almacenan los datos, pero no basta con ello: es necesario fijar también la forma en que los componentes del sistema leen y escriben esa información, porque de esa forma dependen tanto la corrección del comportamiento como la capacidad de evolucionar el sistema sin romper lo existente. En vitalXAI, el acceso a la capa de persistencia no pasa por una capa intermedia de mapeo objeto-relacional ni por un patrón de repositorios que abstraiga las consultas en clases independientes. La aplicación trabaja directamente con SQL parametrizado sobre conexiones obtenidas del pool de MySQLConnectionPool, que el módulo database.py construye de forma diferida y entrega a través de la función `get_db_connection()`. Esta elección responde a la simplicidad y a la trazabilidad: el código que consulta la base de datos es explícito, y la relación entre una consulta y la tabla a la que accede es inmediata.

La configuración del pool, su ciclo de vida y el proceso de inicialización del esquema se describen con detalle en el capítulo 18 dentro del subsistema SSOP-001. Este apartado se centra, por tanto, en dos aspectos complementarios: cómo se distribuye el acceso a las tablas entre los componentes del sistema, y qué consecuencias tiene esa distribución para la concurrencia, el rendimiento y la integridad de los datos. Ambos aspectos son necesarios para comprender el esquema no solo como una estructura estática, sino como un recurso que la aplicación utiliza de forma continuada.

Esta forma de trabajo tiene dos consecuencias directas. La primera es que el acceso a la base de datos queda explícito: cada componente obtiene una conexión, ejecuta sus consultas y la cierra, sin una capa de repositorios que oculte dónde se realiza cada operación. La segunda es que la correspondencia entre el esquema y el código es fácil de localizar: cada tabla tiene componentes concretos que la leen o la escriben. La tabla 57 recoge esa correspondencia.

Tabla de persistencia	Operaciones principales	Componentes de acceso
<i>users</i>	Insertar cuenta, consultar credenciales y perfil, consultar rol.	routers/auth.py, routers/admin.py, routers/history.py, routers/trainer.py.
<i>consultations</i>	Insertar resultado de diagnóstico, consultar historial, renombrar y eliminar.	services/queue_worker.py, routers/history.py, routers/admin.py.
<i>job_queue</i>	Encolar trabajo, reclamarlo, actualizar su estado, cancelarlo.	routers/inference.py, routers/trainer.py, services/queue_worker.py, routers/queue.py.
<i>refresh_tokens</i>	Crear token, verificarlo, revocarlo.	services/auth_service.py, routers/auth.py.
<i>training_jobs</i>	Tabla prevista; sin acceso operativo en la implementación actual.	Ninguno (persistencia real por ficheros en training_results).

Tabla x - Distribución del acceso a las tablas del esquema

La tabla anterior permite verificar que cada tabla tiene un responsable claro en la capa de aplicación, y que la lógica de negocio que rodea a cada entidad reside en los componentes que la utilizan. Por ejemplo, la escritura de consultations es competencia del worker de la cola, que la realiza al finalizar un diagnóstico, mientras que la lectura y la gestión del historial corresponden al router de historial y, en el ámbito administrativo, al router de administración. La cola de trabajos concentra la mayor diversidad de operaciones, porque es el punto de encuentro entre las peticiones que encolan tareas y el worker que las ejecuta. Esta separación evita que la responsabilidad sobre una misma tabla se disperse sin criterio y facilita localizar dónde se introduce o se modifica cada dato durante la depuración y el mantenimiento.

Todas las operaciones de acceso siguen un ciclo de vida similar. En una lectura, el componente obtiene una conexión del pool, crea un cursor normal o de diccionario según el formato de resultado que necesite, ejecuta una consulta parametrizada, procesa el resultado para transformarlo en la estructura requerida y cierra la conexión. En una escritura, la secuencia añade la confirmación explícita mediante `commit()` antes de cerrar. El uso de parámetros evita construir sentencias concatenando datos recibidos del usuario, una práctica recomendada para reducir el riesgo de inyección SQL (OWASP, 2024) y que debe mantenerse en todas las consultas de la aplicación.

Esta uniformidad tiene una implicación práctica sobre la concurrencia. Cada petición solicita una conexión del pool durante el tiempo necesario para su operación y la devuelve al finalizar, lo que limita el número de conexiones simultáneas al tamaño configurado en `DB_POOL_SIZE`. Las operaciones que combinan acceso a la base de datos con procesamiento costoso, como la generación de artefactos, la ejecución de un modelo o la espera de servicios externos, deben liberar la conexión antes de completar el cálculo. El worker respeta este principio: reclama un trabajo, ejecuta el procesamiento pesado fuera de la consulta y vuelve a la base de datos para actualizar el estado o persistir el resultado.

El aislamiento de datos entre usuarios se aplica de forma distinta según el medio de persistencia. Las consultas del historial y los trabajos de la cola filtran por el `user_id` obtenido de la sesión, de modo que un usuario no puede leer filas de otro aunque técnicamente la tabla las contenga. Las operaciones administrativas comprueban previamente el rol del actor. En cambio, las sesiones y los resultados del laboratorio se conservan en `training_results`; su configuración incluye `user_id`, pero las carpetas no tienen una columna ni una clave ajena que las vincule con `users`. El esquema físico no ofrece, por tanto, una garantía equivalente a la de las tablas relacionales. Esta es una excepción relevante a RF-005 y a OBJ-008 que debe valorarse en una evolución posterior mediante una asociación persistente entre la sesión y la cuenta.

En cuanto al rendimiento, el esquema se apoya en los índices asociados a las claves primarias, las restricciones de unicidad y las claves ajenas que crea MySQL/InnoDB (Oracle, 2024). Estos índices ayudan a las consultas habituales, que en su mayoría filtran por el identificador propio de cada tabla y por `user_id`, pero no se ha documentado una medición mediante EXPLAIN. La cola de trabajos accede por estado, tipo y fecha de encolado; si el volumen creciera, deberían valorarse índices adicionales sobre esas columnas y sobre las fechas del historial. La carga de los modelos de inferencia no es una operación de base de datos y no interviene en este apartado.

Conviene precisar, además, la relación entre el acceso a la base de datos y el acceso a los artefactos del sistema de ficheros. Cuando una consulta se completa, el worker escribe los ficheros en el sistema de ficheros y, a continuación, inserta en consultations las rutas que los referencian. La operación inversa, la lectura, recupera primero la fila y después entrega los ficheros desde las rutas almacenadas. Esta secuencia introduce una coordinación que debe ser correcta: si el fichero se genera pero la inserción falla, la fila no existe y el resultado no aparece en el historial; si la inserción se completa pero el fichero no llegó a escribirse, la fila quedaría referenciando un artefacto inexistente. El worker gestiona esta coordinación registrando el estado del trabajo y, en caso de error, dejando constancia en job_queue y evitando presentar como completada una consulta cuyos artefactos no están disponibles.

El acceso a la capa de persistencia queda, así, alineado con la naturaleza híbrida del almacenamiento descrita en el apartado 19.1. Las tablas conservan los datos estructurados que necesitan consultas y relaciones, y se accede a ellas mediante conexiones del pool y consultas parametrizadas; los artefactos grandes se escriben y se leen directamente en el sistema de ficheros, y la base de datos únicamente guarda las rutas que los referencian. Esta combinación mantiene la correspondencia entre el esquema físico y el código que lo utiliza, y deja identificadas las operaciones y los componentes responsables de cada tabla, de modo que la evolución del sistema pueda partir de un conocimiento preciso de dónde reside cada dato y de quién lo gestiona.

Capítulo 20. Diseño de los casos de uso

El capítulo 12 define el comportamiento funcional de los casos de uso y el capítulo 14 muestra sus secuencias desde la perspectiva del análisis. Este capítulo no repite esas especificaciones. Su función es documentar cómo se materializan en la implementación: qué componentes colaboran, qué validaciones se aplican, cómo se persisten los datos y cómo se resuelven las operaciones asíncronas (Jacobson, Booch, & Rumbaugh, 1999; Larman, 2004).

La organización sigue los seis subsistemas de diseño del capítulo 17. Para cada subsistema se resume la correspondencia con los casos de uso, se indican las decisiones técnicas propias del diseño y se incluye un diagrama representativo de interacción entre componentes. Los flujos completos y sus variantes permanecen en el capítulo 12; las secuencias de análisis, en el capítulo 14; la arquitectura general, en el capítulo 17; y los mecanismos compartidos, en el capítulo 18.

20.1. SD-001: Acceso, identidad y gestión de sesiones

SD-001 materializa SS-001 y concentra la autenticación, la gestión de cuentas y el ciclo de vida de las sesiones. La especificación funcional de CU-001 a CU-004 se mantiene en el capítulo 12. En el diseño, estos casos se implementan mediante `routers/auth.py`, `services/auth_service.py`, la tabla `users`, la tabla `refresh_tokens` y los mecanismos transversales registrados en `main.py`.

Casos de uso	Componentes principales	Decisiones de diseño
CU-001 Registrarse	<code>routers/auth.py</code> , <code>auth_service.py</code> , <code>users</code>	Validación de entradas, hash de contraseña, comprobación de unicidad del nombre de usuario y creación de la cuenta.
CU-002 Iniciar sesión	<code>routers/auth.py</code> , <code>auth_service.py</code> , <code>refresh_tokens</code>	Verificación de credenciales, emisión de tokens, cookies <code>HttpOnly</code> y <code>SameSite=Lax</code> , y límite de peticiones.
CU-003 Cerrar sesión	<code>routers/auth.py</code> , <code>auth_service.py</code> , <code>refresh_tokens</code>	Revocación del token de refresco y eliminación de las cookies de sesión.
CU-004 Cambiar idioma	<code>services/lang.py</code> , <code>recursos</code> , JavaScript	Preferencia compartida entre cliente y servidor, con español como idioma de respaldo.

Tabla x - SD-001: Acceso, identidad y gestión de sesiones

El diseño separa la recepción HTTP de la gestión de credenciales. El router valida y traduce la petición, mientras que el servicio concentra el hash, la emisión, la rotación y la revocación de tokens. La autorización administrativa se resuelve en SD-005 mediante el rol obtenido desde la identidad establecida por SD-001.

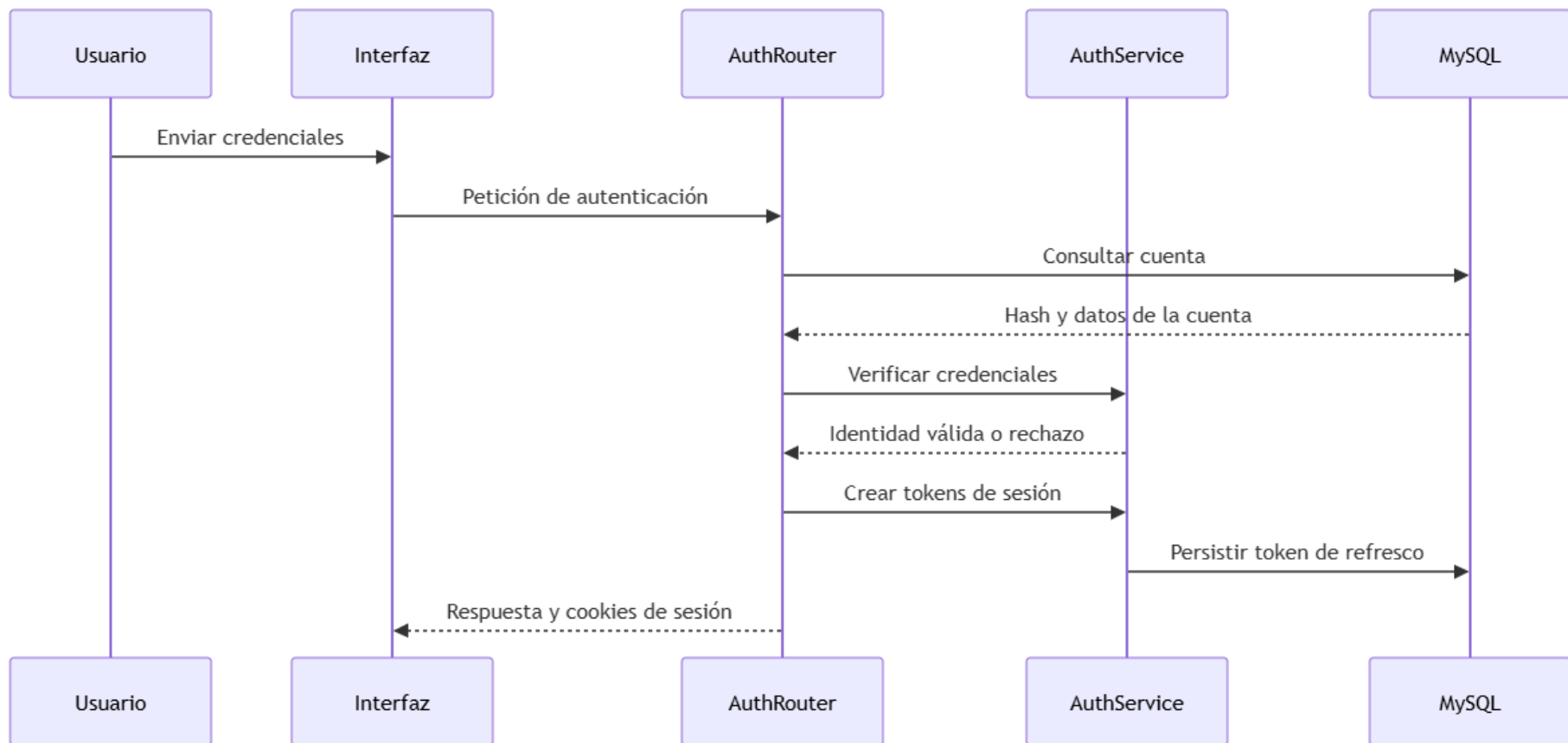


Ilustración x - SD-001: Acceso, identidad y gestión de sesiones

20.2. SD-002: Diagnóstico asistido y resultados

SD-002 materializa SS-002 y cubre CU-005 a CU-010 y CU-037. El capítulo 12 describe el flujo clínico; aquí solo se fijan las decisiones técnicas que lo materializan.

Casos de uso	Componentes principales	Decisiones de diseño
CU-005 a CU-007	routers/inference.py, interfaz de diagnóstico	Validación de sesión, tipo MIME y tamaño de imagen, y selección de arquitectura. La validación del contenido real de la imagen permanece pendiente.
CU-008	routers/inference.py, job_queue, queue_worker.py	La petición valida y encola el trabajo sin ejecutar la inferencia dentro del ciclo HTTP.
CU-009 y CU-010	ml_engine.py, xai_generator.py, consultations	El resultado y los artefactos se generan en segundo plano y se recuperan desde la consulta persistida.
CU-037	pdf_generator.py, consultations	El informe se genera a partir de los datos y artefactos de la consulta en curso, antes de insertar su fila persistida.

Tabla x - SD-002: Diagnóstico asistido y resultados

El router conserva la imagen y crea un trabajo con la arquitectura, la ruta del fichero y el idioma. El worker ejecuta la inferencia, genera los mapas de explicabilidad y el informe, y registra las rutas en consultations. La interfaz consulta posteriormente el estado y muestra el resultado cuando el trabajo termina.

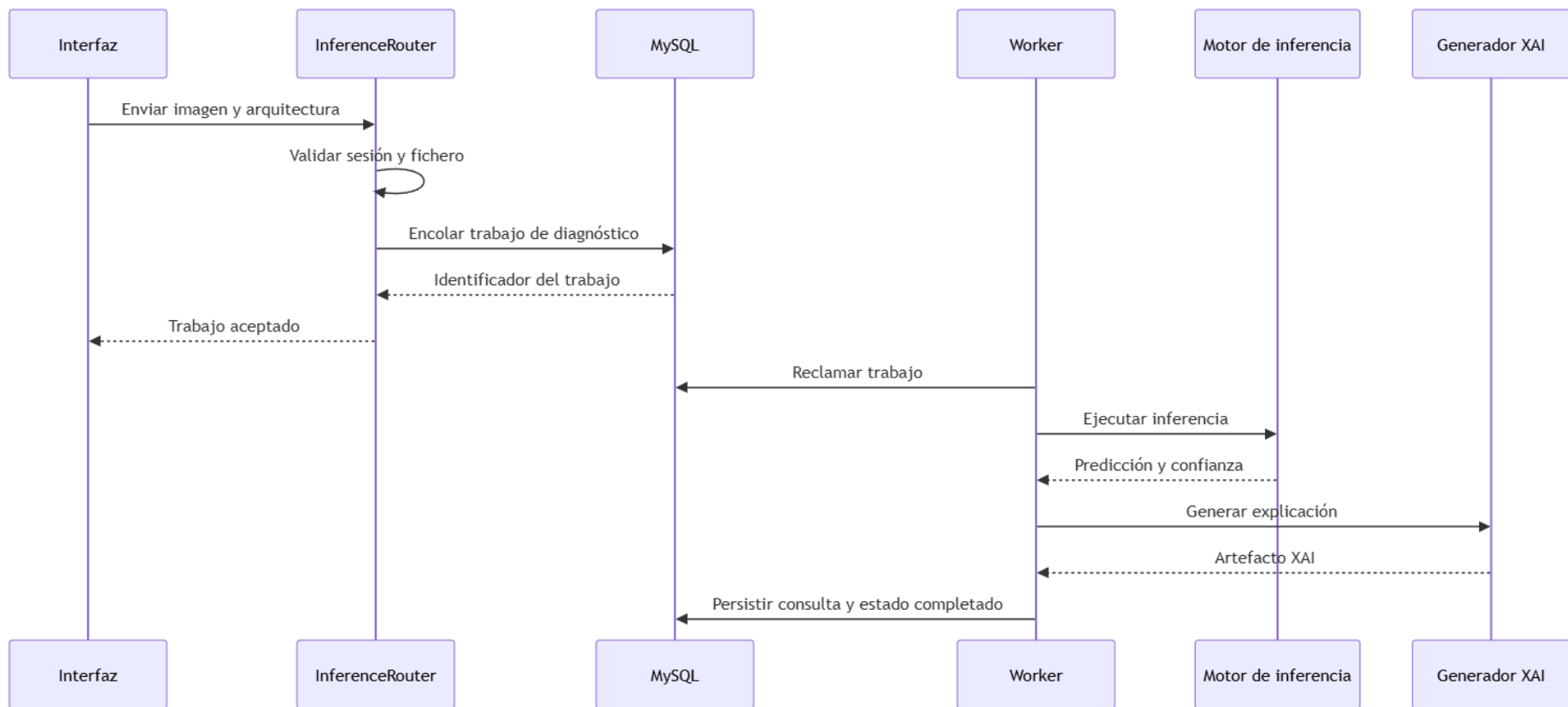


Ilustración x - SD-002: Diagnóstico asistido y resultados

20.3. SD-003: Historial y gestión de consultas

SD-003 materializa SS-003 y cubre CU-011 a CU-014. El comportamiento de consulta, detalle, renombrado y eliminación se especifica en el capítulo 12. El diseño concentra estas operaciones en routers/history.py y en la tabla consultations.

Casos de uso	Componentes principales	Decisiones de diseño
CU-011 y CU-012	routers/history.py, consultations	Filtrado por user_id y recuperación de los artefactos referenciados.
CU-013	routers/history.py, consultations	Actualización únicamente del nombre visible de la consulta propia.
CU-014	routers/history.py, consultations, sistema de ficheros	Comprobación de propiedad y eliminación de la fila de consulta. Los artefactos asociados pueden permanecer en el sistema de ficheros.

Tabla x - SD-003: Historial y gestión de consultas

El historial no vuelve a ejecutar el modelo. Recupera los metadatos y las rutas de los ficheros ya generados. La función de comprobación de propiedad se aplica antes de consultar, modificar o eliminar una consulta, con la excepción administrativa definida en el análisis. La eliminación actual afecta a la fila de consultations, pero no borra explícitamente la imagen, el mapa XAI ni el informe del sistema de ficheros. La descarga posterior utiliza las rutas estáticas montadas por la aplicación, que no aplican una comprobación de propiedad equivalente a la del router.

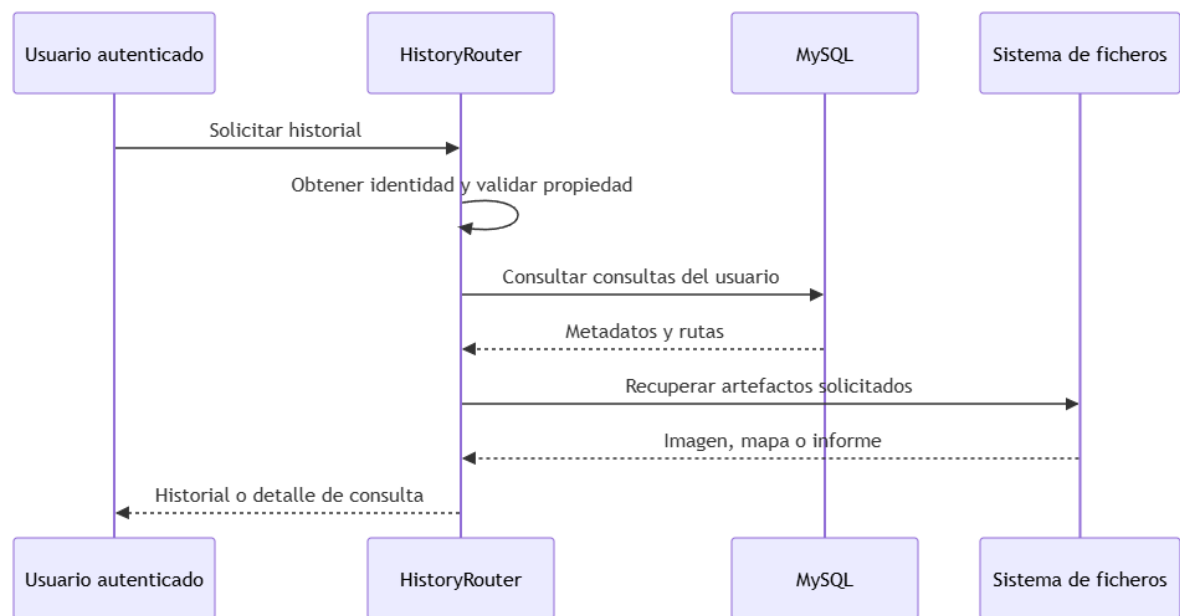


Ilustración x - SD-003: Historial y gestión de consultas

20.4. SD-004: Laboratorio de experimentación MLOps

SD-004 materializa SS-004 y cubre CU-015 a CU-030 y CU-039. El capítulo 12 define las interacciones del laboratorio y el capítulo 17 describe sus servicios. El diseño se limita a concretar la colaboración entre esos servicios y la persistencia híbrida.

Casos de uso	Componentes principales	Decisiones de diseño
CU-015 a CU-017	routers/trainer.py, chatbot_service.py, selección de datasets	La configuración se valida antes de crear una sesión. La restricción de la ruta al directorio permitido está prevista, pero pendiente de implementación.
CU-018 y CU-039	trainer.py, job_queue, queue_worker.py	El entrenamiento se encola. La limitación de trabajos simultáneos y pendientes corresponde a un comportamiento previsto, todavía no implementado.
CU-019 a CU-024	mlops_engine.py, training_results	Las sesiones, resultados y comparativas se recuperan desde los artefactos disponibles.
CU-025	routers/trainer.py, mlops_engine.py, training_results	El análisis XAI se ejecuta directamente mediante el servicio del laboratorio.
CU-026 y CU-027	routers/trainer.py, mlops_engine.py, queue_worker.py, training_results	La validación externa se encola y sus resultados se recuperan desde los artefactos disponibles.
CU-028 a CU-030	pdf_generator_mlops.py, mlops_engine.py, training_results	Los informes, renombrados y eliminaciones operan sobre la sesión y sus artefactos.

Tabla x - SD-004: Laboratorio de experimentación MLOps

La configuración conversacional y la ejecución del entrenamiento son responsabilidades separadas. El asistente devuelve una configuración estructurada, el router valida sus parámetros y el worker ejecuta el trabajo. Los resultados del laboratorio se conservan en training_results; la configuración de cada sesión incluye user_id, pero las carpetas no tienen una relación relacional con users, tal como se documenta en los capítulos 14, 17, 18 y 19.

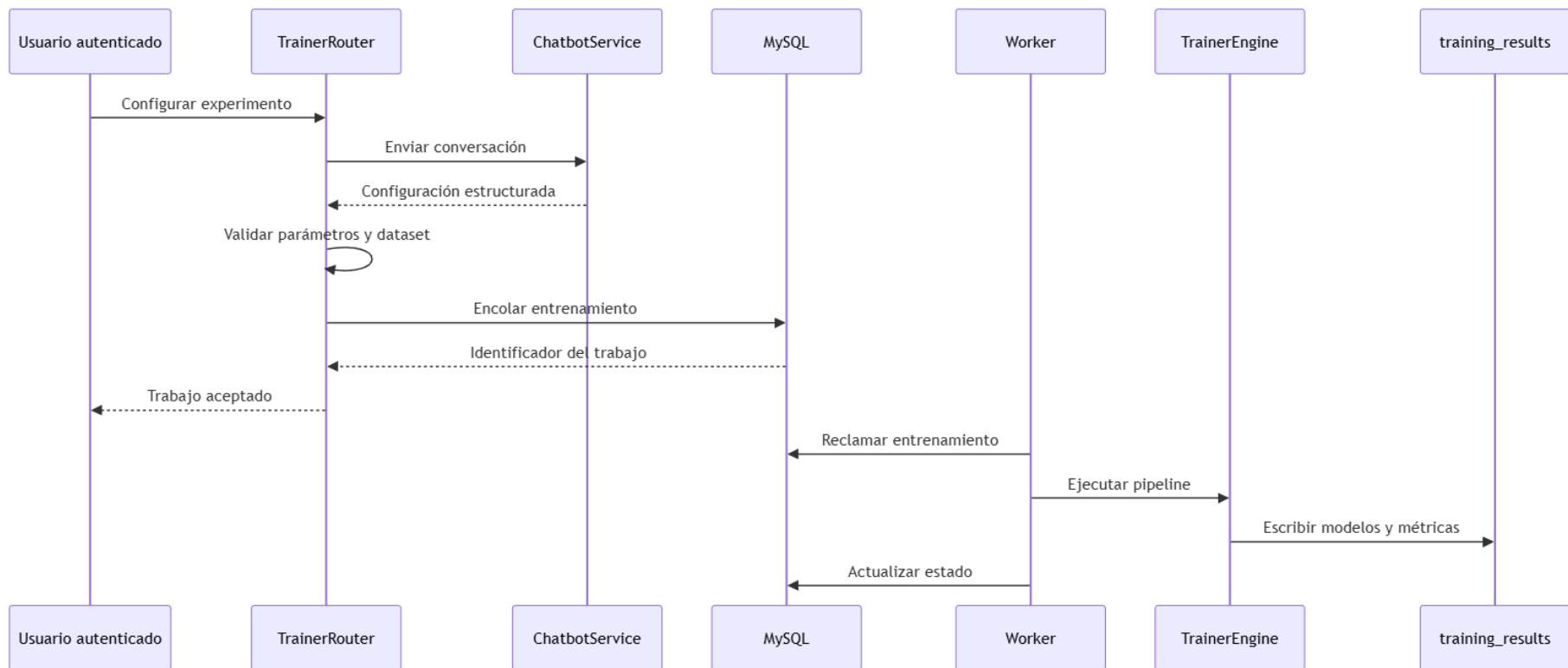


Ilustración x - SD-004: Laboratorio de experimentación MLOps

20.5. SD-005: Supervisión y administración

SD-005 materializa SS-005 y cubre CU-031 a CU-033 y CU-038. Las operaciones de supervisión se restringen al rol de administrador y se implementan principalmente en routers/admin.py; la gestión de cuentas de CU-038 permanece prevista, pero no está implementada.

Casos de uso	Componentes principales	Decisiones de diseño
CU-031	routers/admin.py, users, consultations	Comprobación de rol antes de consultar el listado global.
CU-032 y CU-033	routers/admin.py, consultations, training_results	Consulta supervisada de la actividad de una cuenta sin modificar sus resultados.
CU-038	routers/admin.py, users	Gestión prevista de cuentas, con las limitaciones de persistencia documentadas; no está implementada en el prototipo actual.

Tabla x - SD-005: Supervisión y administración

La función `_require_admin()` centraliza la comprobación del rol. SD-005 reutiliza la identidad de SD-001 y las estructuras de persistencia existentes, pero no sustituye la auditoría administrativa exigida por RNF-006, que permanece pendiente.

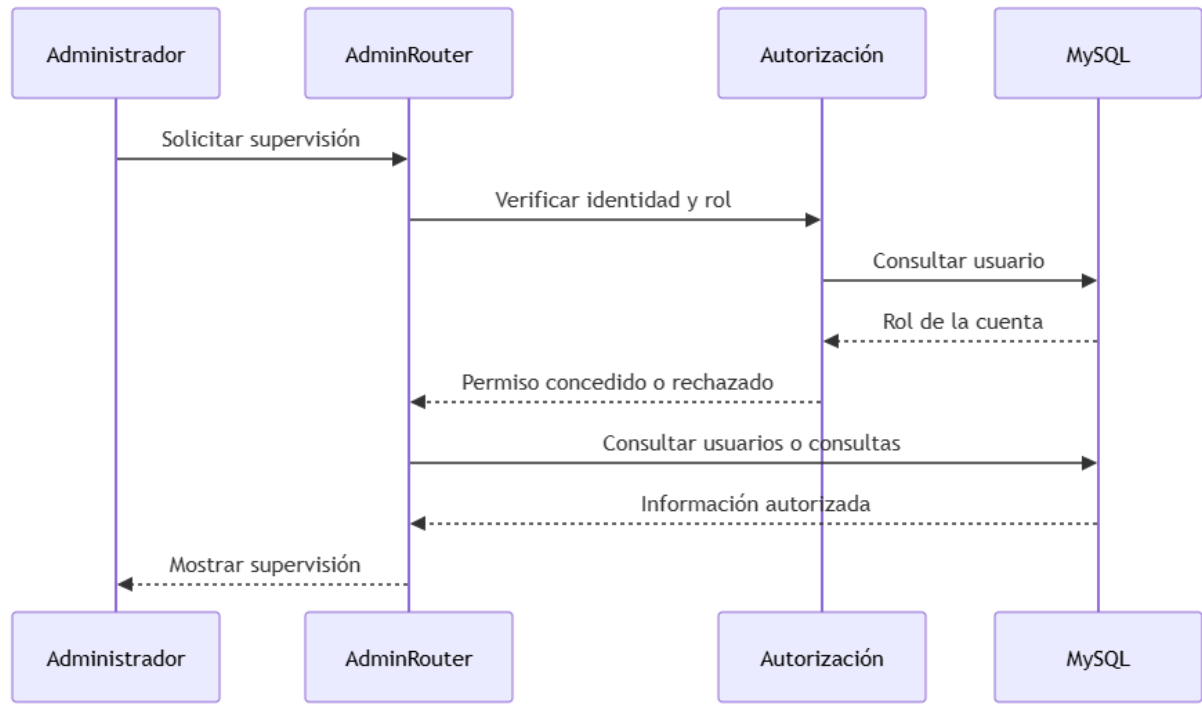


Ilustración x - SD-005: Supervisión y administración

20.6. SD-006: Cola de trabajos y capacidades transversales

SD-006 materializa SS-006 y cubre CU-034 a CU-036. La cola compartida coordina diagnósticos, entrenamientos y validaciones externas. La consulta de trabajos, su cancelación y las preferencias de interfaz se mantienen separadas de los subsistemas que producen los trabajos.

Casos de uso	Componentes principales	Decisiones de diseño
CU-034	routers/queue.py, job_queue	Consulta filtrada por usuario y cálculo de posición de los trabajos pendientes.
CU-035	routers/queue.py, job_queue	Cancelación condicional de trabajos que todavía están pendientes.
CU-036	services/lang.py, JavaScript y recursos estáticos	Cambio de tema e idioma sin recargar ni perder el estado de navegación.

Tabla x - SD-006: Cola de trabajos y capacidades transversales

queue_worker.py reclama los trabajos mediante una actualización condicional, ejecuta el flujo correspondiente y actualiza su estado. La cola no mantiene claves ajenas hacia los resultados que procesa: la relación con consultas y sesiones se conserva mediante el tipo y el contenido del trabajo. Esta decisión se describe con más detalle en los capítulos 17 y 19.

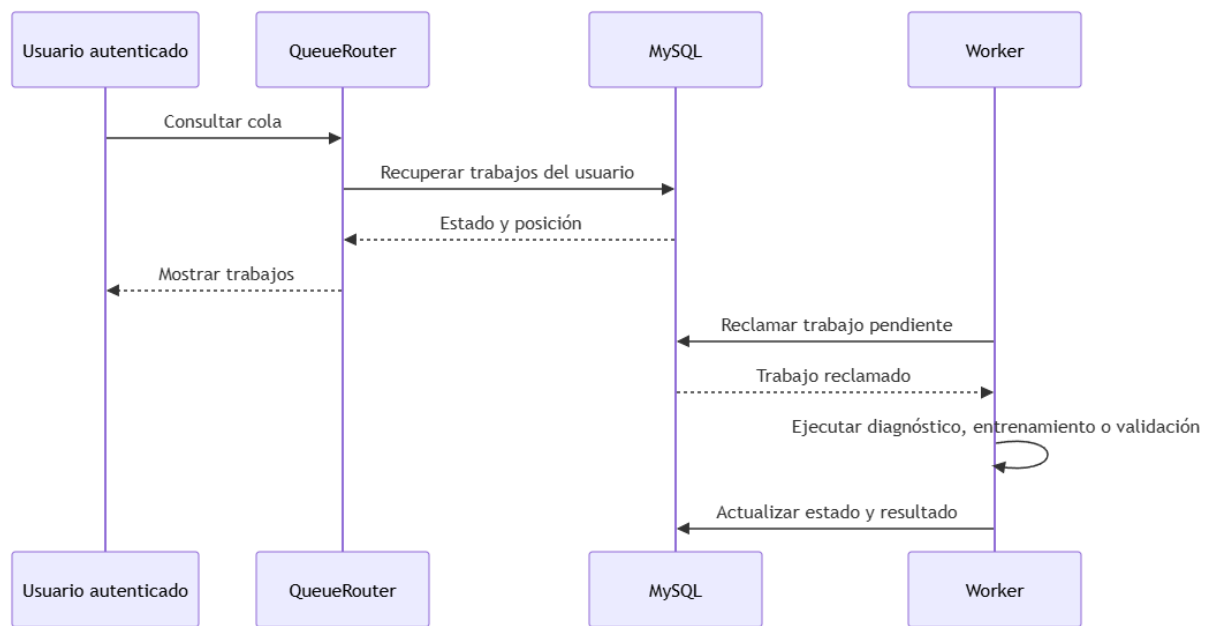


Ilustración x - SD-006: Cola de trabajos y capacidades transversales

20.7. Cierre del diseño de los casos de uso

El capítulo 20 concreta la transición entre el comportamiento especificado y los componentes que lo implementan. El capítulo 12 sigue siendo la fuente de verdad de los casos de uso y el capítulo 14 la del comportamiento de análisis. Las tablas de este capítulo permiten localizar la decisión técnica asociada a cada grupo de casos sin duplicar sus flujos completos. Los capítulos 17, 18 y 23 completan la descripción de la arquitectura, el soporte y la construcción real.

Capítulo 21. Estructura de clases del diseño

Este capítulo traduce la arquitectura de vitalXAI a una vista estática orientada a la implementación. Para cada subsistema se indica qué componentes se han modelado, qué información manejan, qué operaciones ofrecen y con qué otros componentes colaboran. La vista complementa los diagramas de interacción del capítulo 20: aquellos describen el orden temporal de los mensajes, mientras que este capítulo fija la organización de las responsabilidades, los datos y las dependencias. El criterio sigue la separación habitual entre estructura y comportamiento en el modelado UML (Larman, 2004), pero las clases y operaciones descritas a continuación se han derivado de los módulos reales del proyecto.

El capítulo se organiza siguiendo la misma estructura de subsistemas de diseño del capítulo 17: cada apartado corresponde a un subsistema (SD-001 a SD-006) y agrupa las clases que materializan su responsabilidad. Para cada subsistema se presentan dos elementos: el diagrama de clases, que representa gráficamente las clases del subsistema con sus atributos, sus operaciones y sus relaciones, incluidas las clases abstractas, las herencias y las asociaciones cuando existen, y la especificación de las clases, que detalla cada clase mediante la ficha formal CL-NNNN con sus atributos, sus operaciones y sus comentarios. Para las clases más complejas se añade, cuando procede, un diagrama de transición de estados que permite comprender la funcionalidad soportada por dicha clase.

Las clases de diseño de vitalXAI se corresponden con los componentes reales del sistema descritos en el capítulo 17. Aunque la implementación está orientada a módulos funcionales, routers, servicios y worker, cada módulo se modela aquí como una clase de diseño que agrupa sus responsabilidades. La trazabilidad con el diseño de casos de uso se mantiene en cada ficha: las operaciones de cada clase materializan los mensajes que los diagramas de secuencia del capítulo 20 asignaron al componente correspondiente, y las asociaciones del modelo de clases reflejan las dependencias de colaboración entre routers y servicios.

21.1. *Subsistema SD-001: Acceso, identidad y gestión de sesiones*

El subsistema SD-001 materializa las responsabilidades de acceso, identidad y gestión de sesiones descritas en el capítulo 17 y agrupa dos clases de diseño propias. La clase AuthRouter se corresponde con el router routers/auth.py y actúa como fachada HTTP del subsistema: sirve las páginas de inicio de sesión y registro, procesa los formularios y gestiona las cookies de sesión. La clase AuthService se corresponde con el servicio services/auth_service.py y concentra la lógica criptográfica y el ciclo de vida de las credenciales: el hash de las contraseñas, la firma y verificación de los tokens de acceso, la gestión de los tokens de refresco y la rotación con detección de robo. El subsistema depende además del servicio de idioma LangService, que pertenece a SD-006 y proporciona los mensajes localizados que el router emplea en sus respuestas.

21.1.1. Diagrama de clases del subsistema

El modelo de clases de SD-001 se representa en la figura 83. El diagrama muestra las dos clases propias del subsistema, sus atributos y operaciones, y la dependencia hacia el servicio de idioma, que se representa como una clase de otro subsistema. La clase AuthRouter presenta la expresión regular de validación del correo electrónico, el motor de plantillas y las operaciones que sirven las páginas y procesan los formularios; la clase AuthService presenta los parámetros de configuración de las credenciales y las operaciones criptográficas y de gestión de tokens.



Ilustración x - Diagrama de clases del subsistema SD-001

El diagrama identifica la asociación principal del subsistema: la dependencia de AuthRouter hacia AuthService, que materializa la delegación de toda la lógica criptográfica y de tokens del router al servicio. Esta separación de responsabilidades, ya declarada en el capítulo 17, mantiene al router como una fachada ligera que orquesta las peticiones HTTP y delega en el servicio las operaciones que requieren configuración sensible. La dependencia punteada hacia LangService refleja que el router consume los mensajes localizados del servicio de idioma sin que este forme parte del subsistema. El modelo no presenta herencias ni clases abstractas: las dos clases son clases concretas de diseño, y la colaboración se expresa exclusivamente mediante asociaciones de dependencia.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases se corresponden con los mensajes de los diagramas de secuencia del capítulo 20. Las operaciones login(), process_register(), logout() y token_refresh() de AuthRouter materializan los puntos de entrada de los CU-001, CU-002, CU-003 y de la renovación de la sesión. Las operaciones de AuthService, como hash_password(), verify_password(), create_access_token(), create_refresh_token(), verify_refresh_token(), revoke_refresh_token() y rotate_refresh_token(), implementan la gestión de credenciales.

21.1.2. Especificación de las clases

A continuación se definen las dos clases propias de SD-001.

CL-0001	AuthRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/auth.py. Actúa como fachada HTTP del subsistema de acceso: sirve las páginas de inicio de sesión, registro, panel de diagnóstico y laboratorio de entrenamiento; procesa los formularios de registro e inicio de sesión; gestiona el cierre de sesión y la renovación de credenciales, y establece las cookies de sesión en las respuestas. Valida los datos de entrada del registro y delega toda la lógica criptográfica en AuthService.		
Atributos	Nombre	Tipo	Descripción
	_EMAIL_RE	re.Pattern	Expresión regular privada que valida el formato del nombre de usuario como dirección de correo electrónico en el registro.
	templates	Jinja2Templates	Motor de plantillas que compone las páginas HTML servidas por el router (inicio de sesión, registro, panel y laboratorio).
Operaciones	Nombre	Descripción	
	login_page(request, error)	Sirve la página de inicio de sesión, mostrando el error genérico cuando la autenticación previa falló.	
	login(request, username, password)	Procesa el formulario de inicio de sesión: verifica las credenciales mediante AuthService, establece las cookies de sesión y redirige al panel, o redirige al inicio con un mensaje genérico si la verificación falla. Aplicada la limitación de cinco peticiones por minuto.	
	register_page(request)	Sirve el formulario de registro de nuevas cuentas.	
	process_register(username, password, first_name, last_name, role)	Procesa el formulario de registro: valida el formato de los campos, comprueba la unicidad del usuario, solicita el hash de la contraseña a AuthService, crea la cuenta y establece las cookies de sesión.	

	logout(request)	Cierra la sesión: revoca el token de refresco mediante AuthService, elimina las cookies y redirige a la página de inicio de sesión.
	token_refresh(refresh_token)	Renueva la sesión: rota el token de refresco mediante AuthService y emite un nuevo token de acceso, actualizando las cookies.
	dashboard(request)	Sirve el panel de diagnóstico del usuario autenticado, redirigiendo a la página de inicio si no existe sesión válida.
	training_lab(request)	Sirve el laboratorio de entrenamiento del usuario autenticado, con la misma comprobación de sesión que el panel.
	_validate_register_inputs(username, password, first_name, last_name)	Operación privada que valida el formato del correo electrónico, la longitud mínima de la contraseña y la presencia del nombre y los apellidos, devolviendo el campo inválido o None.
Comentarios	La clase no mantiene estado de sesión propio: las credenciales se conservan en las cookies HttpOnly y SameSite=Lax, y la identidad se resuelve mediante AuthService. La operación token_refresh() cubre la renovación de credenciales descrita en el capítulo 17 y materializada en el apartado 20.1. El rol de administrador no se valida en esta clase, sino en las operaciones administrativas de SD-005. El registro actual recibe también un campo role desde el cliente, por lo que la asignación inicial del rol requiere una corrección en el servidor.	

Tabla x - Especificación de clase CL-0001

CL-0002	AuthService		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/auth_service.py. Concentra la lógica criptográfica y el ciclo de vida de las credenciales del subsistema: genera y verifica el hash de las contraseñas con bcrypt, firma y valida los tokens de acceso JWT, gestiona los tokens de refresco en la base de datos mediante su hash SHA-256 y aplica la rotación con detección de uso de credenciales revocadas.		
Atributos	Nombre	Tipo	Descripción
	JWT_SECRET_KEY	str	Clave secreta de firma de los tokens de acceso JWT, obtenida del entorno; se emite una advertencia cuando se utiliza la clave de desarrollo por defecto.
	JWT_ACCESS_EXPIRE_MINUTES	int	Duración en minutos del token de acceso, configurada mediante variable de entorno.
	JWT_REFRESH_EXPIRE_DAYS	int	Duración en días del token de refresco, configurada mediante variable de entorno.
	REFRESH_ROTATION_GRACE_SECONDS	int	Periodo de gracia en segundos que tolera las renovaciones concurrentes del token de refresco sin interpretarlas como un robo.
Operaciones	Nombre	Descripción	
	hash_password(password)	Genera el hash bcrypt de la contraseña con un salt aleatorio en cada operación.	
	verify_password(password, password_hash)	Compara la contraseña introducida con el hash almacenado mediante bcrypt.checkpw.	
	create_access_token(user_id)	Firma un token de acceso JWT con el identificador del usuario y su fecha de expiración, usando la clave secreta del entorno.	
	verify_access_token(token)	Valida el token de acceso y devuelve el identificador del usuario, o None si el token es inválido o ha expirado.	
	create_refresh_token(user_id)	Genera un token de refresco aleatorio y persiste solo su hash SHA-256 en la tabla refresh_tokens, con la fecha de expiración y el indicador de revocación.	
	verify_refresh_token(raw_token)	Verifica el token de refresco: acepta el token activo, tolera el token revocado dentro del periodo de gracia y revoca todos los tokens del usuario si se detecta el uso de una credencial revocada fuera de él.	
	revoke_refresh_token(raw_token)	Marca como revocado el token de refresco en la base de datos.	
	rotate_refresh_token(old_raw_token)	Rota el token de refresco: verifica el token actual, revoca el anterior y emite uno nuevo; devuelve None si la rotación no es válida.	

	get_user_id_from_token(token)	Extrae el identificador del usuario de un token de acceso, devolviendo None si el token falta o no es válido; es el mecanismo común de identificación que consumen el resto de los subsistemas.
Comentarios	La clase no conserva la contraseña original en ningún atributo: una vez completado el hash, la credencial no se transmite a ningún componente de persistencia. La configuración sensible permanece fuera del código y se obtiene del entorno en tiempo de ejecución. Las operaciones de verificación y rotación de tokens de refresco implementan la política de seguridad de sesiones descrita en el capítulo 17 y materializada en los diagramas de secuencia del capítulo 20.	

Tabla x - Especificación de clase CL-0002

21.2. Subsistema SD-002: Diagnóstico asistido y generación de resultados

El subsistema SD-002 materializa el flujo clínico de la plataforma y agrupa cinco clases de diseño propias. La clase `InferenceRouter` se corresponde con el router `routers/inference.py` y actúa como fachada HTTP del diagnóstico: valida la imagen recibida, la conserva en el área de cargas y encola el trabajo de diagnóstico. Las clases `MIEngine`, `XaiGenerator` y `PdfGenerator` se corresponden con los servicios `services/ml_engine.py`, `services/xai_generator.py` y `services/pdf_generator.py`, y concentran respectivamente la predicción, la generación de los mapas de explicabilidad y la construcción del informe PDF. La clase `PDFReport` es la clase concreta del generador de informes y hereda de `FPDF`, la clase de la librería `fpdf`; esta es la única herencia real del subsistema. El procesamiento de los trabajos encolados no pertenece a SD-002: lo ejecuta el worker de la cola de SD-006, que invoca las clases de predicción, explicabilidad e informe del presente subsistema.

21.2.1. Diagrama de clases del subsistema

El modelo de clases de SD-002 se representa en la figura 84. El diagrama muestra las cinco clases propias del subsistema con sus atributos y sus operaciones, la herencia de la clase `PDFReport` sobre la clase `FPDF` de la librería, y las dependencias hacia el servicio de idioma `LangService`, que pertenece a SD-006 y proporciona las etiquetas y los mensajes localizados. Las asociaciones entre las clases del subsistema reflejan la colaboración del flujo de diagnóstico: el generador de explicabilidad depende del motor de predicción para obtener el modelo cargado, y el generador de informes utiliza la clase concreta del documento PDF.

El diagrama identifica la única herencia del subsistema: la clase `PDFReport` especializa la clase `FPDF` de la librería de generación de documentos, redefiniendo la cabecera y el pie de página del informe médico. La clase `FPDF` se representa como una clase externa de la librería, marcada con el estereotipo de librería y con las operaciones básicas de generación que el informe hereda y utiliza, como añadir páginas, configurar la fuente y el color, escribir celdas, insertar imágenes y producir el documento. El resto de las relaciones son asociaciones de dependencia. La dependencia de `XaiGenerator` hacia `MIEngine` refleja que la generación de los mapas de explicabilidad requiere el modelo cargado por el motor; las dependencias hacia `LangService` reflejan la localización de las etiquetas, los títulos y los mensajes; y la dependencia de `PdfGenerator` hacia `PDFReport` materializa la construcción del documento concreto. La clase `InferenceRouter` no presenta dependencias con el resto de las clases del subsistema porque su colaboración se limita al encolado del trabajo: el procesamiento de la predicción, de la explicabilidad y del informe lo orquesta el worker de SD-006, que invoca estas clases desde el ciclo asíncrono.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases materializan los mensajes de los diagramas de secuencia del capítulo 20. La operación `predict()` de `InferenceRouter` implementa el CU-008 y las validaciones del CU-006; las operaciones de `MIEngine`, `XaiGenerator` y `PdfGenerator` materializan los pasos del procesamiento asíncrono descrito en el capítulo 20; y la operación `generate_medical_report()` de `PdfGenerator` materializa la generación del informe del CU-037.

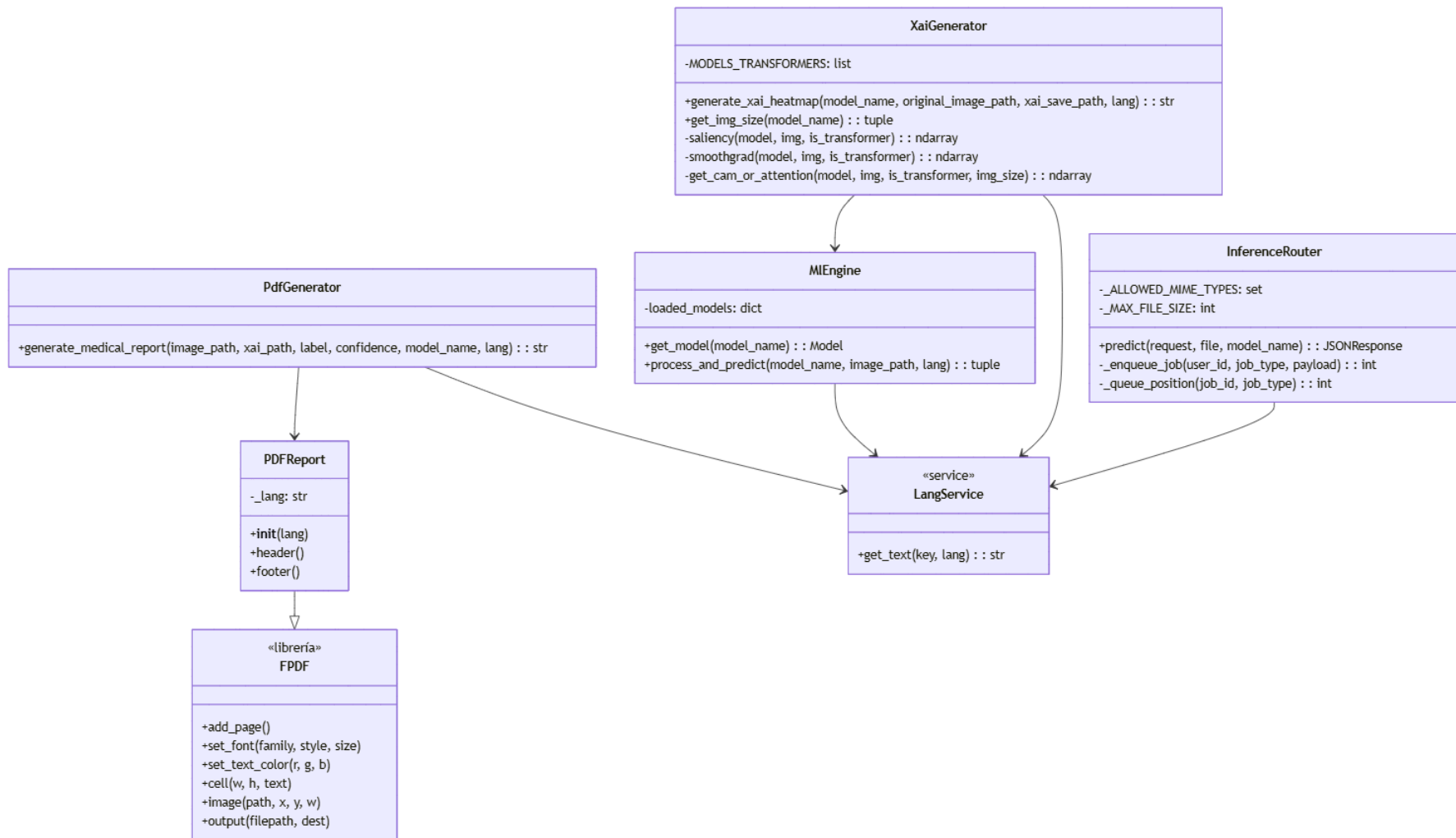


Ilustración x - Diagrama de clases del subsistema SD-002

21.2.2. Especificación de las clases

A continuación se definen las cinco clases propias de SD-002.

CL-0003	InferenceRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/inference.py. Actúa como fachada HTTP de la solicitud de diagnóstico: resuelve la identidad del usuario mediante AuthService, valida el tipo MIME y el tamaño de la imagen, la conserva en el área de cargas y crea el trabajo de diagnóstico en la cola con un payload serializable. No ejecuta la inferencia ni carga modelos.		
Atributos	Nombre	Tipo	Descripción
	_ALLOWED_MIME_TYPES	set	Conjunto de tipos MIME permitidos para la radiografía: image/jpeg, image/jpg e image/png.
	_MAX_FILE_SIZE	int	Tamaño máximo de la imagen en bytes: 10 MB.
Operaciones	Nombre	Descripción	
	predict(request, file, model_name)	Procesa la solicitud de diagnóstico: autentica al usuario (401 sin sesión), valida el tipo MIME y el tamaño de la imagen (400 si no se permiten), guarda el fichero con un nombre basado en la fecha y el nombre recibido, encola el trabajo de tipo diagnosis y responde con el estado queued, el identificador y la posición.	
	_enqueue_job(user_id, job_type, payload)	Operación privada que inserta un trabajo en la tabla job_queue con el tipo y el payload indicados, devolviendo el identificador del trabajo creado.	
	_queue_position(job_id, job_type)	Operación privada que calcula la posición del trabajo en la cola, respetando la prioridad de los diagnósticos frente al resto de tipos.	
Comentarios	La validación del fichero se realiza antes de encolar, de modo que una petición inválida no ocupa una posición en la cola ni consume recursos del worker. El payload del trabajo contiene únicamente el modelo, la ruta de la imagen y el idioma; ni el modelo ni la imagen completa se serializan en la base de datos.		

Tabla x - Especificación de clase CL-0003

CL-0004	MIEngine		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/ml_engine.py. Concentra la inferencia del diagnóstico: carga y reutiliza los modelos de aprendizaje profundo en memoria, prepara la imagen recibida y produce la etiqueta de la predicción y su nivel de confianza. Combina arquitecturas convolucionales con arquitecturas Transformer, una familia de modelos basada en mecanismos de atención (Vaswani & al., 2017; Dosovitskiy & al., 2021), y localiza las etiquetas mediante el servicio de idioma.		
Atributos	Nombre	Tipo	Descripción
	loaded_models	dict	Diccionario global que conserva los modelos cargados en memoria, indexado por nombre de arquitectura, de modo que la primera consulta paga la carga de los pesos y las posteriores reutilizan el modelo.
Operaciones	Nombre	Descripción	
	get_model(model_name)	Carga el modelo de la arquitectura indicada y lo conserva en la caché de modelos; si el modelo ya está cargado, lo devuelve sin volver a leer los pesos.	
	process_and_predict(model_name, image_path, lang)	Preprocesa la imagen según la arquitectura, ejecuta la inferencia y devuelve la etiqueta de la predicción (Neumonía o Normal) y el nivel de confianza, localizados mediante LangService.	
Comentarios	La reutilización de los modelos en memoria implementa el requisito de tiempo de respuesta de la inferencia (RNF-019): la optimización pertenece al motor y no al router. Un modelo no reconocido falla antes de producir un resultado ambiguo, de modo que la validación de la arquitectura queda diferida al motor.		

Tabla x - Especificación de clase CL-0004

CL-0005	XaiGenerator		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/xai_generator.py. Genera la composición visual de explicabilidad adecuada a la arquitectura utilizada: aplica métodos basados en gradientes a las arquitecturas convolucionales y utiliza mapas de atención en las arquitecturas Transformer, y reúne la radiografía original con tres resultados superpuestos. La nomenclatura de estos métodos sigue la literatura de mapas de saliencia y atención (Simonyan, Vedaldi, & Zisserman, 2014; Smilkov & al., 2017; Selvaraju & al., 2017; Chefer, Gur, & Wolf, 2021).		
Atributos	Nombre	Tipo	Descripción
	MODELS_TRANSFORMERS	list	Lista de las arquitecturas Transformer del proyecto, utilizada para bifurcar el método de explicabilidad.
Operaciones	Nombre	Descripción	
	generate_xai_heatmap(model_name, original_image_path, xai_save_path, lang)	Genera la figura de explicabilidad, formada por la imagen original, Saliency, SmoothGrad y la superposición de Grad-CAM o atención, la guarda en la ruta indicada y devuelve dicha ruta. Estos nombres designan técnicas descritas en la bibliografía especializada (Simonyan, Vedaldi, & Zisserman, 2014; Smilkov & al., 2017; Selvaraju & al., 2017; Chefer, Gur, & Wolf, 2021).	
	get_img_size(model_name)	Devuelve el tamaño de entrada de la imagen según la arquitectura (299×299, 384×384 o 224×224).	
	saliency(model, img, is_transformer)	Operación privada que calcula el mapa de Saliency mediante los gradientes de la puntuación respecto a la imagen.	
	smoothgrad(model, img, is_transformer)	Operación privada que promedia los mapas de Saliency con ruido gaussiano y normaliza el resultado.	
	get_cam_or_attention(model, img, is_transformer, img_size)	Operación privada que calcula el Grad-CAM sobre la última capa convolucional para las arquitecturas convolucionales o el mapa de atención para las Transformer, con un fallback basado en gradientes.	
Comentarios	La generación de los mapas se realiza sobre el modelo cargado por MIEngine, de modo que la clase depende del motor de predicción. La figura resultante se sirve como recurso estático y su ruta se persiste en la consulta, sin repetir el cálculo en las visualizaciones posteriores. Las rutas estáticas no aplican una comprobación de propiedad equivalente a la del router, por lo que este acceso requiere protección adicional fuera del entorno de demostración.		

Tabla x - Especificación de clase CL-0005

CL-0006	PdfGenerator		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/pdf_generator.py. Construye el informe PDF del diagnóstico a partir de la imagen original, el mapa de explicabilidad, la etiqueta, la confianza y el modelo utilizado, empleando la clase concreta PDFReport y los textos localizados del servicio de idioma.		
Atributos	Nombre	Tipo	Descripción
	N/A	N/A	La clase no presenta atributos de estado propios: construye el informe a partir de los parámetros recibidos en la operación.
Operaciones	Nombre	Descripción	
	generate_medical_report(image_path, xai_path, label, confidence, model_name, lang)	Compone el informe A4 con la fecha, el modelo, el diagnóstico con su color según el resultado, la confianza y las dos imágenes (original y mapa), lo guarda en static/reports con un nombre basado en la fecha y devuelve la ruta del documento.	
Comentarios	La generación del informe se produce durante el procesamiento del diagnóstico, de modo que el documento está disponible cuando la consulta se completa y la descarga posterior no depende de una nueva operación del sistema. El informe se trata como parte de la información protegida de la consulta, aunque su ubicación bajo static/reports requiere controles adicionales si se utiliza fuera de la demostración.		

Tabla x - Especificación de clase CL-0006

CL-0007	PDFReport		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase concreta del informe PDF de diagnóstico, correspondiente a la clase PDFReport del módulo services/pdf_generator.py. Especializa la clase FPDF de la librería fpdf2 y redefine la cabecera y el pie de página del informe, además de conservar el idioma de los textos (FPDF2, 2024).		
Atributos	Nombre	Tipo	Descripción
	_lang	str	Idioma utilizado para localizar los textos del informe (cabecera, pie y etiquetas).
Operaciones	Nombre	Descripción	
	__init__(lang)	Inicializa el documento de la librería base y conserva el idioma para localizar los textos.	
	header()	Redefine la cabecera de cada página del informe con el título y el color corporativo.	
	footer()	Redefine el pie de página con el número de página.	
Comentarios	Esta clase constituye la herencia real del subsistema: su relación con FPDF se representa en el modelo de clases como una especialización. El generador de informes la instancia para componer el documento, de modo que el conocimiento del formato queda encapsulado en la clase y fuera del router.		

Tabla x - Especificación de clase CL-0007

21.3. Subsistema SD-003: Historial y gestión de consultas

El subsistema SD-003 materializa la recuperación y la gestión de las consultas ya realizadas y agrupa una única clase de diseño propia. La clase HistoryRouter se corresponde con el router routers/history.py y concentra las operaciones del historial: la consulta del listado, la actualización del nombre y la eliminación de una consulta, todas ellas precedidas de la comprobación de propiedad. El subsistema no dispone de clases de servicio propias, porque su lógica se limita a la recuperación de los registros persistidos por SD-002 y a la actualización de la etiqueta de organización; la comprobación de propiedad se resuelve dentro de la propia clase. HistoryRouter depende de AuthService, perteneciente a SD-001, para resolver la identidad del usuario, y de LangService, perteneciente a SD-006, para los mensajes localizados de las respuestas.

21.3.1. Diagrama de clases del subsistema

El modelo de clases de SD-003 se representa en la figura 85. El diagrama muestra la clase propia del subsistema con sus operaciones, y las dependencias hacia las clases AuthService y LangService, que pertenecen a otros subsistemas y se representan con las operaciones que el historial consume. El modelo no presenta herencias ni clases abstractas: la clase HistoryRouter es una clase concreta que orquesta las operaciones del historial y delega la identidad en el subsistema de acceso.

El diagrama refleja la simplicidad estructural del subsistema: una sola clase concentra la consulta del listado, el renombrado y la eliminación, y la operación privada `_check_consultation_ownership()` que protege las dos operaciones de modificación. La dependencia hacia AuthService materializa el mecanismo común de identificación de SD-001, del que el historial obtiene el usuario para filtrar y comprobar la propiedad de las consultas. La dependencia hacia LangService localiza los mensajes de error y de respuesta. La relación de SD-003 con SD-002 es de productor-consumidor de datos y no se representa como una asociación entre clases: los registros de consultations que el historial lee son creados por el worker de SD-002, y la colaboración se produce a través de la persistencia compartida.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases materializan los mensajes de los diagramas de secuencia del capítulo 20. La operación `get_history()` implementa el CU-011; la operación `update_patient_name()` implementa el CU-013; la operación `delete_history_record()` implementa el CU-014; y la operación privada `_check_consultation_ownership()` materializa la comprobación de propiedad que condiciona las respuestas HTTP 404 y 403 de los diagramas de secuencia.

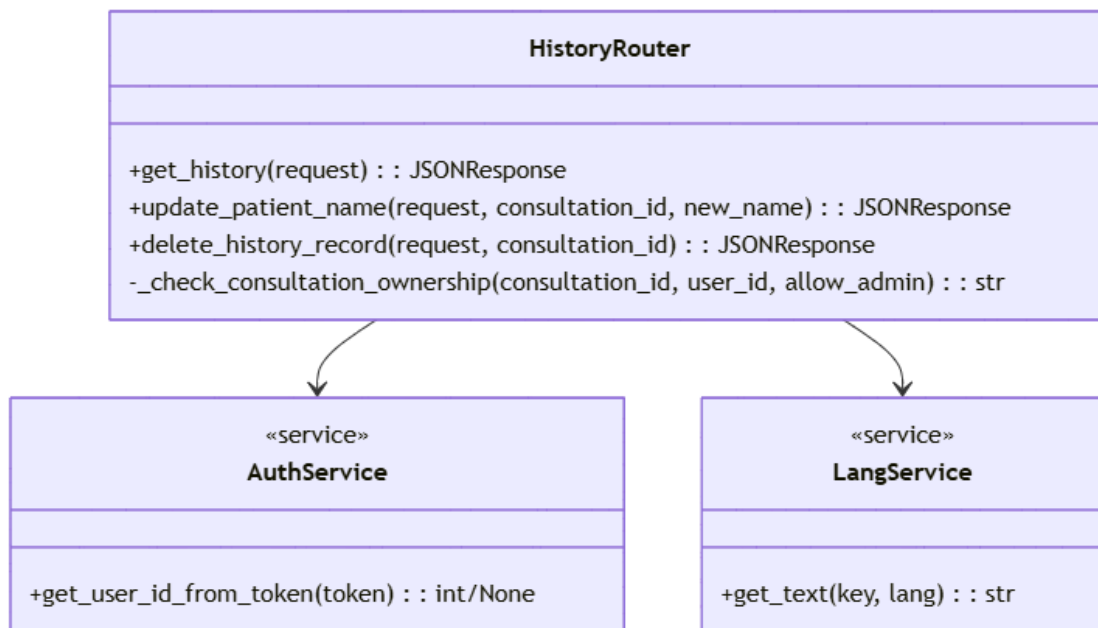


Ilustración x - Diagrama de clases del subsistema SD-003

21.3.2. Especificación de las clases

A continuación se define la clase propia de SD-003.

CL-0008	HistoryRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/history.py. Proporciona la recuperación y la gestión de las consultas de diagnóstico de un usuario: devuelve el listado del historial filtrado por el propietario, actualiza el nombre mostrado de una consulta y elimina una consulta de forma física, aplicando en las operaciones de modificación la comprobación de propiedad con la excepción controlada del administrador.		
Atributos	Nombre	Tipo	Descripción
	N/A	N/A	La clase no presenta atributos de estado propios: cada operación resuelve la identidad y accede a la persistencia a partir de los parámetros recibidos.
Operaciones	Nombre	Descripción	
	get_history(request)	Devuelve el listado de consultas del usuario autenticado, filtrado por user_id y ordenado por fecha descendente; responde HTTP 401 sin sesión y formatea las fechas antes de formar la respuesta.	
	update_patient_name(request, consultation_id, new_name)	Actualiza el nombre mostrado de una consulta: comprueba la propiedad, responde HTTP 404 si la consulta no existe y HTTP 403 si pertenece a otro usuario, y ejecuta la actualización del nombre cuando la propiedad se confirma.	
	delete_history_record(request, consultation_id)	Elimina de forma física una consulta: comprueba la propiedad, responde HTTP 404 o HTTP 403 en los casos de inexistencia o falta de permisos, y ejecuta el borrado cuando la propiedad se confirma.	
	_check_consultation_ownership(consultation_id, user_id, allow_admin)	Operación privada que comprueba que la consulta existe y pertenece al usuario solicitante, permitiendo la excepción del rol admin; devuelve not_found, forbidden u ok.	
Comentarios	La clase no vuelve a ejecutar el modelo para mostrar una consulta anterior: recupera los metadatos y las rutas de los artefactos ya persistidos. El nombre actualizado se aplica sobre la columna patient_name, que funciona como etiqueta de organización y no como identificación clínica del paciente. La eliminación es un borrado físico de la fila y no elimina explícitamente la imagen, el mapa XAI ni el informe; tampoco existe auditoría automática, tal y como se declaró en el capítulo 17.		

Tabla x - Especificación de clase CL-0008

21.4. Subsistema SD-004: Laboratorio de experimentación MLOps

El subsistema SD-004 materializa el laboratorio de experimentación MLOps y agrupa cinco clases de diseño propias. La clase `TrainerRouter` se corresponde con el router `routers/trainer.py` y actúa como fachada ligera del laboratorio: expone los endpoints de configuración, lanzamiento, consulta y gestión de las sesiones, y delega la lógica en los servicios. La clase `ChatbotService` se corresponde con `services/chatbot_service.py` y resuelve la configuración conversacional del experimento mediante el asistente externo. La clase `MlopsEngine` se corresponde con `services/mlops_engine.py` y concentra la organización de las sesiones, la ejecución de los pipelines de entrenamiento y la lectura de los resultados. Las clases `PdfGeneratorMlops` y `MedicalReport`, correspondientes a `services/pdf_generator_mlops.py`, generan el informe consolidado de la sesión; `MedicalReport` es la clase concreta que hereda de la librería `FPDF`, segunda herencia real del diseño de clases de `vitalXAI`. La limitación de entrenamientos de CU-039 se mantiene como capacidad prevista y no como operación implementada del router.

21.4.1. Diagrama de clases del subsistema

El modelo de clases de SD-004 se representa en la figura 86. El diagrama muestra las cinco clases propias del subsistema, la herencia de `MedicalReport` sobre la clase `FPDF` de la librería, la dependencia del servicio conversacional hacia el proveedor externo de inteligencia artificial, y las dependencias del router hacia las clases propias y hacia los servicios transversales `AuthService` y `LangService`. El router concentra las asociaciones del subsistema: delega la exploración, el entrenamiento y la consulta de resultados en `MlopsEngine`, la conversación en `ChatbotService` y la generación del informe en `PdfGeneratorMlops`.

El diagrama identifica la herencia de `MedicalReport` sobre la clase `FPDF` de la librería de generación de documentos, que se representa con las operaciones básicas que el informe hereda y utiliza. El proveedor externo de inteligencia artificial se modela como una clase externa con el estereotipo de proveedor, con la operación de generación de respuestas que el servicio conversacional invoca; la clave de la API no forma parte de la interfaz de la clase, sino del atributo de configuración de `ChatbotService`. Las dependencias hacia `AuthService` y `LangService` reflejan los servicios transversales de SD-001 y SD-006 que el router consume. La clase `MlopsEngine` constituye el núcleo del subsistema: orquesta los scripts de entrenamiento, el análisis XAI y la comparación estadística, y organiza la lectura y escritura de los resultados en el sistema de ficheros, sin depender del servicio de idioma porque sus mensajes de registro son textos fijos del pipeline.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases materializan los casos de uso reales del capítulo 20. Las operaciones de `TrainerRouter` exponen los endpoints de los CU-015 a CU-030; CU-039 se representa como capacidad prevista y pendiente de implementación. Las operaciones de `MlopsEngine` implementan los flujos del procesamiento del entrenamiento y de la validación externa; la operación `generate_pdf_report()` de `PdfGeneratorMlops` materializa el CU-028; y las operaciones de `ChatbotService` materializan la configuración conversacional del CU-016.

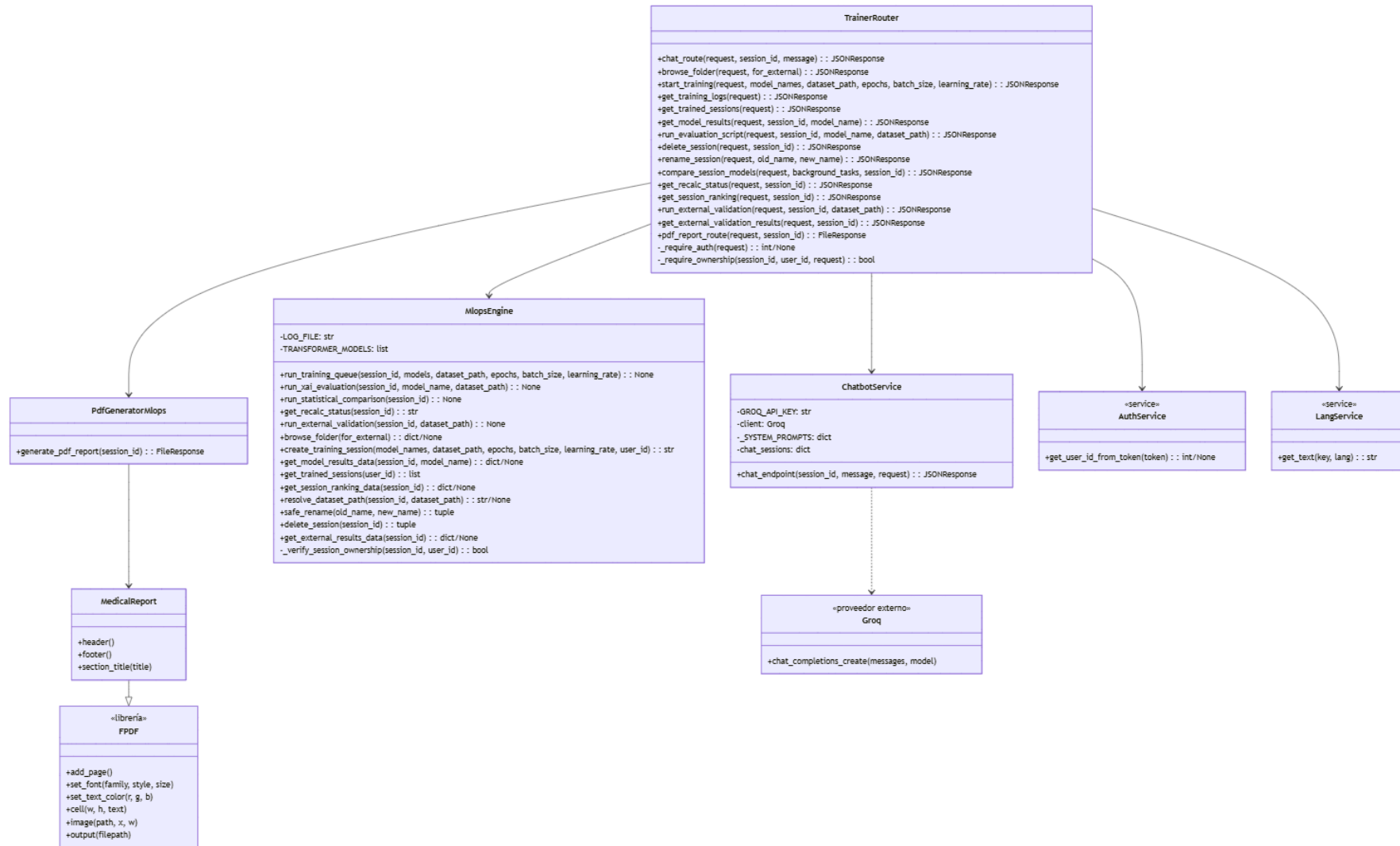


Ilustración x - Diagrama de clases del subsistema SD-004

21.4.2. Especificación de las clases

A continuación se definen las cinco clases propias de SD-004.

CL-0009	TrainerRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/trainer.py. Actúa como fachada ligera del laboratorio MLOps: expone los endpoints de conversación, exploración del dataset, lanzamiento de entrenamientos, consulta de logs, sesiones y resultados, ejecución del análisis XAI, comparativa estadística, validación externa, informes, renombrado y eliminación de sesiones. Comprueba la autenticación y la propiedad de las sesiones y delega la lógica en los servicios del subsistema.		
Atributos	Nombre	Tipo	Descripción
	N/A	N/A	La clase no presenta atributos de estado propios: reexporta constantes del motor para compatibilidad y delega el resto en los servicios.
Operaciones	Nombre	Descripción	
	chat_route(request, session_id, message)	Gestiona la conversación con el asistente de configuración, delegando en ChatbotService.	
	browse_folder(request, for_external)	Explora la carpeta del dataset de entrenamiento o de validación externa mediante MlopsEngine.	
	start_training(request, model_names, dataset_path, epochs, batch_size, learning_rate)	Lanza un experimento: valida la ruta del dataset, crea la sesión mediante MlopsEngine y encola el trabajo de tipo training.	
	get_training_logs(request)	Devuelve las últimas líneas del registro de entrenamiento.	
	get_trained_sessions(request)	Devuelve las sesiones de entrenamiento del usuario mediante MlopsEngine.	
	get_model_results(request, session_id, model_name)	Devuelve los resultados de un modelo de la sesión, con las métricas K-fold, la calibración y las métricas XAI.	
	update_patient_name(request, session_id, model_name, dataset_path, consultation_id, new_name)	Ejecuta el análisis XAI manual de un modelo mediante MlopsEngine.	

	delete_session(request, session_id)	Elimina una sesión y sus artefactos mediante MlopsEngine, tras comprobar la propiedad.
	rename_session(request, old_name, new_name)	Renombra una sesión mediante MlopsEngine, tras comprobar la propiedad.
	compare_session_models(request, background_tasks, session_id)	Programa el recálculo de la comparativa estadística como tarea en segundo plano.
	get_recalc_status(request, session_id)	Devuelve el estado del recálculo de la comparativa (running o completed).
	get_session_ranking(request, session_id)	Devuelve el ranking de modelos y el heatmap de Wilcoxon mediante MlopsEngine; el contraste se basa en el procedimiento descrito por Wilcoxon (Wilcoxon, 1945).
	run_external_validation(request, session_id, dataset_path)	Encola la validación externa de la sesión con el tipo external_validation.
	get_external_validation_results(request, session_id)	Devuelve las métricas, la curva ROC y la matriz de DeLong de la validación externa, cuyo contraste se fundamenta en DeLong, DeLong y Clarke-Pearson (1988).
	pdf_report_route(request, session_id)	Genera y sirve el informe PDF de la sesión mediante PdfGeneratorMlops.
	_require_auth(request)	Operación privada que resuelve la identidad del usuario mediante AuthService, devolviendo None sin sesión.
	_require_ownership(session_id, user_id, request)	Operación privada que comprueba la propiedad de la sesión mediante MlopsEngine, con la excepción del rol admin cuando la ruta lo permite.
Comentarios	La clase concentra un gran número de operaciones porque el laboratorio expone numerosos puntos de consulta y gestión; todas ellas mantienen la frontera de autorización: la comprobación de propiedad se resuelve antes de abrir cualquier operación sobre una sesión. Las operaciones de lanzamiento de entrenamiento y de validación externa encolan trabajos en la cola de SD-006.	

Tabla x - Especificación de clase CL-0009

CL-0010	ChatbotService		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/chatbot_service.py. Gestiona la configuración conversacional del experimento: recibe los mensajes del usuario en lenguaje natural, los envía al asistente externo y devuelve una configuración estructurada del entrenamiento, solicitando los parámetros que falten. Trata al proveedor de inteligencia artificial como una frontera externa y utiliza el cliente de Groq documentado por el proveedor (Groq, 2024).		
Atributos	Nombre	Tipo	Descripción
	GROQ_API_KEY	str	Clave de la API del proveedor del asistente, obtenida del entorno; el único atributo del subsistema que conserva la credencial del proveedor.
	client	Groq	Cliente del proveedor externo, instanciado con la clave de la API; es None si la clave no está configurada.
	_SYSTEM_PROMPTS	dict	Diccionario de prompts del sistema por idioma, que guían la configuración conversacional.
	chat_sessions	dict	Diccionario que conserva el historial de mensajes de cada conversación, indexado por el identificador de la sesión de chat.
Operaciones	Nombre	Descripción	
	chat_endpoint(session_id, message, request)	Procesa un mensaje de la conversación: selecciona el prompt del sistema según el idioma, mantiene el historial de la sesión, solicita la configuración al proveedor externo y devuelve la respuesta con los parámetros de la configuración o la solicitud de los faltantes.	
Comentarios	La clase es el único componente del sistema que necesita la clave de la API del asistente; los routers no la incluyen en las respuestas ni en los registros. Los errores del proveedor externo se tratan como condiciones de la frontera y no se confunden con los errores de persistencia o de entrenamiento.		

Tabla x - Especificación de clase CL-0010

CL-0011	MlopsEngine		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/mlops_engine.py. Constituye el núcleo del laboratorio: organiza las sesiones de entrenamiento en el sistema de ficheros, orquesta la ejecución de los scripts del pipeline, incluidos el entrenamiento K-fold, el análisis XAI, la comparación estadística y la validación externa, y resuelve la lectura de los resultados y la comprobación de propiedad de las sesiones. La referencia a MLOps se utiliza aquí en el sentido de coordinación operativa del ciclo de entrenamiento y sus resultados (Kreuzberger, Kühl, & Hirschl, 2023).		
Atributos	Nombre	Tipo	Descripción
	LOG_FILE	str	Ruta del archivo de registro del entrenamiento, donde el motor escribe la progresión del pipeline.
	TRANSFORMER_MODELS	list	Lista de las arquitecturas Transformer del proyecto, utilizada para bifurcar el script de entrenamiento.
Operaciones	Nombre	Descripción	
	run_training_queue(session_id, models, dataset_path, epochs, batch_size, learning_rate)	Orquesta el entrenamiento completo de la sesión: por cada modelo lanza el script de entrenamiento CNN o Transformer y los scripts de explicabilidad, y al finalizar ejecuta la comparación estadística que genera el ranking y la matriz de Wilcoxon (Wilcoxon, 1945).	
	run_xai_evaluation(session_id, model_name, dataset_path)	Ejecuta el análisis XAI cualitativo y cuantitativo de un modelo en modo manual.	
	run_statistical_comparison(session_id)	Regenera la comparativa estadística de la sesión, gestionando el marcador de estado del recálculo.	
	get_recalc_status(session_id)	Devuelve completed o running según el marcador de estado del recálculo.	
	run_external_validation(session_id, dataset_path)	Ejecuta la validación externa de la sesión y el test estadístico de DeLong sobre la cohorte independiente (DeLong, DeLong, & Clarke-Pearson, 1988).	
	browse_folder(for_external)	Devuelve la ruta del dataset configurada por entorno o abre el selector de carpetas; distingue el dataset de entrenamiento del de validación externa.	
	create_training_session(model_names, dataset_path, epochs, batch_size, learning_rate, user_id)	Crea el directorio de la sesión en training_results y escribe su configuración, con el identificador del usuario propietario; devuelve el identificador de la sesión.	
	get_model_results_data(session_id, model_name)	Lee las métricas K-fold, la calibración y las métricas XAI de un modelo, junto con las rutas de sus artefactos; devuelve None si el modelo no dispone de resultados.	

	get_trained_sessions(user_id)	Enumera las sesiones con resultados que pertenecen al usuario, en orden descendente, con sus modelos.
	get_session_ranking_data(session_id)	Lee el ranking de modelos, el heatmap de Wilcoxon y la configuración de la sesión; devuelve None si no hay ranking.
	resolve_dataset_path(session_id, dataset_path)	Devuelve la ruta del dataset indicada o la conservada en la sesión.
	safe_rename(old_name, new_name)	Valida y sanitiza el nuevo nombre de la sesión y ejecuta el renombrado del directorio, devolviendo el estado y el contenido de la respuesta.
	delete_session(session_id)	Elimina el directorio de la sesión y sus artefactos, devolviendo el estado y el contenido de la respuesta.
	get_external_results_data(session_id)	Lee las métricas, la curva ROC y la matriz de DeLong de la validación externa; devuelve None si no existen resultados.
	_verify_session_ownership(session_id, user_id)	Operación privada que comprueba que la configuración de la sesión contiene el identificador del usuario, como regla de propiedad del laboratorio.
Comentarios	La clase combina la persistencia híbrida de la plataforma: coordina la escritura de los artefactos en el sistema de ficheros y conserva en MySQL únicamente los trabajos de la cola. La comprobación de propiedad de las sesiones se resuelve en esta clase, que la comparte con el router sin duplicarla. Los scripts del pipeline se invocan como procesos externos que leen la configuración desde variables de entorno.	

Tabla x - Especificación de clase CL-0011

CL-0012	PdfGeneratorMlops		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/pdf_generator_mlops.py. Construye el informe consolidado de una sesión de entrenamiento: recopila la configuración del experimento, el ranking con el heatmap de Wilcoxon, la validación externa con la curva ROC y la matriz de DeLong, y el detalle técnico de cada modelo con sus métricas XAI y sus mapas de calor, empleando la clase concreta MedicalReport.		
Atributos	Nombre	Tipo	Descripción
	N/A	N/A	La clase no presenta atributos de estado propios: compone el informe a partir de los artefactos persistidos en la sesión.
Operaciones	Nombre	Descripción	
	generate_pdf_report(session_id)	Compone el informe de la sesión, lo guarda en el directorio de la sesión con el nombre Informe_Completo_{session_id}.pdf y lo devuelve como descarga; responde HTTP 404 si la sesión no existe.	
Comentarios	La generación del informe separa el conocimiento del formato del router: el generador recibe la información preparada por el motor y construye el documento, de modo que el router no conoce el formato interno del informe. El informe se genera bajo demanda, cuando la sesión dispone de los datos necesarios.		

Tabla x - Especificación de clase CL-0012

CL-0013	PdfGeneratorMlops		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase concreta del informe MLOps, correspondiente a la clase MedicalReport del módulo services/pdf_generator_mlops.py. Especializa la clase FPDF de la librería de generación de documentos y redefine la cabecera corporativa, el pie de página y el formato de los títulos de sección del informe.		
Atributos	Nombre	Tipo	Descripción
	N/A	N/A	La clase no presenta atributos de estado propios: compone el informe a partir de los artefactos persistidos en la sesión.
Operaciones	Nombre	Descripción	
	header()	Redefine la cabecera de cada página del informe con el título corporativo y el encabezado del protocolo MLOps.	
	footer()	Redefine el pie de página con el número de página y la indicación de generación automática.	
	section_title(title)	Escribe un título de sección con el formato propio del informe.	
Comentarios	Esta clase constituye la segunda herencia real del diseño de clases de vitalXAI: su relación con FPDF se representa en el modelo de clases como una especialización. El generador de informes la instancia para componer el documento consolidado de la sesión.		

Tabla x - Especificación de clase CL-0013

21.5. Subsistema SD-005: Supervisión y administración

El subsistema SD-005 materializa las operaciones de supervisión y administración reservadas al rol de administrador y agrupa una única clase de diseño propia. La clase AdminRouter se corresponde con el router routers/admin.py y concentra las tres operaciones de consulta administrativa: el listado de usuarios, las consultas de un usuario concreto y el detalle de una consulta, todas ellas precedidas de la comprobación de la autorización administrativa. La gestión de cuentas de CU-038 está prevista, pero no está implementada en el router actual. El subsistema no dispone de clases de servicio propias, porque su responsabilidad es establecer la autorización y coordinar las consultas globales reutilizando los servicios existentes. AdminRouter depende de AuthService, perteneciente a SD-001, para resolver la identidad y consultar el rol, y de MlopsEngine, perteneciente a SD-004, para recuperar las sesiones del laboratorio de un usuario en la operación de consulta de su actividad.

21.5.1. Diagrama de clases del subsistema

El modelo de clases de SD-005 se representa en la figura 87. El diagrama muestra la clase propia del subsistema con sus operaciones, y las dependencias hacia las clases AuthService y MlopsEngine, pertenecientes a SD-001 y SD-004, representadas con las operaciones que la supervisión consume. El modelo no presenta herencias ni clases abstractas: la clase AdminRouter es una clase concreta que centraliza la comprobación de permisos y coordina las consultas globales.

El diagrama refleja la simplicidad estructural del subsistema y su decisión de diseño central: la operación privada `_require_admin()` se antepone a todas las consultas administrativas, de modo que ninguna ruta ejecuta una operación de lectura sobre usuarios o consultas sin haber resuelto antes la identidad y el rol del solicitante. La dependencia hacia AuthService materializa el mecanismo de identificación de SD-001, del que el router obtiene el usuario y consulta su rol; la dependencia hacia MlopsEngine materializa la recuperación de las sesiones del laboratorio en la consulta de la actividad de un usuario. El subsistema no depende del servicio de idioma porque sus mensajes de autorización son textos fijos, y no duplica la lógica de propiedad del historial ni del laboratorio: accede a los datos con las mismas tablas y servicios que el resto de la aplicación.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases materializan los mensajes de los diagramas de secuencia del capítulo 20. La operación `admin_users()` implementa el CU-031; la operación `admin_user_consultations()` implementa el CU-032; la operación `admin_get_consultation()` implementa el CU-033; y la operación privada `_require_admin()` materializa la comprobación de permisos que condiciona las respuestas HTTP 401 y 403 de los diagramas de secuencia.

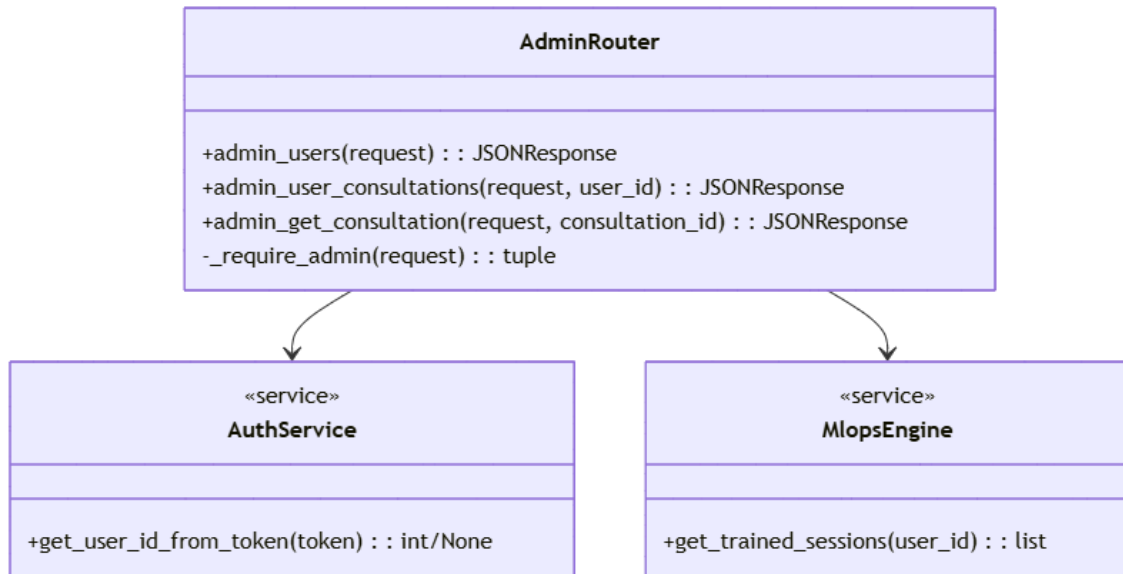


Ilustración x - Diagrama de clases del subsistema SD-005

21.5.2. Especificación de las clases

A continuación se define la clase propia de SD-005.

CL-0014	AdminRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/admin.py. Proporciona las operaciones de supervisión administrativa: el listado de usuarios con sus recuentos de diagnósticos y de sesiones del laboratorio, las consultas y las sesiones de un usuario concreto, y el detalle de una consulta. Centraliza la comprobación de la autorización administrativa en la operación privada _require_admin() y reutiliza los servicios existentes sin duplicar la lógica de propiedad.		
Atributos	Nombre	Tipo	Descripción
	N/A	N/A	La clase no presenta atributos de estado propios: compone el informe a partir de los artefactos persistidos en la sesión.
Operaciones	Nombre	Descripción	
	admin_users(request)	Devuelve el listado de usuarios con el número de diagnósticos, obtenido mediante una consulta agregada sobre users y consultations, y con el número de sesiones del laboratorio, calculado inspeccionando las configuraciones del sistema de ficheros.	
	admin_user_consultations(request, user_id)	Devuelve las consultas de diagnóstico de un usuario concreto, ordenadas por fecha descendente, junto con sus sesiones de entrenamiento recuperadas mediante MlopsEngine; responde HTTP 404 si el usuario no existe.	
	admin_get_consultation(request, consultation_id)	Devuelve el detalle completo de una consulta por su identificador, con las rutas de los artefactos, la predicción, la confianza y los metadatos; responde HTTP 404 si la consulta no existe.	
	_require_admin(request)	Operación privada que resuelve la identidad mediante AuthService y consulta el rol en la tabla users, devolviendo una tupla con el identificador y un indicador de rol administrativo; distingue la ausencia de identidad, la identidad sin privilegios y la identidad administrativa.	
Comentarios	La clase mantiene la frontera entre el control de permisos y la representación de la información: no duplica la lógica de propiedad del historial ni del laboratorio, y la autorización administrativa se comprueba en el servidor, no se infiere de parámetros del navegador. La protección reutiliza la identidad de SD-001 sin una segunda autenticación, y la traza de auditoría administrativa permanece como condición pendiente, tal y como se declaró en el capítulo 17.		

Tabla x - Especificación de clase CL-0014

21.6. Subsistema SD-006: Cola de trabajos y capacidades transversales

El subsistema SD-006 materializa la cola persistente de trabajos y las capacidades transversales de la plataforma, y agrupa cuatro clases de diseño propias. La clase QueueRouter se corresponde con el router routers/queue.py y expone la consulta del estado de los trabajos y la cancelación de un trabajo pendiente. La clase QueueWorker se corresponde con el servicio services/queue_worker.py y actúa como consumidor de la cola: reclama los trabajos pendientes, ejecuta el flujo correspondiente en el executor y actualiza el estado del trabajo. La clase LangService se corresponde con el servicio services/lang.py y concentra la internacionalización de la plataforma. La clase QueueJob modela la entidad persistente de la cola, la fila de la tabla job_queue, que constituye la clase más compleja del subsistema por su máquina de estados, compartida por los subsistemas que generan trabajos.

21.6.1. Diagrama de clases del subsistema

El modelo de clases de SD-006 se representa en la figura 88. El diagrama muestra las cuatro clases propias del subsistema, la dependencia del router hacia el servicio de identidad de SD-001, y las dependencias del worker hacia las clases MIEngine y MlopsEngine, pertenecientes a SD-002 y SD-004, que representan los flujos que el worker ejecuta según el tipo de trabajo. La clase QueueJob se representa con los atributos de su estado persistido y aparece asociada tanto al router, que consulta y cancela los trabajos, como al worker, que los reclama y actualiza.

El diagrama refleja la frontera entre el ciclo de petición y la ejecución asíncrona: el router y el worker comparten la persistencia de la cola, materializada en la clase QueueJob, pero no comparten el ciclo de petición. El worker despacha el trabajo según su tipo: para los diagnósticos invoca la clase MIEngine de SD-002, y para los entrenamientos y las validaciones externas invoca la clase MlopsEngine de SD-004; esas clases se representan con las operaciones que el worker utiliza. La clase LangService no presenta dependencias de colaboración en el diagrama porque sus consumidores pertenecen a otros subsistemas: el mecanismo de internacionalización se ofrece como un servicio transversal que cualquier router o servicio invoca. El modelo no presenta herencias ni clases abstractas.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases materializan los mensajes de los diagramas de secuencia del capítulo 20. La operación queue_status() de QueueRouter implementa el CU-034; la operación cancel_job() implementa el CU-035; y las operaciones del worker materializan el procesamiento asíncrono que orquesta los flujos de SD-002 y SD-004. La clase QueueJob concentra la máquina de estados de la cola, que se describe en su definición.

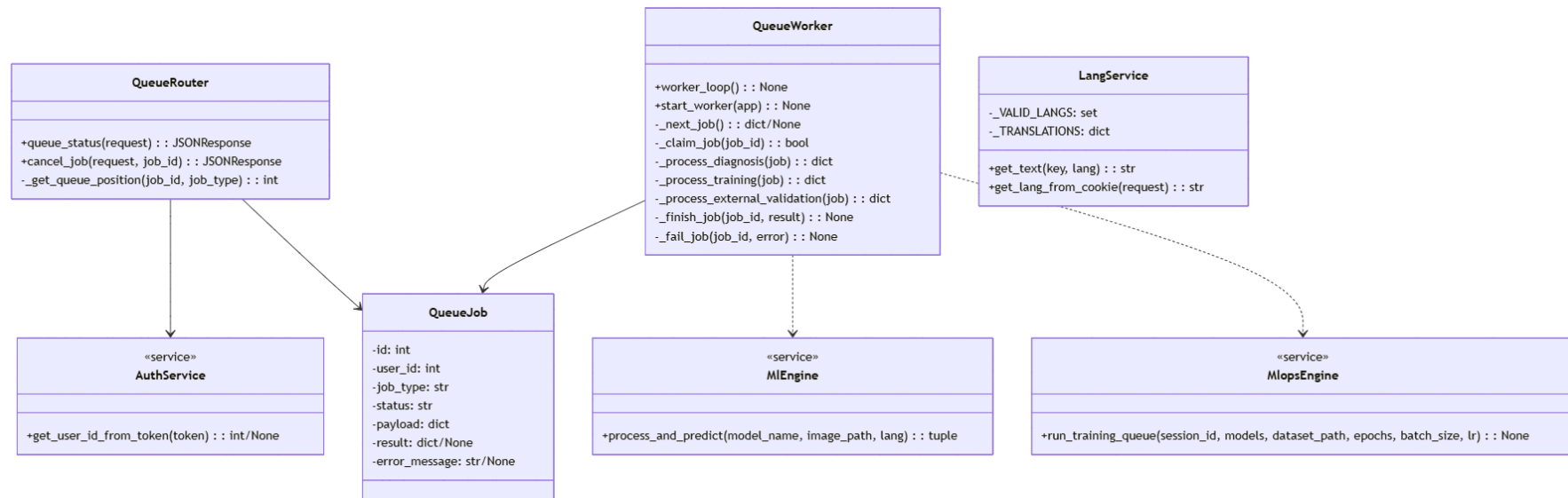


Ilustración x - Diagrama de clases del subsistema SD-006

21.6.2. Especificación de las clases

A continuación se definen las cuatro clases propias de SD-006.

CL-0015	QueueRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/queue.py. Expone la consulta del estado de los trabajos de la cola y la cancelación de un trabajo pendiente: devuelve los trabajos recientes del usuario con su estado y su posición, interpreta el payload según el tipo de trabajo y aplica la cancelación mediante una actualización condicional.		
Atributos	Nombre	Tipo	Descripción
	N/A	N/A	La clase no presenta atributos de estado propios: compone el informe a partir de los artefactos persistidos en la sesión.
Operaciones	Nombre	Descripción	
	queue_status(request)	Devuelve los últimos veinte trabajos del usuario con su estado, sus fechas y la posición de los trabajos encolados; interpreta el payload por tipo (modelo del diagnóstico, sesión y modelos del entrenamiento, o modelos de la validación externa leídos de su configuración) sin enviar el payload completo al navegador.	
	cancel_job(request, job_id)	Cancela un trabajo mediante una actualización condicionada a que pertenezca al usuario y continúe en queued; responde HTTP 200 si la actualización afectó a una fila y HTTP 404 si el trabajo no está en cola.	
	_get_queue_position(job_id, job_type)	Operación privada que calcula la posición de un trabajo encolado, respetando la prioridad de la cola según el tipo de trabajo.	
Comentarios	La cancelación solo opera sobre trabajos pendientes: un trabajo reclamado por el worker no se interrumpe, en coherencia con la máquina de estados de la cola. La consulta del estado filtra siempre por el usuario autenticado, aplicando el aislamiento de datos entre cuentas.		

Tabla x - Especificación de clase CL-0015

CL-0016	QueueWorker		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/queue_worker.py. Actúa como consumidor de la cola de trabajos: al iniciar la aplicación restablece los trabajos que quedaron en estado running, selecciona el siguiente trabajo pendiente, lo reclama mediante una actualización condicionada a que siga en queued y ejecuta el flujo correspondiente en el executor, marcando el resultado como completado o fallido. No comparte el ciclo de petición con los routers.		
Atributos	Nombre	Tipo	Descripción
	N/A	N/A	La clase no presenta atributos de estado propios: compone el informe a partir de los artefactos persistidos en la sesión.
Operaciones	Nombre	Descripción	
	worker_loop()	Bucle infinito del worker: selecciona el siguiente trabajo pendiente, lo reclama y ejecuta el flujo correspondiente en el executor, esperando cuando la cola está vacía.	
	start_worker(app)	Restablece los trabajos en running a queued y crea la tarea asíncrona del bucle del worker, conservándola en el estado de la aplicación.	
	_next_job()	Operación privada que selecciona el primer trabajo queued, con prioridad para los diagnósticos frente a los entrenamientos.	
	_claim_job(job_id)	Operación privada que reclama un trabajo mediante una actualización condicionada a que siga en queued, devolviendo True solo si la actualización afectó a una fila.	
	_process_diagnosis(job)	Operación privada que procesa un trabajo de diagnóstico: invoca MIEngine, XaiGenerator y PdfGenerator de SD-002 y persiste la consulta con sus artefactos.	
	_process_training(job)	Operación privada que procesa un trabajo de entrenamiento, delegando en MlopsEngine.	
	_process_external_validation(job)	Operación privada que procesa una validación externa, delegando en MlopsEngine.	
	_finish_job(job_id, result)	Operación privada que marca el trabajo como completed con su resultado en formato JSON.	
	_fail_job(job_id, error)	Operación privada que marca el trabajo como failed con el mensaje de error limitado.	
Comentarios	La reclamación condicional evita que dos iteraciones del worker procesen el mismo registro, lo que introduce una garantía de consistencia útil si la ejecución evoluciona hacia más de un consumidor. Los flujos de procesamiento se ejecutan en el executor para no bloquear el bucle de eventos de la aplicación.		

Tabla x - Especificación de clase CL-0016

CL-0017	LangService		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/lang.py. Concentra la internacionalización de la plataforma: mantiene el diccionario de mensajes del backend en los cuatro idiomas soportados, obtiene el idioma seleccionado por el usuario y devuelve los textos de la interfaz y de las respuestas. No requiere un servicio externo ni una base de datos adicional.		
Atributos	Nombre	Tipo	Descripción
	_VALID_LANGS	set	Conjunto de los idiomas válidos de la plataforma: español, inglés, chino e hindi.
	_TRANSLATIONS	dict	Diccionario de traducciones de los mensajes del backend, indexado por idioma y por clave de mensaje.
Operaciones	Nombre	Descripción	
	get_text(key, lang)	Devuelve el mensaje correspondiente a la clave en el idioma indicado, aplicando el español como valor por defecto cuando el idioma o la clave no están disponibles.	
	get_lang_from_cookie(request)	Lee la cookie de idioma de la petición, valida que pertenezca al conjunto permitido y devuelve el idioma aplicable, con el español como valor por defecto.	
Comentarios	La clase constituye el mecanismo transversal de presentación del idioma: los routers y los servicios de los subsistemas la invocan para localizar sus mensajes, las etiquetas de predicción, los títulos de los mapas de explicabilidad, los textos de los informes y los mensajes del asistente, en coherencia con el caso de uso CU-004.		

Tabla x - Especificación de clase CL-0017

CL-0018	QueueJob		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño que modela la entidad persistente de la cola de trabajos, correspondiente a la fila de la tabla job_queue. Conserva el estado de cada tarea asíncrona (diagnóstico, entrenamiento o validación externa) junto con el usuario propietario, el tipo, el payload serializable y el resultado o el error. Constituye la clase más compleja del subsistema por su máquina de estados, que determina las transiciones permitidas entre los estados de un trabajo.		
Atributos	Nombre	Tipo	Descripción
	id	int	Identificador del trabajo, asignado automáticamente por la persistencia.
	user_id	int	Identificador del usuario propietario del trabajo, que condiciona la consulta y la cancelación.
	job_type	str	Tipo del trabajo: diagnosis, training o external_validation.
	status	str	Estado del trabajo en la máquina de estados: queued, running, completed, failed o cancelled.
	payload	dict	Configuración serializable del trabajo, que contiene el modelo y la ruta de la imagen, o la sesión y los hiperparámetros.
	result	dict	Resultado del trabajo cuando se completa, en formato JSON; None en el resto de los estados.
	error_message	str	Mensaje de error cuando el trabajo falla, limitado en longitud; None en el resto de los estados.
Operaciones	Nombre	Descripción	
	N/A	La clase no expone operaciones propias: las transiciones de su estado las ejecutan el router (cancelación) y el worker (reclamación, finalización y fallo), de modo que el comportamiento de la máquina de estados se describe mediante el diagrama de transición de estados siguiente.	
Comentarios	La máquina de estados de la clase es compartida por los subsistemas que generan trabajos: SD-002 crea los diagnósticos, SD-004 crea los entrenamientos y las validaciones externas, y el worker de SD-006 aplica las transiciones. La cancelación solo opera sobre queued, y un trabajo en running no se interrumpe mediante una actualización administrativa, de modo que la reclamación condicional del worker mantiene la coherencia entre el estado persistido y la ejecución en curso.		

Tabla x - Especificación de clase CL-0018

Capítulo 22. Diseño de interfaces de los subsistemas

Este capítulo traduce los casos de uso y las clases de los capítulos 20 y 21 a la interfaz que utiliza el usuario. Se describen las ventanas, las vistas integradas, los campos y las acciones disponibles, junto con las rutas de navegación y los informes que genera cada subsistema. La organización sigue criterios de usabilidad relacionados con la claridad de las acciones, la consistencia visual y la presentación comprensible de los errores (Sharp, Rogers, & Preece, 2019; Nielsen, 1994). Las descripciones se han contrastado con las plantillas HTML y los scripts JavaScript del proyecto.

El capítulo se organiza siguiendo la misma estructura de subsistemas de diseño del capítulo 17: cada apartado corresponde a un subsistema (SD-001 a SD-006) y describe las interfaces que materializan su responsabilidad. Para cada subsistema se presentan tres elementos: el flujo de navegación, que define la navegación entre las ventanas del subsistema; la especificación de las interfaces, que documenta cada ventana mediante la ficha formal IU-NNNN con sus campos, sus botones y sus enlaces; y los informes del subsistema, que describen cada documento generado mediante la ficha formal IF-NNNN con sus datos y sus agrupaciones. Las ventanas de la aplicación corresponden a las plantillas reales del sistema: la página de inicio de sesión, el registro, el panel de diagnóstico y el laboratorio de entrenamiento, junto con las vistas que se integran en ellas.

El diagrama de la figura representa la navegación global de la aplicación. El visitante entra por la página de inicio de sesión (IU-0001), desde la que accede al registro (IU-0002); una vez autenticado o registrado, el sistema conduce al panel de diagnóstico (IU-0003), que constituye el centro de la aplicación. Desde el panel se alcanzan las vistas integradas del historial y el detalle (IU-0004), la administración (IU-0006, solo para el rol administrador), la cola de trabajos (IU-0007) y el laboratorio de entrenamiento (IU-0005); desde el laboratorio también se alcanza la cola de trabajos.

La navegación global refleja la estructura funcional de la plataforma: las ventanas públicas de acceso y registro (SD-001) preceden al resto, y el panel de diagnóstico se sitúa como punto central desde el que se accede al laboratorio y a las vistas integradas. Las vistas del historial, la administración y la cola de trabajos no constituyen ventanas independientes de la aplicación, sino secciones y diálogos integrados en el panel de diagnóstico o en el laboratorio, que se describen en los apartados de sus subsistemas correspondientes.

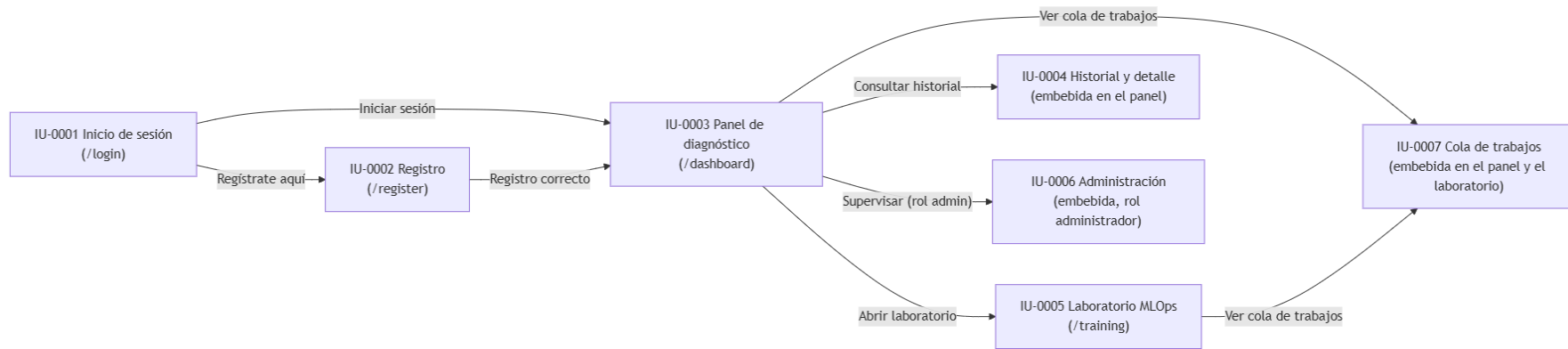


Ilustración x - Diagrama de navegación global de la aplicación

22.1. Subsistema SD-001: Acceso, identidad y gestión de sesiones

El subsistema SD-001 materializa las interfaces de acceso de la plataforma: la página de inicio de sesión y la página de registro. Ambas constituyen las únicas ventanas públicas de la aplicación, accesibles sin autenticación, y comparten los mecanismos transversales de presentación: el selector de idioma y el cambio del tema visual. La página de inicio de sesión permite al visitante autenticarse con sus credenciales, y la página de registro permite crear una cuenta; en ambos casos, la autenticación correcta conduce al panel de diagnóstico del subsistema SD-002.

22.1.1. Flujo de navegación

El flujo de navegación del subsistema SD-001, representado en la figura, refleja la progresión del visitante hasta la entrada al área privada. El visitante llega a la página de inicio de sesión, desde la que puede desplazarse a la página de registro mediante el enlace correspondiente y volver a la página de inicio desde el registro. Tras completar el inicio de sesión o el registro, el sistema conduce al usuario al panel de diagnóstico, que pertenece a SD-002; la navegación entre SD-001 y SD-002 materializa la frontera entre el área pública y el área privada.

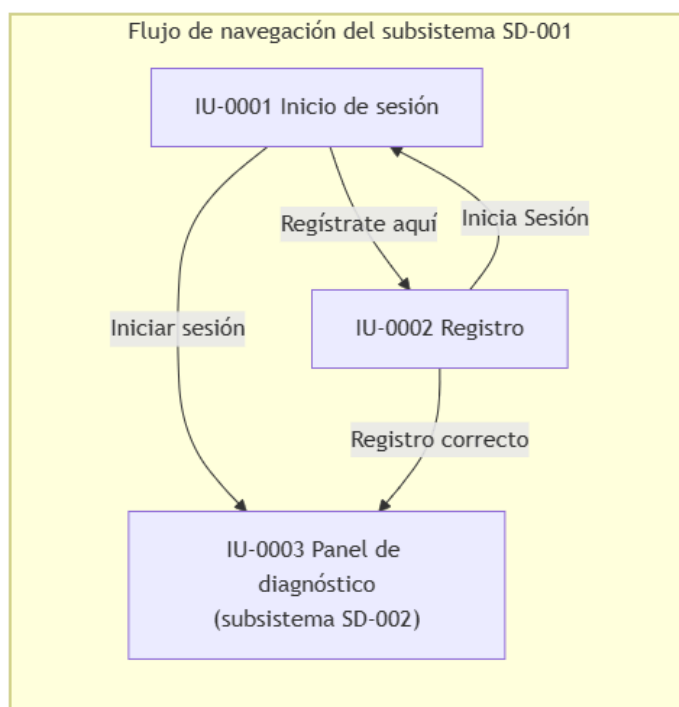


Ilustración x - Flujo de navegación del subsistema SD-001

El flujo no presenta rutas de navegación a otras ventanas del subsistema: las dos ventanas se enlazan mutuamente y ambas desembocan en el panel de diagnóstico. La página de inicio de sesión recibe el parámetro de error en la URL cuando una autenticación falla, de modo que muestra el mensaje genérico sin revelar qué parte de las credenciales era incorrecta, en coherencia con el CU-002 del análisis.

22.1.2. Especificación de las interfaces

A continuación se especifican las dos interfaces propias de SD-001.

IU-0001: Inicio de sesión					
Descripción	Ventana de acceso a la plataforma, correspondiente a la plantilla login.html (ruta /). Presenta el formulario de inicio de sesión con el nombre de usuario y la contraseña, el mensaje de error genérico, el selector de idioma y el cambio del tema visual. Ante una autenticación correcta, redirige al panel de diagnóstico; ante un fallo, muestra un mensaje genérico sin revelar la causa.				
Campos	Nombre	Tipo Datos	Editable/Consulta	Oblig.	Descripción
	username	Texto	Editable	Sí	Nombre de usuario o correo electrónico con el que se identifica la cuenta.
	password	Contraseña	Editable	Sí	Contraseña de la cuenta; se introduce enmascarada.
Botones/Enlaces	Nombre	Acción			
	Entrar al Sistema	Envía el formulario al endpoint POST /login; con credenciales válidas, el sistema establece las cookies de sesión y redirige al panel de diagnóstico; con credenciales inválidas, redirige a la página con el parámetro de error para mostrar el mensaje genérico.			
	Regístrate aquí	Navega a la página de registro (/register).			
	Selector de idioma	Cambia el idioma de la interfaz, materializando el CU-004.			
	Botón de tema	Alterna el tema claro y oscuro de la interfaz, materializando el CU-036 de SD-006.			

Tabla x - Especificación de interfaz IU-0001

La página constituye la puerta de entrada del sistema: todos los flujos privados comienzan en ella. El mensaje de error es genérico para no revelar si el fallo corresponde al nombre de usuario o a la contraseña, y el endpoint de acceso está limitado a cinco peticiones por minuto. La ventana se sirve sin almacenamiento de caché, de modo que no se reutiliza una sesión anterior.

IU-0002: Registro					
Descripción	Ventana de creación de cuenta, correspondiente a la plantilla register.html (ruta /register). Presenta el formulario de registro con el nombre, el apellido, la especialidad, el nombre de usuario y la contraseña, junto con el selector de idioma y el cambio del tema visual. Envía los datos al endpoint de registro de forma asíncrona; ante un registro correcto muestra la confirmación y conduce al panel de diagnóstico, y ante un error muestra el mensaje correspondiente.				
Campos	Nombre	Tipo Datos	Editabile/ Consulta	Oblig.	Descripción
	first_name	Texto	Editabile	Sí	Nombre del profesional.
	last_name	Texto	Editabile	Sí	Apellido del profesional.
	role	Lista de selección	Editabile	Sí	Especialidad del profesional: Médico Residente, Radiólogo Especialista, Médico de Familia o Neumólogo.
	username	Texto	Editabile	Sí	Nombre de usuario o correo electrónico con el que se identificará la cuenta.
	password	Contraseña	Editabile	Sí	Contraseña de la cuenta; el servidor exige una longitud mínima de ocho caracteres.
Botones/Enlaces	Nombre	Acción			
	Registrarme	Envía el formulario al endpoint POST /api/register de forma asíncrona; ante un registro correcto muestra la confirmación y redirige al panel de diagnóstico, y ante un error muestra un mensaje genérico o de usuario duplicado.			
	Inicia Sesión	Navega a la página de inicio de sesión (/).			
	Selector de idioma	Cambia el idioma de la interfaz, materializando el CU-004.			
	Botón de tema	Alterna el tema claro y oscuro de la interfaz, materializando el CU-036 de SD-006.			

Tabla x - Especificación de interfaz IU-0002

La ventana mantiene la puerta de entrada autónoma del análisis: el alta no requiere la intervención de un administrador. La validación del formato del correo, de la longitud de la contraseña y de la unicidad del usuario se resuelve en el servidor, y la interfaz muestra una confirmación o un error sin interrumpir la sesión de navegación. El servidor recibe también el campo role, por lo que la restricción del rol inicial debe comprobarse en el servidor y no solo en las opciones visibles del selector. Tras el registro correcto, el sistema establece las cookies de sesión y el visitante queda autenticado, en coherencia con el CU-001 y con la decisión de diseño del capítulo 20.

22.1.3. Informes del subsistema

El subsistema SD-001 no genera informes: sus ventanas se limitan a la autenticación y a la creación de cuentas, y no producen documentos descargables. Los informes de la plataforma corresponden a los subsistemas funcionales que generan resultados: el informe PDF del diagnóstico, que se describe en el apartado de SD-002, y el informe consolidado de las sesiones de entrenamiento, que se describe en el apartado de SD-004. Por esta razón, no se definen fichas IF-NNNN para este subsistema.

22.2. Subsistema SD-002: Diagnóstico asistido y generación de resultados

El subsistema SD-002 materializa la interfaz clínica de la plataforma: el panel de diagnóstico, correspondiente a la plantilla `dashboard.html` (ruta `/dashboard`), que constituye el entorno central de la aplicación y sobre el que se realiza el ciclo completo de una consulta clínica. La ventana integra además las vistas del historial y el detalle de las consultas, la cola de trabajos y el panel de administración, que pertenecen a los subsistemas SD-003, SD-006 y SD-005, de modo que la ficha de interfaz se centra en las funciones de diagnóstico y se remite a los apartados correspondientes para el resto de las vistas. El subsistema genera además el informe PDF del diagnóstico (IF-0001), que se produce durante el procesamiento de la consulta.

22.2.1. Flujo de navegación

El flujo de navegación del subsistema SD-002, representado en la figura, sitúa el panel de diagnóstico como el punto central de la navegación del usuario autenticado. El usuario accede al panel tras el inicio de sesión o el registro, y desde él realiza el ciclo de diagnóstico sin abandonar la ventana: carga la radiografía, selecciona el modelo, solicita el análisis y visualiza el resultado y los mapas de explicabilidad en la misma vista. Desde el panel se alcanzan el historial y el detalle de las consultas (SD-003), el laboratorio de entrenamiento (SD-004) y la cola de trabajos (SD-006).

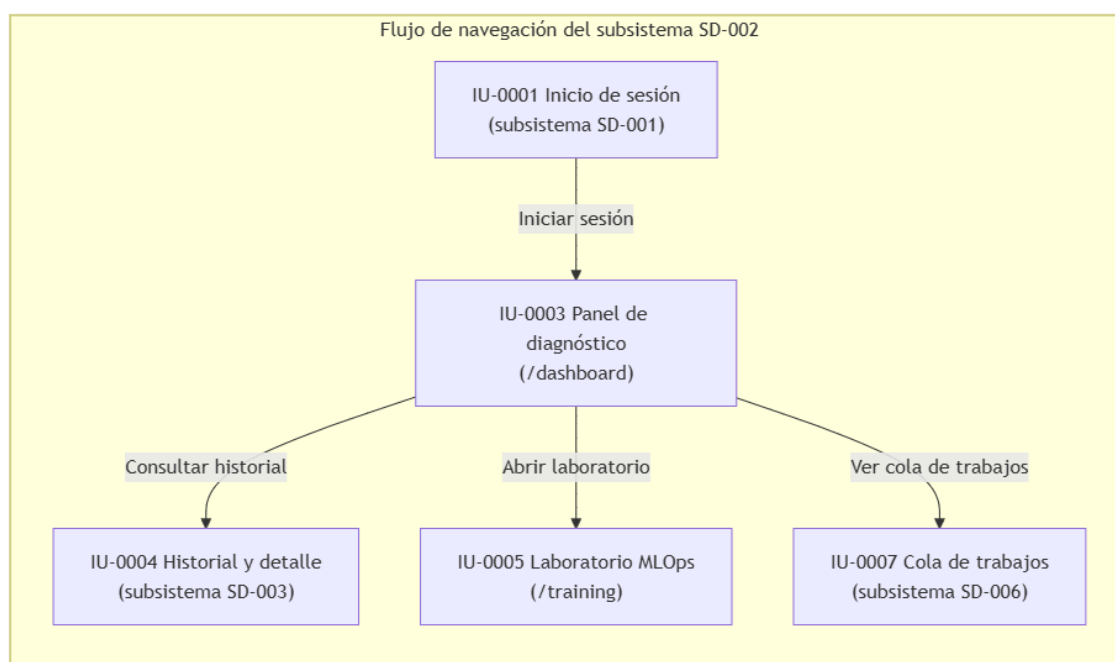


Ilustración x - Flujo de navegación del subsistema SD-002

El flujo refleja la naturaleza central del panel de diagnóstico: el ciclo clínico se resuelve en una única ventana, sin transiciones intermedias, y el resto de los subsistemas se alcanzan desde el panel mediante la barra lateral. La navegación hacia el historial y el detalle, hacia el laboratorio y hacia la cola de trabajos materializa las relaciones de colaboración entre SD-002 y los subsistemas SD-003, SD-004 y SD-006 descritas en el capítulo 17.

22.2.2. Especificación de las interfaces

A continuación se especifica la interfaz propia de SD-002, centrada en las funciones de diagnóstico; las vistas integradas del historial, la cola y la administración se especifican en los apartados de sus subsistemas.

IU-0003: Panel de diagnóstico					
Descripción	Ventana central de la aplicación, correspondiente a la plantilla dashboard.html (ruta /dashboard). Presenta el entorno de diagnóstico rápido: la zona de carga de la radiografía, el selector del modelo de inteligencia artificial, el área de conversación donde se muestra el resultado del análisis y los mapas de explicabilidad, y el visor de imágenes. Integra en su barra lateral la navegación al laboratorio, el historial de consultas, el panel de la cola de trabajos, el acceso a la administración (solo para el rol administrador) y el cierre de sesión. Solo es accesible para usuarios autenticados.				
Campos	Nombre	Tipo Datos	Editable/ Consulta	Oblig.	Descripción
	model-selector	Lista de selección	Editable	Sí	Arquitectura de inteligencia artificial para el diagnóstico, entre las CNN Clásicas disponibles (DenseNet121, ResNet50, MobileNetV2, EfficientNetB0 y ConvNeXtTiny).
	file-input	Archivo	Editable	Sí	Radiografía de tórax en formato JPEG o PNG, de hasta 10 MB, que se adjunta mediante la zona de carga.
	Área de resultado	Contenido	Consulta	N/A	Zona de la ventana que muestra la imagen enviada, el resultado del diagnóstico (etiqueta y confianza), el modelo empleado y el mapa de explicabilidad generado.
Botones/Enlaces	Nombre	Acción			
	Zona de carga	Permite seleccionar o arrastrar la radiografía de tórax; al adjuntarla habilita el envío.			
	Enviar	Envía la solicitud de diagnóstico al endpoint POST /predict, muestra la posición del trabajo en la cola y sondea su estado hasta que se completa.			
	Diagnóstico Rápido	Recarga el panel de diagnóstico (GET /dashboard).			
	Laboratorio de Entrenamiento	Navega al laboratorio de experimentación MLOps (GET /training), del subsistema SD-004.			
	Historial	Muestra el listado de consultas del usuario en la barra lateral, correspondiente a SD-003.			
	Cerrar Sesión	Cierra la sesión del usuario (GET /logout), del subsistema SD-001.			

	Selector de idioma	Cambia el idioma de la interfaz, materializando el CU-004.
	Botón de tema	Alterna el tema claro y oscuro de la interfaz, materializando el CU-036 de SD-006.

Tabla x - Especificación de interfaz IU-0003

La ventana constituye el entorno de trabajo del profesional sanitario: el diagnóstico se solicita y se visualiza sin abandonar la vista, y la interfaz permanece operativa durante el procesamiento asíncrono mediante el sondeo del estado de la cola. La respuesta del sistema localiza las etiquetas según el idioma de la sesión. La ventana integra las vistas del historial, la cola y la administración, que se especifican en los apartados de SD-003, SD-006 y SD-005 respectivamente.

22.2.3. Informes del subsistema

A continuación se especifica el informe propio de SD-002.

IF- 0001: Informe del diagnóstico				
Descripción	Informe PDF de una consulta de diagnóstico, generado durante el procesamiento de la consulta mediante services/pdf_generator.py. Recoge la fecha, el modelo empleado, el diagnóstico con su nivel de confianza, la radiografía original y el mapa de explicabilidad. La ruta del documento se conserva en la consulta, aunque el detalle actual del panel no incluye un control visible para descargarlo.			
Módulo de Interfaz	IU-0003 Panel de diagnóstico.			
Datos	Campo	Ordenación	Tipo Datos	Descripción
	Fecha	N/A	Fecha/hora	Fecha y hora de la consulta.
	Modelo	N/A	Texto	Arquitectura de inteligencia artificial empleada en el diagnóstico.
	Diagnóstico	N/A	Texto	Etiqueta de la predicción (Neumonía o Normal), con el color de diagnóstico acorde con el resultado.
	Confianza	N/A	Numérico	Nivel de confianza de la predicción.
	Radiografía original	N/A	Imagen	Radiografía de tórax enviada por el usuario.
	Mapa de explicabilidad	N/A	Imagen	Mapa de calor XAI que justifica la predicción.
Resumen/Acumulado	Resumen			Campos del Resumen
	N/A			N/A

Tabla x - Especificación de informe IF-0001

El informe se genera durante el procesamiento del diagnóstico y su ruta se almacena junto con la consulta. Sin embargo, el detalle de la consulta no muestra actualmente un botón de descarga, y el fichero se encuentra bajo una ruta estática que no aplica una comprobación de propiedad equivalente a la del historial. Por tanto, la protección efectiva del documento requiere una ruta de descarga autorizada antes de utilizar datos sensibles fuera del entorno de demostración.

22.3. Subsistema SD-003: Historial y gestión de consultas

El subsistema SD-003 materializa la interfaz del historial y el detalle de las consultas, integrada en el panel de diagnóstico. La vista del historial ocupa la barra lateral del panel, donde se muestran las consultas del usuario agrupadas por modelo, y la vista del detalle ocupa el área principal al abrir una consulta, donde se presentan los artefactos y los metadatos de la consulta junto con las acciones de renombrado y eliminación. Ambas vistas constituyen la interfaz IU-0004, correspondiente a las secciones del historial y del detalle de la plantilla dashboard.html, y materializan los casos de uso CU-011 a CU-014.

22.3.1. Flujo de navegación

El flujo de navegación del subsistema SD-003, representado en la figura, refleja la cadena de navegación del análisis: desde el panel de diagnóstico se accede al historial, desde el listado se abre el detalle de una consulta, y desde el detalle se ejecutan, de forma opcional, el renombrado o la eliminación, además de la vuelta al listado. La navegación entre el listado, el detalle y las operaciones de gestión reproduce las relaciones de extensión de los casos de uso del subsistema.

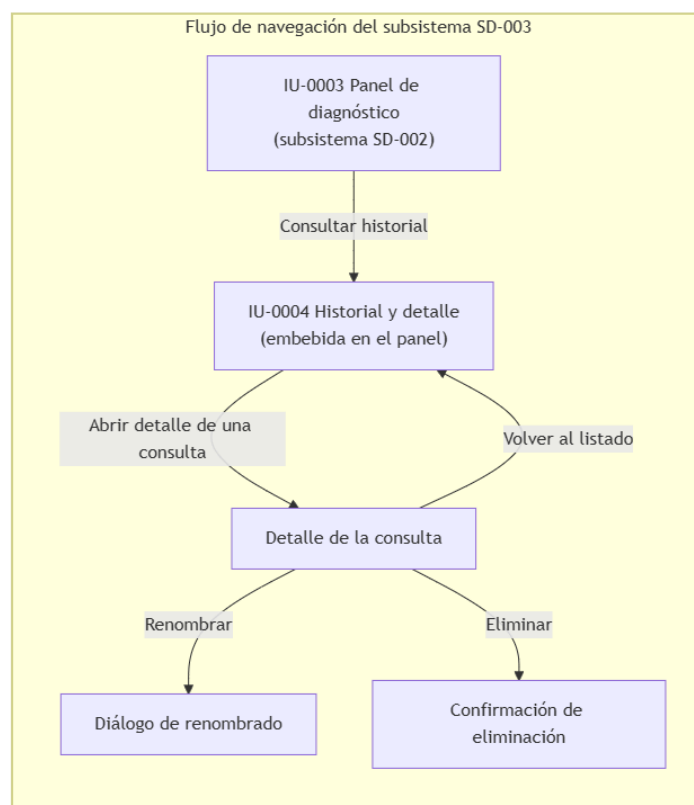


Ilustración x - Flujo de navegación del subsistema SD-003

El flujo refleja la progresión del usuario por el historial: el listado conduce al detalle de una consulta, y desde el detalle se alcanzan las operaciones opcionales de gestión. La navegación hacia el detalle y hacia las operaciones es de ida y vuelta: el usuario puede volver al listado o al panel en cualquier momento, sin que las acciones de gestión interrumpan la navegación principal.

22.3.2. Especificación de las interfaces

A continuación se especifica la interfaz propia de SD-003.

IU-0004: Historial y detalle					
Descripción	Vista integrada del historial y el detalle de las consultas, correspondiente a las secciones del historial y del detalle de la plantilla dashboard.html. La vista del historial ocupa la barra lateral del panel de diagnóstico y muestra las consultas del usuario agrupadas por modelo, con la imagen, la fecha, la etiqueta y la confianza de cada una. La vista del detalle ocupa el área principal al abrir una consulta y muestra la radiografía original, el mapa de explicabilidad, el diagnóstico, la confianza, el modelo, el nombre visible, la fecha y las acciones de renombrado y eliminación.				
Campos	Nombre	Tipo Datos	Editable/Consulta	Oblig.	Descripción
	Listado de consultas	Contenido	Consulta	N/A	Listado de las consultas del usuario, agrupado por modelo, con la imagen, la fecha, la etiqueta y la confianza de cada consulta.
	cd-original	Imagen	Consulta	N/A	Radiografía original de la consulta seleccionada.
	cd-xai	Imagen	Consulta	N/A	Mapa de explicabilidad XAI de la consulta seleccionada.
	cd-label	Texto	Consulta	N/A	Etiqueta de la predicción (Neumonía o Normal) de la consulta.
	cd-confi-dence	Númérico	Consulta	N/A	Nivel de confianza de la predicción.
	cd-model	Texto	Consulta	N/A	Arquitectura del modelo empleado en la consulta.
	cd-patient	Texto	Consulta	N/A	Nombre visible de la consulta, que funciona como etiqueta de organización y no como identificación clínica.
	cd-timesta-mp	Fecha/hora	Consulta	N/A	Fecha de la consulta.
Botones/Enlaces	Nombre		Acción		
	Tarjeta del historial		Abre el detalle de la consulta seleccionada, materializando el CU-012.		

	Volver	Cierra el detalle y regresa al listado del historial.
	Renombrar	Abre el diálogo para indicar el nuevo nombre visible de la consulta y lo actualiza, materializando el CU-013.
	Eliminar	Solicita la confirmación de la eliminación y, si se confirma, elimina la fila de la consulta, materializando el CU-014. Los ficheros asociados no se eliminan explícitamente en la implementación actual.

Tabla x - Especificación de interfaz IU-0004

La vista del historial solo muestra las consultas del usuario propietario, de modo que el acceso al detalle se restringe mediante el filtro del listado. La vista del detalle no requiere una petición adicional al servidor: se construye con los datos recibidos en el listado y las imágenes se sirven desde el almacenamiento estático. Esta última ruta no aplica una comprobación de propiedad equivalente. El informe del diagnóstico se genera en SD-002 y su ruta se conserva en la consulta, pero el detalle actual no incluye un control visible para descargarlo.

22.3.3. Informes del subsistema

El subsistema SD-003 no genera informes propios: su responsabilidad es la recuperación y la gestión de las consultas ya realizadas, y no produce documentos descargables. El informe del diagnóstico de cada consulta (IF-0001) pertenece al subsistema SD-002, que lo genera durante el procesamiento, de modo que el historial únicamente conserva y sirve la ruta del documento ya generado. Por esta razón, no se definen fichas IF-NNNN para este subsistema.

22.4. Subsistema SD-004: Laboratorio de experimentación MLOps

El subsistema SD-004 materializa la interfaz del laboratorio de experimentación, correspondiente a la plantilla training.html (ruta /training). La ventana integra el asistente de configuración conversacional, la exploración de la carpeta del dataset, el listado de los experimentos guardados, la consola de entrenamiento, la vista de resultados de la sesión (con el ranking, la matriz de Wilcoxon y la validación externa) y la vista de resultados de un modelo, con las métricas de validación cruzada, la calibración y las métricas XAI. La ventana integra además la cola de trabajos y el acceso a la administración, pertenecientes a SD-006 y SD-005. El subsistema genera el informe consolidado de la sesión (IF-0002).

22.4.1. Flujo de navegación

El flujo de navegación del subsistema SD-004, representado en la figura, se organiza en torno al laboratorio como ventana principal. El usuario accede al laboratorio desde el panel de diagnóstico, y dentro de la ventana navega entre el asistente de configuración, la vista de resultados de la sesión y la vista de resultados de un modelo, además de la consola de entrenamiento que se muestra durante la ejecución. Desde el laboratorio se alcanzan la cola de trabajos y, en sentido inverso, el panel de diagnóstico.

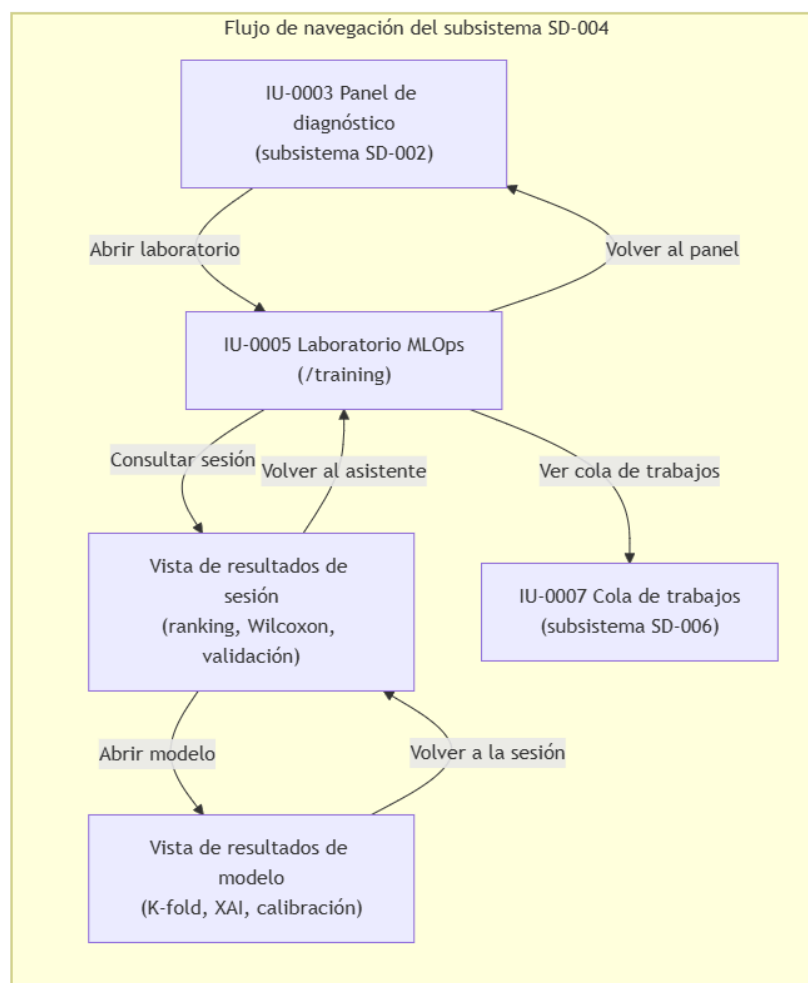


Ilustración x - Flujo de navegación del subsistema SD-004

El flujo refleja la progresión del investigador por el laboratorio: desde el asistente de configuración se lanzan los experimentos, desde el listado de experimentos se consulta la sesión, y desde la vista de la sesión se desciende a los resultados de un modelo concreto. La navegación entre las vistas es de ida y vuelta, y el usuario puede volver al asistente o al panel en cualquier momento sin interrumpir el estado del laboratorio.

22.4.2. Especificación de las interfaces

A continuación se especifica la interfaz propia de SD-004.

IU-0005: Laboratorio de entrenamiento					
Descripción	Ventana del laboratorio de experimentación MLOps, correspondiente a la plantilla training.html (ruta /training). Presenta el asistente de configuración conversacional, la exploración de la carpeta del dataset, el listado de los experimentos guardados, la consola del servidor de entrenamiento, la vista de resultados de la sesión (ranking por AUC, matriz de Wilcoxon, validación externa con curvas ROC y matriz de DeLong, y exploración de modelos) y la vista de resultados de un modelo (validación cruzada K-fold, calibración y métricas XAI). Solo es accesible para usuarios autenticados.				
Campos	Nombre	Tipo Datos	Editabl e/ Consult a	Obli g.	Descripción
	chat-input	Texto	Editabl e	Sí	Mensaje en lenguaje natural dirigido al asistente de configuración del experimento.
	Listado de experimentos	Contenido	Consulta	N/A	Listado de las sesiones de entrenamiento del usuario, con sus modelos, en la barra lateral.
	session-ranking-table	Tabla	Consulta	N/A	Ranking de los modelos de la sesión por la media del AUC de la validación cruzada, con su desviación.
	session-heatmap-img	Imagen	Consulta	N/A	Matriz de significancia estadística del test de Wilcoxon.
	external-ranking-table	Tabla	Consulta	N/A	Rendimiento de la validación externa por modelo: exactitud, F1 y AUC.
	external-roc-img	Imagen	Consulta	N/A	Curvas ROC de la validación externa.
	external-delong-img	Imagen	Consulta	N/A	Matriz de significancia de DeLong.
	res-table-body	Tabla	Consulta	N/A	Resultados de la validación cruzada K-fold de un modelo (exactitud, precisión, sensibilidad, F1 y AUC por pliegue).

	res-xai-table	Tabla	Consulta	N/A	Métricas cuantitativas de explicabilidad del modelo (deletion, insertion, sparsity, entropy y stability por método).
	console-logs	Contenido	Consulta	N/A	Registro de la ejecución del entrenamiento, actualizado durante el procesamiento.
Botones/Enlaces	Nombre		Acción		
	Explorar Carpeta		Abre el explorador del dataset e inserta la ruta seleccionada en la conversación, materializando el CU-017.		
	Enviar		Envía el mensaje al asistente de configuración, materializando el CU-016; si la configuración está completa, lanza el entrenamiento (CU-018).		
	Actualizar		Refresca el listado de experimentos guardados.		
	Volver al Asistente		Oculta la vista de resultados de la sesión y regresa al asistente de configuración.		
	Renombrar		Cambia el nombre visible de la sesión, materializando el CU-029.		
	Reutilizar Configuración		Reutiliza la configuración de la sesión actual en una nueva conversación.		
	Validación Externa		Solicita la validación externa de la sesión, materializando el CU-026.		
	Generar Reporte PDF		Descarga el informe consolidado de la sesión, materializando el CU-028.		
	Recalcular Wilcoxon		Solicita el recálculo de la comparativa estadística de la sesión, materializando el CU-024.		
	Volver a la Sesión		Regresa desde los resultados de un modelo a la vista de la sesión.		
	Generar XAI y Métricas		Ejecuta el análisis de explicabilidad de un modelo, materializando el CU-025.		
	Diagnóstico Rápido		Navega al panel de diagnóstico (GET /dashboard), del subsistema SD-002.		
	Cerrar Sesión		Cierra la sesión del usuario (GET /logout), del subsistema SD-001.		
	Selector de idioma		Cambia el idioma de la interfaz, materializando el CU-004.		
	Botón de tema		Alterna el tema claro y oscuro de la interfaz, materializando el CU-036 de SD-006.		

Tabla x - Especificación de interfaz IU-0005

El informe se genera bajo demanda, cuando la sesión dispone de los datos necesarios, y separa el conocimiento del formato del router: el generador recibe la información preparada por el motor y compone el documento. El informe consolida el análisis estadístico de la sesión y se considera parte de los artefactos de experimentación del laboratorio.

22.4.3. Informes del subsistema

A continuación se especifica el informe propio de SD-004.

IF- 0002: Informe de la sesión de entrenamiento				
Descripción	Informe PDF consolidado de una sesión de entrenamiento, generado bajo demanda mediante <code>services/pdf_generator_mlops.py</code> . Recoge la configuración del experimento, el ranking de modelos con la matriz de Wilcoxon, los resultados de la validación externa con las curvas ROC y la matriz de DeLong, y el detalle técnico de cada modelo con sus métricas de explicabilidad y sus mapas de calor. Se entrega como documento descargable desde la vista de la sesión.			
Módulo de Interfaz	IU-0005 Laboratorio de entrenamiento.			
Datos	Campo	Ordenación	Tipo Datos	Descripción
	ID de sesión	N/A	Texto	Identificador de la sesión de entrenamiento.
	Fecha	N/A	Fecha/hora	Fecha de generación del informe.
	Dataset	N/A	Texto	Ruta del dataset de entrenamiento de la sesión.
	Modelos	N/A	Texto	Arquitecturas entrenadas en la sesión.
	Hiperparámetros	N/A	Texto	Épocas, tamaño de lote y tasa de aprendizaje de la sesión.
	Ranking	1	Tabla	Modelos ordenados por la media del AUC de la validación cruzada, con su desviación típica.
	Matriz de Wilcoxon	N/A	Imagen	Matriz de significancia estadística entre los modelos.
	Validación externa	N/A	Tabla	Exactitud, F1 y AUC de cada modelo sobre la cohorte externa.
	Curvas ROC	N/A	Imagen	Curvas ROC de la validación externa.
	Matriz de DeLong	N/A	Imagen	Matriz de comparación estadística de las curvas ROC.

	Métricas XAI	N/A	Tabla	Métricas de fidelidad de la explicabilidad por modelo y método.
	Mapas de calor XAI	N/A	Imagen	Mapas de explicabilidad de cada modelo sobre imágenes de ejemplo.
Resumen/Acumulado	Resumen			Campos del Resumen
	Ranking de modelos			Modelo, Media AUC, Desviación

Tabla x - Especificación de informe IF-0002

El informe se trata como parte de la información protegida de la consulta: se genera exclusivamente a partir de los datos del usuario propietario y su acceso queda sujeto al control de propiedad del historial. La generación del documento se adelanta al momento del diagnóstico, de modo que la descarga posterior no depende de una nueva operación del sistema.

22.5. Subsistema SD-005: Supervisión y administración

El subsistema SD-005 materializa la interfaz de administración, integrada en las ventanas del panel de diagnóstico y del laboratorio de entrenamiento mediante los diálogos de administración de las plantillas dashboard.html y training.html. La vista de administración es accesible únicamente para el usuario con rol de administrador y presenta el listado de usuarios con sus recuentos de actividad, las consultas de un usuario concreto junto con sus sesiones del laboratorio, y el detalle de una consulta seleccionada. Constituye la interfaz IU-0006 y materializa los casos de uso CU-031 a CU-033.

22.5.1. Flujo de navegación

El flujo de navegación del subsistema SD-005, representado en la figura 95, refleja la progresión del administrador desde el listado general hasta el caso concreto, en coherencia con las relaciones de extensión del análisis. Desde el panel de diagnóstico o el laboratorio, el administrador abre el diálogo de administración con el listado de usuarios; al seleccionar un usuario se muestran sus consultas y sus sesiones del laboratorio, y desde el listado de consultas se abre el detalle completo de una consulta, con la posibilidad de volver en cada paso.

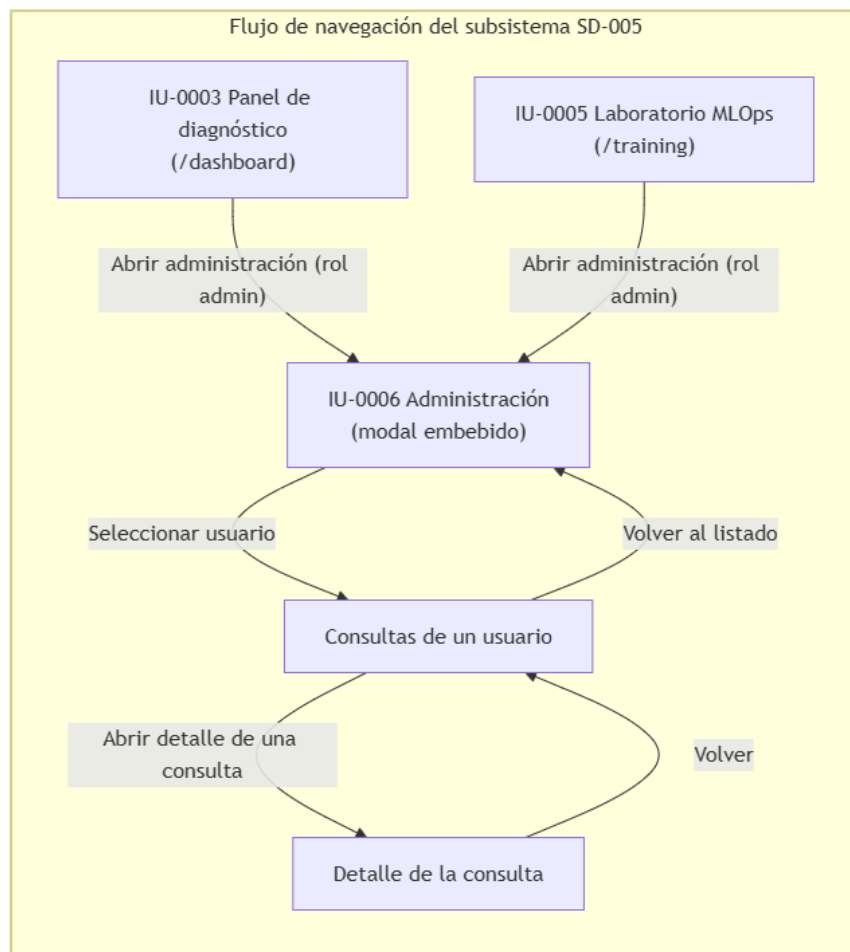


Ilustración x - Flujo de navegación del subsistema SD-005

El flujo refleja la cadena de navegación de la supervisión: el listado de usuarios conduce a la actividad de una cuenta, y desde la actividad se desciende al detalle de una consulta individual. La navegación es de ida y vuelta en cada nivel, de modo que el administrador puede profundizar progresivamente en la información y regresar al nivel anterior sin perder el contexto de la supervisión. El acceso al diálogo de administración queda restringido al rol administrador, y su presencia en la barra lateral depende de la comprobación de rol que realiza el servidor.

22.5.2. Especificación de las interfaces

A continuación se especifica la interfaz propia de SD-005.

IU-0006: Laboratorio de entrenamiento					
Descripción	Vista de supervisión y administración, integrada mediante los diálogos de administración de las plantillas dashboard.html y training.html. Presenta el listado de usuarios con el número de diagnósticos y de sesiones del laboratorio de cada uno, las consultas de un usuario concreto junto con sus sesiones de entrenamiento, y el detalle de una consulta con su imagen, su mapa de explicabilidad, su resultado, su confianza y sus metadatos. Solo es accesible para el usuario con rol de administrador.				
Campos	Nombre	Tipo Datos	Editabl e/ Consult a	Obli g.	Descripción
	admin-us ers-list	Conteni do	Consu lta	N/A	Listado de usuarios registrados con el número de diagnósticos y de sesiones del laboratorio de cada uno.
	admin-us er-consul tations-li st	Conteni do	Consu lta	N/A	Consultas de diagnóstico del usuario seleccionado, con sus sesiones de entrenamiento del laboratorio.
	Detalle de consulta	Conteni do	Consu lta	N/A	Detalle completo de la consulta seleccionada: radiografía original, mapa de explicabilidad, diagnóstico, confianza, modelo, nombre y fecha.
Botones/Enla ces	Nombre		Acción		
	Panel de Administración		Abre el diálogo de administración con el listado de usuarios; solo se muestra al usuario con rol administrador.		
	Usuario del listado		Abre las consultas y las sesiones del laboratorio del usuario seleccionado, materializando el CU-032.		
	Consulta del listado		Abre el detalle completo de la consulta seleccionada, materializando el CU-033.		
	Cerrar		Cierra el diálogo de administración y regresa a la ventana desde la que se abrió.		

Tabla x - Especificación de interfaz IU-0006

La vista de administración reutiliza el detalle de la consulta del panel de diagnóstico para presentar la consulta supervisada, de modo que la interfaz administrativa no duplica la vista del historial. La autorización administrativa se comprueba en el servidor antes de servir los datos, y no se infiere de la interfaz. El acceso del administrador a los datos de otros usuarios constituye la excepción al aislamiento de datos, acotada a la función de supervisión.

22.5.3. Informes del subsistema

El subsistema SD-005 no genera informes: su responsabilidad es la consulta y la supervisión de la actividad de la plataforma, y no produce documentos descargables. Los informes de la plataforma corresponden a los subsistemas que generan resultados, SD-002 y SD-004, y el administrador puede consultarlos a través de las consultas y las sesiones que supervisa, sin generar informes propios. Por esta razón, no se definen fichas IF-NNNN para este subsistema.

22.6. Subsistema SD-006: Cola de trabajos y capacidades transversales

El subsistema SD-006 materializa la interfaz de la cola de trabajos, integrada como panel lateral en las ventanas del panel de diagnóstico y del laboratorio de entrenamiento. El panel muestra los trabajos asíncronos del usuario, ya sean diagnósticos, entrenamientos o validaciones externas, mientras permanecen encolados o en ejecución, e indica su tipo, su estado y su posición. También permite cancelar los trabajos pendientes, materializando los casos de uso CU-034 y CU-035. Además del panel de la cola, el subsistema comprende los mecanismos transversales de presentación de la interfaz: el selector de idioma y el cambio del tema visual, que ya se describieron como acciones comunes en las fichas de interfaz de los subsistemas anteriores.

22.6.1. Flujo de navegación

El flujo de navegación del subsistema SD-006, representado en la figura 96, refleja el carácter transversal de la cola de trabajos: el panel se integra en el panel de diagnóstico y en el laboratorio de entrenamiento, de modo que el usuario puede consultar el estado de sus trabajos desde cualquiera de los dos núcleos funcionales sin abandonar su ventana. Desde el panel de la cola se ejecuta la cancelación de un trabajo pendiente, que lo marca como cancelled.

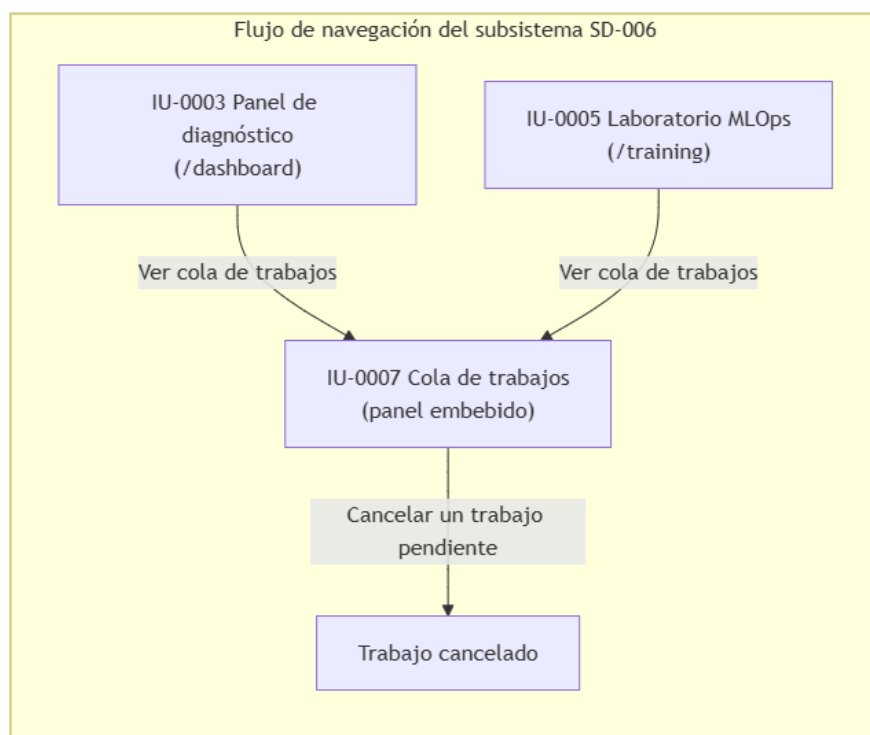


Ilustración x - Flujo de navegación del subsistema SD-006

El flujo refleja la integración transversal del panel de la cola: no constituye una ventana independiente, sino una sección que se muestra en las ventanas de los subsistemas que generan trabajos. La actualización del panel es periódica, de modo que el estado de los trabajos se refleja sin necesidad de navegación adicional, y la cancelación de un trabajo pendiente se resuelve desde el propio panel, sin transiciones intermedias.

22.6.2. Especificación de las interfaces

A continuación se especifica la interfaz propia de SD-006.

IU-0007: Cola de trabajos					
Descripción	Panel lateral de la cola de trabajos, integrado en las ventanas del panel de diagnóstico (dashboard.html) y del laboratorio de entrenamiento (training.html). Muestra los trabajos asincrónicos del usuario mientras están encolados o en ejecución, con su tipo y la posición de los trabajos encolados, y permite cancelar los trabajos pendientes. El panel se actualiza de forma periódica y permanece oculto cuando no hay trabajos pendientes o en ejecución. Los estados completado y fallido se muestran en el mensaje del diagnóstico o en la consola del laboratorio, no como filas persistentes del panel.				
Campos	Nombre	Tipo Datos	Editable/Consulta	Oblig.	Descripción
	queue-items	Contenido	Consulta	N/A	Listado temporal de los trabajos del usuario que están encolados o en ejecución, con su tipo y posición cuando corresponde.
	Estado del trabajo	Texto	Consulta	N/A	Indicación de que el trabajo está en ejecución o de su posición cuando permanece encolado.
Botones/Enlaces	Nombre	Acción			
	Cancelar	Cancela un trabajo pendiente de la cola, tras la confirmación del usuario, mediante el endpoint DELETE /api/queue/cancel/{job_id}; solo se muestra para los trabajos en estado encolado, materializando el CU-035.			

Tabla x - Especificación de interfaz IU-0006

El panel de la cola refleja la máquina de estados de los trabajos y solo ofrece la cancelación para los trabajos pendientes: un trabajo en ejecución no se interrumpe, en coherencia con el diseño de SD-006. La actualización periódica del panel permite al usuario conocer en qué momento estará disponible un resultado o si un trabajo ha fallado, materializando el CU-034. Los mecanismos transversales de idioma y de tema visual se integran en las cabeceras de las ventanas, tal y como se describió en las fichas de interfaz de los subsistemas anteriores.

22.6.3. Informes del subsistema

El subsistema SD-006 no genera informes: su responsabilidad es la ejecución asíncrona de los trabajos y las capacidades transversales de presentación, y no produce documentos descargables. Los informes de la plataforma corresponden a los subsistemas que generan resultados, SD-002 y SD-004, y la cola de trabajos únicamente refleja su estado de procesamiento. Por esta razón, no se definen fichas IF-NNNN para este subsistema.

22.7. Criterios transversales de accesibilidad

La accesibilidad afecta a todas las interfaces y no puede quedar limitada a una única ventana. El diseño toma como referencia las pautas WCAG 2.2 en el nivel AA, conforme al requisito RNF-026. La tabla siguiente resume, con redacción propia, los criterios que resultan relevantes para estas vistas y los convierte en comprobaciones aplicables al proyecto. Los umbrales y requisitos normativos proceden de WCAG 2.2 (W3C, 2023).

Área	Criterio de diseño	Estado de verificación
Navegación por teclado	Todas las acciones disponibles con ratón deben poder ejecutarse mediante teclado, siguiendo un orden lógico y sin trampas de foco.	Pendiente de revisión completa.
Foco visible	Los controles interactivos deben conservar un indicador de foco visible y distinguible sobre fondos claros y oscuros.	Pendiente de revisión homogénea.
Formularios	Cada campo debe tener una etiqueta asociada y los errores deben identificarse mediante texto comprensible y asociado al campo afectado.	Parcialmente presente en las plantillas; pendiente de auditoría global.
Lectores de pantalla	Los botones, enlaces, selectores, diálogos, estados de trabajos e imágenes deben tener un nombre o descripción accesible. Los cambios dinámicos relevantes deben comunicarse sin depender exclusivamente del color.	Pendiente de verificación.
Imágenes y artefactos	Las imágenes informativas deben incluir texto alternativo; las imágenes decorativas deben poder ignorarse; los mapas y resultados deben conservar una alternativa textual cuando aporten información.	Pendiente de revisión de todas las vistas.

Área	Criterio de diseño	Estado de verificación
Contraste	El texto normal debe alcanzar una relación mínima de 4,5:1 y el texto grande una relación mínima de 3:1. Los componentes gráficos y los indicadores necesarios deben alcanzar una relación mínima de 3:1.	Pendiente de medición sobre la interfaz final.
Tamaño y adaptación	El contenido debe seguir siendo utilizable al ampliar el texto y en las resoluciones de referencia del requisito RNF-025, sin pérdida de información ni desplazamiento horizontal innecesario.	Pendiente de verificación conjunta con RNF-025.

Tabla x - Criterios transversales de accesibilidad

La implementación actual ya utiliza etiquetas en los formularios y algunos estados de foco y textos alternativos, pero esos indicios no permiten afirmar la conformidad completa con WCAG 2.2 AA. La comprobación deberá realizarse sobre las vistas principales en sus temas claro y oscuro, incluyendo navegación por teclado, revisión de nombres accesibles y medición del contraste. Mientras no se complete esa revisión, RNF-026 debe considerarse diseñado como criterio de calidad, pero pendiente de verificación.

Capítulo 23. Especificación de la construcción del sistema

Este capítulo concreta cómo se prepara y se pone en marcha el sistema definido en los capítulos anteriores. Describe el entorno tecnológico, la organización de los paquetes y componentes, el procedimiento de construcción y la generación del esquema de base de datos. Con ello se completa la parte de diseño de la memoria y se deja establecido el punto de partida para las pruebas y la implantación (Sharp, Rogers, & Preece, 2019).

El capítulo se organiza en cuatro apartados: el entorno tecnológico de construcción, que describe las herramientas, las dependencias y las restricciones del entorno; los paquetes y componentes de construcción, que representan la estructura del software y sus dependencias; el proceso de construcción, que especifica cómo se prepara el entorno y se lanza la aplicación; y la generación del esquema de base de datos, que describe cómo se materializa el modelo físico definido en el capítulo 19. El contenido se apoya en la arquitectura de soporte del capítulo 18, que establece los subsistemas de soporte sobre los que se asienta la construcción.

La construcción de vitalXAI no produce ejecutables compilados: la aplicación es un sistema Python interpretado, servido por un proceso ASGI y apoyado en un conjunto de dependencias declaradas. Por esta razón, el proceso de construcción consiste en preparar el entorno virtual, instalar las dependencias, configurar las variables de entorno y lanzar el servidor y el worker de la cola, sin una fase de compilación ni de generación de paquetes instalables. Esta naturaleza condiciona las especificaciones de los apartados siguientes, que se centran en la reproducibilidad del entorno y en la configuración necesaria para que la aplicación funcione.

23.1. *Entorno tecnológico de construcción*

El entorno de construcción de vitalXAI reúne las herramientas necesarias para ejecutar y mantener la aplicación. El sistema se construye sobre Python 3.11, que constituye el lenguaje principal y alberga tanto el servidor web como el motor de aprendizaje profundo, y emplea el framework FastAPI para la capa HTTP, servido por el servidor ASGI Uvicorn (FastAPI, 2024; Uvicorn, 2024). La presentación se compone con el motor de plantillas Jinja2 y los recursos estáticos HTML, CSS y JavaScript; el almacenamiento relacional se apoya en MySQL, accedido a través del conector oficial de Python, y en el entorno de desarrollo se ejecuta mediante MariaDB a través de XAMPP (Oracle, 2024). El aprendizaje profundo se apoya en TensorFlow y en la librería Transformers de Hugging Face para las arquitecturas Transformer (TensorFlow, 2024; Hugging Face, 2024), junto con OpenCV para el preprocesamiento de las imágenes y matplotlib para la generación de los mapas de explicabilidad. Las credenciales de sesión se gestionan con bcrypt y python-jose, la limitación de peticiones con slowapi, los informes PDF con FPDF (FPDF2, 2024) y la configuración conversacional con el cliente del proveedor Groq (Groq, 2024).

La gestión de las dependencias se resuelve mediante el archivo requirements.txt, que fija las versiones de las librerías incluidas y favorece la reproducción del entorno de construcción. La instalación se realiza sobre un entorno virtual de Python, y las dependencias principales se agrupan en tres bloques: servidor web (FastAPI, Uvicorn, Jinja2, slowapi, python-multipart, bcrypt y python-jose), aprendizaje automático y análisis (TensorFlow, Keras, Transformers, OpenCV, matplotlib, pandas, scikit-learn y seaborn) y calidad y verificación (pytest, pytest-cov, ruff y mypy), tal y como se describen en el capítulo 18. La lista también incluye las dependencias de integración, persistencia y generación de informes. El peso de estas dependencias condiciona el entorno: la carga de TensorFlow y de los modelos de aprendizaje requiere recursos de memoria y, en su caso, de cómputo acelerado por GPU.

La configuración sensible y operativa permanece fuera del código y se suministra mediante variables de entorno, que se cargan desde un archivo .env en el arranque de la aplicación. La tabla siguiente resume las variables de entorno del sistema, agrupadas por ámbito.

Variable	Variable
JWT_SECRET_KEY	Clave secreta de firma de los tokens de acceso; en producción debe configurarse y no usar la clave de desarrollo.
JWT_ACCESS_EXPIRE_MINUTES	Duración en minutos del token de acceso.
JWT_REFRESH_EXPIRE_DAYS	Duración en días del token de refresco.
REFRESH_ROTATION_GRACE_SECONDS	Periodo de gracia de la rotación del token de refresco.
DB_HOST, DB_USER, DB_PASSWORD, DB_NAME	Parámetros de conexión a MySQL (por defecto localhost, root, sin contraseña y base tfg_pneumonia).
DB_POOL_SIZE	Tamaño del pool de conexiones a la base de datos.
GROQ_API_KEY	Clave de la API del proveedor del asistente conversacional.
TFG_DEMO_DATASET, TFG_DEMO_EXTERNAL_DATASET	Rutas por defecto de los datasets de entrenamiento y de validación externa.
TFG_SESSION_ID, TFG_MODEL_NAME, TFG_DATASET_DIR, TFG_EXTERNAL_DATASET_DIR, TFG_EPOCHS, TFG_BATCH_SIZE, TFG_LEARNING_RATE	Variables de entorno que el pipeline MLOps transmite a los scripts de entrenamiento y de validación externa.

Tabla x - Entorno tecnológico de construcción

El entorno impone además una serie de restricciones que condicionan la construcción y la ejecución. La aplicación requiere una instancia de MySQL accesible y con el esquema inicializable; en el entorno de desarrollo se utiliza XAMPP con el servicio de MariaDB activo, y el arranque de la aplicación crea las tablas si no existen. El motor de diagnóstico necesita los pesos de los modelos entrenados, que se conservan en el directorio pneumoniaccn-main/results y se cargan bajo demanda por la primera consulta con cada arquitectura; sin esos pesos, la solicitud de un diagnóstico falla en el procesamiento. El pipeline de experimentación MLOps requiere el directorio pneumoniaccn-main/code con los scripts de entrenamiento, análisis XAI, estadística y validación externa, que se invocan como procesos externos y escriben sus resultados en el directorio training_results. La generación de los mapas de explicabilidad emplea el backend no interactivo de matplotlib (Agg), que evita la dependencia de un entorno gráfico en el servidor.

El diagrama de despliegue de la siguiente figura representa la disposición física de los componentes del sistema en su entorno de ejecución. En el uso local, el navegador accede al servidor mediante HTTP; cuando la aplicación se publica a través del túnel de demostración, el acceso exterior utiliza HTTPS. Uvicorn aloja la aplicación FastAPI, que orquesta los routers, los servicios y el middleware, atiende las consultas a la base de datos MySQL y sirve los recursos estáticos. El worker asíncrono comparte la persistencia con la aplicación y ejecuta los scripts del pipeline MLOps, que cargan los modelos necesarios y escriben los artefactos en el sistema de ficheros; la aplicación invoca además al proveedor externo Groq para el asistente conversacional.

El diagrama de despliegue refleja la separación entre el ciclo de petición y la ejecución asíncrona: la aplicación atiende las peticiones del navegador y encola los trabajos, mientras que el worker reclama y procesa los diagnósticos y los entrenamientos fuera del ciclo de petición. El almacenamiento de ficheros alberga las plantillas, los recursos estáticos, las imágenes subidas, los informes y los resultados de los entrenamientos, y se expone parcialmente al navegador mediante los montajes de recursos estáticos. Los proveedores externos (el asistente conversacional y el repositorio de modelos) se integran como fronteras del sistema, de modo que los fallos de esos servicios no se confunden con los de la aplicación. Los requisitos no funcionales con impacto sobre el entorno de construcción, como el tiempo de respuesta de la inferencia (RNF-019) o la ejecución sin bloqueo de la interfaz (RNF-020), se satisfacen mediante la reutilización de los modelos en memoria y la cola de trabajos, que no requieren componentes adicionales de construcción.

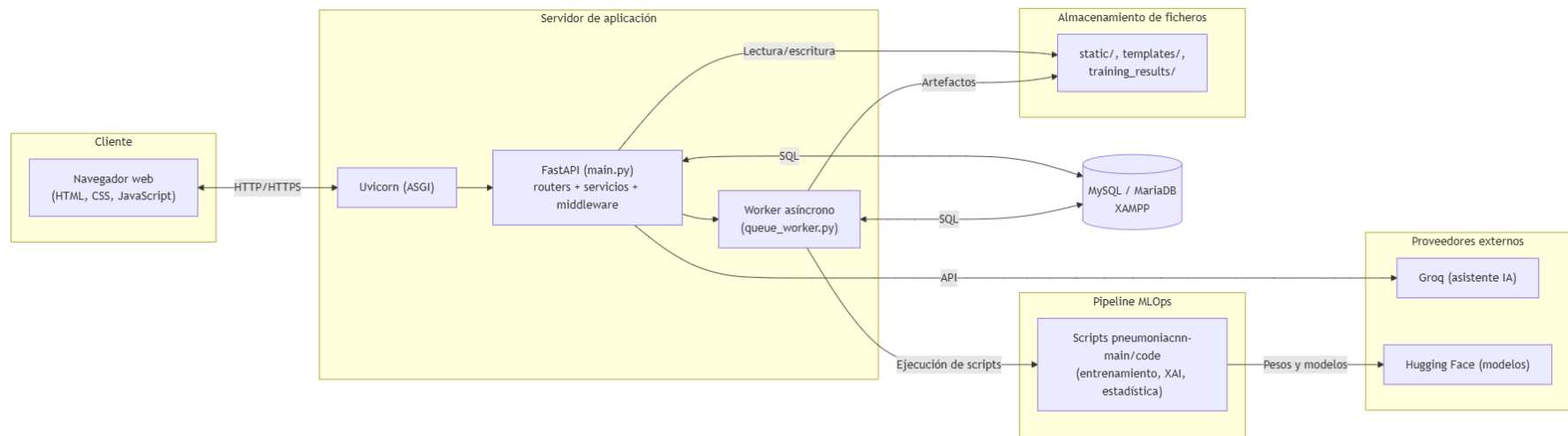


Ilustración x - Diagrama de despliegue del sistema

23.2. Paquetes y componentes de construcción

Los componentes de construcción de vitalXAI se corresponden con las agrupaciones funcionales del código fuente, representadas en la figura. La estructura distingue la capa HTTP (routers/), la capa de servicios (services/), los módulos principales de la raíz (main.py y database.py), la presentación (templates/ y static/), el pipeline de experimentación (pneumoniaccnn-main/) y las pruebas (tests/). El diagrama representa estas agrupaciones y sus dependencias, de modo que la construcción respeta la organización real del repositorio.

El diagrama refleja la dependencia principal del sistema: el núcleo de la aplicación compone la aplicación FastAPI a partir de los routers y los servicios, y monta los paquetes de presentación; los routers dependen de los servicios, que a su vez acceden a la persistencia relacional y al pipeline de experimentación. El paquete de pruebas depende de los routers, los servicios y el núcleo, de modo que su construcción no forma parte de la aplicación ejecutable. El paquete pneumoniaccnn-main/ constituye un componente externo al código de la aplicación: contiene los scripts del pipeline MLOps y los pesos de los modelos entrenados, y los servicios lo invocan como procesos externos, sin acoplamiento de importación.

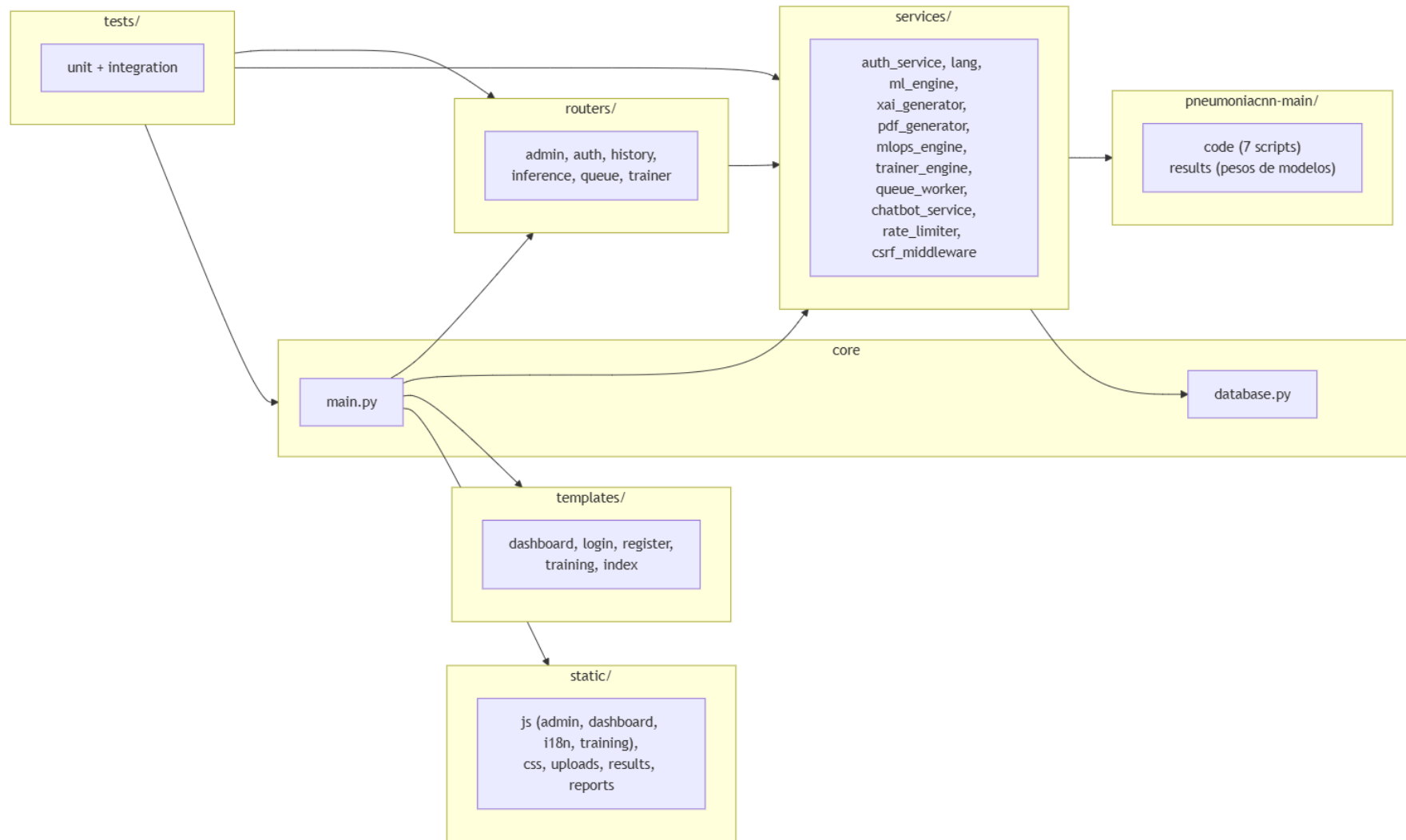


Ilustración x - Diagrama de paquetes de construcción

La tabla siguiente especifica los componentes que forman cada paquete de construcción y su responsabilidad.

Paquete	Componentes	Responsabilidad
Raíz de la aplicación	main.py, database.py	Composición de la aplicación FastAPI, configuración de middleware y montajes, inicialización de la base de datos y pool de conexiones.
routers/	admin.py, auth.py, history.py, inference.py, queue.py, trainer.py	Capa HTTP: definición de los endpoints de la aplicación, autenticación, diagnóstico, historial, cola, laboratorio y administración.
services/	auth_service.py, lang.py, ml_engine.py, xai_generator.py, pdf_generator.py, mlops_engine.py, trainer_engine.py, queue_worker.py, chatbot_service.py, rate_limiter.py, csrf_middleware.py	Lógica de aplicación: criptografía y sesiones, internacionalización, predicción, explicabilidad, informes, laboratorio MLOps, worker de la cola, asistente conversacional, limitación y seguridad.
templates/	login.html, register.html, dashboard.html, training.html, index.html	Plantillas Jinja2 de las páginas de la aplicación.
static/	js/admin.js, js/dashboard.js, js/i18n.js, js/training.js, css, uploads/, results/, reports/	Recursos de presentación: scripts de la interfaz, estilos y directorios de artefactos generados.
pneumoniaccn-main/	code/ (siete scripts), results/ (pesos de modelos)	Pipeline MLOps: entrenamiento K-fold, entrenamiento Transformer, estadística, validación externa, test de DeLong y análisis XAI.
tests/	unit/, integration/, conftest.py	Pruebas unitarias e integración de la aplicación, con sus accesorios y mocks.

Tabla x - Paquetes y componentes de construcción

La estructura de componentes se corresponde con los subsistemas de soporte definidos en el capítulo 18. database.py materializa la persistencia relacional de SSOP-001; los directorios templates/, static/ y training_results/ materializan el almacenamiento de ficheros y recursos de SSOP-002; main.py, las dependencias instaladas y el entorno de ejecución materializan la integración de proveedores externos de SSOP-003; y tests/ materializa la calidad y verificación de SSOP-004. Esta correspondencia mantiene la coherencia entre la organización del repositorio y los subsistemas de soporte del diseño.

23.3. Proceso de construcción del software

El proceso de construcción de vitalXAI establece los pasos necesarios para preparar el entorno y lanzar la aplicación. Como se señaló en la introducción del capítulo, la construcción no incluye una fase de compilación ni de generación de ejecutables: consiste en preparar el entorno virtual, instalar las dependencias declaradas, configurar las variables de entorno, disponer de los artefactos externos y lanzar el servidor. El proceso se describe en la tabla siguiente, con el orden de ejecución y la finalidad de cada paso.

Orden	Paso	Procedimiento
1	Preparar el entorno	Instalar Python 3.11 y crear un entorno virtual para el proyecto.
2	Instalar dependencias	Ejecutar <code>pip install -r requirements.txt</code> dentro del entorno virtual, lo que instala las versiones fijadas de las librerías de la aplicación.
3	Configurar el entorno	Crear el archivo <code>.env</code> con las variables de entorno del sistema (credenciales de MySQL, claves de sesión y del asistente, y rutas de los datasets).
4	Preparar la base de datos	Iniciar el servicio de MySQL (en el entorno de desarrollo, mediante XAMPP); la aplicación crea automáticamente las tablas del esquema en el arranque.
5	Verificar los artefactos	Comprobar que el directorio <code>pneumoniacnn-main/code</code> contiene los scripts del pipeline y que <code>pneumoniacnn-main/results</code> contiene los pesos de los modelos entrenados.
6	Lanzar la aplicación	Ejecutar <code>python main.py</code> , que inicia el servidor Uvicorn, inicializa la base de datos y arranca el worker de la cola.

Tabla x - Proceso de construcción del software

El proceso de construcción depende de dos familias de requisitos adicionales. En primer lugar, los artefactos del aprendizaje automático: los pesos de los modelos entrenados, conservados en `pneumoniacnn-main/results`, son necesarios para el diagnóstico, de modo que la primera consulta con cada arquitectura carga los pesos y las siguientes reutilizan el modelo en memoria. En segundo lugar, las variables de entorno del pipeline MLOps: los scripts de entrenamiento y de validación externa leen la configuración de cada trabajo desde las variables `TFG_*`, que el motor de experimentación establece al invocarlos, y escriben sus resultados en el directorio `training_results`. Sin estas condiciones, el entorno no dispone de la funcionalidad de diagnóstico ni de experimentación completa.

La figura 99 representa el diagrama de componentes del sistema, que detalla la colaboración entre los componentes construidos. La aplicación compone los routers, que delegan en los servicios; los servicios acceden a la persistencia, invocan al asistente externo y orquestan el worker; el worker procesa los diagnósticos con el motor de predicción, el generador de mapas y el generador de informes, y los entrenamientos con el motor MLOps, que ejecuta los scripts del pipeline; la aplicación sirve la presentación desde las plantillas y los recursos estáticos.

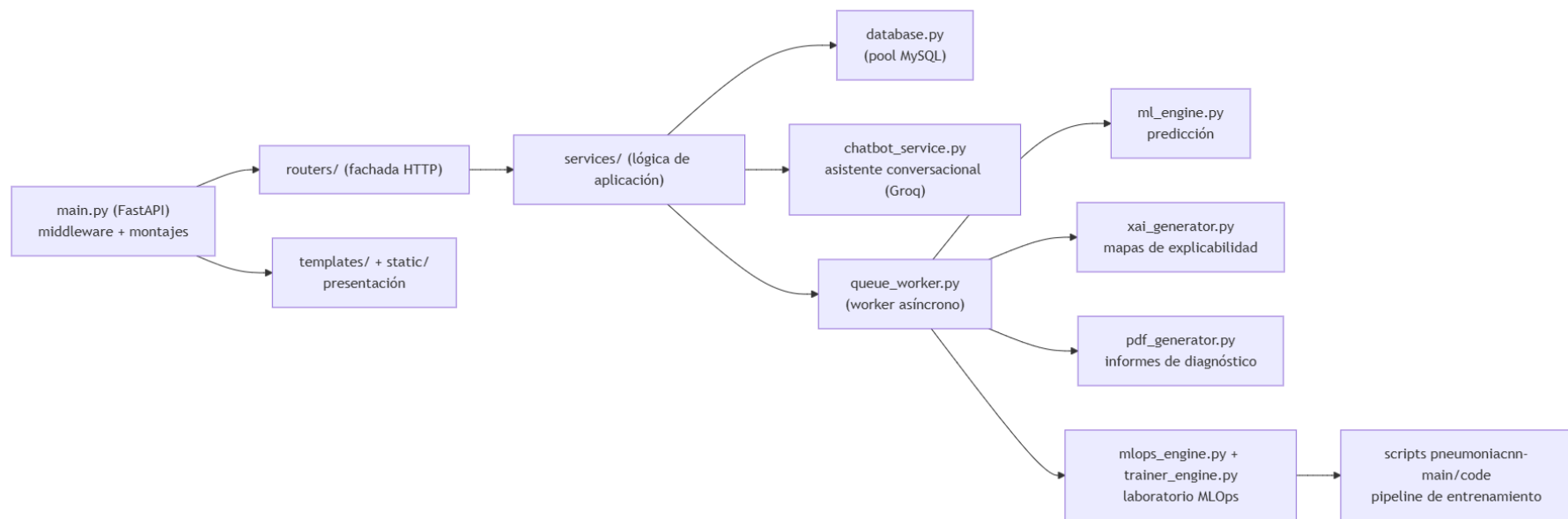


Ilustración x - Diagrama de componentes del sistema

La especificación detallada de los componentes del sistema se resume en la tabla siguiente, indicando qué requiere cada componente para su construcción y cómo se procede.

Componente	Dependencias y requisitos	Procedimiento de construcción
Aplicación web (main.py)	FastAPI, Uvicorn y las dependencias del servidor	Se construye al instalar las dependencias; compone la aplicación, los routers, el middleware y los montajes.
Routers (routers/)	Los módulos de los routers y los servicios	Se incluyen en la aplicación mediante la composición de main.py; no requieren construcción adicional.
Servicios (services/)	Las librerías de seguridad, aprendizaje y generación	Se instalan con requirements.txt; los servicios se importan desde los routers y el worker.
Persistencia (database.py)	MySQL y el conector	Se requiere una instancia de MySQL accesible; el esquema se crea automáticamente en el arranque.
Worker (queue_worker.py)	Las librerías de la aplicación	Se arranca automáticamente en el ciclo de vida de la aplicación.
Motor de diagnóstico (ml_engine.py, xai_generator.py, pdf_generator.py)	TensorFlow, Transformers, OpenCV, matplotlib y FPDF	Se instalan con requirements.txt; requiere los pesos de los modelos para la predicción.
Laboratorio MLOps (mlops_engine.py, trainer_engine.py)	Los scripts del pipeline y los datasets	Requiere pneumoniacnn-main/code y las variables TFG_*; los scripts se invocan como procesos.
Presentación (templates/, static/)	Jinja2 y el navegador	Se sirven mediante los montajes de main.py; no requieren compilación.
Asistente conversacional (chatbot_service.py)	La librería del proveedor y la clave de la API	Se instala con requirements.txt; requiere GROQ_API_KEY configurada.

Tabla x - Especificación detallada de los componentes del sistema

La construcción de la aplicación queda así especificada de forma repetible: con las dependencias instaladas, la configuración suministrada y los artefactos del aprendizaje automático disponibles, la ejecución de main.py produce el sistema en funcionamiento, con la base de datos inicializada y el worker de la cola activo.

23.4. *Elaboración de Especificaciones del Modelo Físico de Datos*

La generación del esquema de la base de datos materializa el modelo físico de datos definido en el capítulo 19 en las tablas del sistema. La aplicación no utiliza un script SQL externo para su instalación: el esquema operativo se define en el módulo database.py y se ejecuta en el arranque mediante la función init_db(), que se invoca en el ciclo de vida de la aplicación antes de iniciar el worker. La función utiliza sentencias CREATE TABLE IF NOT EXISTS, de modo que la base de datos se crea y se verifica automáticamente en cada arranque, sin necesidad de un procedimiento manual de instalación ni de un script de migración adicional (Oracle, 2024).

La tabla siguiente especifica las tablas del esquema, generadas por init_db() a partir del modelo físico, con su propósito y sus columnas principales.

Tabla	Propósito	Columnas principales
users	Cuentas de usuario de la plataforma y credenciales de acceso.	id (clave primaria), username (única), password_hash, first_name, last_name, role.
consultations	Consultas de diagnóstico realizadas por los usuarios, con sus rutas de artefactos y su resultado.	id, user_id (clave ajena a users), model_name, original_image_path, xai_image_path, prediction_label, confidence_score, patient_name, pdf_path, timestamp.
training_jobs	Registro histórico de los trabajos de entrenamiento, con su estado y su progreso.	id, user_id (clave ajena), dataset_path, model_name, status, progress, metrics_json, started_at, finished_at.
job_queue	Cola persistente de trabajos asíncronos: diagnósticos, entrenamientos y validaciones externas.	id, user_id (clave ajena), job_type, status, payload (JSON), result (JSON), error_message, created_at, started_at, finished_at.
refresh_tokens	Tokens de refresco de sesión, conservados mediante su hash y su estado de revocación.	id, user_id (clave ajena), token_hash, expires_at, revoked, created_at.

Tabla x - Especificación de las tablas del esquema

Las relaciones entre las tablas se establecen mediante las claves ajenas definidas en las sentencias de creación. Las tablas consultations, training_jobs, job_queue y refresh_tokens referencian la tabla users a través de la columna user_id, de modo que cada registro relacional de actividad pertenece a una cuenta. Las claves ajenas de training_jobs, job_queue y refresh_tokens se declaran con la política ON DELETE CASCADE, de modo que la eliminación de un usuario elimina sus trabajos y sus credenciales de sesión. La clave ajena de consultations se declara sin esa política, por lo que el borrado de una cuenta que conserva consultas puede ser rechazado por la restricción de integridad. Además, los artefactos de las consultas y las carpetas del laboratorio no se eliminan automáticamente mediante estas claves.

La generación del esquema mantiene la trazabilidad con el modelo físico del capítulo 19: las columnas y los tipos de datos de las sentencias de creación corresponden a las entidades y los atributos definidos en el modelo, con las adaptaciones necesarias para el motor MySQL. La sentencia `CREATE TABLE IF NOT EXISTS` garantiza la idempotencia de la inicialización: si la tabla ya existe, la ejecución no la altera ni elimina datos, de modo que el arranque de la aplicación puede repetirse sin riesgo para los datos persistidos (Oracle, 2024). Cuando el modelo físico evolucione con nuevas tablas o columnas, la generación del esquema deberá actualizarse en `database.py` y registrarse en el registro de cambios de la base de datos, conforme a las prácticas de persistencia del proyecto.

Capítulo 24. Preparación inicial de los datos del sistema

Este capítulo fija las condiciones de datos necesarias para poner vitalXAI en funcionamiento por primera vez y describe cómo se mantiene el esquema cuando evoluciona. Después de definir el entorno y la construcción en el capítulo 23, aquí se concreta qué debe estar disponible, dónde se prepara y en qué orden se realizan las operaciones de inicialización (Elmasri & Navathe, 2016). El apartado completa el diseño de persistencia y sirve de referencia para las pruebas y la implantación.

El capítulo se organiza en dos apartados: el entorno de preparación de los datos, que describe el entorno tecnológico en el que se dispone la carga inicial, y los procedimientos de preparación y evolución del esquema, que definen el proceso, el orden de lanzamiento de los procedimientos y el diseño detallado de cada uno. El contenido se apoya en el modelo físico de datos del capítulo 19 y en la guía de despliegue del sistema, que fija los requisitos operativos de la puesta en marcha.

La carga inicial de vitalXAI no incluye datos semilla ni cuentas precargadas. El esquema relacional se crea y se comprueba automáticamente cuando arranca la aplicación, mientras que las cuentas se crean mediante el formulario de registro. Preparar el sistema consiste, por tanto, en disponer del esquema, de los datasets y pesos necesarios para la demostración y de la configuración que permite acceder a ellos. Los cambios futuros del esquema se gestionan manualmente y se registran en el registro de cambios de la base de datos, sin un proceso de migración formal.

24.1. Entorno de Carga Inicial o Migración

El entorno de preparación de los datos de vitalXAI reúne los elementos necesarios para la puesta en marcha del sistema. La preparación se realiza sobre el mismo equipo que ejecuta la aplicación: el entorno de desarrollo y de demostración del proyecto, que aloja el servidor, la base de datos, los conjuntos de imágenes y los pesos de los modelos. La tabla siguiente resume los elementos del entorno y su papel en la carga inicial.

Elemento de carga	Función en la preparación inicial
MySQL (MariaDB en XAMPP)	Almacén relacional donde la aplicación crea y verifica el esquema en el arranque.
Datasets de demostración	Conjuntos de imágenes de entrenamiento y de validación externa, necesarios para el diagnóstico y la experimentación.
Pesos de los modelos	Pesos de las arquitecturas entrenadas, necesarios para ejecutar la inferencia del diagnóstico.
Configuración del entorno	Variables de entorno con las credenciales y las rutas de los datos, suministradas mediante el archivo .env.

Tabla x - Especificación de entorno de Carga Inicial

La base de datos relacional se prepara mediante el servicio de MySQL del entorno, que en el desarrollo se ejecuta a través de XAMPP. La aplicación no requiere un script de instalación del esquema: la función `init_db()` de `database.py`, invocada en el arranque, ejecuta las sentencias `CREATE TABLE IF NOT EXISTS` que materializan el modelo físico de datos definido en el capítulo 19, creando las tablas `users`, `consultations`, `training_jobs`, `job_queue` y `refresh_tokens` cuando no existen (Oracle, 2024). La verificación de la conexión se realiza al iniciar la aplicación, que informa de la imposibilidad de conectar si el servicio no está activo.

Los datos de demostración comprenden los conjuntos de imágenes y los pesos de los modelos. El dataset de entrenamiento y evaluación XAI se conserva en `pneumoniacnn-main/Images`, y el conjunto independiente para la validación externa en `pneumoniacnn-main/ExternalDataset`; ambos se organizan en las subcarpetas `NORMAL/` y `PNEUMONIA/`. Los pesos de los modelos entrenados se conservan en `pneumoniacnn-main/results`, con los archivos correspondientes a cada arquitectura, y se cargan bajo demanda en la primera consulta de diagnóstico. La disponibilidad de estos artefactos es condición necesaria para el diagnóstico y para el laboratorio de experimentación.

La configuración del entorno se suministra mediante el archivo `.env`, que la aplicación carga en el arranque. La configuración incluye las credenciales de la base de datos (`DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `DB_POOL_SIZE`), las claves de sesión (`JWT_SECRET_KEY` y los parámetros de expiración), la clave del asistente conversacional (`GROQ_API_KEY`) y las rutas de los datasets de demostración (`TFG_DEMO_DATASET` y `TFG_DEMO_EXTERNAL_DATASET`), que permiten lanzar los entrenamientos y la validación externa sin recurrir al selector de carpetas. Las cuentas de usuario no se precargan: se crean mediante el registro en la aplicación, de modo que la carga inicial de cuentas se realiza operativamente, conforme al flujo del subsistema de acceso.

El diagrama de despliegue de la figura representa el entorno de preparación de los datos. La aplicación lee la configuración al arrancar, inicializa el esquema en la base de datos MySQL, carga los pesos de los modelos bajo demanda y accede a los datasets de demostración para la experimentación y la validación externa; el asistente conversacional se integra como proveedor externo.

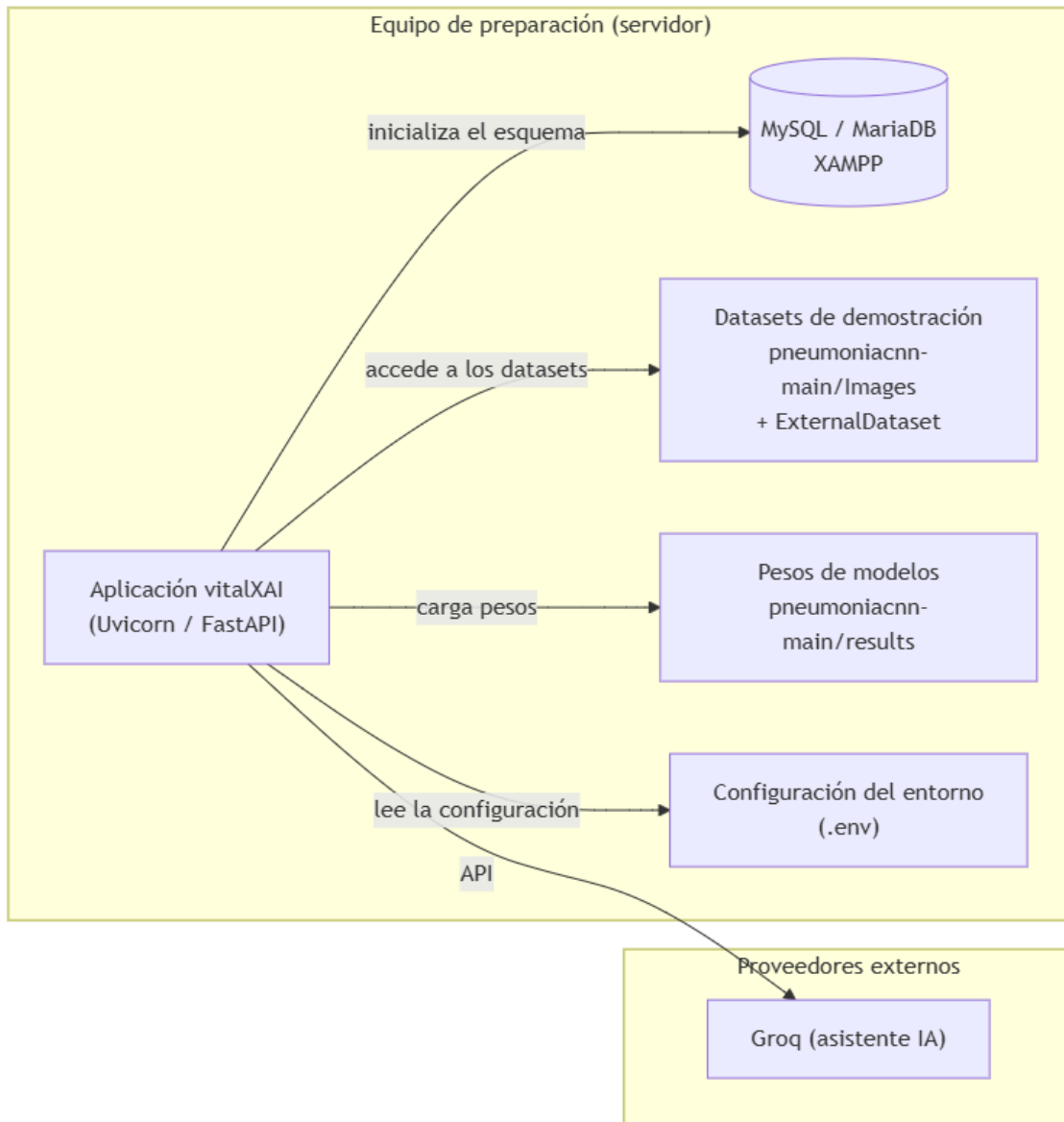


Ilustración x - Diagrama de despliegue del entorno de preparación de los datos

El diagrama refleja que la preparación inicial se resuelve en el equipo que aloja la aplicación, sin componentes de carga adicionales: la base de datos se inicializa desde la propia aplicación, y los datos de demostración y la configuración se disponen como artefactos del sistema de archivos y variables de entorno. La verificación de los requisitos del entorno (el servicio de MySQL activo, la presencia de los datasets y de los pesos, y la configuración correcta) constituye el punto de partida de los procedimientos de preparación descritos en el apartado siguiente.

24.2. Procedimientos de Carga Inicial o Migración

Los procedimientos de preparación de los datos definen el proceso por el que el sistema queda disponible para su primer uso, el orden o la jerarquía de lanzamiento de cada procedimiento y el diseño detallado de cada uno. El proceso se representa mediante el diagrama de actividad de la figura, que refleja la secuencia de preparación: la verificación de los requisitos del entorno, la inicialización del esquema, la disposición de los datos de demostración y de la configuración, y la creación de las cuentas de acceso.

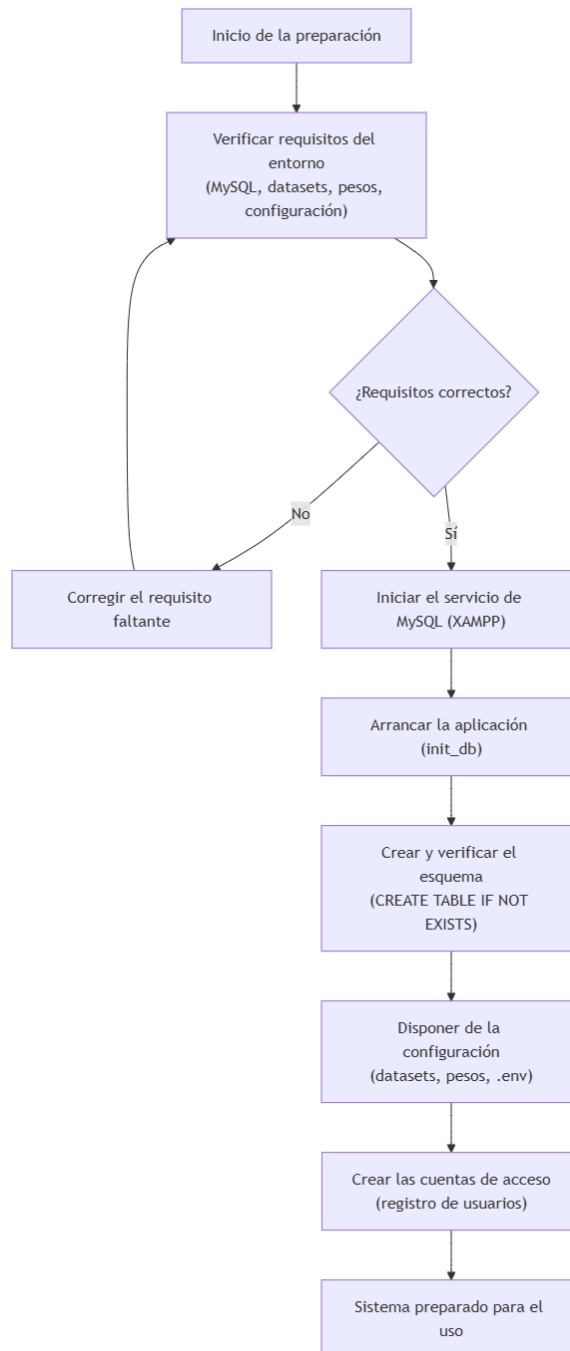


Ilustración x - Diagrama de actividad del proceso de preparación inicial de los datos

El diagrama de actividad refleja el orden operativo de la preparación: primero se comprueban los requisitos y se inicia el servicio de la base de datos; después se dispone el archivo .env y los artefactos necesarios, y se arranca la aplicación, que crea y comprueba el esquema. Finalmente se crean las cuentas mediante el registro. La configuración debe estar disponible antes del arranque porque main.py la carga durante la importación de la aplicación.

El diseño detallado de cada procedimiento que participa en la preparación inicial se especifica en la tabla siguiente, con su orden de lanzamiento, su descripción y la verificación de su resultado.

Procedimiento	Orden	Descripción	Verificación
Verificación de requisitos	1	Comprobar que el servicio de MySQL está disponible, que existen los datasets y los pesos de los modelos y que la configuración del entorno es correcta.	El entorno informa de los requisitos faltantes y permite corregirlos antes de continuar.
Inicialización del esquema	2	Ejecutar la aplicación, que invoca init_db() y crea las tablas del esquema con CREATE TABLE IF NOT EXISTS cuando no existen.	El arranque informa de la conexión a la base de datos y de la verificación de las tablas.
Configuración del entorno	2	Suministrar las variables de entorno del sistema mediante el archivo .env, con las credenciales, las claves y las rutas de los datasets.	La aplicación carga la configuración en el arranque y la utiliza en las operaciones de sesión, diagnóstico y experimentación.
Preparación de los datos de demostración	3	Disponer de los datasets de entrenamiento y de validación externa y de los pesos de los modelos en sus directorios correspondientes.	La solicitud de un diagnóstico carga los pesos del modelo; el laboratorio accede a las rutas de los datasets.
Creación de cuentas	4	Crear las cuentas de acceso mediante el registro de usuarios en la aplicación, sin datos de prueba precargados.	El nuevo usuario inicia sesión y accede al panel de diagnóstico.

Tabla x - Especificación del Procedimientos de Carga Inicial

La configuración del entorno debe estar disponible antes de arrancar la aplicación, mientras que los datasets y los pesos deben estar presentes antes de utilizar las funciones que los necesitan. La creación de cuentas constituye un procedimiento operativo y continuo, que no forma parte de una carga única sino del funcionamiento normal del subsistema de acceso.

La evolución del esquema de la base de datos se gestiona de forma manual, sin un sistema de migraciones automáticas. Cuando el modelo físico de datos cambia, por la incorporación de una tabla, una columna o una restricción, la modificación se aplica actualizando las sentencias de creación de database.py y se registra en el registro de cambios de la base de datos del proyecto, con las sentencias SQL del cambio y las notas de reversión. CREATE TABLE IF NOT EXISTS solo evita el error cuando la tabla ya existe, pero no modifica su definición, por lo que no puede utilizarse como mecanismo de adaptación del esquema (Oracle, 2024). Esta decisión, coherente con la simplicidad del esquema y con el estado del proyecto, evita la complejidad de un sistema de migraciones mientras la base de datos se encuentra en una fase estable; si el esquema evolucionara hacia cambios destructivos o una base de datos con datos persistidos críticos, la incorporación de un mecanismo de migraciones formal debería evaluarse conforme al registro de decisiones del proyecto.

Capítulo 25. Estrategia de verificación del sistema

Este capítulo define cómo se comprobará el comportamiento de vitalXAI una vez construido. El capítulo 16 ya establece qué pruebas deben realizarse y qué resultado permite superarlas; aquí se concretan los entornos, las herramientas y los niveles en los que se organizan esas comprobaciones, sin repetir el catálogo de pruebas. La separación entre el alcance del plan y la forma de ejecutarlo sigue la distinción habitual de la planificación de pruebas (Myers, Sandler, & Badgett, 2011).

El capítulo se organiza en dos apartados: los entornos de verificación, que describen el entorno tecnológico en el que se ejecutan las pruebas, las herramientas utilizadas, el origen de los datos de prueba y las restricciones operativas; y los niveles de verificación y criterios de aceptación, que definen los niveles de prueba, la naturaleza de las pruebas de integración y de sistema con una carga parecida a la de explotación, las validaciones funcionales y no funcionales que cubren las excepciones y los criterios que deben satisfacerse para aceptar cada nivel. El contenido se apoya en la guía de pruebas del proyecto, que fija los comandos, los niveles y el umbral de cobertura, y en el plan de pruebas del capítulo 16.

La verificación de vitalXAI combina pruebas automatizadas con comprobaciones sobre el entorno real. Las pruebas unitarias y la prueba de integración disponible se ejecutan mediante pytest y se incorporan al flujo de integración continua; las pruebas de rendimiento y de sistema se reservan para el despliegue de la plataforma, donde sus criterios pueden medirse sobre el sistema real. Esta combinación ayuda a detectar regresiones en la lógica de la aplicación y permite comprobar los flujos finales en condiciones próximas a las de explotación.

25.1. Entornos de verificación

El entorno de verificación de vitalXAI se organiza en tres entornos que atienden a distintos niveles de aislamiento y de fidelidad con respecto al sistema real: el entorno unitario, el entorno de integración y el entorno de sistema. Cada entorno emplea sus propios dobles de prueba y su propio origen de datos, de modo que la verificación progresa desde el aislamiento total de los componentes hasta la ejecución sobre el despliegue real. La tabla siguiente resume los tres entornos y sus características.

Entorno	Alcance	Aislamiento	Origen de los datos de prueba
Unitario	Verificación de componentes individuales (lógica de negocio, funciones puras, componentes aislados).	Sin red ni base de datos real; las dependencias externas se sustituyen por dobles y las operaciones de ficheros se aíslan con mocks o archivos temporales.	Fixtures de tests/conftest.py: base de datos simulada y modelo TensorFlow simulado, junto con dobles definidos por cada prueba.
Integración	Verificación de la interacción entre capas (API y persistencia) y entre subsistemas.	Base de datos en memoria (SQLite); no requiere la base de datos MySQL.	Esquema y datos preparados por tests/integration/conftest.py; la cobertura actual se centra en los flujos de autenticación e historial.
Sistema	Verificación integral sobre el despliegue real de la plataforma.	Sin aislamiento: entorno real con la base de datos MySQL, los pesos de los modelos y los datasets.	Datos de demostración del entorno de despliegue (imágenes, pesos y cuentas).

Tabla x - Especificación de Entornos de Pruebas

Las herramientas de verificación del proyecto se especifican conforme a la guía de pruebas. La batería automatizada se ejecuta con pytest y pytest-cov, que mide la cobertura de los módulos configurados; los dobles de prueba se construyen principalmente con unittest.mock y fixtures de pytest (pytest, 2024). La calidad estática se comprueba con ruff, que verifica el estilo y las reglas del código, mientras que mypy dispone de configuración para los módulos indicados, aunque no forma parte del flujo de CI actual (Ruff, 2024; Mypy, 2024). La integración continua ejecuta Ruff y la batería de pytest con el umbral de cobertura configurado.

El origen de los datos de prueba se resuelve mediante las fixtures y el esquema preparado en el directorio de pruebas. Las fixtures del módulo conftest.py proporcionan una base de datos simulada y un modelo de TensorFlow simulado para las pruebas unitarias, mientras que cada prueba sustituye las demás dependencias que necesita. La prueba de integración disponible utiliza una base de datos SQLite en memoria y datos creados durante el propio test; actualmente se centra en los flujos de autenticación e historial y no ejerce las APIs externas. Los datos del entorno de sistema proceden del despliegue real: las imágenes de demostración, los pesos de los modelos y las cuentas de acceso, en coherencia con la preparación inicial de los datos del capítulo 24.

Las restricciones operativas del entorno de verificación son las siguientes. Las pruebas unitarias sustituyen las dependencias costosas o externas cuando resulta necesario, de modo que no dependen de la disponibilidad de la red, de los pesos reales o de un proveedor conversacional. La prueba de integración utiliza SQLite en memoria y no reproduce todo el despliegue productivo. El umbral de cobertura está fijado en el setenta por ciento y la configuración de CI hace fallar la batería si no se alcanza. Las pruebas unitarias y de integración se ejecutan con los comandos definidos en el proyecto; las pruebas de rendimiento y de sistema se plantean manual o semiautomáticamente sobre el despliegue.

El diagrama de despliegue de la figura representa el entorno de verificación del sistema. Las pruebas unitarias se ejecutan sobre las fixtures aisladas, la prueba de integración disponible sobre la base de datos en memoria y las pruebas de sistema sobre el despliegue real con MySQL y los artefactos del aprendizaje automático. Las comprobaciones de calidad estática acompañan a la batería automatizada.

El diagrama refleja la progresión prevista de la verificación por entornos: la batería automatizada parte del aislamiento del entorno unitario, pasa por la interacción entre capas en el entorno de integración y culmina en la comprobación sobre el despliegue real. Ruff acompaña a la batería en la integración continua; mypy queda configurado, pero no se ejecuta actualmente en ese flujo. Los criterios de aceptación de cada nivel se definen en el apartado siguiente.

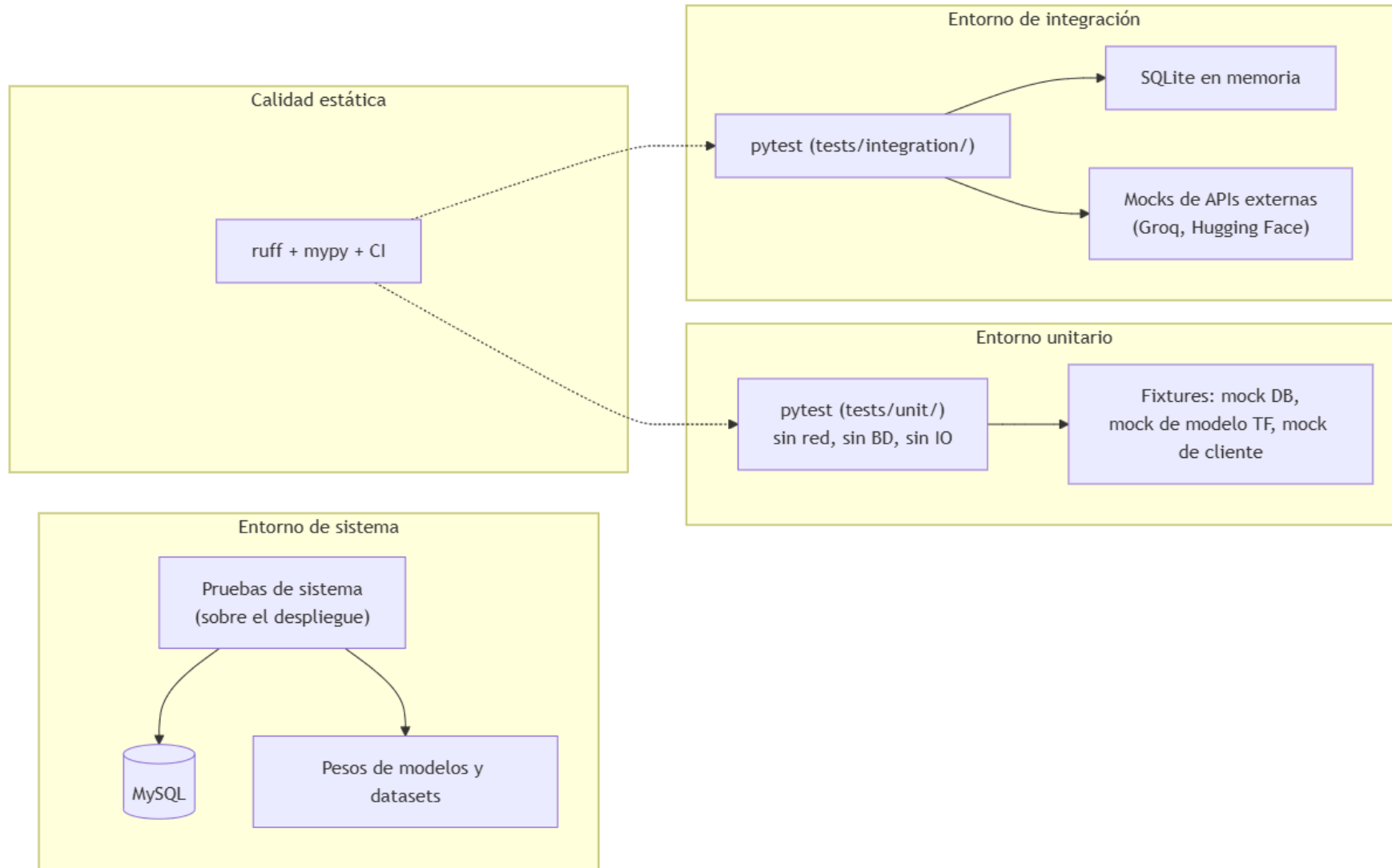


Ilustración x - Diagrama de despliegue del entorno de verificación

25.2. Niveles de verificación y criterios de aceptación

Los niveles de verificación definen la profundidad de la comprobación del sistema, en correspondencia con los entornos descritos en el apartado anterior. Cada nivel agrupa las verificaciones que se ejecutan en su entorno y establece los criterios que deben satisfacerse para aceptarlo, de modo que la superación de un nivel es condición para considerar verificada la parte del sistema que abarca. La definición de los niveles se mantiene coherente con el plan de pruebas del capítulo 16, que detalla las pruebas concretas de cada categoría; en este apartado se especifican los niveles, su alcance y sus criterios de aceptación, sin reproducir el detalle de cada prueba. Los tres niveles se presentan a continuación, con la tabla de criterios de aceptación de cada uno.

25.2.1. Nivel unitario

El nivel unitario verifica el comportamiento de los componentes de forma aislada. Las pruebas no dependen de la red, de una base de datos real ni de los servicios externos, que se sustituyen por dobles cuando es necesario; sí pueden utilizar ficheros temporales para probar operaciones de entrada y salida. Cubre la gestión de cuentas y sesiones, la validación de entradas, el motor de inferencia con modelos simulados, la generación de explicaciones, el historial, el laboratorio, la cola, la internacionalización y el acceso a datos. También recorre caminos alternativos como entradas inválidas, credenciales incorrectas, usuarios duplicados, tokens revocados, modelos ausentes, sesiones ajenas y trabajos no interrumpibles. Este nivel se corresponde con la categoría de verificación de componentes del capítulo 16.

Criterio	Descripción
Batería unitaria superada	Todos los tests unitarios del directorio tests/unit/ se ejecutan y superan sin fallos.
Cobertura de código	La cobertura de los módulos de la aplicación alcanza al menos el umbral del setenta por ciento.
Calidad estática	La verificación con ruff no reporta errores de estilo y la verificación con mypy no reporta errores de tipos en los módulos configurados.
Determinismo	La batería unitaria se ejecuta sin dependencia de la red, de la base de datos ni de los servicios externos.
Cobertura de excepciones	Los caminos alternativos y de error de los componentes verificados están cubiertos por las pruebas.

Tabla x - Especificación de Nivel unitario

La superación del nivel unitario se comprueba cuando se ejecutan los comandos de pruebas del proyecto, y la integración continua repite esta comprobación en los eventos configurados del repositorio. Así se obtiene una detección temprana de regresiones en los componentes cubiertos.

25.2.2. Nivel de integración

El nivel de integración verifica la colaboración entre las capas del sistema y entre los subsistemas, en el entorno de integración, superando el aislamiento del nivel unitario. Su alcance previsto cubre la interacción entre la API, la lógica de aplicación y la persistencia mediante una base de datos en memoria. El plan incluye como flujos críticos el acceso a la plataforma, el diagnóstico desde la subida hasta el resultado, el aislamiento de datos entre usuarios, la supervisión administrativa y el lanzamiento de un experimento desde el asistente hasta la cola. En el estado actual del repositorio, la prueba de integración disponible cubre principalmente el acceso, el registro, el panel y el historial; el resto de los flujos requiere ampliar la cobertura antes de poder considerarse verificado. Este nivel se corresponde con la categoría de verificación de flujos entre subsistemas del capítulo 16.

Criterio	Descripción
Batería de integración superada	Todos los tests de integración del directorio tests/integration/ se ejecutan y superan sin fallos.
Flujos críticos	Los flujos de extremo a extremo entre la API, la capa de negocio y la persistencia se verifican correctamente.
Aislamiento y autorización	Las condiciones de aislamiento de datos entre usuarios y de autorización administrativa se respetan en los flujos combinados.
Carga concurrente	Se ejecutan operaciones de consulta y gestión del historial con diez usuarios autenticados concurrentes, sin superar el doble del tiempo medido con carga baja.
Dependencias externas	La prueba de integración disponible no depende de servicios externos; la cobertura de otros flujos que requieran simulaciones queda pendiente.

Tabla x - Especificación de Nivel de integración

La superación del nivel de integración se verifica antes de la integración de los cambios en el flujo de trabajo del proyecto, de modo que la colaboración entre subsistemas queda confirmada antes de la verificación sobre el entorno real.

25.2.3. Nivel de sistema

El nivel de sistema verifica integralmente la plataforma sobre el despliegue real, sin aislar la base de datos ni los artefactos del aprendizaje automático. Las pruebas previstas ejercitan los flujos completos con MySQL, los pesos de los modelos y los datasets del entorno de despliegue, utilizando una carga de trabajo parecida a la de explotación. Incluyen el diagnóstico asistido, el historial, el laboratorio, la ejecución asíncrona, la supervisión administrativa y las capacidades transversales, además de las comprobaciones no funcionales de rendimiento y de protección y control de acceso. La usabilidad se mantiene como criterio de aceptación, pero no existe una validación empírica con usuarios reales en el proyecto. Este nivel se corresponde con las categorías de protección, rendimiento y verificación integral de los flujos completos del capítulo 16.

Criterio	Descripción
Flujos completos	Los flujos de uso de la plataforma se realizan correctamente sobre el despliegue real: diagnóstico, historial, laboratorio, cola, administración y capacidades transversales.
Validaciones funcionales	El comportamiento de los flujos coincide con el especificado en los casos de uso, incluidos los escenarios alternativos y de error.
Seguridad	Los mecanismos de protección (CSRF, cabeceras de seguridad, limitación de peticiones y gestión de tokens) se verifican sobre el sistema real.
Rendimiento	La petición de diagnóstico devuelve su identificador en menos de 2 segundos, la inferencia con el modelo ya cargado finaliza en menos de 15 segundos y la interfaz permanece operativa durante las tareas largas. La prueba de concurrencia utiliza diez usuarios autenticados y no permite superar el doble del tiempo medido con carga baja.
Usabilidad	Una persona que no participe en el desarrollo completa correctamente, sin asistencia del evaluador, los cuatro flujos principales, con una tasa de éxito del 100 %.
Adaptación visual	Las vistas principales se muestran sin pérdida de contenido ni desplazamiento horizontal no previsto en 1366 × 768 píxeles y 390 × 844 píxeles.
Artefactos	Los informes PDF del diagnóstico y de la sesión, y los mapas de explicabilidad, se generan correctamente y se entregan al usuario.
Carga de explotación	La verificación se realiza con una carga de trabajo parecida a la de explotación y cubre las excepciones previstas.

Tabla x - Especificación de Nivel de sistema

La superación del nivel de sistema se verifica sobre el despliegue de la plataforma antes de su presentación y confirma el comportamiento de los flujos y requisitos incluidos en el alcance de las pruebas. La comprobación de usabilidad se limita a una revisión estructurada y no equivale a una evaluación con usuarios reales, que no se realizó durante el proyecto. Los requisitos RNF-006, RNF-009 y RNF-017 permanecen pendientes de implementación y, por tanto, no pueden considerarse verificados en el prototipo actual.

Capítulo 26. Implantación del sistema

Este capítulo define las condiciones necesarias para poner vitalXAI en funcionamiento y para que sus usuarios puedan operar con ella. Parte de la construcción descrita en el capítulo 23 y de la preparación de datos del capítulo 24, y añade la documentación, la formación, la infraestructura y el acceso necesarios para la operación. La implantación se describe como una preparación para la demostración y no como una certificación de uso clínico (Sharp, Rogers, & Preece, 2019).

El capítulo se organiza en dos apartados: la documentación necesaria para la operación, que especifica el manual de usuario y sus características; y la implantación en el entorno de operación, que define las necesidades de formación, los requisitos de infraestructura e instalación y el proceso de puesta en marcha. El contenido se apoya en la guía de despliegue del proyecto y en los capítulos precedentes de diseño, de modo que la implantación mantiene la coherencia con la construcción y con la preparación de los datos.

La implantación de vitalXAI se orienta a dos escenarios de operación. El escenario principal es el despliegue de la plataforma en un equipo que aloja el servidor, la base de datos y los artefactos del aprendizaje automático, al que acceden los usuarios finales a través de la red; en la demostración del proyecto, el acceso se realiza mediante un túnel HTTPS que expone el servidor a través de una URL pública. La documentación y los requisitos de implantación se especifican para ambos escenarios, de modo que la operación del sistema queda cubierta tanto en el uso local como en el acceso remoto.

26.1. Documentación necesaria para la operación

Los entregables documentales comprometidos para el proyecto son la memoria del Trabajo Fin de Grado y el manual de usuario. La memoria recoge el análisis, el diseño, la implementación, las pruebas y las conclusiones. El manual de usuario proporciona las instrucciones necesarias para que los profesionales utilicen las funciones principales de la plataforma. La información técnica de explotación, administración, construcción y despliegue se mantiene distribuida en los capítulos de la memoria y en la guía interna de despliegue del proyecto, pero no se presenta como un manual independiente entregable.

Entregable	Perfil destinatario	Alcance del contenido
Manual de usuario	Profesionales de la plataforma: facultativos e investigadores.	Acceso y registro; diagnóstico asistido (subida de la radiografía, selección del modelo, solicitud del diagnóstico, visualización del resultado y de los mapas de explicabilidad, y generación del informe PDF); gestión del historial (listado, detalle, renombrado y eliminación); laboratorio de entrenamiento (configuración con el asistente, lanzamiento de experimentos, consulta de sesiones y resultados e informe de la sesión); y capacidades transversales (idioma, tema visual y cola de trabajos).
Memoria del Trabajo Fin de Grado	Tutor, tribunal y personas interesadas en el proyecto.	Plan de proyecto, análisis, diseño, implementación, pruebas, resultados, limitaciones y conclusiones.

Tabla x - Especificación de documentación necesaria para la operación

El manual de usuario se elabora en formato digital, en el mismo formato de documentación del proyecto, y se organiza por rol y por flujo funcional. Incluye los pasos de cada operación, las condiciones que deben cumplirse, los resultados esperados y los escenarios alternativos y de error. La memoria constituye el segundo entregable documental y conserva la descripción técnica necesaria para comprender la arquitectura, la construcción, la preparación de los datos y la implantación.

La guía de despliegue del proyecto es un documento interno de apoyo. Recoge los requisitos operativos y el procedimiento de la demostración, pero no forma parte de los entregables comprometidos. La documentación de diseño de la memoria constituye la referencia técnica de la arquitectura, el diseño de casos de uso, de clases y de interfaces, y de la construcción y preparación de los datos.

26.2. *Requisitos de Implantación*

La implantación en el entorno de operación define las necesidades de formación de los usuarios, los requisitos de infraestructura e instalación y el proceso de puesta en marcha de la plataforma. Los requisitos se apoyan en los capítulos de construcción y de preparación de los datos: la implantación dispone el sistema sobre el entorno descrito en el capítulo 23 y aplica la preparación inicial definida en el capítulo 24, y añade las condiciones de formación y de acceso propias de la operación.

Las necesidades de formación se orientan a los usuarios previstos para la plataforma. El manual de usuario describe la operación clínica de los facultativos y el uso del laboratorio de entrenamiento por los investigadores. La información sobre administración y puesta en marcha se encuentra en los capítulos técnicos correspondientes y en la guía interna de despliegue. No se realizaron sesiones de formación ni evaluaciones con usuarios reales durante el proyecto; por tanto, el manual no constituye evidencia de usabilidad.

Los requisitos de infraestructura e instalación se especifican para el entorno de operación, atendiendo al software, al hardware y a las comunicaciones. En cuanto al software, el sistema requiere Python en su versión 3.11 o superior, el servicio de MySQL (en el entorno de desarrollo, MariaDB mediante XAMPP) y las dependencias declaradas en el archivo requirements.txt (FastAPI, 2024; Oracle, 2024). En cuanto al hardware, la plataforma requiere recursos de memoria y de cómputo suficientes para la carga de los modelos de TensorFlow y la ejecución del pipeline MLOps, además de espacio de almacenamiento para los datasets, los pesos de los modelos y los resultados de los entrenamientos; una GPU puede reducir el tiempo de los entrenamientos, aunque no forma parte de un requisito mínimo documentado (TensorFlow, 2024). En cuanto a las comunicaciones, la operación local accede al servidor mediante la dirección local del equipo, mientras que el acceso remoto de la demostración se expone mediante un túnel HTTPS que publica el servidor a través de una URL pública; en un despliegue de producción, la comunicación debe cifrarse mediante HTTPS con un certificado válido (IETF, 2018).

El diagrama de implantación de la figura representa el escenario remoto de demostración. Los usuarios finales, el facultativo, el investigador y el administrador, acceden mediante su navegador a través del túnel HTTPS; el servidor de operación aloja la aplicación, la base de datos MySQL, los pesos de los modelos y los datasets, y el worker asíncrono. En el escenario local, el navegador accede directamente a la dirección local del servidor.

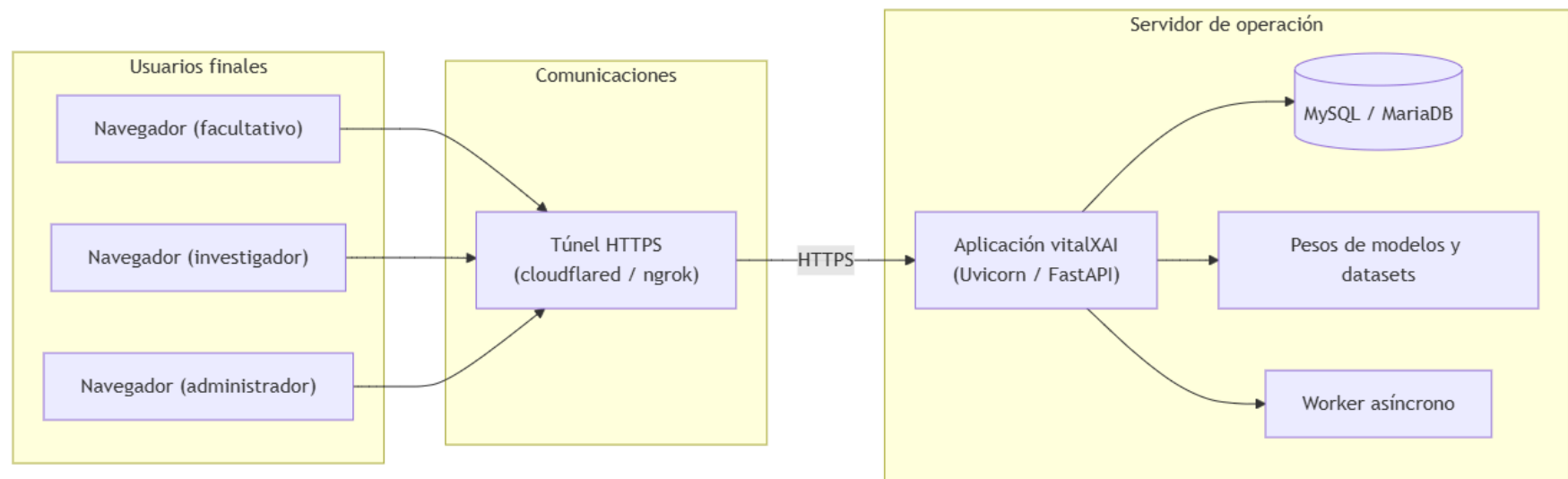


Ilustración x - Diagrama de implantación del sistema en el entorno de operación

El diagrama refleja la disposición de la demostración remota: los usuarios finales no acceden directamente al equipo que aloja el servidor, sino mediante el túnel HTTPS, de modo que los datos de la plataforma permanecen en el servidor de operación. El servidor agrupa la aplicación, la persistencia y los artefactos del aprendizaje automático, y el worker atiende las tareas asíncronas de la cola.

El proceso de implantación aplica los procedimientos definidos en los capítulos precedentes. Primero se construye el sistema conforme al capítulo 23: se prepara el entorno virtual y se instalan las dependencias. Después se prepara el entorno de datos conforme al capítulo 24: se verifican los requisitos, se inicia MySQL, se dispone el archivo `.env` junto con los datasets y los pesos, y se crean las cuentas de acceso. A continuación se ejecuta `main.py`, que inicializa el esquema y arranca el worker. Finalmente se establece el acceso local o remoto y se verifica el sistema conforme al capítulo 25. Con la infraestructura dispuesta y la documentación comprometida entregada, la plataforma queda preparada para su demostración.

Parte IV.

Codificación del Sistema

Capítulo 27. Introducción a la codificación

Esta parte de la memoria describe cómo se implementó vitalXAI a partir del diseño de los capítulos 17 a 26. El capítulo introductorio sitúa la organización del repositorio, las convenciones de código, las capas tecnológicas y las herramientas de calidad; los capítulos siguientes muestran las decisiones y los fragmentos más representativos de cada módulo. La implementación resultante es una aplicación Python servida por FastAPI, con persistencia en MySQL y un laboratorio MLOps basado en los servicios y scripts del proyecto.

La parte se organiza en seis capítulos. Este capítulo introductorio presenta la visión general de la implementación, la estructura del repositorio, las convenciones de código y las herramientas de calidad; los cinco capítulos siguientes describen la codificación de cada capa tecnológica: el backend (capítulo 28), la ejecución asíncrona (capítulo 29), el motor de diagnóstico y de explicabilidad (capítulo 30), el laboratorio de experimentación MLOps (capítulo 31) y el frontend (capítulo 32). Cada capítulo de codificación presenta los componentes implementados, los fragmentos de código representativos con su explicación y la coherencia con el diseño del sistema.

La implementación de vitalXAI se acompaña de una batería automatizada de pruebas unitarias y de integración, integrada en el flujo de integración continua. En el estado actual del repositorio se han ejecutado más de ciento ochenta pruebas; el plan del capítulo 16 define su alcance y sus criterios, pero esta cifra no implica que cada capacidad se desarrollara estrictamente mediante TDD (Myers, Sandler, & Badgett, 2011). Los capítulos siguientes describen la implementación real, con referencias a los módulos, los ficheros de pruebas y las decisiones técnicas que la sustentan.

27.1. *Visión general de la implementación*

La implementación de vitalXAI se organiza en torno a las capas definidas en el diseño: la capa HTTP, la capa de servicios, la persistencia relacional, la ejecución asíncrona, el motor de inteligencia artificial y la capa de presentación. La capa HTTP agrupa los routers que exponen los endpoints de la API; la capa de servicios concentra la lógica de aplicación, como criptografía, sesiones, predicción, explicabilidad, informes, laboratorio y cola; la persistencia relacional se resuelve con MySQL a través de un pool de conexiones; la ejecución asíncrona se materializa en la cola de trabajos y en el worker; el motor de inteligencia artificial agrupa la carga de los modelos y la generación de las explicaciones; y la capa de presentación se compone de las plantillas Jinja2 y los recursos JavaScript del navegador. Estas responsabilidades se distribuyen entre los directorios y módulos de la raíz, no en paquetes independientes para cada capa.

El lenguaje de implementación es Python en su versión 3.11, que alberga tanto el servidor web como el motor de aprendizaje profundo. El servidor se construye con FastAPI y se sirve mediante Uvicorn; las plantillas se componen con Jinja2; la persistencia se accede mediante el conector MySQL de Python; el aprendizaje profundo se apoya en TensorFlow y en la librería Transformers de Hugging Face; y la seguridad de las sesiones se resuelve con bcrypt, python-jose y slowapi. Estas tecnologías, junto con el resto de las dependencias del sistema, se describieron en el entorno de construcción del capítulo 23, y su implementación se detalla en los capítulos de codificación de esta parte.

La coherencia entre el diseño y la implementación se mantiene mediante la trazabilidad de los módulos: cada router y cada servicio del código se corresponde con los componentes descritos en el capítulo 17 y con las clases de diseño del capítulo 21, y cada endpoint materializa los casos de uso del capítulo 20. Esta correspondencia permite verificar que la implementación satisface el diseño y que los capítulos de codificación pueden leerse como la materialización concreta de las decisiones técnicas ya documentadas.

27.2. Estructura del repositorio

La estructura del repositorio del proyecto refleja la organización por capas de la implementación. El árbol de directorios de la siguiente ilustración representa la estructura del repositorio de vitalXAI y los ficheros principales de cada paquete, con sus agrupaciones funcionales.

La estructura distingue los ficheros raíz de la aplicación, los paquetes de la implementación y los directorios de artefactos. Los ficheros raíz incluyen el punto de entrada `main.py`, la persistencia `database.py`, las dependencias `requirements.txt`, la configuración de calidad `pyproject.toml`, la configuración de la integración continua en `.github/workflows/` y las variables de entorno. Los paquetes `routers/`, `services/`, `templates/` y `static/` contienen la implementación de cada capa; el directorio `pneumoniacnn-main/` alberga el pipeline de experimentación con sus scripts y los pesos de los modelos; el directorio `tests/` contiene las pruebas unitarias y de integración; el directorio `scripts/` reúne los scripts operativos de arranque y de migración; y los directorios `sql/`, `training_results/` y los subdirectorios de `static/` conservan los artefactos generados y los recursos de la aplicación.

```

vitalXAI/
├── .github/
│   └── workflows/
│       └── ci.yml
├── routers/
│   ├── admin.py
│   ├── auth.py
│   ├── history.py
│   ├── inference.py
│   ├── queue.py
│   └── trainer.py
├── services/
│   ├── auth_service.py
│   ├── chatbot_service.py
│   ├── csrf_middleware.py
│   ├── lang.py
│   ├── ml_engine.py
│   ├── mlops_engine.py
│   ├── pdf_generator.py
│   ├── pdf_generator_mlops.py
│   ├── queue_worker.py
│   ├── rate_limiter.py
│   ├── trainer_engine.py
│   └── xai_generator.py
├── static/
│   ├── js/
│   │   ├── admin.js
│   │   ├── dashboard.js
│   │   ├── i18n.js
│   │   └── training.js
│   ├── uploads/
│   ├── results/
│   └── reports/
├── templates/
│   ├── dashboard.html
│   ├── index.html
│   ├── login.html
│   ├── register.html
│   └── training.html
├── tests/
│   ├── unit/
│   └── integration/
├── pneumoniaccnn-main/
│   ├── code/
│   │   ├── 1_train_kfold.py
│   │   ├── 2_train_transformer_kfold.py
│   │   ├── 3_evaluate_statistics.py
│   │   ├── 4_external_validation.py
│   │   ├── 5_evaluate_delong.py
│   │   ├── 6_xai_qualitative.py
│   │   └── 7_xai_quantitative.py
│   └── results/
├── scripts/
│   ├── demo_start.bat
│   ├── demo_start.ps1
│   ├── migrate_passwords.py
│   └── migrate_roles.py
├── sql/
├── training_results/
├── .env.example
├── .gitignore
├── database.py
├── main.py
├── pyproject.toml
├── README.md
└── requirements.txt

```

Ilustración x - Estructura del repositorio de vitalXAI

27.3. Convenciones de código y decisiones transversales

La codificación de vitalXAI toma como referencia las convenciones de estilo de Python de PEP 8 (van Rossum, Warsaw, & Coghlan, 2001). Ruff revisa automáticamente el conjunto de reglas seleccionado por el proyecto, que incluye errores sintácticos, advertencias, importaciones, convenciones de nombres, actualizaciones de sintaxis, comprobaciones de seguridad y simplificaciones. La configuración fija una longitud máxima de línea de 140 caracteres y estas comprobaciones se ejecutan en el flujo de integración; esto ayuda a mantener un estilo uniforme, pero no equivale por sí solo a una conformidad completa con PEP 8.

Las decisiones transversales de codificación incluyen el uso de anotaciones de tipos, la política de comentarios y la gestión de los recursos. El proyecto emplea anotaciones en los módulos de mayor criticidad, especialmente los servicios de autenticación y seguridad, y dispone de una configuración de mypy para comprobar esos módulos cuando se ejecuta la herramienta. Los comentarios se reservan para explicar la intención y las decisiones no evidentes de cada bloque, sin narrar la mecánica obvia del código. Las variables de entorno se leen mediante la función de carga del entorno, de modo que la configuración sensible permanece fuera del código y se suministra en tiempo de ejecución. Los identificadores de los módulos y de las funciones siguen las convenciones de nombres de Python, con módulos en minúsculas separados por guiones bajos y funciones con la misma convención.

27.4. Herramientas y calidad del código

La calidad del código de vitalXAI se comprueba mediante herramientas integradas en el flujo de trabajo. pytest ejecuta las pruebas unitarias y de integración y pytest-cov mide la cobertura de los módulos de la aplicación; los dobles de prueba se construyen principalmente con unittest.mock y fixtures de pytest (pytest, 2024). La cobertura está sujeta a un umbral mínimo del setenta por ciento, de modo que la configuración de CI hace fallar la batería si no se alcanza. Ruff comprueba el estilo y las reglas del código (Ruff, 2024), mientras que mypy dispone de una configuración para la verificación de tipos, aunque no se ejecuta actualmente en CI (Mypy, 2024).

La verificación se integra en la integración continua mediante el workflow del repositorio, que se ejecuta en los eventos configurados para las ramas principales y las peticiones de cambios. El workflow instala las dependencias, ejecuta Ruff y ejecuta pytest con medición de cobertura. De este modo se comprueban automáticamente el comportamiento cubierto por las pruebas, el estilo y el umbral de cobertura; la verificación de tipos requiere ejecutar mypy por separado. Esta estrategia es coherente con la estrategia de verificación del capítulo 25, aunque no permite afirmar que toda la implementación esté verificada.

Capítulo 28. Codificación del backend

El backend de vitalXAI materializa las capas de la aplicación descritas en el capítulo 23: la capa HTTP, la capa de servicios, la persistencia relacional y los mecanismos de seguridad transversales. Este capítulo describe la codificación de estas capas, presentando los componentes implementados y los fragmentos de código más representativos de cada uno, con la explicación de las decisiones de implementación que los sustentan. La codificación del backend se organiza en cinco apartados: el arranque y la configuración de la aplicación, la capa HTTP de los routers, los servicios de aplicación, la persistencia y el acceso a los datos, y la seguridad transversal.

El backend se construye sobre FastAPI, servido por Uvicorn, y se apoya en MySQL para la persistencia (FastAPI, 2024; Uvicorn, 2024; Oracle, 2024). La implementación sigue el patrón de fachada ligera descrito en el diseño: los routers se ocupan de la recepción y la serialización de las peticiones HTTP, y delegan la lógica en los servicios, que concentran las operaciones de aplicación y acceden a la persistencia. Esta separación, ya declarada en el capítulo 17, se refleja en la estructura de los paquetes routers/ y services/ descrita en el capítulo 27, y permite verificar cada componente de forma aislada mediante las pruebas unitarias del capítulo 16.

Los fragmentos incluidos en este capítulo muestran solo las partes necesarias para explicar las decisiones principales del backend. La implementación completa está disponible en el repositorio público del proyecto: github.com/eLeCe2611/vitalXAI. El contenido de este capítulo corresponde principalmente a `main.py`, `database.py`, `routers/`, `services/auth_service.py` y los módulos de configuración y seguridad del proyecto. Las rutas y nombres utilizados en la memoria corresponden a la estructura del repositorio consultada para esta versión.

28.1. Arranque y configuración de la aplicación

El punto de entrada de la aplicación se implementa en el módulo `main.py`, que compone la aplicación FastAPI, configura los mecanismos de seguridad transversales, monta los recursos estáticos e integra los routers de los subsistemas. El arranque se gestiona mediante el ciclo de vida de la aplicación: al iniciarse, se inicializa la base de datos y se arranca el worker de la cola. El código no define una operación de parada propia del worker al finalizar el ciclo de vida; su terminación depende del apagado del proceso servidor. El fragmento siguiente muestra la implementación del ciclo de vida y de la composición de la aplicación.

La implementación del arranque refleja las decisiones del diseño. La inicialización de la base de datos se realiza mediante la función `init_db()` de `database.py`, que crea las tablas del esquema si no existen, de modo que el arranque no requiere un script de instalación previo; el fallo de conexión se captura y se informa sin interrumpir el arranque, en coherencia con el entorno de construcción del capítulo 23. El worker de la cola se arranca mediante `start_worker()`, que crea la tarea asíncrona del procesamiento en segundo plano. Los mecanismos de seguridad se añaden como middleware (las cabeceras de seguridad y la protección CSRF) y el limitador de peticiones se registra en el estado de la aplicación con su manejador de excepciones. Los recursos estáticos y el directorio de resultados de entrenamiento se montan como directorios servidos por la aplicación, y los routers de los seis subsistemas se integran mediante la inclusión de sus enrutadores.


```

@asynccontextmanager
async def lifespan(app: FastAPI):
    try:
        init_db()
        print("Base de datos conectada e inicializada correctamente.")
    except Exception:
        print("ATENCIÓN: No se pudo conectar a la base de datos MySQL.")
    start_worker(app)
    print("Worker de cola iniciado.")
    yield

app = FastAPI(title="X-Ray AI Consultant", lifespan=lifespan)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)
app.add_middleware(SecurityHeadersMiddleware)
app.add_middleware(CSRFMiddleware)
app.mount("/static", StaticFiles(directory="static"), name="static")
os.makedirs("training_results", exist_ok=True)
app.mount("/training_results", StaticFiles(directory="training_results"), name="training_results")

app.include_router(admin.router)
app.include_router(auth.router)
app.include_router(history.router)
app.include_router(inference.router)
app.include_router(queue.router)
app.include_router(trainer.router)

```

Código 1 - Arranque y composición de la aplicación (main.py)

28.2. Capa HTTP: los routers

La capa HTTP se implementa en el paquete `routers/`, con un módulo por subsistema: `auth.py` para el acceso y las sesiones, `inference.py` para el diagnóstico, `history.py` para el historial, `queue.py` para la cola de trabajos, `trainer.py` para el laboratorio MLOps y `admin.py` para la administración. Cada router sigue el patrón de fachada ligera del diseño: resuelve la identidad del usuario, valida los datos de entrada, delega en los servicios y devuelve las respuestas HTTP con los códigos diferenciados. El fragmento siguiente muestra la implementación del endpoint de solicitud de diagnóstico del router `inference.py`, que materializa la subida de la radiografía y el encolado del trabajo.

La implementación del endpoint refleja las decisiones del diseño del subsistema de diagnóstico. La identidad se resuelve mediante el mecanismo común de identificación de SD-001, y las validaciones del tipo MIME y del tamaño se realizan antes de escribir el fichero en disco, de modo que una petición inválida no ocupa espacio ni una posición en la cola. La imagen se conserva en el área de cargas con un nombre basado en la fecha, y el trabajo se encola con un payload serializable que solo contiene el modelo, la ruta de la imagen y el idioma. La respuesta devuelve el estado `queued` con la posición, y el mensaje se localiza mediante el servicio de idioma. Este patrón (resolver la identidad, validar, delegar y responder con códigos diferenciados) se repite en los routers de los demás subsistemas, con las variaciones propias de cada flujo.

```

@router.post("/predict")
async def predict(request: Request, file: UploadFile = File(...), model_name: str = Form(...)):
    user_id = get_user_id_from_token(request.cookies.get("access_token"))
    if not user_id:
        return JSONResponse(status_code=401, content={"error": get_text("no_autenticado")})

    if file.content_type not in _ALLOWED_MIME_TYPES:
        return JSONResponse(status_code=400, content={"status": "error", "message":
get_text("solo_imagenes")})

    file.file.seek(0, 2)
    file_size = file.file.tell()
    file.file.seek(0)
    if file_size > _MAX_FILE_SIZE:
        return JSONResponse(status_code=400, content={"status": "error", "message":
get_text("imagen_muy_grande")})

    timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"{timestamp}_{file.filename}"
    upload_path = os.path.join("static", "uploads", filename)
    with open(upload_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    job_id = _enqueue_job(user_id, "diagnosis", {
        "model_name": model_name, "image_path": upload_path,
        "lang": get_lang_from_cookie(request),
    })
    position = _queue_position(job_id, "diagnosis")
    return JSONResponse(content={
        "status": "queued", "job_id": job_id, "position": position,
        "message": get_text("diagnostico_encolado").format(position=position),
    })

```

Código 2 - Solicitud de diagnóstico (routers/inference.py)

28.3. Servicios de aplicación

La capa de servicios se implementa en el paquete `services/` y concentra la lógica de aplicación del sistema. Los servicios se organizan por responsabilidad: `auth_service.py` gestiona la criptografía y las sesiones, `lang.py` la internacionalización, `ml_engine.py` y `xai_generator.py` la predicción y la explicabilidad, `pdf_generator.py` los informes del diagnóstico, `mlops_engine.py` y `trainer_engine.py` el laboratorio, `queue_worker.py` el procesamiento asíncrono, `chatbot_service.py` el asistente conversacional y `rate_limiter.py` y `csrf_middleware.py` los mecanismos de seguridad. El fragmento siguiente muestra la implementación de la creación y la verificación de los tokens de sesión del servicio de autenticación.

La implementación de los tokens refleja la política de seguridad de sesiones del diseño. El token de acceso se firma con JWT mediante la clave secreta del entorno e incluye el identificador del usuario y la fecha de expiración; el token de refresco se genera como un valor aleatorio con `secrets.token_urlsafe`, y en la base de datos solo se conserva su hash SHA-256 junto con la expiración, de modo que una credencial reutilizable no se almacena en texto plano. La gestión de la rotación y de la detección de robo, descrita en el capítulo 17, se implementa en las funciones de verificación del mismo servicio, que distinguen el token activo, el revocado dentro del periodo de gracia y el uso de una credencial revocada, revocando todos los tokens del usuario en este último caso.

```

def create_access_token(user_id: int) -> str:
    expire = datetime.datetime.now(datetime.UTC) +
datetime.timedelta(minutes=JWT_ACCESS_EXPIRE_MINUTES)
    payload = {"sub": str(user_id), "exp": expire}
    return jwt.encode(payload, JWT_SECRET_KEY, algorithm="HS256")

def create_refresh_token(user_id: int) -> str:
    raw_token = secrets.token_urlsafe(48)
    token_hash = _hash_token(raw_token)
    expires_at = datetime.datetime.now(datetime.UTC) +
datetime.timedelta(days=JWT_REFRESH_EXPIRE_DAYS)
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO refresh_tokens (user_id, token_hash, expires_at) VALUES (%s, %s, %s)",
        (user_id, token_hash, expires_at),
    )
    conn.commit()
    conn.close()
    return raw_token

```

Código 3 - Creación de los tokens de sesión (services/auth_service.py)

28.4. Persistencia y acceso a los datos

La persistencia se implementa en el módulo `database.py`, que gestiona el pool de conexiones a MySQL y la inicialización del esquema. El acceso a los datos se resuelve mediante un pool de conexiones configurado con las variables de entorno, de modo que las peticiones concurrentes reutilizan las conexiones del pool sin abrir una conexión por operación. El fragmento siguiente muestra la configuración del pool y la inicialización del esquema.

La implementación de la persistencia refleja el modelo físico del capítulo 19 y la generación del esquema del capítulo 23. El pool se configura con los parámetros del entorno y se crea perezosamente en la primera conexión; la inicialización del esquema ejecuta las sentencias `CREATE TABLE IF NOT EXISTS` de las cinco tablas del sistema, `users`, `consultations`, `training_jobs`, `job_queue` y `refresh_tokens`, de modo que el arranque crea el esquema sin un script de migración manual. El fragmento muestra el patrón aplicado a la tabla `users`; las restantes tablas se crean en la misma función. Los routers y los servicios obtienen las conexiones mediante la función común, aplican las operaciones y normalmente las cierran tras confirmar la transacción, manteniendo la integridad de los datos.

```

def _get_pool():
    global _pool
    if _pool is None:
        _pool = MySQLConnectionPool(
            pool_name="mypool",
            pool_size=int(os.getenv("DB_POOL_SIZE", "5")),
            host=os.getenv("DB_HOST", "localhost"),
            user=os.getenv("DB_USER", "root"),
            password=os.getenv("DB_PASSWORD", ""),
            database=os.getenv("DB_NAME", "tfg_pneumonia"),
        )
    return _pool

def get_db_connection():
    return _get_pool().get_connection()

def init_db():
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INT AUTO_INCREMENT PRIMARY KEY,
            username VARCHAR(255) NOT NULL UNIQUE,
            password_hash VARCHAR(255) NOT NULL,
            first_name VARCHAR(255) NOT NULL,
            last_name VARCHAR(255) NOT NULL,
            role VARCHAR(255) NOT NULL
        )
    """)
    conn.commit()
    conn.close()

```

Código 4 - Pool de conexiones e inicialización del esquema (database.py)

28.5. Seguridad transversal

La seguridad transversal del backend se implementa en los servicios `csrf_middleware.py` y `rate_limiter.py`, que se aplican a toda la aplicación mediante el middleware y el limitador configurados en el arranque. La protección CSRF se implementa como un middleware que establece una cookie de token en las peticiones seguras y exige su correspondencia en las peticiones que modifican el estado; las cabeceras de seguridad se añaden a todas las respuestas. El fragmento siguiente muestra la implementación de la protección CSRF.

La implementación de la protección CSRF refleja los requisitos de seguridad del análisis y las prácticas habituales de protección de aplicaciones web (OWASP, 2024). Las peticiones seguras, GET, HEAD y OPTIONS, se eximen de la comprobación y establecen la cookie del token si no existe; el endpoint de inicio de sesión se exime porque se limita en frecuencia de forma independiente. Las peticiones que modifican el estado exigen que el token de la cookie coincida con el de la cabecera `x-csrf-token`, que la interfaz envía de forma automática en las peticiones asíncronas; si no coinciden, el middleware responde HTTP 403 sin procesar la petición. Las cabeceras de seguridad se añaden en todas las respuestas. El limitador aplica un límite global de sesenta peticiones por minuto y, mediante el decorador específico del inicio de sesión, un límite de cinco peticiones por minuto para ese endpoint, en coherencia con las pruebas de protección del capítulo 16.

El backend de vitalXAI queda así codificado de forma coherente con el diseño: la aplicación se compone en el arranque con los mecanismos de seguridad y los routers de los seis subsistemas, la capa HTTP sigue el patrón de fachada ligera, los servicios concentran la lógica de aplicación, la persistencia se resuelve con el pool de conexiones y la inicialización del esquema, y la seguridad transversal protege las peticiones y las respuestas de toda la aplicación. La implementación de la ejecución asíncrona, que constituye el soporte del procesamiento en segundo plano, se describe en el capítulo siguiente.


```

class CSRFMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next: RequestResponseEndpoint) -> Response:
        if request.method in SAFE_METHODS or request.url.path in CSRF_EXEMPT_PATHS:
            response = await call_next(request)
            if CSRF_COOKIE_NAME not in request.cookies:
                token = secrets.token_urlsafe(32)
                response.set_cookie(key=CSRF_COOKIE_NAME, value=token, httponly=False,
samesite="lax")
            return response

        csrf_cookie = request.cookies.get(CSRF_COOKIE_NAME)
        csrf_token = request.headers.get(CSRF_HEADER_NAME)
        if not csrf_cookie or not csrf_token or csrf_cookie != csrf_token:
            return JSONResponse(status_code=403, content={"status": "error", "message": "CSRF
validation failed"})

        response = await call_next(request)
        return response

```

Código 5 - Protección CSRF (services/csrf_middleware.py)

Capítulo 29. Codificación de la ejecución asíncrona

La ejecución asíncrona constituye el soporte del procesamiento en segundo plano de vitalXAI: los diagnósticos, los entrenamientos y las validaciones externas se procesan fuera del ciclo de petición HTTP, de modo que la interfaz permanece operativa durante las tareas de larga duración. Este capítulo describe la codificación de este mecanismo, que se materializa en la cola persistente de trabajos y en el worker de la cola. La implementación se organiza en cinco apartados: la cola de trabajos y su máquina de estados, el bucle del worker, la reclamación y el despacho de los trabajos, la tolerancia a fallos y la recuperación, y la consulta y la cancelación desde la API.

El código completo de la cola, el worker y sus pruebas se encuentra en el repositorio del proyecto: github.com/eLeCe2611/vitalXAI. El contenido de este capítulo corresponde principalmente a `services/queue_worker.py`, `routers/queue.py`, la tabla `job_queue` definida en `database.py` y las pruebas de `tests/unit/test_queue_worker.py` y `tests/unit/test_queue_router.py`. En este capítulo se reproducen únicamente los fragmentos que explican la máquina de estados, la reclamación, el despacho y la recuperación de trabajos.

29.1. *La cola de trabajos y su máquina de estados*

La cola de trabajos se implementa en la tabla `job_queue`, cuyas filas representan los trabajos asíncronos del sistema. Cada trabajo conserva el usuario propietario, el tipo (`diagnosis`, `training` o `external_validation`), el estado, el `payload` serializable, el resultado y el mensaje de error, de modo que la tabla materializa la clase `QueueJob` definida en el diseño de clases del capítulo 21 y su máquina de estados, representada en la figura 89. Las transiciones de estado se implementan en el worker mediante operaciones que actualizan la fila del trabajo: la reclamación, la finalización y el fallo. El fragmento siguiente muestra la implementación de estas transiciones.

La implementación de las transiciones refleja la máquina de estados de la cola. La reclamación utiliza una actualización condicionada a que el trabajo continúe en `queued`, de modo que solo afecta a una fila si el trabajo seguía pendiente; el resultado de la actualización determina si el worker ha adquirido el trabajo, lo que impide que dos iteraciones procesen el mismo registro. La finalización marca el trabajo como `completed` con el resultado en formato JSON, y el fallo lo marca como `failed` con el mensaje de error limitado a quinientos caracteres, de modo que el estado persistido nunca conserva un error desbordado. La cancelación desde la API se implementa con la misma técnica condicionada, limitada a los trabajos en `queued`, tal y como se describe en el apartado 29.5.

```

def _claim_job(job_id: int):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE job_queue SET status = 'running', started_at = %s WHERE id = %s AND status = 'queued'",
        (datetime.now(), job_id),
    )
    conn.commit()
    affected = cursor.rowcount
    conn.close()
    return affected > 0

def _finish_job(job_id: int, result: dict | None = None):
    conn = get_db_connection()
    cursor = conn.cursor()
    result_json = json.dumps(result) if result else None
    cursor.execute(
        "UPDATE job_queue SET status = 'completed', finished_at = %s, result = %s WHERE id = %s",
        (datetime.now(), result_json, job_id),
    )
    conn.commit()
    conn.close()

def _fail_job(job_id: int, error: str):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE job_queue SET status = 'failed', finished_at = %s, error_message = %s WHERE id = %s",
        (datetime.now(), error[:500], job_id),
    )
    conn.commit()
    conn.close()

```

Código 6 - Transiciones de estado del trabajo (services/queue_worker.py)

29.2. *El bucle del worker*

El worker se implementa como un bucle asíncrono que selecciona el siguiente trabajo pendiente, lo reclama, procesa el flujo correspondiente y actualiza su estado mediante las primitivas de `asyncio` (Python Software Foundation, 2024b). El bucle se crea en el arranque de la aplicación mediante `start_worker()`, que restablece los trabajos en ejecución y lanza la tarea asíncrona del procesamiento. El fragmento siguiente muestra la implementación del bucle del worker.

La implementación del bucle refleja las decisiones de diseño del procesamiento asíncrono. El worker selecciona el primer trabajo pendiente mediante la consulta de prioridad, que ordena los diagnósticos antes que los entrenamientos; si no hay trabajos, espera un segundo y vuelve a consultar. Cuando hay un trabajo, lo reclama con la actualización condicionada; si la reclamación no afecta a una fila, otro consumidor ya lo tomó y el worker continúa con el siguiente. La ejecución del flujo se delega en el executor de eventos mediante `run_in_executor`, de modo que el procesamiento intensivo no bloquea el bucle de eventos de la aplicación; al terminar, el trabajo se marca como completado con el resultado, y ante cualquier excepción se marca como fallido con el mensaje de error. El arranque del worker se integra en el ciclo de vida de la aplicación descrito en el capítulo 28, y conserva la tarea en el estado de la aplicación.

```

async def worker_loop():
    loop = asyncio.get_running_loop()
    while True:
        try:
            job = _next_job()
            if job is None:
                await asyncio.sleep(1)
                continue

            claimed = _claim_job(job["id"])
            if not claimed:
                continue

            try:
                result = await loop.run_in_executor(None, _execute_job, job)
                _finish_job(job["id"], result)
            except Exception as e:
                _fail_job(job["id"], str(e))
        except Exception:
            await asyncio.sleep(1)

def start_worker(app):
    _reset_running_jobs()
    task = asyncio.create_task(worker_loop())
    app.state.queue_worker = task

```

Código 7 - Bucle del worker y su arranque (services/queue_worker.py)

29.3. Reclamación, despacho y procesamiento de los trabajos

El procesamiento de cada trabajo se resuelve mediante el despacho por tipo, que selecciona el flujo correspondiente según el tipo del trabajo. La reclamación condicionada, descrita en el apartado 29.1, hace que el despacho solo continúe cuando el worker ha adquirido el trabajo. El fragmento siguiente muestra la implementación del despacho y del procesamiento de un diagnóstico.

La implementación del procesamiento de un diagnóstico refleja el flujo asíncrono definido en el diseño del subsistema SD-002. El worker deserializa el payload del trabajo, invoca el motor de predicción, genera el mapa de explicabilidad y construye el informe PDF, y persiste la consulta en la tabla consultations con las rutas de los artefactos y el resultado. El procesamiento se ejecuta en el executor, de modo que la predicción y la generación de los artefactos no bloquean la interfaz. Los procesamientos de los entrenamientos y de las validaciones externas siguen el mismo patrón y delegan en el motor del laboratorio MLOps, que se describe en el capítulo 31. El resultado devuelto se conserva en el campo result del trabajo completado.

```

def _execute_job(job):
    if job["job_type"] == "diagnosis":
        return _process_diagnosis(job)
    elif job["job_type"] == "training":
        return _process_training(job)
    elif job["job_type"] == "external_validation":
        return _process_external_validation(job)
    else:
        raise ValueError(f"Unknown job type: {job['job_type']}")

def _process_diagnosis(job):
    payload = _get_payload(job)
    label, confidence = process_and_predict(payload["model_name"], payload["image_path"],
    lang=payload.get("lang", "es"))

    xai_path = os.path.join("static", "results", "xai_" + os.path.basename(payload["image_path"]))
    generate_xai_heatmap(payload["model_name"], payload["image_path"], xai_path,
    lang=payload.get("lang", "es"))

    pdf_path = generate_medical_report(payload["image_path"], xai_path, label, confidence,
    payload["model_name"], lang=payload.get("lang", "es"))

    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO consultations (user_id, model_name, original_image_path, xai_image_path,
    prediction_label, confidence_score, patient_name, pdf_path) VALUES (%s, %s, %s, %s, %s, %s, %s,
    %s)",
        (job["user_id"], payload["model_name"], payload["image_path"], xai_path, label, confidence,
    default_patient_name, pdf_path),
    )
    conn.commit()
    conn.close()
    return {"label": label, "confidence": confidence, "original_image": f"/{payload['image_path']}",
        "xai_image": f"/{xai_path}", "pdf_report": f"/{pdf_path}", "model_used": payload["model_name"]}

```

Código 8 - Despacho por tipo y procesamiento de un diagnóstico (services/queue_worker.py)

29.4. Tolerancia a fallos y recuperación

La ejecución asíncrona incorpora mecanismos de tolerancia a fallos y de recuperación que garantizan la coherencia del sistema ante errores y reinicios. Ante un fallo del procesamiento, el worker captura la excepción y marca el trabajo como fallido con el mensaje de error, de modo que un fallo del motor, de la imagen o del informe no se transforma en una consulta completada con datos ambiguos. Ante el reinicio de la aplicación, los trabajos que quedaron en estado running se restablecen a queued, de modo que el procesamiento interrumpido vuelve a la cola y puede ser retomado. El fragmento siguiente muestra la implementación del restablecimiento de los trabajos en ejecución.

La implementación del restablecimiento refleja la decisión de recuperación del diseño. Al arrancar la aplicación, `start_worker()` ejecuta `_reset_running_jobs()`, que devuelve a la cola los trabajos que quedaron en running cuando el proceso anterior se detuvo, con su fecha de inicio restablecida. La reclamación condicionada evita que dos consumidores adquieran el mismo registro en una misma carrera, pero no permite reconstruir el punto intermedio del cálculo ni elimina los artefactos parciales. Esta técnica, junto con la captura de excepciones del bucle, permite reintentar trabajos sin dejar su estado operativo oculto, aunque la idempotencia completa requiere controles adicionales.


```
def _reset_running_jobs():
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("UPDATE job_queue SET status = 'queued', started_at = NULL WHERE status = 'running'")
    conn.commit()
    conn.close()
```

Código 9 - Restablecimiento de los trabajos en ejecución (services/queue_worker.py)

29.5. *La cola desde la API*

La consulta del estado y la cancelación de los trabajos se exponen en el router `routers/queue.py`, que materializa los casos de uso CU-034 y CU-035 del análisis. La consulta del estado devuelve los trabajos del usuario con su tipo, su estado, su posición y el nombre del modelo o de la sesión, interpretando el payload según el tipo; la cancelación aplica una actualización condicionada que solo afecta a los trabajos en `queued` pertenecientes al usuario. El fragmento siguiente muestra la implementación de la cancelación.

La implementación de la cancelación refleja la restricción del diseño: solo un trabajo en estado `queued` y perteneciente al usuario puede cancelarse. La actualización condicionada a esos tres criterios afecta a una fila únicamente cuando el trabajo sigue pendiente; si el worker ya lo reclamó o el trabajo finalizó, la actualización no afecta a ninguna fila y el router responde HTTP 404 informando de que el trabajo no está en cola. De este modo, un trabajo en ejecución no se interrumpe de forma abrupta, en coherencia con la máquina de estados de la cola. La consulta del estado, por su parte, interpreta el payload de cada tipo de trabajo para mostrar el nombre del modelo o de la sesión sin exponer el contenido interno completo del payload.

La ejecución asíncrona de `vitalXAI` queda descrita en sus elementos principales: la cola persistente materializa la máquina de estados de los trabajos, el worker reclama y procesa los trabajos fuera del ciclo de petición, el despacho por tipo orquesta los flujos de los subsistemas, la recuperación permite reintentar trabajos tras un reinicio y la API expone la consulta y la cancelación a los usuarios. La implementación de los motores que el worker invoca, la predicción y la explicabilidad del diagnóstico y el laboratorio MLOps, se describe en los capítulos siguientes.

```

@router.delete("/api/queue/cancel/{job_id}")
async def cancel_job(request: Request, job_id: int):
    user_id = get_user_id_from_token(request.cookies.get("access_token"))
    if not user_id:
        return JSONResponse(status_code=401, content={"error": "No autenticado"})

    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE job_queue SET status = 'cancelled', finished_at = NOW() WHERE id = %s AND user_id"
        = %s AND status = 'queued'",
        (job_id, user_id),
    )
    affected = cursor.rowcount
    conn.commit()
    conn.close()

    if affected == 0:
        return JSONResponse(status_code=404, content={"error": "Trabajo no encontrado o ya no está"
        en cola"})
    return JSONResponse(content={"status": "success", "message": "Trabajo cancelado"})

```

Código 10 - Cancelación de un trabajo desde la API (routers/queue.py)

Capítulo 30. Codificación del motor de diagnóstico y XAI

Este capítulo describe la implementación del motor que procesa una radiografía, obtiene una predicción, genera su explicación visual y prepara el informe PDF asociado. Estas responsabilidades se reparten entre tres servicios: `ml_engine.py`, que carga los modelos y ejecuta la predicción; `xai_generator.py`, que genera los mapas de explicabilidad; y `pdf_generator.py`, que construye el informe. El contenido se organiza en cuatro apartados dedicados a la carga y caché de modelos, el preprocesado y la predicción, los mapas de explicabilidad y el informe PDF.

El motor se apoya en las librerías de aprendizaje profundo y de visión por computador descritas en el entorno de construcción del capítulo 23: TensorFlow y la librería Transformers de Hugging Face para la carga de las arquitecturas, OpenCV para el preprocesado de las imágenes y FPDF para la generación del informe (TensorFlow, 2024; Hugging Face, 2024; FPDF2, 2024). La implementación distingue las arquitecturas convolucionales de las arquitecturas Transformer, cuyas decisiones de carga, preprocesado e inferencia se bifurcan en cada servicio, en coherencia con el diseño del subsistema SD-002 descrito en el capítulo 17.

La implementación completa del motor de diagnóstico, la generación XAI y los informes está disponible en github.com/eLeCe2611/vitalXAI, especialmente en `services/ml_engine.py`, `services/xai_generator.py` y `services/pdf_generator.py`. Los fragmentos de este capítulo son representativos y se incluyen para explicar el funcionamiento, no como sustituto del código completo.

30.1. Carga y caché de los modelos

La carga de los modelos se implementa en el servicio `ml_engine.py`, que diferencia el procedimiento según la familia de la arquitectura. Los modelos convolucionales se cargan desde sus pesos entrenados en formato Keras; los modelos Transformer se cargan desde su arquitectura preentrenada de Hugging Face y, a continuación, se aplican los pesos entrenados del proyecto. En ambos casos, el modelo se conserva en un diccionario global de caché, de modo que la primera consulta con cada arquitectura paga la carga de los pesos y las posteriores reutilizan el modelo en memoria. El fragmento siguiente muestra la implementación de la carga.

La implementación de la carga refleja las decisiones del diseño del motor. La diferenciación por familia permite cargar cada arquitectura con el procedimiento adecuado: las CNN se cargan directamente con el cargador de modelos de Keras, y los Transformer se construyen sobre la arquitectura de Hugging Face, con la salida de atención habilitada para la generación de los mapas de explicabilidad, y reciben los pesos entrenados del proyecto. La caché en memoria implementa el requisito de tiempo de respuesta de la inferencia (RNF-019): la primera consulta con cada arquitectura asume el coste de la carga, mientras que las posteriores reutilizan el modelo. La ausencia de los pesos se resuelve con un error explícito, de modo que un modelo no disponible falla antes de producir un resultado ambiguo, tal y como se declaró en el diseño.

```

loaded_models = {}
MODELS_TRANSFORMERS = ["deit", "swin_base", "vit_384"]
HF_MODEL_IDS = {
    "deit": "facebook/deit-base-distilled-patch16-224",
    "swin_base": "microsoft/swin-base-patch4-window7-224",
    "vit_384": "google/vit-base-patch16-384",
}

def get_model(model_name: str):
    if model_name not in loaded_models:
        is_transformer = model_name in MODELS_TRANSFORMERS

        if is_transformer:
            from transformers import TFAutoModelForImageClassification
            model_path = os.path.join("pneumoniaccnn-main", "results", model_name, "best_fold1.h5")
            if not os.path.exists(model_path):
                model_path = os.path.join("pneumoniaccnn-main", "results", model_name,
"best_fold1.weights.h5")
            if not os.path.exists(model_path):
                raise FileNotFoundError(f"Pesos de Transformer no encontrados en: {model_path}")

            model = TFAutoModelForImageClassification.from_pretrained(
                HF_MODEL_IDS[model_name], num_labels=1, ignore_mismatched_sizes=True,
output_attentions=True
            )
            model.load_weights(model_path)
            loaded_models[model_name] = model
        else:
            model_path = os.path.join("pneumoniaccnn-main", "results", model_name, "best_fold1.keras")
            if not os.path.exists(model_path):
                raise FileNotFoundError(f"Modelo CNN no encontrado en: {model_path}")
            loaded_models[model_name] = load_model(model_path)

    return loaded_models[model_name]

```

Código 11 - Carga y caché de los modelos (services/ml_engine.py)

30.2. Preprocesado y predicción

La predicción se implementa en la función `process_and_predict`, que prepara la imagen según la arquitectura y ejecuta la inferencia. El preprocesado ajusta la resolución de entrada a la arquitectura seleccionada (299×299 para InceptionV3 y Xception, 384×384 para ViT-384 y 224×224 para el resto), convierte la imagen al espacio RGB y la normaliza. La inferencia se bifurca por familia: los Transformer procesan la imagen con los `pixel_values` y aplican una sigmoide sobre el logit, mientras que las CNN utilizan la predicción directa del modelo. El fragmento siguiente muestra la implementación del preprocesado y de la inferencia.

La implementación de la predicción refleja las decisiones del diseño clínico. La normalización divide los píxeles entre 255 para escalarlos al intervalo de entrada de los modelos; la inferencia devuelve la probabilidad de la clase neumonía, y el umbral de 0.5 decide la etiqueta de la predicción. El valor mostrado como confianza se expresa en el intervalo 0-100: para la clase neumonía coincide con la probabilidad, y para la clase normal se calcula como su complementaria. Esta cifra es la salida transformada del modelo y no constituye por sí sola una garantía de calibración clínica, cuya evaluación requiere medidas específicas como el Brier Score o el Expected Calibration Error (Guo, Pleiss, Sun, & Weinberger, 2017). Las etiquetas se localizan mediante el servicio de idioma, en coherencia con la internacionalización de la plataforma.

```

def process_and_predict(model_name: str, image_path: str, lang: str = "es"):
    model = get_model(model_name)
    is_transformer = model_name in MODELS_TRANSFORMERS

    if model_name in ["InceptionV3", "Xception"]:
        img_size = (299, 299)
    elif model_name == "vit_384":
        img_size = (384, 384)
    else:
        img_size = (224, 224)

    img = cv2.imread(image_path)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img_resized = cv2.resize(img_rgb, img_size)
    img_array = img_resized / 255.0
    img_tensor = np.expand_dims(img_array, axis=0)

    if is_transformer:
        img_tensor_tf = tf.convert_to_tensor(img_tensor, dtype=tf.float32)
        logits = model(pixel_values=img_tensor_tf).logits
        prediction = float(tf.nn.sigmoid(logits)[0][0])
    else:
        prediction = float(model.predict(img_tensor, verbose=0)[0][0])

    is_pneumonia = prediction > 0.5
    label = get_text("label_pneumonia", lang) if is_pneumonia else get_text("label_normal", lang)
    confidence = prediction if is_pneumonia else (1 - prediction)
    confidence_percent = round(confidence * 100, 2)

    return label, confidence_percent

```

Código 12 - Preprocesado y predicción (services/ml_engine.py)

30.3. *Generación de los mapas de explicabilidad*

La generación de los mapas de explicabilidad se implementa en el servicio `xai_generator.py`, que calcula los tres mapas de la explicación visual y compone la figura con la radiografía original. Para las arquitecturas convolucionales se aplican los métodos basados en gradientes (Saliency, SmoothGrad y Grad-CAM), mientras que para las Transformer se utiliza el mapa de atención de la última capa, con un respaldo basado en gradientes ante un fallo del cálculo de la atención. El fragmento siguiente muestra la implementación de la generación del mapa de explicabilidad.

La implementación de la explicabilidad refleja las decisiones del diseño del subsistema de diagnóstico. El mapa de Saliency se calcula mediante los gradientes de la puntuación de la clase respecto a la imagen, aplicados con una cinta de gradientes de TensorFlow; SmoothGrad promedia treinta mapas de Saliency con ruido gaussiano y normaliza el resultado; y Grad-CAM pondera las activaciones de la última capa convolucional por los gradientes, mientras que las Transformer producen un mapa de atención a partir de la última capa de atención. La figura resultante combina la radiografía original, el Saliency, el SmoothGrad y la superposición del mapa principal, se guarda con el backend no interactivo de matplotlib y se persiste su ruta en la consulta, de modo que las visualizaciones posteriores no repiten el cálculo. La descripción de estas técnicas se apoya en la bibliografía especializada citada anteriormente.

```

def generate_xai_heatmap(model_name: str, original_image_path: str, xai_save_path: str, lang: str =
"es"):
    model = get_model(model_name)
    is_transformer = model_name in MODELS_TRANSFORMERS
    img_size = get_img_size(model_name)
    xai_method_name = "Attention Map" if is_transformer else "Grad-CAM"

    img = load_img_tf(original_image_path, img_size)
    sal = saliency(model, img, is_transformer)
    sm = smoothgrad(model, img, is_transformer)
    cam = get_cam_or_attention(model, img, is_transformer, img_size)

    plt.figure(figsize=(15, 5))
    plt.subplot(1, 4, 1)
    plt.imshow(img)
    plt.title(get_text("xai_original", lang), fontsize=12)
    plt.axis("off")
    # ... subplots de Saliency, SmoothGrad y la superposición del Grad-CAM o la atención ...
    plt.tight_layout()
    plt.savefig(xai_save_path, bbox_inches="tight", dpi=150)
    plt.close()
    return xai_save_path

```

Código 13 - Generación del mapa de explicabilidad (services/xai_generator.py)

30.4. Generación del informe PDF

La generación del informe PDF se implementa en el servicio `pdf_generator.py`, que compone el documento de la consulta mediante la clase `PDFReport`, especialización de la librería `FPDF`. El informe incluye la fecha, el modelo utilizado, el diagnóstico con su color según el resultado, el nivel de confianza y las dos imágenes (la radiografía original y el mapa de explicabilidad). El fragmento siguiente muestra la implementación de la generación del informe.

La implementación del informe refleja las decisiones del diseño del CU-037. El documento se compone con la clase `PDFReport`, que hereda de la librería y redefine la cabecera y el pie de página; el color del diagnóstico se determina según el resultado (rojo para la neumonía y verde para el resultado normal), considerando las etiquetas en los idiomas soportados. Las imágenes se insertan en paralelo y, si la carga de alguna falla, se muestra un texto de error en su lugar, de modo que el documento no queda incompleto. El informe se guarda en el área de informes con un nombre basado en la fecha y se devuelve la ruta, que se persiste en la consulta para su descarga posterior.

El motor de diagnóstico y de explicabilidad de `vitalXAI` reúne las funciones implementadas en este capítulo: la carga y la caché de los modelos atienden la inferencia con las dos familias de arquitecturas, el preprocesado y la predicción producen la etiqueta y la confianza, la generación de mapas ofrece una representación visual de la señal utilizada por el modelo y la generación del informe PDF materializa el documento de la consulta. Estos servicios, invocados por el worker de la cola descrito en el capítulo 29, forman el núcleo técnico del diagnóstico asistido; la codificación del laboratorio de experimentación MLOps se describe en el capítulo siguiente.

```

def generate_medical_report(image_path, xai_path, label, confidence, model_name, lang="es"):
    pdf = PDFReport(lang=lang)
    pdf.add_page()
    pdf.set_font("helvetica", "", 12)

    pdf.cell(0, 10, f" {get_text('pdf_date', lang)} {datetime.now().strftime('%Y-%m-%d %H:%M')}")
    pdf.cell(0, 10, f" {get_text('pdf_model', lang)} {model_name}")
    pdf.ln(10)

    pdf.set_font("helvetica", "B", 14)
    if "neumonía" in label.lower() or "pneumonia" in label.lower() or "肺炎" in label or "न्यूमोनिया" in
label:
        pdf.set_text_color(192, 57, 43)
    else:
        pdf.set_text_color(39, 174, 96)

    pdf.cell(0, 10, get_text("pdf_diagnosis", lang).format(label=label.upper()))
    pdf.set_text_color(0)
    pdf.cell(0, 10, get_text("pdf_confidence", lang).format(confidence=confidence))
    pdf.ln(10)

    try:
        pdf.image(image_path, 20, 100, 70)
        pdf.image(xai_path, 110, 100, 70)
    except Exception:
        pdf.cell(0, 10, get_text("pdf_error_images", lang))

    filename = f"report_{datetime.now().strftime('%Y%m%d%H%M%S')}.pdf"
    filepath = os.path.join("static", "reports", filename)
    pdf.output(filepath, "F")
    return filepath

```

Código 14 - Generación del informe PDF (services/pdf_generator.py)

Capítulo 31. Codificación del laboratorio MLOps

El laboratorio MLOps constituye el segundo núcleo funcional de vitalXAI: permite configurar, lanzar y analizar experimentos de entrenamiento de modelos de detección de neumonía, con validación cruzada, análisis de explicabilidad, comparación estadística y validación externa sobre cohortes independientes. El término MLOps se utiliza aquí para referirse a la coordinación del ciclo de entrenamiento, evaluación y gestión de resultados (Kreuzberger, Kühn, & Hirschl, 2023). Este capítulo describe su implementación, principalmente en `mlops_engine.py`, que organiza las sesiones y orquesta el pipeline, y en `pdf_generator_mlops.py`, que genera el informe consolidado de cada sesión. El contenido se organiza en seis apartados: metodología experimental, orquestación del entrenamiento, validación externa y comparación estadística, gestión de sesiones, lectura de resultados y generación del informe.

La codificación del laboratorio sigue la decisión de diseño del capítulo 23: el pipeline de experimentación se ejecuta mediante los scripts del directorio `pneumoniacnn-main/code`, que el motor invoca como procesos externos con la configuración de cada trabajo transmitida por variables de entorno. Los scripts realizan el entrenamiento con validación cruzada, el análisis XAI cualitativo y cuantitativo, la comparación estadística y la validación externa, y escriben sus resultados en el directorio `training_results` de la sesión. El motor orquesta la ejecución de los scripts y resuelve la lectura de los resultados, manteniendo la persistencia híbrida de la plataforma: la cola y el estado en MySQL, y los artefactos en el sistema de ficheros.

El código completo del laboratorio y de los scripts experimentales puede consultarse en github.com/eLeCe2611/vitalXAI, en `services/mlops_engine.py`, `services/trainer_engine.py`, `services/pdf_generator_mlops.py` y `pneumoniacnn-main/code/`. Los fragmentos incluidos aquí se limitan a las decisiones necesarias para comprender la ejecución del pipeline y la lectura de sus resultados.

31.1. Metodología experimental del pipeline

El pipeline se ha diseñado para comparar diecinueve arquitecturas, dieciséis convolucionales y tres basadas en Transformers, bajo un procedimiento común. La evaluación utiliza cinco pliegues estratificados: en cada iteración se entrena con cuatro pliegues y se valida con el restante, de forma que cada imagen participa una vez en la validación. El resultado de cada arquitectura se resume mediante la media y la desviación típica de las métricas obtenidas en los cinco pliegues.

El conjunto de entrenamiento presenta un mayor número de imágenes de neumonía que de imágenes normales. Para evitar que esta diferencia determine por sí sola el aprendizaje, el pipeline aplica submuestreo aleatorio de la clase mayoritaria dentro de cada partición de entrenamiento. La validación conserva la distribución completa de cada pliegue. Esta decisión facilita la comparación entre modelos, aunque implica descartar parte de las imágenes disponibles y limita las conclusiones frente a otras estrategias de balanceo que no se han comparado en un experimento independiente (He & García, 2009).

Las CNN utilizan pesos preentrenados en ImageNet, mantienen congelada la base convolucional y actualizan un cabezal de clasificación binaria. Los Transformers parten de pesos preentrenados y ajustan el modelo completo con una tasa de aprendizaje menor y AdamW (Loshchilov & Hutter, 2019). En ambas familias se aplican transformaciones ligeras de las imágenes y se guarda el mejor modelo de cada pliegue según la pérdida de validación. También se utilizan parada temprana, reducción de la tasa de aprendizaje cuando el aprendizaje se estanca y la semilla global 42. En las CNN, mantener congelada la base reduce los parámetros que se ajustan con las radiografías, aunque puede limitar la adaptación de las características al dominio médico (Deng & al., 2009; Yosinski & al., 2014).

El rendimiento se resume mediante exactitud, precisión, sensibilidad, F1 y AUC. El pipeline conserva las métricas de cada pliegue, su media y su desviación típica, junto con las predicciones y los pesos del mejor modelo. La curva ROC y su AUC permiten valorar la discriminación sin limitarse a un único umbral (Fawcett, 2006). La evaluación de las explicaciones añade Deletion AUC, Insertion AUC, Sparsity, Entropy y Stability SSIM, además del Brier Score y el Expected Calibration Error para valorar la calibración. En el script cuantitativo, estas dos últimas métricas se calculan sobre una muestra de diez imágenes, cinco por clase, por lo que no deben interpretarse como una evaluación clínica general de la calibración (Linardatos, Papastefanopoulos, & Kotsiantis, 2021; Guo, Pleiss, Sun, & Weinberger, 2017).

La comparación entre modelos mediante Wilcoxon se mantiene como análisis exploratorio sobre las AUC de los pliegues (Wilcoxon, 1945). Con cinco pares dependientes, el p-valor bilateral mínimo es 0,0625 y los pliegues no son observaciones independientes; por tanto, la matriz no permite afirmar significación estadística con el umbral habitual de 0,05 (Nadeau & Bengio, 2003). El ranking ordena las arquitecturas por AUC media y la matriz se conserva como apoyo descriptivo, sin una figura que sugiera conclusiones confirmatorias. La validación externa utiliza modelos congelados sobre la cohorte adulta independiente y emplea DeLong para comparar sus curvas ROC (DeLong, DeLong, & Clarke-Pearson, 1988), teniendo en cuenta que este resultado también debe interpretarse junto con las limitaciones del cambio de población y de las fuentes de las imágenes (Varma & Simon, 2006).

31.2. Orquestación del pipeline de entrenamiento

La orquestación del entrenamiento se implementa en la función `run_training_queue`, que coordina las etapas de una sesión: por cada modelo configurado, lanza el script de entrenamiento adecuado, convolucional o Transformer, y los scripts de análisis XAI; al finalizar ejecuta el script de comparación estadística. La configuración de cada trabajo se transmite a los scripts mediante variables de entorno, y el registro de la ejecución se escribe en el archivo de log del entrenamiento. El fragmento siguiente muestra la implementación del bucle de orquestación.

La implementación de la orquestación refleja la decisión de delegar la computación intensiva en los scripts del pipeline. El motor itera sobre los modelos de la sesión, transmite la configuración mediante las variables de entorno `TFG_*` y ejecuta el script de entrenamiento correspondiente, el script 1 para las CNN o el script 2 para las Transformer, seguido de los scripts de análisis XAI cualitativo y cuantitativo. La salida de los scripts se dirige al archivo de registro, de modo que la consola del laboratorio refleja la progresión del entrenamiento. Al completar todos los modelos, se ejecuta el script de comparación estadística, que genera el ranking y la matriz de Wilcoxon. Este flujo se ejecuta en el worker de la cola descrito en el capítulo 29, fuera del ciclo de petición.

```

def run_training_queue(session_id, models, dataset_path, epochs, batch_size, learning_rate):
    base_path = os.getcwd()
    script_train_cnn = os.path.join(base_path, "pneumoniaccnn-main", "code", "1_train_kfold.py")
    script_train_trans = os.path.join(base_path, "pneumoniaccnn-main", "code",
"2_train_transformer_kfold.py")
    script_img = os.path.join(base_path, "pneumoniaccnn-main", "code", "6_xai_qualitative.py")
    script_math = os.path.join(base_path, "pneumoniaccnn-main", "code", "7_xai_quantitative.py")

    for model_name in models:
        env_vars = os.environ.copy()
        env_vars.update({
            "TFG_SESSION_ID": session_id, "TFG_MODEL_NAME": model_name,
            "TFG_DATASET_DIR": dataset_path, "TFG_EPOCHS": str(epochs),
            "TFG_BATCH_SIZE": str(batch_size), "TFG_LEARNING_RATE": str(learning_rate),
        })
        is_trans = model_name in TRANSFORMER_MODELS
        script_to_run = script_train_trans if is_trans else script_train_cnn
        with open(LOG_FILE, "a", encoding="utf-8", errors="replace") as log:
            subprocess.Popen(["python", script_to_run], stdout=log, stderr=subprocess.STDOUT,
env=env_vars, text=True, encoding="utf-8", errors="replace").wait()
            subprocess.Popen(["python", script_img], stdout=log, stderr=subprocess.STDOUT,
env=env_vars, text=True, encoding="utf-8", errors="replace").wait()
            subprocess.Popen(["python", script_math], stdout=log, stderr=subprocess.STDOUT,
env=env_vars, text=True, encoding="utf-8", errors="replace").wait()

        env_vars_comp = os.environ.copy()
        env_vars_comp["TFG_SESSION_ID"] = session_id
        script_comp = os.path.join(base_path, "pneumoniaccnn-main", "code", "3_evaluate_statistics.py")
        with open(LOG_FILE, "a", encoding="utf-8", errors="replace") as log:
            subprocess.Popen(["python", script_comp], stdout=log, stderr=subprocess.STDOUT,
env=env_vars_comp, text=True, encoding="utf-8", errors="replace").wait()

```

Código 15 - Orquestación del pipeline de entrenamiento (services/ml_ops_engine.py)

31.3. Validación externa y comparación estadística

La validación externa y la comparación estadística se implementan en las funciones `run_external_validation` y `run_statistical_comparison`, que ejecutan los scripts correspondientes del pipeline. La validación externa evalúa los modelos de la sesión sobre el dataset independiente y aplica el test de DeLong; la comparación estadística regenera el ranking y la matriz de Wilcoxon, gestionando el marcador de estado del recálculo. El fragmento siguiente muestra la implementación de la validación externa.

La implementación de la validación externa refleja la separación entre la validación y la configuración original del experimento: la función recibe la sesión y la ruta del dataset externo, transmite la ruta mediante la variable de entorno `TFG_EXTERNAL_DATASET_DIR` y ejecuta los scripts de validación y del test estadístico de DeLong, que escriben las métricas, la curva ROC y la matriz de DeLong en el subdirectorío `external_validation` de la sesión. La comparación estadística, por su parte, ejecuta el script 3 con el identificador de la sesión, limpia el marcador de estado del recálculo antes de ejecutarlo y lo recrea al finalizar, de modo que la interfaz puede consultar si el recálculo está en curso o completado.


```

def run_external_validation(session_id, dataset_path):
    base_path = os.getcwd()
    env_vars = os.environ.copy()
    env_vars["TFG_SESSION_ID"] = session_id
    env_vars["TFG_EXTERNAL_DATASET_DIR"] = dataset_path
    script_val = os.path.join(base_path, "pneumoniaccn-main", "code", "4_external_validation.py")
    script_delong = os.path.join(base_path, "pneumoniaccn-main", "code", "5_evaluate_delong.py")
    with open(LOG_FILE, "a", encoding="utf-8", errors="replace") as log:
        subprocess.Popen(["python", script_val], stdout=log, stderr=subprocess.STDOUT, env=env_vars,
text=True, encoding="utf-8", errors="replace").wait()
        subprocess.Popen(["python", script_delong], stdout=log, stderr=subprocess.STDOUT,
env=env_vars, text=True, encoding="utf-8", errors="replace").wait()

```

Código 16 - Validación externa (services/ml_ops_engine.py)

31.4. Gestión de las sesiones de experimentación

La gestión de las sesiones se implementa en el motor MLOps, que organiza el directorio de cada sesión en `training_results` y resuelve las operaciones de creación, consulta, renombrado y eliminación. La creación de una sesión materializa el identificador, la configuración y el propietario; la consulta enumera las sesiones del usuario con sus modelos; el renombrado y la eliminación operan sobre el directorio con las comprobaciones de validez y de propiedad. El fragmento siguiente muestra la implementación de la creación de la sesión.

La implementación de la creación de la sesión refleja la persistencia híbrida del laboratorio: la sesión se materializa en un directorio de `training_results` con su configuración en `config.json` y la ruta del dataset en `dataset_path.txt`. El identificador se genera a partir de la fecha y la hora, y la configuración incorpora los modelos, los hiperparámetros y el identificador del usuario propietario, que la comprobación de propiedad de las sesiones utiliza para aislar los datos entre usuarios. Las operaciones de renombrado y de eliminación operan sobre el directorio mediante el sistema de ficheros, con la sanitización del nuevo nombre y las comprobaciones de existencia y de propiedad descritas en el diseño.

```

def create_training_session(model_names, dataset_path, epochs, batch_size, learning_rate,
user_id=None):
    session_id = f"RUN_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}"
    session_dir = f"training_results/{session_id}"
    os.makedirs(session_dir, exist_ok=True)
    with open(os.path.join(session_dir, "dataset_path.txt"), "w", encoding="utf-8") as f:
        f.write(dataset_path)
    config = {"dataset_path": dataset_path, "epochs": epochs, "batch_size": batch_size,
        "learning_rate": learning_rate, "models": [m.strip() for m in model_names.split(",")]}
    if user_id is not None:
        config["user_id"] = user_id
    with open(os.path.join(session_dir, "config.json"), "w", encoding="utf-8") as f:
        json.dump(config, f)
    return session_id

```

Código 17 - Creación de la sesión de entrenamiento (services/mlops_engine.py)

31.5. *Lectura de los resultados*

La lectura de los resultados se implementa en las funciones de consulta del motor, que transforman los artefactos persistidos en el sistema de ficheros en estructuras para la interfaz. La consulta de los resultados de un modelo lee las métricas de validación cruzada, la calibración, las métricas XAI y las rutas de los artefactos visuales; la consulta del ranking lee el ranking por AUC y el heatmap de Wilcoxon; y la consulta de la validación externa lee las métricas, la curva ROC y la matriz de DeLong. El fragmento siguiente muestra la implementación de la lectura del ranking.

La implementación de la lectura de los resultados refleja la decisión de no ejecutar el modelo en las consultas: las funciones transforman los ficheros de resultados persistidos por el pipeline (los CSV de métricas y los artefactos visuales) en las estructuras que la interfaz presenta, devolviendo None cuando los resultados no existen para que el router informe con el código correspondiente. Esta decisión reduce el coste de acceso a los resultados y conserva la separación entre el cálculo y la visualización, en coherencia con el diseño del laboratorio.

```

def get_session_ranking_data(session_id):
    session_dir = f"training_results/{session_id}"
    csv_path = f"{session_dir}/session_ranking.csv"
    if not os.path.exists(csv_path):
        return None
    data = []
    with open(csv_path, encoding="utf-8") as f:
        for row in csv.DictReader(f):
            data.append(row)
    heatmap = f"training_results/{session_id}/wilcoxon_heatmap.png"
    return {"ranking": data, "heatmap": heatmap, "config": read_config(session_dir)}

```

Código 18 - Lectura del ranking de la sesión (services/mlops_engine.py)

31.6. Generación del informe de la sesión

La generación del informe de la sesión se implementa en el servicio `pdf_generator_mlops.py`, que compone el documento consolidado mediante la clase `MedicalReport`, especialización de la librería `FPDF2` (FPDF2, 2024). El informe recoge la configuración del experimento, el ranking con el heatmap de Wilcoxon, los resultados de la validación externa con la curva ROC y la matriz de DeLong, y el detalle técnico de cada modelo con sus métricas XAI y sus mapas de calor. El fragmento siguiente muestra el inicio de la implementación del generador.

La implementación del informe refleja la decisión de separar el conocimiento del formato del router: el generador recibe la sesión, lee los artefactos persistidos en el sistema de ficheros, como la configuración, el ranking, los heatmaps, las métricas de la validación externa y las métricas XAI de cada modelo, y compone el documento con la clase `MedicalReport`, que redefine la cabecera corporativa, el pie de página y los títulos de sección. El informe se guarda en el directorio de la sesión y se devuelve como documento descargable; si la sesión no existe, el generador responde HTTP 404.

El laboratorio MLOps de vitalXAI queda codificado en sus funciones principales: el motor orquesta el pipeline de entrenamiento y de validación externa mediante los scripts del proyecto, gestiona las sesiones en el sistema de ficheros, lee los resultados persistidos para la interfaz y genera el informe consolidado de cada sesión. La ejecución experimental documentada sigue siendo parcial, por lo que esta implementación no equivale a un benchmark completo de todas las arquitecturas previstas. La codificación del frontend, que presenta estas capacidades al usuario, se describe en el capítulo siguiente.

```

async def generate_pdf_report(session_id):
    session_dir = f"training_results/{session_id}"
    if not os.path.exists(session_dir):
        return JsonResponse(status_code=404, content={"message": "Sesión no encontrada"})

    pdf = MedicalReport()
    pdf.set_auto_page_break(auto=True, margin=20)
    pdf.add_page()
    pdf.section_title("1. CONFIGURACIÓN DEL SISTEMA Y PARÁMETROS")

    config_path = os.path.join(session_dir, "config.json")
    if os.path.exists(config_path):
        with open(config_path, encoding="utf-8") as f:
            cfg = json.load(f)
            # ... secciones de configuración, ranking, Wilcoxon, validación externa y detalle por modelo ...

    pdf_output_path = os.path.join(session_dir, f"Informe_Completo_{session_id}.pdf")
    pdf.output(pdf_output_path)
    return FileResponse(pdf_output_path, filename=f"Reporte_MLOps_{session_id}.pdf")

```

Código 19 - Generación del informe de la sesión (services/pdf_generator_mlops.py)

Capítulo 32. Codificación del frontend

Este capítulo explica cómo se implementa la parte de interfaz de vitalXAI. Las plantillas Jinja2 sirven las vistas definidas en el capítulo 22 y los scripts JavaScript permiten al profesional sanitario y al investigador interactuar con el backend. La aplicación no utiliza un framework de componentes independiente. El contenido se organiza en cuatro apartados: plantillas de presentación, scripts por ámbito, comunicación asíncrona con la API e internacionalización y tema visual.

La presentación se resuelve con las plantillas Jinja2 del directorio `templates/`, que componen las páginas en el servidor, y con los recursos estáticos del directorio `static/`, que incluyen los scripts JavaScript y los estilos. La interactividad se implementa en JavaScript nativo, con módulos por ámbito funcional, y la comunicación con la API se realiza mediante la interfaz de fetch del navegador, con la protección CSRF y la renovación automática de la sesión en las vistas privadas (MDN Web Docs, 2024). El estilo visual se apoya en las utilidades de Tailwind CSS, cargadas mediante CDN y configuradas con el modo oscuro mediante clase (Tailwind CSS, 2024).

La implementación completa de las plantillas, los scripts y los recursos estáticos está disponible en github.com/eLeCe2611/vitalXAI, en los directorios `templates/` y `static/`. Este capítulo utiliza fragmentos representativos para explicar la estructura de la interfaz y su comunicación con el backend; el repositorio contiene el código completo y su evolución.

32.1. Las plantillas de presentación

Las plantillas Jinja2 componen las páginas del sistema en el servidor, inyectando los datos del usuario y los textos de la interfaz. Las páginas principales son el inicio de sesión (`login.html`), el registro (`register.html`), el panel de diagnóstico (`dashboard.html`) y el laboratorio de entrenamiento (`training.html`). Las plantillas combinan la estructura HTML con los atributos de internacionalización `data-i18n`, que el script de idioma traduce dinámicamente, y con las variables de contexto del servidor, como el nombre y el rol del usuario. El fragmento siguiente muestra una sección de la plantilla del panel de diagnóstico.

La separación por ámbito refleja la organización del código del frontend: cada módulo conoce los elementos de su ventana y las operaciones de su subsistema, de modo que los cambios de un ámbito no afectan al resto. El módulo `i18n.js` es el único compartido, y se carga en todas las páginas para aplicar el idioma y el tema visual. La coordinación entre módulos se resuelve mediante la ventana global, que expone las funciones de interacción que las plantillas invocan desde los atributos de los elementos, como el envío de la imagen o la apertura del detalle de una consulta.


```

<div class="p-4 border-b ... bg-blue-50 ...">
  <div class="flex items-center gap-3">
    <div class="w-10 h-10 rounded-full bg-blue-600 ...">
      <i class="fa-solid fa-user-md text-lg"></i>
    </div>
    <div class="overflow-hidden">
      <p class="text-[10px] font-bold ...">{{ role }}</p>
      <p class="text-sm font-black ...">{{ full_name }}</p>
    </div>
  </div>
</div>

<div class="p-3 space-y-1 border-b ...">
  <a href="/dashboard" class="flex items-center gap-3 px-3 py-2 rounded-lg bg-blue-100 ...">
    <i class="fa-solid fa-stethoscope w-5 text-center"></i>
    <span id="ui-nav-diag" data-i18n="navDiag">Diagnóstico Rápido</span>
  </a>
  <a href="/training" class="flex items-center gap-3 px-3 py-2 rounded-lg ...">
    <i class="fa-solid fa-flask w-5 text-center"></i>
    <span id="ui-nav-lab" data-i18n="navLab">Laboratorio de Entrenamiento</span>
  </a>
</div>

```

Código 20 - Plantilla del panel de diagnóstico (templates/dashboard.html)

32.2. Los recursos JavaScript por ámbito

La interactividad del frontend se implementa en JavaScript nativo, organizado en módulos por ámbito funcional: `dashboard.js` para el panel de diagnóstico, `training.js` para el laboratorio, `admin.js` para la administración y la cola, e `i18n.js` para la internacionalización compartida. Cada módulo se encarga de la interacción de su ventana: la carga y el envío de la radiografía, el sondeo del estado de la cola, el render de los resultados, la gestión de los experimentos del laboratorio y los diálogos de la administración. Los módulos se cargan en las plantillas al final del documento, y se comunican con la API mediante las peticiones asíncronas descritas en el apartado siguiente.

La separación por ámbito refleja la organización del código del frontend: cada módulo conoce los elementos de su ventana y las operaciones de su subsistema, de modo que los cambios de un ámbito no afectan al resto. El módulo `i18n.js` es el único compartido, y se carga en todas las páginas para aplicar el idioma y el tema visual. La coordinación entre módulos se resuelve mediante la ventana global, que expone las funciones de interacción que las plantillas invocan desde los atributos de los elementos, como el envío de la imagen o la apertura del detalle de una consulta.

32.3. Comunicación asíncrona con la API

La comunicación con la API se realiza mediante la interfaz de `fetch` del navegador, con el envío de los formularios de forma asíncrona mediante `FormData`, de modo que las operaciones se procesan sin recargar la página. La protección CSRF se gestiona mediante un interceptor global de peticiones, que añade la cabecera `X-CSRF-Token` a las peticiones que modifican el estado y gestiona la renovación automática de la sesión ante una respuesta de no autenticado. El fragmento siguiente muestra la implementación del interceptor global.

La implementación de la comunicación asíncrona refleja las decisiones de seguridad y de experiencia del sistema. El interceptor reemplaza la función `fetch` global en las vistas que cargan `admin.js`: añade la cabecera `X-CSRF-Token` con el token de la cookie a las peticiones que modifican el estado, en correspondencia con la protección CSRF del backend descrita en el capítulo 28, y ante una respuesta de no autenticado intenta renovar la sesión mediante el endpoint de refresco. Si la renovación tiene éxito reintenta la petición original; en caso contrario redirige a la página de inicio. El mecanismo centraliza esta gestión, aunque no constituye una garantía de disponibilidad continua de la sesión.

```

(function() {
  const c = document.cookie.match(/csrf_token=([^;]+)/);
  if (!c) return;
  const csrfToken = c[1];
  const origFetch = window.fetch;
  let _isRefreshing = false;
  window.fetch = async function(url, options) {
    if (options && (options.method || 'GET').toUpperCase() !== 'GET') {
      options = Object.assign({}, options);
      options.headers = Object.assign({}, options.headers || {});
      options.headers['X-CSRF-Token'] = csrfToken;
    }
    let response = await origFetch(url, options);
    if (response.status === 401 && !_isRefreshing) {
      _isRefreshing = true;
      try {
        const refreshResp = await origFetch('/api/token/refresh', { method: 'POST', headers: {
          'X-CSRF-Token': csrfToken } });
        if (refreshResp.ok) {
          response = await origFetch(url, options);
        } else {
          window.location.href = '/';
          return response;
        }
      } catch (e) {
        window.location.href = '/';
      } finally {
        _isRefreshing = false;
      }
    }
    return response;
  };
})();

```

Código 21 - Interceptor global de peticiones con CSRF y renovación de sesión (static/js/admin.js)

32.4. Internacionalización y tema visual

La internacionalización y el tema visual se implementan en el módulo `i18n.js`, compartido por todas las páginas. La internacionalización mantiene un diccionario de traducciones en el navegador para los cuatro idiomas soportados, aplica los textos a los elementos `data-i18n` y persiste la preferencia del idioma en el almacenamiento local. El tema visual alterna entre los modos claro y oscuro mediante la clase del elemento raíz y, cuando no existe una preferencia guardada, consulta la preferencia del sistema mediante `matchMedia`; estas APIs de almacenamiento y consulta del entorno pertenecen a la plataforma web del navegador (MDN Web Docs, 2024). El fragmento siguiente muestra la implementación de la traducción y del cambio de idioma.

La implementación de la internacionalización refleja las decisiones del diseño del CU-004: el selector de idioma actualiza la preferencia en el almacenamiento local y traduce de forma inmediata los textos de la interfaz marcados con `data-i18n`, sin recargar la página, con el español como valor por defecto. El tema visual alterna la clase del modo oscuro en el elemento raíz, de modo que el cambio se propaga a toda la interfaz, y persiste la preferencia en el almacenamiento local; al cargar la página, se restaura la preferencia guardada y, en ausencia de esta, se respeta la preferencia de color del sistema. Los módulos de las ventanas complementan la traducción re-renderizando las vistas dinámicas, como el historial o los resultados del laboratorio, en el idioma seleccionado.

El frontend de vitalXAI queda descrito en sus elementos principales: las plantillas Jinja2 componen las ventanas en el servidor, los recursos JavaScript por ámbito gestionan la interacción de cada vista, la comunicación asíncrona se protege con el interceptor CSRF y la renovación de sesión, y la internacionalización con el tema visual se aplican de forma transversal. Junto con la codificación del backend, de la ejecución asíncrona y de los motores, este capítulo completa la descripción de la implementación del sistema vitalXAI.

```

function t(key) {
  const lang = currentLang || localStorage.getItem('appLang') || 'es';
  const langDict = dict[lang] || dict.es;
  return langDict[key] || dict.es[key] || key;
}

function changeLanguage() {
  const selector = document.getElementById('lang-selector');
  if (selector) currentLang = selector.value;
  localStorage.setItem('appLang', currentLang);
  document.querySelectorAll('[data-i18n]').forEach(el => {
    const key = el.getAttribute('data-i18n');
    el.innerText = t(key);
  });
}

function toggleTheme() {
  const html = document.documentElement;
  if (html.classList.contains('dark')) {
    html.classList.remove('dark');
    localStorage.setItem('theme', 'light');
  } else {
    html.classList.add('dark');
    localStorage.setItem('theme', 'dark');
  }
}

```

Código 22 - Internacionalización y tema visual (static/js/i18n.js)

[OBJ]
[OBJ]

[OBJ] **Parte V.**

Pruebas

[OBJ]

Capítulo 33. Introducción a la verificación del sistema

La parte de verificación de la memoria presenta los resultados de las pruebas realizadas sobre el sistema vitalXAI, describiendo cómo se comprobó que la implementación satisface los requisitos funcionales y no funcionales del análisis. El plan de pruebas del capítulo 16 definió las verificaciones previstas del sistema, con sus códigos y sus criterios de éxito, y la estrategia de verificación del capítulo 25 especificó los entornos y los niveles de prueba; esta parte recoge la ejecución de esa verificación y sus resultados, de modo que la memoria documenta tanto lo que se planificó como lo que se comprobó realmente. La verificación se apoya en la batería de pruebas automatizadas del proyecto y en las comprobaciones estáticas de calidad, cuya metodología se describió en la introducción a la codificación del capítulo 27 (Myers, Sandler, & Badgett, 2011).

La parte se organiza en cuatro capítulos. Este capítulo introductorio presenta el alcance de la verificación, los entornos y las herramientas empleadas y la organización de la parte; el capítulo 34 describe las pruebas automatizadas, unitarias y de integración, con sus resultados; el capítulo 35 describe las pruebas de seguridad; y el capítulo 36 describe las pruebas de sistema y la validación funcional de los flujos completos de uso. Las pruebas de rendimiento PR-001 a PR-005 quedan identificadas como trabajo pendiente y no se presentan como resultados obtenidos.

La verificación de vitalXAI se ejecutó sobre la implementación descrita en la parte de codificación, siguiendo la estrategia definida en el capítulo 25: la batería automatizada se ejecutó en los entornos unitario y de integración, con los servicios externos simulados, y las pruebas de rendimiento y de sistema se definieron sobre el despliegue de la plataforma. Los resultados de la ejecución y de las comprobaciones estáticas, que se detallan en los capítulos siguientes, documentan el alcance realmente comprobado. No permiten afirmar que todos los criterios del plan estén satisfechos: las pruebas de integración ejecutadas se concentran en autenticación y los flujos PI-004 a PI-007 permanecen pendientes.

33.1. Alcance de la verificación

El alcance de la verificación de vitalXAI abarca las cinco categorías del plan de pruebas del capítulo 16: la verificación de los componentes individuales, la verificación de los flujos entre subsistemas, las pruebas de protección y control de acceso, las pruebas de rendimiento y capacidad de respuesta, y la verificación integral de los flujos completos de uso. Las dos primeras categorías se materializan en la batería automatizada de pruebas unitarias y de integración; la tercera se materializa en las pruebas de seguridad automatizadas; y las dos últimas se definen sobre el entorno de despliegue, conforme a la estrategia del capítulo 25. La tabla siguiente resume el alcance de la verificación y su correspondencia con las categorías del plan.

Categoría del plan	Nivel	Materialización
Verificación de componentes individuales	Unitario	Pruebas unitarias del directorio tests/unit/, por subsistema.
Verificación de los flujos entre subsistemas	Integración	Pruebas de integración del directorio tests/integration/.
Protección y control de acceso	Seguridad	Pruebas de seguridad automatizadas (CSRF, cabeceras, limitación, tokens).
Rendimiento y capacidad de respuesta	Pendiente	Pruebas PR-001 a PR-005 previstas sobre el despliegue, todavía no ejecutadas.
Verificación integral de los flujos completos	Sistema	Pruebas de sistema de los flujos de uso (PE-001 a PE-006).

Tabla x - Alcance de la verificación

El alcance se define de modo que la verificación progrese desde el aislamiento de los componentes hasta la comprobación integral del sistema: cada nivel cubre un grado de fidelidad mayor con respecto al entorno real, y la superación de cada nivel es condición para considerar verificada la parte del sistema que abarca, en coherencia con los criterios de aceptación definidos en el capítulo 25.

33.2. Entornos y herramientas de verificación

La verificación se ejecutó en los tres entornos definidos en la estrategia de verificación del capítulo 25: el entorno unitario, sin dependencia de una red o base de datos real y con dobles de prueba; el entorno de integración, con una base de datos SQLite en memoria; y el entorno de sistema, sobre el despliegue real de la plataforma. Las herramientas empleadas son las especificadas en el capítulo 25 y en la guía de pruebas del proyecto: pytest con pytest-cov para medir la cobertura, dobles contruidos principalmente con unittest.mock y fixtures, y las comprobaciones estáticas de ruff y mypy (pytest, 2024; Ruff, 2024; Mypy, 2024).

La ejecución de la batería automatizada y de las comprobaciones estáticas sobre la implementación actual del proyecto produjo los resultados globales que se resumen en la tabla siguiente. La batería completa de pruebas unitarias y de integración se ejecutó en el entorno de verificación automatizada, la cobertura de los módulos de la aplicación se midió sobre el código implementado y las comprobaciones estáticas se aplicaron sobre el repositorio.

Verificación	Resultado
Batería de pruebas automatizadas	190 pruebas superadas, sin fallos.
Cobertura de código	74,04 % sobre los módulos de la aplicación, por encima del umbral del 70 %.
Análisis estático (ruff)	Todas las comprobaciones superadas sin errores.
Verificación de tipos (mypy)	Sin problemas en los módulos configurados.

Tabla x - Entornos y herramientas de verificación

Los resultados detallados de cada nivel (la distribución de las pruebas por subsistema, la cobertura por módulo y los resultados de las comprobaciones de seguridad) se presentan en los capítulos 34 y 35, de modo que este resumen global se desglosa en la verificación específica de cada componente y de cada mecanismo.

33.3. Organización de la parte

La parte de verificación se organiza por niveles de prueba, de modo que cada capítulo presenta el alcance y los resultados disponibles. El capítulo 34 describe las pruebas automatizadas: las pruebas unitarias de los componentes y las pruebas de integración ejecutadas, con los ficheros de pruebas, la distribución de las verificaciones y la cobertura alcanzada. El capítulo 35 describe las pruebas de seguridad automatizadas: la protección CSRF, las cabeceras de seguridad, la limitación de peticiones y la gestión de los tokens de sesión. El capítulo 36 describe las pruebas de sistema y la validación funcional de los flujos completos de uso, presentando cada flujo con su criterio de éxito y el resultado de su verificación. Las pruebas de rendimiento previstas en el plan no disponen de resultados y quedan identificadas como trabajo pendiente. Esta organización permite distinguir lo ejecutado de lo que aún debe verificarse.

33.4. Trazabilidad de los objetivos específicos

La verificación de los requisitos no sustituye la revisión de los objetivos del proyecto. Para hacer explícita la relación entre ambos niveles, la tabla siguiente indica dónde se documenta cada objetivo específico y qué evidencia aporta esta parte de la memoria. En los objetivos científicos, la evidencia puede consistir en la documentación de la metodología y de la capacidad implementada; no implica por sí sola que los resultados de los modelos sean clínicamente generalizables.

Objetivo	Evidencia principal	Comprobación en la memoria
OE1. Revisar el estado del arte	Capítulo 1	Revisión documentada y utilizada como fundamento del proyecto.
OE2. Diseñar e implementar la arquitectura de persistencia	Capítulos 12, 14, 19, 28 y 34	Requisitos, diseño, implementación y pruebas de persistencia e historial.
OE3. Diseñar e implementar el control de acceso	Capítulos 12, 20, 28, 35 y 37	Pruebas de autenticación, sesión, seguridad y aislamiento de los flujos.
OE4. Implementar el pipeline de entrenamiento	Capítulos 12, 23, 31 y 37	Metodología y orquestación documentadas; flujo del laboratorio comprobado.
OE5. Desarrollar el módulo de explicabilidad	Capítulos 12, 30, 31 y 37	Implementación de mapas y métricas descrita; consulta de artefactos incluida en los flujos clínico y del laboratorio.

Objetivo	Evidencia principal	Comprobación en la memoria
OE6. Ejecutar la validación externa y el análisis estadístico	Capítulos 12, 31 y 37	Procedimiento de validación y comparación documentado; la disponibilidad de resultados depende de la ejecución de una sesión.
OE7. Garantizar la reproducibilidad y la trazabilidad	Capítulos 12, 19, 23 y 31	Configuración, semilla, resultados y artefactos contemplados en el diseño y la implementación.
OE8. Desarrollar la interfaz clínica de diagnóstico asistido	Capítulos 12, 22, 32 y 36	Flujo clínico completo comprobado mediante la prueba PE-001.
OE9. Desarrollar el laboratorio MLOps y el asistente conversacional	Capítulos 12, 22, 31 y 36	Configuración, lanzamiento y consulta de una sesión comprobados mediante la prueba PE-003.
OE10. Diseñar e implementar el procesamiento asíncrono de tareas	Capítulos 12, 29, 36 y 36	Ejecución sin bloqueo y gestión de la cola contempladas en PR-003 y PE-004.
OE11. Incorporar internacionalización	Capítulos 12, 22, 32 y 36	Cambio de idioma comprobado mediante la prueba PE-006.

Tabla x - Trazabilidad de los objetivos específicos

Esta tabla permite localizar la evidencia de cada objetivo sin convertir la verificación de la plataforma en una afirmación sobre la validez clínica de los modelos. Los capítulos 34 a 36 presentan los resultados de las pruebas que respaldan las comprobaciones indicadas; la interpretación de las métricas de entrenamiento y validación debe hacerse junto con sus condiciones experimentales y limitaciones.

Capítulo 34. Pruebas automatizadas

Las pruebas automatizadas constituyen el primer nivel de la verificación de vitalXAI: comprueban el comportamiento de los componentes y de los flujos entre subsistemas de forma repetible, sin depender del entorno real de despliegue. Este capítulo describe las pruebas automatizadas del proyecto y presenta los resultados reales de su ejecución: las pruebas unitarias de los componentes, las pruebas de integración de los flujos entre subsistemas, la ejecución de la batería con las comprobaciones estáticas y la cobertura de los módulos de la aplicación. La distinción entre pruebas aisladas y pruebas de colaboración sigue la terminología habitual de la verificación de software (Myers, Sandler, & Badgett, 2011), y estas pruebas se corresponden con las categorías del plan de pruebas del capítulo 16.

La batería automatizada se ejecuta con `pytest` y `pytest-cov`, y se organiza en los directorios `tests/unit/` y `tests/integration/` del repositorio (pytest, 2024). Las pruebas unitarias sustituyen la base de datos, los modelos de TensorFlow y las dependencias externas por dobles de prueba, de modo que no dependen de la red ni de los pesos reales; las pruebas de integración utilizan una base de datos SQLite en memoria y comprueban principalmente los flujos de autenticación e historial.

34.1. Las pruebas unitarias

Las pruebas unitarias verifican el comportamiento de los componentes del sistema de manera aislada, organizadas por subsistema conforme a la estructura del capítulo 16. El conjunto comprende 186 pruebas distribuidas en 19 ficheros del directorio `tests/unit/`, que cubren los servicios y los enrutadores del backend: la gestión de cuentas y sesiones, la validación de las entradas, el motor de inferencia, la generación de las explicaciones, la gestión del historial, el laboratorio de entrenamiento, la cola de trabajos, la internacionalización y la capa de acceso a datos. La tabla siguiente desglosa las pruebas unitarias por fichero, agrupadas por subsistema.

La distribución de las pruebas refleja la complejidad funcional de cada subsistema: el laboratorio MLOps concentra el mayor número de verificaciones, por ser el bloque de mayor complejidad, seguido del acceso y la gestión de cuentas y del motor de diagnóstico. Las pruebas de cada fichero materializan las verificaciones unitarias del plan de pruebas del capítulo 16 (los códigos PU-001 a PU-037) y se ejecutan de forma aislada, con la base de datos, los modelos y los servicios externos sustituidos por dobles.

Subsistema	Fichero de pruebas	Nº de pruebas
Laboratorio MLOps	test_trainer_router.py	30
Acceso y gestión de cuentas	test_auth_router.py	15
Laboratorio MLOps	test_trainer_engine.py	14
Motor de diagnóstico	test_xai_generator.py	13
Capacidades transversales	test_lang.py	12
Laboratorio MLOps	test_mlops_engine.py	11
Motor de diagnóstico	test_ml_engine.py	11
Capacidades transversales	test_queue_worker.py	10
Supervisión y administración	test_admin_router.py	10
Acceso y gestión de cuentas	test_auth_service.py	10
Historial	test_history_router.py	9
Acceso y gestión de cuentas	test_input_validation.py	8
Seguridad	test_security_headers.py	7
Persistencia	test_database.py	6
Capacidades transversales	test_queue_router.py	6
Seguridad	test_csrf_middleware.py	5
Motor de diagnóstico	test_pdf_generator.py	4
Motor de diagnóstico	test_inference_router.py	3
Seguridad	test_rate_limiting.py	2

Tabla x - Recuento de pruebas unitarias

34.2. Las pruebas de integración

Las pruebas de integración verifican la colaboración entre las capas del sistema y entre los subsistemas, superando el aislamiento de las pruebas unitarias. El conjunto comprende cuatro pruebas distribuidas en el fichero `test_auth_flow.py` del directorio `tests/integration/`. Comprueban el registro de un usuario, el inicio de sesión con las credenciales registradas, el acceso al panel, la redirección de un usuario no autenticado y el historial vacío de un usuario nuevo. Utilizan una base de datos SQLite en memoria y se centran en la colaboración entre la API, la capa de negocio y la persistencia.

Las pruebas de integración materializan las verificaciones PI-001 a PI-003 del plan de pruebas del capítulo 16, cubriendo el comportamiento especificado en los casos de uso CU-001, CU-002 y CU-005. Las ampliaciones previstas del plan, que incluyen los flujos del diagnóstico, el aislamiento de datos, la supervisión administrativa y el laboratorio, requieren pruebas adicionales conforme a la estrategia de verificación del capítulo 25.

34.3. Ejecución y resultados de la batería

La ejecución de la batería completa de pruebas automatizadas produjo 190 pruebas superadas sin fallos, con una duración aproximada de un minuto y quince segundos. La batería comprende las 186 pruebas unitarias y las cuatro pruebas de integración descritas en los apartados anteriores. El análisis estático con `ruff` también se superó sin errores y `mypy` no encontró problemas en los módulos configurados; esta última comprobación se ejecuta por separado de CI (Ruff, 2024; Mypy, 2024). Estos resultados respaldan los componentes unitarios y el flujo de autenticación cubierto por las pruebas de integración, pero no permiten afirmar que se hayan satisfecho todos los criterios del nivel de integración: PI-004 a PI-007, incluido el aislamiento entre usuarios, permanecen pendientes.

La ejecución de la batería se integra en el flujo de integración continua del repositorio. El workflow configurado ejecuta `Ruff` y `pytest` con medición de cobertura en los eventos definidos para las ramas y las peticiones de cambios, conforme al flujo descrito en el capítulo 27. El resultado global de la batería se resume en la tabla siguiente.

Verificación	Resultado
Pruebas unitarias	186 pruebas superadas.
Pruebas de integración	4 pruebas superadas.
Total de la batería	190 pruebas superadas, sin fallos.
Análisis estático (<code>ruff</code>)	Todas las comprobaciones superadas.
Verificación de tipos (<code>mypy</code>)	Sin problemas en los módulos configurados.

Tabla x - Ejecución y resultados

34.4. Cobertura de los módulos

La cobertura de los módulos de la aplicación se midió con pytest-cov sobre los paquetes de servicios, los routers y el módulo de persistencia, obteniendo una cobertura total del 74,04 %, por encima del umbral mínimo del setenta por ciento definido en la guía de pruebas del proyecto. La tabla siguiente desglosa la cobertura alcanzada por cada módulo de la aplicación.

Módulo	Cobertura
services/csrf_middleware.py	100 %
services/lang.py	100 %
services/rate_limiter.py	100 %
routers/inference.py	96 %
services/ml_engine.py	96 %
routers/admin.py	95 %
services/auth_service.py	94 %
services/chatbot_service.py	92 %
routers/auth.py	86 %
routers/history.py	85 %
routers/trainer.py	84 %
services/xai_generator.py	80 %
services/pdf_generator.py	79 %
routers/queue.py	74 %
services/trainer_engine.py	71 %
services/pdf_generator_mlops.py	57 %
services/queue_worker.py	48 %
services/mlops_engine.py	41 %
Total	74,04 %

Tabla x - Cobertura de los módulos

La cobertura refleja la estrategia de verificación de la plataforma: los módulos de seguridad transversal alcanzan el cien por cien de cobertura, y los módulos del diagnóstico y del acceso presentan valores elevados. Los módulos del laboratorio y de la ejecución asíncrona presentan los valores más bajos, porque su lógica se resuelve en gran medida en los scripts del pipeline y en los procesos del worker, que las pruebas unitarias verifican de forma parcial mediante los dobles de prueba. La batería automatizada permite considerar respaldadas las comprobaciones ejecutadas, pero no cubre de extremo a extremo los flujos PI-004 a PI-007. Las pruebas de seguridad, que comparten esta batería, se presentan con detalle en el capítulo siguiente.

Capítulo 35. Pruebas de seguridad

Este capítulo presenta la evidencia automatizada obtenida para los mecanismos de seguridad implementados en vitalXAI. El alcance cubre la protección CSRF, las cabeceras de seguridad, la limitación de peticiones y el ciclo de vida de los tokens de sesión, controles relacionados con riesgos habituales de las aplicaciones web autenticadas (OWASP, 2024). Las pruebas se corresponden con la categoría de protección y control de acceso del plan de pruebas del capítulo 16, concretamente con los códigos PS-001 a PS-009, y forman parte de la batería descrita en el capítulo 34.

Las pruebas de seguridad se implementan en los ficheros `test_csrf_middleware.py`, `test_security_headers.py`, `test_rate_limiting.py` y `test_auth_service.py` del directorio `tests/unit/`, y se ejecutan de forma aislada, sin dependencia del entorno real, verificando los mecanismos de seguridad implementados en el backend descrito en el capítulo 28. La ejecución específica de estos ficheros produjo un resultado de 24 pruebas superadas sin fallos, como se detalla en cada apartado.

35.1. Protección CSRF

La protección CSRF se verifica con las cinco pruebas del fichero `test_csrf_middleware.py`, que comprueban el comportamiento del middleware de protección frente a las peticiones que modifican el estado. Las pruebas cubren los casos PS-001 a PS-004 del plan: el rechazo de una petición POST sin token CSRF, el procesamiento de una petición POST con un token válido, el rechazo de una petición POST con un token erróneo y la exención de las peticiones GET de la protección. La tabla siguiente desglosa estas verificaciones.

ID	Verificación	Resultado
PS-001	Una petición POST sin token CSRF es rechazada.	Superada.
PS-002	Una petición POST con token CSRF válido se procesa.	Superada.
PS-003	Una petición POST con token CSRF erróneo es rechazada.	Superada.
PS-004	Las peticiones GET están exentas de la protección CSRF.	Superada.

Tabla x - Protección CSRF

Las verificaciones confirman que el middleware implementado en `csrf_middleware.py` establece la cookie del token en las peticiones seguras y exige su correspondencia con la cabecera en las peticiones que modifican el estado, de modo que una petición de modificación sin el token o con un token erróneo responde HTTP 403 sin procesarse, tal y como se describió en la codificación del backend.

35.2. Cabeceras de seguridad

Las cabeceras de seguridad se verifican con las siete pruebas del fichero `test_security_headers.py`, que comprueban que todas las respuestas de la aplicación incluyen las cabeceras configuradas. Las pruebas cubren los casos PS-005 a PS-007 del plan y las cabeceras complementarias: la política de seguridad de contenido (Content-Security-Policy), la seguridad de transporte (Strict-Transport-Security), la opción de tipo de contenido (X-Content-Type-Options), el marco (X-Frame-Options), la protección de XSS y la política de referente. La tabla siguiente resume las verificaciones principales.

ID	Verificación	Resultado
PS-005	Las respuestas incluyen la cabecera Content-Security-Policy.	Superada.
PS-006	Las respuestas incluyen la cabecera Strict-Transport-Security.	Superada.
PS-007	Las respuestas incluyen la cabecera X-Content-Type-Options.	Superada.

Tabla x - Cabeceras de seguridad

Las verificaciones confirman que el middleware de cabeceras de seguridad añade las cabeceras configuradas a todas las respuestas de la aplicación, en coherencia con las directivas definidas en la codificación del backend, de modo que el navegador aplica las restricciones de contenido, transporte y tipo MIME en cada respuesta.

35.3. Limitación de peticiones

La limitación de peticiones se verifica con las dos pruebas del fichero `test_rate_limiting.py`, que comprueban el comportamiento del limitador configurado en el backend. Las pruebas cubren la limitación del endpoint de inicio de sesión, que permite cinco peticiones por minuto, verificando que la sexta petición en el mismo minuto es rechazada, y la aplicación de la limitación global sobre las peticiones de la aplicación. La tabla siguiente resume estas verificaciones.

ID	Verificación	Resultado
PS-008	El inicio de sesión está limitado en frecuencia.	Superada.

Tabla x - Limitación de peticiones

La verificación confirma que el limitador implementado en `rate_limiter.py` rechaza los intentos reiterados de acceso desde una misma dirección, en coherencia con el requisito de protección frente a ataques de fuerza bruta y con la codificación del backend descrita en el capítulo 28.

35.4. Gestión de los tokens de sesión

La gestión de los tokens de sesión se verifica con las diez pruebas del fichero `test_auth_service.py`, que comprueban el comportamiento del servicio de autenticación respecto a las credenciales y a los tokens. Las pruebas cubren la creación y la verificación de los tokens de acceso, la creación y la verificación de los tokens de refresco, la invalidación de los tokens revocados, la rotación del token de refresco y la detección del robo de sesión. Entre ellas se encuentra la verificación PS-009 del plan, que comprueba que el acceso con un token de refresco revocado es rechazado. La tabla siguiente resume las verificaciones principales.

ID	Verificación	Resultado
PS-009	El acceso con un token de refresco revocado es rechazado.	Superada.
N/A	La rotación del token de refresco invalida el anterior.	Superada.
N/A	La detección de robo de sesión revoca todos los tokens del usuario.	Superada.

Tabla x - Gestión de los tokens de sesión

Las verificaciones confirman que el servicio de autenticación implementado en `auth_service.py` gestiona correctamente el ciclo de vida de los tokens: la verificación distingue el token activo, el revocado dentro del periodo de gracia y el uso de una credencial revocada, y ante este último revoca todos los tokens del usuario, tal y como se describió en la codificación del backend y en el diseño del subsistema de acceso.

35.5. Resultados de la verificación de seguridad

La ejecución específica de las pruebas de seguridad produjo un resultado de 24 pruebas superadas sin fallos, distribuidas en los cuatro ficheros de verificación. La tabla siguiente resume el resultado de la verificación de seguridad.

Fichero de pruebas	Verificaciones	Resultado
test_csrf_middleware.py	5	Superadas.
test_security_headers.py	7	Superadas.
test_rate_limiting.py	2	Superadas.
test_auth_service.py	10	Superadas.
Total	24	Superadas.

Tabla x - Resultados de la verificación de seguridad

Las verificaciones de seguridad confirman el funcionamiento de los mecanismos evaluados: la protección frente a la falsificación de peticiones en sitios cruzados, las cabeceras de seguridad, la limitación de los intentos de acceso y la gestión de las credenciales de sesión. Estos resultados no equivalen a una certificación de seguridad completa ni cubren los controles pendientes de auditoría, protección de datos y despliegue; tampoco sustituyen una evaluación de amenazas más amplia basada en el contexto de operación (OWASP, 2024). La verificación en el entorno real, junto con las pruebas de rendimiento y de los flujos completos, se describe en los capítulos siguientes de la parte de verificación.

Capítulo 36. Pruebas de sistema y validación funcional

Este capítulo reúne la evidencia funcional obtenida al recorrer vitalXAI desde su interfaz. Se describen las seis pruebas de sistema del plan de pruebas del capítulo 16, identificadas como PE-001 a PE-006, junto con sus pasos, criterios de éxito y alcance. Las comprobaciones se realizaron manualmente sobre el despliegue definido en el capítulo 25 y cada flujo se acompaña de una captura de la aplicación en funcionamiento. Este enfoque sigue la comprobación de flujos completos entre componentes propia de las pruebas de sistema (Myers, Sandler, & Badgett, 2011).

Las pruebas de sistema verifican los seis flujos completos de la plataforma: el flujo clínico del diagnóstico, la gestión del historial, el flujo del laboratorio de entrenamiento, la ejecución asíncrona de la cola, la supervisión administrativa y las capacidades transversales de idioma y tema. Cada prueba recorre las operaciones del flujo sobre la interfaz real, desde el registro o el acceso hasta la obtención del resultado, y comprueba que el comportamiento coincide con el especificado en los casos de uso del análisis. La ejecución de cada prueba se apoya en la documentación de operación de la plataforma y en las cuentas de acceso del entorno de despliegue.

36.1. *PE-001: Flujo clínico completo*

La prueba PE-001 verifica el flujo clínico completo del diagnóstico asistido: la subida de una radiografía, la selección del modelo, la solicitud del diagnóstico, la visualización del resultado y de los mapas de explicabilidad, y la generación del informe PDF. La prueba se inicia desde el panel de diagnóstico del usuario autenticado y recorre las operaciones descritas en los casos de uso CU-005 a CU-010 y CU-037. Los pasos de la prueba son los siguientes.

1. El usuario autenticado accede al panel de diagnóstico.
2. El usuario sube una radiografía de tórax en la zona de carga.
3. El usuario selecciona una arquitectura de inteligencia artificial.
4. El usuario solicita el diagnóstico y espera el procesamiento en la cola.
5. El sistema muestra el resultado con la etiqueta, la confianza y el modelo empleado.
6. El usuario visualiza los mapas de explicabilidad y el sistema genera el informe PDF de la consulta.

El criterio de éxito de la prueba es que el flujo completo se realiza sin errores y el profesional obtiene el diagnóstico, su confianza y los mapas de explicabilidad, mientras el sistema genera el informe de la consulta. El resultado esperado se confirma con la captura del panel de diagnóstico mostrando el resultado y los artefactos generados.

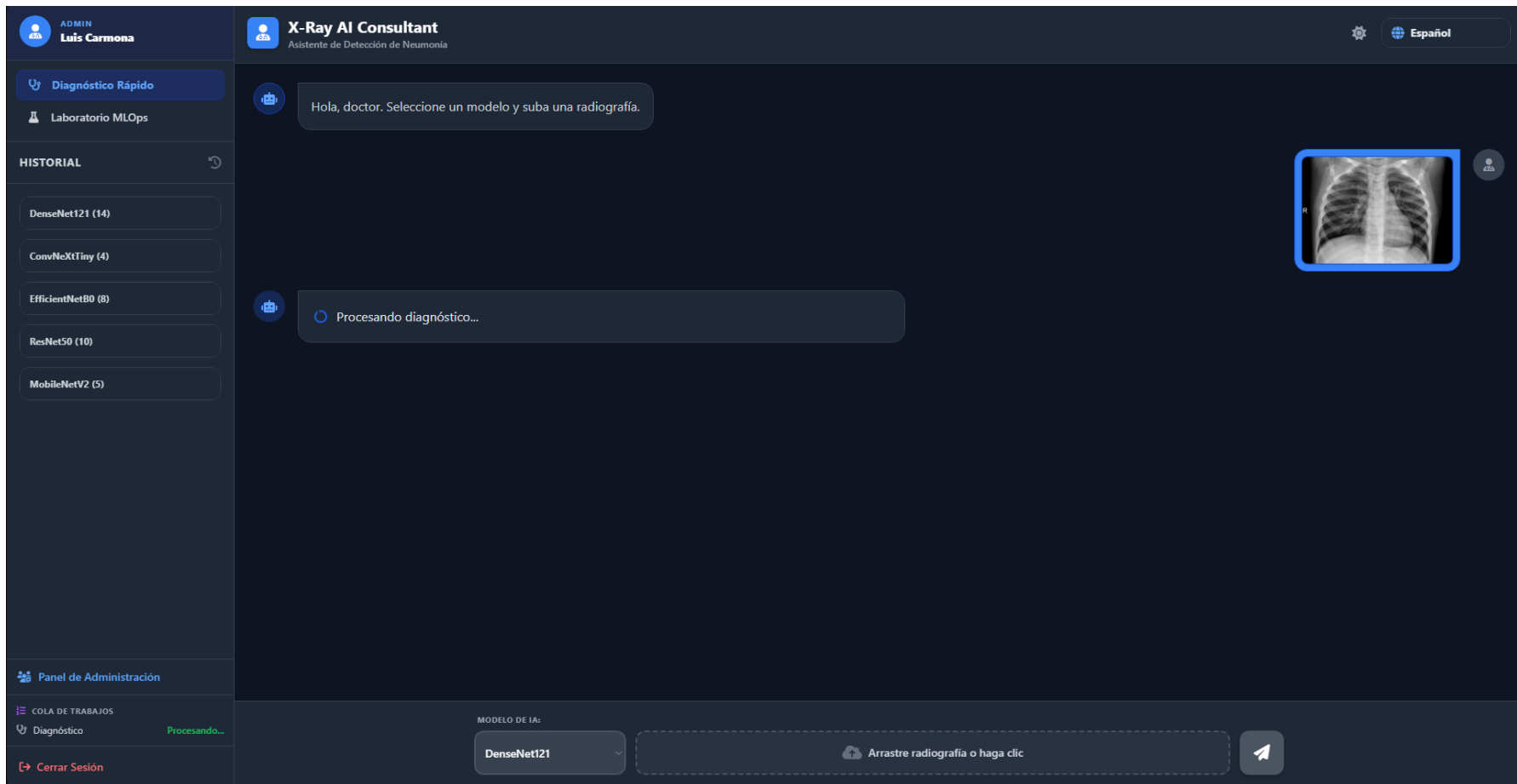


Ilustración x - Visualización de Diagnóstico Rápido y Cola de Trabajo tras el lanzamiento de un diagnóstico

ADMIN
Luis Carmona

Diagnóstico Rápido

Laboratorio MLOps

HISTORIAL

DenseNet121 (15)

ConvNeXtTiny (4)

EfficientNetB0 (8)

ResNet50 (10)

MobileNetV2 (5)

Panel de Administración

Cerrar Sesión

X-Ray AI Consultant
Asistente de Detección de Neumonía

Volver

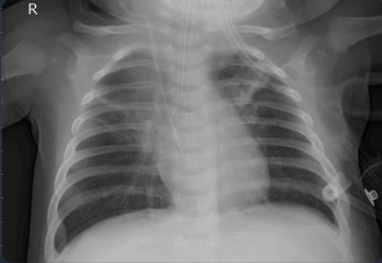
Detalle de Consulta

2026-08-25 20:54

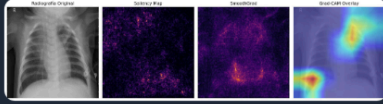
Renombrar

Eliminar

RADIOGRAFÍA ORIGINAL



MAPAS DE CALOR XAI



DIAGNÓSTICO

Neumonía

CONFIANZA

91.78%

MODELO

DenseNet121

PACIENTE

Paciente sin nombre
DenseNet121 #14

MODELO DE IA:

DenseNet121

Arrastre radiografía o haga clic

Ilustración x - Visualización de resultados y mapas de explicabilidad

571

36.2. PE-002: Gestión del historial

La prueba PE-002 verifica la gestión del historial de consultas: la consulta del listado, el detalle de una consulta, el renombrado y la eliminación. La prueba se realiza desde el panel de diagnóstico, sobre las consultas del usuario, y recorre las operaciones descritas en los casos de uso CU-011 a CU-014. Los pasos de la prueba son los siguientes.

1. El usuario consulta su historial de consultas en el panel.
2. El usuario abre el detalle de una consulta del listado.
3. El usuario renombra la consulta con un nuevo nombre visible.
4. El usuario elimina una consulta, confirmando la operación.

El criterio de éxito de la prueba es que todas las operaciones del historial se ejecutan correctamente, el listado se actualiza después de los cambios y el aislamiento de datos entre usuarios se respeta. El resultado esperado se confirma con la captura del historial y del detalle de una consulta.

ADMIN

Luis Carmona

X-Ray AI Consultant

Asistente de Detección de Neumonía

⚙️

🌐 Español

🏠 Diagnóstico Rápido

🏭 Laboratorio MLOps

HISTORIAL

DenseNet121 (15)

ConvNeXtTiny (4)

EfficientNetB0 (8)

ResNet50 (10)

MobileNetV2 (5)

2026-07-29 22:48

Neumonía

Paciente sin nombre MobileNetV2 #5

2026-03-23 18:09

Normal

Paciente sin nombre MobileNetV2 #4

2026-03-23 18:03

Normal

Paciente sin nombre MobileNetV2 #3

2026-03-22 20:42

Neumonía

Paciente sin nombre MobileNetV2 #2

2026-03-22 20:42

Normal

Juan Manuel

⚙️ Panel de Administración

➡ Cerrar Sesión

← Volver

Detalle de Consulta

2026-03-22 20:42

Renombrar

Eliminar

RADIOGRAFÍA ORIGINAL

MAPAS DE CALOR XAI

DIAGNÓSTICO

CONFIANZA

MODELO

PACIENTE

Normal

88.04%

MobileNetV2

Juan Manuel

MODELO DE IA:

DenseNet121

Arrastre radiografía o haga clic

📁

Ilustración x - Visualización del historial antes de la eliminación de un Diagnóstico y tras el cambio de nombre del paciente

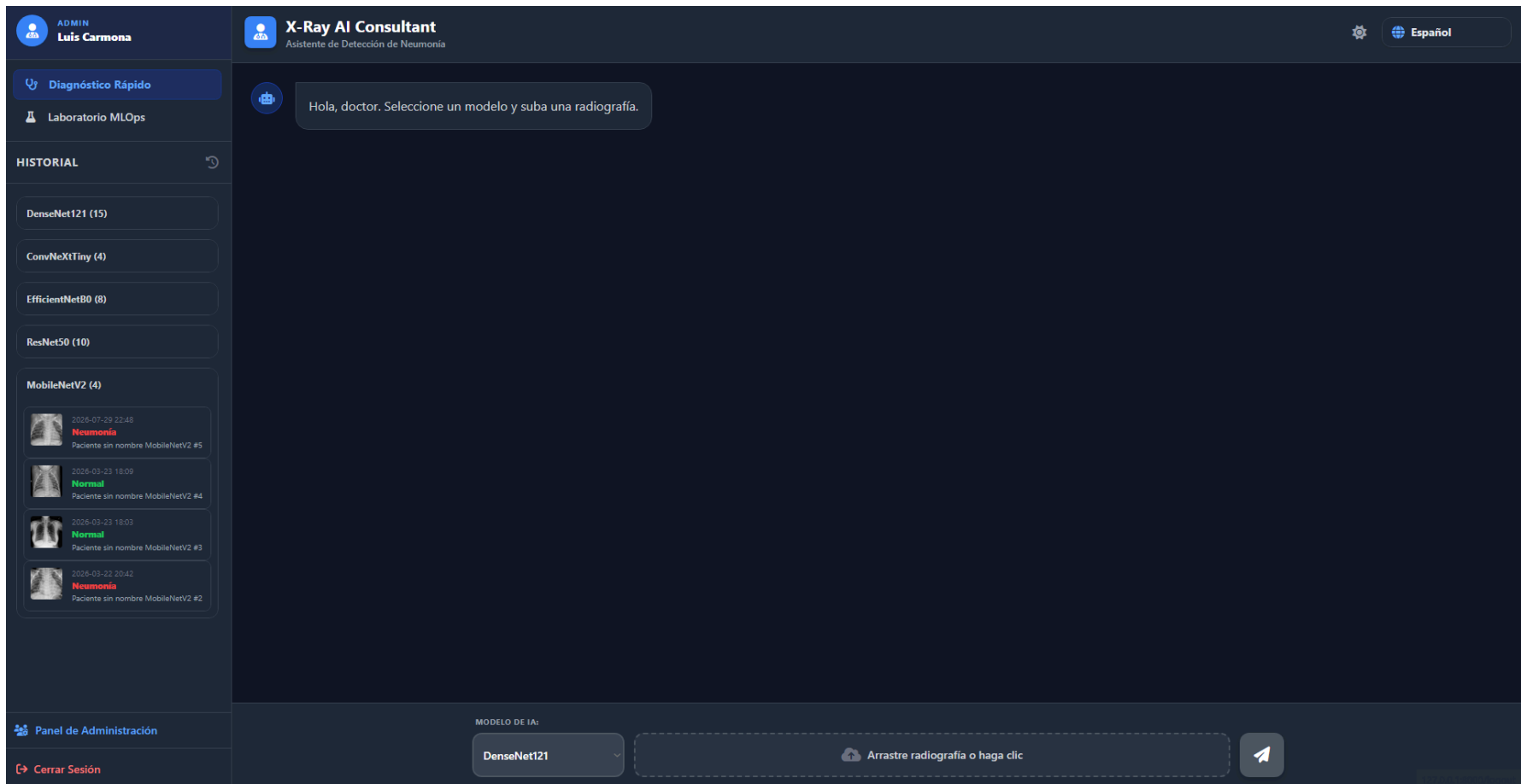


Ilustración x - Visualización del historial tras la eliminación de un Diagnóstico

36.3. PE-003: Flujo del laboratorio

La prueba PE-003 verifica el flujo del laboratorio de entrenamiento: la conversación con el asistente, el lanzamiento del experimento, la consulta de las sesiones y de los resultados, y la generación del informe PDF de la sesión. La prueba se realiza desde el laboratorio del usuario investigador y recorre las operaciones descritas en los casos de uso CU-015 a CU-030. Los pasos de la prueba son los siguientes.

1. El usuario accede al laboratorio de entrenamiento.
2. El usuario configura el experimento con el asistente conversacional y selecciona la carpeta del dataset.
3. El usuario lanza el experimento y lo encola en segundo plano.
4. El usuario consulta la sesión, el ranking de modelos y los resultados de un modelo.
5. El usuario genera el informe PDF de la sesión.

El criterio de éxito de la prueba es que el experimento se configura y lanza mediante el asistente, se ejecuta en segundo plano y sus resultados e informe se obtienen sin errores. El resultado esperado se confirma con la captura de la vista de resultados de la sesión.

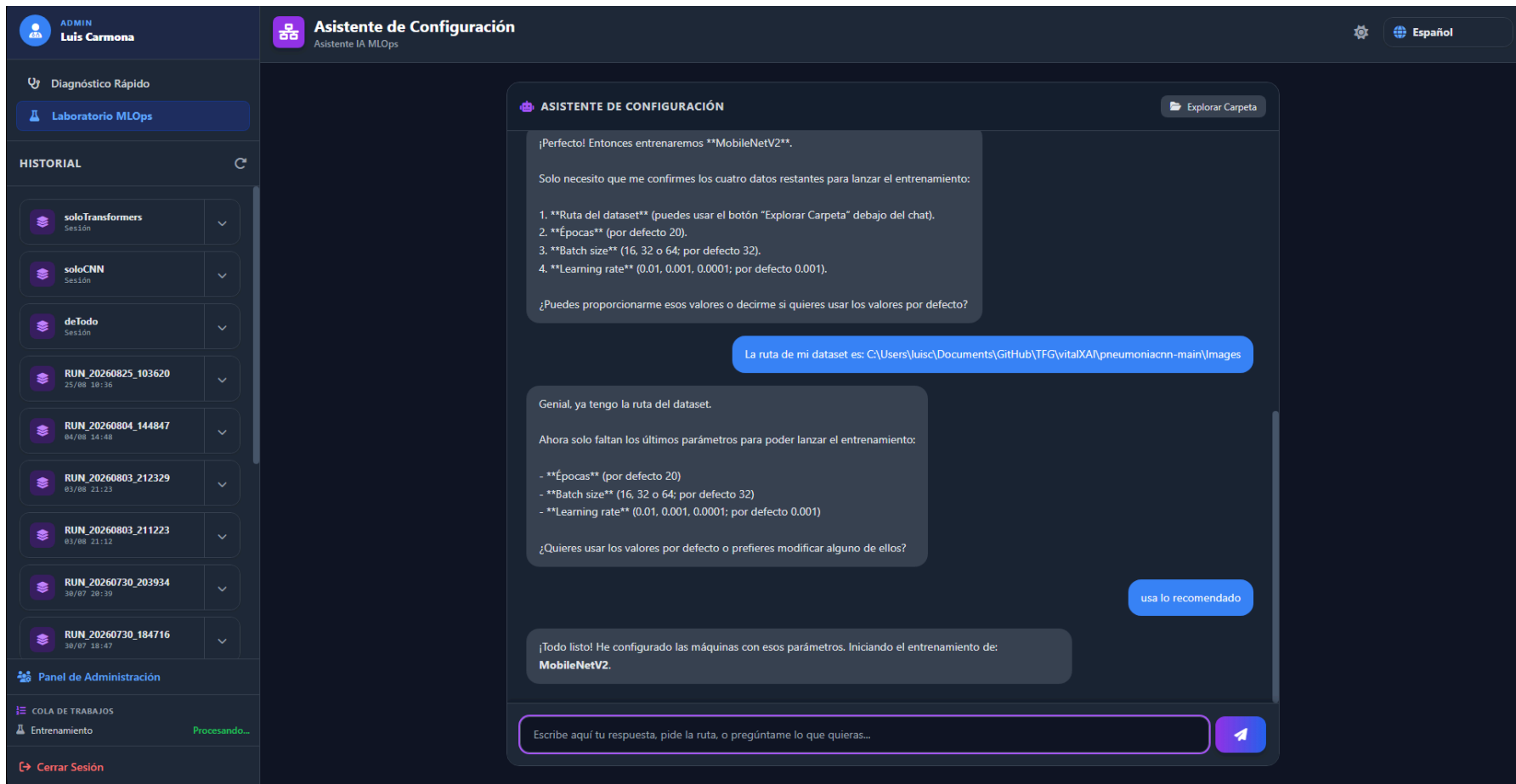


Ilustración x - Visualización del panel del Laboratorio y la Cola de Trabajo tras lanzar un entrenamiento

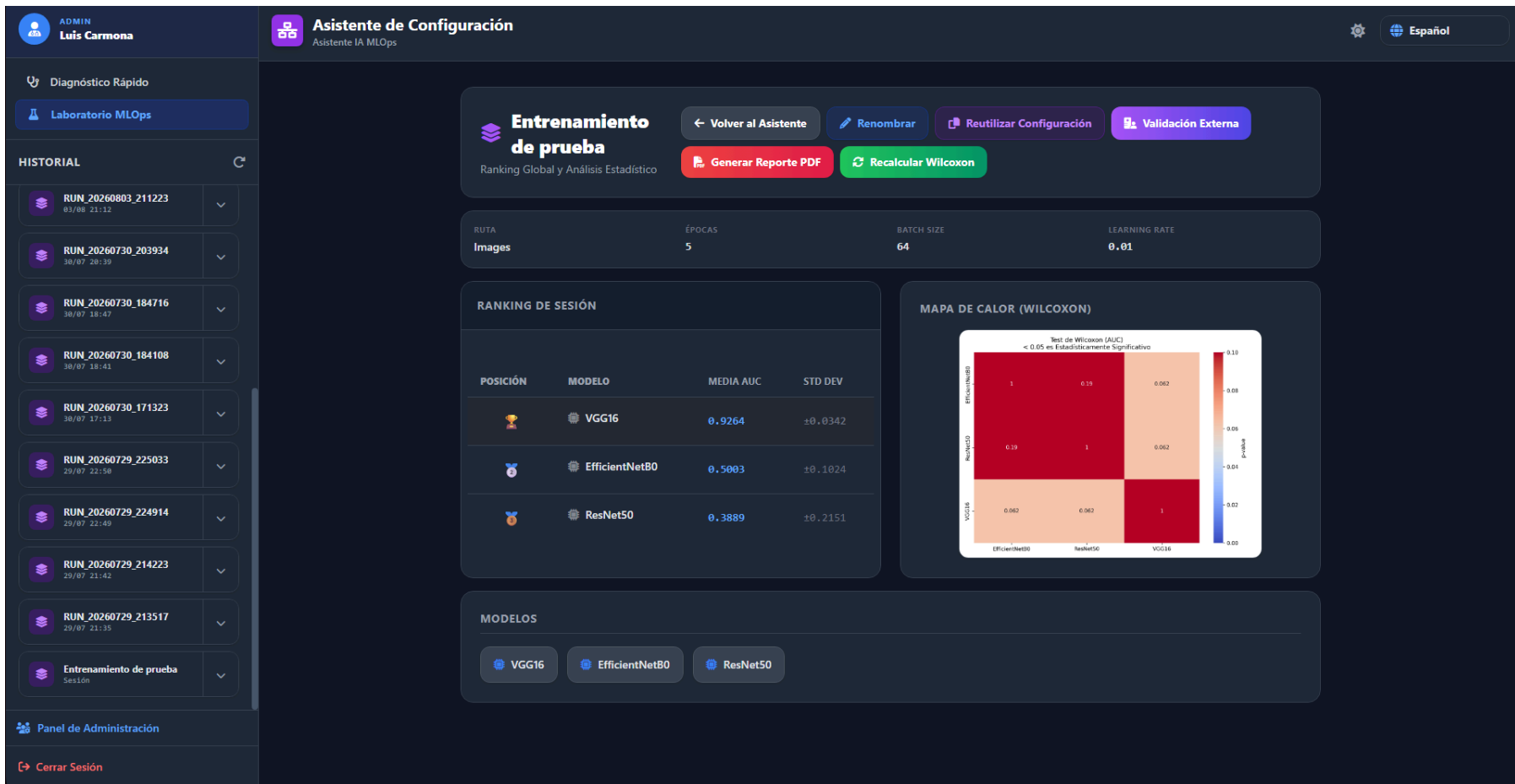


Ilustración x - Visualización de los resultados del entrenamiento

X-RAY CONSULTANT AI - MEDICAL REPORT

Protocolo MLOps: Deep Learning para Detección de Neumonía

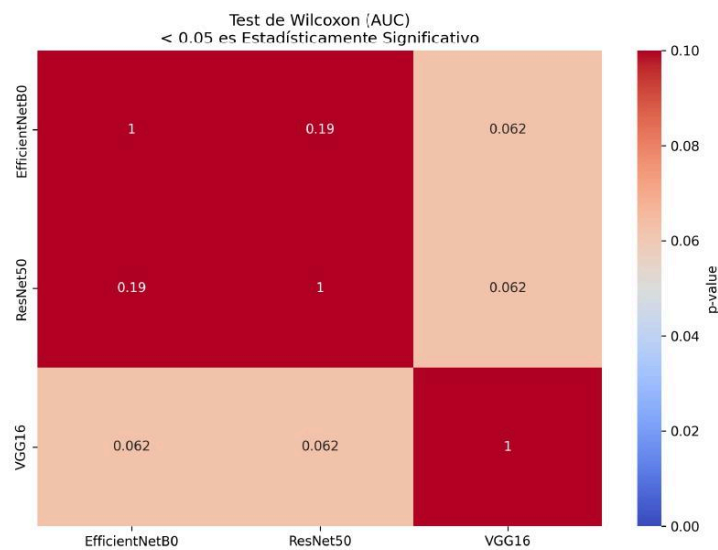
1. CONFIGURACIÓN DEL SISTEMA Y PARÁMETROS

ID Sesión:	Entrenamiento de prueba
Fecha:	25/08/2026 21:18
Dataset:	C:\\Users\\luisc\\Documents\\GitHub\\TFG\\vitalXA\\pneumoniaccn-main\\Images
Modelos:	ResNet50, EfficientNetB0, VGG16
Hiperparámetros:	Epochs: 5 Batch: 64 LR: 0.01

2. RENDIMIENTO GLOBAL (K-FOLD CROSS-VALIDATION)

Arquitectura de Modelo	Media AUC	Desviación Estándar
VGG16	0.9263888888888887	0.034207997810417584
EfficientNetB0	0.5003472222222222	0.10237698693409425
ResNet50	0.3888888888888884	0.21509008394847717

Matriz de Significancia Estadística (P-Values):



36.4. PE-004: Ejecución asíncrona de la cola

La prueba PE-004 verifica la ejecución asíncrona de la cola de trabajos: la consulta del estado de la cola y la cancelación de un trabajo pendiente. La prueba se realiza desde el panel de diagnóstico o el laboratorio, y recorre las operaciones descritas en los casos de uso CU-034 y CU-035. Los pasos de la prueba son los siguientes.

1. El usuario consulta el panel de la cola de trabajos.
2. El usuario comprueba el estado de sus trabajos encolados o en ejecución; la finalización o el fallo se muestran en el mensaje del diagnóstico o en la consola del laboratorio.
3. El usuario cancela un trabajo pendiente, confirmando la operación.

El criterio de éxito de la prueba es que el usuario conoce el estado de sus trabajos y puede cancelar un trabajo pendiente, mientras que un trabajo en ejecución no se interrumpe. El resultado esperado se confirma con la captura del panel de la cola de trabajos.

Diagnóstico Rápido

Laboratorio MLOps

HISTORIAL

soloTransformers
Sesión

soloCNN
Sesión

deTodo
Sesión

RUN_20260825_103620
25/08 10:36

RUN_20260804_144847
04/08 14:48

RUN_20260803_212329
03/08 21:23

RUN_20260803_211223
03/08 21:12

RUN_20260730_203934
30/07 20:39

Panel de Administración

COLA DE TRABAJOS

Validación Externa

Posición #2

Entrenamiento

Modelos: MobileNetV2, MobileNetV3Large, ConvNeXtTiny, deit, swin_base

Diagnóstico

Posición #1

Entrenamiento

Procesando...

Cerrar Sesión

ASISTENTE DE CONFIGURACIÓN

Explorar Carpeta

¡Hola! Soy tu asistente de MLOps. Para empezar un nuevo entrenamiento de modelos para detectar neumonía, necesito que me digas qué arquitecturas quieres usar y la ruta de la carpeta de tus imágenes. ¿Con qué empezamos?

➤

Ilustración x - Visualización de la Cola de Trabajo

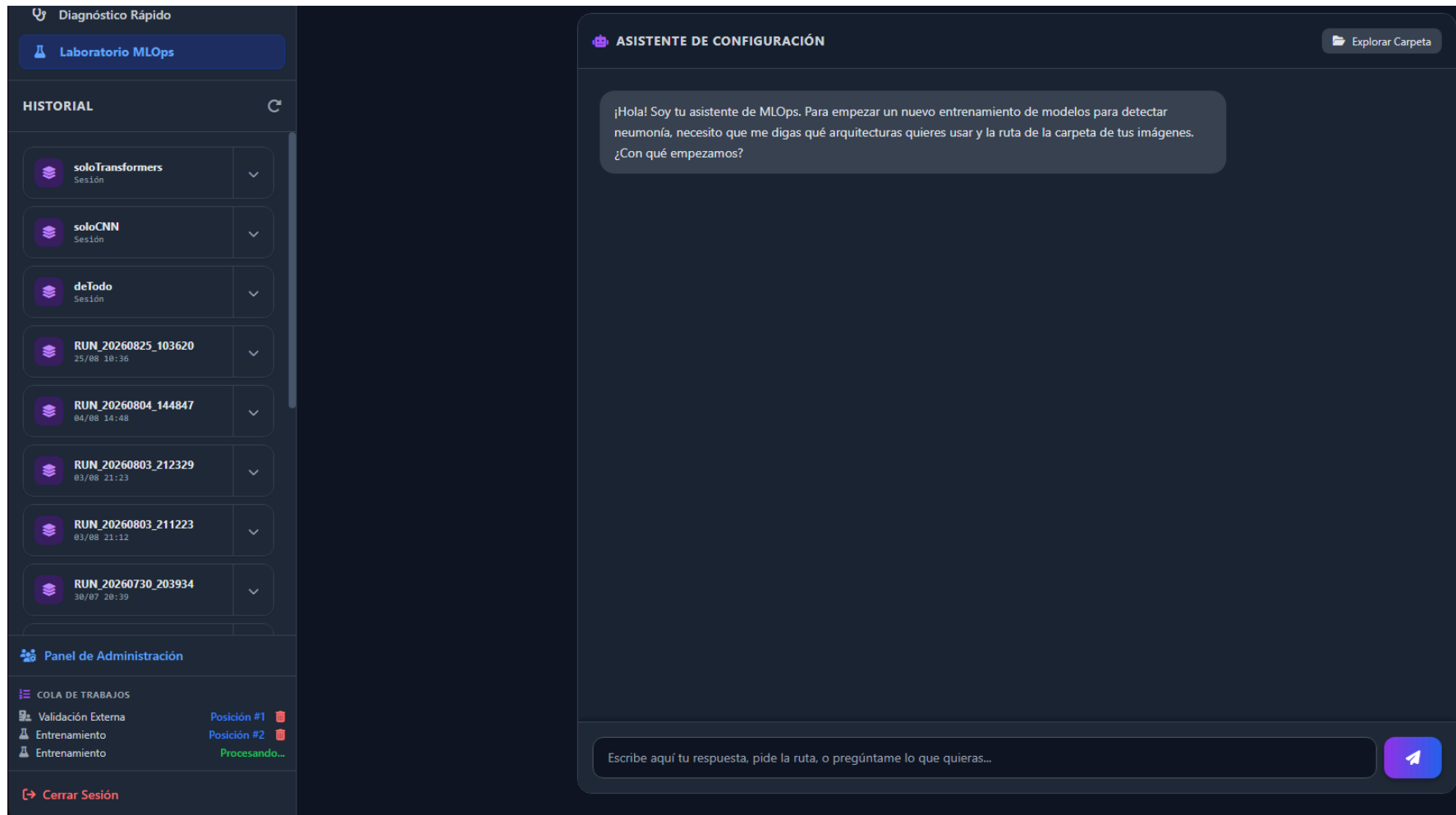


Ilustración x - Visualización de la Cola de Trabajo tras la eliminación del Diagnóstico pendiente

36.5. PE-005: Supervisión administrativa

La prueba PE-005 verifica la supervisión administrativa de la plataforma: el listado de usuarios, la consulta de las consultas de un usuario y su detalle. La prueba se realiza desde el panel del usuario con rol de administrador, y recorre las operaciones descritas en los casos de uso CU-031 a CU-033. Los pasos de la prueba son los siguientes.

1. El administrador abre el panel de administración desde su ventana.
2. El administrador consulta el listado de usuarios con sus recuentos de actividad.
3. El administrador consulta las consultas de un usuario concreto y el detalle de una consulta.

El criterio de éxito de la prueba es que el administrador supervisa la actividad de un usuario concreto desde el panel de administración, con la comprobación de rol aplicada por el sistema. El resultado esperado se confirma con la captura del diálogo de administración.

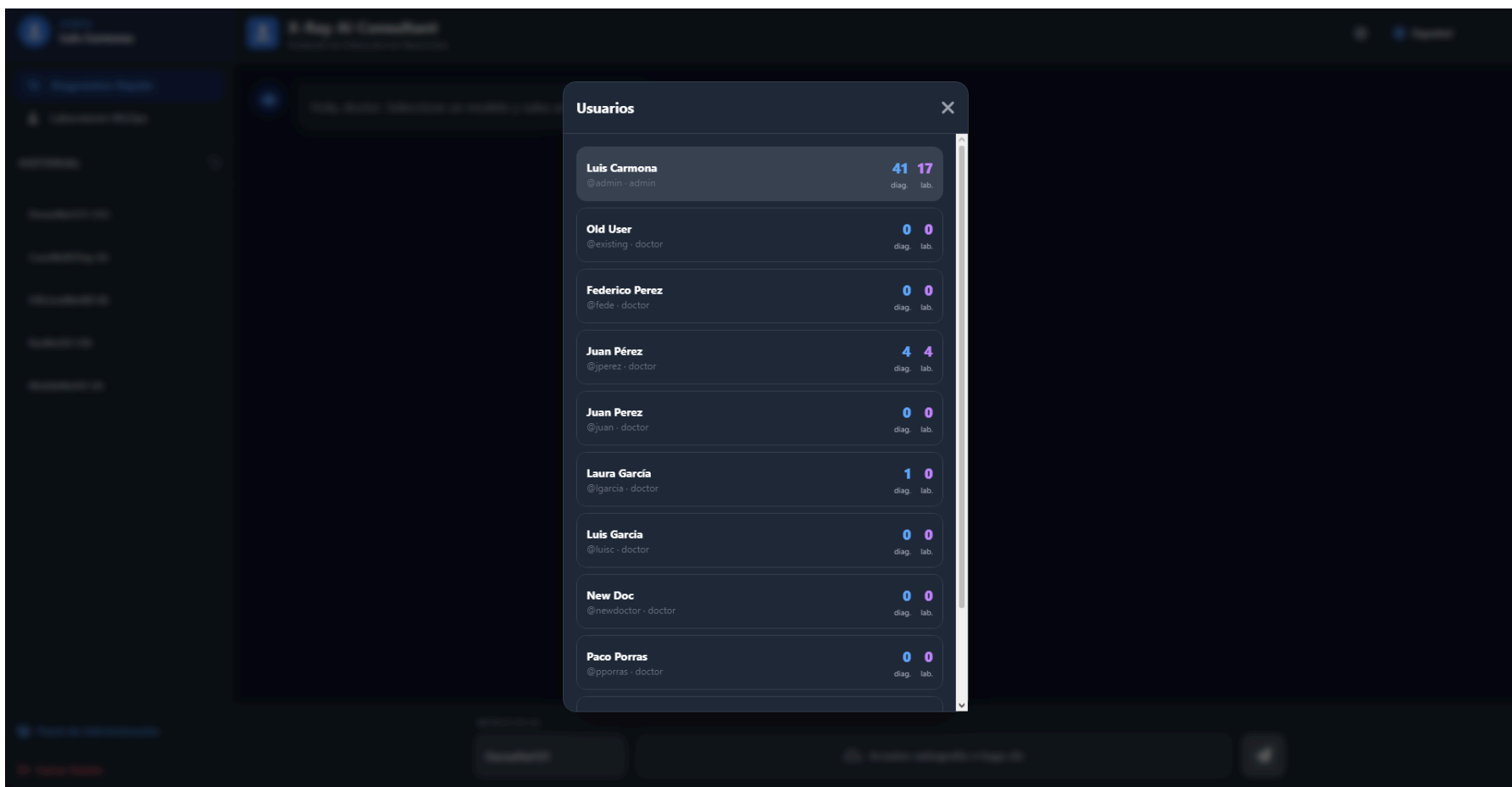


Ilustración x - Visualización del Panel de Administrador

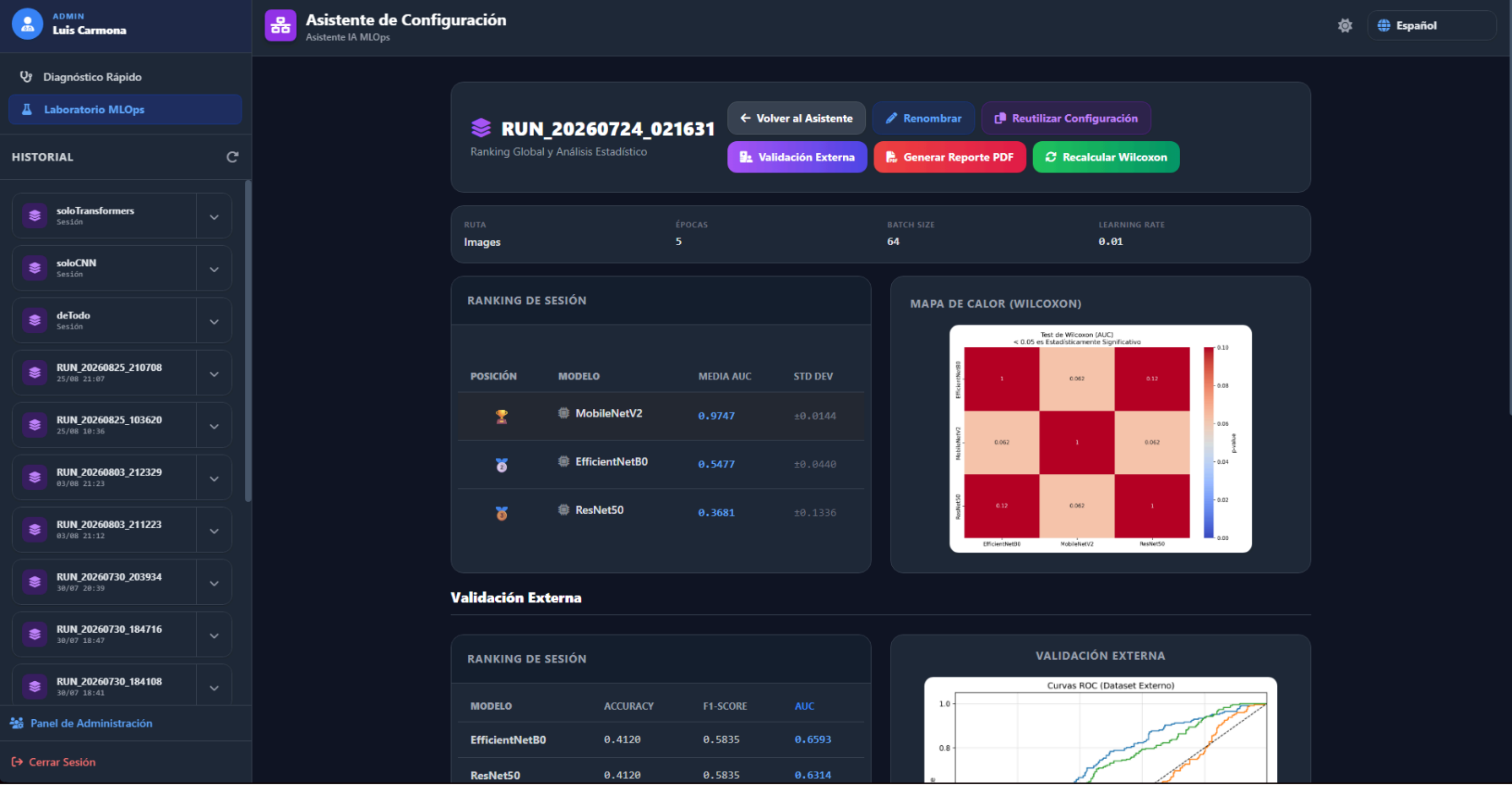


Ilustración x - Visualización de los detalles de una consulta de otro usuario

36.6. PE-006: Capacidades transversales

La prueba PE-006 verifica las capacidades transversales de la plataforma: el cambio de idioma y el cambio del tema visual de la interfaz. La prueba se realiza desde cualquiera de las ventanas del sistema y recorre las operaciones descritas en los casos de uso CU-004 y CU-036. Los pasos de la prueba son los siguientes.

1. El usuario cambia el idioma de la interfaz mediante el selector de idioma.
2. El sistema aplica las traducciones en toda la interfaz sin recargar la página.
3. El usuario alterna el tema visual entre el claro y el oscuro.
4. El sistema aplica el tema en toda la interfaz y conserva la preferencia.

El criterio de éxito de la prueba es que el idioma y el tema se aplican en toda la interfaz sin interrumpir la navegación. El resultado esperado se confirma con la captura de una ventana en el idioma y el tema seleccionados.

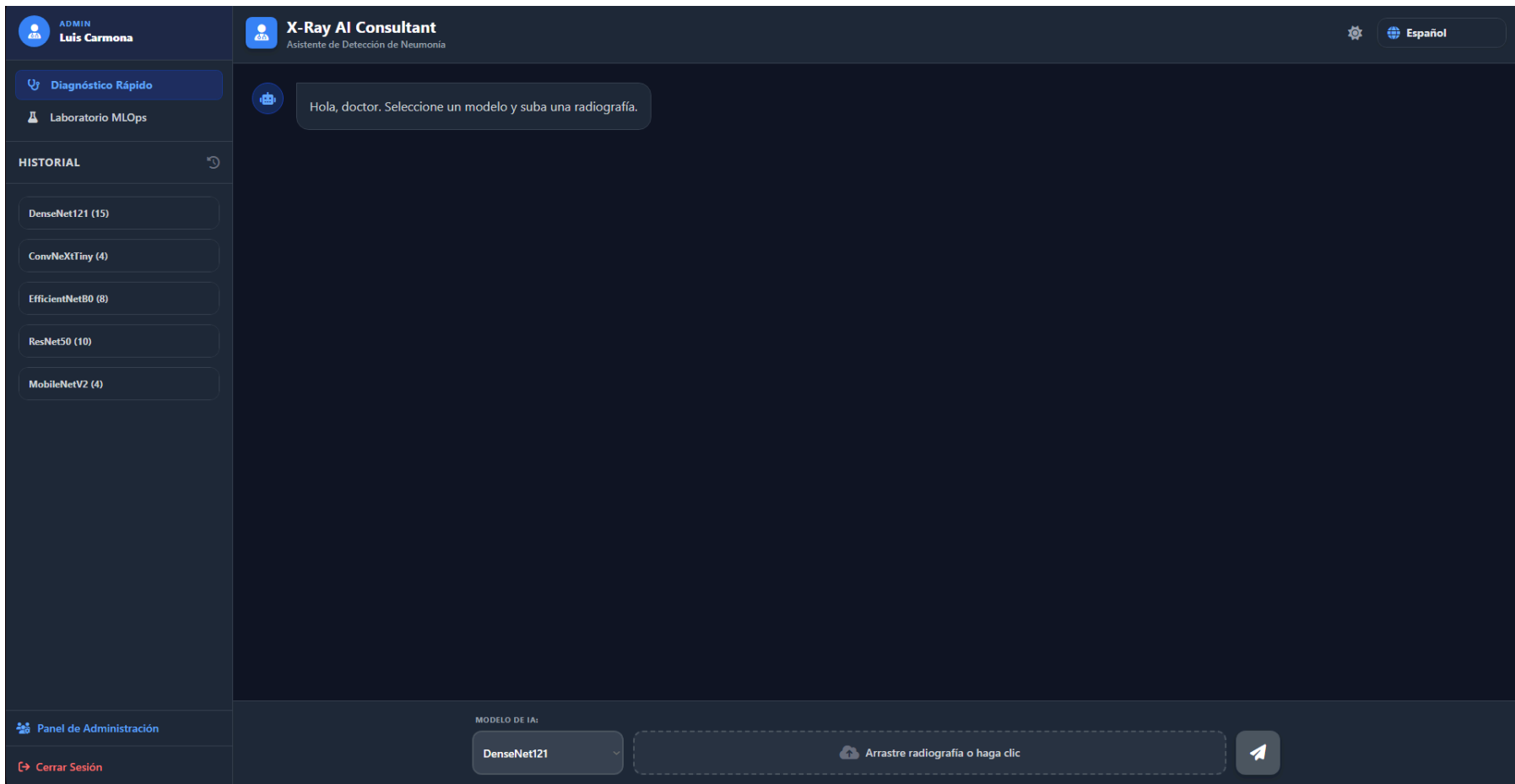


Ilustración x - Visualización de la interfaz en Español y estilo oscuro

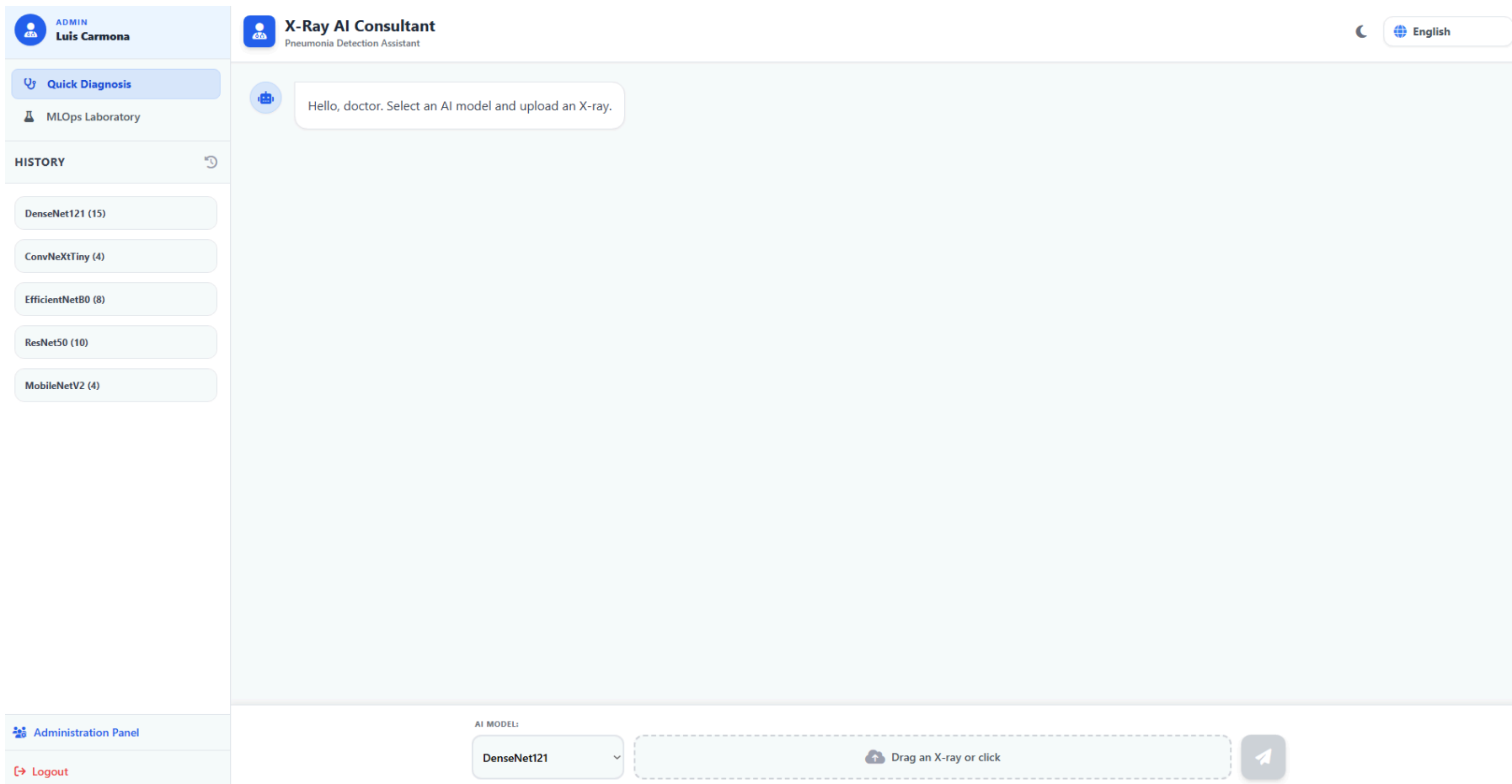


Ilustración x - Visualización de la interfaz en Inglés y estilo claro

36.7. Resultados de la validación funcional

Las seis pruebas de sistema documentan los flujos principales de uso de la plataforma sobre el despliegue real, cubriendo el diagnóstico, el historial, el laboratorio, la cola, la administración y las capacidades transversales. La gestión funcional del historial se comprueba, pero el aislamiento entre dos usuarios no cuenta con una verificación de extremo a extremo documentada. La tabla siguiente resume el resultado de la validación funcional.

ID	Flujo verificado	Resultado
PE-001	Flujo clínico completo del diagnóstico	Comprobación funcional documentada.
PE-002	Gestión del historial de consultas	Comprobación funcional documentada. El aislamiento entre dos usuarios no queda verificado de extremo a extremo.
PE-003	Flujo del laboratorio de entrenamiento	Comprobación funcional documentada.
PE-004	Ejecución asíncrona de la cola de trabajos	Comprobación funcional documentada.
PE-005	Supervisión administrativa	Comprobación funcional documentada.
PE-006	Capacidades transversales (idioma y tema)	Comprobación funcional documentada.

Tabla x - Resultados de la validación funcional

Las capturas incluidas en este capítulo son evidencia ilustrativa de una ejecución manual, pero no constituyen por sí solas una validación reproducible: no se documentan de forma completa los datos de entrada, la identidad de la persona que ejecutó cada flujo, la fecha, el entorno exacto ni un registro detallado de cada resultado. Por ello, el estado de las seis pruebas se expresa como comprobación funcional documentada y no como superación completa.

La verificación documentada no permite afirmar que todos los requisitos estén satisfechos. Las pruebas automatizadas de los capítulos 34 y 35 respaldan los componentes ejecutados y los mecanismos de seguridad cubiertos, pero las cuatro pruebas de integración se concentran en autenticación. Las pruebas PI-004 a PI-007, incluido el aislamiento de datos entre usuarios exigido por RF-005, permanecen pendientes. Tampoco existe verificación documentada de los requisitos de protección de datos, anonimización, retención, copias de seguridad y recuperación tras reinicio. Las pruebas de rendimiento PR-001 a PR-005 no se han ejecutado, por lo que no existe evidencia para declarar verificados RNF-019, RNF-020 y RNF-021. Finalmente, no se realizó una evaluación con usuarios reales que respalde empíricamente los requisitos de usabilidad y accesibilidad. El conjunto de resultados debe interpretarse, por tanto, como evidencia parcial del comportamiento de la plataforma y no como una certificación completa del cumplimiento del catálogo.

Parte VI.

Conclusiones del Sistema

Parte VII.

Referencias Bibliográficas

Capítulo 37. Referencias bibliográficas

Apostolopoulos, I. D., & Mpesiana, T. A. (2020). Covid-19: Automatic detection from X-ray images utilizing transfer learning with convolutional neural networks. *Physical and Engineering Sciences in Medicine*, 43(2), 635-640.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Brown, T., & al., e. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.

Chefer, H., Gur, S., & Wolf, L. (2021). Transformer interpretability beyond attention visualization. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 782-791.

Chollet, F. (2017). Xception: Deep learning with depthwise separable convolutions. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 1251-1258.

Chowdhury, M. E. H., & al., e. (2020). Can AI help in screening viral and COVID-19 pneumonia? *IEEE Access*, 8, 132665-132676. <https://doi.org/10.1109/ACCESS.2020.3010287>

DeGrave, A. J., Janizek, J. D., & Lee, S.-I. (2021). AI for radiographic COVID-19 detection selects shortcuts over signal. *Nature Machine Intelligence*, 3, 610-619. <https://doi.org/10.1038/s42256-021-00338-7>

DeLong, E. R., DeLong, D. M., & Clarke-Pearson, D. L. (1988). Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach. *Biometrics*, 44(3), 837-845.

Nadeau, C., & Bengio, Y. (2003). Inference for the generalization error. *Machine Learning*, 52, 239-281. <https://doi.org/10.1023/A:1024068626366>.

Deng, J., & al., e. (2009). ImageNet: A large-scale hierarchical image database. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 248-255.

Dosovitskiy, A., & al., e. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations (ICLR)*.

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861-874.

Franquet, T. (2001). Imaging of pneumonia: Trends and algorithms. *European Respiratory Journal*, 18(1), 196-208.

Gonzalez, R. C., & Woods, R. E. (2008). *Digital Image Processing (3rd ed.)*. Prentice Hall.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

Groq. (2024). Groq: Instant AI inference. Obtenido de <https://groq.com>

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *Proceedings of the 34th International Conference on Machine Learning*, 1321-1330.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770-778.

Holzinger, A., & al., e. (2022). Explainable AI methods — A brief overview. *Lecture Notes in Computer Science*, 13200, 1-19.

Hopstaken, R. M., Witbraad, T., van Engelshoven, J. M., & Dinant, G. J. (2004). Inter-observer variation in the interpretation of chest radiographs for pneumonia in community-acquired lower respiratory tract infections. *Clinical Radiology*, 59(8), 743-752. <https://doi.org/10.1016/j.crad.2004.01.011>

Howard, A. G., & al., e. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*.

Howard, A., & al., e. (2019). Searching for MobileNetV3. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 1314-1324.

Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 4700-4708.

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263-1284. <https://doi.org/10.1109/TKDE.2008.239>.

Jiang, A. Q., & al., e. (2023). Mistral 7B. *arXiv preprint arXiv:2310.06825*.

Kermany, D. S., & al., e. (2018). Identifying medical diagnoses and treatable diseases by image-based deep learning. *Cell*, 172(5), 1122-1131.e9. <https://doi.org/10.1016/j.cell.2018.02.010>.

Kreuzberger, D., Kühl, N., & Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11, 31866-31879. <https://doi.org/10.1109/ACCESS.2023.3262138>

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 1097-1105.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444.

Linardatos, P., Papastefanopoulos, V., & Kotsiantis, S. (2021). Explainable AI: A review of machine learning interpretability methods. *Entropy*, 23(1), 18.

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*. Obtenido de <https://arxiv.org/abs/1711.05101>

Liu, Z., & al., e. (2021). Swin Transformer: Hierarchical vision transformer using shifted windows. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 10012-10022.

Liu, Z., & al., e. (2022). A ConvNet for the 2020s. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 11976-11986.

Mandell, L. A., & al., e. (2007). Infectious Diseases Society of America/American Thoracic Society consensus guidelines on the management of community-acquired pneumonia in adults. *Clinical Infectious Diseases*, 44(Supplement 2), S27-S72.

Meta AI. (2024). Introducing Meta Llama 3: The most capable openly available LLM to date. Obtenido de <https://ai.meta.com/blog/meta-llama-3/>

Minaee, S., Kafieh, R., Sonka, M., Yazdani, S., & Jamalipour Soufi, G. (2020). Deep-COVID: Predicting COVID-19 from chest X-ray images using deep transfer learning. *Medical Image Analysis*, 65, 101794. <https://doi.org/10.1016/j.media.2020.101794>

Murphy, Z. R., Venkatesh, K., Sulam, J., & Yi, P. H. (2022). Visual transformers and convolutional neural networks for disease classification on radiographs: A comparison of performance, sample efficiency, and hidden stratification. *Radiology: Artificial Intelligence*, 4(6), e220012. <https://doi.org/10.1148/ryai.220012>

Nagendran, M., Chen, Y., Lovejoy, C. A., Gordon, A. C., Komorowski, M., Harvey, H., Topol, E. J., Ioannidis, J. P. A., Collins, G. S., & Maruthappu, M. (2020). Artificial intelligence versus clinicians: Systematic review of design, reporting standards, and claims of deep learning studies. *BMJ*, 368, m689. <https://doi.org/10.1136/bmj.m689>.

Ouyang, L., & al., e. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 27730-27744.

Petsiuk, V., Das, A., & Saenko, K. (2018). RISE: Randomized input sampling for explanation of black-box models. *Proceedings of the British Machine Vision Conference*.

Rahman, T., & al., e. (2021). Exploring the effect of image enhancement techniques on COVID-19 detection using chest X-ray images. *Computers in Biology and Medicine*, 132, 104319.

Rajaraman, S., Thoma, G., Antani, S., & Candemir, S. (2019). Visualizing and explaining deep learning predictions for pneumonia detection in pediatric chest radiographs. *Medical Imaging 2019: Computer-Aided Diagnosis*, 10950, 109500S-109500S-12. <https://doi.org/10.1117/12.2512752>

Rajpurkar, P., & al., e. (2017). CheXNet: Radiologist-level pneumonia detection on chest X-rays with deep learning. *arXiv preprint arXiv:1711.05225*.

Russakovsky, O., & al., e. (2015). ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3), 211-252.

Sandler, M., & al., e. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 4510-4520.

Selvaraju, R. R., & al., e. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *Proceedings of the IEEE International Conference on Computer Vision*, 618-626.

- Simonyan, K., Vedaldi, A., & Zisserman, A. (2014). Deep inside convolutional networks: Visualising image classification models and saliency maps. arXiv preprint arXiv:1312.6034.
- Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *International Conference on Learning Representations (ICLR)*.
- Singhal, K., & al., e. (2023). Large language models encode clinical knowledge. *Nature*, 620, 172-180.
- Smilkov, D., & al., e. (2017). SmoothGrad: Removing noise by adding noise. arXiv preprint arXiv:1706.03825.
- Szegedy, C., & al., e. (2016). Rethinking the Inception architecture for computer vision. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2818-2826.
- Szeliski, R. (2010). *Computer Vision: Algorithms and Applications*. Springer.
- Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *International Conference on Machine Learning (ICML)*, 6105-6114.
- Tan, M., & Le, Q. V. (2021). EfficientNetV2: Smaller models and faster training. *International Conference on Machine Learning (ICML)*, 10096-10106.
- Thirunavukarasu, A. J., & al., e. (2023). Large language models in medicine. *Nature Medicine*, 29(8), 1930-1940.
- Tjoa, E., & Guan, C. (2020). A survey on explainable artificial intelligence (XAI): Toward medical XAI. *IEEE Transactions on Neural Networks and Learning Systems*, 32(11), 4793-4813.
- Torres, A., & al., e. (2021). Pneumonia. *Nature Reviews Disease Primers*, 7(1), 25.
- Touvron, H., & al., e. (2021). Training data-efficient image transformers & distillation through attention. *Proceedings of the 38th International Conference on Machine Learning*, 10347-10357.
- Touvron, H., & al., e. (2023). Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971.
- Vaswani, A., & al., e. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998-6008.
- Varma, S., & Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7, 91. <https://doi.org/10.1186/1471-2105-7-91>.
- Wang, Z., & al., e. (2004). Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4), 600-612.
- Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6), 80-83.

World Health Organization. (2024). Pneumonia fact sheet. Obtenido de <https://www.who.int/news-room/fact-sheets/detail/pneumonia>

Yosinski, J., & al., e. (2014). How transferable are features in deep neural networks? *Advances in Neural Information Processing Systems*, 27, 3320-3328.

Zech, J. R., Badgeley, M. A., Liu, M., Costa, A. B., Titano, J. J., & Oermann, E. K. (2018). Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs: A cross-sectional study. *PLOS Medicine*, 15(11), e1002683. <https://doi.org/10.1371/journal.pmed.1002683>

Aggarwal, R., & al., e. (2021). Diagnostic accuracy of deep learning in medical imaging: A systematic review and meta-analysis. *NPJ Digital Medicine*, 4, 65.

Amershi, S., & al., e. (2019). Guidelines for human-AI interaction. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 1-13.

De Fauw, J., & al., e. (2018). Clinically applicable deep learning for diagnosis and referral in retinal disease. *Nature Medicine*, 24, 1342-1350.

Esteva, A., & al., e. (2017). Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542, 115-118.

Kelly, C. J., & al., e. (2019). Key challenges for delivering clinical impact with artificial intelligence. *BMC Medicine*, 17, 195.

Tudor Car, L., Dhinakaran, D. A., Kyaw, B. M., Kowatsch, T., Joty, S., Theng, Y.-L., & Atun, R. (2020). Conversational agents in health care: Scoping review and conceptual analysis. *Journal of Medical Internet Research*, 22(8), e17158. <https://doi.org/10.2196/17158>

Laranjo, L., & al., e. (2018). Conversational agents in healthcare: A systematic review. *Journal of the American Medical Informatics Association*, 25(9), 1248-1258.

Liu, X., & al., e. (2019). A comparison of deep learning performance against health-care professionals in detecting diseases from medical imaging: A systematic review and meta-analysis. *The Lancet Digital Health*, 1(6), e271-e297.

McKinney, S. M., & al., e. (2020). International evaluation of an AI system for breast cancer screening. *Nature*, 577, 89-94.

OWASP. (2021). OWASP Top 10:2021 – The ten most critical web application security risks. Obtenido de <https://owasp.org/Top10/>.

Park, S. H., & Han, K. (2018). Methodologic guide for evaluating clinical performance and effect of artificial intelligence technology for medical diagnosis and prediction. *Radiology*, 286(3), 800-809.

Sculley, D., & al., e. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28, 2503-2511.

Varoquaux, G., & Cheplygina, V. (2019). Machine learning for medical imaging: Methodological failures and recommendations for the future. *NPJ Digital Medicine*, 2, 48.

Anderson, D. J. (2010). *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press.

Beck, K. (2000). *Extreme Programming Explained: Embrace Change*. Addison-Wesley.

Boehm, B., & Turner, R. (2004). *Balancing Agility and Discipline: A Guide for the Perplexed*. Addison-Wesley.

Dybå, T., & Dingsøyr, T. (2008). Empirical studies of agile software development: A systematic review. *Information and Software Technology*, 50(9-10), 833-859.

Kniberg, H., & Skarin, M. (2010). *Kanban and Scrum – Making the Most of Both*. C4Media.

Reddy, A. (2016). *The Scrumban [R]Evolution: Getting the Most Out of Agile, Scrum, and Lean Kanban*. Addison-Wesley.

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*.

Sutherland, J. (2014). *Scrum: The Art of Doing Twice the Work in Half the Time*. Crown Business.

España. (2018). Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales. *Boletín Oficial del Estado*, 294, 119788-119857.

IETF. (2018). RFC 8446 – The Transport Layer Security (TLS) Protocol Version 1.3. Obtenido de <https://datatracker.ietf.org/doc/html/rfc8446>.

Parlamento Europeo y Consejo de la Unión Europea. (2016). Reglamento (UE) 2016/679 relativo a la protección de las personas físicas en lo que respecta al tratamiento de los datos personales y a la libre circulación de estos datos (RGPD). *Diario Oficial de la Unión Europea*, L119, 1-88.

Parlamento Europeo y Consejo de la Unión Europea. (2017). Reglamento (UE) 2017/745 sobre los productos sanitarios, por el que se modifican la Directiva 2001/83/CE, el Reglamento (CE) n.º 178/2002 y el Reglamento (CE) n.º 1223/2009 y por el que se derogan las Directivas 90/385/CEE y 93/42/CEE del Consejo. *Diario Oficial de la Unión Europea*, L117, 1-175.

Parlamento Europeo y Consejo de la Unión Europea. (2024). Reglamento (UE) 2024/1689 por el que se establecen normas armonizadas en materia de inteligencia artificial y por el que se modifican los Reglamentos (CE) n.º 300/2008, (UE) n.º 167/2013, (UE) n.º 168/2013, (UE) 2018/858, (UE) 2018/1139 y (UE) 2019/2144 y las Directivas 2014/90/UE, (UE) 2016/797 y (UE) 2020/1828 (Reglamento de Inteligencia Artificial). *Diario Oficial de la Unión Europea*, serie L, 12.7.2024.

Python Software Foundation. (2024). PEP 8 – Style Guide for Python Code. Obtenido de <https://peps.python.org/pep-0008/>.

Universidad Pablo de Olavide. (2014). Guía técnica para la realización del Trabajo Fin de Grado en Ingeniería Informática en Sistemas de Información. Escuela Politécnica Superior.

van Rossum, G., Warsaw, B., & Coghlan, N. (2001). PEP 8 – Style Guide for Python Code. Obtenido de <https://peps.python.org/pep-0008/>.

Cockburn, A. (2001). Writing Effective Use Cases. Addison-Wesley.

Jacobson, I., Booch, G., & Rumbaugh, J. (1999). The Unified Software Development Process. Addison-Wesley.

Larman, C. (2004). Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development (3rd ed.). Prentice Hall.

W3C. (2023). Web Content Accessibility Guidelines (WCAG) 2.2. World Wide Web Consortium. Obtenido de <https://www.w3.org/TR/WCAG22/>.

Wiegers, K., & Beatty, J. (2013). Software Requirements (3rd ed.). Microsoft Press.

Myers, G. J., Sandler, C., & Badgett, T. (2011). The Art of Software Testing (3rd ed.). Wiley.

Chen, P. P.-S. (1976). The entity-relationship model: Toward a unified view of data. ACM Transactions on Database Systems, 1(1), 9-36.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

Oracle. (2024). MySQL 8.0 Reference Manual. Obtenido de <https://dev.mysql.com/doc/refman/8.0/en/>.

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd ed.). Addison-Wesley.

Nielsen, J. (1994). Usability Engineering. Morgan Kaufmann.

Sharp, H., Rogers, Y., & Preece, J. (2019). Interaction Design: Beyond Human-Computer Interaction (5th ed.). Wiley.

FastAPI. (2024). FastAPI Documentation. Obtenido de <https://fastapi.tiangolo.com/>

FPDF2. (2024). FPDF2 Documentation. Obtenido de <https://py-pdf.github.io/fpdf2/>

Hugging Face. (2024). Transformers Documentation. Obtenido de <https://huggingface.co/docs/transformers/>

TensorFlow. (2024). TensorFlow Documentation. Obtenido de <https://www.tensorflow.org/>

Uvicorn. (2024). Uvicorn: An ASGI web server for Python. Obtenido de <https://www.uvicorn.org/>

Mypy. (2024). Mypy Documentation. Obtenido de <https://mypy.readthedocs.io/>

pytest. (2024). pytest Documentation. Obtenido de <https://docs.pytest.org/>

Ruff. (2024). Ruff Documentation. Obtenido de <https://docs.astral.sh/ruff/>

eLeCe2611. (2026). vitalXAI [Software]. GitHub. <https://github.com/eLeCe2611/vitalXAI>

MDN Web Docs. (2024). JavaScript and Fetch API. Obtenido de <https://developer.mozilla.org/>

Tailwind CSS. (2024). Tailwind CSS Documentation. Obtenido de <https://tailwindcss.com/>

OWASP. (2024). OWASP Top 10: Application Security Risks. Obtenido de <https://owasp.org/www-project-top-ten/>